Reference textbook: Introduction to Algorithms, 3rd Edition
Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein

Algorithms

Full course handout

A4 · Two slides per page · Lin Cing

Contents

1. Algorithm Analysis and Correctness

01 · Algorithm Analysis and Correctness 3

2. Recursion, Recurrences, and Divide-and-Conquer

02 · Recursion, Recurrences, and Divide-and-Conquer 29

3. Sorting, Selection, and Randomized Analysis

03 · Sorting, Selection, and Randomized Analysis 60

4. Data Structures and Amortized Analysis

04 · Data Structures and Amortized Analysis 137

5. Dynamic Programming

15 · Dynamic programming 252

6. Greedy Algorithms

16 · Greedy algorithms 270

7. Graph Traversal and Connectivity

22 · Elementary graph algorithms 282

8. Minimum Spanning Trees and Shortest Paths

21 · Disjoint sets: merge groups, never split them 300
23 · Minimum spanning trees 320
24 · Single-source shortest paths 336
25 · All-pairs shortest paths 353

9. Network Flow

26 · Maximum flow 368

10. Computational Geometry

33 · Computational geometry 393

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Algorithms

Full course handout

A4 · Two slides per page · Lin Cing

Contents

11. Complexity and Approximation

34 · NP-completeness 408

35 · Approximation algorithms 434

12. String Matching

32 · String matching 454

13. Matrix Operations and FFT

28 · Matrix operations 468

30 · Polynomials and FFT 481

14. Number-Theoretic Algorithms

31 · Number-theoretic algorithms 494

15. Linear Programming

29 · Linear programming 517

16. Parallel Algorithms

27 · Multithreaded algorithms 536

Reference Appendices

A · Summations 561

B · Sets, relations, functions, graphs, and trees 575

C · Counting and probability 593

D · Matrices 617

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Algorithm Analysis and Correctness

Running a program tells us what happened on one execution. Analyzing an algorithm asks more: **Does it return a correct answer for every legal input? How does its cost grow with the input?**

We develop specifications and proofs, exact operation counts, asymptotic comparisons, and loop sums. Insertion sort provides one continuous example, from individual moves to a general cost formula.

Prerequisites: arrays, loops, elementary algebra and logarithms. Area and limit arguments also use basic calculus.

Reference textbook: Cormen et al., *Introduction to Algorithms*, 3rd Edition, Chapters 1–3; summations in Appendix A.

Reading route

1. Specifications and termination → linear-search invariants → a complete insertion-sort analysis.
2. Asymptotic quantifiers → growth comparisons → sums and loop counts.
3. Integer boundaries and space → interpreting costs → mathematical extensions.

Read the main sequence before the extensions; revisit definitions when a symbol is unfamiliar.

Specify the answer before reading the code

A problem has a legal input domain D and an output relation $R(x)$. One input $x \in D$ may admit several answers. An algorithm is a precise procedure for producing one of them.

A sorting input is a sequence of records with a key and a fixed identity. In $(3, a), (1, b), (3, c)$, the letters distinguish records; they are not secondary sorting keys.

Output requirement	How to check it
Nondecreasing keys	Adjacent keys satisfy $A[i].key \leq A[i+1].key$
No lost or duplicated records	The output is a permutation, including multiplicities
Stability, when required	Equal keys retain their original relative order

Sortedness alone is insufficient: deleting every record leaves a sorted empty sequence, but does not sort the original input.

Identities distinguish equal keys

Input [3a, 1b, 3c] → sort by key; letters identify records

Sorted, preserved, stable	Sorted and preserved	Sorted, but missing a record
1b 3a 3c	1b 3c 3a	1b 3a
All three requirements hold	Equal-key a and c are reversed	3c is lost: invalid sorting

Stability is an additional requirement. Swapping equal-key records can preserve sortedness and the input multiset while violating stability. Stability also requires the original order of a and c.

Empty and singleton inputs are already sorted. A correct implementation must handle them without reading a nonexistent first record.

A correct result and termination are separate obligations

Partial correctness means that if execution terminates, its output satisfies the specification. Total correctness additionally requires termination on every legal input.

```
while true
    skip
```

This program never returns an incorrect answer, but never returns an answer at all. A proof about its result after termination does not establish that it completes the task.

For a loop, a common termination argument uses a nonnegative integer that strictly decreases in every iteration. A finite program listing or a finite collection of tests does not replace that argument.

Linear search: specify which occurrence to return

Given $A[1..n]$ and target x, return the **first** index with $A[i] = x$, or NIL if none exists. Allow $n = 0$; one record read or comparison costs a constant.

```
FIRST-MATCH(A, x)
    i = 1
    while i <= n
        if A[i] == x: return i
        i = i + 1
    return NIL
```

Arrays are indexed from 1. An interval $A[a..b]$ with $a > b$ is empty; it does not access an element. Here and in the sorting pseudocode, n denotes the input array length.

Array references are different from record identities

Our pseudocode accesses an array through a reference. Updating $A[i]$ changes the array visible to the caller. Rebinding the procedure's local variable to another array does not rebind the caller's variable.

Operations	Result
$A = [3, 1]$; $B = A$; then $B[1] = 8$	Both names refer to one array; A becomes $[8, 1]$
$A = [3, 1]$; $B = \mathit{copy}(A)$; then $B[1] = 8$	The arrays have separate storage; A stays $[3, 1]$

A shallow array copy may still contain references to the same objects. Independent array storage does not imply a deep copy of every object. The identities a, b, c used in sorting identify records, not necessarily their memory addresses.

The search invariant records what has been ruled out

Before each `i <= n` test: $1 \leq i \leq n+1$, and no position in $A[1..i-1]$ matches x.

Stage	Reason
Initialization	$i = 1$; the excluded prefix is empty
Maintenance	A failed comparison followed by $i \leftarrow i + 1$ extends that prefix by one
Match found	$A[i] = x$, and every earlier position has been excluded
Loop exhausted	$i = n + 1$; every position has been excluded, so NIL is correct

Every iteration that does not return decreases the nonnegative integer $n - i + 1$ by 1. Thus execution stops. For $n = 0$, it returns NIL immediately without reading $A[1]$.

Before i is excluded; after i is still unknown

A = [6, 2, 6, 5]; target x = 5

	1	2	3	4	
i = 1	6	2	6	5	Excluded: empty Current: no match
i = 3	6	2	6	5	Excluded: A[1..2] Current: no match
i = 4	6	2	6	5	Excluded: A[1..3] Current: match; return 4

Green: excluded. Teal border: current. White: not yet tested.

For $A = [6, 2, 6, 5]$, search for 5. Before $i = 3$, only positions 1–2 are excluded. A failed comparison extends that prefix; at $i = 4$, the first match returns. **Untested does not mean excluded.**

Three complete executions of the same code

Input and target	Positions compared	Return value	Key comparisons
$[4, 2, 4]$, target 4	1	1	1
$[4, 2, 4]$, target 7	1, 2, 3	NIL	3
Empty array, target 4	None	NIL	0

The cases test first occurrence, exhaustion, and empty input; the invariant supplies the general proof.

If the first matching position, conditional on success, is uniform over $1..n$, the expected comparison count conditional on success is $(n + 1)/2$. Another distribution requires another calculation.

State the cost model before measuring cost

Initially use a word-RAM model: indices and records occupy a constant number of machine words; reads, writes, comparisons, and supported basic arithmetic operations have constant cost. This is a model, not a claim that every real operation takes the same time.

Object of analysis	Possible cost unit
Array search	Key comparisons or reads
Sorting	Comparisons, moves, auxiliary space
Large-integer algorithms	Bit lengths and multiword arithmetic
External storage	Block transfers

For integers with b bits, arbitrary-precision multiplication is not automatically one constant-time operation. Read every bound together with its model.

Input length is not necessarily the numerical value

A positive integer N has binary length $b = \lfloor \lg N \rfloor + 1$, where $\lg = \log_2$. A loop with N iterations may be exponential in the input length b.

For example, $N = 13$ is `1101`, only four bits long. Enumerating every integer up to N takes $\Theta(N)$ iterations. Since $2^{b-1} \leq N < 2^b$, that is $\Theta(2^b)$ iterations as a function of b.

Array size usually means the number of records n. Graph bounds often need both the vertex count V and edge count E. Writing $O(V + E)$ preserves that distinction.

Choose the cost function: best, worst, or average

For inputs of length n in D_n, let $C(x)$ be the cost on input x.

$$B(n) = \min_{x \in D_n} C(x), \qquad W(n) = \max_{x \in D_n} C(x).$$

Average cost additionally needs a distribution P_n:

$$A(n) = \mathbb{E}_{x \sim P_n}[C(x)] = \sum_{x \in D_n} P_n(x) C(x).$$

Big-O does not mean worst case. Each of B, W, A is a function with its own upper and lower bounds. For a randomized algorithm, also distinguish randomness in the input from the algorithm's own random choices.

Insertion sort extends a sorted prefix

Use $[(7, a), (2, b), (5, c), (2, d), (1, e)]$, abbreviated $[7a, 2b, 5c, 2d, 1e]$. Save the next record as `key` and shift larger records on its left one position right.

```
INSERTION-SORT(A)
    for j = 2 to n
        key = A[j]
        i = j - 1
        while i >= 1 and A[i].key > key.key
            A[i+1] = A[i]
            i = i - 1
        A[i+1] = key
```

`and` short-circuits left to right, so $A[i]$ is not read when $i = 0$. For $n \leq 1$ the outer loop is empty; strict comparison keeps older equal-key records before the new one.

Every iteration leaves a checkable result

j / saved record	Key comparisons, right to left	Shifts h_j	Array after insertion
2 / $2b$	$7 > 2$; reach boundary	1	$2b, 7a, 5c, 2d, 1e$
3 / $5c$	$7 > 5$; $2 > 5$ is false	1	$2b, 5c, 7a, 2d, 1e$
4 / $2d$	$7 > 2, 5 > 2$; $2 > 2$ is false	2	$2b, 2d, 5c, 7a, 1e$
5 / $1e$	$7, 5, 2d, 2b$ all exceed 1	4	$1e, 2b, 2d, 5c, 7a$

$2b$ remains before $2d$. There are eight shifts but ten key comparisons: iterations 3 and 4 each have a final failed comparison; iterations 2 and 5 reach $i = 0$ without another key comparison.

A logical hole explains temporary duplicates

Fixed j = 4; saved key = 2d

	1	2	3	4	
1	2b	5c	7a	□	7 > 2: shift 7a right
2	2b	5c	□	7a	5 > 2: shift 5c right
3	2b	□	5c	7a	2 > 2 is false: stop shifting
4	2b	2d	5c	7a	Insert key; 2b stays before 2d

□ is a logical hole, not a written value. The saved key remains outside the array.

The box is a **logical hole**, not a written empty marker. The positions outside it plus saved key retain the input. After the first shift, physical memory holds two copies of $7a$; the whole physical prefix is not a permutation.

Outer invariant: order, preservation, and stability

Let S be a snapshot of the original input. Before each iteration, $A[1..j-1]$ is a stable sorting of $S[1..j-1]$, and $A[j..n] = S[j..n]$.

Stage	How the invariant holds
Start, $j = 2$	One record is already sorted; the suffix is unchanged
Maintenance	Shift the larger suffix and insert key; equal keys do not cross; the unprocessed suffix is unchanged
Stop	At $j = n + 1$, the sorted prefix contains the whole original input

The inner index decreases at every shift, with at most $j - 1$ shifts; there are at most $n - 1$ outer iterations. Thus the algorithm terminates. Inputs with $n \leq 1$ are handled separately by the empty loop.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Inner invariant: include the position of the hole

Fix j. Let $B = A[1..j]$ at the start of this iteration, with $B[1..j-1]$ sorted. Save $key = B[j]$ and place the logical hole at $i + 1$.

Before each inner test:

1. $A[1..i] = B[1..i]$.
2. $A[i + 2..j] = B[i + 1..j - 1]$; every key in this right segment exceeds key.
3. Excluding the hole and adding the saved key preserves exactly the records in B.

Initially the right segment is empty. Copying $A[i]$ into the hole and decrementing i preserves all three properties. On stopping, the left segment is at most key and the right segment exceeds it; filling the hole restores a sorted, record-preserving prefix.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Count comparisons, moves, and guard tests separately

For $n \geq 2$, let h_j count shifts in iteration j. Let $q_j = 1$ if it stops at a failed key comparison with $i \geq 1$, and $q_j = 0$ otherwise.

Operation	Iteration j	Entire execution
Shift write	h_j	$\sum_{j=2}^{n} h_j$
Final key write	1	$n-1$
Complete inner guard test	$h_j + 1$	$\sum h_j + n - 1$
Key comparison	$h_j + q_j$	$\sum h_j + \sum q_j$

Here $\sum h_j = 8$ and $\sum q_j = 2$: ten key comparisons, twelve guard tests, and twelve array writes. Reading key into a local variable is not an array write.

Why does the shift count equal the inversion count?

An inversion is a pair of original positions $p < q$ with $S[p].key > S[q].key$. Denote the total by $I(S)$.

When inserting the original record $S[j]$, the earlier $j-1$ records are sorted. Exactly those with keys larger than $S[j]$ shift right. Thus h_j counts precisely the inversions whose right endpoint is j.

Every inversion has a unique right endpoint, giving

$$\sum_{j=2}^{n} h_j = I(S), \qquad T(S) = \Theta(n + I(S)).$$

Equal keys do not form inversions. The example has $1, 1, 2, 4$ inversions by right endpoint, totaling eight, exactly as in the shift table.

Best, worst, and average for one algorithm

Input assumption	Inversions	Time
Already nondecreasing	0	$\Theta(n)$
Distinct keys in decreasing order	$n(n-1)/2$	$\Theta(n^2)$
All permutations of distinct keys equally likely	Expected $n(n-1)/4$	Expected $\Theta(n^2)$

For the last row, let X_{pq} indicate whether positions $p < q$ form an inversion. A uniform permutation gives $\mathbb{E}[X_{pq}] = 1/2$. By linearity of expectation,

$$\mathbb{E}[I] = \sum_{p<q} \mathbb{E}[X_{pq}] = \binom{n}{2}\frac{1}{2}.$$

Independence between these indicators is unnecessary. Repeated keys or a nonuniform input model require a new distribution argument.

What asymptotic analysis ignores, and what it retains

We study growth as $n \to \infty$, allowing fixed multiplicative constants and finitely many small inputs to be disregarded for the bound. Small inputs still require correct behavior.

Here n is a positive integer, f is eventually nonnegative, and g is eventually strictly positive. $O(g), \Omega(g), \Theta(g)$ are **sets of functions**. The conventional notation $f = O(g)$ means $f \in O(g)$.

For example, $n \in O(n^2)$ is true but loose. Use Θ when both sides have been established; Big-O alone does not mean an exact growth order.

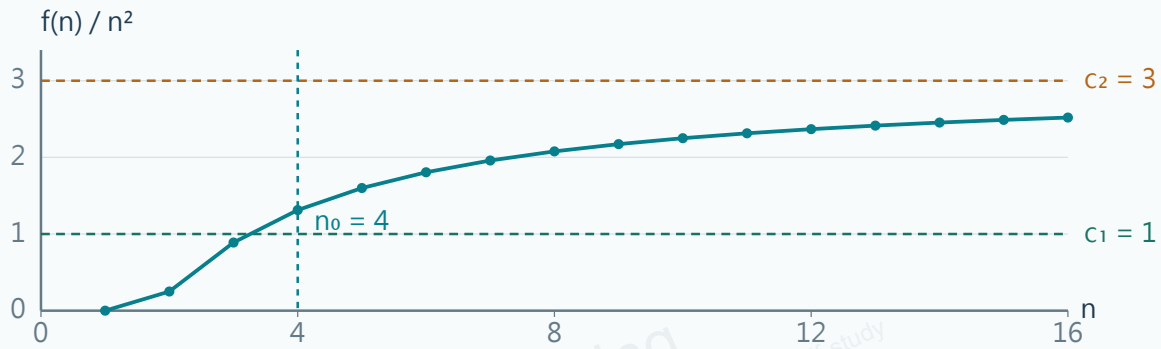
The quantifiers in upper, lower, and tight bounds

Notation	Formal requirement: for every integer $n \geq n_0$
$f \in O(g)$	There exist $c > 0, n_0$ with $0 \leq f(n) \leq cg(n)$
$f \in \Omega(g)$	There exist $c > 0, n_0$ with $0 \leq cg(n) \leq f(n)$
$f \in \Theta(g)$	There exist $c_1, c_2 > 0, n_0$ with $0 \leq c_1 g(n) \leq f(n) \leq c_2 g(n)$

Constants may depend on the fixed functions, **not on input size** n. The threshold need not be the smallest possible one.

To refute $f \in O(g)$, every candidate c, n_0 must admit an $n \geq n_0$ with $f(n) > cg(n)$. Defeating only one proposed pair is insufficient.

Plot the constants and the threshold together



For every n ≥ 4, the ratio lies between 1 and 3; earlier values need not.

Let $f(n) = 3n^2 - 8n + 5$. For $n \geq 4$, $f(n)/n^2 = 3 - 8/n + 5/n^2 \geq 1$; for $n \geq 1$, $f(n) \leq 3n^2$.

Thus $c_1 = 1, c_2 = 3, n_0 = 4$ prove $f \in \Theta(n^2)$. The plot illustrates the ratio. The inequalities, rather than finitely many plotted points, cover every $n \geq 4$.

Prove that the same function has no linear upper bound

We have $f(n) = 3n^2 - 8n + 5 \geq n^2$ for $n \geq 4$. Suppose $f \in O(n)$. Then some fixed c, n_0 satisfy $f(n) \leq cn$ for every $n \geq n_0$.

Choose an integer $n > \max\{4, n_0, c\}$. It follows that

$$n^2 \leq f(n) \leq cn \quad \Longrightarrow \quad n \leq c,$$

contradicting $n > c$. Therefore $f \notin O(n)$.

The order of choices matters: allow arbitrary candidate constants first, then choose a large enough input. One fixed test input cannot refute every candidate bound.

Use the larger of two thresholds

Suppose (c_U, N_U) witnesses $f \in O(g)$ and (c_L, N_L) witnesses $f \in \Omega(g)$. Both inequalities hold for $n \geq \max(N_U, N_L)$, so

$$\Theta(g) = O(g) \cap \Omega(g).$$

Other familiar rules also have assumptions:

Property	Reason or condition
Transitivity of O	Multiply the constants; take the larger threshold
$f \in O(g) \Leftrightarrow g \in \Omega(f)$	Read the same inequality in reverse with reciprocal constant
$f + g = \Theta(\max(f, g))$	For nonnegative functions, $\max(f, g) \leq f + g \leq 2\max(f, g)$

If cancellation is allowed, the last rule cannot be used without adjustment: $n + (-n) = 0$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

little-o works for every positive multiplier

$$f \in o(g) \iff \forall c > 0\ \exists n_0(c)\ \forall n \geq n_0(c) :\ 0 \leq f(n) < cg(n).$$

Big-O asks for one multiplier; little-o asks for **every positive multiplier**. For $7n \in o(n^2)$, given $c > 0$, choose an integer $n_0 > 7/c$.

Likewise $f \in \omega(g)$ requires $0 \leq cg(n) < f(n)$ eventually for every $c > 0$. Hence $n^2 \in \omega(7n)$.

For eventually positive functions, little-o is equivalent to $f/g \to 0$, and little-omega to $f/g \to \infty$. $7n^2$ and n^2 have the same order; their ratio does not tend to zero.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

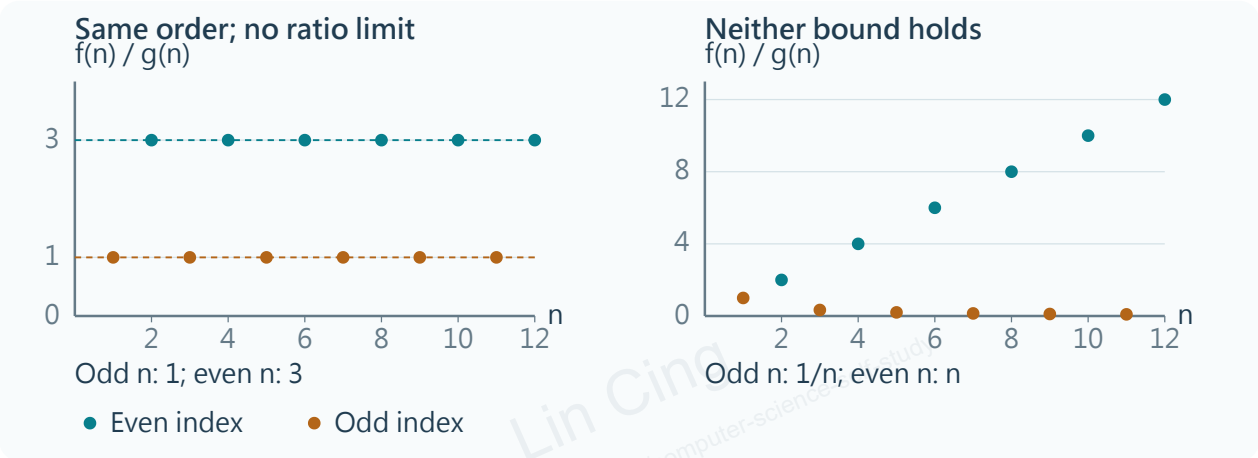
A ratio limit is a tool, not the only criterion

$\lim f(n)/g(n)$	Conclusion
0	$f \in o(g)$
Finite positive L	$f \in \Theta(g)$
$+\infty$	$f \in \omega(g)$
Does not exist	Continue the analysis; incomparability does not follow

For finite positive L, choose $\varepsilon = L/2$. Eventually $L/2 < f/g < 3L/2$, providing two fixed positive multipliers.

The first three rows require the stated limit to exist. Logarithmic comparison also requires positive arguments: $\lg f / \lg g \to 1$ alone does **not** imply $f = \Theta(g)$.

An oscillating ratio can still be tightly bounded



Left: $f = n(2 + (-1)^n), g = n$ satisfy $n \le f \le 3n$ despite oscillation. Right: $g = n$; use $f = n^2$ for even n and $f = 1$ for odd n. Even ratios are unbounded, refuting $O(g)$; odd ratios tend to zero, refuting $\Omega(g)$.

Do not exchange function bounds and input quantifiers

Let $C(x)$ be input cost and $W(n) = \max_{x \in D_n} C(x)$. If $W(n) \in O(n^2)$, there are **shared** constants c, n_0 such that every $n \geq n_0$ and every $x \in D_n$ satisfy $C(x) \leq cn^2$.

If a separate constant may grow with each input, the statement does not provide that uniform upper bound.

Insertion sort has $W(n) = \Theta(n^2)$, yet a sorted input takes $\Theta(n)$. A tight worst-case bound does not say that every input requires quadratic time.

Three practical approaches to comparing growth

Approach	Useful for	Common mistake
Definition and inequalities	Polynomials, oscillation, explicit constants	Only sampling values or proving one side
Ratio limit	Positive functions with a manageable limit	Treating nonexistence as incomparability
Take logarithms	Positive products, powers, exponentials	Inferring original tight bounds from tight logarithms

For $f = n \lg n, g = n$, $\lg f / \lg g = 1 + \lg\lg n / \lg n \to 1$, but $f/g = \lg n \to \infty$. Thus $f \in \omega(g)$.

Logs can simplify comparison. A fixed multiplicative bound still requires the original ratio, or a sufficiently precise bound on the difference of logarithms.

Growth hierarchies require fixed constants

For fixed $a > 1, b > 0, \varepsilon > 0$, and $k > 0$,

$$1 \prec \lg\lg n \prec (\lg n)^b \prec n^\varepsilon \prec a^n \prec n! \prec n^n.$$

Here $f \prec g$ means $f = o(g)$, for sufficiently large positive integers.

With $x = \ln n$, $(\lg n)^b / n^\varepsilon$ is a fixed constant times $x^b / e^{\varepsilon x}$, which tends to zero. Also $\ln(n^k / a^n) = k \ln n - n \ln a \to -\infty$, proving $n^k = o(a^n)$.

The following panel justifies the basic limit $\ln n / n \to 0$. If a base or exponent varies with n, these fixed-constant conclusions need not apply.

Why does a positive power dominate a logarithm?

For real $x \geq 4$, the derivative of $\sqrt{x} - \ln x$ is $(\sqrt{x} - 2)/(2x) \geq 0$, and its value at 4 is positive. Hence $0 \leq \ln x / x \leq 1/\sqrt{x} \to 0$.

Now fix $b, \varepsilon > 0$ and set $t = \ln n$. Taking natural logarithms gives

$$\ln \frac{(\ln n)^b}{n^\varepsilon} = b \ln t - \varepsilon t = t \left(b \frac{\ln t}{t} - \varepsilon \right).$$

Since $\ln t / t \to 0$, the parenthesis is eventually at most $-\varepsilon/2$. The logarithm tends to $-\infty$, so the original ratio tends to zero. Thus $(\ln n)^b = o(n^\varepsilon)$; a fixed change of log base changes only a constant.

Logarithms, powers, and changes of base

Expression	Interpretation
$\log_a n = \lg n / \lg a$	Fixed $a > 1$: log bases differ by a constant
$\lg(n^k) = k \lg n$	Take a power, then a logarithm
$\lg^k n = (\lg n)^k$	Take a logarithm, then a power
$2^{\lg n} = n$	Matching exponential and logarithm bases
$2^{2 \lg n} = n^2$	A constant in an exponent cannot simply be discarded

$\lg n$ and $2 \lg n$ have the same order; $2^{\lg n}$ and $2^{2 \lg n}$ do not. A varying log base is another issue: $\log_n n = 1$ does not behave like a fixed-base logarithm.

From a loop to a sum: recheck each iteration's range

```
count = 0
for i = 1 to n
    for j = 1 to i
        count = count + 1
```

Iteration i performs i increments, so

$$\sum_{i=1}^{n} i = \frac{n(n+1)}{2} = \Theta(n^2).$$

Saying the inner loop averages about half the maximum is not a proof. The exact sum supplies a formula. Alternatively, n^2 is an upper bound, while the final half of the iterations each perform at least $n/2$ operations, giving a lower bound $n^2/4$.

A triangular iteration space is still quadratic

n = 6; one cell is one visit(i, j)

Upper: at most 6 × 6 = 36 cells. Lower: three cells in each of rounds 4–6.

Each cell is one inner operation. For $n = 6$, $1 + 2 + 3 + 4 + 5 + 6 = 21$ cells. Taking three cells from each of rows 4–6 gives $3 \times 3 = 9$: a concrete instance of the preceding general quadratic lower bound.

A doubling loop counts how often 2 is multiplied

```
i = 1
while i <= n
    visit(i)
    i = 2*i
```

With unbounded integer arithmetic and $n \geq 1$, it visits $1, 2, 4, \ldots, 2^{\lfloor \lg n \rfloor}$, exactly $\lfloor \lg n \rfloor + 1$ values. For example, input 13 visits $1, 2, 4, 8$: four values.

A fixed-width implementation must avoid overflow in `2*i`. After visiting, stop if $i > \lfloor n/2 \rfloor$; otherwise doubling is safe. A mathematical termination proof does not automatically cover machine overflow.

A geometric sum has the order of its last term

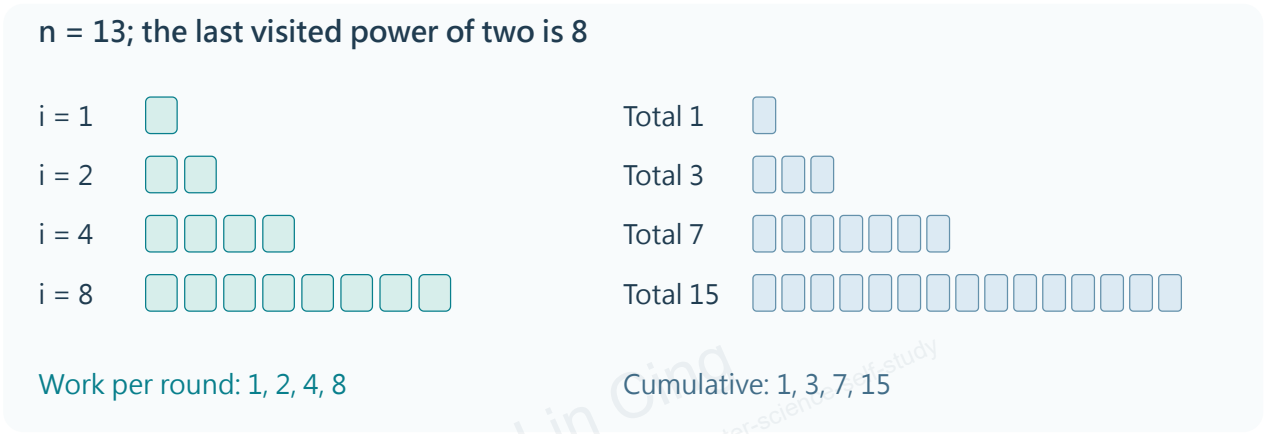
If the outer values are $i = 1, 2, 4, \ldots, 2^k \leq n$ and each iteration performs i operations, the total is

$$1 + 2 + 4 + \cdots + 2^k = 2^{k+1} - 1.$$

Since $2^k \leq n < 2^{k+1}$, we have $n/2 < 2^k \leq n$. Thus the total is $\Theta(n)$, not $\Theta(n \lg n)$.

For $n = 13$, the four iterations perform $1, 2, 4, 8$ operations, totaling 15. Replacing every iteration's work by n gives an upper bound, but it does not establish a tight bound.

Sum the changing work, not the final work times the rounds



Rounds cost $1, 2, 4, 8$; cumulative work is $1, 3, 7, 15$. Generally, $2^k \le \sum_{r=0}^{k} 2^r < 2 \cdot 2^k$. The total is within twice the last round, with no extra factor for the number of rounds.

Bound a harmonic sum by areas on both sides

Let $H_n = \sum_{i=1}^{n} 1/i$. The function $1/x$ decreases on the positive real axis, so

$$\int_1^{n+1} \frac{dx}{x} \le H_n \le 1 + \int_1^{n} \frac{dx}{x}.$$

Thus $\ln(n+1) \le H_n \le 1 + \ln n$. For $n \ge 3$, $1 + \ln n \le 2 \ln n$, giving $H_n = \Theta(\ln n) = \Theta(\lg n)$.

For the upper bound, separate the first term 1 and place later terms below the curve on the preceding intervals. For the lower bound, left-endpoint rectangles cover the integral. The following panel separates the two constructions.

Power sums need not have a closed form to be bounded

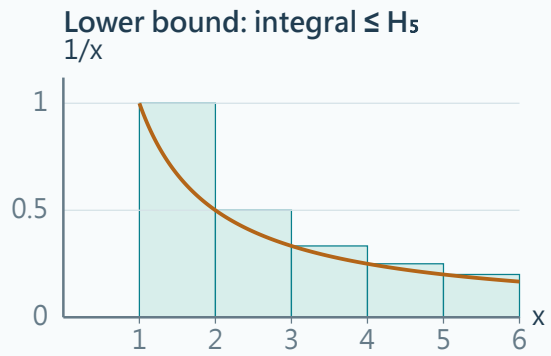
For fixed real $p \ge 0$, let $S_p(n) = \sum_{i=1}^{n} i^p$. There are n terms, each at most n^p, so $S_p(n) \le n^{p+1}$.

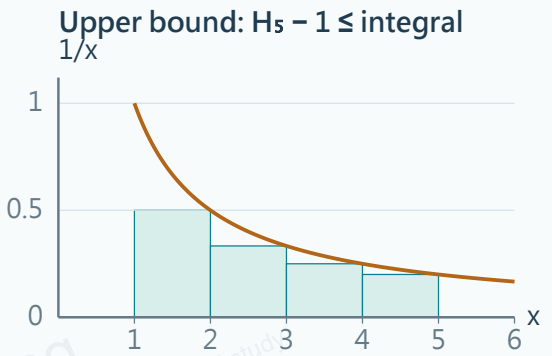
For a lower bound, retain $i = \lceil n/2 \rceil .. n$. There are at least $n/2$ such terms, each at least $(n/2)^p$, so

$$S_p(n) \ge \frac{n}{2} \left(\frac{n}{2}\right)^p = \frac{n^{p+1}}{2^{p+1}}.$$

Thus $S_p(n) = \Theta(n^{p+1})$. The factors are fixed because p is fixed. The case $p = -1$ is harmonic and does not satisfy this conclusion.

One curve, two rectangle constructions





Left: height $1/i$ on $[i, i+1]$ covers the curve. Right: terms $i = 2..5$ lie below it on $[i-1, i]$. The common-scale panels give $1.7918 \approx \ln 6 \leq H_5 = 137/60 \approx 2.2833 \leq 1 + \ln 5 \approx 2.6094$.

Dependent nested loops can take n log n time

```
for i = 1 to n
    j = i
    while j <= n
        visit(i, j)
        j = j + i
```

The inner loop visits $i, 2i, \ldots, \lfloor n/i \rfloor i$, giving

$$S(n) = \sum_{i=1}^{n} \lfloor n/i \rfloor .$$

For $x \geq 1$, $x/2 \leq \lfloor x \rfloor \leq x$. Hence $nH_n/2 \leq S(n) \leq nH_n$, proving $\Theta(n \lg n)$. At $n = 6$, the counts are $6, 3, 2, 1, 1, 1$, totaling 14.

Take logarithms before comparing factorial growth

For $n \geq 4$, $n! \leq n^n$, so $\lg(n!) \leq n \lg n$.

At least $n/2$ factors in the latter half are at least $n/2$, giving

$$\lg(n!) \geq \frac{n}{2} \lg(n/2) \geq \frac{n}{4} \lg n.$$

Hence $\lg(n!) = \Theta(n \lg n)$. This does not imply $n! = \Theta(n^n)$: that ratio tends to zero.

For fixed $a > 1$, $\ln(n!/a^n) \geq (n/2) \ln(n/2) - n \ln a \to \infty$, proving $a^n = o(n!)$. Use Stirling's formula for a more precise factor; an error inside a logarithm cannot be exponentiated and discarded indiscriminately.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Boundaries for floor, ceiling, and remainder

For real x, $x - 1 < \lfloor x \rfloor \leq x \leq \lceil x \rceil < x + 1$. For integer n, this implies

$$\lfloor n/2 \rfloor + \lceil n/2 \rceil = n.$$

For integer a and positive integer m, use the mathematical remainder

$$a \bmod m = a - m\lfloor a/m \rfloor, \qquad 0 \leq a \bmod m < m.$$

For example, $-7 \bmod 3 = 2$, since $-7 = 3(-3) + 2$. A language's `%` operator may instead use a quotient truncated toward zero and produce a negative remainder. Check the semantics before translating a formula into code.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

When can successive integer divisions be combined?

For $x \geq 0$ and positive integers a, b,

$$\left\lfloor \frac{\lfloor x/a \rfloor}{b} \right\rfloor = \left\lfloor \frac{x}{ab} \right\rfloor, \qquad \left\lceil \frac{\lceil x/a \rceil}{b} \right\rceil = \left\lceil \frac{x}{ab} \right\rceil.$$

For floor, set $q = \lfloor x/(ab) \rfloor$, so $abq \leq x < ab(q+1)$. Divide by a and take floor: $bq \leq \lfloor x/a \rfloor \leq bq + b - 1$. Dividing by b and taking floor now gives q.

For $x = 17, a = 3, b = 2$, successive floors give $17 \to 5 \to 2$, matching $\lfloor 17/6 \rfloor = 2$. Positivity and integrality of the divisors support the integer-interval argument.

The ceiling proof uses a strict lower endpoint

Set $q = \lceil x/(ab) \rceil$. Then $ab(q-1) < x \leq abq$. Divide by a and take ceiling:

$$b(q-1) + 1 \leq \lceil x/a \rceil \leq bq.$$

Dividing by b and taking ceiling gives q. For $x = 17, a = 3, b = 2$, the steps $17 \to 6 \to 3$ match $\lceil 17/6 \rceil = 3$.

For positive integers u, v, $\lceil u/v \rceil \leq (u + v - 1)/v$ and $\lfloor u/v \rfloor \geq (u - v + 1)/v$. Write $u = qv + r$, $0 \leq r < v$, and check $r = 0$ and $r > 0$ separately. The implementation `(u+v-1)/v` still needs an overflow check for the addition.

Peak space counts what is alive simultaneously

Auxiliary space excludes the input but normally includes temporary arrays and the call stack. If output space is also excluded, state that convention.

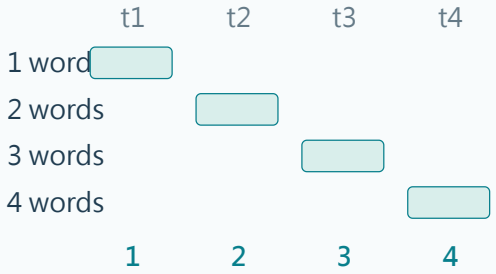
Situation	Cumulative allocation	Peak live storage
Allocate and free n single-word blocks in turn	$\Theta(n)$	$\Theta(1)$
Retain all n single-word blocks	$\Theta(n)$	$\Theta(n)$
K calls; peak of d constant-size live frames	$\Theta(K)$	$\Theta(d)$

Insertion sort keeps a constant number of indices and temporary records. Its auxiliary space is $\Theta(1)$ when each record occupies a constant number of words. For large objects, clarify whether the algorithm moves objects or references.

Draw allocation lifetimes to find the peak

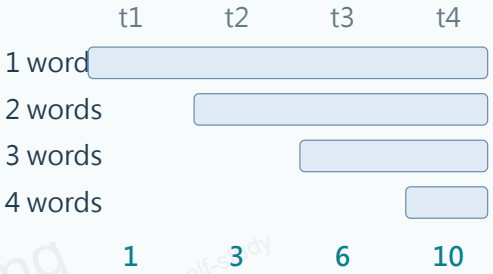
Allocate 1, 2, 3, 4 words; horizontal positions are processing intervals

Free after processing



Peak 4; allocated total 10

Retain all blocks



Peak 10; allocated total 10

Allocate blocks of 1, 2, 3, and 4 words. Immediate release gives peak 4; retaining all gives peak 10. Both cumulatively allocate ten words. Count these live blocks only, excluding allocator fragmentation, caches, and operating-system resident memory.

Constants and crossover points still matter

In the same cost unit, let $T_A(n) = 8n^2$ and $T_B(n) = 64n \lg n$, $n \geq 2$. A is faster exactly when $n < 8 \lg n$.

n	$8n^2$	$64n \lg n$	Comparison
16	2048	4096	A faster
32	8192	10240	A faster
64	32768	24576	B faster

For real $n \geq 4$, $n/\lg n$ is strictly increasing. Checking 43 and 44 on opposite sides, together with 2 and 3, locates the integer crossover between 43 and 44. If these formulas came from measurements, the result would be conditional on that model and environment.

Hardware speed has units too

If A executes 10^9 model operations per second and B executes 10^8, compare

$$t_A(n) = T_A(n)/10^9, \qquad t_B(n) = T_B(n)/10^8.$$

Do not compare an operation count directly with a time in seconds. Hardware changes constants; an algorithmic change may change the growth order. Both comparisons need an explicit model.

Also check space: an n^2-entry table may exceed memory even when its running time is acceptable.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Put the analysis together

For a new program, build the argument in this order rather than guessing a Big-O first:

1. Fix legal inputs, input size, and the comparison/arithmetic model.
2. Trace a small input completely to identify what each iteration does.
3. State invariants, boundary cases, and a decreasing quantity.
4. Write a general sum or recurrence for the cost.
5. Establish upper and lower bounds; identify the best/worst/average function.
6. Find peak space and check overflow, aliasing, and distribution assumptions.

For insertion sort, $T(S) = \Theta(n + I(S))$ explains how input structure affects work, beyond the looser statement of worst-case $O(n^2)$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Extension: why does a polynomial's leading term dominate?

Let $p(n) = a_d n^d + \sum_{i<d} a_i n^i$, with fixed $d \geq 1$, $a_d > 0$, and $M = \sum_{i<d} |a_i|$.

For $n \geq 1$, the lower terms have total absolute value at most Mn^{d-1}. If also $n \geq 2M/a_d$, then

$$\frac{a_d}{2} n^d \leq p(n) \leq \frac{3a_d}{2} n^d.$$

Thus $p(n) = \Theta(n^d)$ even with negative lower coefficients. A negative leading coefficient violates our eventually nonnegative cost convention. If $d = 0, a_0 > 0$, the result is $\Theta(1)$ instead.

Extension: an asymptotic equality need not be reversible

The expression $n^2 + n = n^2 + O(n)$ means that a function in $O(n)$ makes the equality true; here it is the function n.

It implies $n^2 + n = \Theta(n^2)$, but $f = \Theta(n^2)$ does not imply $f = n^2 + O(n)$. Take $f = 2n^2$: the difference n^2 is not in $O(n)$.

To calculate with anonymous functions, name them first. For $u, v \in O(n)$, the triangle inequality gives $|u + v| \leq (c_u + c_v)n$ eventually. Subtraction requires a convention for signed error bounds; do not silently ignore the nonnegative-function convention used in a definition.

Extension: e, ln, and approximation error

For $x > -1$, $\ln(1 + x) \leq x$. Therefore $(1 + 1/n)^n \leq e$ for positive integers n.

For $x \geq 0$, a more precise pair of bounds is

$$x - \frac{x^2}{2} \leq \ln(1 + x) \leq x.$$

For the lower bound, set $h(x) = \ln(1 + x) - x + x^2/2$. Then $h(0) = 0$ and $h'(x) = x^2/(1 + x) \geq 0$. Substituting $x = 1/n$ gives

$$e^{1-1/(2n)} \leq (1 + 1/n)^n \leq e.$$

This bounds finite-n error, not just the limit. Approximations require a domain and an error bound; higher-order terms cannot always be discarded.

Extension: an exponential remainder with explicit bounds

Taylor's theorem gives $e^x = 1 + x + (e^\xi/2)x^2$, with ξ between 0 and x. For $|x| \le 1$,

$$\frac{e^{-1}}{2}x^2 \le e^x - 1 - x \le \frac{e}{2}x^2.$$

Thus $e^x = 1 + x + \Theta(x^2)$ as $x \to 0$, rather than as $x \to \infty$. Convexity gives $e^x \ge 1 + x$ for every real x.

The standard power series $e^x = \sum_{k=0}^{\infty} x^k/k!$ converges for every real x. Using only a few terms requires a remainder estimate such as the one above. This chapter uses the basic calculus theorem, rather than deriving Taylor's theorem itself.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Extension: logarithmic bounds also work for negative x

For $x > -1$, the general pair is

$$\frac{x}{1+x} \le \ln(1+x) \le x.$$

For the upper bound, $h = x - \ln(1+x)$ has derivative $x/(1+x)$ and its minimum at 0 is zero. For the lower bound, $q = \ln(1+x) - x/(1+x)$ has derivative $x/(1+x)^2$ and the same minimum. Both arguments cover negative and positive x.

For fixed real z, substitute $x = z/n$ for sufficiently large n: $z/(1+z/n) \le n\ln(1+z/n) \le z$. Both sides tend to z, proving $(1+z/n)^n \to e^z$.

For $|x| < 1$, the standard series is $\ln(1+x) = \sum_{k=1}^{\infty}(-1)^{k+1}x^k/k$; its stated domain matters.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Extension: how much precision does Stirling supply?

For positive integers n, Stirling's formula is

$$n! = \sqrt{2\pi n}(n/e)^n\big(1 + O(1/n)\big).$$

Stirling's theorem controls the relative error of the leading approximation. Taking natural logarithms gives

$$\ln(n!) = n\ln n - n + \tfrac{1}{2}\ln(2\pi n) + O(1/n).$$

The relative error $O(1/n)$ becomes a logarithmic error $O(1/n)$ since $\ln(1+u) = O(u)$ for sufficiently small u. Conversely, a difference $O(n)$ between logarithms can become an exponential factor, not a constant relative error.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Extension: turn Stirling's error into a numerical interval

A sharper form of Stirling's theorem is

$$n! = \sqrt{2\pi n}(n/e)^n e^{\alpha_n}, \qquad \frac{1}{12n+1} < \alpha_n < \frac{1}{12n} \quad (n \geq 1).$$

For $n = 6$, the leading approximation $L = \sqrt{12\pi}(6/e)^6 \approx 710.078$ is below $6! = 720$. The bounds give

$$Le^{1/73} < 720 < Le^{1/72}.$$

The correction is bounded, not an arbitrary rounding choice. Since $\alpha_n = \Theta(1/n)$ and $e^u - 1 = \Theta(u)$ near zero, $e^{\alpha_n} = 1 + \Theta(1/n)$. Coarse product bounds establish the order of $\lg(n!)$; this theorem additionally controls the leading approximation's relative error.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Extension: iterated logs need a stopping rule

Set $\lg^{(0)} n = n$ and $\lg^{(k+1)} n = \lg(\lg^{(k)} n)$, taking another log only when the previous value exceeds 1. Define

$$\lg^* n = \min\{k \geq 0 : \lg^{(k)} n \leq 1\}.$$

Input	Successive values	$\lg^* n$
1	1	0
2	$2 \to 1$	1
16	$16 \to 4 \to 2 \to 1$	3
65536	$65536 \to 16 \to 4 \to 2 \to 1$	4

$\lg^k n$ raises a logarithm to a power; $\lg^{(k)} n$ repeatedly applies a function. The notation and growth behavior differ.

Extension: verify the Fibonacci closed form

Define $F_0 = 0, F_1 = 1, F_n = F_{n-1} + F_{n-2}$. Set $\varphi = (1 + \sqrt{5})/2$ and $\psi = (1 - \sqrt{5})/2$; both satisfy $r^2 = r + 1$.

Therefore $G_n = (\varphi^n - \psi^n)/\sqrt{5}$ satisfies the same recurrence, with $G_0 = 0, G_1 = 1$. Induction gives $G_n = F_n$ for every nonnegative integer.

Since $|\psi| < 1 < \varphi$, $F_n = \Theta(\varphi^n)$. But the **size of the answer** is not the time needed to compute it. Naive recursion, a table, and fast doubling require separate analyses in the recurrence chapter.

References and attribution

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Recursion, Recurrences, and Divide-and-Conquer

A recursive call has a contract. We must explain what it returns, why its subcalls finish, and how their work combines with the work of the current call.

Read in this order: **one recursive path → merge sort → tools for proving recurrence bounds → maximum subarrays and matrix multiplication → unequal-split extensions**.

Prerequisites: Chapter 1's invariants, induction, asymptotic notation, and sums. Matrix examples use elementary matrix multiplication; the extension uses integration.

Reference textbook: Cormen et al., *Introduction to Algorithms*, 3rd Edition, Chapters 2 and 4; supplementary teaching reference: MIT 6.046J (2005), Lectures 2–3.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A subcall contract is the interface for induction

Let $P(n)$ mean: for every legal input of size n, the procedure terminates and returns a result satisfying its specification.

1. **Base cases:** check the smallest inputs, including whether empty input is legal.
2. **Shrinking:** every subproblem is legal and strictly smaller than n.
3. **Combination:** assuming the smaller calls are correct, prove the current result correct.

This is strong induction. Assuming the current call is correct would be circular. Shrinking proves termination, not the result specification by itself.

Analyze cost separately: $T(n) = \sum_i T(n_i) + f(n)$. Charge only the current call's comparisons, partitioning, copies, and combination to f; **do not charge a subtree twice**.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Binary search continues down one branch

Preconditions: $A[1..N]$ is nondecreasing over totally ordered keys. Return **any** matching index, or NIL if none exists. Call with $(1, N)$; $N = 0$ returns immediately.

```
BINARY-SEARCH(A, low, high, x)
    if low > high: return NIL
    mid = low + floor((high-low)/2)
    if A[mid] == x: return mid
    if A[mid] < x:
        return BINARY-SEARCH(A, mid+1, high, x)
    return BINARY-SEARCH(A, low, mid-1, x)
```

Intervals include both endpoints. A nonempty call has $1 \leq low \leq high \leq N$; an empty interval reads no element. This version does not promise the first duplicate.

Why may an entire interval be discarded?

If $A[mid] < x$, then $A[i] \leq A[mid] < x$ for every $i \leq mid$. Any match must lie on the right. If $A[mid] > x$, keep only the left. Equality already satisfies the contract.

For current length $n = high - low + 1 \geq 1$, a recursive interval has length at most $\lfloor n/2 \rfloor < n$. The empty interval is the failure base case. Strong induction on length proves termination and correctness.

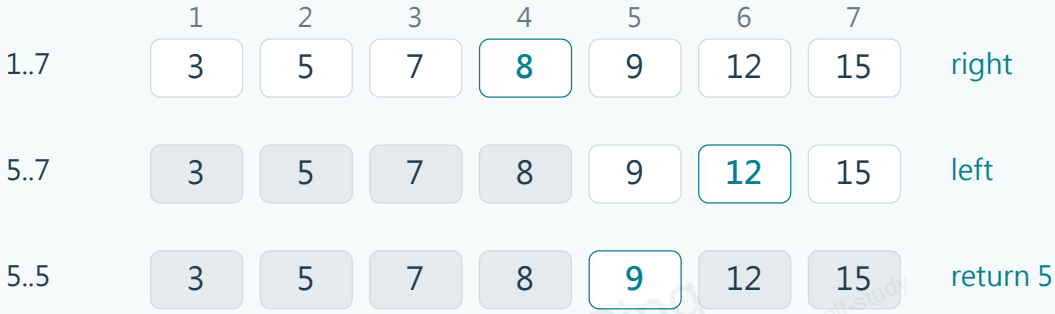
With constant-cost comparisons and index-only arguments, worst-case cost satisfies

$$W(0) = \Theta(1), \qquad W(n) = W(\lfloor n/2 \rfloor) + \Theta(1).$$

The longest path has $\Theta(\lg(n + 1))$ calls; a match at the root costs $\Theta(1)$. Recursive execution has peak stack space $\Theta(\lg(n + 1))$.

Trace both success and failure on the same array

Find 9: white cells remain; teal border marks the probe



For 10: call the empty interval 6..5 next → NIL.

For $A = [3, 5, 7, 8, 9, 12, 15]$, searching for 9 checks $(low, mid, high) = (1, 4, 7), (5, 6, 7), (5, 5, 5)$ and returns 5.

Searching for 10 makes the same first three probes. Since $9 < 10$, it then calls $(6, 5)$ and returns NIL on the empty interval. Gray cells are excluded candidates; the array itself is unchanged.

Copying can change the cost of the same search logic

Keep original length N distinct from current interval length n. On a worst-case search path:

Argument representation	Local work per call	Total worst-case cost
Indices or a view	$\Theta(1)$	$\Theta(\lg(N+1))$
Copy the current interval	$\Theta(n)$	$\Theta(N)$
Copy the original whole array	$\Theta(N)$	$\Theta(N \lg(N+1))$

The second row sums $N + N/2 + N/4 + \cdots = \Theta(N)$. In the third row, N stays fixed throughout recursion; replacing it by n changes the model.

A recursion tree may have only one child per node. It then becomes a path. Multiple recursive calls are not a prerequisite for the method.

Repeated squaring: compute the subproblem once

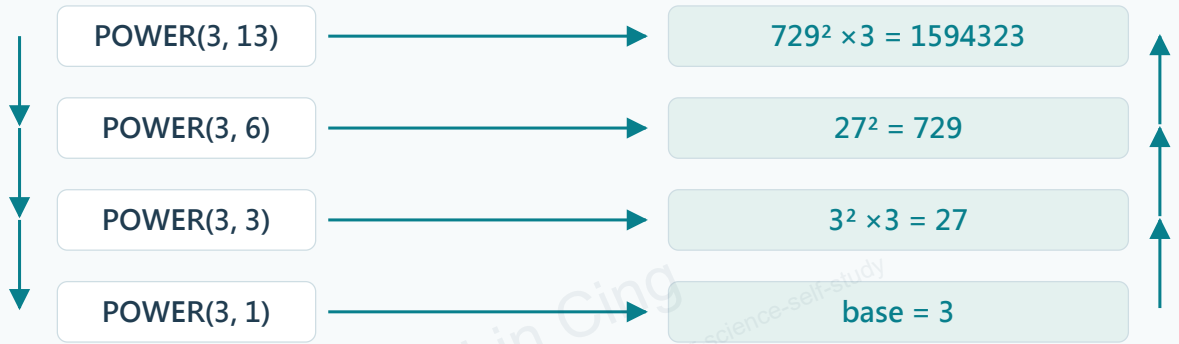
Input is a nonnegative integer n; output is a^n, with $a^0 = 1$. We first count multiplications; this does not make arbitrary-precision multiplication constant-time.

```
POWER(a, n)
    if n == 0: return 1
    if n == 1: return a
    y = POWER(a, floor(n/2))
    z = y*y
    if n is odd: z = z*a
    return z
```

If $n = 2k$, the subcall returns $y = a^k$, so $y^2 = a^n$. If $n = 2k + 1$, multiply once more by a. The two bases and $\lfloor n/2 \rfloor < n$ complete strong induction.

Descend with suspended calls, then compute on return

POWER(3, 13): descend on the left, return on the right

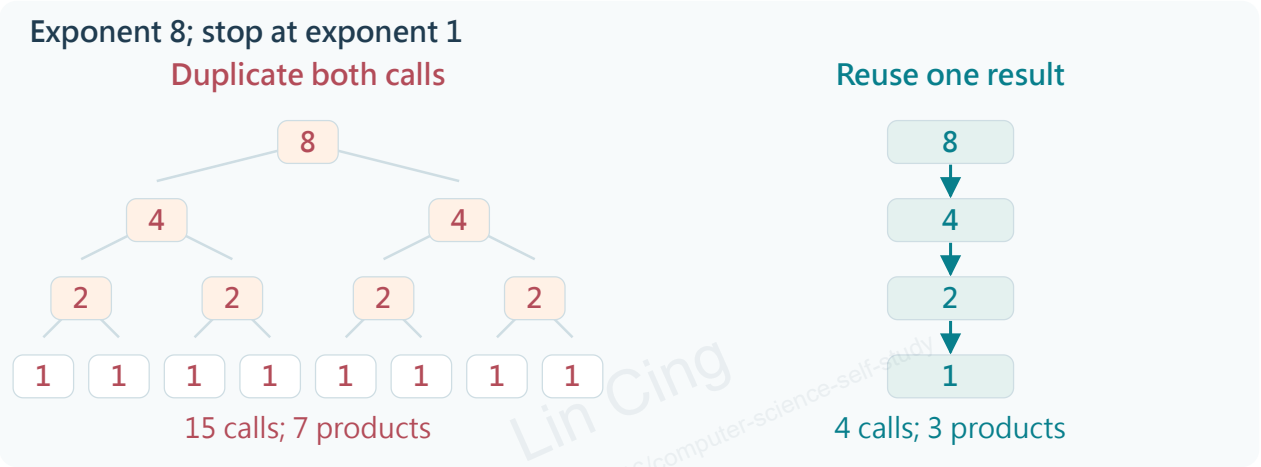


Suspend exponent and return position; each return uses 1 or 2 products.

For 3^{13}, exponents descend as $13 \to 6 \to 3 \to 1$. Returned values are $3 \to 27 \to 729 \to 1594323$, using 0, 2, 1, and 2 multiplications: five in total.

For $n \geq 1$, there are $\lfloor \lg n \rfloor$ squarings and $\text{popcount}(n) - 1$ extra multiplications by a. Total multiplications are their sum. At $n = 0$, the count is zero.

Two occurrences in an identity need not mean two calls



Writing two separate POWER calls for $a^{n/2}a^{n/2}$ gives $D(n) = 2D(n/2) + \Theta(1) = \Theta(n)$ on powers of two, despite equal returned values.

Saving one returned value before squaring gives $R(n) = R(\lfloor n/2 \rfloor) + \Theta(1) = \Theta(\lg n)$ under unit-cost multiplication. Count **executed calls**, not occurrences left after algebraic simplification.

Fixed-width code must handle multiplication overflow. Modular exponentiation may reduce after each product, but the intermediate product must itself be computed safely.

Fibonacci: repeated calls and the size of the answer

Define $F_0 = 0, F_1 = 1, F_n = F_{n-1} + F_{n-2}$. Naive recursion repeats work. Its call count obeys $C_0 = C_1 = 1$ and $C_n = C_{n-1} + C_{n-2} + 1$.

Setting $D_n = C_n + 1$ proves $C_n = 2F_{n+1} - 1$ from the recurrence and bases. Computing $F_5 = 5$ takes 15 calls. Time is $\Theta(\varphi^n)$ but stack depth is only $\Theta(n)$, where $\varphi = (1 + \sqrt{5})/2$.

Computing in increasing order while retaining the previous two values takes $n - 1$ additions for $n \geq 2$. This introduces reuse; full dynamic-programming design comes later.

Separate arithmetic steps from bit complexity. The answer has $\Theta(n)$ bits. Two retained integers need not occupy a constant number of machine words. Sequential ordinary integer additions take $\Theta(n^2)$ bit operations.

Compute Fibonacci exactly through a matrix power

Let $Q = \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}$. For $n \geq 1$,

$$Q^n = \begin{bmatrix} F_{n+1} & F_n \\ F_n & F_{n-1} \end{bmatrix}.$$

The identity holds at $n = 1$. Right multiplication by Q adds the two old columns to form the first new column and copies the first old column to the second. The Fibonacci recurrence proves the next case.

For example, $Q^2 = \begin{bmatrix} 2 & 1 \\ 1 & 1 \end{bmatrix}$ and $Q^4 = \begin{bmatrix} 5 & 3 \\ 3 & 2 \end{bmatrix}$, so $Q^5 = \begin{bmatrix} 8 & 5 \\ 5 & 3 \end{bmatrix}$ gives $F_5 = 5$. Return 0 directly at $n = 0$.

Repeated squaring needs $O(\lg(n+1))$ exact 2×2 matrix multiplications. This counts **arithmetic operations**. Writing the $\Theta(n)$-bit answer already rules out logarithmic bit time. Floating-point evaluation of the closed form is not an unconditional exact algorithm.

Merge Sort and Divide-and-Conquer

Divide and conquer solves strictly smaller instances and combines their results. State boundaries, base cases, merge preconditions, and results. For $A[p..r]$, length is $m = r - p + 1$, with $p > r$ denoting an empty interval.

```
MERGE-SORT(A,p,r)
    if p >= r: return
    q = p + floor((r-p)/2)
    MERGE-SORT(A,p,q)
    MERGE-SORT(A,q+1,r)
    MERGE(A,p,q,r)
```

A legal nonempty call has $1 \leq p \leq r \leq n$. The full call is $(1, n)$, giving immediate return at $(1, 0)$ for empty input. Child lengths are $\lceil m/2 \rceil$ and $\lfloor m/2 \rfloor$, both smaller for $m \geq 2$. Computing the midpoint from a difference avoids the usual $p + r$ overflow, provided the index range itself is representable.

MERGE without an extra key value

Assume $1 \leq p \leq q < r \leq n$ and two stably sorted halves. L and R are **independent copies**, not views affected by overwriting A. Their lengths are $q - p + 1$ and $r - q$.

```
MERGE(A,p,q,r)
    L = copy(A[p..q]); R = copy(A[q+1..r])
    i = 1; j = 1; k = p
    while i <= L.length and j <= R.length
        if L[i].key <= R[j].key: A[k] = L[i]; i = i+1
        else: A[k] = R[j]; j = j+1
        k = k+1
    while i <= L.length
        A[k] = L[i]; i = i+1; k = k+1
    while j <= R.length
        A[k] = R[j]; j = j+1; k = k+1
```

Choose L on equality. Every original left-half record precedes every right-half record, so stability is preserved. Length checks require no key-domain value named infinity.

Proof of merge preservation, order, and termination

Before the main loop,

$$k - p = (i - 1) + (j - 1).$$

The output $A[p..k-1]$ is exactly the stable merge of consumed prefixes $L[1..i-1], R[1..j-1]$. It is sorted, and every emitted key is at most every unconsumed key. Remaining records still exist in independent buffers.

The empty output establishes initialization. When both inputs remain nonempty, their heads are their smallest remaining records. Choosing the smaller head, with left priority on ties, gives the next record of the stable order. Consuming one record and incrementing k preserves lengths, data, order, and stability. Remaining record count decreases by one, proving termination.

At main-loop exit, at least one buffer is exhausted. The other is sorted with keys no smaller than the output tail, so appending it preserves the invariant. Finally $i = |L| + 1, j = |R| + 1, k = r + 1$, and all m records occur once. Without copies, an output write could destroy unread input and invalidate the proof.

Complete merge with duplicate keys

Let $L = [1a, 4b, 4c]$, $R = [2d, 4e, 6f]$, with L originally before R. Letters label records; compare only numeric keys.

i,j before test	Comparison	Output prefix
1,1	$1 \leq 2$; take 1a	1a
2,1	$4 > 2$; take 2d	1a,2d
2,2	$4 \leq 4$; take left 4b	1a,2d,4b
3,2	$4 \leq 4$; take left 4c	1a,2d,4b,4c
4,2	L exhausted; no key comparison	append 4e,6f

There are 4 key comparisons, 6 output writes, and 6 copied buffer records. Interleaving until the last two records can require $m - 1 = 5$ comparisons. $\Theta(m)$ work does not mean exactly m key comparisons.

Complete merge with duplicate keys — diagram

Take left on equality:4b and 4c must precede 4e

	L			R			Output prefix
start	1a	4b	4c	2d	4e	6f	ø
1≤2	1a	4b	4c	2d	4e	6f	1a
4>2	1a	4b	4c	2d	4e	6f	1a, 2d
4≤4	1a	4b	4c	2d	4e	6f	1a, 2d, 4b
4≤4	1a	4b	4c	2d	4e	6f	1a, 2d, 4b, 4c
tail	1a	4b	4c	2d	4e	6f	1a, 2d, 4b, 4c, 4e, 6f

Tinted cells are consumed; unconsumed records remain in independent copies.

Sentinel preconditions

A sentinel implementation can append an infinity record to each buffer and select the smaller head exactly m times, avoiding repeated exhaustion tests. The sentinel key must strictly exceed every legal key, and its record must not be legal input. The maximum legal integer is unsafe as a sentinel.

If legal keys include $+\infty$, take $L = [+\infty]$, $R = [+\infty, +\infty]$. Left priority can select the real left record, then its equal-valued sentinel, then read beyond L on the third output. This is not an out-of-domain marker.

Length checks work for any totally ordered key domain. A tagged sentinel is another option, but it must rank strictly above every real record and never be emitted as data.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

From merge to complete sorting

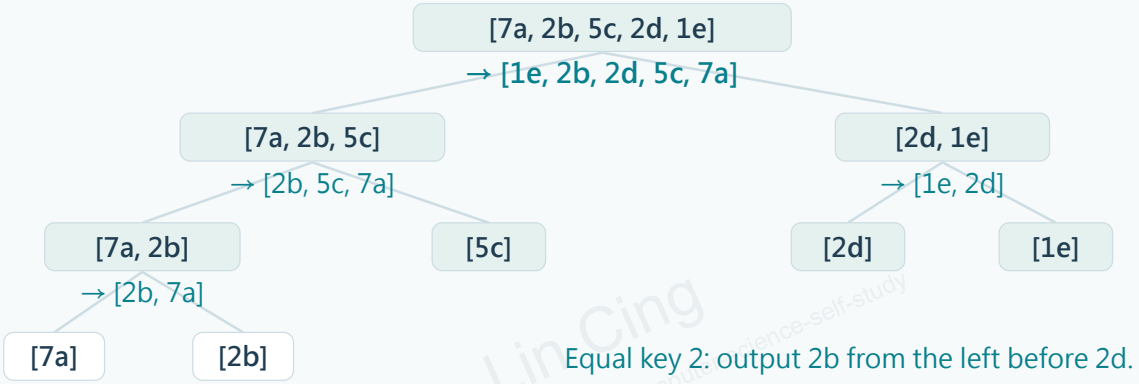
Use strong induction on interval length m. At $m \leq 1$, input is sorted and unchanged. For $m \geq 2$, both smaller calls stably sort and preserve their records by induction. MERGE's preconditions hold, and its proof gives a stable sorting of the whole interval, leaving outside storage unchanged. Child sizes decrease to 0 or 1, proving termination without a power-of-two assumption.

For [7a,2b,5c,2d,1e], splits have lengths 3 and 2, then 2/1 and 1/1. The figure shows every merged node. At the root tie, left 2b still precedes right 2d.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

From merge to complete sorting — diagram

Divide downward; labels show each returned sorted result



Equal key 2: output 2b from the left before 2d.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Include merging in the recurrence

Midpoint computation costs $\Theta(1)$. Copying and outputting m records costs $\Theta(m)$, giving

$$T(m) = T(\lceil m/2 \rceil) + T(\lfloor m/2 \rfloor) + \Theta(m), \quad T(0), T(1) = \Theta(1).$$

For $m = 2^h$, first solve the exact surrogate $U(1) = d$, $U(m) = 2U(m/2) + cm$, with positive c,d. Depth $\ell < h$ has 2^ℓ nodes costing $cm/2^\ell$ each, hence cm per level. There are h internal levels and m leaves of cost d, so $U(m) = cm \lg m + dm$.

Actual internal costs lie between fixed positive multiples of size, and leaves have constant lower and upper costs. Apply the expansion to both bounds to prove Θ(m log m), rather than assuming one exact c describes every instruction and base case.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Arbitrary lengths and a six-record cost table

At depth ℓ, each surviving interval has size at most $\lceil m/2^\ell \rceil$, so height is at most $\lceil \lg m \rceil$. Intervals at one depth are disjoint with total size at most m, proving O(m log m).

At depths $\ell < \lfloor \lg m \rfloor$, each interval has size at least $\lfloor m/2^\ell \rfloor \geq 2$; none has become a leaf. Total size is exactly m, giving Ω(m) at each of $\lfloor \lg m \rfloor$ levels. For $m \geq 2$, this is at least $(\lg m)/2$, proving the lower bound for arbitrary m.

Depth for m=6	Sizes	Total internal size
0	6	6
1	3,3	6
2	2,1,2,1	4
3	1,1,1,1; two earlier leaves	0

Buffer copies and output writes each total 16, not 6 at every depth. There are six base cases.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Peak space differs from cumulative allocation

MERGE allocates L/R only after both recursive calls return and releases them after merging. A length-m merge holds m buffer records. Child buffers do not coexist with parent buffers, giving $\Theta(n)$ peak auxiliary record space plus $\Theta(\lg n)$ stack space, hence $\Theta(n)$ overall for $n \geq 2$.

Cumulative allocation may be $\Theta(n \lg n)$, but it is not simultaneous demand. Retaining completed buffers changes the lifetime model and invalidates this peak bound. A reusable n-record scratch buffer is also possible with the same nonaliasing requirement.

Insertion sort needs only a saved record and constant index words. This merge sort guarantees $\Theta(n \lg n)$ time but is not in-place. Both are stable under their respective strict-shift and left-tie rules.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Substitution: prove one fixed bound

Guess a bound, then use strong induction with the same fixed constants at every size.

Let $T(1) = 1$ and $T(n) = 2T(\lfloor n/2 \rfloor) + n$ for $n \geq 2$. Try $T(n) \leq cn \lg n$. For $n \geq 4$, put $m = \lfloor n/2 \rfloor \geq 2$. Then

$$\begin{aligned} T(n) &\leq 2cm \lg m + n \\ &\leq cn \lg(n/2) + n \\ &= cn \lg n - (c-1)n \leq cn \lg n \end{aligned}$$

for fixed $c \geq 1$. At $n = 1$ the candidate right side is zero, so that input cannot establish this induction. Instead, compute $T(2) = 4, T(3) = 5$ from the recurrence. Fixed $c = 2$ covers both, and every child at $n \geq 4$ is at least 2, closing induction.

A recurrence base and an induction base may differ. The proof must still cover every reached small input.

Lower bounds and stronger hypotheses

First restrict to powers of two, with $T(1) = 1$ and $T(n) = 2T(n/2) + n$. Assuming $T(n/2) \geq d(n/2) \lg(n/2)$ gives

$$T(n) \geq dn \lg n + (1-d)n \geq dn \lg n, \qquad 0 < d \leq 1.$$

The base compares 1 with 0 and holds. Induction proves $\Omega(n \lg n)$. The balanced-level argument earlier extends the bound to integer sizes.

Now consider $T(n) = T(\lfloor n/2 \rfloor) + T(\lceil n/2 \rceil) + 1$, with $T(1) = 1$. The guess $T(n) \leq cn$ yields only $cn + 1$. Strengthen it to

$$T(n) \leq cn - d.$$

Substitution yields $T(n) \leq cn - 2d + 1 \leq cn - d$ for $d \geq 1$. Choosing $c = 2, d = 1$ covers the base; induction in fact gives the exact answer $2n - 1$.

The lower-order deductions at children pay the new work at the parent. Saying that $cn + n$ is still $O(n)$ does not establish the same fixed bound cn and cannot close induction.

Variable changes and strict shrinking

Start on $n = 2^{2^k}$ with $T(n) = 2T(\sqrt{n}) + \lg n$ and $T(2) = \Theta(1)$. Set $m = \lg n$ and $S(m) = T(2^m)$. Then

$$S(m) = 2S(m/2) + m = \Theta(m \lg m).$$

Restoring the variable gives $T(n) = \Theta(\lg n \lg \lg n)$, not a bound with the new variable mistaken for the original input. The next argument treats an integer version with a floor.

A fixed shift also needs a shrinking check. In $T(n) = 2T(n/2 + 17) + n$, children shrink only for $n > 34$. Put $n = 34 + x$ and $U(x) = T(34 + x)$, obtaining

$$U(x) = 2U(x/2) + x + 34.$$

With a sufficiently large fixed base interval, level sums give $\Theta(x \lg x) = \Theta(n \lg n)$. This argument first uses real arguments; integer versions must also bound rounding. Giving only $T(1)$ would leave $n = 34$ calling itself, so the recurrence would not be well-defined.

How to justify a floor inside a square root

Define $T(n) = 1$ for integers $1 \le n \le 3$, and for $n \ge 4$ let

$$T(n) = 2T(\lfloor \sqrt{n} \rfloor) + \lg n.$$

Here $2 \le \lfloor \sqrt{n} \rfloor < n$, so the recurrence is well-defined and terminates.

Set $a_k = 2^{2^k}$ and $U_k = T(a_k)$. Since $\sqrt{a_k} = a_{k-1}$, these points have no rounding error. We have $U_0 = 1$ and $U_k = 2U_{k-1} + 2^k$.

Dividing by 2^k and summing yields $U_k = (k+1)2^k$. Thus at these special inputs, $T(n) = \lg n(1 + \lg \lg n)$.

This establishes a result only on the special sequence. Extending it to every integer requires the monotonicity and sandwich argument next.

Prove monotonicity before sandwiching inputs

The bases satisfy $T(1) = T(2) = T(3) = 1 < T(4)$. Assume smaller-input values are nondecreasing. Since $\lfloor \sqrt{n+1} \rfloor \geq \lfloor \sqrt{n} \rfloor$, both child arguments are below n, and $\lg(n+1) > \lg n$, we get $T(n+1) \geq T(n)$. Strong induction proves monotonicity.

Choose k with $a_k \leq n < a_{k+1}$, equivalently $2^k \leq \lg n < 2^{k+1}$. Monotonicity gives

$$(k+1)2^k \leq T(n) \leq (k+2)2^{k+1}.$$

Within this interval, $\lg n$ is within a factor two of 2^k, and $1 + \lg\lg n$ is within fixed factors of $k+1$. Therefore

$$T(n) = \Theta\big(\lg n(1 + \lg\lg n)\big), \qquad n \geq 2.$$

For sufficiently large n, this is $\Theta(\lg n \lg\lg n)$. The sandwich uses proved monotonicity, not an assumption that the floor can be discarded.

A recursion tree charges local work once

Each internal node stores only its current nonrecursive work $f(\text{size})$; its descendants account for recursive work. Storing the full $T(\text{size})$ at a node and also summing its descendants would double-count.

Take $T(n) = 3T(n/4) + n^2$, $T(1) = 1$, initially on $n = 4^L$. Depth j has 3^j nodes, each costing $(n/4^j)^2$, so its total is $n^2(3/16)^j$. There are $3^L = n^{\log_4 3}$ leaves. Thus

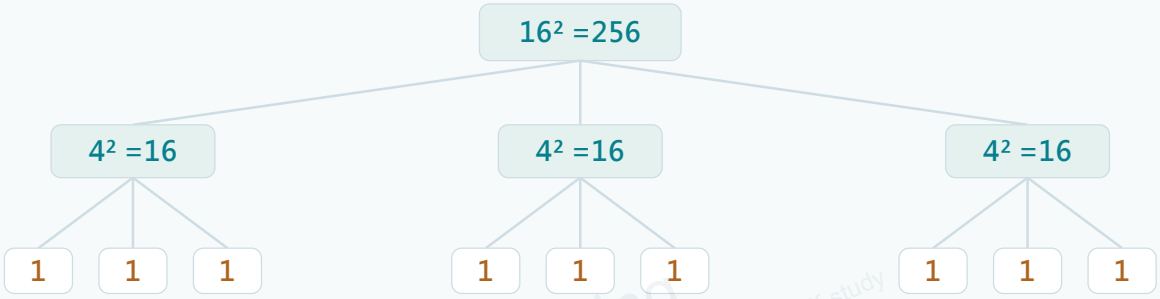
$$T(n) = n^2 \sum_{j=0}^{L-1} (3/16)^j + n^{\log_4 3}.$$

The geometric sum is at most $16/13$, and leaf work is at most n^2, proving $O(n^2)$. Root work supplies the matching $\Omega(n^2)$ lower bound.

At $n = 16$, level costs are 256,48,9, totaling 313. The final 9 already pays for nine base cases; do not add $f(1)$ again at each leaf.

A recursion tree charges local work once — diagram

$T(n)=3T(n/4)+n^2$, $T(1)=1$; $n=16$



Level costs:256 →48 →9 (nine base calls)
Total 313; do not add f(1) again at leaves.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A contracting level sum still needs leaf costs

For integer $n \geq 8$, define

$$T(n) = T(\lfloor n/2 \rfloor) + T(\lfloor n/4 \rfloor) + T(\lfloor n/8 \rfloor) + n,$$

with $T(n) = 1$ for $0 \leq n \leq 7$. Child sizes sum to at most $7n/8$. Internal work at depth j is at most $n(7/8)^j$, totaling at most $8n$.

Let I and L count internal nodes and leaves. Every internal node has three children, so counting edges gives $3I = I + L - 1$ and $L = 2I + 1$. Each internal node costs at least 8, hence $8I \leq 8n$ and $L \leq 2n + 1$.

Thus $T(n) \leq 8n + 2n + 1 = O(n)$. The root supplies $T(n) \geq n$, proving $\Theta(n)$. **The geometric sum pays for internal work; a separate node count pays for leaves.**

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Unequal splits: depth and mass per level

Let $a = \lfloor n/3 \rfloor$, $b = n - a$, and $T(n) = T(a) + T(b) + cn$ for fixed $c > 0$, with fixed positive bounds on small-input costs.

Along any path, the depth-j size q_j satisfies

$$\frac{n}{3^j} - \frac{3}{2} \le q_j \le n(2/3)^j + 3.$$

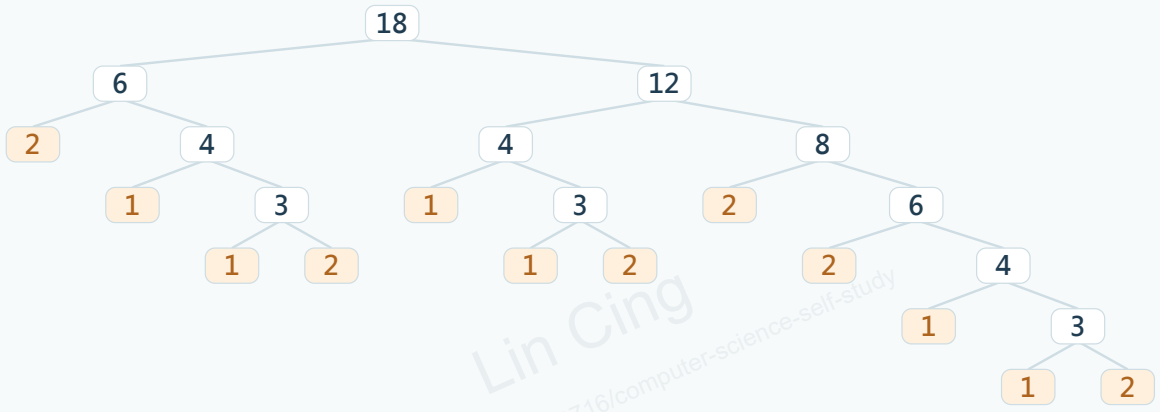
Induct from $q/3 - 1 \le q_{\text{next}} \le 2q/3 + 1$ and sum the geometric rounding errors. Absorb all work at sizes up to fixed $K \ge 6$ into base cases; this changes only constants. The upper bound gives $O(\lg n)$ height. Disjoint intervals at a depth have total length at most n and cost at most cn. Positive integer leaf lengths sum to n, so there are at most n leaves. Total cost is $O(n \lg n)$.

For $0 \le j \le \lfloor \log_3(n/(K+2)) \rfloor$, the lower bound keeps every $q_j > K$. No path has stopped: each complete level retains mass n and costs cn. There are $\Omega(\lg n)$ such levels, proving the matching lower bound.

With only $O(n)$ local work, only the upper bound follows. Zero combination work can leave linear leaf cost alone.

Unequal splits: the complete call tree

n=18; white nodes split, orange nodes stop at n≤2

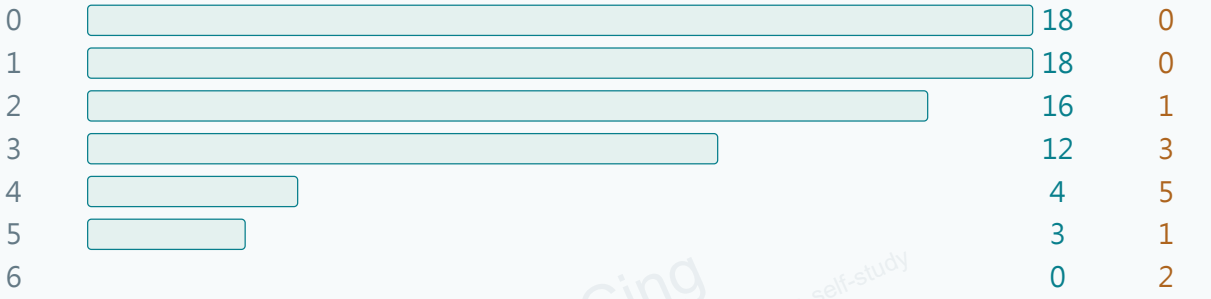


11 internal nodes, 12 leaves; leaf depths range from 2 to 6.

Each edge is one executed call. Orange leaves stop at different depths; the deepest path is 18 →12 →8 →6 →4 →3 →2.

Unequal splits: distinguish complete and truncated levels

n=18; stop at n≤2; bars measure internal mass per level



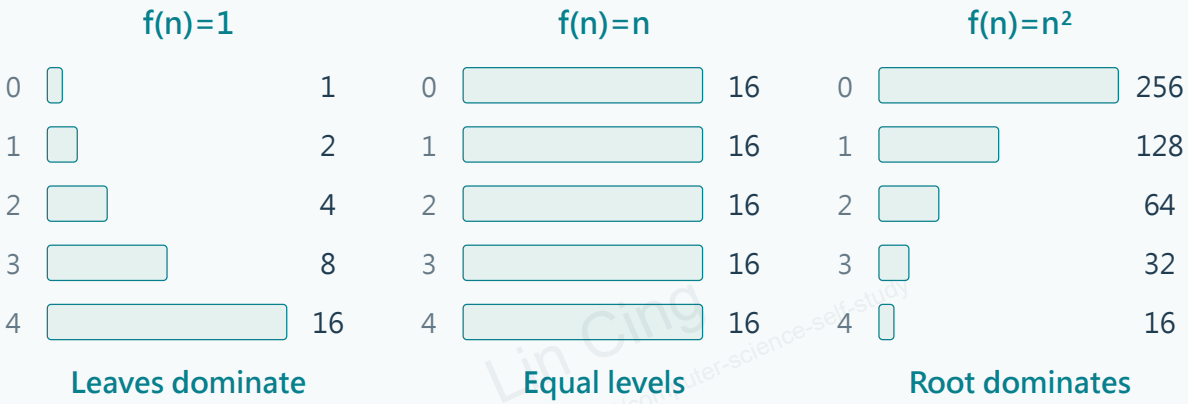
Charge leaves separately; stopped nodes produce no descendants.

The figure stops at $n \leq 2$; larger nodes split into $\lfloor n/3 \rfloor$ and $n - \lfloor n/3 \rfloor$. Complete levels initially have mass 18. After leaves appear, **only internal nodes** pay combination work.

The shortest path limits the complete levels usable for a lower bound. The longest path controls the depth needed for an upper bound. Multiplying the deepest height by n does not establish a lower bound for truncated levels.

Read the level work behind the three cases — diagram

Independent scale per panel; labels are total cost per level



Read the level work behind the three cases

All three use $n = 16, a = b = 2, T(1) = 1$. Nonrecursive terms are $1, n, n^2$. Costs by level, including leaves:

$f(n)$	Depths 0,1,2,3,4	Total
1	1,2,4,8,16	31
n	16,16,16,16,16	80
n^2	256,128,64,32,16	496

These illustrate leaf, equal-level, and root domination. The theorem still requires fixed parameters, polynomial gaps, and case 3 regularity. A drawing alone does not verify those conditions.

Master theorem: check the hypotheses

Let fixed $a \geq 1, b > 1$ and nonnegative local work f define

$$T(n) = aT(n/b) + f(n), \qquad p = \log_b a.$$

Base costs on a constant-size interval have fixed positive upper and lower bounds. Start on exact powers; rounding conditions follow later. An actual call count a is integral; a real a can instead be a recurrence weight.

Case	Required hypotheses	Result
1	Fixed $\epsilon > 0$; $f(n) = O(n^{p-\epsilon})$	$\Theta(n^p)$
2	$f(n) = \Theta(n^p)$	$\Theta(n^p \lg n)$
3	Fixed $\epsilon > 0$; $f(n) = \Omega(n^{p+\epsilon})$; eventually $af(n/b) \leq cf(n)$ for fixed $c < 1$	$\Theta(f(n))$

Merely smaller or larger than n^p is insufficient. Cases 1 and 3 require a fixed polynomial gap; case 3 also requires regularity.

Complete case classification examples

Every row assumes fixed positive base costs. Compute $p = \log_b a$ rather than guessing it from a plot.

$(a, b, f(n))$	Concrete hypothesis check	Tight bound
$(9, 3, n)$	$p = 2$; choose $\epsilon = 1$	$\Theta(n^2)$
$(1, 3/2, 1)$	$p = 0$; $f(n) = \Theta(n^0)$	$\Theta(\lg n)$
$(3, 4, n \lg n)$	$p \approx 0.7925$; choose $\epsilon = 0.1$; regularity ratio at most $3/4$	$\Theta(n \lg n)$
$(8, 2, n^2)$	$p = 3$; choose $\epsilon = 1$	$\Theta(n^3)$
$(7, 2, n^2)$	$p = \lg 7$; choose $\epsilon = \lg 7 - 2 > 0$	$\Theta(n^{\lg 7})$

For the third row,

$$\frac{3f(n/4)}{f(n)} = \frac{3}{4}\frac{\lg n - 2}{\lg n} \le \frac{3}{4} \quad (n \ge 4).$$

By contrast, $2T(n/2) + n \lg n$ has no fixed polynomial gap. None of the classical three cases applies directly, but the level sums next still solve it.

Sum logarithmic boundary cases

Let $n = b^L, p = \log_b a$, and $f(n) = \Theta(n^p (\lg n)^k)$ for fixed real k. Above a fixed base interval, renumber depth j by distance $r = L - j$ from the leaf end. Level cost is $\Theta(n^p r^k)$.

Add $\Theta(n^p)$ leaf work to n^p times the resulting power sum:

$$T(n) = \begin{cases} \Theta(n^p (\lg n)^{k+1}) & k > -1, \\ \Theta(n^p \lg \lg n) & k = -1, \\ \Theta(n^p) & k < -1. \end{cases}$$

For $k > -1$, integral comparison gives $\sum_{r=1}^{L} r^k = \Theta(L^{k+1})$. At $k = -1$ it is harmonic; for $k < -1$ it converges to a positive constant. For negative k, apply the formula only above a fixed base interval larger than 1, avoiding negative powers of $\lg 1$.

Thus $2T(n/2) + n/\lg n$ gives $\Theta(n \lg \lg n)$; replacing its local term by $n/(\lg n)^2$ gives $\Theta(n)$. These follow from level sums, not classical case 1.

A counterexample without regularity

Restrict to $n = 2^L$, with $T(1) = 1$ and

$$f(n) = \begin{cases} n^2 & L \text{ even}, \\ n^3 & L \text{ odd}, \end{cases} \qquad T(n) = 2T(n/2) + f(n).$$

Always $f(n) \geq n^2$, giving a fixed polynomial gap above $n^p = n$. For even $L \geq 2$, however, the child exponent is odd:

$$T(n) \geq 2f(n/2) = n^3/4, \qquad \frac{T(n)}{f(n)} \geq n/4 \to \infty.$$

Hence $T(n) \notin \Theta(f(n))$. The regularity ratio itself equals $n/4$, so no common $c < 1$ works eventually.

This illustrates omission of a hypothesis, not failure of the theorem. For an oscillating function, a bounded change in its argument need not produce comparable values.

Why the master theorem holds

Start with $n = b^L, T(1) = t > 0$. Expanding L times gives

$$T(n) = tn^p + \sum_{j=0}^{L-1} a^j f(n/b^j).$$

Depth j has total weight a^j; when a counts actual calls, it is also the node count. The leaf term tn^p gives a positive lower bound in every case.

In case 1, eventually $f(x) \leq Cx^{p-\epsilon}$. Large-size levels contribute at most

$$Cn^{p-\epsilon} \sum_{j=0}^{L-1} (b^\epsilon)^j = O(n^{p-\epsilon} b^{\epsilon L}) = O(n^p).$$

The ratio $b^\epsilon > 1$ makes this sum increase toward the leaves. Extending the bound across finitely many small arguments adds at most $O(n^p)$: their total weight has that order, and their fixed function values are absorbed into constants.

Adding leaves gives $O(n^p)$; their positive cost supplies $\Omega(n^p)$, proving $\Theta(n^p)$.

Equal-level and root-dominated proofs

Case 2. Since $a = b^p$, every sufficiently large-size level costs

$$a^j \Theta((n/b^j)^p) = \Theta(n^p).$$

There are $L - O(1) = \Theta(\lg n)$ such levels. Summing their upper and lower bounds gives $\Theta(n^p \lg n)$ and covers leaf work.

Case 3. Repeated regularity gives, while arguments remain sufficiently large,

$$a^j f(n/b^j) \le c^j f(n), \qquad 0 < c < 1.$$

Their sum is at most $f(n)/(1-c)$. Remaining fixed-size levels and leaves cost $O(n^p)$, which is $O(f(n))$ by polynomial separation. The root's local work $f(n)$ supplies the matching lower bound, proving $\Theta(f(n))$.

Small-input work is not constant across the entire tree. There can be $\Theta(n^p)$ small nodes; their total must be included.

Rounding errors remain bounded

Along an actual call path, let $q_0 = n$ and let q_{j+1} be either $\lfloor q_j/b \rfloor$ or $\lceil q_j/b \rceil$. Both satisfy $|q_{j+1} - q_j/b| \le 1$. For fixed $C \ge b/(b-1)$, induction gives

$$|q_j - n/b^j| \le C,$$

because the next error is at most $C/b + 1 \le C$. It does not accumulate linearly with depth.

Choose a fixed base interval $q \le n_0$, with $n_0 > C + 2$ and large enough for strict shrinking. The two error bounds place every leaf at depth $\log_b n + O(1)$. Where $n/b^j \ge 2C$, actual sizes and unrounded sizes are within fixed factors.

With an integral number a of actual children, leaf depths differ by only a constant, giving $\Theta(a^{\log_b n}) = \Theta(n^p)$ leaves and $O(n^p)$ total fixed-size work. For $a = 1$, the tree is one path with constant bottom work.

Conditions for all integer sizes

Here a is an integral number of actual subcalls. Real weights on exact powers were handled by the earlier expansion.

Cases 1 and 2: Actual sizes q_j and n/b^j are within fixed factors on large-size levels. Even for a negative fixed exponent,

$$q_j^{p-\epsilon} = \Theta((n/b^j)^{p-\epsilon}).$$

Case 1 therefore retains its polynomial bound and geometric sum. In case 2, each complete large-size level still costs $\Theta(n^p)$, with $\Theta(\lg n)$ levels. Charge incomplete fixed-size levels separately as $O(n^p)$. The logarithmic extension also survives: away from the fixed base interval, $\log q_j$ is comparable to distance from the bottom.

Case 3: Apply regularity to actual rounded arguments. A sufficient condition is

$$a \max\{f(\lfloor x/b \rfloor), f(\lceil x/b \rceil)\} \le cf(x), \qquad c < 1.$$

Then level work contracts by c, and polynomial separation absorbs bottom work. Bounded argument error alone does not justify dropping this condition for arbitrary f.

Regularity can imply a polynomial gap

On exact powers, suppose $af(x/b) \le cf(x)$ for fixed $0 < c < 1$, and $f(x_0) > 0$ at a fixed bottom point. Reversing the inequality through k levels gives

$$f(b^k x_0) \ge (a/c)^k f(x_0).$$

Since $\log_b(a/c) = p + \log_b(1/c)$, set $\epsilon = \log_b(1/c) > 0$. Along this sequence, $f(n) = \Omega(n^{p+\epsilon})$.

This explains why some versions omit a separate polynomial-gap hypothesis under suitable positive bottom conditions. We retain the complete three-case assumptions for direct checking.

The argument covers $n = b^k x_0$. Extending it to every real or integer input requires control of the final base interval and a positive lower bound for f there. A proof on one sequence alone does not cover all inputs.

Maximum nonempty contiguous subarray

Given $n \geq 1$ finite real values $A[1..n]$, find a nonempty contiguous interval $[i, j]$ maximizing

$$\sum_{k=i}^{j} A[k],$$

and return (i, j, sum). Any maximizing interval is acceptable. We count exact additions and comparisons; floating-point error, NaN, and overflow are outside this model.

For prices $P_0, \ldots, P_n$, let $A_i = P_i - P_{i-1}$. Then $P_j - P_{i-1} = \sum_{k=i}^{j} A_k$: interval $i..j$ means buying on day $i - 1$ and selling on day j.

Allowing no transaction adds an empty interval with value 0 and changes all-negative cases. Enumerating each start and extending its end with a running sum takes $\Theta(n^2)$ time and $O(1)$ auxiliary space. Re-summing every interval instead takes $\Theta(n^3)$.

Crossing means suffix plus prefix

For $low < high$, set $mid = low + \lfloor (high - low)/2 \rfloor$. Every nonempty interval belongs to exactly one class:

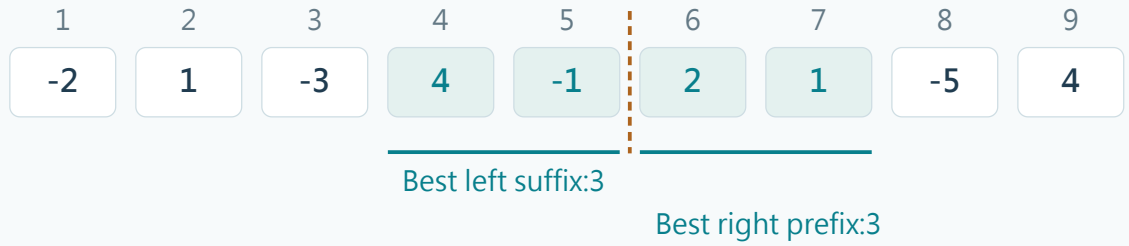
1. Entirely in $low..mid$.
2. Entirely in $mid + 1..high$.
3. Crossing the boundary, with $i \leq mid < j$.

A crossing interval contains both mid and $mid + 1$. It splits uniquely into a left suffix $i..mid$ and a right prefix $mid + 1..j$.

The two endpoints can be chosen independently. Thus its best sum is **the best nonempty left suffix plus the best nonempty right prefix**. Scan both sides separately rather than enumerating endpoint pairs. The best subarray within each half need not touch the boundary, so adding those two answers is invalid.

Crossing means suffix plus prefix — diagram

mid=5: a crossing interval includes positions 5 and 6



Left sums:−1, 3, 0, 1,−1 → best 3, start 4

Right sums:2, 3,−2, 2 → best 3, end 7

Combine A[4..7]=[4,−1, 2, 1], sum 6.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Complete crossing scan

```
CROSS(A, low, mid, high):
    left = -infinity; total = 0
    for i = mid downto low:
        total += A[i]
        if total > left: left = total; best_i = i
    right = -infinity; total = 0
    for j = mid+1 to high:
        total += A[j]
        if total > right: right = total; best_j = j
    return (best_i, best_j, left+right)
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

After each left-loop update, `total` is the sum of $A[i..mid]$, and `left` is the maximum among all left suffixes examined so far. The initial $-\infty$ is a comparison marker, not input data; the first finite candidate updates it and sets an endpoint.

Each iteration adds exactly one candidate and takes the maximum, preserving the invariant. All left suffixes are covered at termination; the right prefixes are analogous. Adding the two maxima gives the best crossing sum. Total scan iterations are

$$(mid - low + 1) + (high - mid) = high - low + 1.$$

Recursive maximum subarray and proof

```
MAX-SUBARRAY(A, low, high):
    if low == high: return (low, high, A[low])
    mid = low + floor((high-low)/2)
    L = MAX-SUBARRAY(A, low, mid)
    R = MAX-SUBARRAY(A, mid+1, high)
    C = CROSS(A, low, mid, high)
    return a tuple with largest sum among L, R, C
```

Use strong induction on interval length. A singleton is its only answer. For a longer interval, both children are nonempty and smaller. Their contracts solve the first two classes; CROSS solves the third. The classes are disjoint and exhaustive, so taking their maximum is correct. Strict shrinking also guarantees termination.

Operation cost is

$$T(n) = T(\lfloor n/2 \rfloor) + T(\lceil n/2 \rceil) + \Theta(n) = \Theta(n \lg n).$$

Sequential subcalls with index arguments retain only constant many indices and sums per frame, giving $O(\lg n)$ peak stack space. Copying subarrays changes allocation and lifetime costs and must be analyzed separately.

Nine-element crossing trace

For $A = [-2, 1, -3, 4, -1, 2, 1, -5, 4]$, use $low = 1, high = 9, mid = 5$.

Left index i	Sum of $A[i..5]$	Best sum and start so far
5	-1	-1,5
4	3	3,4
3	0	3,4
2	1	3,4
1	-1	3,4

Right-prefix sums at $j = 6, 7, 8, 9$ are $2, 3, -2, 2$, with best interval $6..7$ of sum 3. The crossing answer is $4..7$ of sum 6. Left-only and right-only maxima are $4..4$ and $9..9$, both of sum 4. Return $(4, 7, 6)$.

For all-negative input $[-8, -2, -5]$, the nonempty contract returns $(2, 2, -2)$, not 0.

A linear algorithm's state and invariant

Let E_i be the best nonempty sum ending exactly at i, and B_i the best nonempty sum within $1..i$:

$$E_1 = B_1 = A_1, \qquad E_i = \max(A_i, E_{i-1} + A_i), \quad B_i = \max(B_{i-1}, E_i).$$

An interval ending at i either consists only of A_i or extends one ending at $i-1$. In the latter case, the best previous ending sum suffices. This classification and the bases prove both state meanings by induction.

Choosing A_i resets the start to i; extending preserves the old start. Save that start and endpoint i when the global best improves. Constant work per iteration gives $\Theta(n)$ time and $O(1)$ auxiliary space.

For the nine-value example, $E_i = [-2, 1, -2, 4, 3, 5, 6, 1, 5]$ and $B_i = [-2, 1, 1, 4, 4, 5, 6, 6, 6]$. Divide-and-conquer is not the fastest solution here; it illustrates a different classification and combination proof.

From scalar products to block multiplication

Standard multiplication computes $C_{ij} = \sum_{k=1}^{n} A_{ik} B_{kj}$. After step k, the accumulator contains the first k products. The n^2 outputs each take n multiply-add steps: $\Theta(n^3)$ operations for this algorithm, not a universal lower bound.

For even n, use $n/2$-order blocks:

$$A = \begin{bmatrix} a & b \\ c & d \end{bmatrix}, \qquad B = \begin{bmatrix} e & f \\ g & h \end{bmatrix}.$$

Their product satisfies

$$C_{11} = ae + bg, \quad C_{12} = af + bh, \quad C_{21} = ce + dg, \quad C_{22} = cf + dh.$$

Preserve multiplication order; matrix blocks generally do not commute.

Full cost of eight recursive products

Pad to a power-of-two order, then extract the original-size result. Each call reads its own view's order rather than reusing the original order.

```
BLOCK-MULTIPLY(A,B):
    n = A.rows
    if n == 1: return [[A[1,1]*B[1,1]]]
    view A as [[a,b],[c,d]], B as [[e,f],[g,h]]
    C11 = BLOCK-MULTIPLY(a,e)+BLOCK-MULTIPLY(b,g)
    C12 = BLOCK-MULTIPLY(a,f)+BLOCK-MULTIPLY(b,h)
    C21 = BLOCK-MULTIPLY(c,e)+BLOCK-MULTIPLY(d,g)
    C22 = BLOCK-MULTIPLY(c,f)+BLOCK-MULTIPLY(d,h)
    return the four blocks assembled into C
```

An index view retains an origin, stride, and order, permitting constant-time splitting. Explicit block copies cost $\Theta(n^2)$, the same order as fixed many block additions. Hence $T(n) = 8T(n/2) + \Theta(n^2)$ with a $\Theta(1)$ base.

Eight recursive products give 8^j calls at depth j and $8^{\lg n} = n^3$ leaves. The branch count changes the exponent; it cannot be hidden in one child's constant.

All seven Strassen products

Compute each product recursively with STRASSEN. Additions and subtractions inside parentheses are elementwise block operations.

$$\begin{aligned} M_1 &= (a+d)(e+h), & M_2 &= (c+d)e, \\ M_3 &= a(f-h), & M_4 &= d(g-e), \\ M_5 &= (a+b)h, & M_6 &= (c-a)(e+f), \\ M_7 &= (b-d)(g+h). \end{aligned}$$

Assemble the four output blocks:

$$\begin{aligned} C_{11} &= M_1 + M_4 - M_5 + M_7, & C_{12} &= M_3 + M_5, \\ C_{21} &= M_2 + M_4, & C_{22} &= M_1 - M_2 + M_3 + M_6. \end{aligned}$$

For $n = 1$, multiply the two scalars. For $n > 1$, split, compute these seven products, and assemble the four blocks. Together with the base case, these steps specify the recursive procedure.

Correctness of all four output blocks

Use distributivity and additive cancellation, preserving every product's factor order:

$$\begin{aligned} C_{11} &= (ae + ah + de + dh) + (dg - de) \\ &\quad - (ah + bh) + (bg + bh - dg - dh) = ae + bg, \\ C_{12} &= (af - ah) + (ah + bh) = af + bh, \\ C_{21} &= (ce + de) + (dg - de) = ce + dg, \\ C_{22} &= (ae + ah + de + dh) - (ce + de) \\ &\quad + (af - ah) + (ce + cf - ae - af) = cf + dh. \end{aligned}$$

Scalar multiplication is a correct base. If the seven smaller products are correct by induction, these identities assemble all four blocks of AB, completing the proof.

The identities require addition, subtraction, and distributivity. They do not directly apply to a general semiring without subtraction.

Numerical products and reconstruction

Take $A = \begin{bmatrix} 2 & -1 \\ 3 & 4 \end{bmatrix}$ and $B = \begin{bmatrix} 5 & 2 \\ -2 & 1 \end{bmatrix}$.

Product	Substitution	Value
M_1	$(2 + 4)(5 + 1)$	36
M_2	$(3 + 4)5$	35
M_3	$2(2 - 1)$	2
M_4	$4(-2 - 5)$	-28
M_5	$(2 - 1)1$	1
M_6	$(3 - 2)(5 + 2)$	7
M_7	$(-1 - 4)(-2 + 1)$	5

The four blocks are $36 - 28 - 1 + 5 = 12$, $2 + 1 = 3$, $35 - 28 = 7$, and $36 - 35 + 2 + 7 = 10$. Direct multiplication verifies:

$$AB = \begin{bmatrix} 10 + 2 & 4 - 1 \\ 15 - 8 & 6 + 4 \end{bmatrix} = \begin{bmatrix} 12 & 3 \\ 7 & 10 \end{bmatrix}.$$

Numerical products and reconstruction — diagram

Keep C11,C12,C21,C22 in their correct block positions

A			B			C	
2	-1	×	5	2	=	12	3
3	4		-2	1		7	10

C11: $36-28-1+5=12$

C12: $2+1=3$

C21: $35-28=7$

C22: $36-35+2+7=10$

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Time, padding, and memory

The seven Strassen subproducts need ten input-block additions/subtractions and eight more to assemble the outputs. View splitting is $O(1)$, giving

$$T(n) = 7T(n/2) + \Theta(n^2) = \Theta(n^{\lg 7}), \qquad \lg 7 \approx 2.80735.$$

Larger addition constants may make small matrices slower. A fixed cutoff to standard multiplication preserves the exponent. Algebraic correctness alone gives no floating-point error guarantee.

For arbitrary $n \geq 1$, pad to $m = 2^{\lceil \lg n \rceil}$, where $n \leq m < 2n$. The top-left $n \times n$ result is the original product, and $m^{\lg 7} < 7n^{\lg 7}$ preserves the order. Return an empty matrix directly at zero size.

Execute subcalls sequentially and release temporaries promptly. Retaining only fixed many current-level blocks gives peak space

$$S(n) \leq S(n/2) + O(n^2) = O(n^2),$$

plus $O(\lg n)$ frames. Simultaneously retaining every child workspace requires a different space recurrence.

Extension: conditions for Akra–Bazzi

We cite the theorem in MIT's *Mathematics for Computer Science* (2010), § 10.4.3, using positive bounded base costs as sufficient conditions. **Applications below are not a full proof of this theorem.**

$$T(x) = \sum_{i=1}^{k} a_i T(b_i x + h_i(x)) + g(x), \qquad x > x_0.$$

- Fixed finite k, fixed $a_i > 0$, and fixed $0 < b_i < 1$.
- Fixed $0 < d_1 \le T(x) \le d_2$ on $0 \le x \le x_0$.
- Nonnegative differentiable g, with $|g'(x)|$ polynomially bounded; $|h_i(x)| = O(x/\log^2 x)$.
- Choose x_0 so every child lies in $[0, \rho x]$ for a fixed $\rho < 1$, making recursion well-defined and terminating.

These are the conditions used here. Zero base costs and identically zero g would give identically zero cost, invalidating a conclusion with a positive leaf term.

Find the unique exponent before integrating

Let p satisfy $\sum_i a_i b_i^p = 1$. The cited theorem gives

$$T(x) = \Theta\left(x^p \left[1 + \int_1^x \frac{g(u)}{u^{p+1}}\, du\right]\right).$$

Since $0 < b_i < 1$, $F(p) = \sum_i a_i b_i^p$ is continuous and strictly decreasing, tending to infinity as $p \to -\infty$ and to zero as $p \to +\infty$. The real root is unique.

For $T(x) = T(x/3) + T(2x/3) + x$, with positive constant base costs, the equation $(1/3)^p + (2/3)^p = 1$ gives $p = 1$. Therefore

$$T(x) = \Theta\left(x \left[1 + \int_1^x \frac{du}{u}\right]\right) = \Theta(x \log x).$$

This agrees with the earlier complete-level and maximum-depth bounds. Keep the assumptions of both arguments explicit.

Rounding and a changed ratio are different

Floor, ceiling, and fixed shifts have bounded additive error, satisfying $O(x/\log^2 x)$. Choose a sufficiently large base interval to ensure shrinking.

In $T(n) = T(n/2 + n/4) + T(n/2) + n$, however, the first child is $3n/4$. The term $n/4$ is not a small perturbation; two one-half ratios give the wrong equation.

The correct equation is $(3/4)^p + (1/2)^p = 1$, with unique root $p \approx 1.507126591639 > 1$. Since

$$\int_1^n u^{-p}\,du = \frac{1 - n^{1-p}}{p-1} = \Theta(1),$$

we obtain $T(n) = \Theta(n^p)$. The equation defines the exact exponent; the decimal is approximate.

If another source writes child sizes as n/c_i, its parameters correspond to $b_i = 1/c_i$ here. Convert the equation together with the notation.

Further reading and theorem sources

- Cormen, Leiserson, Rivest, Stein, *Introduction to Algorithms*, 3rd Edition: Chapter 2 for merge sort; Chapter 4 for divide-and-conquer, substitution, recursion trees, and the master theorem with its proof.
- Demaine and Leiserson, MIT 6.046J (Fall 2005), Lectures 2–3: recurrences and their connection to binary search, powering, Fibonacci, and matrix multiplication.
- Lehman, Leighton, Meyer, *Mathematics for Computer Science* (2010), § 10.4.3, Theorem 10.4.1: official MIT notes. We use positive bounded base costs as sufficient conditions.

The chapter proves the three master-theorem cases. Akra–Bazzi is a cited extension; its applications verify hypotheses, solve the exponent equation, and evaluate the integral.

References and attribution

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Sorting, Selection, and Randomized Analysis

Sorting determines the order of every record; selection asks for one rank. They can share partitioning and heaps without sharing the same time bound.

Read in this order: **comparison-model limits → heaps and sorting → probability tools → partitioning and quicksort → counting, radix, and bucket sorting → order statistics**. Further examples use collisions, loads, waiting times, and online selection to practice probabilistic modeling.

Prerequisites: specifications, invariants, and sums from Chapter 1; divide-and-conquer and recurrences from Chapter 2. Probability tools are developed before their use in expected-time analysis.

Reference textbook: Cormen et al., *Introduction to Algorithms*, 3rd Edition, Chapters 5–9; supplementary reference: MIT 6.046J (2005), Lectures 4–6.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Specify the records and the sorting guarantee

A record contains a **key** and fields that move with it. Identity labels a, b, c distinguish equal-key records; comparisons inspect only the key unless stated otherwise.

A correct sort produces nondecreasing keys and preserves the input multiset of records. **Stability** additionally preserves the original order of equal-key records.

For input $[2_a, 1_b, 2_c]$, output $[1_b, 2_a, 2_c]$ is stable. Output $[1_b, 2_c, 2_a]$ is sorted but unstable. Output $[1_b, 2_a, 2_a]$ loses 2_c and violates the sorting contract itself.

Use **1-based closed intervals** $A[p..r]$, with $p > r$ denoting an empty interval. Key-indexed counting tables may start at 0. Selection rank k is always 1-based and local to the current subarray.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Count comparisons, time, and space separately

Unless stated otherwise, comparing or moving a fixed-size record and accessing one index take constant time. Long strings, large integers, and external objects require explicit comparison, digit-access, and movement costs.

Question	Conditions that must be fixed
Worst-case time $\max_A T(A)$	Length and legal input set
Average-case time $\mathbb{E}_A[T(A)]$	Joint input distribution
Randomized time $\mathbb{E}_R[T(A, R)]$	Fixed input A; average over internal randomness R
Auxiliary space	Whether output, buffers, and live call frames are counted

An in-place partition does not remove quicksort's recursion stack. A small expected time does not exclude an expensive random execution.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A comparison-tree leaf must identify an ordering

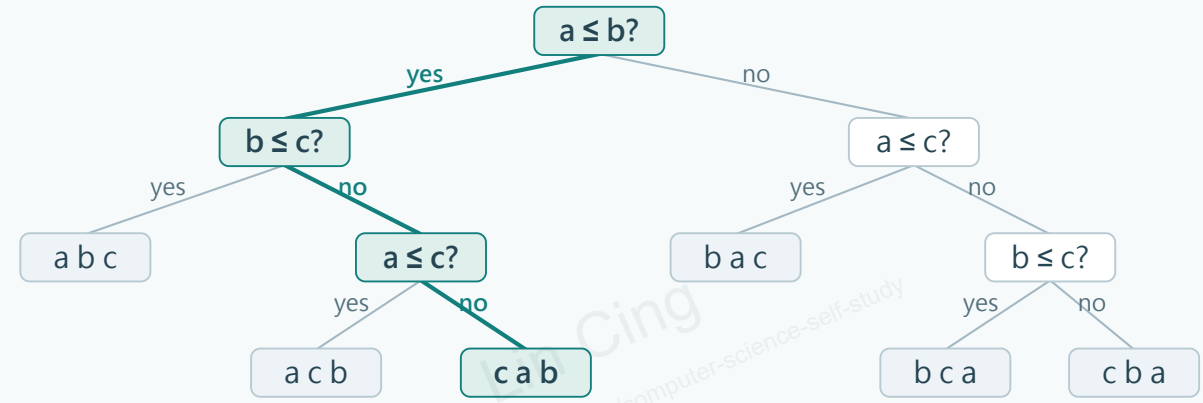
First restrict attention to n distinct keys. A comparison sort learns order only through comparisons between keys, without using key values as array indices. Fix n and represent all executions by a binary tree.

Each internal node identifies two original records. Its branches record $\leq$ or $>$. A root-to-leaf path records the observations, and its leaf must determine the complete identity order.

The $n!$ relative orders require distinct reachable leaves. If two orders reached the same leaf, the algorithm would observe the same outcomes and output the same identity permutation; at least one would be incorrect. Thus at least $n!$ leaves are reachable.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Follow the comparisons to the identity order



a=4, b=7, c=1: yes → no → no → c a b. Leaves name record identities.

Leaf c, a, b means $c < a < b$. For $a = 4, b = 7, c = 1$, follow the comparison outcomes to that leaf to check one complete execution.

A worst-case lower bound requires some hard input

A binary tree of height h has at most 2^h leaves, so

$$n! \le 2^h, \qquad h \ge \lceil \lg(n!) \rceil.$$

For $n \ge 2$, at least $\lfloor n/2 \rfloor$ final factors of $n!$ are at least $n/2$, while every factor is at most n. Hence

$$\lfloor n/2 \rfloor \lg(n/2) \le \lg(n!) \le n \lg n.$$

These give $\lg(n!) = \Theta(n \lg n)$ for sufficiently large n; finitely many smaller sizes are covered separately.

Thus **every correct comparison sort has a worst-case input requiring $\Omega(n \lg n)$ comparisons**. It need not spend that much on every input: insertion sort uses $n - 1$ key comparisons on sorted data.

Allowing duplicates still includes the hard distinct-key subset. Counting and radix sorts inspect key representations and leave this model, introducing range and bit-length costs instead.

The deepest leaf alone does not bound the average

Let the $N = n!$ relative orders be uniform, and select one representative leaf for each, at depths $d_1, \ldots, d_N$. Their root paths are prefix-free. Associate a depth-d_i leaf with its binary interval of length 2^{-d_i} in $[0, 1)$. The intervals are disjoint, so

$$\sum_{i=1}^{N} 2^{-d_i} \leq 1.$$

The geometric mean is at most the arithmetic mean:

$$2^{-\frac{1}{N}\sum_i d_i} \leq \frac{1}{N} \sum_i 2^{-d_i} \leq \frac{1}{N}.$$

Taking logarithms yields $\frac{1}{N}\sum_i d_i \geq \lg N = \Omega(n \lg n)$. This bounds average depth under uniform inputs; the deepest path alone would not suffice.

For a randomized sort correct for every random choice r, this bound holds for each fixed r. Averaging over r gives $\max_A E_r[C(A, r)] \geq \lg(n!)$, since a maximum is at least the uniform-input average.

Three keys: distinguish the two averaging domains

Example: choose fairly between the earlier three-key tree L and its identity-renamed version R (input order a, c, b).

Identity order	Tree L	Tree R	Fair-choice mean
a, b, c	2	3	$5/2$
a, c, b	3	2	$5/2$
b, a, c	2	3	$5/2$
b, c, a	3	3	3
c, a, b	3	2	$5/2$
c, b, a	3	3	3

Uniform-input column means equal $16/6 = 8/3 \geq \lg 6$; the maximum fixed-input row mean is three. The averaging domains differ.

A heap has both a shape rule and a key-order rule

A binary heap is a **complete binary tree**: all levels before the last are full, and the last fills from left to right. In a 1-based array,

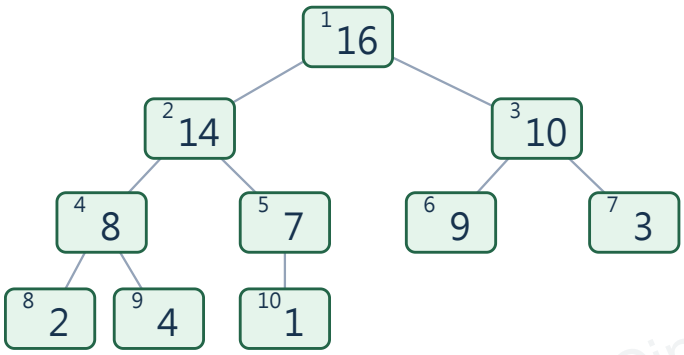
$$parent(i) = \lfloor i/2 \rfloor, \quad left(i) = 2i, \quad right(i) = 2i + 1.$$

Only positions $1 \le i \le s$ belong to the current heap, where s is its heap-size. Array capacity, array length, and s are separate quantities.

A max-heap requires $A[parent(i)] \ge A[i]$ for every nonroot position i. Here $A[i]$ abbreviates its key; swaps move entire records. Following ancestors proves the root is a maximum. Levels and different subtrees need not be sorted.

Height counts **edges** on a longest downward path: a singleton has height 0, and a nonempty n-node heap has height $\lfloor \lg n \rfloor$. Counting levels instead gives $\lfloor \lg n \rfloor + 1 = \lceil \lg(n+1) \rceil$. Handle an empty heap separately, without evaluating $\lg 0$.

One heap, two representations



Node: small index, large key

Fixed indices; moving keys

1	2	3	4	5
16	14	10	8	7
6	7	8	9	10
9	3	2	4	1

Every edge: parent key ≥ child key
One level need not be sorted

Node labels are array positions. Swaps move keys between positions; a parent index stays fixed.

State the precondition for sifting down

Precondition: $1 \leq i \leq s$, both child subtrees are already max-heaps, and only edges from i to its children may violate order. Missing children are not read; `and` uses short-circuit evaluation.

```
MAX-HEAPIFY(A,i,s):
    while true:
        largest = i
        if 2*i <= s and A[2*i] > A[largest]:
            largest = 2*i
        if 2*i+1 <= s and A[2*i+1] > A[largest]:
            largest = 2*i+1
        if largest == i: return
        swap(A[i],A[largest])
        i = largest
```

Each iteration finds a maximum among the current node and its existing children before deciding whether to swap. Comparing the two children alone cannot determine whether their parent needs repair.

Why sifting down is correct and terminates

If the current node is a maximum, its child edges satisfy heap order. The two subtrees already satisfy it, so the entire subtree is repaired.

Otherwise promote the larger child root. It dominates its descendants, the other child root, and the old parent key. Both new edges from the parent are valid. Only the old parent's new position may violate order; the other subtree is unchanged.

Induction on remaining height proves correctness. Each swap changes i to $2i$ or $2i + 1$, moving down one level. At height h, there are at most h descents, giving $O(h + 1)$ time and $O(1)$ auxiliary space for this iterative version.

For $A = [16, 4, 10, 8, 7, 9, 3]$, $i = 2$, $s = 7$, choose child key 8 and swap positions $2, 4$, producing $[16, 8, 10, 4, 7, 9, 3]$. Child indices $8, 9$ of position 4 are out of range, so stop. Key 3 at index 7 belongs to the other subtree and is not compared in this descent.

Build bottom-up so the child subtrees are ready

```
BUILD-MAX-HEAP(A):
    s = length(A)
    for i = floor(s/2) downto 1:
        MAX-HEAPIFY(A,i,s)
```

Invariant: at each iteration's start, every node whose index exceeds i roots a valid max-heap.

Initially $i = \lfloor n/2 \rfloor$, and all larger indices are leaves. Child indices exceed i, so MAX-HEAPIFY's precondition holds. It repairs i while preserving all descendant heaps. Decrementing i restores the invariant for the next iteration.

At $i = 0$, root 1 is valid and the entire heap is complete. For $n = 0, 1$, the loop is skipped and the empty or singleton heap is already valid.

Index $\lfloor n/2 \rfloor$ is the last possible internal node, **not the first leaf**. For $n = 10$, position 5 still has left child 10 and must be processed.

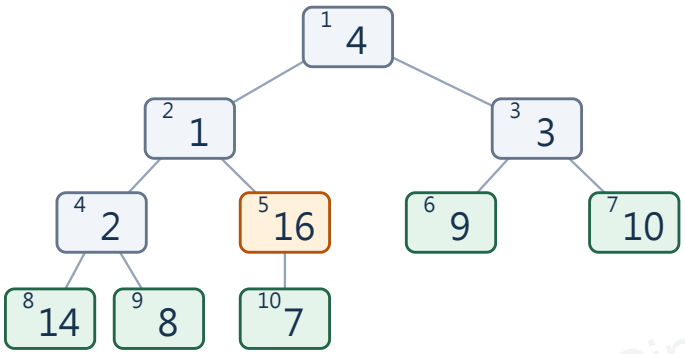
Build backward from position five: complete states

Start with $A = [4, 1, 3, 2, 16, 9, 10, 14, 8, 7]$, active size $s = 10$. All indices are **one-based**. Each root is processed only after both child subtrees have been repaired.

Root i	Child choice and exchanged indices	Repaired $A[1..10]$
5	Only child key 7; $16 \geq 7$, no swap	$[4, 1, 3, 2, 16, 9, 10, 14, 8, 7]$
4	Choose 14 over 8; swap $4 \leftrightarrow 8$	$[4, 1, 3, 14, 16, 9, 10, 2, 8, 7]$
3	Choose 10 over 9; swap $3 \leftrightarrow 7$	$[4, 1, 10, 14, 16, 9, 3, 2, 8, 7]$
2	Choose 16, then 7; swap $2 \leftrightarrow 5$, $5 \leftrightarrow 10$	$[4, 16, 10, 14, 7, 9, 3, 2, 8, 1]$
1	Choose $16, 14, 8$; swap $1 \leftrightarrow 2$, $2 \leftrightarrow 4$, $4 \leftrightarrow 9$	$[16, 14, 10, 8, 7, 9, 3, 2, 4, 1]$

The following tree-and-array figures use these same records. Orange marks the root being processed; green marks certified heap roots. Position relationships stay fixed while swaps move keys and their associated records.

Position five has only a left child



Node: small index, large key

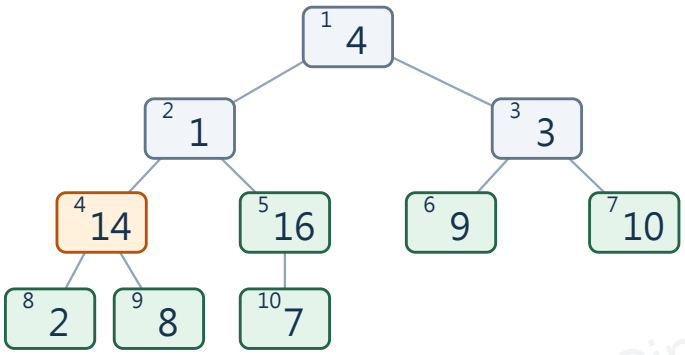
Fixed indices; moving keys

1	2	3	4	5
4	1	3	2	16
6	7	8	9	10
9	10	14	8	7

Orange: root just repaired
Green: legal; gray: pending

Key 16 at position 5 already dominates key 7 at left-child position 10, so no swap occurs. Right-child index $11 > s$ must not be read. Positions $6..10$ are leaves; the repaired subtree at five extends the bottom-up invariant.

Position four promotes the larger child, fourteen



Node: small index, large key

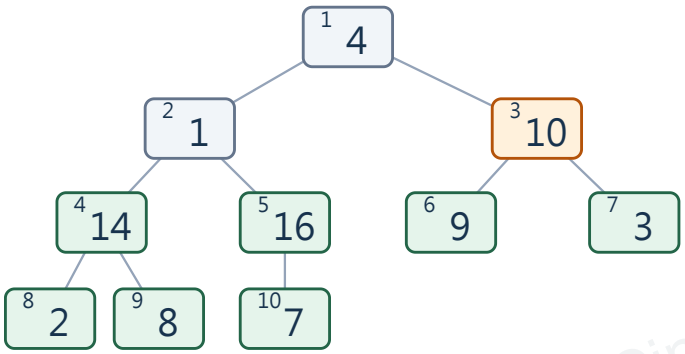
Fixed indices; moving keys

1	2	3	4	5
4	1	3	14	16
6	7	8	9	10
9	10	2	8	7

Orange: root just repaired
Green: legal; gray: pending

Position four contains key two; children at eight and nine contain fourteen and eight. Choose fourteen, compare it with two, then swap positions four and eight. The old key reaches leaf eight and stops; key eight at position nine is unchanged.

Position three chooses ten over nine



Node: small index, large key

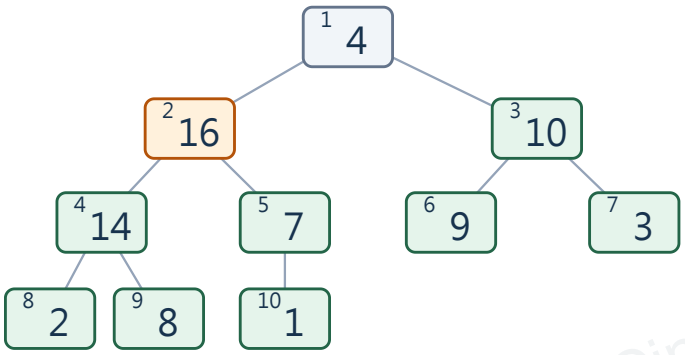
Fixed indices; moving keys

1	2	3	4	5
4	1	10	14	16
6	7	8	9	10
9	3	2	8	7

Orange: root just repaired
Green: legal; gray: pending

Position three has key three; children six and seven contain nine and ten. Choose ten and swap positions three and seven. The old key stops at leaf seven; key nine at six is unchanged. Previously valid child heaps now combine into a valid heap rooted at three.

Position two sends the old key through five and ten



Node: small index, large key

Fixed indices; moving keys

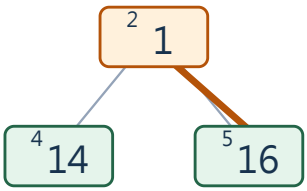
1	2	3	4	5
4	16	10	14	7
6	7	8	9	10
9	3	2	8	1

Orange: root just repaired
Green: legal; gray: pending

Position two has child keys fourteen and sixteen; swap with sixteen first. The old key one reaches position five, remains below its sole child seven, and swaps again with position ten. The path is $2 \to 5 \to 10$; key fourteen at position four does not move.

Follow the same old key through local child choices

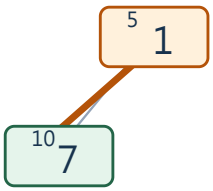
Comparison 1



16 > 1 → swap(2,5)

A[2]=16; A[5]=1

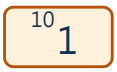
Comparison 2



7 > 1 → swap(5,10)

A[5]=7; A[10]=1

Leaf: stop



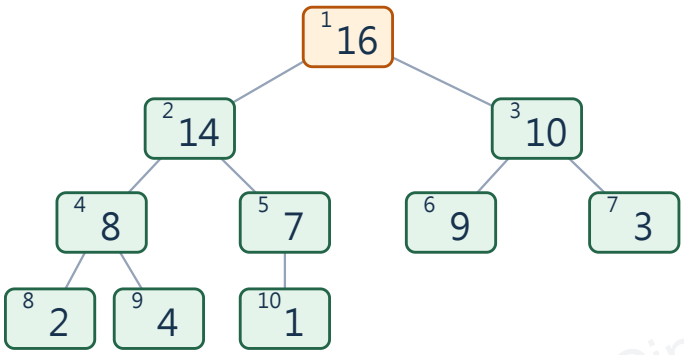
20, 21 > heap-size 10

No child access

Old key 1 ends at index 10

Orange follows the old key one; the heavy edge identifies the chosen larger child. Index labels and key values are separate. First choose child five over four; at position five, only child ten exists. Unshown subtrees remain valid and need no rebuilding.

Position one sends the old root through two, four, and nine



Node: small index, large key

Fixed indices; moving keys

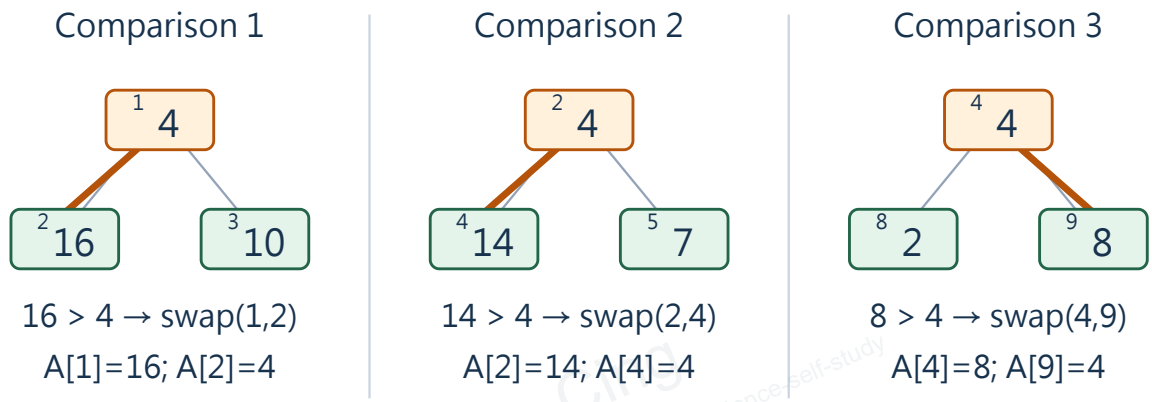
1	2	3	4	5
16	14	10	8	7
6	7	8	9	10
9	3	2	4	1

Orange: root just repaired

Green: legal; gray: pending

The old root key four swaps with sixteen, fourteen, then eight, stopping at leaf nine. Final parent-child checks are $16 \geq 14, 10$, $14 \geq 8, 7$, $10 \geq 9, 3$, $8 \geq 2, 4$, and $7 \geq 1$. They cover all nine tree edges, certifying the heap while preserving all ten records.

Three downward steps: distinguish positions from keys



Each panel shows local state before a swap and the resulting keys below it. Positions follow $1 \to 2 \to 4 \to 9$, while the same old key remains four; chosen child keys are sixteen, fourteen, and eight. At position nine, child indices eighteen and nineteen exceed $s = 10$, so the repair stops.

The complete array after every exchange

Continue after repairing position three. All array indices stay fixed at $1..10$:

Root being repaired	Exchanged indices	Resulting $A[1..10]$
2	$2 \leftrightarrow 5$	$[4, 16, 10, 14, 1, 9, 3, 2, 8, 7]$
2	$5 \leftrightarrow 10$	$[4, 16, 10, 14, 7, 9, 3, 2, 8, 1]$
1	$1 \leftrightarrow 2$	$[16, 4, 10, 14, 7, 9, 3, 2, 8, 1]$
1	$2 \leftrightarrow 4$	$[16, 14, 10, 4, 7, 9, 3, 2, 8, 1]$
1	$4 \leftrightarrow 9$	$[16, 14, 10, 8, 7, 9, 3, 2, 4, 1]$

At each step, choose the largest valid child before comparing it with the parent. One swap changes only two cells; eight other cells and the record multiset stay unchanged. Keep intermediate states to reconstruct a downward path: the final array alone does not record its history.

Linear-time building by summing node heights

For $n \geq 1$, position i has height at least h exactly when its leftmost depth-h descendant exists: $i2^h \leq n$. Hence **exactly** $\lfloor n/2^h \rfloor$ nodes have height at least h.

Write each height as a sum of unit layers and exchange the finite sums:

$$\sum_{i=1}^{n} height(i) = \sum_{h=1}^{\lfloor \lg n \rfloor} \#\{i : height(i) \geq h\} = \sum_{h=1}^{\lfloor \lg n \rfloor} \left\lfloor \frac{n}{2^h} \right\rfloor < n.$$

One repair costs $O(height(i) + 1)$ and there are $\lfloor n/2 \rfloor$ calls, giving $O(n)$ total time. For $n \geq 2$, the loop itself performs at least $\lfloor n/2 \rfloor$ constant-work iterations, supplying $\Omega(n)$ and therefore $\Theta(n)$.

For $n = 15$, the height sum is $7 + 3 + 1 = 11$. Most nodes are not as high as the root. For an empty heap, the height sum is zero and no repair calls occur; return immediately without evaluating $\lg 0$ or asserting $0 < 0$. Building instead by repeated insertion can require $\Theta(n \lg n)$ because new keys may bubble to the root; that is a different construction algorithm.

Heapsort shrinks the active prefix

```
HEAPSORT(A):
    BUILD-MAX-HEAP(A)
    for end = length(A) downto 2:
        swap(A[1],A[end])
        MAX-HEAPIFY(A,1,end-1)
```

At each iteration's start, $A[1..end]$ is a max-heap. Suffix $A[end + 1..n]$ is sorted increasingly and every suffix key is at least every heap key.

The swap moves the active maximum to $A[end]$, extending the sorted suffix. With heap-size $end - 1$, the old root's two subtrees remain valid, so only the new root needs repair. Swaps preserve the record multiset. At termination, the remaining singleton prefix completes the sorted array.

Building costs $O(n)$, followed by at most $n - 1$ repairs of $O(\lg(n + 1))$ time each. Total time is $O(n \lg(n + 1))$, with worst case $\Theta(n \lg n)$ from the comparison-sort lower bound. This implementation stops on equal keys and can take only $\Theta(n)$ on all-equal input; a worst-case bound need not describe every input.

Iterative repair uses $O(1)$ auxiliary space. Recursive repair requires additional stack space.

Ten-record heapsort: nine maximum extractions

Start from $[16, 14, 10, 8, 7, 9, 3, 2, 4, 1]$. Each round first exchanges position one with the old last position $s + 1$, moving the maximum to the sorted suffix, then repairs the prefix of new size s. Paths below record only repair swaps, excluding the root-to-end exchange.

New size s	Repair path	Repaired heap prefix	Sorted suffix
9	$1 \to 2 \to 4 \to 9$	$[14, 8, 10, 4, 7, 9, 3, 2, 1]$	$[16]$
8	$1 \to 3 \to 6$	$[10, 8, 9, 4, 7, 1, 3, 2]$	$[14, 16]$
7	$1 \to 3 \to 7$	$[9, 8, 3, 4, 7, 1, 2]$	$[10, 14, 16]$
6	$1 \to 2 \to 5$	$[8, 7, 3, 4, 2, 1]$	$[9, 10, 14, 16]$

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Finish the remaining five extractions

New size s	Repair path	Repaired heap prefix	Sorted suffix
5	$1 \to 2 \to 4$	$[7, 4, 3, 1, 2]$	$[8, 9, 10, 14, 16]$
4	$1 \to 2$	$[4, 2, 3, 1]$	$[7, 8, 9, 10, 14, 16]$
3	$1 \to 3$	$[3, 2, 1]$	$[4, 7, 8, 9, 10, 14, 16]$
2	$1 \to 2$	$[2, 1]$	$[3, 4, 7, 8, 9, 10, 14, 16]$
1	—	$[1]$	$[2, 3, 4, 7, 8, 9, 10, 14, 16]$

In round two, the old final key one moves to the root and descends through $1 \to 3 \to 6$, swapping with ten and then nine. Every suffix is ascending and dominates every active heap key. The final prefix and suffix together form the complete ascending sequence.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Priority queues: maximum, extraction, increase, and insertion

$Q.A[1..Q.s]$ is a max-heap with writable capacity `Q.capacity`. The fixed-capacity code checks empty/full conditions first. Swaps move whole records; keys determine priority.

```
MAXIMUM(Q):
    if Q.s == 0: return EMPTY
    return Q.A[1]

EXTRACT-MAX(Q):
    if Q.s == 0: return EMPTY
    result = Q.A[1]; Q.A[1] = Q.A[Q.s]; Q.s -= 1
    if Q.s > 0: MAX-HEAPIFY(Q.A,1,Q.s)
    return result
```

`MAXIMUM` costs $O(1)$. `EXTRACT-MAX` saves the maximum record, replaces the root with the last record, decreases active size, and repairs downward in worst-case $O(\lg(s+1))$. Extracting a singleton leaves size zero and does not inspect a new root.

Increase a key: repair only toward ancestors

```
SIFT-UP(Q,i):
    while i > 1 and key(Q.A[floor(i/2)]) < key(Q.A[i]):
        swap(Q.A[i],Q.A[floor(i/2)])
        i = floor(i/2)

INCREASE-KEY(Q,i,k):
    require 1 <= i <= Q.s and k >= key(Q.A[i])
    set-key(Q.A[i],k)
    SIFT-UP(Q,i)
```

An increase can violate only ancestor edges: the new key still dominates its old children. Each exchange repairs the current parent edge and moves the candidate upward. Stop at the root or a dominating parent, in worst-case $O(\lg(s+1))$. Evaluate `and` left to right with short-circuiting, checking $i > 1$ first.

Insert directly without a sentinel value

```
INSERT(Q,record):
    if Q.s == Q.capacity: return FULL
    Q.s += 1
    Q.A[Q.s] = record
    SIFT-UP(Q,Q.s)
```

Write the new record directly into the new final position and sift upward; no negative-infinity sentinel is needed. Nonfull fixed-capacity insertion costs worst-case $O(\lg(s+1))$. Dynamic-array reallocation can move $\Theta(s)$ records in one operation and needs a separate amortized analysis.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Trace the increase from position eight

Increase-key follows parent indices 8 → 4 → 2

	1	2	3	4	5	6	7	8
Original heap	20	15	18	7	12	9	17	2
Write 16 at 8	20	15	18	7	12	9	17	**16**
Swap 8 ↔ 4	20	15	18	**16**	12	9	17	7
Swap 4 ↔ 2	20	**16**	18	15	12	9	17	7

Compare 16 with parents 7, 15, then 20; stop because 20 ≥ 16.

After writing sixteen at eight, compare parent keys seven, fifteen, and twenty. Exchanges $8 \leftrightarrow 4$ and $4 \leftrightarrow 2$ suffice. Indices identify positions; record identities move during swaps.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Decrease, indexed deletion, and equal priorities

Decreasing a max-heap key can violate only child edges, so repair downward. For deletion at position i, replace it with the last record and decrease size; deleting the old last position simply decreases size.

Let replacement key be z. If $i > 1$ and $z > A[parent(i)]$, the old parent dominated the deleted key, which dominated its children; consequently z dominates those children too and needs only upward repair. Otherwise the parent edge is valid and only downward repair is needed. Each operation costs worst-case $O(\lg(s+1))$ given a position; searching for an arbitrary key can still cost $\Theta(s)$.

A handle identifies a record. Every swap must update the handle-to-index map, and deletion removes the deleted handle. An array index does not permanently identify a record.

Sequence numbers have different roles in sorting and queues

Heapsort is unstable: $[2a, 2b, 1c]$ can become $[1c, 2b, 2a]$. Stable ascending sorting can compare $(key, original_index)$, placing later equal records into the suffix first. FIFO ties in a max-priority queue instead compare $(priority, -arrival_index)$. The sequence-number direction differs, and both choices change the comparison and storage specification.

Let a, b arrive in that order, with sequence numbers $1, 2$ and equal keys 5:

Purpose	Key for a	Key for b	First maximum extracted	Required order
Stable ascending sort	$(5, 1)$	$(5, 2)$	b, placed in the suffix	Output a, b
FIFO priority ties	$(5, -1)$	$(5, -2)$	a, served immediately	Service a, b

Ascending sorting places extracted maxima from right to left; a queue serves each extracted maximum immediately. Determine how extraction order maps to the desired order before choosing the sequence-number sign.

Hiring: distinguish interview and replacement costs

There are n candidates with distinct, comparable ranks; larger is better. After each interview, replace the incumbent exactly when the new candidate is better. Dummy candidate `dummy` ranks below everyone.

```
HIRE(A):
    best = dummy; hires = 0
    for i = 1 to n:
        interview A[i]
        if rank(A[i]) > rank(best):
            best = A[i]
            hire best; hires += 1
    return (best, hires)
```

After iteration i, `best` is the best of the first i candidates: initially none have arrived; a better arrival replaces the incumbent, otherwise the current best remains. Thus the final result is the overall best.

One procedure, three different cost questions

With X hires, interview cost c_I and hiring cost c_H, total cost is $nc_I + Xc_H$. Empty input returns `dummy` with $X = 0$.

For $n \geq 1$, increasing ranks give $X = n$ and reverse order gives $X = 1$. Now distinguish three questions:

Question	Random source or quantifier	Quantity
Worst case	Maximize over legal inputs	$\max_A X(A) = n$
Average case	Specify an input distribution, e.g. uniform over $n!$ orders	$\mathbb{E}_A[X(A)]$
Randomized procedure	Fix any input and uniformly shuffle it internally	$\mathbb{E}_R[X(A, R)]$

A uniform shuffle makes every fixed distinct-rank input a uniform permutation; the processing order must be controllable. Expected bounds permit bad runs: increasing order still has probability $1/n!$. Fewer hires reduce hiring cost, not the number of interviews.

Indicators turn a count into event probabilities

Fix a finite sample space Ω with nonnegative outcome probabilities summing to 1. A random variable has expectation $\mathbb{E}[X] = \sum_{\omega \in \Omega} \Pr(\omega) X(\omega)$.

The indicator I_E is 1 when event E occurs and 0 otherwise, so

$$\mathbb{E}[I_E] = 1 \Pr(E) + 0 \Pr(E^c) = \Pr(E).$$

If a target count equals $X = \sum_{i=1}^{m} X_i$, exchanging finite sums gives

$$\mathbb{E}[X] = \sum_{\omega} \Pr(\omega) \sum_{i} X_i(\omega) = \sum_{i} \sum_{\omega} \Pr(\omega) X_i(\omega) = \sum_{i} \mathbb{E}[X_i].$$

This is **linearity of expectation**; the X_i need not be independent. Independence may instead be needed to calculate a probability or factor an intersection probability.

For example, let $X_1 = X_2$ both indicate heads on the same fair toss. Then $\mathbb{E}[X_1 + X_2] = 1$, but $\Pr(X_1 = X_2 = 1) = 1/2$, not $1/4$.

Three tools: union bound, Markov bound, and tail sums

First identify whether the target is an event probability, an expected count, or the expected maximum. Indicators yield three basic tools.

Since $I_{\cup_i E_i} \leq \sum_i I_{E_i}$, taking expectations gives

$$\Pr\left(\bigcup_i E_i\right) \leq \sum_i \Pr(E_i).$$

For $X \geq 0$ and $t > 0$, inequality $X \geq t I_{\{X \geq t\}}$ gives Markov's bound

$$\Pr(X \geq t) \leq \frac{\mathbb{E}[X]}{t}.$$

Tail sums: count the levels reached by each outcome

For nonnegative integer L, every outcome satisfies $L = \sum_{j \geq 1} I_{\{L \geq j\}}$, hence

$$\mathbb{E}[L] = \sum_{j \geq 1} \Pr(L \geq j).$$

Finite bounds allow finite-sum exchange. Infinite tails follow by monotone limits of nonnegative partial sums, allowing both sides to be infinite. None of these three tools requires independence.

Concrete probability products may still require independent trials. Do not drop those model assumptions merely because the general tool needs no independence.

Example: $L = 0, 1, 3$ with probabilities $1/2, 1/4, 1/4$. Direct expectation gives $0/2 + 1/4 + 3/4 = 1$. The tails are $\Pr(L \geq 1) = 1/2$, $\Pr(L \geq 2) = 1/4$, $\Pr(L \geq 3) = 1/4$; their sum is also 1.

Hiring count: the probability of a record at each position

Assume uniformity over all $n!$ distinct-rank permutations, and let X_i indicate hiring at position i. Condition on the set of the first i candidates: all $i!$ relative orders remain equally likely, so their best candidate is last with probability $1/i$.

Hiring at i is exactly the event that this candidate is best among the first i. Thus

$$\mathbb{E}[X] = \sum_{i=1}^{n} \Pr(X_i = 1) = \sum_{i=1}^{n} \frac{1}{i} = H_n.$$

Since $1/x$ decreases, comparing rectangles with integrals gives

$$\int_1^{n+1} \frac{dx}{x} \leq H_n \leq 1 + \int_1^n \frac{dx}{x}, \qquad n \geq 1.$$

Consequently $\mathbb{E}[X] = \Theta(\ln n)$, but total expected cost remains $nc_I + c_H H_n$: interview cost does not become logarithmic.

A six-candidate run and its expected-cost comparison

Position i	Rank	Best after interview	Hire?	Total hires
1	2	2	Yes	1
2	5	5	Yes	2
3	1	5	No	2
4	4	5	No	2
5	6	6	Yes	3
6	3	6	No	3

Input $[2, 5, 1, 4, 6, 3]$ sets records at positions $1, 2, 5$, making three hires; the uniform mean at this size is $H_6 = 49/20 = 2.45$. Exactly one hire has probability $1/n$; hiring everyone requires the unique increasing permutation, with probability $1/n!$. For empty input define $H_0 = 0$ separately.

The same method counts inversions and fixed points

Uniformly permute n distinct keys. For positions $i < j$, let I_{ij} indicate $A[i] > A[j]$. Exchanging the two entries bijectively pairs inversion and noninversion outcomes, so $\Pr(I_{ij} = 1) = 1/2$.

$$V = \sum_{i<j} I_{ij}, \qquad \mathbb{E}[V] = \binom{n}{2}\frac{1}{2} = \frac{n(n-1)}{4}.$$

Inversion events sharing positions can be dependent; linearity still applies. When $n = 2$, the count is either 0 or 1, although its expectation is $1/2$.

Now uniformly return $n \geq 1$ hats one-to-one to their original owners. Each person receives their own hat with probability $1/n$. The fixed-point count satisfies

$$F = \sum_{i=1}^{n} I_i, \qquad \mathbb{E}[F] = n\frac{1}{n} = 1.$$

For $n = 2$, both hats are correct or both wrong, each with probability $1/2$. Mean one does not imply exactly one correct hat in every outcome. For $n = 0$, directly take $F = 0$.

Uniform integers by rejecting out-of-range bit patterns

The contract of `RANDOM(a,b)` is an inclusive integer with probability $1/(b-a+1)$ per value, conditionally uniform given prior history. Assume integer $a \le b$ and access to independent fair bits.

Set $m = b - a + 1$. If $m = 1$, return a. Otherwise let $k = \lceil \lg m \rceil$, repeatedly read k bits as $z \in \{0, \ldots, 2^k - 1\}$, and return $a + z$ when $z < m$. Reject other patterns and redraw the whole block.

Acceptance probability per round is $p = m/2^k > 1/2$. For any legal value v, its eventual return probability is

$$\sum_{t=0}^{\infty} (1-p)^t \frac{1}{2^k} = \frac{1}{p2^k} = \frac{1}{m}.$$

Expected rounds are $1/p < 2$, giving $O(\lg m)$ expected bit cost. Termination is almost sure; there is no fixed worst-case retry limit.

Taking a remainder directly can bias the result: eight three-bit patterns reduced modulo 3 give preimage counts $3, 3, 2$ for remainders $0, 1, 2$. The pseudorandom generator and bounded-integer sampling procedure are distinct parts of an implementation.

In-place shuffling fixes one more position per round

Select uniformly from the unfixed suffix and swap the selected record into position i. Distinguish record identities even when keys coincide.

```
SHUFFLE(A):
    for i = 1 to n:
        j = RANDOM(i,n)
        swap(A[i],A[j])
```

After round i, later swaps never touch $A[1..i]$. Empty input has one empty output; a singleton swaps only with itself. Under unit-cost integer sampling, time is $\Theta(n)$ and extra space is $O(1)$; bit-sampling cost is separate.

Uniform shuffling: extend a prescribed prefix

Claim: any prescribed sequence of $i-1$ distinct identities occupies $A[1..i-1]$ with probability $(n-i+1)!/n!$. For $i=1$, the empty prefix has probability one.

Conditioned on that prefix, the next desired identity occupies one of the $n-i+1$ remaining positions. Conditional uniform sampling selects it with probability $1/(n-i+1)$, so

$$\frac{(n-i+1)!}{n!}\frac{1}{n-i+1}=\frac{(n-i)!}{n!}.$$

Induction gives probability $0!/n!=1/n!$ for every complete permutation. For $n=4$ and target $[C,D,A,B]$:

Fixed length	0	1	2	3	4
Prescribed prefix	empty	C	C,D	C,D,A	C,D,A,B
Probability	1	$1/4$	$1/12$	$1/24$	$1/24$

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Four-record shuffle: track the fixed prefix at every round

Start with $[A,B,C,D]$ and draw $j=3,4,3,4$ in successive rounds. Letters identify records rather than comparison keys.

Round i	Legal indices	Drawn j	Full array after swap	Fixed prefix
1	$1..4$	3	$[C,B,A,D]$	C
2	$2..4$	4	$[C,D,A,B]$	C,D
3	$3..4$	3	$[C,D,A,B]$	C,D,A
4	4	4	$[C,D,A,B]$	C,D,A,B

This path has probability $(1/4)(1/3)(1/2)(1)=1/24$. Every prescribed output uniquely determines a legal path: locate its next identity in the remaining suffix and swap it into position i.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

The fixed prefix never moves again

Start [A,B,C,D]: select only from the unfixed suffix

	Pos 1	Pos 2	Pos 3	Pos 4	Round probability
i=1, j=3	C	B	A	D	1/4
i=2, j=4	C	D	A	B	1/3
i=3, j=3	C	D	A	B	1/2
i=4, j=4	C	D	A	B	1

Green cells stay fixed. This path has probability 1/4·1/3·1/2·1=1/24.

Later rounds touch only unfixed positions, so the prefix remains fixed. A trace checks the procedure; the preceding joint invariant proves general uniformity.

Uniform positions do not guarantee a uniform permutation

A uniform cyclic shift puts each record in every position with probability $1/n$, yet for $n \geq 3$ it produces only n outputs rather than $n!$. Uniform marginals do not imply a uniform joint permutation.

Another incorrect procedure runs $i = 1..n$, drawing j from all of $1..n$ each round before swapping. For $n = 3$, its $3^3 = 27$ random paths are equally likely. Uniformity over six permutations would require $27/6$ paths per output, impossible for an integer path count.

This procedure can move earlier positions again, so the fixed-prefix proof no longer applies. A demonstrated bias is stronger than saying the shuffle looks insufficiently random.

Running only the first m rounds of the correct shuffle, $0 \leq m \leq n$, gives each ordered sample probability $(n - m)!/n!$. Each unordered m-subset has $m!$ orders, hence probability $1/\binom{n}{m}$. This saves later swaps, but constructing an indexable array can still cost $\Theta(n)$.

Random priorities: collision handling is part of uniformity

Give $n \geq 1$ records independent uniform priorities $P_i \in \{1, \ldots, M\}$ for integer $M \geq n$, and sort by priority. Conditioned on all priorities being distinct, exchanging record labels bijectively maps orders while preserving probability. Thus all $n!$ relative orders are equally likely.

Each pair collides with probability $1/M$. The union bound gives

$$\Pr(\exists i < j : P_i = P_j) \leq \frac{\binom{n}{2}}{M}.$$

With $M = n^3$ and $n \geq 2$, this is $(n-1)/(2n^2) < 1/(2n)$. Rare collisions are still possible.

Keeping original order on ties biases the unconditional output. For $n = 2$, the first record precedes the second with probability $\Pr(P_1 \leq P_2) = (M+1)/(2M) > 1/2$.

One repair redraws the entire set after any collision and accepts only distinct priorities. For this M, acceptance probability has a positive constant lower bound, so expected rounds are constant. Alternatively, uniformly shuffle each tie group independently. Comparison-based priority sorting takes $\Theta(n \lg n)$ time and $O(n)$ priority storage; redraws must also detect collisions, which is not free.

After randomization, specify correctness and what is controllable

RANDOMIZED-HIRE uniformly shuffles, then runs HIRE. Every fixed distinct-rank input becomes a uniform permutation, giving H_n expected hires and expected cost $nc_I + c_H H_n$. Another $\Theta(n)$ shuffle operations must be accounted for separately from monetary hiring costs.

If candidates arrive externally and cannot be reordered, the algorithm lacks this control. The H_n analysis still requires a uniform-arrival assumption; an unavailable shuffle cannot remove it.

For the shuffle and randomized-pivot sort in this chapter, cost is random while every returned result satisfies its specification. These are Las Vegas algorithms. Monte Carlo algorithms allow a small error probability, which must be stated with its input quantifiers. Expected time alone says nothing about correctness probability.

The later online-selection strategy has a different objective: it follows its selection rule even when the chosen candidate is not globally best. Its success probability needs a separate model and proof.

Quicksort partitions, then sorts both sides

Use the 1-based closed interval $A[p..r]$. Partitioning around a pivot establishes order between regions; it does not sort either region internally.

```
QUICKSORT(A,p,r):
    if p >= r: return
    q = PARTITION-LAST(A,p,r)
    QUICKSORT(A,p,q-1)
    QUICKSORT(A,q+1,r)
```

An empty interval ($p > r$) or singleton ($p = r$) returns immediately. Partition returns the pivot position q, and both recursive intervals **exclude** q, so their lengths strictly decrease. Correctness and termination depend on this return contract.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Last-position partition: save the key, move whole records

The last-position partition is given below. Save pivot key x first. Swaps move complete records, keeping every associated field with its key.

```
PARTITION-LAST(A,p,r):
    x = key(A[r]); i = p-1
    for j = p to r-1:
        if key(A[j]) <= x:
            i += 1
            swap(A[i],A[j])
    swap(A[i+1],A[r])
    return i+1
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Loop boundaries: i ends the small region; j is next

The pivot stays at A[r]. Use **1-based indices and the inclusive interval A[p..r]**. The boundary i=p−1 means that the ≤ region is empty.

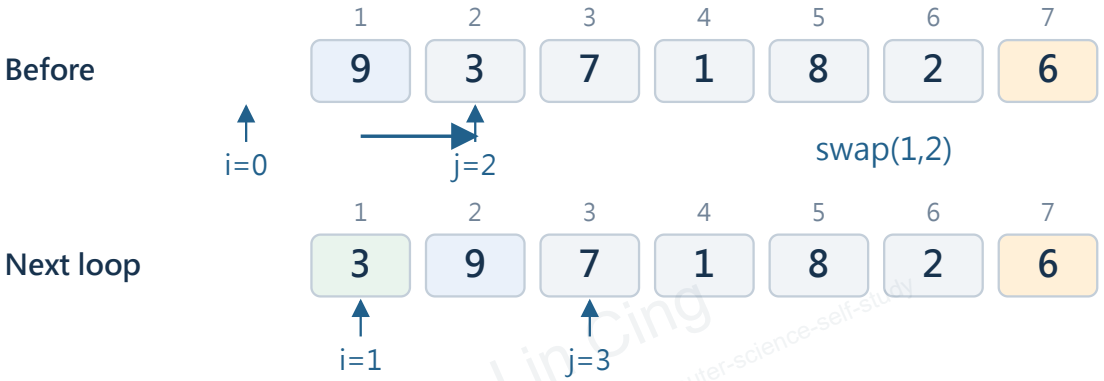
Region	Known relation	Initially i=p−1, j=p
A[p..i]	≤ pivot	Empty
A[i+1..j−1]	> pivot	Empty
A[j..r−1]	Unclassified	All non-pivot keys
A[r]	Pivot	Fixed until the final exchange

Diagram key: green ≤, blue >, gray unknown, orange pivot. Both transition diagrams use [9,3,7,1,8,2,6] with pivot 6.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A small key: exchange, then advance both boundaries

3 ≤ 6: advance i, then exchange A[1] and A[2]



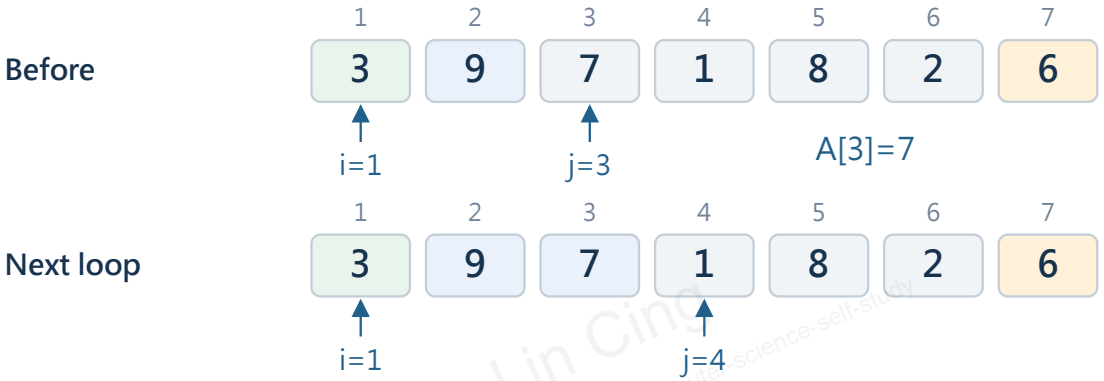
3 joins ≤; displaced 9 stays in >. Pivot 6 remains at index 7.

For A[2]=3, advance i to 1 and exchange A[1],A[2]. The key 3 joins ≤; displaced 9 stays in >. Finally advance j. In general i may equal j, giving a harmless self-exchange.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A large key: leave values fixed and advance j

7 > 6: no exchange; keep i and advance j

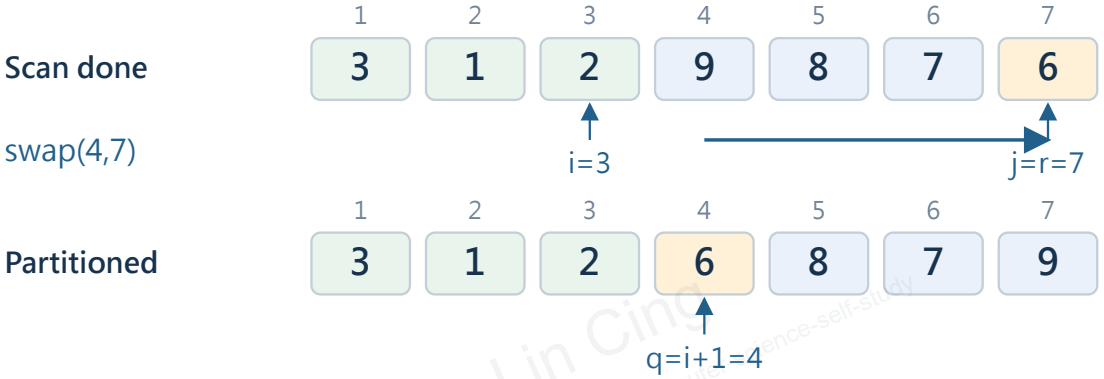


7 moves from unknown to >. Values stay fixed; the boundary moves.

Since A[3]=7>6, leave i and the array unchanged. Advancing j adds 7 to the > region. **The values stay fixed; our classification changes.**

The final exchange establishes the pivot position q

Only after scanning: place the pivot between the two regions



≤6: [3,1,2] | pivot 6 | >6: [8,7,9]; the sides remain unsorted.

The unknown length r−j decreases once per loop, proving termination. At j=r, exchange A[i+1],A[r] and return q=i+1. The left region is ≤pivot and the right region >pivot; neither side is necessarily sorted.

From the partition invariant to sorting correctness

1. **Initialization:** i=p−1,j=p makes both classified regions empty.
2. **Maintenance:** a small key expands $\leq$ after an exchange; a large key expands >. Either branch removes one unknown key and leaves the pivot fixed.
3. **Partition termination:** no unknown keys remain. The final exchange places the pivot at q. Exchanges preserve the input multiset.
4. **Recursive induction:** empty and singleton intervals are sorted. Both children exclude q and are strictly shorter. Sorting them and adjoining the pivot therefore sorts the whole interval.

The ordering relation alone is insufficient: the proof also needs **multiset preservation, strictly smaller recursive intervals**, and sorted children upon return.

Complete partition trace: every test and exchange

Input is $A = [9, 3, 7, 1, 8, 2, 6]$ with $p = 1, r = 7$. Save last key $x = 6$ and initialize $i = 0$. The table gives i after each test.

Scan index j	Test	i	Full array after action
1	$9 > 6$, no swap	0	$[9, 3, 7, 1, 8, 2, 6]$
2	$3 \leq 6$, swap $1, 2$	1	$[3, 9, 7, 1, 8, 2, 6]$
3	$7 > 6$, no swap	1	$[3, 9, 7, 1, 8, 2, 6]$
4	$1 \leq 6$, swap $2, 4$	2	$[3, 1, 7, 9, 8, 2, 6]$
5	$8 > 6$, no swap	2	$[3, 1, 7, 9, 8, 2, 6]$
6	$2 \leq 6$, swap $3, 6$	3	$[3, 1, 2, 9, 8, 7, 6]$
End of scan	Swap $4, 7$	3	$[3, 1, 2, 6, 8, 7, 9]$

Return $q = 4$; both sides remain unsorted. Six key comparisons; $\Theta(r - p + 1)$ time and $O(1)$ extra space.

Worst-case time: a family attaining quadratic work

Sorted distinct input with a fixed final pivot always chooses the maximum. The two child lengths are n−1 and 0.

Current length	Key comparisons	Next nonempty length
n	n−1	n−1
n−1	n−2	n−2
…	…	…
2	1	1

The sum is $n(n-1)/2$, proving a worst-case Ω(n²) lower bound for an entire input family, rather than extrapolating from one measurement.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

The quadratic upper bound must cover every split

Let C(n) be the worst comparison count. Child sizes satisfy a+b=n−1. Induction on smaller sizes gives

$$C(a)+C(b)+(n-1)\le \frac{a(a-1)}{2}+\frac{b(b-1)}{2}+n-1=\frac{n(n-1)}{2}-ab\le \frac{n(n-1)}{2}.$$

With C(0)=C(1)=0, this covers all splits. Equality holds when a=0 or b=0. Each pivot leaves recursion, so there are O(n) calls. Total time is O(n+C); the matching family proves **worst-case Θ(n²)**.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Balanced splits: count the comparison work by level

Take n=15 and always choose the true median. Count only Lomuto key comparisons; singletons do not call partition.

Depth d	Subproblems	Size each	Comparisons at level
0	1	15	14
1	2	7	2×6=12
2	4	3	4×2=8
3	8	1	0

Total: 34 comparisons, versus 105 for endpoint splits on 15 keys. Partition still scans linearly; the difference is how much work survives at each depth.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A fixed split ratio also gives logarithmic depth

If the larger child always has size at most ρ times the parent size, for a fixed ρ<1, the longest branch has O(log n) levels. Intervals at one depth are disjoint, giving at most n scanned keys per level and O(n log n) total work. A fixed 1:9 split suffices.

For perfectly balanced $n = 2^k - 1$, depth d has 2^d subproblems of length $2^{k-d} - 1$. Its comparison work is

$$2^d(2^{k-d} - 2) = n + 1 - 2^{d+1}, \qquad 0 \le d \le k - 2.$$

The first roughly k/2 levels each require Θ(n) comparisons, proving Ω(n log n) for that execution and hence Θ(n log n) overall. This lower bound counts actual recursive work; a comparison-sorting worst-input lower bound does not apply to every individual input.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Duplicates: a random pivot cannot repair all-equal input

For $[5, 5, 5, 5, 5]$, every `<= x` test succeeds, so every partition returns $q = r$. Successive sizes $5, 4, 3, 2$ cost $4 + 3 + 2 + 1 = 10$ comparisons; for n records the total is $n(n-1)/2$.

A randomly chosen pivot is still 5 and cannot change this path. The later expected $\Theta(n \lg n)$ analysis of the two-way version therefore assumes **distinct keys**. The order-statistics section gives three-way partitioning into keys less than, equal to, and greater than the pivot. Only the strict regions recurse. All-equal input then finishes in one $\Theta(n)$ partition.

Stability is a separate guarantee. Input $[2_a, 2_b, 1_c]$ with last pivot 1 becomes $[1_c, 2_b, 2_a]$ after the final swap. The two key-2 records have reversed order. Random pivot selection does not make this swapping implementation stable.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Recurse on the smaller side to bound stack space

The original two-recursion procedure can retain $\Theta(n)$ stack frames. In-place partitioning alone does not give constant-space quicksort. Recurse only on the smaller side and handle the larger side in the current frame's loop.

```
QUICKSORT-SMALL-STACK(A,p,r):
    while p < r:
        q = PARTITION-LAST(A,p,r)
        if q-p < r-q:
            QUICKSORT-SMALL-STACK(A,p,q-1)
            p = q+1
        else:
            QUICKSORT-SMALL-STACK(A,q+1,r)
            r = q-1
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Smaller-side recursion bounds the live call chain

Side lengths satisfy $a + b = n - 1$, so the actual recursive call has size at most $\lfloor (n-1)/2 \rfloor$. Sizes at least halve along the chain of unreturned calls, yielding worst-case $O(\lg(n+1))$ stack space.

If every pivot is largest, the recursive side is empty but the loop still partitions sizes $n, n-1, \ldots, 2$. Stack space is then constant while time remains $\Theta(n^2)$. Analyze these two resources separately.

Median pivots: a chain of live calls

Nested frames

15 → 7 → 3 → 1: four simultaneous frames

Largest pivots: successive loop iterations

One working frame

Each empty recursive call returns immediately; the larger side stays in this frame.

Randomization bounds expectation for each fixed input

At each partition, draw integer k uniformly from closed interval $p..r$, swap $A[k]$ with $A[r]$, then use the same last-position partition. Future choices must remain uniform conditional on their current states.

For distinct keys, every local rank has pivot probability $1/n$, whatever the current arrangement. Expectation is over **internal algorithm choices**; no uniform-input assumption is required.

Every fixed input can still encounter a path that repeatedly chooses an endpoint, costing $\Theta(n^2)$. An expectation is neither a per-execution upper bound nor an unproved tail-probability bound.

Separate mathematical uniform sampling from its implementation. A fixed seed reproduces a pseudorandom sequence; merely calling random does not prove uniformity. Bounded-integer sampling can use the rejection procedure developed earlier in this chapter.

A pair compares exactly when an interval endpoint is first

Name distinct keys in increasing order $z_1 < \cdots < z_n$. Let X_{ij} indicate whether z_i, z_j are directly compared. They compare at most once: one must be the pivot, which then leaves recursion.

Consider $Z_{ij} = \{z_i, \ldots, z_j\}$. An outside pivot keeps this entire interval on one side. If the first pivot inside it is endpoint z_i or z_j, the pair compares immediately. If it is an interior key, the endpoints enter different recursive regions and never meet again.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Outside pivots preserve the entire rank interval

Sorted ranks for analysis: $Z_{2,5}$ = {2,3,4,5}; not an array snapshot

Pivot 7 first: ranks 2..5 remain together in its left child; $X_{2,5}$ is undecided.

First within interval: endpoint 2 (or 5)

This partition compares 2 and 5 → $X_{2,5}$=1

First within interval: interior 3 (or 4)

2 left, 5 right → never compare, $X_{2,5}$=0

Four interval keys are equally likely to be first; two are endpoints. P($X_{2,5}$=1)=2/4=1/2.

Thus $X_{ij} = 1$ exactly when the first pivot selected from Z_{ij} is one of its endpoints. It need not be the first pivot of the entire execution.

From four first choices to the general comparison probability

For z_2, z_5, inspect only the first pivot from $Z_{2,5} = \{z_2, z_3, z_4, z_5\}$.

First pivot inside the interval	What happens to z_2, z_5	$X_{2,5}$
z_2	Compare immediately	1
z_3	Separate forever	0
z_4	Separate forever	0
z_5	Compare immediately	1

Each key has probability $1/4$ of being first inside the interval, so the comparison probability is $1/2$. Generally, for interval length $m = j - i + 1$,

$$\Pr(X_{ij} = 1) = \frac{2}{m} = \frac{2}{j - i + 1}.$$

Why the first pivot inside an interval is uniform

Prove uniformity rather than assume it. Fix an interval Z of m keys and a designated $z \in Z$. Induct on the size N of the current subproblem containing Z. If $N = m$, the first pivot is uniform, giving probability $1/m$. For $N > m$, choosing z immediately has probability $1/N$; choosing another key of Z fails.

An outside pivot leaves all of Z in a strictly smaller subproblem. Induction gives designated-first probability $1/m$ there. Summing over the $N - m$ outside keys yields

$$\frac{1}{N} + \frac{N - m}{N} \frac{1}{m} = \frac{1}{m}.$$

The first pivot inside the interval is therefore uniform; the denominator is m, not original input size n. Other recursive branches may execute in between but do not move or compare these keys.

Expected comparisons: sum over rank-interval lengths

Total key comparisons are $C = \sum_{1 \le i < j \le n} X_{ij}$. Linearity of expectation requires no independence. There are $n + 1 - k$ rank intervals of length $k = j - i + 1$, so

$$\begin{aligned}\mathbb{E}[C] &= \sum_{i<j} \frac{2}{j-i+1} = 2\sum_{k=2}^{n} \frac{n+1-k}{k} \\ &= 2(n+1)\sum_{k=2}^{n} \frac{1}{k} - 2(n-1) \\ &= 2(n+1)H_n - 4n, \qquad H_n = \sum_{k=1}^{n} \frac{1}{k}.\end{aligned}$$

Bounds $\ln(n+1) \le H_n \le 1 + \ln n$ give expected $\Theta(n \lg n)$ comparisons. Scanning work is proportional to these comparisons, with $O(n)$ additional call overhead, so expected time is also $\Theta(n \lg n)$.

The exact formula assumes distinct keys, conditionally uniform pivots, and one <= test per nonpivot key. It excludes index tests, swaps, and random-number generation. Comparison events are dependent; independent-variable variance or tail formulas cannot be imported without further proof.

A first-position pivot has its own invariant

This single-scan version keeps pivot $x = A[p]$ at the first position until cleanup. The interval is still the closed interval $A[p..r]$.

```
PARTITION-FIRST(A,p,r):
    x = A[p]; i = p
    for j = p+1 to r:
        if key(A[j]) <= key(x):
            i = i+1
            swap(A[i],A[j])
    swap(A[p],A[i])
    return i
```

Before each test, $A[p] = x$, $A[p+1..i] \le x$, $A[i+1..j-1] > x$, and $A[j..r]$ is unknown. Small keys expand the left region; large keys only advance j. The unknown length decreases each round. Cleanup places the pivot at i; exclude i from both recursive sorts.

Check both partitions using the same pivot key

Use first-position input $[6, 10, 13, 5, 8, 3, 2, 11]$, pivot $x = 6$, and initial $i = 1$.

j	Test and action	Array after action
2	$10 > 6$, $i = 1$	$[6, 10, 13, 5, 8, 3, 2, 11]$
3	$13 > 6$, $i = 1$	Unchanged
4	$5 \leq 6$, $i = 2$, swap $2, 4$	$[6, 5, 13, 10, 8, 3, 2, 11]$
5	$8 > 6$, $i = 2$	Unchanged
6	$3 \leq 6$, $i = 3$, swap $3, 6$	$[6, 5, 3, 10, 8, 13, 2, 11]$
7	$2 \leq 6$, $i = 4$, swap $4, 7$	$[6, 5, 3, 2, 8, 13, 10, 11]$
8	$11 > 6$, $i = 4$	Unchanged
End	Swap $1, 4$, return 4	$[2, 5, 3, 6, 8, 13, 10, 11]$

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Check the same pivot in both variants

Same pivot key, different internal orders

Both: q=4, pivot=6; left <=6, right >6. Regions need not be sorted.

To choose the same key 6 in the last-position version, first exchange the endpoints, then partition $[11, 10, 13, 5, 8, 3, 2, 6]$. The result can be $[5, 3, 2, 6, 8, 10, 13, 11]$. Both place 6 at position 4 and make 7 scan-key comparisons; their internal region orders may differ.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Compare contracts, boundaries, and return values

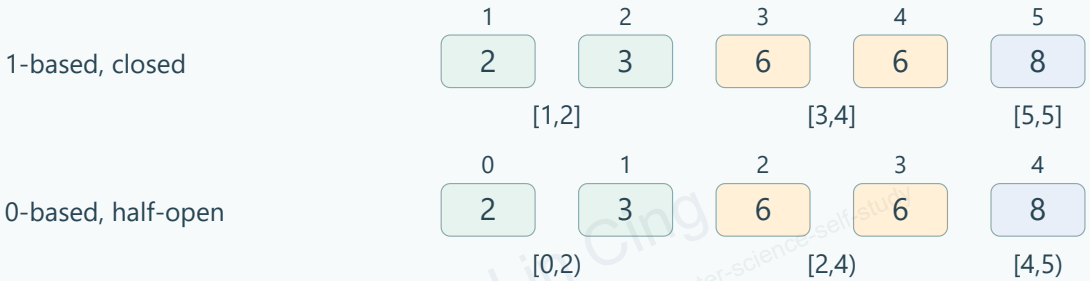
Version	Pivot and initial boundaries	Return value and recursion
CLRS last-position single scan	$x = A[r], i = p - 1, j = p$	One pivot index q; recurse on $[p, q - 1], [q + 1, r]$
MIT 2005 first-position single scan	$x = A[p], i = p, j = p + 1$	One pivot index q; exclude q again
First-position inward scan with final placement	$x = A[p]$; both scans stop on equality	Exchange pivot after crossing; return its position q
This chapter's three-way partition	Save pivot key x; $lt = i = p$, $gt = r$	Equal closed interval $[lt, gt]$; exclude that whole interval

Inward scanning still leaves two return contracts to distinguish. Classical Hoare partition returns a split boundary without guaranteeing the pivot's final position. Another inward-scan variant finally exchanges the pivot and returns its position. Their recursive intervals differ; the name Hoare alone is insufficient. The inward-placement code below specifies the latter procedure.

Two interval contracts for the same records

The two highlighted keys are the same records: closed $[3, 4]$ in 1-based indexing, half-open $[2, 4)$ in 0-based indexing.

Same records; different interval contracts



Changing notation moves no records. Exclude both equal keys from recursion.

Convert closed $[p, r]$ to half-open $[p - 1, r)$: exclude the right endpoint. Likewise $[p, q - 1]$ becomes $[p - 1, q - 1)$. Empty intervals have length zero.

Inward scanning with final placement stops on equal keys

Use closed interval $p \le r$ with the first record as pivot. Evaluate and left to right with short-circuiting: check the boundary before reading an array entry.

```
PARTITION-INWARD-PLACE(A,p,r):
    x = key(A[p]); i = p+1; j = r
    while i <= j:
        while i <= j and key(A[i]) < x: i += 1
        while i <= j and key(A[j]) > x: j -= 1
        if i >= j: break
        swap(A[i],A[j])
        i += 1; j -= 1
    swap(A[p],A[j])
    return j
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Inward scanning: invariant and complete exchanges

Classified regions satisfy $A[p+1..i-1] \le x$ and $A[j+1..r] \ge x$. Scans skip strictly smaller or strictly larger keys. Before crossing, exchange the stopped records and advance both indices, so equal keys cannot stall progress. The unknown region shrinks; finally exchange the pivot into j. Recurse on $[p, j-1]$ and $[j+1, r]$.

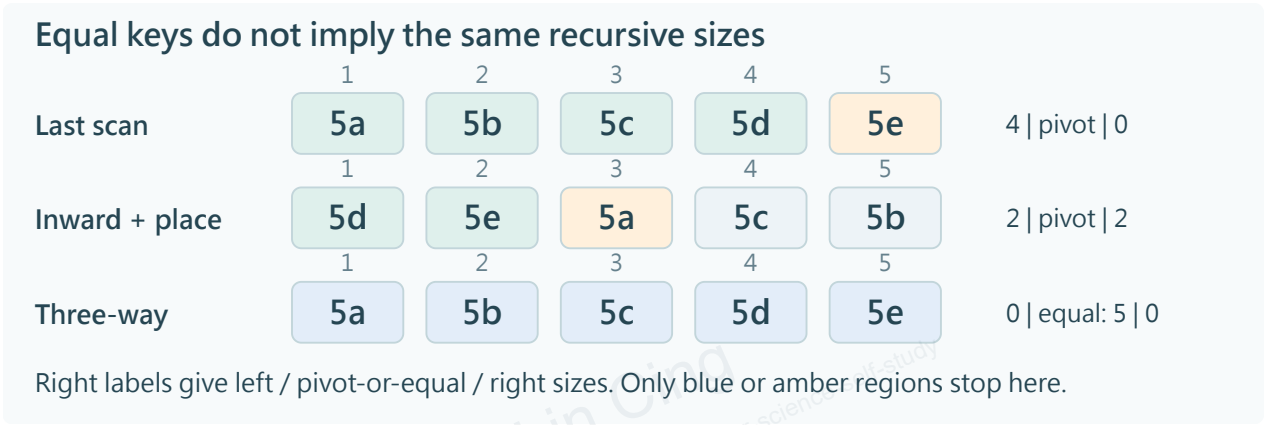
Input $[6, 10, 13, 5, 8, 3, 2, 11]$, $x = 6$. Scan stops are recorded before the exchange.

Stopped (i, j)	Exchange	Complete array afterward
$(2, 7)$	$2, 7$	$[6, 2, 13, 5, 8, 3, 10, 11]$
$(3, 6)$	$3, 6$	$[6, 2, 3, 5, 8, 13, 10, 11]$
$(5, 4)$ crossed	Final $1, 4$	$[5, 2, 3, 6, 8, 13, 10, 11]$

Return $q = 4$. Left keys are ≤ 6 and right keys are ≥ 6; neither side needs to be internally sorted.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Five equal keys produce three different partition outcomes



Letters identify records. Inward placement swaps $2, 5$, then $3, 4$, then $1, 3$. For all-equal inputs, its nearly halved sides give $\Theta(n \lg n)$ time, versus $\Theta(n^2)$ for last-position scanning and $\Theta(n)$ for three-way partitioning. These are input-family bounds; inward scanning can still be quadratic on other inputs.

One classification can require two key comparisons

The exact formula counts executions of `key(A[j]) <= x` in the last-position single scan: one per nonpivot key. A three-way version that tests `< x` and then, on failure, `> x` can use two Boolean comparisons per classification, including for an equal key.

For keys $[1, 4, 3]$ classified against pivot value $x = 3$, key 1 needs one test; keys $4, 3$ need two each, totaling five rather than three. In-place three-way swapping may change processing order, but its branch-test rule must still be counted.

The data-structures chapter uses 0-based half-open $[l, r)$ and equal region $[lt, gt)$. This chapter uses 1-based closed $[p, r]$ and equal region $[lt, gt]$. Both exclude the entire equal region, but boundaries and exact comparison constants require separate verification.

With distinct keys and conditionally uniform pivots, three-way quicksort still has expected $\Theta(n \lg n)$ time: each level performs at most constant-factor linear classification work, with the same recursive sizes. This asymptotic conclusion does not make its Boolean count exactly $2(n + 1)H_n - 4n$.

Four ranks: pair probabilities differ from run costs

Name four distinct keys z_1, z_2, z_3, z_4 by increasing rank; their input order is arbitrary.

Rank pairs	Interval length	Direct-comparison probability
$(1,2), (2,3), (3,4)$	2	1 each
$(1,3), (2,4)$	3	$2/3$ each
$(1,4)$	4	$1/2$

Thus $\mathbb{E}[C] = 3 + 4/3 + 1/2 = 29/6$. Substituting $H_4 = 25/12$ gives the same $2 \cdot 5 \cdot 25/12 - 16 = 29/6$.

A run uses an integer number of comparisons. A first pivot of rank 2 or 3 gives four comparisons with probability $1/2$. An endpoint first, then the median of the remaining three, gives five with probability $(1/2)(1/3) = 1/6$. Remaining runs use six, with probability $1/3$. Hence $4/2 + 5/6 + 6/3 = 29/6$, independently checking the expected run cost.

Adjacent ranks always compare, but not every pair does. Average $29/6$ does not mean any run performs a fractional comparison.

Counting sort assigns an output block to each key

Preconditions: integer $K \geq 0$ and keys in $0..K$. Input/output positions are $1..n$. Key-indexed storage must support $K + 1$ entries, and each counter must represent values $0..n$.

```
COUNTING-SORT(A,K):
    C[0..K] = 0
    for j = 1 to n: C[key(A[j])] += 1
    for v = 1 to K: C[v] += C[v-1]
    for j = n downto 1:
        v = key(A[j])
        B[C[v]] = A[j]
        C[v] -= 1
    return B
```

After counting, $C[v]$ is the frequency of key v. After prefix accumulation, it counts records with keys $\leq v$: the **last position** of that key's output block. Placement consumes the counters; copy the unconsumed prefix table to P first if range queries will be needed.

Ending-position placement: trace every write

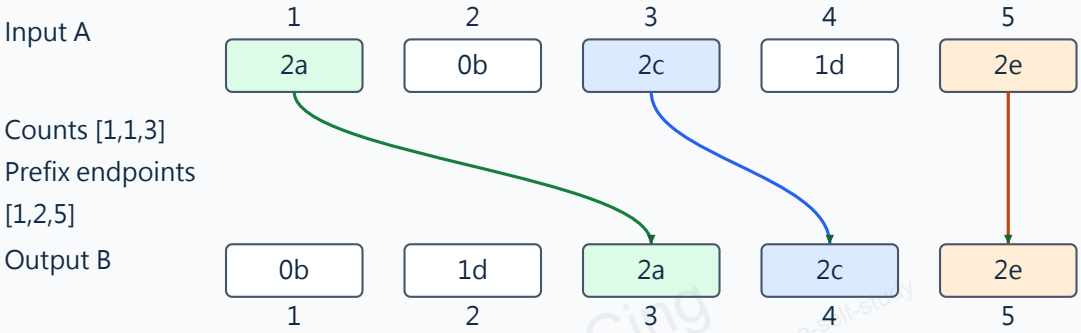
Take $A = [2_a, 0_b, 2_c, 1_d, 2_e]$ and $K = 2$. Frequencies of keys $0, 1, 2$ are $[1, 1, 3]$, with prefix ending positions $[1, 2, 5]$. Their output blocks are position 1, position 2, and positions $3..5$.

Scan index j	Record	Position written in B	C after placement
5	2_e	5	$[1, 2, 4]$
4	1_d	2	$[1, 1, 4]$
3	2_c	4	$[1, 1, 3]$
2	0_b	1	$[0, 1, 3]$
1	2_a	3	$[0, 1, 2]$

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Equal keys retain their identity order

Counting sort: track destinations of equal-key identities



Process e→d→c→b→a backward; positions are 5→2→4→1→3.

Green a, blue c, orange e all have key 2. Output keeps a→c→e.

Output is $[0_b, 1_d, 2_a, 2_c, 2_e]$. Key-2 identities still occur in order a, c, e, establishing stability. Bare keys $[0, 1, 2, 2, 2]$ cannot show whether stability holds.

Placement invariant, stability, and cost

Fix key v and let $L = P[v-1]$ (use $L = 0$ for $v = 0$), $R = P[v]$. Key-v records in the processed input suffix occupy exactly the right end $[C[v]+1, R]$ of their block. Counter $C[v]$ identifies its rightmost unfilled position.

Another key-v record is still unplaced, so $C[v] > L$. Its write is legal and cannot overwrite a placed record; decrementing the counter preserves the invariant. At termination $C[v] = L$, every block contains exactly its frequency of records. Increasing block order proves sortedness and multiset preservation.

If equal-key record x precedes y in input, reverse scanning places y to the right before placing x to its left. This proves stability. Forward scanning with the same ending-position decrement reverses ties; the next section's starting-position increment is the matching forward rule.

Initialization, counting, prefix accumulation, and placement give $\Theta(n + K + 1)$ time and auxiliary space including output. This is linear when $K = O(n)$ and $n \geq 1$; a huge key domain still incurs table cost for a small input.

Save prefix counts before consuming positions

For $0 \leq a \leq b \leq K$, subtract prefix counts to count the keys in $[a, b]$:

$$Q(a,b) = \begin{cases} P[b], & a = 0, \\ P[b] - P[a-1], & a > 0. \end{cases}$$

Copy P before placement. For the earlier five records:

Key v	0	1	2
Frequency $F[v]$	1	1	3
Saved prefix $P[v]$	1	2	5
Consumed counter $C[v]$	0	1	2

$Q(1,2) = 5 - 1 = 4$; consumed counters give the wrong answer $2 - 0 = 2$. For $a = 0$, use $Q(0,0) = P[0] = 1$, with no negative index.

Preprocessing: $\Theta(n + K + 1)$. Each query: $O(1)$.

Stable counting sort can also fill from left to right

The ending-position version scans right to left and decrements its counter. Another stable version starts from each key's **first output position** $S[v]$, scans left to right, and increments that position.

Let $F[v]$ be the frequency of key v in domain $0..K$:

```
S[0] = 1
for v = 1 to K:
    S[v] = S[v-1] + F[v-1]
for j = 1 to n:
    v = key(A[j])
    B[S[v]] = A[j]
    S[v] = S[v] + 1
```

Initially $S[v] = 1 + \sum_{u<v} F[u]$, the first position of that key's block. Empty key blocks may share a start position because no records are written for them.

Trace all eight records through starting positions

Take $A = [3_a, 1_b, 4_c, 2_d, 6_e, 1_f, 2_g, 2_h]$ and $K = 6$. Frequencies for keys $0..6$ are $[0, 2, 3, 1, 1, 0, 1]$, giving initial $S = [1, 1, 3, 6, 7, 8, 8]$.

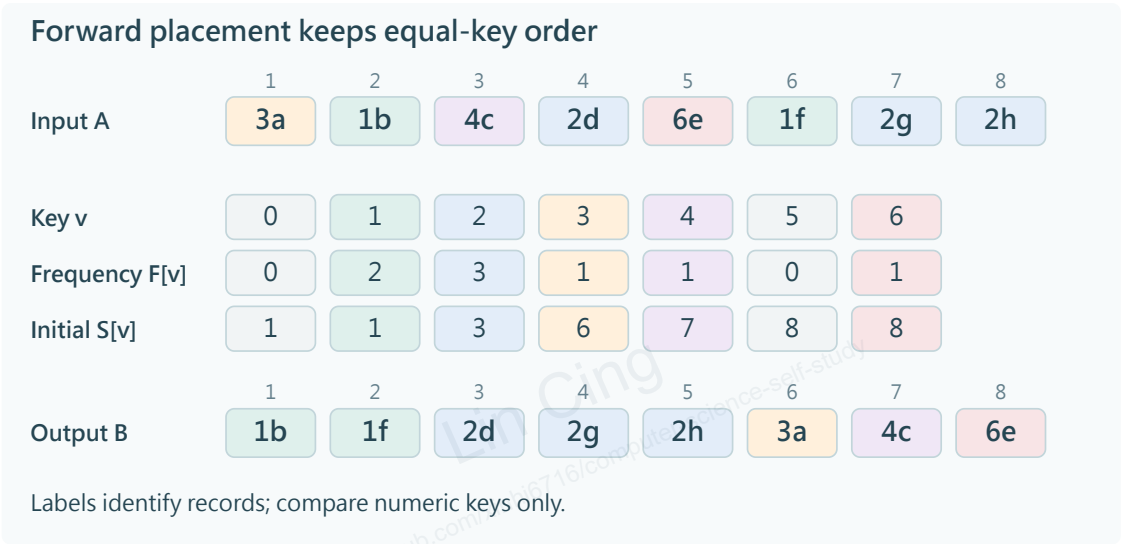
j	Record	Output position	Next position for its key
1	3_a	6	$S[3] = 7$
2	1_b	1	$S[1] = 2$
3	4_c	7	$S[4] = 8$
4	2_d	3	$S[2] = 4$

Finish the eight writes and interpret the final pointers

j	Record	Output position	Next position for its key
5	6_e	8	$S[6] = 9$
6	1_f	2	$S[1] = 3$
7	2_g	4	$S[2] = 5$
8	2_h	5	$S[2] = 6$

The result is $[1_b, 1_f, 2_d, 2_g, 2_h, 3_a, 4_c, 6_e]$. A final pointer may be one past its key block: it denotes the next unused position, not a position that will necessarily be accessed.

Equal-key records fill their block in input order



1_b takes position 1 before 1_f takes position 2. Records $2_d, 2_g, 2_h$ take positions 3,4,5 in that order. Color connects each equal-key block; identity labels check stability.

The filled prefix and next-free-position invariant

Let $S_0[v]$ be the unconsumed start and let $c_j(v)$ count key-v records in the already-scanned first $j-1$ inputs. At each iteration's start,

$$S[v] = S_0[v] + c_j(v).$$

Block $[S_0[v], S[v]-1]$ contains exactly the scanned key-v records in input order. Different key blocks do not overlap.

Initially $c_1(v) = 0$ and all filled prefixes are empty. On reading another key-v record, $c_j(v) < F[v]$ because that record is still unplaced; the write is inside its legal block. Writing its next free position and incrementing $S[v]$ preserves records and order, without changing other blocks.

At termination $c_{n+1}(v) = F[v]$, so every key block is full and blocks occur in increasing key order. The result is correct and stable. Counting, start construction, and scanning take $\Theta(n+K+1)$ time and $\Theta(n+K+1)$ auxiliary space including output.

Pair scan direction with position meaning: forward/start/increment, or backward/end/decrement. Changing only the scan direction may preserve sorted keys while reversing equal-key identities.

LSD radix sorting preserves lower-digit order within ties

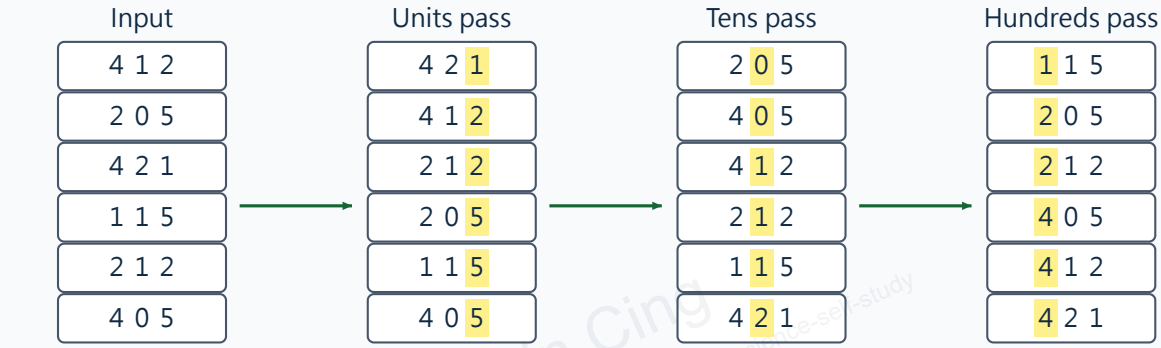
Let every key be a fixed $d \geq 1$-digit nonnegative integer in base $r \geq 2$, padded with leading zeros. LSD processes digit positions $1, 2, \ldots, d$ from least to most significant, using a **stable** sort at every pass.

Invariant: after pass t, the sequence is sorted by its lowest t digits. The $t = 0$ condition is empty. Pass t groups records by that digit. It determines order between different groups, while stability retains the previous $t-1$-digit order inside a group. Thus the lowest t digits are sorted; $t = d$ gives complete order.

Lower-digit order is retained **within an equal higher-digit group**, not globally across all records. For $[12, 11]$, units sorting gives $[11, 12]$. Tens digits are both 1; an unstable reversal would incorrectly restore $[12, 11]$.

Sorting the entire sequence by hundreds, tens, then units makes units the final primary key. MSD must recurse within equal higher-digit groups; it is not LSD with its loop simply reversed.

Three radix passes with every full sequence



Yellow marks the current digit. Equal current digits retain the previous column order.

In the hundreds pass, 205 precedes 212 because their shared hundreds group retains lower-two-digit order $05 < 12$. Each pass inspects the current digit; the invariant builds full-key order progressively.

Verify every radix pass

Digits processed	Complete sequence
None	$[412, 205, 421, 115, 212, 405]$
Units	$[421, 412, 212, 205, 115, 405]$
Units and tens	$[205, 405, 412, 212, 115, 421]$
All three	$[115, 205, 212, 405, 412, 421]$

In the tens pass, $205, 405$ both have tens digit 0 and retain their previous order. Records $412, 212, 115$ all have tens digit 1 and also retain their prior order; their lowest two digits are $12, 12, 15$.

Account jointly for bit length, radix, and passes

Stable counting sort takes $\Theta(n + r)$ per pass, giving $\Theta(d(n + r))$ over d passes and $\Theta(n + r)$ reusable auxiliary space. Linear time follows directly only for fixed d and $r = O(n)$.

For $b \geq 1$-bit keys and s bits per pass, $1 \leq s \leq b$,

$$r = 2^s, \qquad d = \lceil b/s \rceil, \qquad T = \Theta(\lceil b/s \rceil (n + 2^s)) .$$

Pad a short highest group with zeros. This assumes constant-time key/digit access; integers longer than a machine word require explicit extraction and movement costs.

For $n \geq 2$, choose $s = \min(b, \lfloor \lg n \rfloor)$, so $2^s \leq n$ and time is $O(n\lceil b/s \rceil)$. When $b = O(\lg n)$ this is $O(n)$.

Under a cost of $n + r$ per pass, 32-bit keys with $s = 8$ cost $4(n + 256)$, while $s = 16$ costs $2(n + 65536)$. The latter is smaller exactly when $n > 65024$, with equality at that threshold. Actual allocation, cache, and bandwidth costs lie outside this specified model.

Distribution is a shared technique, not one algorithm

Method	Distribution rule	What follows distribution
LSD radix sort	Digits from least to most significant	Stable full-sequence sort at each pass
MSD radix sort	Start with the most significant digit	Recurse on the next digit inside each equal-prefix group
Interval bucket sort	Map $x \in [0, 1)$ to $\lfloor nx \rfloor$	Fully sort each bucket, then concatenate buckets

All can use buckets, but their **digits, recursion rules, and cost assumptions differ**. Ten digit buckets are not the same as n equal-width intervals. Fully sorting a bucket is not the same procedure as recursively distributing its next digit.

Grouping $[312, 305, 121, 118, 214, 207]$ by hundreds yields $[121, 118]$, $[214, 207]$, $[312, 305]$. This is not yet sorted. MSD processes lower digits within each group; interval bucket sorting uses its specified local sorting algorithm. Concatenating the unsorted groups is incorrect.

Interval bucket sorting distributes, sorts, and concatenates

For $n \geq 1$ keys $x \in [0, 1)$, create n buckets $B[0..n-1]$ and append each key to $B[\lfloor nx \rfloor]$ in input order. Insertion-sort each bucket, then concatenate by bucket index. Empty input returns immediately. Key $x = 1$ is outside the domain and cannot index $B[n]$.

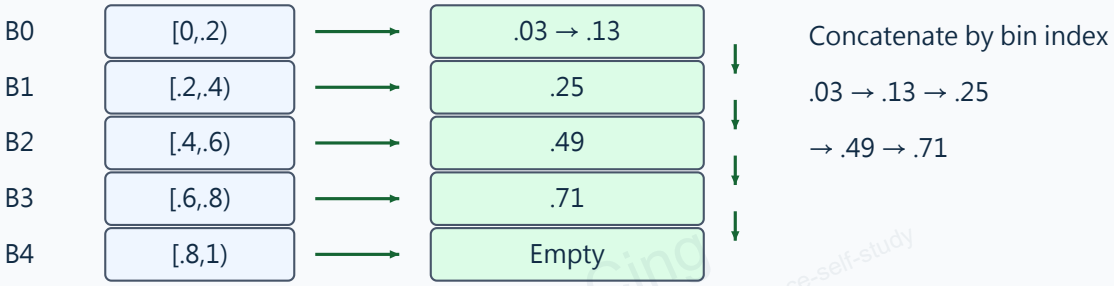
Correctness needs no distribution assumption: $x \leq y$ implies $\lfloor nx \rfloor \leq \lfloor ny \rfloor$. Different buckets are ordered by disjoint half-open intervals; local sorting orders records within each bucket. Input-order appending plus stable insertion sort also guarantees stability; head insertion may change equal-key order first.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Five buckets: interval, local order, and output

Bucket intervals guarantee cross-bin order

Input .13,.25,.03,.71,.49 → bins 0,1,0,3,2



Blue: half-open bin intervals. Green: sorted contents. No further cross-bin comparisons.

For $n = 5$, input $[.13, .25, .03, .71, .49]$ maps to bucket indices $[0, 1, 0, 3, 2]$. Bucket 0 becomes $[.03, .13]$; buckets $1, 2, 3$ contain $.25, .49, .71$ respectively; bucket 4 is empty. Output is $[.03, .13, .25, .49, .71]$.

Expected linear time uses squared bucket sizes

Let N_i be bucket i's size. Initialization, distribution, and concatenation cost $\Theta(n)$; insertion sorting costs at most a constant times $\sum_i N_i^2$. Thus

$$T \le c_0 n + c_1 \sum_{i=0}^{n-1} N_i^2.$$

Now introduce the input assumption: keys are **mutually independent and uniform** on $[0, 1)$. Indicator X_{ij} records whether input j enters bucket i. Then $N_i = \sum_j X_{ij}$, $X_{ij}^2 = X_{ij}$, and

$$\begin{aligned}\mathbb{E}[N_i^2] &= \sum_j \mathbb{E}[X_{ij}] + \sum_{j \ne k} \mathbb{E}[X_{ij} X_{ik}] \\ &= n\frac{1}{n} + n(n-1)\frac{1}{n^2} = 2 - \frac{1}{n}.\end{aligned}$$

Hence $\mathbb{E}[\sum_i N_i^2] = 2n - 1$, giving $\mathbb{E}[T] = O(n)$. Reading the input supplies $\Omega(n)$, so expected time is $\Theta(n)$.

Independence is used for the destinations of different records j, k. Bucket sizes are dependent because they sum to n, but expectation linearity still applies. Knowing $\mathbb{E}[N_i] = 1$ does not imply $\mathbb{E}[N_i^2] = 1$.

Concentration, dependence, and a genuine worst-case input

If records remain independent but bucket probabilities are p_i, with $\sum_i p_i = 1$, the same expansion gives

$$\mathbb{E}\left[\sum_i N_i^2\right] = n + n(n-1)\sum_i p_i^2.$$

The expected upper bound remains linear when $\sum_i p_i^2 = O(1/n)$. Uniform probabilities give $1/n$; concentrating all probability in one bucket makes the squared-size sum n^2.

Uniform marginals alone do not suffice. Choose uniform $J \in \{0, \dots, n-1\}$ and set every $x_j = (J + 1/2)/n$. Every record has uniform bucket marginals, yet all enter one bucket, so $\sum_i N_i^2 = n^2$. This refutes a squared-cost estimate; equal-key insertion sorting itself may still be linear.

For an actual quadratic-time family, take $n \ge 2$ and $x_j = (n - j + 1)/(2n^2)$. These keys are distinct, strictly decreasing, and all in bucket 0. Insertion sort performs $n(n-1)/2$ shifts, establishing worst-case $\Theta(n^2)$ time.

Replacing each local sort by a worst-case $O(m \lg(m+1))$ method gives total worst-case $O(n \lg(n+1))$. Independent uniform inputs retain linear expected time because $m \lg(m+1) = O(m^2)$.

Order statistics and extrema: specify the target

Inputs contain $n \geq 1$ totally ordered keys. The kth smallest value occupies position k in the sorted multiset, for $1 \leq k \leq n$. Repeated occurrences take separate positions but can yield the same answer: ranks $1, 2$ of $[2, 5, 2, 8]$ both return 2.

Our lower median has rank $\lceil n/2 \rceil$. For even n, it is the smaller central value, unlike the statistical convention that averages the two central values. Reject empty inputs and invalid ranks at entry.

A minimum scan initializes from the first record and compares each remaining key with the current minimum. Its invariant is that the current value is the minimum of the processed prefix. It uses $n - 1$ key comparisons; maximum is symmetric.

Distinct-key inputs give a matching comparison lower bound: every nonminimum record must be shown larger than something before it can be eliminated. A comparison can eliminate at most one new minimum candidate. Thus at least $n - 1$ comparisons are needed. Using distinct keys for a worst-case lower bound does not restrict the algorithm to distinct-key inputs.

Paired extrema: eight records, ten comparisons

Compare each pair first: only its smaller record challenges the minimum, its larger the maximum. Initialize from the first pair; later pairs preserve both extrema of the processed prefix.

Input: $[14, 3, 9, 1, 8, 5, 12, 6]$.

Pair	Smaller, larger	Minimum	Maximum	Comparisons
$14, 3$	$3, 14$	3	14	1
$9, 1$	$1, 9$	1	14	4
$8, 5$	$5, 8$	1	14	7
$12, 6$	$6, 12$	1	14	10

Even n: $1 + 3(n - 2)/2$. Odd n: initialize from one record, then use $3(n - 1)/2$. Both give $\lceil 3n/2 \rceil - 2$ comparisons (zero for $n = 1$). Ties preserve correct extremal values.

Why paired extrema achieve the worst-case lower bound

For distinct keys, every record initially qualifies as both a minimum and a maximum candidate. Only one qualification of each type survives, so $2n - 2$ qualifications must be eliminated.

An adversary answers comparisons as follows. Let U mean untested, H maximum-only, L minimum-only, and M neither.

Compared states	Answer and new eliminations
U, U	Choose a larger and a smaller key; eliminate 2 qualifications
H, H or L, L	Eliminate one maximum or minimum candidate
H, L	Answer $H > L$; eliminate none
U versus a classified key	Against H, answer $U < H$; against L, answer $U > L$; against M, choose either consistent direction. Eliminate one U qualification
Involving M, with no U	Maintain $H > M > L$; for M, M use a linear extension of the existing order. At most one new elimination

Count the eliminated candidate qualifications

The answers stay consistent: H keys have no known larger key, L keys no known smaller key, and U keys no constraints. Place H above M above L, extending the comparison order within each class; no cycle is introduced.

Only U, U removes two qualifications, at most $\lfloor n/2 \rfloor$ times. Other comparisons remove at most one. Thus

$$2n - 2 \le c + \lfloor n/2 \rfloor, \qquad c \ge \lceil 3n/2 \rceil - 2.$$

Initially
16 qualifications
8 × (min + max)

−8 →

Four pair comparisons
8 qualifications
H: 14, 9, 8, 12
L: 3, 1, 5, 6

−6 →

Six further comparisons
2 qualifications
max: 14; min: 1

Pair tests first: 4 × 2 + 6 × 1 = 14 eliminations; 4 + 3 + 3 = 10 tests.

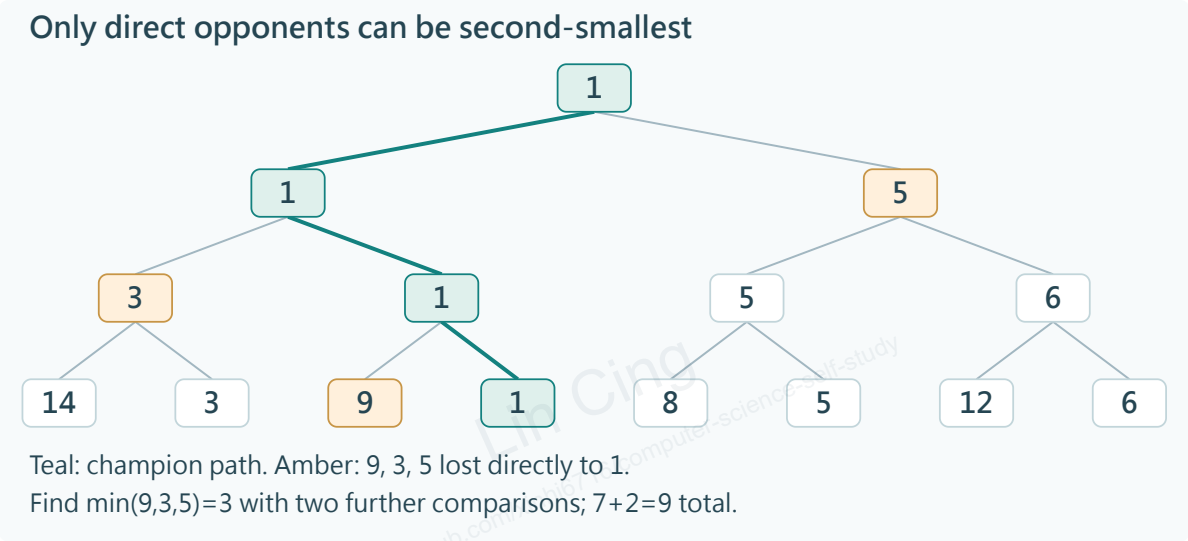
A tournament preserves the second-minimum candidates

Advance the smaller key and retain each winner's directly defeated opponents. The second-smallest key must lose directly to the minimum: losing to a different key would put two keys below it. Assume distinct keys initially.

For $n \geq 2$, a balanced bracket can use byes rather than fictional keys. Eliminating one record per comparison costs $n - 1$. The champion faces $d \leq \lceil \lg n \rceil$ actual opponents; searching that list uses $d - 1$ more. The worst-case upper bound is $n + \lceil \lg n \rceil - 2$. With duplicate keys, break ties by $(key, identity)$; the result then means the second record, not necessarily the second distinct value.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Eight keys: whom did the champion defeat?



Champion 1 directly defeated $9, 3, 5$, so the second minimum is their minimum, 3. The tournament uses 7 comparisons and the opponent list uses 2, totaling 9.

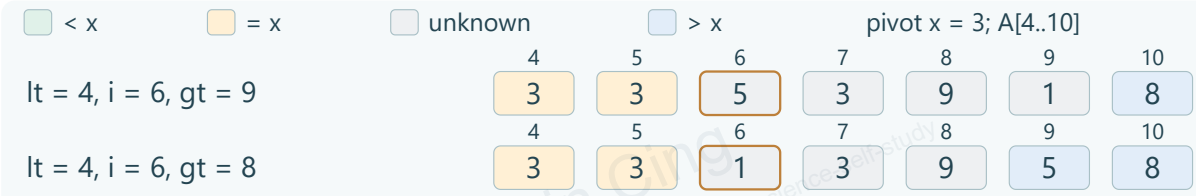
Three-way selection removes the whole equal-key region

Sorting processes both sides; selection retains only the side containing the answer. Array indices use **one-based closed intervals** $A[p..r]$. Target k is the local rank in this subarray, with $1 \le k \le r - p + 1$. Compare keys and exchange whole records; x is a key present in the subarray.

```
PARTITION3(A,p,r,x):
    lt = p; i = p; gt = r
    while i <= gt:
        if key(A[i]) < x:
            swap(A[lt],A[i]); lt += 1; i += 1
        elif key(A[i]) > x:
            swap(A[i],A[gt]); gt -= 1
        else:
            i += 1
    return (lt,gt)
```


Three-way partition: invariant and progress

At loop entry, $A[p..lt-1] < x$, $A[lt..i-1] = x$, $A[i..gt]$ is unknown, and $A[gt+1..r] > x$; empty ranges are permitted. A smaller key extends the less-than and equal regions; an equal key extends the equal region. A larger key swaps in an unclassified right-end record, so i must stay. Each iteration decreases unknown length by one, and swaps preserve the multiset. Termination removes the unknown region.



Swap A[6] = 5 with A[9] = 1; gt becomes 8, but i stays 6.
Next classify the new A[6] = 1. Since p = lt = 4, the < x region is empty.

Randomized selection changes the local rank only on the right

```
RANDOM-SELECT(A,p,r,k):
    require 1 <= k <= r-p+1
    while p < r:
        x = key(A[uniform-integer(p,r)])
        (lt,gt) = PARTITION3(A,p,r,x)
        L = lt-p; E = gt-lt+1
        if k <= L: r = lt-1
        elif k <= L+E: return x
        else: k -= L+E; p = gt+1
    return key(A[p])
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Selection: preserve the requested value and shrink

Because x is drawn from current records, $E \geq 1$. Local ranks $L+1..L+E$ all return x. A left move preserves k; a right move subtracts the L smaller and E equal occurrences.

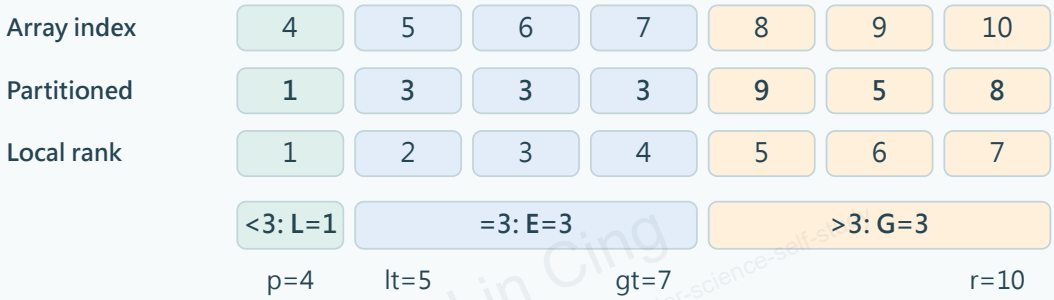
Loop invariant: the current subarray's kth smallest key equals the original requested key. The left branch has $1 \leq k \leq L$; the right branch has $1 \leq k - L - E \leq r - gt$, so neither retains an empty range. Every nonreturning step reduces size by at least E. A singleton returns its sole key, proving correctness and termination even with duplicates.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Separate local ranks, region lengths, and absolute indices

Array indices differ from local ranks

A[4..10] = [8,3,5,3,9,1,3]; pivot x=3



k=1: left rank 1; k=2..4: return 3; k=6: right rank 6-1-3=2.

Equal indices [5,7], local ranks [2,4]; L=lt-p, E=gt-lt+1.

For $A[4..10] = [8, 3, 5, 3, 9, 1, 3]$ with $x = 3$, partition returns $lt = 5, gt = 7$, hence $L = 1, E = 3, G = 3$. Equal-region local ranks are $2..4$.

Convert each target using region lengths

Original local rank	Test	Next state or answer
1	$1 \le L$	Rank 1 in $A[4..4]$
4	$L < 4 \le L + E$	Return 3
6	$6 > L + E$	Rank $6 - 1 - 3 = 2$ in $A[8..10]$

Comparing k directly with lt mixes origins: k starts at the current subarray, whereas lt starts at the full array's index origin. An alternative absolute target $t = p + k - 1$ stays unchanged through iterations; do not also decrement a local rank under that convention.

Randomized selection: strong induction over all states

Conditioned on the current array, choose the pivot uniformly among its n records. Future pivots are freshly uniform within their then-current subarrays. For distinct keys, local pivot rank R is uniform on $1..n$. Whatever the target rank, the next size is at most

$$m_R = \max(R-1, n-R).$$

Let $F(A,k)$ be the expected remaining cost from that state. Bound current scanning and overhead by cn. Assume $F(B,j) \le a|B|$ for **every shorter state**. Conditioning on the chosen pivot and applying induction gives

$$F(A,k) \le cn + \frac{a}{n}\sum_{r=1}^{n} \max(r-1, n-r).$$

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Average the larger-side sizes explicitly

For $n = 2h$, larger-side sizes $h, \dots, 2h-1$ each occur twice: the sum is $2\sum_{t=h}^{2h-1} t = 3h^2 - h$. For $n = 2h+1$, the middle rank contributes h and all other sizes occur in symmetric pairs: the sum is $h + 2\sum_{t=h+1}^{2h} t = 3h^2 + 2h$. Dividing by n gives at most $3n/4$, so

$$F(A,k) \le (c + 3a/4)n \le an \qquad (a \ge 4c).$$

Choose a large enough for the $n = 1$ base case. This proves $O(n)$; the initial partition scans $n > 1$ records, giving expected $\Theta(n)$ time.

No monotonicity of an unknown cost function or product of dependent expectations was assumed. The $3n/4$ bound concerns the expected larger side, not a deterministic upper bound on every partition.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Duplicates preserve expected linear time with three-way selection

For analysis only, break equal-key ties by a fixed identity order to assign unique ranks $R \in \{1, \ldots, n\}$. Actual partitioning still compares keys, not identities. Uniformly sampling a record makes this analytical rank uniform.

If the answer is in $< x$, its region has length at most $R - 1$; if in $> x$, at most $n - R$; an equal-region answer returns immediately. Every pivot identity therefore admits the same bound $m_R = \max(R - 1, n - R)$. The preceding strong induction over all smaller states applies, yielding expected $\Theta(n)$ time.

Worst-case execution remains $\Theta(n^2)$: with distinct keys, seek the minimum while choosing the current maximum every time. Scan sizes are $n, n - 1, \ldots, 2$, a possible positive-probability path. Expected and worst-case bounds differ. The iterative version with in-place three-way partition needs $O(1)$ extra space and changes its input.

All-equal input returns after one scan. Switching back to a two-way Lomuto partition that sends every equal key to one side loses this guarantee, even with random pivots.

Deterministic selection: align code with the worked trace

All lists and sorted results below are **one-based**. SORT-COPY sorts a copy by key. STABLE-SPLIT-COPY scans the input into three fresh lists for keys less than, equal to, and greater than x, retaining their input order within each region. This trace-friendly version uses $O(n)$ extra space; it is not an in-place implementation.

Groups contain at most five records; take each lower median. With $g = \lceil n/5 \rceil$ medians, recursively select rank $\lceil g/2 \rceil$ as the pivot.

```
MEDIAN-PIVOT(A):
    M = empty list
    for each consecutive group G of at most 5 records in A:
        B = SORT-COPY(G)
        append key(B[ceil(length(B)/2)]) to M
    return DETERMINISTIC-SELECT(M,ceil(length(M)/2))
```

Deterministic selection: return or recurse on one region

```
DETERMINISTIC-SELECT(A,k):
    n = length(A); require 1 <= k <= n
    if n <= 5: return key(SORT-COPY(A)[k])
    x = MEDIAN-PIVOT(A)
    (L,E,G) = STABLE-SPLIT-COPY(A,x)
    if k <= length(L):
        return DETERMINISTIC-SELECT(L,k)
    elif k <= length(L)+length(E): return x
    else:
        return DETERMINISTIC-SELECT(G,k-length(L)-length(E))
```

Correctness follows from region order and induction on size. Pivot recursion has size $g < n$; answer recursion has size at most $n - 1$, since x is present and the equal region is nonempty. Both terminate, and both must be charged in the time analysis.

Groups of five certify exclusions, not an exact median

Median-of-medians: three witnesses per full group

One group per column, larger above. Middle row: group medians. Pivot x=12.

	G1	G2	G3	G4	G5
	25	24	23	22	21
	17	18	19	20	16
Median→	4	8	12	14	15
	2	5	9	10	13
	1	3	6	7	11

Medians [4,8,12,14,15] → third smallest 12. The overall median is 13.

Blue+purple: 9 values ≤12 Orange+purple: 9 values ≥12 Purple: shared pivot

These 25 records contain exactly keys $1..25$. Group medians are $[4, 8, 12, 14, 15]$, giving pivot $x = 12$, while the true lower median is rank 13, namely 13.

List the actual witnesses on both sides

For the three groups whose medians are ≤ 12, their smallest triples are $[1, 2, 4]$, $[3, 5, 8]$, and $[6, 9, 12]$: nine certified values ≤ 12. For the three groups with medians ≥ 12, their largest triples are $[12, 19, 23]$, $[14, 20, 22]$, and $[15, 16, 21]$: nine values ≥ 12.

Overlap at 12 is harmless. A strict greater-than recursion excludes all $\leq x$; a strict less-than recursion excludes all $\geq x$. These are alternative branches, not simultaneous claims about disjoint witnesses. With duplicates, the non-strict certificates still hold and three-way partitioning removes the entire equal region.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Rounding and incomplete groups: finish the linear-time induction

Among $g = \lceil n/5 \rceil$ group medians, their lower median x has at least $\lceil g/2 \rceil$ medians on each non-strict side. Conservatively discard the possible incomplete group and the group containing the selected pivot identity. At least $\lceil g/2 \rceil - 2$ complete groups remain on either side.

Each supplies three witnesses, so each non-strict side has at least

$$3(\lceil g/2 \rceil - 2) \geq 3n/10 - 6$$

records. A strict recursive side therefore has size at most $7n/10 + 6$. A negative small-input lower bound is merely uninformative; the actual nonempty equal region still ensures strict shrinkage.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Pay for both recursive calls in the induction

Let nonrecursive work be at most cn. Inductively bound every shorter state by a times its length. For $n \geq 140$,

$$\begin{aligned} T(n) &\leq a\lceil n/5 \rceil + a(7n/10 + 6) + cn \\ &\leq (9a/10 + c)n + 7a \leq an. \end{aligned}$$

Choose $a \geq 20c$, so $a/10 - c \geq a/20$ and $n \geq 140$ gives $an/20 \geq 7a$. Increase a to cover all finite sizes $1 \leq n < 140$; this proof threshold does not change the code's size-five base case. Initial grouping costs $\Omega(n)$, establishing worst-case $\Theta(n)$ time.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Rank sixteen: trace groups, region sizes, and the next rank

Concatenate these five groups as the input, without first sorting the whole collection:

$[1, 2, 4, 17, 25]$, $[3, 5, 8, 18, 24]$, $[6, 9, 12, 19, 23]$, $[7, 10, 14, 20, 22]$, $[11, 13, 15, 16, 21]$.

Current (n, k)	Group medians M and pivot x	$(\lvert L\rvert, \lvert E\rvert, \lvert G\rvert)$	Next step
$(25, 16)$	$[4, 8, 12, 14, 15] \to 12$	$(11, 1, 13)$	Greater side, rank 4
$(13, 4)$	$[19, 20, 16] \to 19$	$(6, 1, 6)$	Less side, rank 4
$(6, 4)$	$[15, 16] \to 15$	$(2, 1, 3)$	Greater side, rank 1
$(3, 1)$	Base sort $[16, 17, 18]$	No partition	Return 16

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Reconstruct the next groups from the retained list

The first greater side retains scan order: $[17, 25, 18, 24, 19, 23, 14, 20, 22, 13, 15, 16, 21]$. Groups of five, five, and three have sorted copies $[17, 18, 19, 24, 25]$, $[13, 14, 20, 22, 23]$, and $[15, 16, 21]$. Their medians are $19, 20, 16$; a size-three base call selects 19.

The next less side is $[17, 18, 14, 13, 15, 16]$. Its first five have median 15; the singleton contributes 16, so their lower median is 15. The greater side $[17, 18, 16]$ returns rank one, 16. This trace follows the stable-list code. In-place swaps may change region order and subsequent groups, so they need not produce the same intermediate path.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Changing group size changes the proof obligations

For fixed odd group size $s = 2t + 1$, each full group median certifies $t + 1$ records on a non-strict side. Ignoring rounding and additive constants, the two recursive-size fractions sum to

$$\frac{1}{s} + 1 - \frac{t+1}{2s} = 1 + \frac{1-t}{2s}.$$

Groups of five give $9/10 < 1$, and seven give $6/7 < 1$, leaving room to pay linear work. Incomplete groups and finite base cases still require justification. Groups of three give sum one and only the coarse bound

$$T(n) \le T(\lceil n/3 \rceil) + T(2n/3 + O(1)) + O(n).$$

This proves $O(n \lg n)$, not an $\Omega(n \lg n)$ lower bound for the actual algorithm. A lower bound needs additional evidence that such bad splits persist across levels.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Account for copied lists and the active call chain

The stable-list code uses $O(n)$ extra space: each call retains $O(n)$ copied records and executes its two recursive calls sequentially. For sufficiently large sizes, every active call chain shrinks by a fixed ratio below one, giving a geometric $O(n)$ sum; the finite small-size tail is constant. Worst-case call depth is $O(\lg n)$. In-place partitioning alone does not establish $O(1)$ total space: medians, copies, and the recursive stack also count.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Birthday collisions: build the no-collision complement

Model $k \geq 0$ people as independent uniform choices among $n \geq 1$ dates. This simplified model is not an assertion about real birthday distributions.

Conditioned on j distinct dates so far, the next person has $n - j$ noncolliding choices, with conditional probability $(n - j)/n$. The chain rule gives, for $k \leq n$,

$$\Pr(\text{all dates distinct}) = \prod_{j=0}^{k-1} \left(1 - \frac{j}{n}\right),$$

$$\Pr(\text{at least one collision}) = 1 - \prod_{j=0}^{k-1} \left(1 - \frac{j}{n}\right).$$

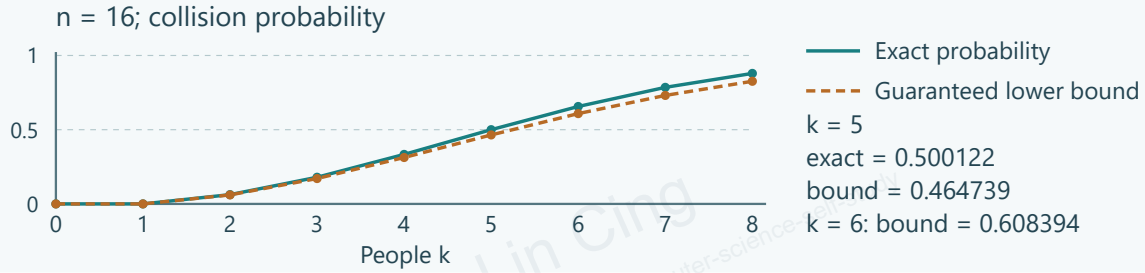
For $k = 0, 1$ there is no collision; the empty product is 1. For $k > n$, the pigeonhole principle gives probability 1 directly. Do not extend the product into negative factors.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A sufficient threshold can be conservative

For $2 \le k \le n$, apply $1 - x \le e^{-x}$ to each factor and take the complement:

$$\Pr(\text{collision}) \ge 1 - \exp\left(-\sum_{j=0}^{k-1} \frac{j}{n}\right) = 1 - e^{-k(k-1)/(2n)}.$$



The bound reaches $1/2$ when $k(k-1) \ge 2n \ln 2$: a **sufficient condition**, not an exact threshold. For $n = 16$, the true probability first exceeds $1/2$ at $k = 5$; the bound certifies it only at $k = 6$.

Expected collision pairs are not collision probability

Let I_{ij} indicate that people i, j share a date. Independence and uniformity give

$$\Pr(I_{ij} = 1) = \sum_{d=1}^{n} \Pr(B_i = d, B_j = d) = n\frac{1}{n^2} = \frac{1}{n}.$$

The collision-pair count is $X = \sum_{i<j} I_{ij}$, hence

$$\mathbb{E}[X] = \frac{\binom{k}{2}}{n}.$$

A collision is event $X \ge 1$. Markov only yields $\Pr(X \ge 1) \le \mathbb{E}[X]$, not a reverse implication. For example, let $X = M^2$ with probability $1/M$ and zero otherwise. Its mean is M, but its positive-value probability is only $1/M$.

Under the birthday model, $k = o(\sqrt{n})$ makes collision probability vanish by this upper bound. For $k \ge C\sqrt{n}$ with sufficiently large fixed C, the exponential complement bound gives a fixed positive collision probability. Both directions establish the $\sqrt{n}$ transition scale.

Sixteen dates: multiply every conditional probability

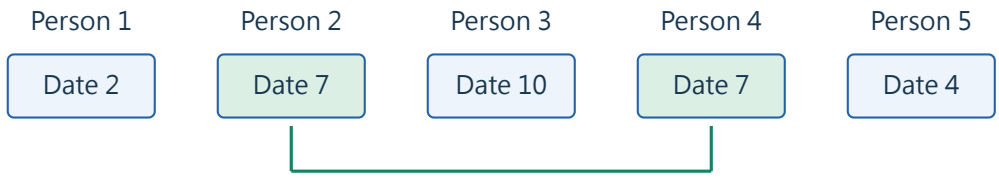
Take $n = 16, k = 5$ and accumulate no-collision probabilities person by person.

Person added	Conditional noncollision probability	Probability all dates remain distinct
1	1	1
2	$15/16$	$15/16$
3	$14/16$	$105/128$
4	$13/16$	$1365/2048$
5	$12/16$	$4095/8192$

Collision probability is $4097/8192 \approx 0.500122$, whereas expected collision pairs are $\binom{5}{2}/16 = 5/8$. At $k = 5$, the exponential complement bound $e^{-10/16} > 1/2$ cannot prove a probability above one half. At $k = 6$, $e^{-15/16} < 1/2$ can. A sufficient condition can be conservative without contradicting the exact result.

A collision in one sample is not a model average

One date sample and its model: distinguish counts from expectations



Only colliding pair in this sample: (2,4)

Model: 16 dates, 5 people. P(collision)=4097/8192≈0.500122.

E[colliding pairs]=10/16=5/8, distinct from the event probability above.

The illustrated sample is $[2, 7, 10, 7, 4]$. Only people $2, 4$ collide, so $X = 1$ and $I_{\{X \geq 1\}} = 1$. These are values in one outcome, unlike averages $5/8$ and $4097/8192$ over all outcomes.

Pairwise independence does not justify a joint product

Let U, V be independent uniform elements of $\{0, 1, 2\}$ and define

$$B_1 = U, \qquad B_2 = V, \qquad B_3 = (2U - V) \bmod 3.$$

Every pair of dates uniquely determines (U, V). The first pair is immediate; B_1, B_3 give $V = 2B_1 - B_3$, and B_2, B_3 give $U = 2(B_2 + B_3)$, modulo 3. Hence every date pair is uniform over nine ordered pairs and is independent.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Enumerate all nine equally likely outcomes

U	V	(B_1, B_2, B_3)	Collision pairs X
0	0	$(0, 0, 0)$	3
0	1	$(0, 1, 2)$	0
0	2	$(0, 2, 1)$	0
1	0	$(1, 0, 2)$	0
1	1	$(1, 1, 1)$	3
1	2	$(1, 2, 0)$	0
2	0	$(2, 0, 1)$	0
2	1	$(2, 1, 0)$	0
2	2	$(2, 2, 2)$	3

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Equal means can hide different collision probabilities

Model	$\Pr(X=0)$	$\Pr(X=1)$	$\Pr(X=3)$	$\Pr(X>0)$	$\mathbb{E}[X]$
Pairwise independent construction	$2/3$	0	$1/3$	$1/3$	1
Three fully independent dates	$2/9$	$2/3$	$1/9$	$7/9$	1

With full independence, the 27 equally likely triples include 6 with all dates distinct, 3 with all dates equal, and $3 \cdot 3 \cdot 2 = 18$ with exactly one equal pair: choose the pair, its date, then the different date. Exactly two equal pairs are impossible: equality is transitive.

Here collision probability is $3/9 = 1/3$. Three fully independent dates give $1 - 6/27 = 7/9$, yet both models have expected collision-pair count 1. Equal expectations do not determine distributions; the exact complement product needs stronger joint assumptions.

Balls into bins: fixed toss count, random loads

Toss $t \geq 0$ balls independently and uniformly into $b \geq 1$ bins. A fixed bin's load N is the sum of t indicators, so $\mathbb{E}[N] = t/b$. Choosing which r tosses hit that bin gives

$$\Pr(N = r) = \binom{t}{r}(1/b)^r(1-1/b)^{t-r}, \quad 0 \leq r \leq t.$$

Let Z_0, Z_1 count empty bins and singleton bins. Applying indicators to each bin gives

$$\mathbb{E}[Z_0] = b(1-1/b)^t,$$

$$\mathbb{E}[Z_1] = b\left[t\frac{1}{b}(1-1/b)^{t-1}\right] = t(1-1/b)^{t-1}, \quad t \geq 1.$$

For $t = 0$, directly take $Z_0 = b, Z_1 = 0$. For $b = 1$, evaluate the events directly or interpret boundary formulas with empty-product convention $0^0 = 1$.

For $t = b = 4$, a fixed bin has mean load 1, yet $\mathbb{E}[Z_0] = 81/64$ and $\mathbb{E}[Z_1] = 27/16$. Loads sum to four and are dependent. Mean one does not mean exactly one ball in every bin.

Collecting every bin: wait for one new bin at each stage

Tosses are independent and uniform. For a fixed bin, let T count tosses through its first hit, including the successful toss. Since $\Pr(T > t) = (1 - 1/b)^t$ for $t \geq 0$, tail summation gives

$$\mathbb{E}[T] = \sum_{t=0}^{\infty} (1 - 1/b)^t = b.$$

Four bins; wait for the first hit on bin 3

1	2	3	4	5	6
2	2	4	1	4	3
miss	miss	miss	miss	miss	hit

T = 6

E[T] = 4

P(T = 6) = $(3/4)^5 \times (1/4) = 243/4096$

Five misses, then one hit; a single waiting time need not equal its mean.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Add the waiting times for successive new bins

For complete collection, after i distinct bins have appeared, new-bin probability is $(b - i)/b$. Conditioned on the currently seen set, the waiting count T_i has expectation $b/(b - i)$. Future tosses remain independent and uniform; elapsed time does not change this value.

Total stopping time is $C = \sum_{i=0}^{b-1} T_i$, so linearity gives

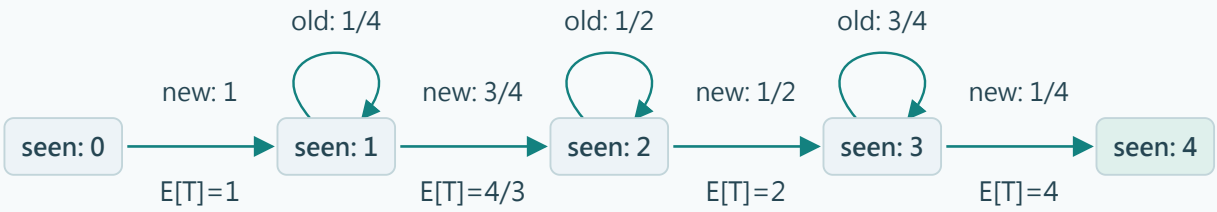
$$\mathbb{E}[C] = \sum_{i=0}^{b-1} \frac{b}{b - i} = bH_b.$$

This is $\Theta(b \ln b)$ for $b \geq 2$; for $b = 1$ it is exactly one toss. Adding expectations does not require a separate proof of independence between stages.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Four bins: repeated hits stay in the same state

A new bin advances; an old bin repeats the state



Tosses 2,2,4,1,4,3: new bins at tosses 1,3,4,6; actual stage lengths 1,2,1,2.

Stage means sum to 1+4/3+2+4=25/3; a particular run may stop at toss 6.

Four bins have stage means $1, 4/3, 2, 4$, summing to $25/3$. The illustrated tosses are $2, 2, 4, 1, 4, 3$, discovering new bins at tosses $1, 3, 4, 6$. Actual stage lengths $1, 2, 1, 2$ sum to six. A run's stopping time need not equal its expectation.

First collision versus complete collection: different stopping times

First bound complete-collection time C. After t tosses, a designated bin is empty with probability $(1 - 1/b)^t \le e^{-t/b}$. Incomplete collection means at least one empty bin, so

$$\Pr(C > t) \le \min\{1, be^{-t/b}\}.$$

For integer $t \ge b(\ln b + c)$ and $c \ge 0$, failure probability is at most e^{-c}. Empty-bin events may be dependent; the union bound is sufficient.

Same tosses; the stopping rule changes the answer

t = 1	t = 2	t = 3	t = 4	t = 5	t = 6
2	2	4	1	4	3
{2}	{2}	{2,4}	{1,2,4}	{1,2,4}	{1,2,3,4}

D = 2: first repeat

C = 6: all four bins seen

Sets show distinct bins seen so far; compare D with C, not two different samples.

First-collision time has square-root expectation

Let D be the toss index of the first repeated bin. Event $D > k$ means the first k tosses are distinct. For $k > b$ its probability is zero; otherwise use the birthday complement. Since $k(k-1) \geq k^2/2$ for $k \geq 2$,

$$\begin{aligned}\mathbb{E}[D] &= \sum_{k \geq 0} \Pr(D > k) \\ &\leq 2 + \sum_{k \geq 2} e^{-k^2/(4b)} \\ &\leq 2 + \int_0^\infty e^{-x^2/(4b)} dx = 2 + \sqrt{\pi b}.\end{aligned}$$

For a lower bound, integer $k \leq \sqrt{b}/2$ has collision probability at most $k(k-1)/(2b) \leq 1/8$. Thus the first $\Theta(\sqrt{b})$ tail terms are at least $7/8$, giving $\mathbb{E}[D] = \Omega(\sqrt{b})$.

Consequently $\mathbb{E}[D] = \Theta(\sqrt{b})$, whereas $\mathbb{E}[C] = \Theta(b \ln b)$. The same balls-and-bins model has different answers for first repetition and complete coverage.

Heads streaks: window count and longest length differ

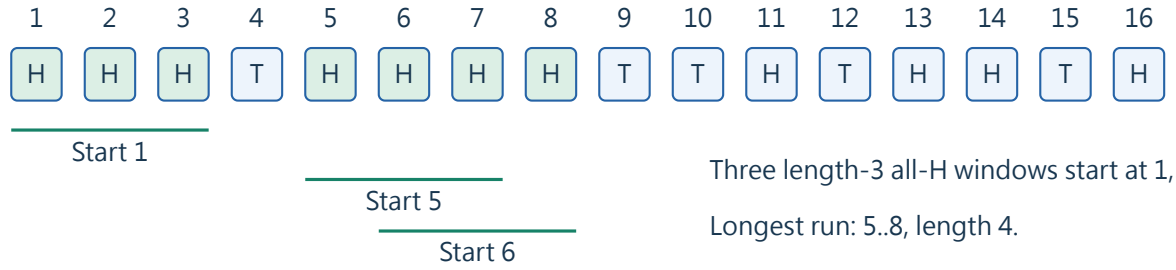
Flip a fair coin independently n times. Let L be the longest consecutive-heads length, with $L = 0$ when no head occurs. For $1 \leq k \leq n$, event $A_{i,k}$ means positions i through $i + k - 1$ are all heads; valid starts satisfy $1 \leq i \leq n - k + 1$. Thus

$$X_k = \sum_{i=1}^{n-k+1} \mathbf{1}_{A_{i,k}}, \qquad \mathbb{E}[X_k] = (n - k + 1)2^{-k}.$$

Overlapping windows need not be independent. For example, $\Pr(A_{1,3} \cap A_{2,3}) = 2^{-4} = 1/16$ because only four heads are needed, whereas the product of marginals is $2^{-6} = 1/64$. Linearity of expectation remains valid; exact union probabilities require accounting for overlaps.

Count windows without confusing them with runs

Four consecutive Hs contain two dependent length-3 windows



Three length-3 all-H windows start at 1,5,6.

Longest run: 5..8, length 4.

Overlapping windows can both occur; starts 5 and 6 share two flips.

The illustrated length-three windows start at $1, 5, 6$, giving $X_3 = 3$. The heads run at positions $5..8$ contributes two of these windows. Its longest length is $L = 4$, distinct from both the window count and the number of heads runs.

Upper bounds: from streak tails to expected length

Event $\{L \geq k\}$ is the union of all all-heads windows of length k. For $1 \leq k \leq n$, the union bound gives

$$\Pr(L \geq k) \leq \min\{1, (n - k + 1)2^{-k}\} \leq \min\{1, n2^{-k}\}.$$

For $n \geq 1$, put $k_0 = \lceil \lg n \rceil$. Integer tail sums and a geometric series yield

$$\begin{aligned} \mathbb{E}[L] &= \sum_{k=1}^{n} \Pr(L \geq k) \\ &\leq k_0 + \sum_{k=k_0+1}^{\infty} n2^{-k} = k_0 + n2^{-k_0} \leq \lceil \lg n \rceil + 1. \end{aligned}$$

Here $\lg$ denotes base-two logarithm. For $k > n$ the event is impossible; for $n = 0$, set $L = 0$ directly without taking a logarithm.

For $r \geq 1$, choosing integer threshold $k = \lceil r \lg n \rceil$ gives $\Pr(L \geq k) \leq n^{1-r}$. At $n = 1000, k = 20$, the sharper bound is $981/2^{20} < 10^{-3}$; at $k = 30$, even $1000/2^{30} < 10^{-6}$ suffices. These are probability guarantees for an unusually long streak in one run.

Lower bound: disjoint blocks supply independence

For sufficiently large n, let $s = \lfloor (\lg n)/2 \rfloor \geq 1$ and $m = \lfloor n/s \rfloor$. Partition the first ms flips into m blocks of length s, discarding the incomplete final block. Each block is all heads with probability $2^{-s} \geq n^{-1/2}$. Disjoint blocks use independent flips, so their success events are mutually independent.

If $L < s$, every block must fail. The converse need not hold: a long run can cross a block boundary. Consequently,

$$\Pr(L < s) \leq (1 - 2^{-s})^m \leq e^{-m2^{-s}} \leq e^{-m/\sqrt{n}} \longrightarrow 0.$$

Since $m = \Theta(n/\lg n)$, we have $m/\sqrt{n} \to \infty$. Nonnegativity then gives

$$\mathbb{E}[L] \geq s \Pr(L \geq s) = \Omega(\lg n).$$

Combining with the upper bound proves $\mathbb{E}[L] = \Theta(\lg n)$. Many overlapping windows in expectation alone would not justify the high-probability existence step; independence of the selected blocks does.

For $n = 16, s = 2, m = 8$, at least one designated block is HH with probability $1 - (3/4)^8 = 58975/65536$. Hence $\mathbb{E}[L] \geq 117950/65536$, a checkable lower bound rather than an exact mean.

Online selection: specify information and decisions

There are $n \geq 2$ candidates with distinct abilities, arriving in a uniformly random permutation. Interviews permit comparisons only with candidates already seen; rejected candidates cannot be recalled, and at most one hire is allowed. The objective is to select the globally best candidate, rather than minimize interview cost.

Choose integer $1 \leq k < n$. Reject the first k candidates while recording their highest score M, then accept the first later score exceeding M. If none does, accept the last candidate.

The second loop need not update M: before the first threshold crossing every observed score is at most M, and a crossing returns immediately. The final fallback accepts the current last candidate, without recalling anyone. Handle $n = 1$ by selecting its sole candidate; define $k = 0$ separately as selecting the first candidate.

Online selection: code and stopping invariant

```
ONLINE-SELECT(n,k):
    require n >= 2 and 1 <= k < n
    M = -infinity
    for i = 1 to k:
        M = max(M, score(i))      // interview, then reject
    for i = k+1 to n:
        if score(i) > M: return i
    return n
```

Before inspecting each later candidate, no earlier candidate has crossed the sample maximum M. A crossing ends the search immediately; if none occurs, the fallback accepts the last candidate.

Success probability: condition on the best arrival

Condition on the globally best candidate arriving at position i. If $i \le k$, rejection during observation makes success impossible. If $i > k$, success occurs exactly when the best of the first $i-1$ candidates lies among the first k: otherwise a later candidate would already have crossed M, while if it does, candidate i is the first crossing.

The best position is uniform, with probability $1/n$. Conditional on that position, the first $i-1$ candidates still have a uniform relative order. Their maximum lies among the first k with conditional probability $k/(i-1)$. Summing disjoint position events gives

$$P_k = \sum_{i=k+1}^{n} \frac{1}{n}\frac{k}{i-1} = \frac{k}{n}\sum_{j=k}^{n-1}\frac{1}{j}.$$

This uses conditional probabilities, without assuming independence of the two maximum-position events.

Define boundaries separately: for $n = 1$ success is certain; for $k = 0$ choosing the first candidate succeeds with probability $1/n$. The harmonic sum formula applies only to $k \ge 1$.

Optimal threshold: finite differences before limits

Put $q_k = \sum_{j=k}^{n-1} 1/j$, with empty sum $q_n = 0$. Since $q_k = 1/k + q_{k+1}$, for $1 \le k \le n-2$,

$$P_{k+1} - P_k = \frac{(k+1)q_{k+1} - kq_k}{n} = \frac{q_{k+1} - 1}{n}.$$

The strictly decreasing sequence q_k makes success unimodal. A threshold satisfying $q_k \ge 1 \ge q_{k+1}$ maximizes it; equality can produce adjacent tied optima. For $n = 2$, the sole positive threshold $k = 1$ ties $k = 0$ at success $1/2$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Why the optimal success rate tends to 1/e

Integral bounds for decreasing $1/x$ yield

$$\ln(n/k) \le q_k \le \ln(n/k) + 1/k.$$

Thus $f(k/n) \le P_k \le f(k/n) + 1/n$, where $f(x) = x\ln(1/x)$. Derivatives $f'(x) = \ln(1/x) - 1$ and $f''(x) = -1/x < 0$ give maximum $1/e$ at $x = 1/e$.

Choose the nearest legal integer to n/e, making k/n differ from $1/e$ by $O(1/n)$. Continuous differentiability gives $f(k/n) = 1/e - O(1/n)$, while every threshold satisfies $P_k \le 1/e + 1/n$. Optimal success therefore converges to $1/e$. At finite n, use the exact difference rather than treating n/e as an exact integer solution.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

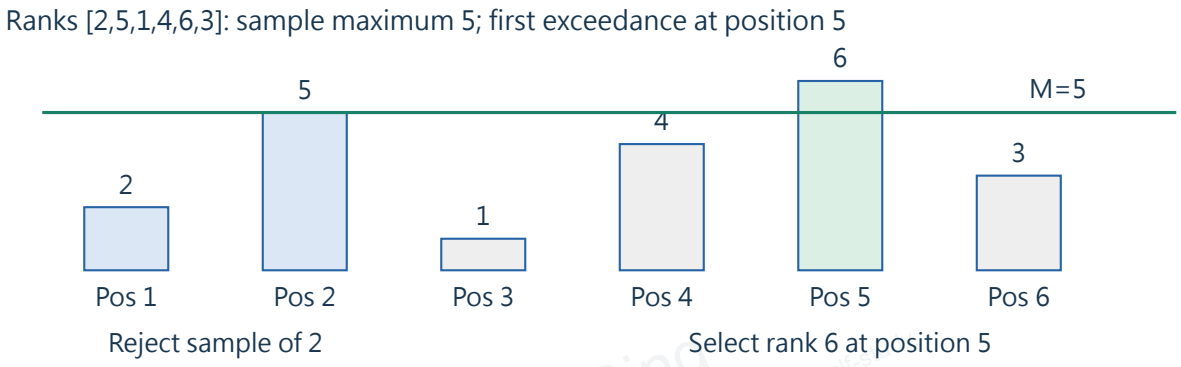
Six candidates: verify thresholds and individual decisions

For $n = 6$, substitute each threshold into $P_k = (k/6)q_k$:

k	q_k	Exact success probability P_k
1	$137/60$	$137/360$
2	$77/60$	$77/180$
3	$47/60$	$47/120$
4	$9/20$	$3/10$
5	$1/5$	$1/6$

Since $q_2 > 1 > q_3$, the best positive threshold is $k = 2$; threshold 0 succeeds with probability only $1/6$.

Follow the threshold through every candidate



For ranks $[2, 5, 1, 4, 6, 3]$, reject the first two and set $M = 5$. Reject candidate three because $1 \le 5$, and four because $4 \le 5$. Candidate five has $6 > 5$ and is accepted immediately; candidate six need not be interviewed. For failure example $[5, 6, 1, 2, 3, 4]$, the best candidate is rejected during observation, nobody later exceeds 6, and the fallback accepts rank 4.

Random recursive splitting: conditioning and leaf conservation

Consider a separate cost model. A call of size $n \geq 2$ prints n stars, chooses p uniformly from $\{1, \ldots, n-1\}$, and processes sizes $p, n-p$. A size-one call prints one exclamation mark and stops. Every draw is uniform conditional on prior history.

Let L_n, S_n denote expected exclamation marks and stars, with $L_1 = 1, S_1 = 0$. Condition on p, add the two expected costs, then average over p. Symmetry under $j \leftrightarrow n-j$ gives

$$L_n = \frac{2}{n-1} \sum_{j=1}^{n-1} L_j, \qquad S_n = n + \frac{2}{n-1} \sum_{j=1}^{n-1} S_j.$$

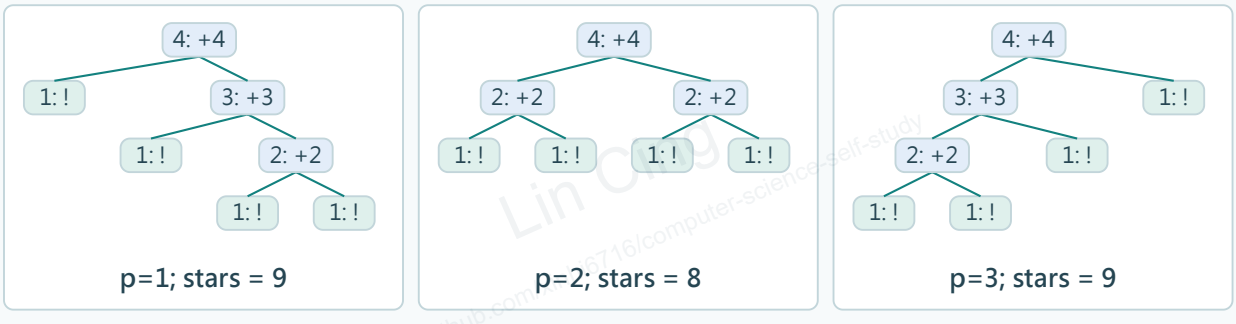
Linearity needs no independence between the two subtree costs. Every branch strictly decreases its positive integer size, ensuring termination.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Four items: the same leaf count, different work

Splitting conserves total size and all final leaves have size one, so **every execution has exactly n leaves**, not merely n in expectation. Star count varies. At $n = 4$, a balanced split costs $4 + 2 + 2 = 8$. A $1 + 3$ split costs $4 + (3 + 2) = 9$. Averaging the three equally likely root splits gives $S_4 = (9 + 8 + 9)/3 = 26/3$.

Three equally likely root splits; four leaves in every tree



Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Solving split cost: subtract adjacent sizes

Strong induction verifies the leaf mean: if $L_j = j$ for $j < n$, then $L_n = \frac{2}{n-1}\sum_{j<n} j = n$.

For stars, clear the denominators:

$$(n-1)S_n = n(n-1) + 2\sum_{j<n} S_j,$$

$$nS_{n+1} = n(n+1) + 2\sum_{j<n} S_j + 2S_n.$$

Subtract to obtain $nS_{n+1} - (n-1)S_n = 2n + 2S_n$, hence $S_{n+1} = 2 + \frac{n+1}{n}S_n$. The subtraction applies for $n \geq 2$; at $n = 1$, check the difference directly from $S_1 = 0, S_2 = 2$.

With $Q_n = S_n/n$, we have $Q_{n+1} = Q_n + 2/(n+1)$. Sum from $Q_1 = 0$ to get

$$Q_n = 2(H_n - 1), \qquad S_n = 2n(H_n - 1) = \Theta(n \ln n).$$

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

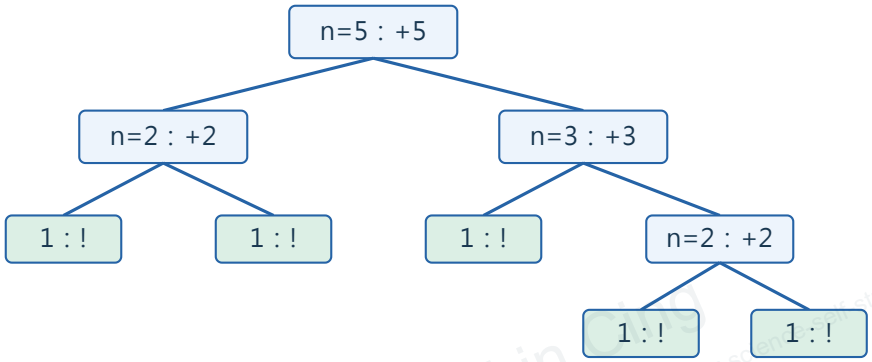
Verify the closed form on small inputs

n	Leaves in every execution	$\mathbb{E}[\text{stars}]$
2	2	2
3	3	5
4	4	$26/3$
5	5	$77/6$
6	6	$87/5$

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Expected cost and worst execution are different

Five unit leaves: leaf count is conserved; star cost varies



Leaves here=5

Stars here=12

E[stars]=77/6

+number: stars at this call

!: one exclamation mark

Shown: p=2, then 3=1+2. All splits give 5 leaves; total stars depend on splits.

The worst execution splits off one each time, printing $2 + 3 + \cdots + n = n(n+1)/2 - 1$ stars. The expected linearithmic bound is not a bound on every execution.

References and attribution

Reference textbook: Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition, Chapters 5–9. Supplementary notes: MIT 6.046J/18.401J, Fall 2005. Lecture 4: Quicksort (Charles E. Leiserson, 2005-09-21); Lecture 5: Sorting Lower Bounds / Linear-Time Sorting (Erik Demaine, 2005-09-26); Lecture 6: Order Statistics (Erik Demaine, 2005-09-28).

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Data Structures and Amortized Analysis

A data structure turns abstract operations into maintained states. First specify legal states and operation guarantees; then ask how much work an entire operation sequence requires.

Read **representations and contracts → amortized analysis → hashing → search trees and augmentation → B-trees → Fibonacci heaps → integer universes**.

Prerequisites are specifications and invariants from Chapter 1, recurrence analysis from Chapter 2, and probability and heaps from Chapter 3. Two stacks and a resizable array introduce the cost tools needed later.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

The same operation name can hide different contracts

An **abstract data type (ADT)** specifies observable behavior; arrays, links, and trees implement it. Complexity depends on the contract.

Operation	Question to settle first
DELETE	Is the argument a key, an array index, or an existing node handle?
SEARCH	Are duplicate keys allowed? Which record is returned?
POP / DEQUEUE	Is an empty call rejected, given a special result, or forbidden?
Pointer update	Must node identity and external handles remain valid?

Deleting a known doubly linked-list node can take constant time. Finding a node by key and then deleting it also incurs search cost. Every later bound uses its own argument and representation contract.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Circular queue: physical position is not logical order

Fix capacity $C \geq 1$ and use array $A[0..C-1]$, front index h, and size s. A valid state satisfies $0 \leq h < C$ and $0 \leq s \leq C$. Logical item j is

$$Q[j] = A[(h + j) \bmod C], \qquad 0 \leq j < s.$$

Empty means $s = 0$; full means $s = C$. Do not infer occupancy from equal indices. The next insertion position is $(h + s) \bmod C$, writable only when not full. All physical slots are usable.

Old values left in the array are not necessarily queue members. Clearing a removed slot may release resources, but h, s determine membership. Fixed-capacity ENQUEUE and DEQUEUE take worst-case $O(1)$, excluding allocation and nonconstant element-copy costs.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Check boundaries before changing queue state

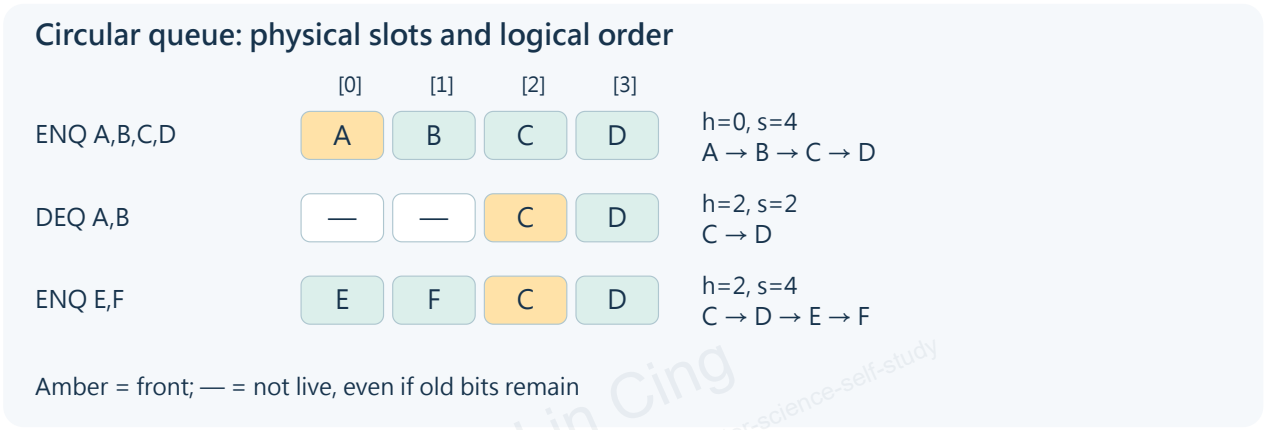
```
ENQUEUE(x):
    if s == C: return overflow
    A[(h+s) mod C] = x
    s = s + 1

DEQUEUE():
    if s == 0: return underflow
    x = A[h]
    h = (h+1) mod C
    s = s - 1
    return x
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Assume successful element reads/writes and sequential execution. A rejected call changes neither metadata nor live records. Distinguish error status from a returned data value. This is not a concurrent lock-free queue; sharing it requires synchronization.

After wrapping, dequeue still returns C, D, E, F



Enqueue A–D, dequeue A and B, then enqueue E and F. Storage is [E,F,C,D], but $h = 2, s = 4$: read $2 \to 3 \to 0 \to 1$. Reject G; the next-insertion index equals h because the queue is full.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Why circular indexing preserves FIFO

Before insertion, $s < C$. Position $(h + s) \bmod C$ differs from every live position $(h + j) \bmod C$, $0 \le j < s$: equality would imply $C \mid (s - j)$, contradicting $1 \le s - j < C$. Thus insertion overwrites no live record and appends exactly at the logical rear.

Dequeue returns $A[h] = Q[0]$. With $h' = (h + 1) \bmod C$ and $s' = s - 1$, each $0 \le j < s'$ satisfies

$$A[(h' + j) \bmod C] = A[(h + j + 1) \bmod C] = Q[j + 1].$$

The remaining sequence is precisely the old sequence without its first item. The empty initial state is valid; successful calls preserve the invariant and failed calls preserve state. Induction covers every legal call sequence.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Another representation: reserve one slot instead of size

CLRS § 10.1 uses head for the first item and tail for the next insertion slot, reserving one slot. In zero-based notation, empty is $h = t$ and full is $(t + 1) \bmod C = h$, so usable capacity is $C - 1$.

Another convention places f immediately before the first item and r at the last item. Empty is $f = r$ and full is $(r + 1) \bmod C = f$. Convert by $h = (f + 1) \bmod C$ and $t = (r + 1) \bmod C$. Renaming variables alone does not preserve read/write order.

This chapter uses h, s and all C slots. The derived next-insertion index is $t = (h + s) \bmod C$; $t = h$ can mean empty or full, distinguished by s. A reserved slot, a count, and a full flag are different designs with different tests.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Singly linked insertion: preserve the old successor first

Given a valid node p, insert a detached new node x after it. Node identity is fixed and keys may repeat. Node x must not already belong to the list.

```
q = p.next
x.next = q
p.next = x
```

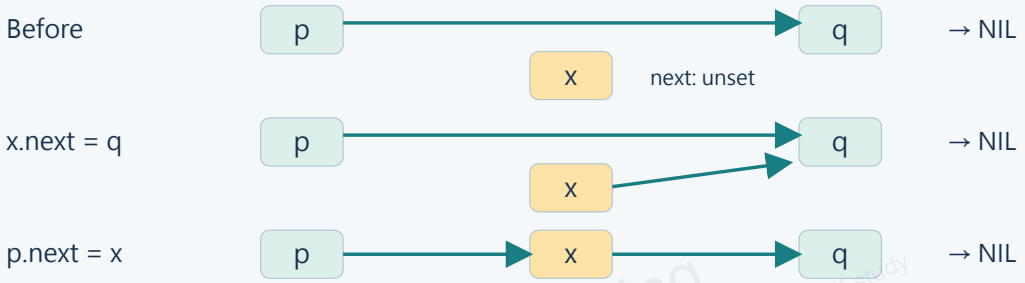
The path $p \to q$ becomes $p \to x \to q$; all other next links stay unchanged. The same code handles q=NIL. If a tail pointer is maintained and p was the tail, update tail=x.

Writing `p.next=x` before `x.next=p.next` creates a self-loop at x and can lose reachability to q. The $O(1)$ bound covers rewiring with known p, excluding search, allocation, and nonconstant data-copy costs.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Three states: connecting a new node to the old path

Initialize the new node before changing the old path



Connect x.next to q before connecting p.next to x. In the intermediate state, x is not yet a member reachable from head. Node labels denote identity rather than key values.

Singly linked deletion: a node is not its predecessor

To delete the node after p, save `x=p.next`; reject x=NIL. Otherwise save `q=x.next`, then set `p.next=q`. If x was the tail, set tail=p. Reclaim x last. Given x alone without its predecessor, deleting a tail generally requires searching from head, taking worst-case $\Theta(n)$.

Copying the successor's data into x and deleting that successor changes which node owns which record and fails at the tail. It does not implement deletion with the node-identity contract used here.

In-place reversal preserves access to the untouched suffix

Reverse an acyclic, NIL-terminated singly linked list:

```
prev = NIL; cur = head
while cur != NIL:
    nxt = cur.next
    cur.next = prev
    prev = cur
    cur = nxt
head = prev
```

If a tail is stored, save old_head first and set tail=old_head at the end. During the loop, prev is the reversed processed prefix and cur begins the untouched suffix. Saving nxt preserves access after rewiring. Each iteration consumes one node: $\Theta(n)$ time and $O(1)$ extra space.

Circular doubly linked list: a sentinel joins the endpoints

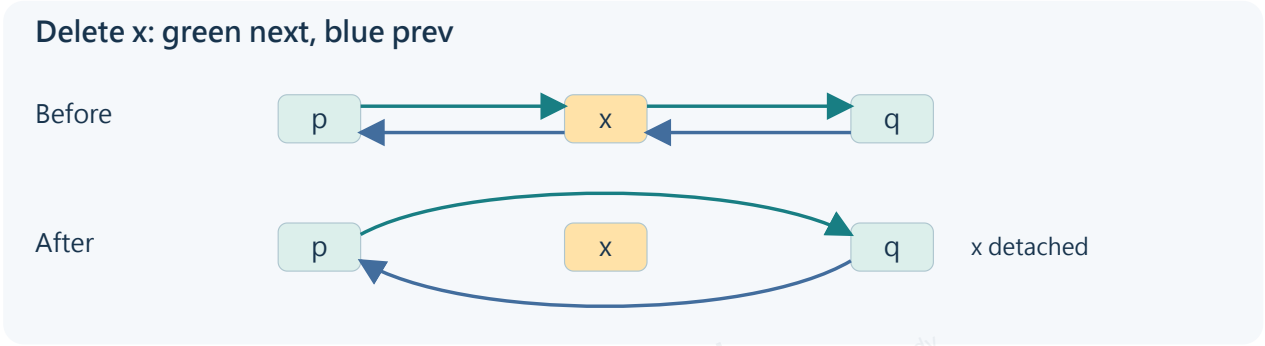
Allocate a permanent dummy node z, the sentinel, excluded from the data count. Empty means `z.next=z.prev=z`; otherwise z.next is the first item and z.prev the last.

For every node v in the ring, including z, maintain `v.next.prev=v` and `v.prev.next=v`. Require a valid member handle and forbid deleting z. Validating an arbitrary foreign handle is not included in constant-time rewiring of a known member.

```
DELETE(x):                // x != z, valid member
    p = x.prev; q = x.next
    p.next = q
    q.prev = p
```

Either neighbor may be z, so the code handles first, last, and sole data nodes. Reclaim x only after rewiring; external handles to x then become invalid. These are sequential operations, not intermediate states safe for unsynchronized concurrent readers.

Doubly linked deletion changes two neighbor links



Replace p.next=x and q.prev=x with p.next=q and q.prev=p. Other members keep their adjacencies. Old fields inside x do not make it a member after removal.

For a sole data item, p=q=z. The two writes restore z.next=z and z.prev=z, exactly the empty state. Two pointer writes do not mean deletion by an arbitrary key takes two steps: finding that key can still scan the entire list.

Doubly linked insertion must establish four links

Insert a detached x after a known p; p may be z. First save q=p.next:

```
x.prev = p
x.next = q
q.prev = x
p.next = x
```

The result is p↔x↔q, with all other links unchanged. For an empty list p=q=z, both z.next and z.prev become x. If p was last, q=z and z.prev automatically becomes the new last node.

Each new adjacency has its matching backlink, restoring the invariant. This order establishes correctness after a sequential call, not concurrent publication safety. A sentinel simplifies boundaries at the cost of one extra node per list; it does not improve search asymptotically.

Array indices as node handles: location and order differ

Parallel arrays key, next, and prev represent nodes without native pointers. Three fields at the same index belong to one object. Reserve index 0 for NIL and use indices 1–5 for objects.

Index	State	key	next	prev
1	free	—	4	—
2	live	B	3	5
3	live	C	0	2
4	free	—	0	—
5	live	A	2	0

With head=5, follow $5 \to 2 \to 3 \to 0$, giving A→B→C. With free=1, unused slots follow $1 \to 4 \to 0$. Adjacent physical indices need not be logical neighbors.

Allocation and reclamation keep live and free lists disjoint

Pool slots have three disjoint ownership classes: live, free, and caller-owned. ALLOCATE transfers a free slot to the caller; initialization and linking make it live. The free list uses only next and behaves as a stack.

```
ALLOCATE():
    if free == 0: return exhausted
    x = free; free = next[x]
    return x                    // initialize before linking

FREE(x):                        // detached, not already free
    next[x] = free
    free = x
```

Both operations change constantly many fields and take $O(1)$ in a fixed pool. Growing the pool and allocating variable-length records are outside this bound.

Object-pool trace: allocation through reclamation

Use the same five slots; head remains 5:

Completed operation	Live chain (NIL=0)	Free chain	Caller-owned
Initial	5(A)→2(B)→3(C)→0	1→4→0	none
ALLOCATE returns 1	5→2→3→0	4→0	1
Append D after C	5→2→3→1(D)→0	4→0	none
Unlink and FREE(2)	5→3→1→0	2→4→0	none

Append: key[1]=D, prev[1]=3, next[1]=0, next[3]=1. Remove B: set next[5]=3 and prev[3]=5, then next[2]=4 and free=2.

Double-free corrupts the free list; a stale handle can refer to an unrelated reused object. An in-range index does not establish a valid lifetime.

How two stacks preserve FIFO

Use S for incoming records and T for outgoing records. Read each sequence in the stated direction:

$$Q = \mathrm{reverse}(T_{\mathrm{bottom}\to\mathrm{top}}) \parallel S_{\mathrm{bottom}\to\mathrm{top}}.$$

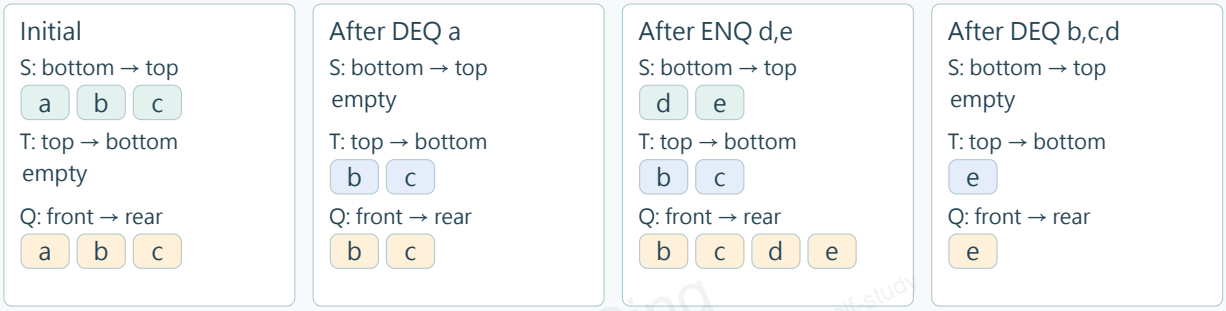
Here $\parallel$ denotes concatenation. **The top of T is the queue front.** Transfer all of S to T only when T is empty.

ENQUEUE pushes onto S, appending to Q. If T is nonempty, DEQUEUE pops its top, removing the first record of Q. If T is empty, transferring reverses S, making its oldest record the top of T while preserving queue order.

If both are empty, return underflow without changing state. Transferring before every dequeue would put new records ahead of older records still waiting in T.

Follow the order before counting the expensive transfer

One queue, two stacks: older records leave first



Read S bottom to top and T top to bottom. Enqueuing d, e leaves b, c waiting in T. Transfer S only after both b, c have left; then dequeue d.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

One expensive call does not make every call expensive

Count one stack PUSH or POP as one unit, temporarily omitting fixed queue-call overhead. ENQUEUE costs 1. A DEQUEUE transferring k records costs $2k + 1$: pop and push each transferred record, then pop T.

Starting empty, each record can be **pushed onto S → popped from S → pushed onto T → popped from T** with each of these four primitives occurring at most once for that record. Thus e enqueues and d successful dequeues cost at most $3e + d$.

Any legal sequence of m operations has $e + d \leq m$, giving at most $3m$ primitive operations. Restoring fixed per-call overhead preserves $O(m)$. No distribution over sequences is assumed.

A single DEQUEUE can still cost $\Theta(n)$. A sequence guarantee is not a bound on each call's latency.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Potential: attach deferred work to the state

Operation i changes D_{i-1} into D_i at actual cost c_i. Choose a state function Φ and define

$$\hat{c}_i = c_i + \Phi(D_i) - \Phi(D_{i-1}).$$

A positive $\Delta\Phi$ stores credit for future work; a negative change releases previously stored credit. Amortized cost is not a second measurement of execution time.

Summing a prefix of m operations cancels intermediate potentials:

$$\sum_{i=1}^{m} c_i = \sum_{i=1}^{m} \hat{c}_i + \Phi(D_0) - \Phi(D_m).$$

If all reachable states satisfy $\Phi(D_i) \geq \Phi(D_0)$, total amortized cost bounds total actual cost. A common sufficient condition is $\Phi(D_0) = 0$ and $\Phi \geq 0$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Initial state and negative amortized charges matter

If only $\Phi \geq 0$ is known, without $\Phi(D_0) = 0$, the correct conclusion is

$$\sum c_i \leq \sum \hat{c}_i + \Phi(D_0).$$

A queue starting with n records in S may transfer all of them on its first dequeue. Either charge for constructing that initial state or retain an $O(n)$ initial term.

An individual $\hat{c}_i$ may be negative. The telescoping identity remains valid under the endpoint condition: the operation releases more credit than the actual work it performs.

An **amortized** bound controls total cost over legal sequences. An **expected** bound needs a probability model. An expected amortized bound can combine both; it must state both quantifiers.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Queue potential counts records not yet transferred

Let $\Phi(S, T) = 2|S|$. Both stacks start empty, so $\Phi_0 = 0$.

Operation	Actual c	$\Delta\Phi$	$\hat{c}$
ENQUEUE	1	2	3
DEQUEUE with nonempty T	1	0	1
Transfer $k \geq 1$ records, then dequeue	$2k + 1$	$-2k$	1
Report underflow on empty queue	0	0	0

The last row counts stack primitives only. Including queue-call overhead adds a fixed constant to each row. Records in T have already paid their transfer cost.

Every prefix has total cost at most $3e + d$, agreeing with the aggregate argument that each record incurs at most four stack primitives.

A complete ledger for interleaved operations

Both stacks run **bottom to top**; $\Phi = 2|S|$. Both totals are 19 because endpoint potentials are zero.

Operation	S	T	c	Φ	$\hat{c}$
ENQ a	a	empty	1	2	3
ENQ b	a,b	empty	1	4	3
ENQ c	a,b,c	empty	1	6	3
DEQ → a	empty	c,b	7	0	1
ENQ d	d	c,b	1	2	3
ENQ e	d,e	c,b	1	4	3
DEQ → b	d,e	c	1	4	1
DEQ → c	d,e	empty	1	4	1
DEQ → d	empty	e	5	0	1

Accounting: every prefix must pay its own bill

Operation i costs c_i but is charged a_i. The bank balance is

$$B_t = B_0 + \sum_{i=1}^{t} (a_i - c_i).$$

Start empty with $B_0 = 0$. Proving $B_t \geq 0$ for every prefix of every legal sequence gives $\sum_{i=1}^{t} c_i \leq \sum_{i=1}^{t} a_i$.

Borrowing against future operations is insufficient. The caller may stop while the bank is overdrawn. An initial credit $B_0 > 0$ must appear in the total-cost bound; it is not free work.

Attach credit to objects to show that it cannot be spent twice. If total credit depends only on the current state, setting $\Phi(D) = B(D)$ recovers the potential method.

MULTIPOP: who pays for a batch of removals?

An initially empty stack supports PUSH, POP, and MULTIPOP(k), with $k \geq 0$. The last operation removes at most k items and stops when empty. Each successful push or pop costs one unit.

Charge PUSH 2: pay 1 now and leave 1 on the new item. Charge POP and MULTIPOP 0: removed items pay with their stored credit. **Every item still on the stack carries exactly one unit.** Induction proves the bank never goes negative.

Among m calls, let p be the number of pushes and q the number of successful removals. Since $q \leq p$, actual cost is $p + q \leq 2p \leq 2m$. Pushing the same object again is a new insertion requiring new payment.

This model counts successful item operations. Adding one unit of checking/dispatch per API call changes the bound to $m + p + q \leq 3m$; an empty MULTIPOP does not literally take zero time.

Binary counter: fix width and overflow semantics

Use a fixed width $k \geq 1$. Bit $A[0]$ is least significant; the value is $\sum_{j=0}^{k-1} A[j]2^j$. Start at zero. INCREMENT adds one modulo 2^k, so all ones wraps to all zeros.

```
INCREMENT(A, k):
    j = 0
    while j < k and A[j] == 1:
        A[j] = 0
        j = j + 1
    if j < k:
        A[j] = 1
```

Cost c_i counts flipped bits. Tests and loop control take $\Theta(1 + c_i)$ actual time, so a constant overhead per call preserves the asymptotic sequence bound. Arbitrary-precision integers that increase their width require a separate representation contract.

A carry does not flip every bit on every call

Carry: trailing ones and the next zero

Amber = flipped on this call; most significant bit first

	A[3]	A[2]	A[1]	A[0]	
0110 → 0111	0	1	1	1	c = 1 ΔΦ = +1 ĉ = 2
0111 → 1000	1	0	0	0	c = 4 ΔΦ = -2 ĉ = 2
1000 → 1001	1	0	0	1	c = 1 ΔΦ = +1 ĉ = 2

The transition 0111→1000 flips four bits but also spends stored potential. Next, distinguish an ordinary carry from overflow.

Analyze ordinary carry and overflow separately

For a nonoverflowing carry through t trailing ones, clear those ones and set the next zero: $c = t + 1$. Let Φ be the number of set bits. Then $\Delta\Phi = 1 - t$ and $\hat{c} = 2$.

Overflow is a separate branch: $t = k$, every bit clears and no bit is set. Thus $c = k$, $\Delta\Phi = -k$, and $\hat{c} = 0$. The formula $t + 1$ does not apply to this branch.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

One increment sequence, three consistent analyses

Starting at zero, perform $m \geq 1$ increments. Bit j flips once every 2^j calls, including wraps. The exact total is

$$T(m, k) = \sum_{j=0}^{k-1} \left\lfloor \frac{m}{2^j} \right\rfloor < 2m.$$

Aggregate analysis sums flips per position. **Accounting** charges 2 per call: setting a zero pays 1 now and leaves 1 for its later clearing. Every set bit owns its credit. **Potential analysis** counts those set bits, charging 2 for an ordinary increment and 0 for a wrap.

For $k = 4, m = 8$, positions flip $8, 4, 2, 1$ times, totaling 15 rather than $8 \times 4 = 32$. The proof covers every m; the numerical example checks the count.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Complete ledger: from 0000 through wraparound

Four bits, initially $\Phi_0 = 0$. Bit strings are most significant first. The first eight charges total 16; subtract final potential 1 to get actual cost 15.

Call	Result	c_i	Φ_i	Cumulative cost
1	0001	1	1	1
2	0010	2	1	3
3	0011	1	2	4
4	0100	3	1	7
5	0101	1	2	8
6	0110	2	2	10
7	0111	1	3	11
8	1000	4	1	15

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Complete ledger: paying for the highest bit

Continue the same sequence. Calls 9–15 each charge 2; call 16 wraps and charges 0. Total actual and amortized costs both equal 30, with no credit remaining.

Call	Result	c_i	Φ_i	Cumulative cost
9	1001	1	2	16
10	1010	2	2	18
11	1011	1	3	19
12	1100	3	2	22
13	1101	1	3	23
14	1110	2	3	25
15	1111	1	4	26
16	0000	4	0	30

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Which changes invalidate the constant amortized bound?

A nonzero initial state: starting with k ones makes the first call cost k. The same potential only yields $\sum c_i \leq 2m + \Phi_0 \leq 2m + k$. Initial credit cannot be discarded.

Adding DECREMENT: for $k \geq 2$, increment from zero to $2^{k-1} - 1$, then alternate INCREMENT and DECREMENT. The states alternate between $0\underbrace{11\cdots1}_{k-1}$ and $1\underbrace{00\cdots0}_{k-1}$. Every alternating call flips all k bits.

Let preparation take $r = 2^{k-1} - 1$ calls, followed by $2r$ alternating calls. There are $m = 3r$ calls, and the alternating portion alone costs $2rk = \Omega(mk)$. The upper bound is $O(mk)$ since each call flips at most k bits, giving $\Theta(mk)$ on this family.

An expensive carry can now be immediately undone by a borrow. The previous credit attached to set bits is insufficient. Changing the supported operations requires a new cost proof.

Resizable arrays: specify what moving costs

State (n, M) records the number of live elements and capacity. Start at $(0, 0)$; allocate one slot on the first insertion, and double a full table before inserting.

Count **one new-record write plus the number of old records moved**. Temporarily omit fixed allocation, release, and test overhead. Initializing every newly allocated slot adds work and changes exact constants.

Insertion	Result (n, M)	Actual cost	Cumulative cost
1	(1,1)	1	1
2	(2,2)	2	3
3	(3,4)	3	6
4	(4,4)	1	7
5	(5,8)	5	12

Five insertions move $1 + 2 + 4 = 7$ old records. The moved-record count need not be smaller than the insertion count; the full geometric sum proves the linear total.

For growth alone, aggregate and potential bounds agree

After $m \geq 2$ insertions, let L be the largest power of two below m. Moves total $1 + 2 + \cdots + L = 2L - 1$, giving $T(m) = m + 2L - 1 < 3m$. Costs for $m = 0, 1$ are 0 and 1.

Choose $\Phi(0, 0) = 0$ and $\Phi(n, M) = 2n - M$ for nonempty tables. Reachable insertion-only states satisfy $M/2 < n \leq M$, so potential is nonnegative.

Branch	c	$\Delta\Phi$	$\hat{c}$
First insertion $(0, 0) \to (1, 1)$	1	1	2
Insert without moving	1	2	3
Full $(M, M) \to (M + 1, 2M)$	$M + 1$	$2 - M$	3

Hence $T(m) \leq 3m$. The first insertion has amortized cost 2; a formula involving $\log 0$ cannot be applied to the initial state.

Shrinking too eagerly repeatedly moves the same records

Consider the bad rule: double when full, and halve as soon as deletion leaves the table half full.

Let $k \geq 2$ be a power of two. Start full at (k, k). An insertion grows to $(k + 1, 2k)$ and moves k records. Deleting that record shrinks back to (k, k) and moves k again. Each repeated insert/delete pair moves $2k$ records and costs exactly $2k + 2$.

Building the initial state costs $O(k)$. Another $2r$ alternating operations cost $\Omega(kr)$. Taking $r = k$ gives $O(k)$ operations but $\Omega(k^2)$ total work.

Separate the thresholds: **grow only when full; shrink only when deletion leaves at most one-quarter occupancy**. Shrinking restores half occupancy, leaving room to accumulate credit before the opposite resize.

Resize contract: thresholds, empty tables, and record order

This example supports **APPEND and DELETE-LAST**. Deleting from the middle may shift a suffix; that cost is separate.

- The first APPEND allocates one slot. A full table doubles before writing the new record.
- DELETE-LAST removes the last record. If $M \geq 2$ and the resulting $n \leq M/4$, halve capacity and move the n survivors; otherwise retain capacity.
- Capacities are powers of two. Once allocated, minimum retained capacity is 1. An empty delete reports underflow without changing state.
- Each successful operation costs $1+$ records moved. Moving preserves survivor order but can invalidate external raw pointers.

Threshold convention: this example uses $n \leq M/4$. Textbook § 17.4 uses $n < M/4$, delaying contraction by one deletion when $M \geq 4$. Both allow constant amortized bounds, but their exact ledgers differ.

At most one resize occurs per operation. For $M \geq 4$, a shrink is triggered at exactly $n = M/4$: the prior state was above the threshold and one operation removes one record.

Follow the same records through growth and contraction

Separate resize thresholds leave time to save credit

Full	n=4, M=4	a b c d	cost —
APPEND e	n=5, M=8	a b c d e · · ·	cost 5 = 1 + 4
DELETE e	n=4, M=8	a b c d · · · ·	cost 1
DELETE d	n=3, M=8	a b c · · · · ·	cost 1
DELETE c	n=2, M=4	a b · ·	cost 3 = 1 + 2

Half full: retain capacity. Quarter full: halve capacity.

After growing to eight slots, retain capacity when four records remain. Shrink to four slots only when two remain. This separation between thresholds is **hysteresis**: it prevents immediate resize reversals.

Dense and sparse states store credit for different resizes

For $M \geq 2$, define

$$\Phi(n, M) = \begin{cases} 2n - M, & n \geq M/2, \\ M/2 - n, & n < M/2. \end{cases}$$

Also set $\Phi(0, 0) = \Phi(0, 1) = 0$ and $\Phi(1, 1) = 1$. Both branches equal zero at half occupancy; every reachable state has nonnegative potential.

Dense states accumulate credit during insertions to pay for growth. Sparse states accumulate credit during deletions to pay for contraction. A state uses exactly one branch, so credit is not counted twice.

The next page checks each branch. Empty-table cases at capacities 1 and 2 are separate because $M/4$ need not be an attainable integer count.

The general-capacity branches all have constant amortized bounds

Here M is capacity before the operation. Substitute into both branches when a nonresizing operation crosses half occupancy.

Case	c	$\Delta\Phi$	$\hat{c}$
Insert without growth	1	-1 or 2	0 or 3
Grow a full table	$M + 1$	$2 - M$	3
Delete without shrinking	1	-2 or 1	-1 or 2
Shrink after reaching $M/4$, $M \geq 4$	$1 + M/4$	$1 - M/4$	2

For the last row, before deletion $n = M/4 + 1$ and potential is $M/4 - 1$. Afterwards capacity is $M/2$ and count is $M/4$: exactly half full, with potential zero.

Example: $(3, 8) \to (2, 4)$ moves two records, so $c = 3$. Potential falls from 1 to 0, giving $\hat{c} = 2$.

Minimum-capacity cases and the total-cost conclusion

Check the remaining small-capacity boundaries separately:

State change	c	$\Delta\Phi$	$\hat{c}$
First $(0,0) \to (1,1)$	1	1	2
Reuse $(0,1) \to (1,1)$	1	1	2
Delete last $(1,1) \to (0,1)$	1	-1	0
Smallest shrink $(1,2) \to (0,1)$	1	0	1

Every successful operation has $\hat{c} \leq 3$. Initial potential is zero and final potential is nonnegative, so any m successful APPEND / DELETE-LAST operations cost at most $3m$ in this model.

Charging one unit for an underflow test adds an operation with unchanged potential and amortized cost 1. The bound does not cover arbitrary-position deletion, arbitrary shrinking policies, or unmodeled record-moving costs.

Dictionary: decide what inserting an existing key means

Maintain a key→record mapping with at most one record per key. INSERT updates the value of an existing key. DELETE takes a key and reports absence if missing. Equal hash values do not imply equal keys.

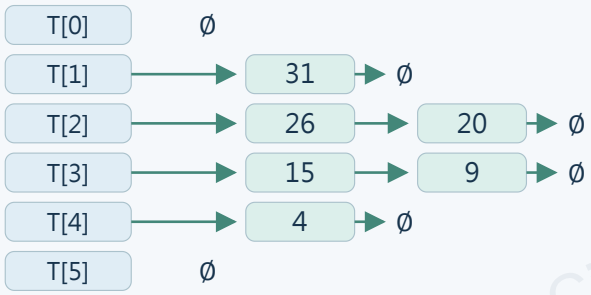
For universe $\{0, \ldots, M-1\}$, direct addressing uses the key as an index. Operations take worst-case $O(1)$, with $\Theta(M)$ space and ordinary full-table initialization. Hashing becomes useful when only $n \ll M$ records are present.

Use $h : U \to \{0, \ldots, m-1\}$. If $|U| > m$, every fixed h has collisions by the pigeonhole principle, so retain full keys for verification. Bounds assume keys, hashing, and equality take constantly many word operations. Long strings and arbitrary-precision integers require their own reading and arithmetic costs.

Chaining keeps colliding records in a list

m = 6; h(k) = k mod 6

Insert at heads: 4, 9, 15, 20, 26, 31



Find 20

26 → 20 ✓

Find 14

26 → 20 → ø

Same bucket: compare full keys

With $m = 6, h(k) = k \bmod 6$, insert 4,9,15,20,26,31 at chain heads. Slot 2 contains 26→20 and slot 3 contains 15→9. Searching for 20 checks two records; absent key 14 also checks both records in slot 2.

Constant-time head insertion does not cover all of INSERT

Let $\alpha = n/m$; chaining allows $\alpha > 1$. Search scans the designated chain until finding the key or reaching its end. Worst-case time is $\Theta(n)$.

Head insertion rewires links in $O(1)$ when the key is known to be new. Our update-on-existing-key INSERT must first search for duplicates. DELETE(key) also pays for search; only deletion through a valid doubly linked node handle supports direct $O(1)$ unlinking.

CLRS § 11.2 gives constant-time insertion and deletion under these different input contracts: insertion assumes a new object and deletion receives the object itself. The bounds are compatible once the contracts are distinguished.

Unsuccessful search: add the cost of each collision

First assume simple uniform hashing: fixed distinct keys have independent uniform bucket locations. For a fixed absent query q, define $I_x = \mathbf{1}[h(x) = h(q)]$. Then

$$L_q = \sum_{x \in K} I_x, \qquad E[L_q] = \sum_{x \in K} \frac{1}{m} = \frac{n}{m} = \alpha.$$

Search checks every record in that chain. Adding constant hashing and indexing work gives expected time $\Theta(1 + \alpha)$. At $\alpha = 0$, inspecting an empty bucket still costs time.

Linearity of expectation itself needs no independence between collision events. The full uniform model is sufficient; a universal family will later give an upper bound using only pairwise collision bounds.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Successful search averages records rather than buckets

Fix distinct insertion order $x_1, \ldots, x_n$, insert at heads, perform no deletions, and choose a successful query uniformly among stored records. Finding x_i checks itself and later inserted records in its bucket.

$$E[C_i] = 1 + \frac{n - i}{m}, \qquad E[C] = \frac{1}{n} \sum_{i=1}^{n} E[C_i] = 1 + \frac{n - 1}{2m}.$$

Adding constant hashing work gives $\Theta(1 + \alpha)$. Larger buckets contain more keys that may be queried; successful queries do not select buckets uniformly.

Our six-key table has two length-two chains and two singletons. Summed successful checks are $(1 + 2) + (1 + 2) + 1 + 1 = 8$, averaging $4/3$. This fixed-table average need not equal the random-model expectation $1 + 5/12$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Division and string hashing: a formula is not a distribution guarantee

Division hashing is $h(k) = k \bmod m$. For $m = 2^p$, it retains only p low bits: 0 and 8 both map to 0 when m=8. A prime modulus can avoid some input patterns, but does not prove uniformity for arbitrary key sets.

Horner evaluation uses $H_0 = 0$ and $H_{i+1} = (BH_i + c_i) \bmod m$. Induction shows this equals the full base-B integer modulo m. A length-ℓ string requires $\Theta(\ell)$ constant-cost steps.

With B=10, m=7, codes [1,2,3] yield intermediate values $1, 5, 4$, matching $123 \bmod 7 = 4$. If $B \equiv 1 \pmod m$, the result is just the code sum modulo m, unchanged by permutation. Computing the formula correctly does not establish low collision probability.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Multiplication hashing: take high bits of the low word

Use $h(k) = \lfloor m \operatorname{frac}(kA) \rfloor$. For $A = s/2^w, m = 2^p$, compute $r_0 = ks \bmod 2^w$, then $h(k) = \lfloor r_0 / 2^{w-p} \rfloor$, with $1 \le p \le w$.

Write $ks = q2^w + r_0$. Then $kA = q + r_0/2^w$, so its fractional part is $r_0/2^w$. Multiplying by 2^p and flooring discards the low w−p bits of the low word, leaving its p high bits.

Next use w=8, s=159, k=13, p=4 to trace every bit; the result should be 1.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Multiplication example: from full product to slot index

w = 8; s = 159; k = 13; p = 4

ks = 2067 = 8 × 256 + 19

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	1	0	0	0	0	0	0	1	0	0	1	1

Discard high word 00001000 = 8 Keep low word 00010011 = 19

Select bits 7..4 of the low word: 0001 → h = 1

$2067 = 8 \cdot 256 + 19$. The low word is 00010011; its four high bits are 0001. Do not take the highest bits of the full product. Respect the stated word width.

Universal families randomize the choice of function

A finite family H is universal if every fixed pair of distinct keys x,y satisfies

$$\Pr_h[h(x) = h(y)] \leq \frac{1}{m}$$

when h is chosen uniformly from H. Keys and queries must be independent of that draw. No uniform input distribution or mutual independence of all collisions is required. An individual sampled table may still have a long chain.

Source convention: we use the “at most” definition of CLRS § 11.3.3. MIT 2005 Lecture 8 defines an exact collision probability and demonstrates a prime-modulus dot-product family. Its equality implies our inequality, but the reverse implication is invalid.

Universal chaining: the complete expected upper bound

Fix set K and query q. Let $Y_q = \sum_{x \in K, x \neq q} \mathbf{1}[h(x) = h(q)]$. Linearity and universality give

$$E[Y_q] \leq \frac{|K \setminus \{q\}|}{m}.$$

An absent query scans at most n/m records in expectation. A present query's entire chain has expected length at most $1 + (n-1)/m$, and search costs no more than scanning it. Thus each fixed query has expected time $O(1+\alpha)$ without requiring uniform successful queries.

For a fixed operation sequence with at most $O(m)$ live keys, update-style INSERT and DELETE(key) search first. Total expected time for t calls is $O(t)$ plus $\Theta(m)$ initialization. Queries chosen adaptively from h or earlier observations are not covered by this fixed-pair proof.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Affine family: prove the prime-field mapping first

Choose prime $P > \max U$, independently uniform $a \in \{1, \ldots, P-1\}$ and $b \in \{0, \ldots, P-1\}$, and define

$$h_{a,b}(k) = ((ak + b) \bmod P) \bmod m.$$

For fixed x≠y, let $r = (ax + b) \bmod P$ and $s = (ay + b) \bmod P$. Since $a(x-y) \not\equiv 0 \pmod P$, r≠s.

Conversely, each r≠s uniquely determines $a = (r-s)(x-y)^{-1} \bmod P$ and $b = r - ax \bmod P$. Uniform (a, b) therefore produces uniform ordered distinct (r, s) among $P(P-1)$ possibilities. Primality guarantees the inverse of nonzero x−y.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

The final reduction still bounds collisions by 1/m

Condition on r. Each of the other P−1 residues s is equally likely. At most $\lceil P/m \rceil - 1 \leq (P-1)/m$ share r's residue modulo m. Hence

$$\Pr[h(x) = h(y) \mid r] \leq \frac{\lceil P/m \rceil - 1}{P-1} \leq \frac{1}{m}.$$

Averaging over r proves universality. For m≥P, reducing modulo m merges no distinct r,s, so collision probability is zero.

To check the count inequality, write $P = qm + d, 0 \leq d < m$. The count is q−1 when d=0 and q otherwise. Table size m need not be prime; P must be prime. Constant-time bounds assume the required modular arithmetic fits the word model.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Why the MIT dot-product family also works

Let m be prime and represent keys by fixed-length vectors $x = (x_0, \ldots, x_r) \in \mathbb{F}_m^{r+1}$. Draw uniform coefficient vector a and use $h_a(x) = \sum_i a_i x_i \bmod m$.

For x≠y, choose j with $x_j - y_j \neq 0$. After fixing all other coefficients, collision requires

$$a_j(x_j - y_j) = -\sum_{i \neq j} a_i(x_i - y_i) \pmod{m}.$$

The difference is invertible, so exactly one of m values of a_j satisfies the equation. Collision probability is exactly 1/m. Here table modulus m must be prime, unlike the arbitrary table size of the affine family. Variable vector length also changes hashing cost.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Open addressing uses three distinct slot states

Store one record per slot, so n≤m. Each key has a reproducible fixed sequence $h(k,0),\ldots,h(k,m-1)$. To find space whenever available, these m positions must form a permutation of the table.

State	Search action	Deletion effect
OCCUPIED	Compare full keys	May become DELETED
DELETED (tombstone)	Continue probing	Not a stopping point
EMPTY	Stop: absent	Never created by ordinary deletion

EMPTY means unused since this table was built. An inserted key could not have skipped an EMPTY earlier in its sequence. Deletion leaves DELETED rather than breaking that path. A rebuild establishes a new table state.

Search and deletion probe at most m slots

```
FIND(k):
    for i = 0 .. m-1:
        j = h(k,i)
        if T[j] is EMPTY: return absent
        if T[j] is OCCUPIED and T[j].key == k:
            return j
    return absent
```

DELETE(k) runs FIND, changing a found slot to DELETED and otherwise preserving state. EMPTY and DELETED require separate tags rather than reserved values that could also be legal keys.

The loop bound matters: a table may contain tombstones but no EMPTY slots. An unsuccessful search must return after m probes rather than wait indefinitely for an empty slot.

Update-style INSERT remembers a tombstone but keeps searching

```
first = NONE
for i = 0 .. m-1:
    j = h(k,i)
    if T[j] is OCCUPIED and T[j].key == k:
        T[j].value = value; return j
    if T[j] is DELETED and first == NONE: first = j
    if T[j] is EMPTY:
        target = first if first != NONE else j
        T[target] = OCCUPIED(k,value); return target
if first != NONE: T[first] = OCCUPIED(k,value); return first
return full
```

An existing key may lie beyond the first DELETED. Inserting immediately could create duplicates. Establish absence at EMPTY or after all m probes, then reuse the saved slot. This assumes a full-table probe permutation.

Tombstones preserve search paths while allowing reuse

m = 7; home(k) = k mod 7

∅ = EMPTY; D = DELETED

	0	1	2	3	4	5	6	
Insert 1, 8, 15	∅	1	8	15	∅	∅	∅	Probes: 1 → 2 → 3
Delete 1	∅	D	8	15	∅	∅	∅	Find 15: 1 → 2 → 3
Update 8	∅	D	8	15	∅	∅	∅	Update value at slot 2
Insert 22	∅	22	8	15	∅	∅	∅	Probe to 4; write slot 1

Continue past D; rule out an existing key before reusing it.

Use $m = 7, h(k,i) = (k \bmod 7 + i) \bmod 7$. Keys 1,8,15 occupy slots 1,2,3. After deleting 1, finding 15 still follows $1 \to 2 \to 3$; a tombstone does not stop search.

A complete probe ledger for the same records

Three records remain. INSERT(22) confirms absence at slot 4, then writes slot 1.

Call	Slots probed	Write / result	Probes
INSERT(1,a)	1	store (1,a) at 1	1
INSERT(8,b)	1,2	store (8,b) at 2	2
INSERT(15,c)	1,2,3	store (15,c) at 3	3
DELETE(1)	1	slot 1=DELETED	1
FIND(15)	1,2,3	return slot 3	3
INSERT(8,b′)	1,2	update slot 2 only	2
INSERT(22,d)	1,2,3,4	reuse slot 1	4
FIND(29)	1,2,3,4	absent	4

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Probe formulas: first establish table coverage

Linear probing uses $(h_0(k) + i) \bmod m$ and covers every slot. Contiguous occupied runs attract further insertions, producing primary clustering.

Quadratic probing uses $(h_0(k) + c_1 i + c_2 i^2) \bmod m$ with constrained parameters. For m=7 and offset i^2, the first seven offsets are $0, 1, 4, 2, 2, 4, 1$, visiting only four slots. Insertion can fail despite unused slots elsewhere.

Equal home positions give identical sequences in these two schemes, the source of secondary clustering. Double hashing varies the step as well, but coverage still requires a step coprime to m; adding a function alone is not a proof.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Two provable full-coverage probe schemes

Double hashing: use $h(k,i) = (h_1(k) + id) \bmod m$. Returning home requires $m \mid id$. With $g = \gcd(m,d)$, cancellation gives minimal positive period m/g. Full coverage therefore requires g=1. Use 1≤d<m for prime m, or odd d for power-of-two m.

Triangular offsets: for $m = 2^w$, use $f(i) = i(i+1)/2 \bmod m$, dividing as an integer first. For m=8 the offsets are $0, 1, 3, 6, 2, 7, 5, 4$.

If f(i)=f(j), then $2^{w+1} \mid (i-j)(i+j+1)$. The factors have opposite parity, so the even factor is divisible by 2^{w+1}. But $|i-j| < 2^w$ and $1 \le i+j+1 < 2^{w+1}$. The only possibility is i−j=0. Thus f is injective and covers all m slots.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Uniform probing is a stronger idealized model

The following bounds assume each new key has a uniformly random permutation relative to the current occupied state, with no deletions or tombstones and n<m. This is sampling without replacement, not selecting a fresh random slot independently on every probe.

Linear, quadratic, and double hashing generate restricted sequence families. Uniform home locations or a universal family do not automatically imply the ideal distribution over m! permutations.

Separate collision probabilities for chaining, full-table probe coverage, and the distribution of probe permutations. The first two do not imply the expected formulas that follow.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Unsuccessful-search bound from tail probabilities

Let X count probes through the first EMPTY and $\alpha = n/m < 1$. The probability that the first i−1 positions are occupied is

$$\Pr(X \geq i) = \prod_{j=0}^{i-2} \frac{n-j}{m-j} \leq \alpha^{i-1}, \quad 1 \leq i \leq n+1.$$

For i=1 the empty product is 1; for i>n+1 the probability is zero. Each ratio is at most n/m since $m \geq n$. Using $X = \sum_{i \geq 1} \mathbf{1}[X \geq i]$,

$$E[X] = \sum_{i \geq 1} \Pr(X \geq i) \leq \sum_{i \geq 1} \alpha^{i-1} = \frac{1}{1-\alpha}.$$

The same bound covers insertion of a new key. At m=10,n=8 it is 5, not an exact count per query or an unconditional guarantee for linear probing.

Successful-search bound replays the insertion path

Fix n distinct insertions with no deletions or relocations. Choose successful queries uniformly among records. Insertion i+1 sees i occupied slots and expects at most $m/(m-i)$ probes; later searches replay that path.

$$E[C] \leq \frac{1}{n} \sum_{i=0}^{n-1} \frac{m}{m-i} = \frac{1}{\alpha} \sum_{k=m-n+1}^{m} \frac{1}{k}$$
$$\leq \frac{1}{\alpha} \int_{m-n}^{m} \frac{dx}{x} = \frac{1}{\alpha} \ln \frac{1}{1-\alpha}.$$

Since 1/x decreases, its integral over [k−1,k] is at least 1/k. No successful query exists at α=0; do not substitute zero into 1/α. The right side tends to 1 as α→0. Both the ideal probe model and uniform successful queries are needed for this formula.

Low live load can still require a full-table search

For m=7, fill all slots with seven distinct home positions, then delete every key. Now n=0 and α=0, but every slot is DELETED. Any absent query probes all seven positions.

The tombstone-free $1/(1-\alpha)$ bound is inapplicable. Tracking $\beta = (n+d)/m$, where d counts tombstones, exposes β=1 and the absence of an early EMPTY stopping point. β alone is not a performance proof for every implementation.

Rebuilding removes tombstones but costs allocation, rehashing, and reinsertion; open-address reinsertion itself requires probes. An expected amortized rebuild claim needs explicit thresholds, a random model, and rebuild rules, not just a promise to rehash periodically.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Perfect hashing moves random retries into construction

FKS stores a fixed set of n distinct keys. After construction, queries take worst-case $O(1)$ word operations with $O(n)$ space. Random retries give expected linear construction time, not worst-case linear time.

For n=0, return an empty dictionary without modulo zero. For n≥1 use m=n, draw a universal first-level h, and let bucket j contain n_j keys. Allocate n_j^2 secondary slots.

Let Y count unordered colliding pairs. Applying $n_j^2 = n_j + 2\binom{n_j}{2}$ to each bucket gives

$$\sum_j n_j^2 = n + 2Y, \qquad E\left[\sum_j n_j^2\right] \le n + 2\binom{n}{2}\frac{1}{n} = 2n - 1.$$

This controls total squared space through collision pairs; it does not require every bucket to be small.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

First level: check space before accepting the draw

Let $S = \sum_j n_j^2$. Markov gives $\Pr(S \geq 4n) \leq E[S]/(4n) < 1/2$. Independently redraw until the measured value satisfies S<4n.

Each trial clears n bucket counts and processes n keys in O(n). **Check S before allocating quadratic secondary tables for a rejected draw.** Acceptance probability exceeds 1/2, so expected trials are below 2 and first-level expected work is O(n).

Acceptance establishes a deterministic S<4n for the built structure. Merely knowing E[S]<2n would not establish that every output has linear size. Rare runs can require many retries; expected trial count is not a hard limit.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Second level: squared space buys collision-free placement

Condition on an accepted first-level partition. A bucket of s≥2 keys gets s^2 slots and an independent universal h_j. Its collision-pair count Z satisfies

$$E[Z] \leq \binom{s}{2}\frac{1}{s^2} = \frac{s-1}{2s} < \frac{1}{2}, \qquad \Pr(Z \geq 1) \leq E[Z] < \frac{1}{2}.$$

Accept only after verifying zero collisions. Each trial clears s^2 slots and places s keys in $O(s^2)$, with fewer than two expected trials. Summing conditionally over the fixed accepted partition gives $O(\sum_j s_j^2) = O(n)$ expected work; averaging over the first level preserves O(n) total construction.

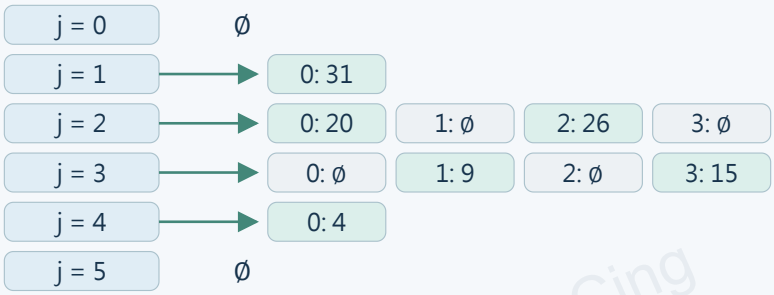
Empty buckets allocate nothing; singleton buckets store their record directly. Sizes, parameters, and pointers add O(n) space. Worst-case queries assume constant-cost hashing and key comparison; inserting new keys is outside this static contract.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Two-level layout: place the same keys in actual slots

P = 37; h(k) = (k mod 37) mod 6

Two-key buckets: $h_j(k) = (k \bmod 37) \bmod 4$



26, 14 → j = 2 → slot 2

Stored key = 26

26 = 26 ✓

14 ≠ 26 ✗

Total secondary slots: 1 + 4 + 4 + 1 = 10

Use $K = \{4, 9, 15, 20, 26, 31\}$, P=37, and $h(k) = (k \bmod 37) \bmod 6$. Bucket sizes [0,1,2,2,1,0] have square sum 10<4n=24. Both two-key buckets use $h_j(k) = (k \bmod 37) \bmod 4$, occupying (0,2) and (1,3), respectively, with verified zero collisions.

An absent key can still reach an occupied perfect-hash slot

Query 26 follows first-level bucket 2 and secondary slot 2, reads key 26, and succeeds after equality comparison. Query 14 follows the same route but reads 26, so it is absent.

Query 5 reaches empty first-level bucket 5 and stops without evaluating a nonexistent secondary hash. Query 10 reaches singleton bucket 4 but reads 4≠10, so it also fails.

Perfect hashing separates stored keys; it does not certify membership for absent keys. Handle empty buckets, check slot occupancy, and compare full keys. After construction, present and absent queries both traverse constantly many levels, establishing the worst-case query bound.

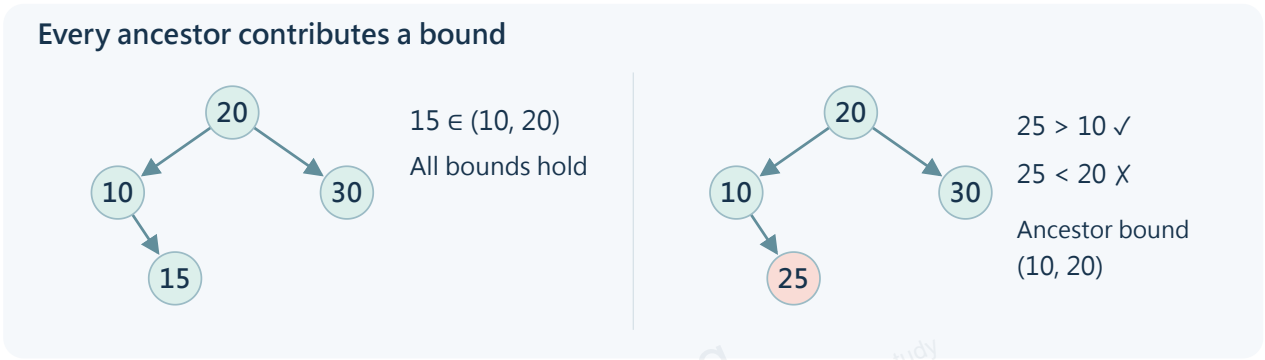
BST constraints come from every ancestor

A binary search tree turns comparisons into branches. Continue the unique-key dictionary contract: a node stores key, value, left, right, and parent; missing links are NIL and root.parent=NIL. Inserting an existing key updates its value.

Every key in a node's entire left subtree is smaller; every key in its entire right subtree is larger. Parent-child comparisons alone are insufficient. A left turn tightens the upper bound; a right turn tightens the lower bound.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Ancestor intervals detect globally invalid placement



CLRS § 12.1 also permits duplicates under non-strict inequalities. Here order is strict. Duplicates could instead use a count or a (key,unique-ID) ordering; these require an explicit contract.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Search rules out an entire subtree at each comparison

```
FIND(T,k):
    x = T.root
    while x != NIL:
        c = compare(k,x.key)   // one three-way comparison
        if c == 0: return x
        x = x.left if c < 0 else x.right
    return absent
```

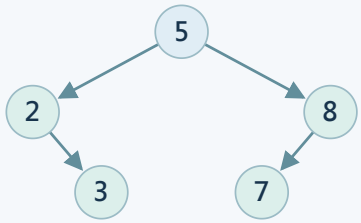
Invariant: if k exists, it lies in the subtree rooted at x. Global BST order rules out the other subtree after an unequal comparison. Each iteration descends one level; reaching NIL proves absence.

Depth d counts **edges** from the root, whose depth is 0. Successful search at depth d visits d+1 records. Failed search counts only visited non-NIL nodes. Count one three-way key comparison per visit; NIL tests and comparator instructions are separate costs.

Height h counts edges on a longest path; the empty tree has height −1. For nonempty trees search costs O(h+1); an empty tree costs O(1). A uniform bound is O(h+2).

Trace successful and unsuccessful searches in one tree

Root depth is 0; successful comparisons = depth + 1



d = 0

Find 7: 5 → 8 → 7; 3 comparisons

d = 1

Find 6: 5 → 8 → 7 → NIL

Failure also uses 3 comparisons

d = 2

Queries 9 and 10 fail to the right of 8

Finding 7 visits 5→8→7, using three key comparisons. Finding 6 visits those same three nodes before reaching NIL. Queries 9 and 10 visit 5→8→NIL, using two comparisons.

Count visited records for either outcome. The final NIL pointer test is not another key comparison.

Average cost depends on how queries are drawn

Uniform queries among the five stored keys give total successful comparisons 1+2+2+3+3=11, hence mean **11/5**. Uniform integer queries from 1 through 10 also include five failures.

query	1	4	6	9	10
keys visited	5,2	5,2,3	5,8,7	5,8	5,8
comparisons	2	3	3	2	2

The mixed mean is (11+12)/10=**2.3**. Gaps are not equally likely: $(8,+\infty)$ contains queries 9 and 10 here, with weight 2/10. Equal weights on NIL links would model a different experiment.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Inorder traversal and extrema

Inorder traversal lists left subtree, current node, then right subtree. Inductively, both subtrees produce increasing sequences. Global BST order places every left key below the root and every right key above it, so concatenation is sorted. Each node is processed once, giving $\Theta(n+1)$ time and $O(h+2)$ recursive stack space.

MINIMUM(x) requires $x \neq \text{NIL}$ and follows left links. If a left subtree exists it contains the minimum; otherwise x is smaller than everything to its right. MAXIMUM is symmetric. A dictionary-level minimum on an empty tree should return absent.

BST sorting includes construction. Inserting 1,2,…,n produces a right chain and costs $0+1+\dots+(n-1)=n(n-1)/2$ comparisons. Adding linear inorder traversal still gives $\Theta(n^2)$ total time.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Successor: descend right or climb to the first left turn

```
SUCCESSOR(x):                   // valid node handle
    if x.right != NIL: return MINIMUM(x.right)
    p = x.parent
    while p != NIL and x == p.right:
        x = p; p = p.parent
    return p                   // NIL means no successor
```

With a right subtree, its minimum is the next key. Without one, each climb from a right child passes a smaller ancestor; the first ancestor reached from its left side is the smallest larger key.

For root 5 with children 2,8, plus 2.right=3 and 8.left=7: succ(5)=7 via 8→7; succ(3)=5 via 3→2→5; succ(8)=NIL. PREDECESSOR swaps left and right.

A call follows one path and costs O(h+1). Starting at the minimum and repeatedly finding successors traverses each edge at most once downward and once upward, giving O(n) for the full sequence. Multiplying individual worst-case bounds does not prove Θ(nh).

Insertion finds a missing link, not an arbitrary leaf

```
INSERT(T,k,v):
    p = NIL; x = T.root
    while x != NIL:
        p = x; c = compare(k,x.key)
        if c == 0: x.value = v; return x
        x = x.left if c < 0 else x.right
    z = Node(k,v,NIL,NIL,p)
    if p == NIL: T.root = z
    elif k < p.key: p.left = z
    else: p.right = z
    return z
```

The search maintains the interval imposed by ancestors. At NIL, the new key satisfies every bound. The duplicate branch changes only the value, preserving key and identity. Arbitrarily changing a key in place can violate order.

Worst-case time is O(h+2), including empty-tree insertion, with O(1) new-node space.

Transplant changes the incoming parent link

```
TRANSPLANT(T,u,v):              // u is a valid member node
    p = u.parent
    if p == NIL: T.root = v
    elif u == p.left: p.left = v
    else: p.right = v
    if v != NIL: v.parent = p
```

This redirects the incoming link to u and updates a nonempty v's parent. **It does not reconnect v.left or v.right.** NIL is allowed as v but is not a writable parent-bearing node in this section.

If z has no left child, delete it with TRANSPLANT(T,z,z.right), including a leaf. If it has only a left child, transplant that child. Every retained subtree remains within the old ancestor bounds.

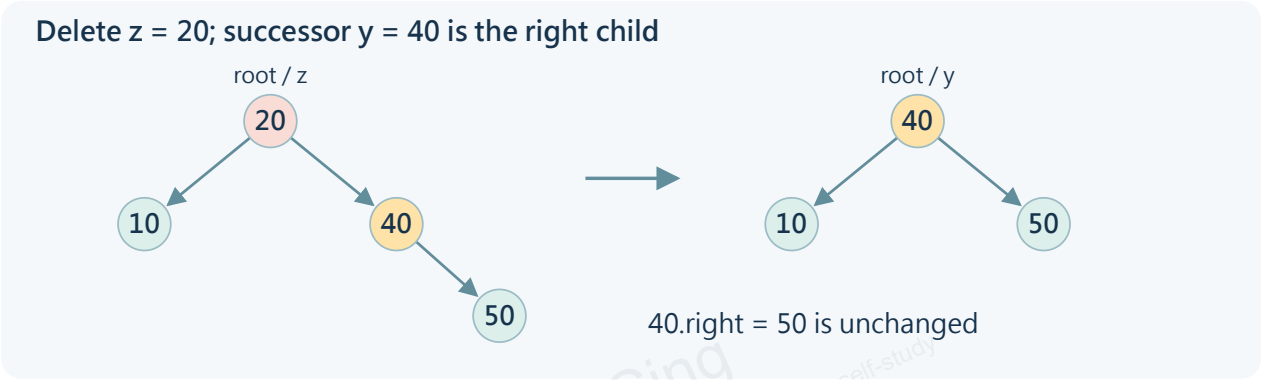
Deleting root 20 with sole right child 40 makes root=40 and 40.parent=NIL. Deleting the last node makes root=NIL. These cases change a constant number of links; DELETE(key) must also pay for finding z.

Two children: detach the successor before moving it

```
DELETE-NODE(T,z):
    if z.left == NIL: TRANSPLANT(T,z,z.right)
    elif z.right == NIL: TRANSPLANT(T,z,z.left)
    else:
        y = MINIMUM(z.right)
        if y.parent != z:
            TRANSPLANT(T,y,y.right)
            y.right = z.right; y.right.parent = y
        TRANSPLANT(T,z,y)
        y.left = z.left; y.left.parent = y
```

The right-subtree minimum y has no left child but **may have a right child**. A deeper y must first be replaced by y.right before receiving z's old right subtree. If y=z.right, leave its right link unchanged: assigning y.right=z.right would create y.right=y.

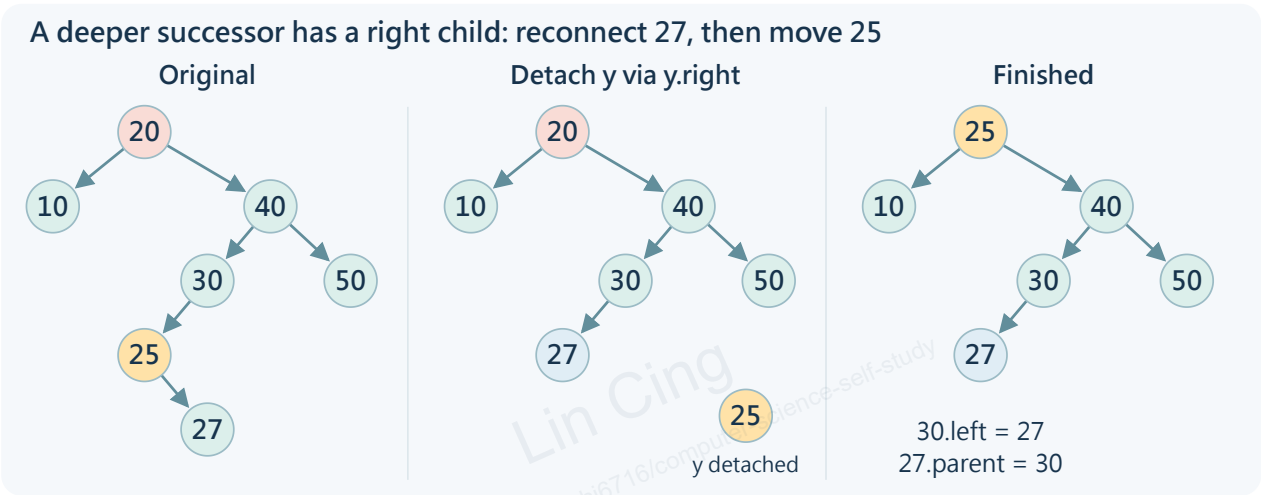
An immediate successor keeps its existing right child



Delete z=20: y=40 and y.parent=z. Redirect root to y and set y.left=10 and 10.parent=y. **The link y.right=50 never changes.**

Inorder changes from [10,20,40,50] to [10,40,50]. Every retained key, value, and object identity is unchanged; only z leaves the tree. For nonroot z, redirect its old parent's left or right link instead of root.

A deeper successor must not lose its right child



Delete root z=20. Its successor y=25 has right child 27. Writes are: ① 30.left=27 and 27.parent=30; ② y.right=40 and 40.parent=y; ③ root=y and y.parent=NIL; ④ y.left=10 and 10.parent=y.

Check order and reachability after a deep transplant

The middle panel shows y temporarily detached. Its stale links must be cleared or overwritten; the whole object pool is not a valid single BST during the update. Finally root=25,25.right=40,40.left=30,30.left=27.

Inorder [10,20,25,27,30,40,50] loses only 20. Writing 30.left=NIL would lose 27. Forgetting parent updates would break later upward successor searches.

Why is order preserved? Successor 25 is the minimum of the old right subtree. Every old left-subtree key is below 25, and every remaining right-subtree key is above 25. Key 27 lies between 25 and 30, so reconnecting it as 30.left is valid.

Every original object except z remains reachable exactly once. Finally check root.parent=NIL, matching child/parent links, and absence of cycles. Finding the successor costs O(h+1); rewiring is O(1). Reclaim z only after reconnection.

Deleting a key and deleting an object are different contracts

Let z.key=20 and its successor y.key=25. Both methods can preserve the same key set, but handles behave differently.

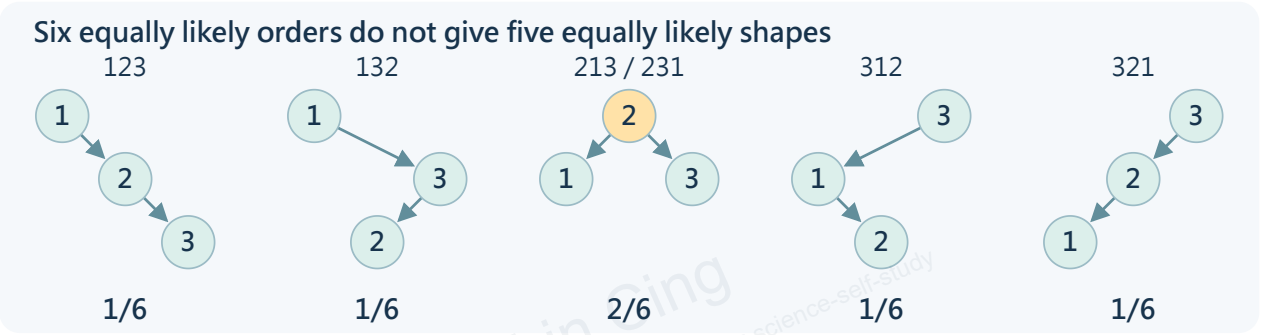
Method	Object occupying z's old position	Removed object	Handle effect
Copy y's payload, then delete y	Original z, now representing 25	y	z changes meaning; y is invalidated
Transplant node y	Original y in a new position	z	z is invalidated; y retains identity

Source convention: the handwritten notes illustrate copying; CLRS third edition § 12.3 transplants nodes, as we do here. Copying a dictionary record must move the matching value, not just its key. Extra fields, resource ownership, and external indices also need maintenance.

Either method may use the predecessor instead, with every direction mirrored consistently. Do not combine a predecessor diagram with successor code.

Uniform insertion orders do not give uniform tree shapes

Draw uniformly from n! orders on n distinct keys. The first key becomes root; given root rank r, each side retains uniform relative order. Analyze insertion-only construction.



Orders 213 and 231 share the balanced tree (2/6). Each other order gives a different shape (1/6). Sampling five shapes with probability 1/5 each is a different model.

Expected depth of a fixed rank: count ancestors

Write keys in order $x_1<\ldots<x_n$ and let D_i be the depth of x_i. For $j\neq i$, x_j is an ancestor of x_i **iff x_j is inserted first among the keys in the closed rank interval between i and j**.

Earlier keys outside the interval route both to the same side. If x_j appears first inside, it lies on the search path to x_i. Any other first key either is x_i or separates them. Thus

$$\Pr(x_j \text{ is an ancestor of } x_i) = \frac{1}{|j-i|+1}.$$

Depth counts ancestors. Linearity of expectation, without independent events, gives

$$E[D_i] = \sum_{j<i} \frac{1}{i-j+1} + \sum_{j>i} \frac{1}{j-i+1} = H_i + H_{n+1-i} - 2.$$

Here $H_t = \sum_{k=1}^{t} 1/k$. For n=5,i=3, expected depth is $2H_3 - 2 = 5/3$. A successful search adds one comparison, giving 8/3.

Expected construction cost sums all node depths

The nodes compared during insertion become exactly that node's ancestors in the finished tree; later insertions do not change its depth. Hence total construction comparisons are $C = \sum_i D_i$. Using

$$\sum_{i=1}^{n} H_i = \sum_{k=1}^{n} \frac{n-k+1}{k} = (n+1)H_n - n,$$

we obtain

$$E[C] = 2(n+1)H_n - 4n = \Theta(n \log n), \qquad n \geq 2.$$

Allocation and linking add Θ(n). Uniform successful search has expected mean comparisons E[C]/n+1. For n=3 these are 8/3 and 17/9 respectively; n=1 uses zero construction comparisons and one successful-search comparison.

This connects to quicksort using the first item as pivot recursively: comparison of a key pair depends on which key in their rank interval appears first. However, **logarithmic expected average depth does not yet bound expected maximum depth**.

Height analysis handles the maximum before expectation

Let X_n be edge height and $Y_n = 2^{X_n}$ for n≥1. Set **Y_0=0 as an auxiliary value**, not $2^{X_0} = 2^{-1}$. A singleton has Y_1=1. For n≥2, at least one subtree is nonempty; conditional on root rank R=r,

$$Y_n = 2 \max(Y_{r-1}^{L}, Y_{n-r}^{R}).$$

Write a_j=E[Y_j]. Conditional subtree orders are uniform. Since max(u,v)≤u+v,

$$a_n \leq \frac{2}{n} \sum_{r=1}^{n} (a_{r-1} + a_{n-r}) = \frac{4}{n} \sum_{j=0}^{n-1} a_j, \qquad n \geq 2.$$

This uses **conditional expectation**, not unconditional independence of random subtree sizes and R. It also avoids the false equality E[max(U,V)]=max(E[U],E[V]).

For example, $a_1 = 1$ and $a_2 = 2$. Among the six orders on three keys, two have height 1 and four have height 2, so a_3=(2 • 2+4 • 4)/6=10/3.

Bound exponential height, then return to height

Strong induction gives $a_n \le n^3$ for $n \ge 1$. The base is $a_1 = 1$. For $n \ge 2$, use $a_0 = 0$ and $\sum_{j=1}^{n-1} j^3 = [n(n-1)/2]^2$:

$$a_n \le \frac{4}{n} \sum_{j=1}^{n-1} j^3 = n(n-1)^2 \le n^3.$$

Let $\mu = E[X_n]$. Inequality $e^u \ge 1 + u$ implies $2^x \ge 2^\mu [1 + (\ln 2)(x - \mu)]$. Taking expectation cancels the linear term, yielding the needed Jensen inequality:

$$2^{E[X_n]} \le E[2^{X_n}] = a_n \le n^3 \quad \Longrightarrow \quad E[X_n] \le 3 \log_2 n.$$

The derivative of $e^u - 1 - u$ is $e^u - 1$, so its minimum is 0 at $u = 0$. This proves the tangent bound. Height has finite support, so these expectations exist.

A binary tree of edge height h contains at most $2^{h+1} - 1$ nodes; thus every tree has $h \ge \log_2(n+1) - 1$. Together the bounds give $\Theta(\log n)$ expected height. A single run may still have worst-case height $n - 1$.

Red-black trees control height through path invariants

A red-black tree is still a strictly ordered BST, with one extra color field. B means black and R means red. Figures include these labels as well as colors.

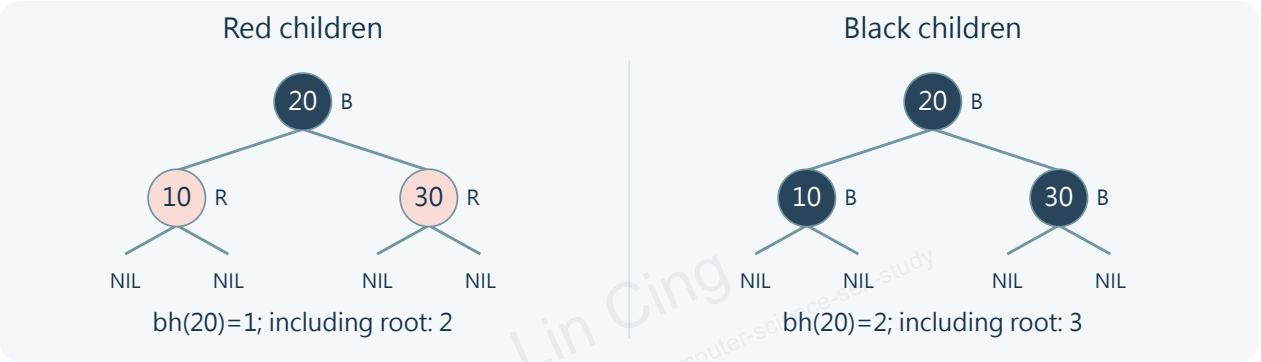
1. Each node is red or black, and the root is black.
2. Every external NIL position is black. A data node with no child still links to NIL.
3. Both children of a red node are black: consecutive red nodes are forbidden.
4. **From every node, all downward paths to NIL contain the same number of black nodes.** Checking only selected root–NIL paths is insufficient.

Use one shared sentinel with NIL.left=NIL.right=NIL, NIL.color=B, and root.parent=NIL. Require matching parent links only for non-NIL children. A duplicate-key insertion updates the value without changing shape or colors.

Earlier BST operations did not write NIL.parent. Deletion repair will temporarily use it for one deficient position. It cannot represent all missing-child parents simultaneously. Rotations still skip the parent write when β=NIL.

Fix exactly which nodes black-height counts

Define $bh(x)$ as the black nodes on a downward path from x to NIL, **excluding x and including the terminal NIL**; set $bh(NIL) = 0$. The path invariant makes this well-defined.



For a non-NIL child c, a red c has $bh(c) = bh(x)$; a black c has $bh(c) = bh(x) - 1$. The parent's count includes c, whereas c's own black-height excludes c. For c=NIL the relation is $bh(x) = 1 + bh(NIL)$.

Prove the height bound by counting required nodes

Retain internal edge height h: a singleton has h=0 and an empty tree h=−1. Separately let H count edges from root to external NIL. Then H=h+1 for a nonempty tree, and H=0 for an empty tree. Use H in the height proof without silently changing the meaning of h.

Lemma: The subtree at x contains at least $2^{bh(x)} - 1$ internal nodes. Induct on subtree height. For x=NIL, $0 = 2^0 - 1$. Otherwise each child has black-height at least bh(x)−1. Apply induction and count x:

$$n_x \geq 2\left(2^{bh(x)-1} - 1\right) + 1 = 2^{bh(x)} - 1.$$

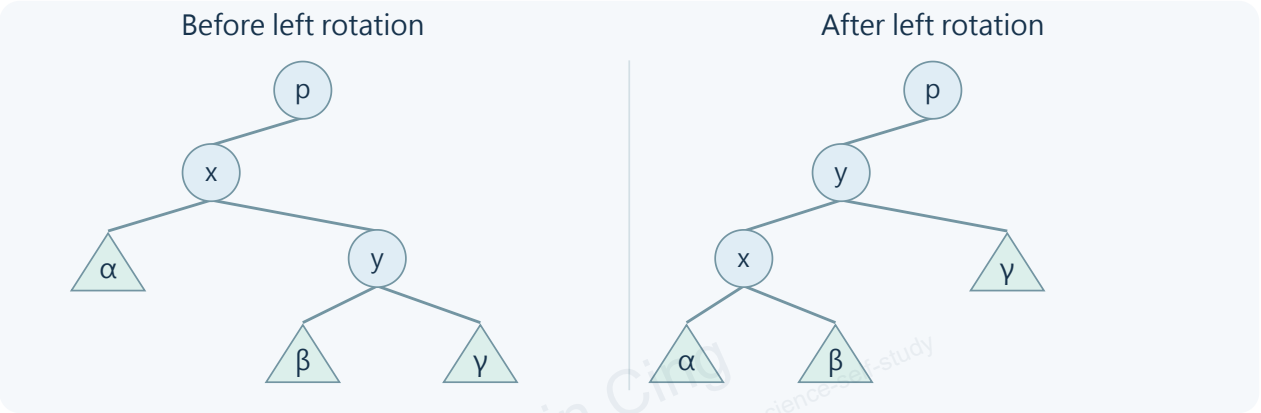
On a longest root–NIL path, let B count black nodes including both endpoints, and R count red nodes. Black endpoints and no adjacent reds imply $R \leq B - 1$. Its edge length is

$$H = B + R - 1 \leq 2(B - 1) = 2bh(root).$$

Hence $n \geq 2^{bh(root)} - 1 \geq 2^{H/2} - 1$ and $H \leq 2\log_2(n + 1)$. The empty tree has H=0. Searching and its terminating test cost O(H+1), or uniformly O(log(n+2)), including the empty-tree constant cost.

This bounds search height; update repair still needs a termination and cost argument. Red-black trees do not require child heights to differ by at most one: that is an AVL condition.

Left rotation changes the parent of β and the child of p



Require x≠NIL and y=x.right≠NIL. The symbols α, β, γ denote entire subtrees, possibly NIL; p is x's **old parent**. Node y becomes the local root, and β moves from y.left to x.right.

Complete, symmetric pointer updates for both rotations

```
LEFT-ROTATE(T,x):                RIGHT-ROTATE(T,y):
    p=x.parent; y=x.right            p=y.parent; x=y.left
    beta=y.left                      beta=x.right
    x.right=beta                     y.left=beta
    if beta!=NIL: beta.parent=x      if beta!=NIL: beta.parent=y
    y.parent=p                       x.parent=p
    if p==NIL: T.root=y              if p==NIL: T.root=x
    elif p.left==x: p.left=y          elif p.left==y: p.left=x
    else: p.right=y                  else: p.right=x
    y.left=x                         x.right=y
    x.parent=y                       y.parent=x
```

Save the old parent p. Skip the sentinel.parent write for β=NIL. Require x.right≠NIL or y.left≠NIL, respectively.

Check each rotation write against a concrete tree

Take p=30, x=10, y=20, β=15, with p.left=x. Leave all other subtree contents unchanged.

Step	Write	Purpose
1	10.right=15	Reconnect β
2	15.parent=10	Update β's parent
3	20.parent=30	Connect the new local root upward
4	30.left=20	Replace the old outside child slot
5	20.left=10	Make the old root a left child
6	10.parent=20	Update the old root's parent

For a root rotation, step 3 becomes 20.parent=NIL and step 4 becomes T.root=20.

Intervals α<x<β<y<γ remain valid, preserving BST order and node identities. Keys and values do not change. **Rotation alone does not ensure red-black validity.**

Insert a red leaf and identify the only possible violation

Use the previous INSERT search. An existing key only updates its value. Insert a new node z as R with left=right=NIL and a correct parent link.

Replacing a black NIL by a red z with black NIL children preserves every affected path's black count. Inserting a black node would add one black only to the paths through that position.

At loop entry maintain: **z is red; if its parent is the root, that parent is black; the only possible violation is a red root or the red pair z and its parent.** BST order and equal black counts remain valid.

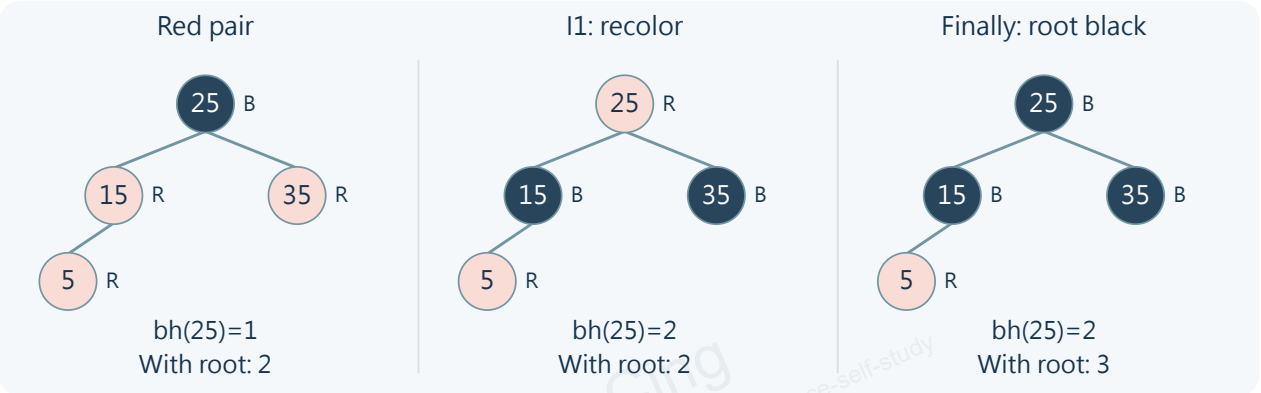
A black parent needs no repair. A new root is simply blackened at the end. A red parent p enters the loop and cannot be the root, so grandparent g exists and is black. The uncle u is g's other child; it may be NIL, whose color is safe to read.

I2 is not an alternative terminating case: it falls through into I3. Rotations change parent links. I3 uses the current z.parent and grandparent, not cached pre-rotation p and g.

Insertion repair when the parent is a left child

```
while z.parent.color==R:
    if z.parent==z.parent.parent.left:
        u=z.parent.parent.right
        if u.color==R:                          // I1
            z.parent.color=B; u.color=B
            z.parent.parent.color=R
            z=z.parent.parent
        else:
            if z==z.parent.right:               // I2
                z=z.parent
                LEFT-ROTATE(T,z)
            z.parent.color=B                    // I3
            z.parent.parent.color=R
            RIGHT-ROTATE(T,z.parent.parent)
    else: apply the explicitly mirrored branch
T.root.color=B
```


I1 preserves the black count seen by ancestors



Insert 25,15,35, then 5. The new 5R conflicts with parent 15R; uncle 35R is red too. Color 15 and 35 black, color 25 red, and move z to 25.

I1: which black count changes?

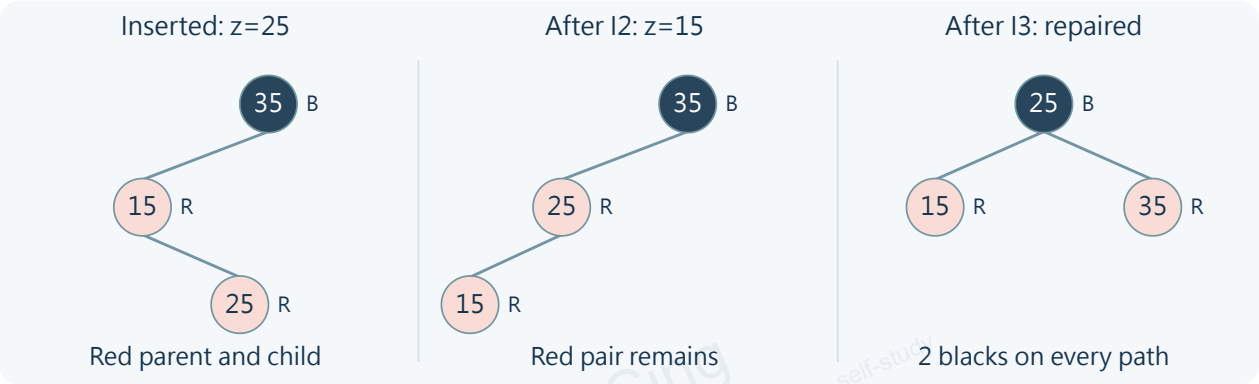
Let every path below p/u, excluding p/u, contribute b black nodes. A complete path through g originally contributes 1+b, and still contributes 1+b after g becomes red and p/u become black. **The ancestor-to-NIL black count through g stays unchanged**, although bh(g), which excludes g, rises from b to b+1.

Only g and its parent can now form a red pair. If g is the root, the final root-blackening step adds one black to all root–NIL paths. That final step must not be applied to a non-root g.

Counting range	Before I1	After I1
bh(g), excluding g	b	b+1
g through NIL, including g	1+b	1+b
Ancestor through g to NIL	outside count+1+b	outside count+1+b

For root g=25 and b=1, I1 leaves bh(25)=2 and a full-root count of 2. Final root blackening raises only the latter to 3.

I2 followed by I3: straighten the bend, then repair it

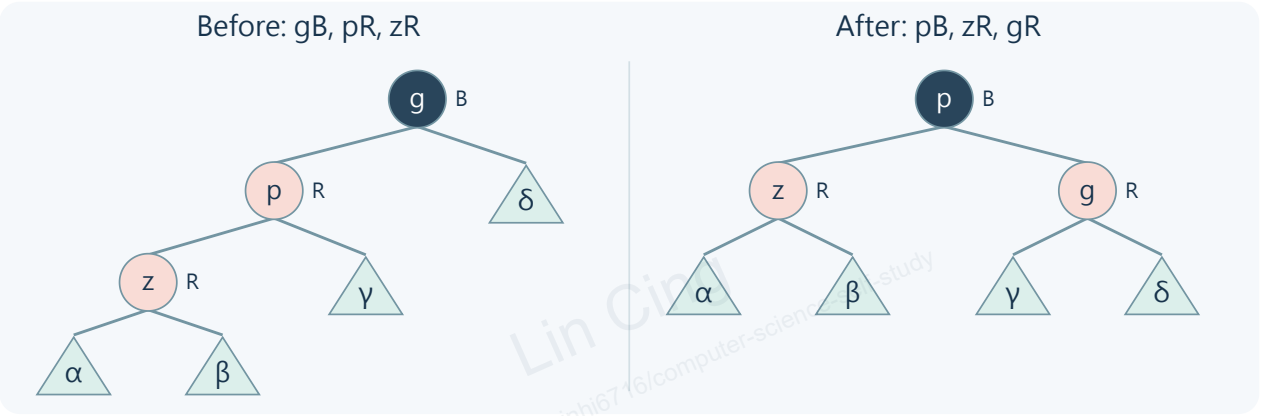


Insert 35,15,25. The uncle is black NIL; parent 15 is a left child and the new 25 is a right child. I2 first sets z to the old parent 15 and left-rotates 15. Now 25R is the parent of 15R: **the red pair still exists**.

I3 blackens the current parent 25, reddens grandparent 35, and right-rotates 35. Result: 25B with 15R,35R, and two blacks per path. Do not return after I2.

Why I3 preserves black counts and removes the red pair

Start with gB-left pR-left zR and a black uncle. Let α,β be z's children, γ be p's old right subtree, and δ be g's old right subtree. Blacken p, redden g, and right-rotate g.



I3: verify every repaired invariant

Red z forces α,β to have black roots; red p forces γ black; δ is the black uncle. Equal path counts make every boundary subtree contribute b blacks, including its root.

Destination subtree	Local black nodes before	Local black nodes after
α,β	gB, then b	pB, then b
γ	gB, then b	pB; gR adds zero; then b
δ	gB, then b	pB; gR adds zero; then b

All four destinations retain 1+b blacks. Each boundary subtree keeps its internal links and colors, preserving its own path equalities.

New local root p is black, so no outside red pair is possible. Red z has black children α,β; red g has black children γ,δ. An outer insertion chain skips I2; the same proof applies.

Write out the mirrored right-parent branch

This replaces the preceding code's else branch when z.parent is the grandparent's right child. The uncle is now on the left, and both rotation directions reverse.

```
u=z.parent.parent.left
if u.color==R:                                  // I1 mirrored
    z.parent.color=B; u.color=B
    z.parent.parent.color=R
    z=z.parent.parent
else:
    if z==z.parent.left:                         // I2 mirrored
        z=z.parent
        RIGHT-ROTATE(T,z)
    z.parent.color=B                            // I3 mirrored
    z.parent.parent.color=R
    LEFT-ROTATE(T,z.parent.parent)
```


Why insertion needs at most two rotations

I1 moves z upward by two levels without rotating, at most O(H) times. I2 falls through to I3, which blackens the local root and terminates. A black parent also terminates the loop; final root blackening resolves the remaining root-color case.

Branch	Primitive rotations	What follows
I1: red uncle	0	Move z up two levels; may repeat
I2: black uncle, inner child	1	Immediately perform I3
I3: black uncle, outer child	1	Black local root; terminate
Final root blackening	0	Done

The rotating suffix is either I3 or I2→I3. Repeated I1 steps add recolorings, not rotations.

Therefore there are at most two primitive rotations in total. Search plus recoloring costs O(log(n+2)); a new node uses O(1) space.

Two repair schedules must not be mixed

Comparison	Bottom-up here	Top-down 2-3-4 splitting
Two red children	Insert a red leaf, then repair upward	Split/recolor a 4-node during descent
Possible rotation	Black-uncle I2/I3	A red pair created during downward repair
Shared invariants	BST order, no red pair, equal black counts	Same goals, different repair-entry conditions

These algorithms may produce different valid shapes. Do not mix their recoloring schedules while retaining one algorithm's proof. A “double-rotation adjustment” may count as two primitive rotations; fix the unit and algorithm before comparing counts.

Coloring the edge to each red child gives a 2-3-4 interpretation after contracting red edges. General red-black trees allow either red-edge direction. Left-leaning red-black trees impose an additional convention.

Distinguish z, y, x, and y's original color

z is the record to remove. If z has fewer than two children, y=z; otherwise y is the minimum in z.right and moves as a whole object into z's position. x is the sole possible child filling **y's old position**, namely y.right for a successor with no left child; when y=z, use z's only child or NIL. Save yOriginalColor before giving moved y z.color. Missing black depends on yOriginalColor, not y's new color or merely z.color.

```
RB-TRANSPLANT(T,u,v):
    if u.parent==NIL: T.root=v
    elif u==u.parent.left: u.parent.left=v
    else: u.parent.right=v
    v.parent=u.parent              // write even when v==NIL
```

NIL.parent now records the parent of the current empty x position so repair can find its sibling. It is not a universal parent for all empty links. The root's parent is NIL. Rotations skip beta.parent for beta=NIL to preserve this context. After a stable operation, NIL.parent may be reset to NIL; NIL.color must always remain black.

RB-DELETE: relocate first, then inspect the original color

```
RB-DELETE(T,z):                  // z is an existing non-NIL record
    original=z.color
    if z.left==NIL:
        x=z.right; RB-TRANSPLANT(T,z,x)
    elif z.right==NIL:
        x=z.left; RB-TRANSPLANT(T,z,x)
    else:
        x,original=RELOCATE-SUCCESSOR(T,z)
    if original==B: DELETE-FIXUP(T,x)
    NIL.parent=NIL                // only after repair finishes
    reclaim z
```

Only object z is removed; all other identities and key/value pairs survive. A red original color removes no black; a black original color starts repair at replacement x. A key-based API must first search and return absent on failure. Never call this procedure on NIL.

Two children: preserve the successor's original color

```
RELOCATE-SUCCESSOR(T,z):          // requires two non-NIL children
    y=MINIMUM(z.right); original=y.color; x=y.right
    if y.parent==z:
        x.parent=y                // even when x==NIL
    else:
        RB-TRANSPLANT(T,y,x)
        y.right=z.right; y.right.parent=y
    RB-TRANSPLANT(T,z,y)
    y.left=z.left; y.left.parent=y
    y.color=z.color
    return x,original
```

Successor relocation: trace every changed link

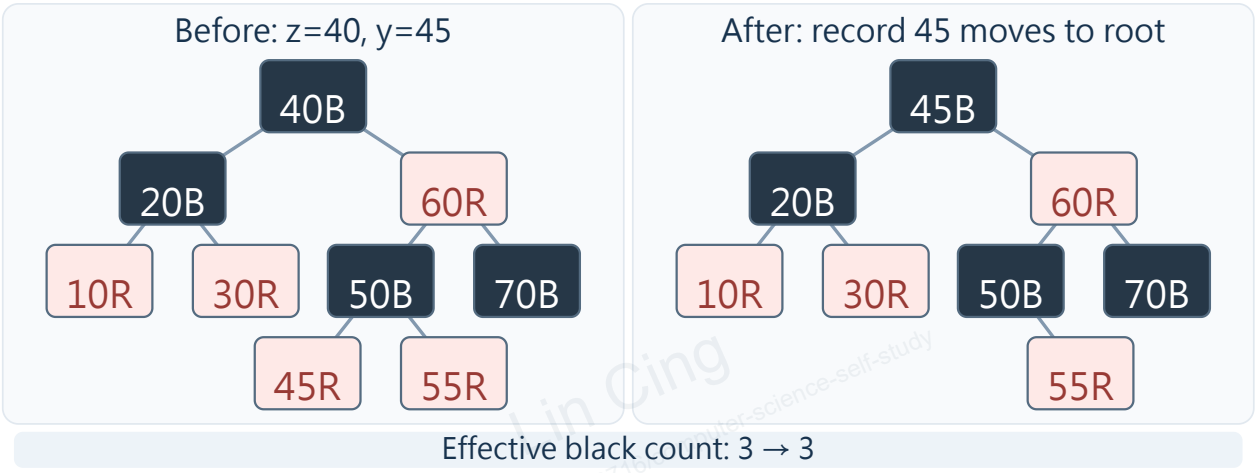
Insert 40,20,60,10,30,50,70,45,55. The tree has root 40B, left 20B(10R,30R), and right 60R with left 50B(45R,55R) and right 70B. Deleting z40 chooses y45R and x=NIL, with y.parent 50≠z.

Remove 45 from 50.left, giving 50.left=NIL and NIL.parent=50. Attach 60 as y.right and set 60.parent=45. Replace root 40 with y, then attach 20 as y.left and set 20.parent=45. Give y color B. Since y's original color was R, no black repair runs. Root is 45B, original 45's data moves with 45, and only record 40 is removed.

Successor y has no left child, so only y.right can fill its old position. If y is z's immediate right child, x's new parent is y, not the soon-reclaimed z. For a deeper y, the first transplant sets x.parent; moving y afterward must preserve that context.

Assigning y.color=z.color preserves the contribution at z's old position. The repair decision depends on **y's color before relocation**, a separate quantity.

A deeper successor: before and after relocation



Both panels show the same deletion: node 45 moves from the left of 50 to the root. Its old position is filled by x=NIL. The first transplant sets NIL.parent=50; moving 45 must preserve that context until repair or final cleanup resets NIL.parent=NIL. Omitted empty children are black NIL nodes.

Track a missing black at x without a third color

Removing an originally red y reduces no path black count. Giving moved y z's old color preserves the other local relationships, so no fix is needed. If y was black, temporarily assign its missing black to x's position. Count one extra black on each path through x to match the pre-deletion effective counts.

This proof-only extra black does not change the actual R/B color domain. A red x absorbs it by becoming black. At the root, drop it from all tree paths together. Otherwise the loop has a black nonroot x. Its sibling w cannot be NIL: w would contribute only one ordinary NIL black while x's side contributes at least NIL plus extra, contradicting equal effective black counts.

Below, x=p.left, w=p.right, near=w.left, and far=w.right. A right-side deficit exchanges left and right, giving near=w.right and far=w.left, and reverses rotations.

Figure legend: x +1 marks the proof-only extra black, not a third stored color. Unshown missing children are black NIL. Complete-path counts include the starting node and NIL.

The repair cursor tracks the unique extra-black position

```
DELETE-FIXUP(T,x):
    while x!=T.root and x.color==B:
        if x==x.parent.left: x=REPAIR-LEFT(T,x)
        else: x=REPAIR-RIGHT(T,x)
    x.color=B
```

The helpers return the next repair position: D2 returns the parent; D4 returns the root to terminate. Final blackening absorbs the extra black when x is red. When x is already the black root, remove the proof's extra contribution from every path without changing its stored color.

Left-side deficit: the full D1–D4 control flow

```
REPAIR-LEFT(T,x):
    p=x.parent; w=p.right
    if w.color==R:                          // D1
        w.color=B; p.color=R; LEFT-ROTATE(T,p)
        w=p.right
    if w.left.color==B and w.right.color==B:  // D2
        w.color=R
        return p
    if w.right.color==B:                    // D3
        w.left.color=B; w.color=R; RIGHT-ROTATE(T,w)
        w=p.right
    w.color=p.color                         // D4
    p.color=B; w.right.color=B; LEFT-ROTATE(T,p)
    return T.root
```


Right-side deficit: write the mirror explicitly

```
REPAIR-RIGHT(T,x):
    p=x.parent; w=p.left
    if w.color==R:                          // D1 mirrored
        w.color=B; p.color=R; RIGHT-ROTATE(T,p)
        w=p.left
    if w.left.color==B and w.right.color==B:  // D2 mirrored
        w.color=R
        return p
    if w.left.color==B:                     // D3 mirrored
        w.right.color=B; w.color=R; LEFT-ROTATE(T,w)
        w=p.left
    w.color=p.color                         // D4 mirrored
    p.color=B; w.left.color=B; RIGHT-ROTATE(T,p)
    return T.root
```

Select cases by sibling and near/far colors

Case	Condition, after handling a red sibling	Next state
D1	Red sibling	Black sibling; continue into D2/D3/D4
D2	Black sibling, both children black	Move the deficit to the parent
D3	Black sibling, red near/black far child	Convert to D4
D4	Black sibling, red far child	Remove extra black and terminate

For x on the left, near=w.left and far=w.right. Refresh w after D1 and D3. D3 falls through into D4. Returning p in D2 hands control back to the outer loop; it does not declare deletion complete.

For x on the right, near=w.right and far=w.left. Define these geometrically relative to x; near does not always mean left. Neither helper changes NIL.parent: the transplant establishes that context until repair finishes.

Shared NIL does not make the side test ambiguous here: the loop invariant requires a non-NIL sibling, so the parent cannot have NIL on both sides.

D1 converts a red sibling to a black sibling — black-count proof

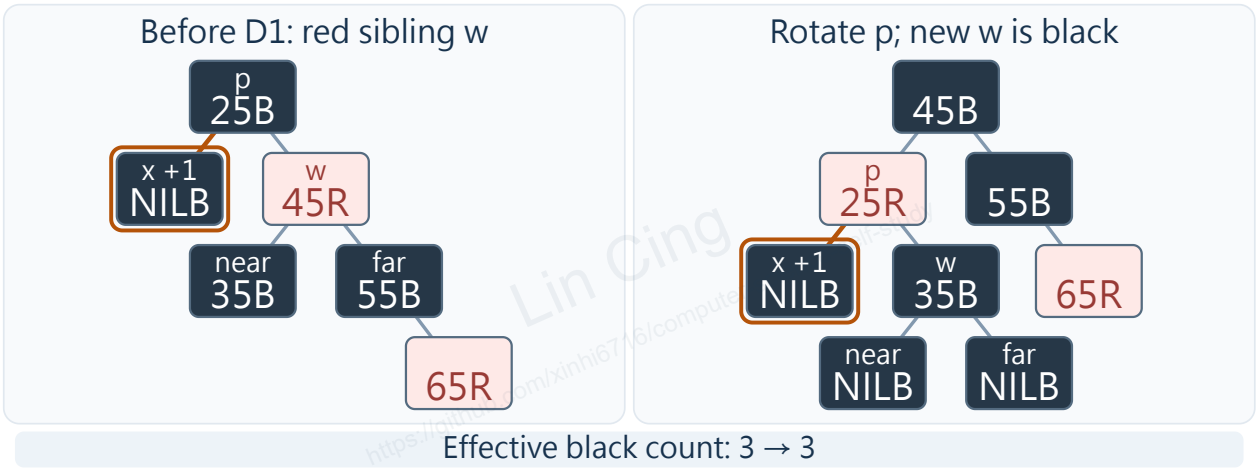
Let x's **actual** complete black count be b, including x, or NIL if x=NIL. Extra makes it b+1. Red w forces p and both w children black. Each w-child path contributes b+1, so every effective complete path including p contributes b+2.

Make w black and p red, then left-rotate p. New black local root w has a red left child p whose x-side effective count b+1 equals the old near-child count b+1. Its right side, the old far child, also contributes b+1. Total remains b+2. Extra remains at x; its new sibling is w's old black near child. Proceed to D2, D3, or D4.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

D1 converts a red sibling to a black sibling — worked trace

Insert 15,25,35,45,55,65 and delete 15. After removal x=NIL is 25B.left and w45R. D1 makes 45 black and 25 red, then left-rotates 25. New root 45B has left 25R(right 35B) and right 55B(right 65R); x remains 25.left. Complete black counts through ancestors are unchanged.



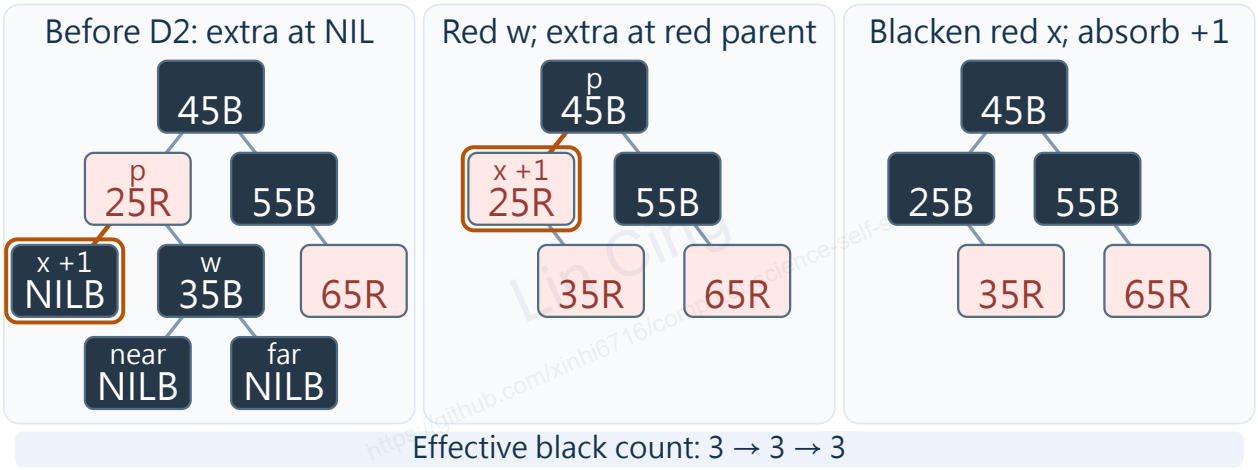
D2 moves the deficit upward or absorbs it at a red parent — black-count proof

Now w and both its children are black. x contributes actual b and effective b+1; w contributes b+1 and each child b. Making w red reduces its actual contribution to b. Both branches now have equal **actual** counts, but the whole p-subtree is one black short relative to its ancestors. Move extra to p by setting x=p.

If p was red, final blackening absorbs the extra and stops. If p was black and nonroot, iterate upward. At the root, drop extra consistently from all paths. D1 just made p red, so D1 followed by D2 absorbs in this round and cannot climb to another D1.

D2 moves the deficit upward or absorbs it at a red parent — worked trace

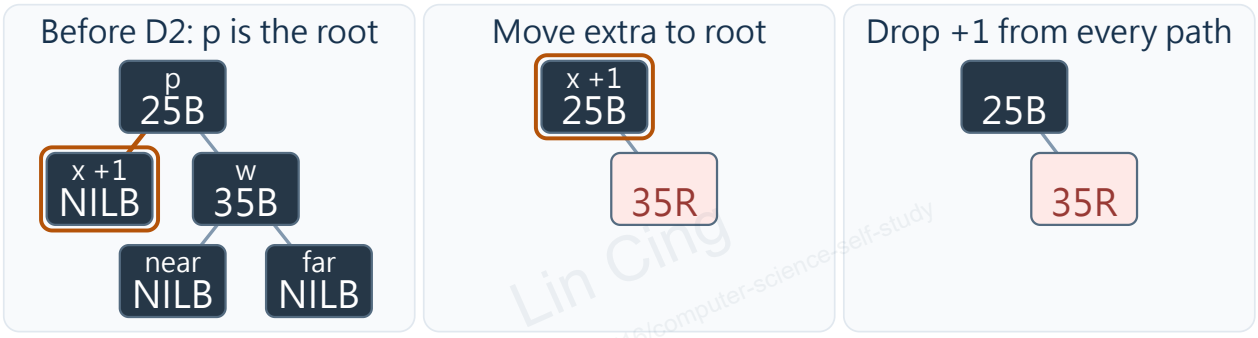
Continue the example: w35B has NIL children. Make 35 red and move x from NIL to 25R. The loop exits and 25 becomes black. Finally 45B has left 25B(right 35R) and right 55B(right 65R).



Effective black count: 3 → 3 → 3

D2 at the root reduces every path by one black

Start with 25B and black leaves 15B,35B. Deleting 15 leads D2 to make 35 red and move x to root 25. Dropping extra from all root-leaf paths leaves 25B(right 35R). Root bh decreases from 2 to 1, with equal path counts. This differs from absorption by blackening a red parent.



Effective black count: 3 → 3 → 2

D3 moves an available red node to the far side — black-count proof

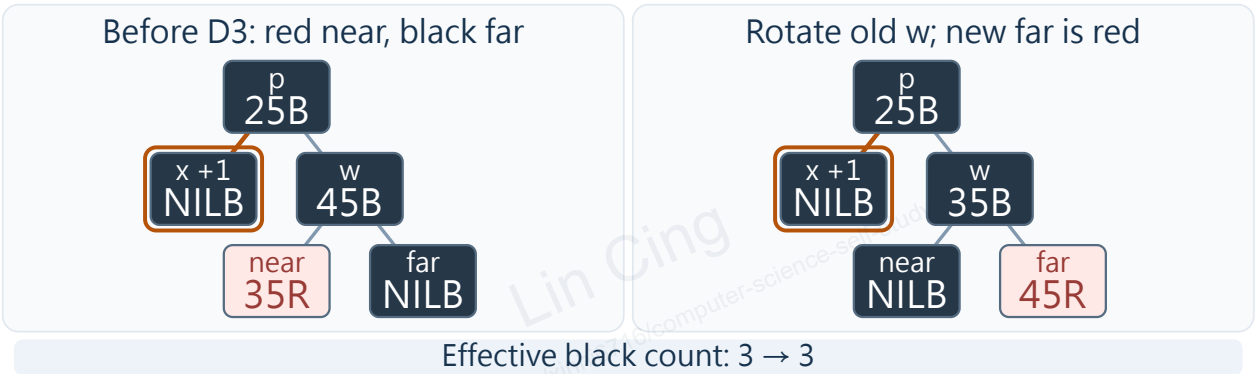
w is black, near N is red, and far is black. Let x's actual count be b. Both w children contribute b; red N's children each contribute b, as does far. Make N black and w red, then right-rotate w. New black sibling N has old N.left contributing b and old red w whose old N.right and far also contribute b. Its total remains b+1, preserving everything above p.

New sibling's far child is old red w, so execute D4 immediately. Extra remains at x; this case neither moves the deficit upward nor terminates.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

D3 moves an available red node to the far side — worked trace

Insert 15,25,45,35 to obtain 25B with left 15B and right 45B(left 35R). Delete 15, leaving x=NIL, w45B, near 35R, and black far=NIL. Make 35 black and 45 red, right-rotate 45, and obtain 25B with right 35B(right 45R). D4 must follow.



Lin Cing
https://github.com/xinhi6716/computer-science-self-study

D4 repairs both sides by recoloring and rotation — black-count proof

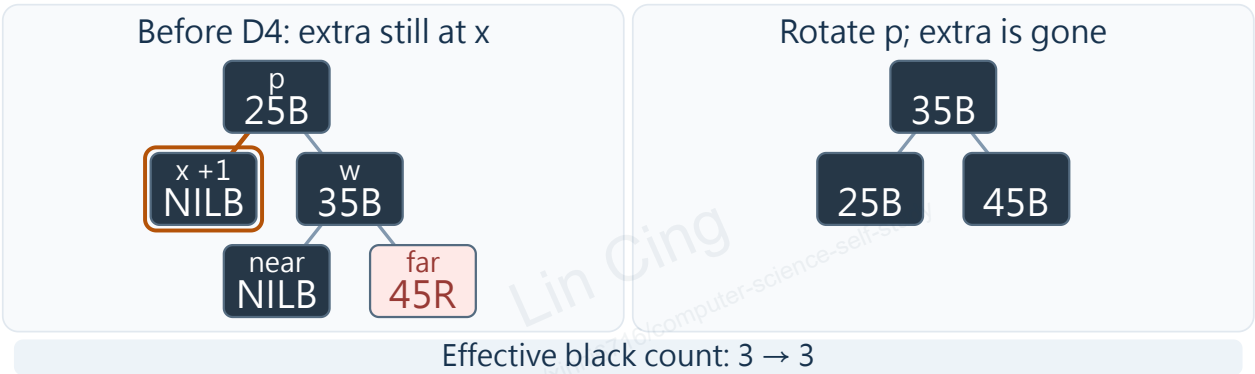
Let p's original black contribution be c∈{0,1}, x's actual count b, and both children of black w contribute b; far is red. Give w p's old color, make p and far black, and left-rotate p. New root w has black left child p with equal actual b on x and near, so its complete left count is b+1. Far changes red to black and also contributes b+1. New local total is c+b+1, equal to original p's effective c+(b+1).

If new w is red, its children p and far are black. Making p black creates no red-red conflict, and w retains p's external color relationship. Extra is eliminated. Set x=root to stop and ensure the root is black.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

D4: finish the numeric example

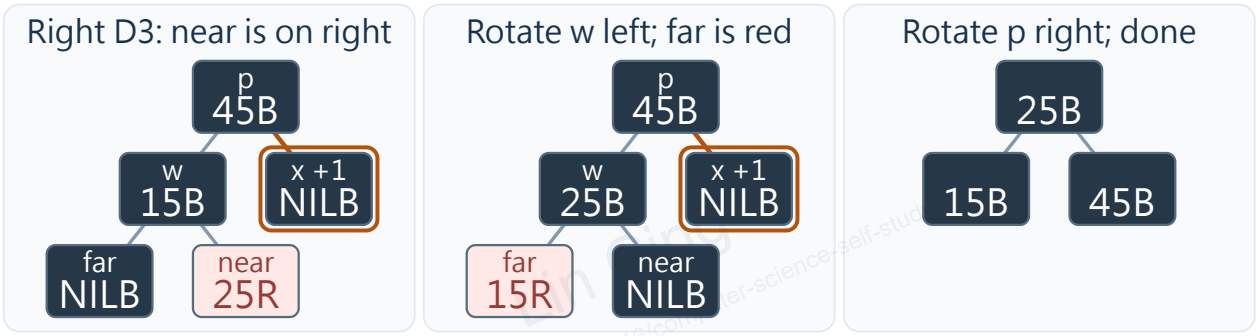
Continue D3:35 takes 25's old black color;25 and far 45 become black. Left-rotate 25 to obtain 35B with 25B,45B children. Inorder is 25,35,45, and every root-NIL path contains 3 blacks with no extra.



Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Right-side D3 → D4

Mirror example: insert 15,35,45,25, then delete 35. Direct successor 45B becomes root; x=NIL is 45.right, so NIL.parent must be 45. Sibling 15B has near 25R and far=NIL. Right-side D3 left-rotates 15, then right-side D4 right-rotates 45, yielding 25B with 15B,45B children.



Count the black nodes along the same D3→D4 example

Every row **includes its starting node and NIL**. D3 leaves x=NIL in place; D4 removes the extra black.

Stage and path	Actual black	Extra	Effective total
Before D3: 25B → NIL(x)	2	1	3
Before D3: 25B → 45B → 35R → NIL	3	0	3
After D3: 25B → 35B → 45R → NIL	3	0	3
After D4: 35B → 25B → NIL	3	0	3
After D4: 35B → 45B → NIL	3	0	3

Each black node adds 1 and each red node adds 0. Add the extra only on paths through x. The other NIL paths also total 3 by the same rule; check them in the figures. After D4 both sides have the needed black without an extra.

From local conservation to the general proof and cost

BST relocation preserves order and every undeleted record. Removing red directly leaves a valid tree. Removing black assigns extra to x so effective ancestor-to-leaf counts remain balanced. D1 and D3 preserve its position and effective counts; D2 moves the sole extra upward; D4 eliminates it. A red x is repaired by final blackening. In D2, newly red w has black children; rotations and recolorings satisfy the checked local color conditions. At termination all five properties hold.

D2 ascends one level with no rotation, at most O(log(n+2)) times. After D1, p is red, so D2 absorbs, or D3/D4 terminates. Therefore D1, D3, and D4 occur at most once each, for at most 3 rotations. Searching for z and its successor plus relocation and repair totals O(log(n+2)). NIL, root, and red-child absorption are valid base cases; deleting the last node leaves black root=NIL.

The rotation bound limits structural rearrangements. D2 propagation and recoloring may still traverse an ancestor path, so complete deletion takes O(log(n+2)), not O(1).

Dynamic order statistics from subtree sizes

Repeated updates and rank/select queries motivate an **order-statistic tree**: a balanced search tree augmented with subtree size, including the node itself. NIL.size=0 and

$$\text{size}(x) = 1 + \text{size}(x.left) + \text{size}(x.right).$$

Ranks are 1-based inorder positions of node identities. Examples use distinct keys; duplicates require a declared order such as (key,identity). A value alone does not identify one duplicate's rank.

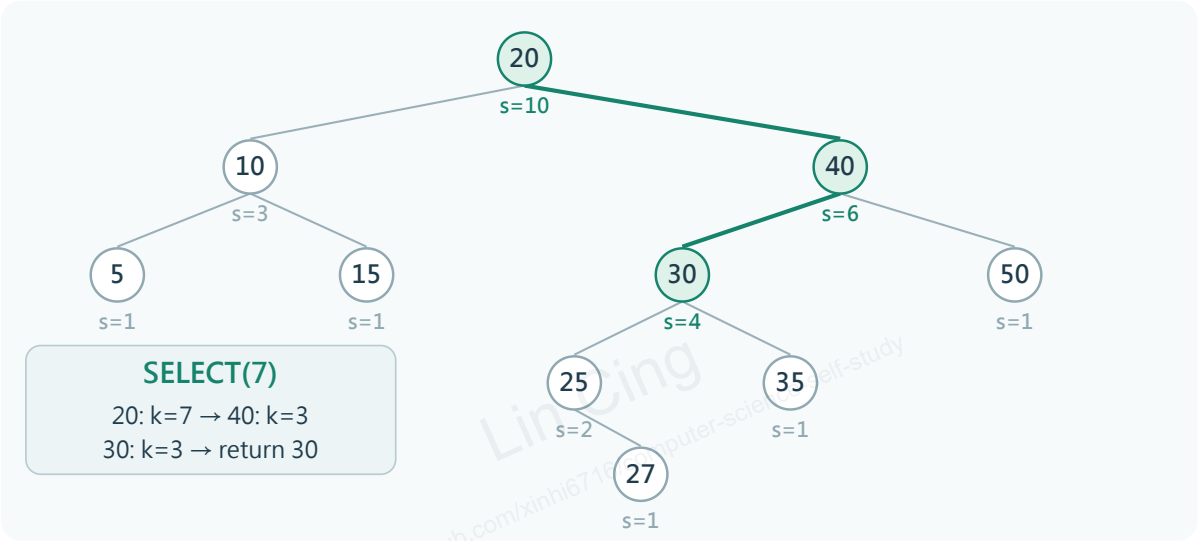
An augmentation must specify the summary, its maintenance after every modification, and the correctness of new queries. Here summaries use local data and child summaries in O(1) and do not depend on colors; color-dependent summaries require separate recoloring costs. First bound costs by height h: O(h+1). Red-black invariants then give O(log(n+2)); adding a field does not balance an ordinary BST.

Select uses a local rank at every level

```
select(x,k):
    require 1<=k<=x.size
    while x!=NIL:
        r=x.left.size+1
        if k==r: return x
        if k<r: x=x.left
        else: k-=r; x=x.right
```

Invariant: the answer is local rank k in the current x-subtree. The left subtree has r−1 nodes and x is position r. Going left retains k; going right subtracts the entire left subtree plus x, r nodes. A valid k guarantees the chosen side is nonempty and its new k valid, so descent never incorrectly enters NIL. Each iteration descends one level and returns the correct node.

One tree: keys, subtree sizes, and the selection path



Keys appear inside circles; s below each circle is subtree size. Green traces SELECT(7): 20→40→30. Rank and deletion examples start from the same tree.

Select(7): subtract only the skipped prefix

Current key	k	left.size	r=left.size+1	Action and new k
20	7	3	4	Right to 40; 7−4=3
40	3	4	5	Left to 30; still 3
30	3	2	3	Return this node

Inorder is 5,10,15,20,25,27,30,35,40,50, confirming key 30 at rank 7. **Subtract the left subtree and current node only when moving right.** Reject k=0 or k>root.size before querying. An empty tree has no valid k.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Rank adds disjoint prefixes while climbing

```
rank(T,x):
    r=x.left.size+1; u=x
    while u!=T.root:
        if u==u.parent.right:
            r+=u.parent.left.size+1
        u=u.parent
    return r
```

Invariant: r is original x's inorder rank inside current u's subtree. If u is a left child, the parent and right side follow x, so add nothing. If u is a right child, its parent's left subtree and parent precede x and lie outside the previously counted u-subtree. Add exactly left.size+1 without duplication. At root, r is the global rank.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Rank(27): which ancestors add a prefix?

Find the rank of node 27, starting with r=0+1=1.

u → parent	u's side	Added prefix	New r
27 → 25	right	size(NIL)+1=1	2
25 → 30	left	0	2
30 → 40	left	0	2
40 → 20	right	size(10)+1=4	6

Each added set lies outside the old u subtree and precedes 27. The sets are disjoint and exhaustive, confirming inorder rank 6. This API requires a valid node belonging to the tree; NIL and removed objects are invalid arguments.

Repair summaries bottom-up, including during rotations

```
PULL(v):                      # v != NIL
    v.size=1+v.left.size+v.right.size
    v.max=max(v.high,v.left.max,v.right.max)
REPAIR-UP(v):
    while v!=NIL:
        PULL(v); v=v.parent
```

This version stores both size and the largest interval high; omit max in a size-only tree. NIL.size=0 and NIL.max=−∞. A new leaf has size 1; after attaching it, repair upward from its parent.

For deletion, finish BST relocation, repair along final parent links, and then run red-black fixup. After changing rotation pointers, pull the lowered node before the promoted node. Recoloring does not affect size/max. Deletion or a shorter interval may decrease max; keeping a historical upper bound is unsafe.

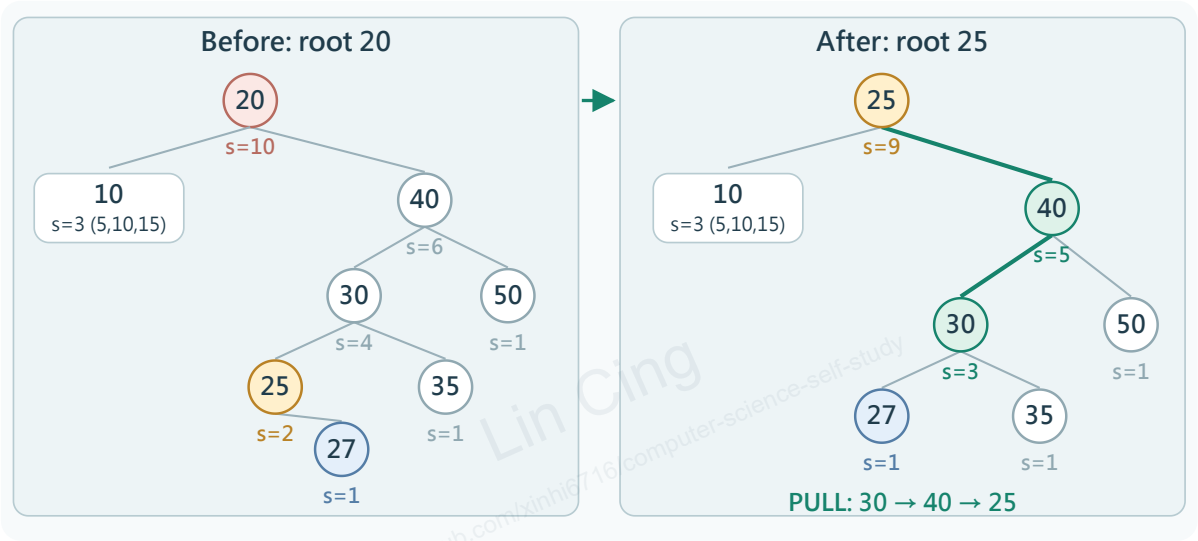
Where does summary repair start after deletion?

In the preceding RB-DELETE/RELOCATE-SUCCESSOR, save a before relocation according to the table. After all transplants, run REPAIR-UP(a), then repair colors.

Branch	Saved a	Why this final chain suffices
z has at most one child	z's old parent	The child's own subtree is unchanged; ancestors lose one record
Successor y is z.right	y	y has a new left subtree; repair upward from y
Successor y is deeper	y's old parent	Its final chain visits the old right subtree, moved y, and z's old ancestors

Never read z.parent after freeing z. NIL.parent is not a universal parent for all empty links. Save a separately as a real node or NIL; NIL.size=0 and NIL.max=−∞ never change.

Delete 20 and relocate 25: track links and sizes



Node 25 keeps its identity;27 replaces it under 30, and 25 takes the old root position. Colors distinguish transplant roles, not red-black colors. Complete BST relocation and summary repair before the preceding red-black fixup.

Pull 30, then 40, then the new root 25

Successor 25 has right child 27. Replace 30.left by 27, then move 25 to the root with old left subtree 10 and right subtree 40.

Repair order	Calculation	New size
30	1+size(27)+size(35)=1+1+1	3
40	1+size(30)+size(50)=1+3+1	5
25	1+size(10)+size(40)=1+3+5	9

The old size of 25, namely 2, is obsolete. Pulling 25 first reads stale 40.size=6 and wrongly produces 10. Merely decrementing z's old ancestors updates nothing here because z was the root.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

When can a rotation update only two summaries?

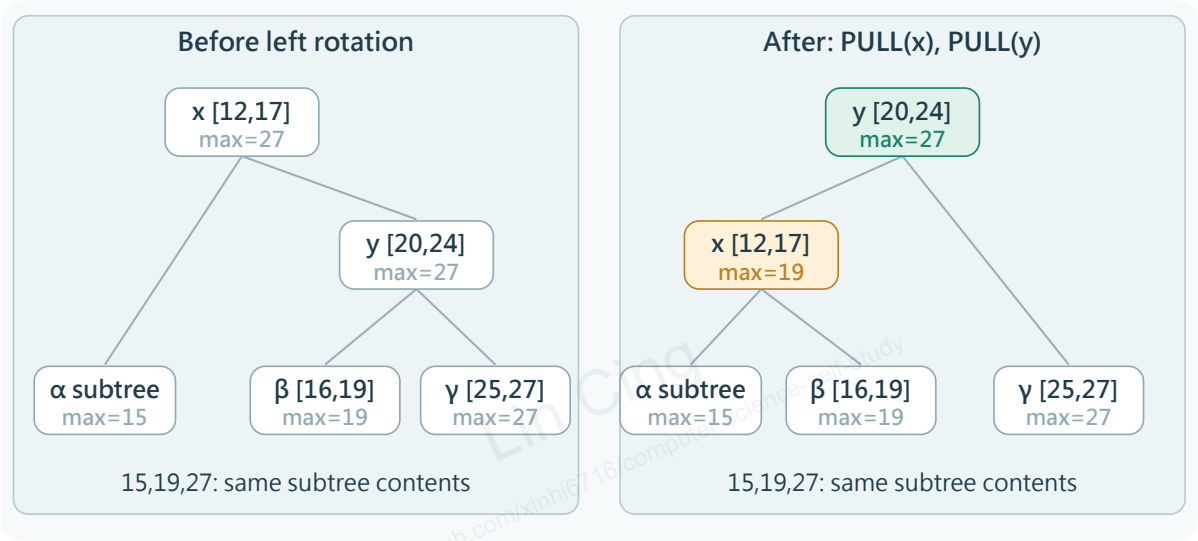
In left-rotate(x), y=x.right rises and β=y.left moves to x. Finish pointer changes, then pull(x) before pull(y), because new y depends on new x. Reversing the order reads stale x information.

Size and maximum interval high depend only on subtree contents. New y contains exactly old x's entire subtree after rotation, so its summary equals old x's summary and other ancestors do not change. These two summaries need O(1) local recomputation.

A local recurrence alone does not imply a two-node update. Height can change at the local root after rotation, requiring ancestor propagation. A general constant-time child-based summary costs O(h+1) per rotation; the constant rotation bounds of red-black updates retain O(log(n+2)) overall. Storing global ranks instead would require Θ(n) explicit writes when a new minimum is inserted.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

The same interval set before and after rotation



First x.max=max(17,15,19)=19; then y.max=max(24,19,27)=27. The new local root still represents the entire old set.

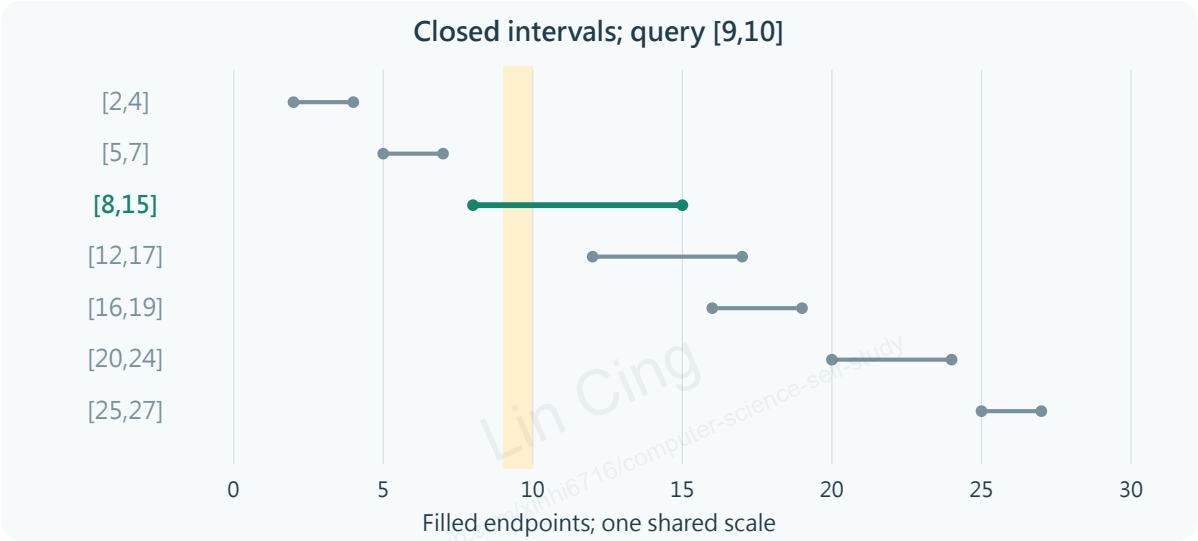
Interval trees: order by low, summarize high

Intervals here are closed [l,h], with l≤h, including singletons [l,l]. Intervals i,q overlap exactly when i.low≤q.high and q.low≤i.high. Equivalently, neither lies completely left of the other: i.high<q.low and q.high<i.low are the two rejection cases.

Order the BST primarily by low; use identity to break equal-low ties, retaining left.low≤x.low≤right.low. Store max(x)=max(x.high,left.max,right.max), with NIL.max=−∞. This is the highest high in the **whole subtree**, not the node's high or the largest low.

Root [12,17] has high 17 but max 27. Left[5,7] has max 15 from right descendant [8,15]. Looking only at the current interval can miss a farther-reaching descendant.

All seven intervals share one coordinate scale



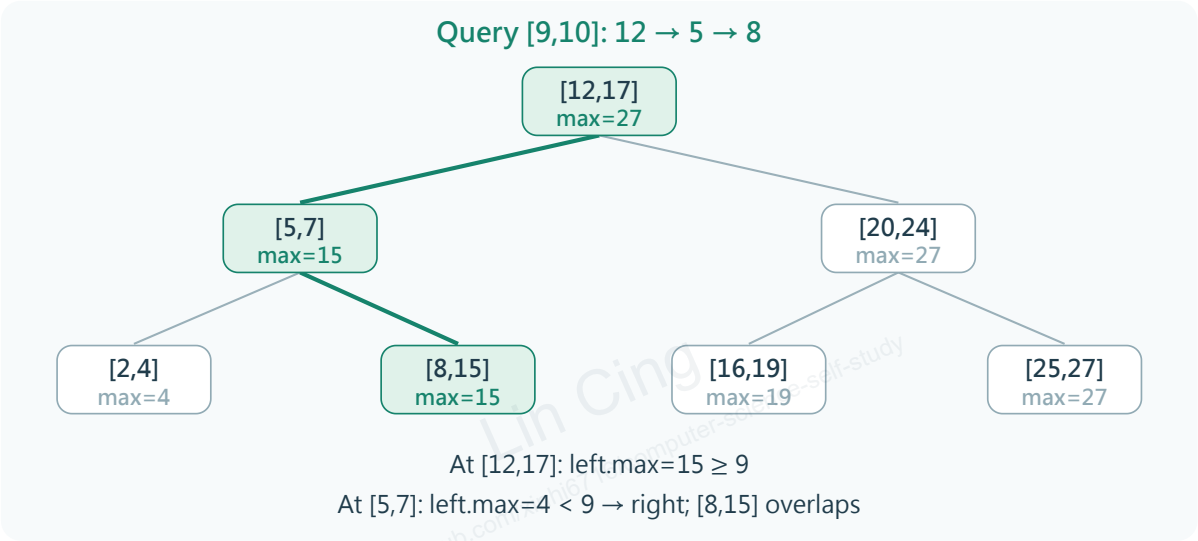
The shaded query is [9,10]; only [8,15] overlaps it. The tree uses exactly these intervals.

Overlap search chooses one subtree at a time

```
interval-search(T,q):
    x=T.root
    while x!=NIL and not overlaps(x.interval,q):
        if x.left!=NIL and x.left.max>=q.low:
            x=x.left
        else:
            x=x.right
    return x
```

This returns any one overlap, not the leftmost or all matches. Prove that every chosen subtree retains an answer whenever the current subtree has one.

One interval tree and the query [9,10] path



Read left.max at the current node: 15≥9 chooses left; then 4<9 chooses right and reaches [8,15].

Three queries with traceable branch conditions

Query [9,10] does not overlap root [12,17], but left.max 15≥9 leads to [5,7]. That also misses; left [2,4].max 4<9 leads right to overlapping [8,15].

Query [18,18] goes from nonoverlapping root right because left.max 15<18. At [20,24], left.max 19≥18 leads to matching [16,19]. Query [28,29] follows [12,17]→[20,24]→[25,27]→NIL and fails.

Going right: rule out the entire left subtree

Maintain: if an overlap exists anywhere in the tree, the current x-subtree contains one. This holds initially at root. Return if x overlaps. Otherwise prove that the chosen direction cannot discard all answers.

If left=NIL, it has no candidate. If left.max<q.low, every left interval j has j.high≤left.max<q.low and lies completely left of the query. Exclude the whole left side. Since x also misses, any answer must be right, preserving the invariant.

This uses a bound on every left descendant's high. The single condition x.high<q.low does not imply every left interval lies too far left. Change only the example root to [12,13] and query [14,14]: root.high 13<14, yet left descendant [8,15] overlaps, so the left side cannot be pruned on that basis.

Going left: why no right-side backtracking is needed

When left.max≥q.low, choose a left interval j attaining that maximum. If left has any overlap, descending left is safe. If it has none, j.high≥q.low while j misses, so j.low>q.high; it cannot have j.high<q.low.

For every right descendant k, BST order gives `k.low≥x.low≥j.low>q.high`. The whole right side also lies too far right. With x itself nonoverlapping, this means the x-subtree had no answer. Thus either way, if an answer existed, the chosen left side retains one.

This proves the one-path algorithm. It does not assert that left.max≥q.low guarantees a left overlap. At NIL the chosen subtree is empty, so the invariant implies global absence. Each step descends once, giving O(height+1), or O(log(n+2)) with a red-black foundation.

An overestimated stale max can miss an answer

Change [8,15] to [8,27], then delete it. The left subtree now contains only [5,7] and [2,4], so its exact max is 7. If it incorrectly remains 27, query [22,22] behaves as follows.

Position	Comparison used	Incorrect choice
[12,17]	Stale left.max=27≥22	Left, abandoning the real answer on the right
[5,7]	left.max=4<22	Into empty right child
NIL	Loop ends	Incorrect failure

The actual answer is [20,24] in the right subtree. With exact left.max=7 the first step goes right and finds it. **This one-path algorithm requires an exact maximum, not just an upper bound.** Underestimates can prune true answers directly.

Recompute max bottom-up after modifications, allowing decreases.

Endpoint and update conventions are part of the contract

Convention	Nonempty condition	overlap(i,q)	Left may contain an answer
Closed [l,h]	l≤h	i.low≤q.high and q.low≤i.high	left.max≥q.low
Half-open [l,h)	l<h	i.low<q.high and q.low<i.high	left.max>q.low

[4,5] and [5,7] meet at 5; [4,5) and [5,7) do not overlap. Half-open [5,5) is empty, unlike a singleton closed interval.

Changing low changes the BST ordering key: delete and reinsert the record instead of editing the field in place. Changing only high preserves order, provided the interval stays nonempty, but requires max repair upward from that node.

One match, leftmost match, and all matches differ

The algorithm promises any overlapping record, not minimum low or all matches. To enumerate all, recurse left only if left.max≥q.low, test x, and recurse right only if x.low≤q.high and right.max≥q.low. Each visited node costs O(1), with worst-case O(n). The one-path O(log(n+2)) bound cannot be transferred to enumeration.

Another provable enumeration method repeats overlap-search, temporarily deletes each returned record, and records it in a list. k matches plus one final failure cost O((k+1)log(n+2)); reinsertion has the same order. This temporarily mutates the tree and must fit the caller's contract. Output alone costs Ω(k). Achieving exactly O(log n+k) requires another analysis or representation; saying “output-sensitive” is no proof.

Size and max can be stored together, each pulled in O(1), with the same maintenance order. Rank then follows BST order on (low,identity), not high order or number of overlaps, which need other summaries or algorithms.

B-trees: use a page to reduce tree height

A node stores several sorted keys and child pointers separating their ranges. Search chooses a range within the current page, then reads another page. Separate **page accesses (I/O)** from **within-page comparisons and movement (CPU)**.

We use a set of distinct keys. Each key travels with its payload or record pointer; internal nodes can store actual records. Duplicate insertion returns EXISTS without adding a record. A B+ tree uses a different representation: records reside in leaves and internal nodes store separators.

Whether one node fits a page depends on page size, key/payload sizes, child pointers, and headers. The I/O analysis assumes this layout requirement holds.

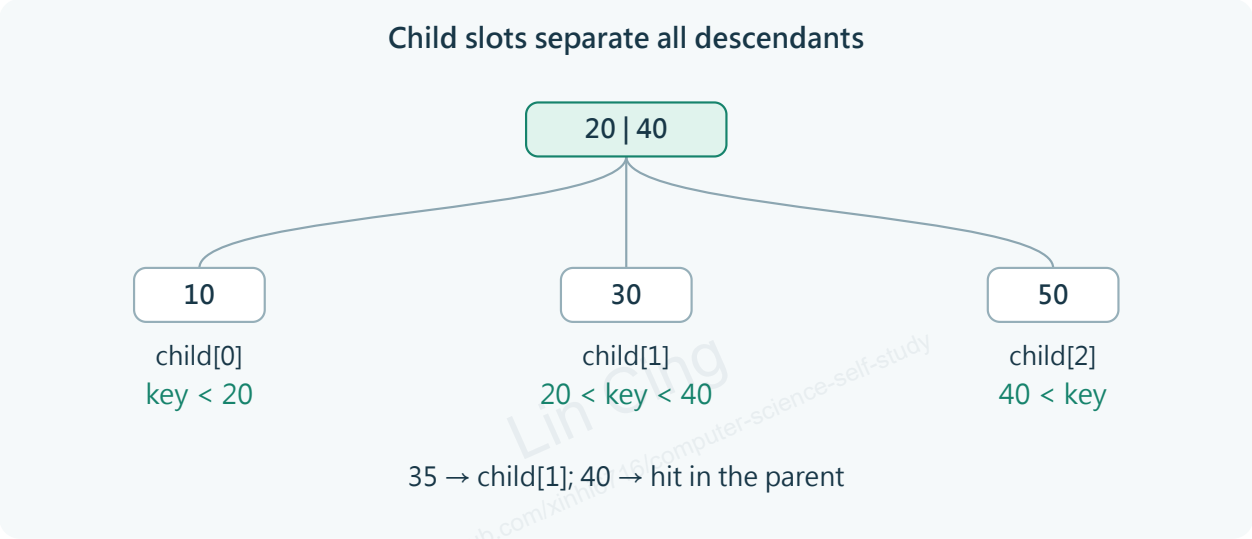
Minimum degree t: fix the occupancy contract

Fix an integer $t \geq 2$. Key and child arrays are zero-based.

Node type	Key count k	Child count
Nonroot	$t-1 \leq k \leq 2t-1$	0 for a leaf; exactly k+1 otherwise
Nonempty root	$1 \leq k \leq 2t-1$	0 for a leaf; at least 2 otherwise
Empty tree	Zero-key leaf root	0

All leaves have the same depth. The root has depth 0 and leaf depth is height h. A nonempty height-zero tree stores everything in its root. We also assign height 0 to the empty tree, but the nonempty height theorem does not apply at n=0.

Every child pointer owns a whole key range



Keys $a_0 < \ldots < a_{k-1}$ divide the subtree into k+1 ranges. Each restriction applies to **every descendant**, not merely the first key of a child. Search for 35 chooses the middle child; search for 40 stops at the parent.

Do not mix order m with minimum degree t

Convention used here	Nonroot internal children	Nonroot keys
Order m means at most m children	⌈m/2⌉…m	⌈m/2⌉−1…m−1
Minimum degree t	t…2t	t−1…2t−1

Capacities match exactly when m=2t. Order 3 gives a 2-3 tree. It is neither t=1.5, which is invalid, nor t=2, which permits four children and gives a 2-3-4 tree.

Books do not use “order” uniformly. Check whether it means maximum children or another capacity before applying a formula. The table explicitly fixes the maximum-child convention.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Preemptive and overflow splitting have different premises

Our insertion splits a full node with 2t−1 keys before descending: promote one key, leaving t−1 keys on each side. Both sides are immediately legal.

An order-m overflow algorithm first permits m temporary keys, then promotes a median. With m=3, inserting 5 into [2,8] temporarily gives [2,5,8]; promote 5 and retain [2] and [8].

Preemptively splitting [2,8] at order 3 leaves one side empty regardless of which key rises, violating the one-key minimum. **Capacity, repair timing, and proof must agree.** Deletion likewise has both repair-before-descent and repair-after-underflow schedules.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Height upper bound: count the sparsest legal tree

Assume $n \geq 1$. The root has at least one key and, if $h \geq 1$, at least two children. Depth $i \geq 1$ contains at least $2t^{i-1}$ nodes, each with at least $t-1$ keys. Thus

$$n \geq 1 + \sum_{i=1}^{h} 2t^{i-1}(t-1) = 2t^h - 1.$$

Rearranging and taking base-t logarithms gives

$$h \leq \log_t \frac{n+1}{2}.$$

For $h=0$ the sum is empty, giving $n \geq 1$ as required. The root minimum is one key; do not count it as another node with $t-1$ keys.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Height lower bound and a billion-key example

Each node has at most $2t-1$ keys and $2t$ children, so

$$n \leq (2t-1)\sum_{i=0}^{h}(2t)^i = (2t)^{h+1} - 1.$$

For $t=100$ and $n=10^9$, height 2 holds at most 7,999,999 keys, too few; height 3 holds at most 1,599,999,999. Height 4 requires at least 199,999,999 keys, while height 5 requires 19,999,999,999, too many. A legal tree therefore has $h=3$ or 4.

Search visits 4 or 5 nodes. With a resident root and no other path pages cached, it performs **3 or 4 page reads**. A nonresident root adds one more read.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Search: locate a slot, then choose the only possible subtree

```
SEARCH(x,k):
    i=lower_bound(x.keys,k)       // first key >= k, or len(keys)
    if i<len(x.keys) and x.keys[i]==k: return (x,i)
    if x.children is empty: return NOT_FOUND
    read x.children[i] if not cached
    return SEARCH(x.children[i],k)
```

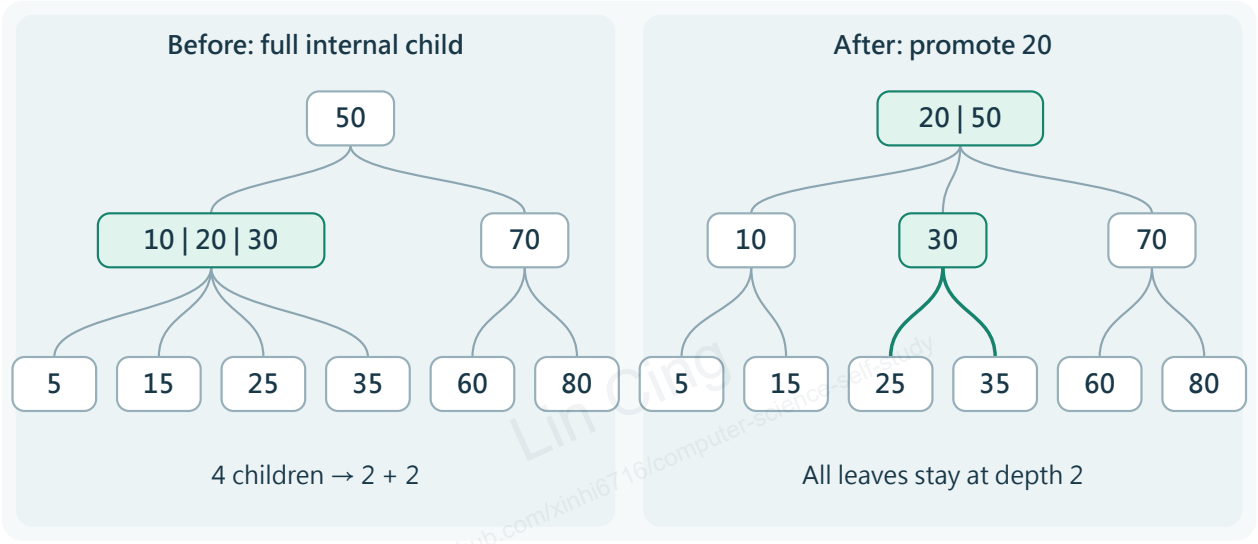
Without an equal key, global range separation leaves only child i as a possible location. Every step descends one level and eventually succeeds or fails at a leaf. This proves termination and correctness. Linear scanning and binary search implement the same slot choice and page path.

SPLIT: move the median and divide the child pointers

```
SPLIT(p,i):                       // p nonfull; child i full
    y=p.children[i]; old=y.keys; kids=y.children
    median=old[t-1]
    z=Node(keys=old[t:], children=kids[t:] if kids else [])
    y.keys=old[:t-1]
    y.children=kids[:t] if kids else []
    p.keys.insert(i,median); p.children.insert(i+1,z)
    write y,z,p
```

Slices include the left endpoint and exclude the right. old and kids refer to the pre-change arrays. Both halves have t−1 keys and, if internal, t children. The median moves to the parent, with **no copy retained in either child**. A nonfull parent can safely gain one key.

Splitting an internal node must preserve all four child subtrees

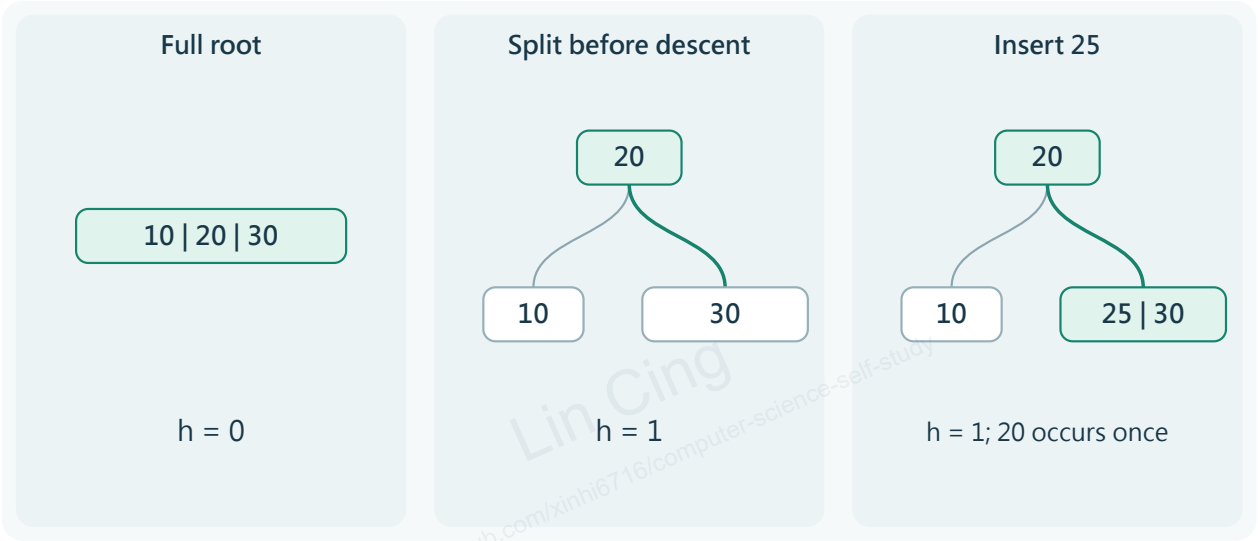


For t=2, parent [50] has full left child [10,20,30]. Promoting 20 gives parent [20,50]. Node [10] keeps [5],[15]; node [30] receives [25],[35]. Old right subtree [70] is unchanged. All leaves remain at depth 2.

Insertion: always descend into a nonfull node

```
INSERT(k):
    if SEARCH(root,k) succeeds: return EXISTS
    if len(root.keys)==2*t-1:
        root=Node([], [root]); SPLIT(root,0)
    x=root
    while x.children is not empty:
        i=lower_bound(x.keys,k)
        if len(x.children[i].keys)==2*t-1:
            SPLIT(x,i)
            if k>x.keys[i]: i+=1
        x=x.children[i]
    insert k in sorted order into x.keys; write x
    return INSERTED
```

Root splitting increases height at the top



For t=2, insert 25 into full leaf root [10,20,30]. Promote 20 first, then choose the right leaf since 25>20, yielding [25,30]. Only root splitting adds a level; all old leaves deepen together and remain at equal depth.

Insertion correctness: why no upward repair is needed

At loop entry, x is nonfull. A full original root is split under a new root, establishing the invariant initially. If the next child is full, nonfull x can accept its promoted key. Both split children have t−1 keys, so the chosen child is again nonfull.

Each iteration descends one level. A leaf gains just one key, at most from 2t−2 to 2t−1, without overflowing. SPLIT preserves keys and subtree ranges; the final new key lies in the range chosen by search. The represented set gains exactly k.

Repair precedes descent, unlike an overflow-based schedule, so insertion need not return to an earlier parent to fix overflow. The separate duplicate check adds another search path but does not change the asymptotic bound.

Deletion: give the next subproblem one key of slack

Ordinary nonroot occupancy is $t-1$ keys. Deletion strengthens the entry condition: **before recursing into a nonroot node, ensure it has at least t keys**. Losing one key at that level then leaves the legal $t-1$ minimum.

Situation	Action
k found in a leaf	Remove it
k found internally; left child has $\geq t$ keys	Replace by predecessor; delete it on the left
Left child lacks t; right has $\geq t$	Replace by successor; delete it on the right
Both adjacent children have $t-1$	Merge both with the parent key, then delete k

Even when k is not found locally, repair its target child before descending. The root is exempt from the stronger minimum; a final zero-key internal root is replaced by its only child.

Borrow left: lower the separator and raise the sibling maximum

```
BORROW-LEFT(p,i):                // i>0
    y=p.children[i]; s=p.children[i-1]
    y.keys.insert(0,p.keys[i-1])
    p.keys[i-1]=s.keys.pop()
    if s.children: y.children.insert(0,s.children.pop())
    write s,y,p
```

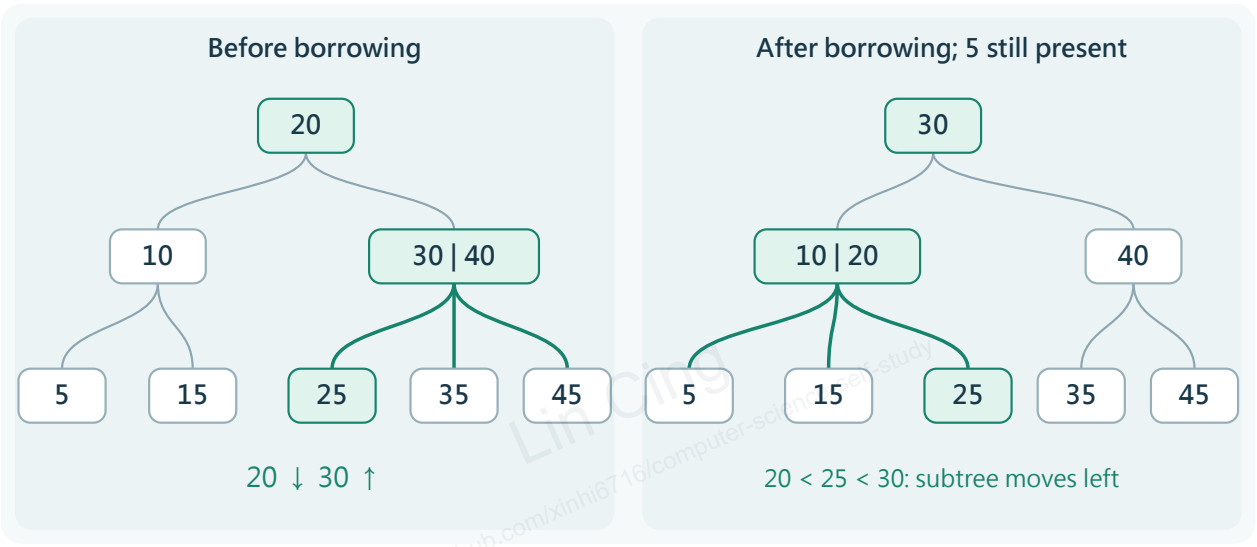
Require $t-1$ keys in y and at least t in s, at the same depth. The rightmost subtree of s lies between the new and old separators and must become the leftmost subtree of y. Afterward y has t keys, s retains at least $t-1$, and the parent key count is unchanged.

Borrow right: move the child pointer to the correct side

```
BORROW-RIGHT(p,i):                // i<len(p.keys)
    y=p.children[i]; s=p.children[i+1]
    y.keys.append(p.keys[i])
    p.keys[i]=s.keys.pop(0)
    if s.children: y.children.append(s.children.pop(0))
    write s,y,p
```

Again require t−1 keys in y and at least t in s. The leftmost child of s becomes the rightmost child of y. Moving keys without moving that pointer assigns an entire key range to the wrong subtree, even if every individual node remains sorted.

Borrowing example: why the 25 subtree must move left



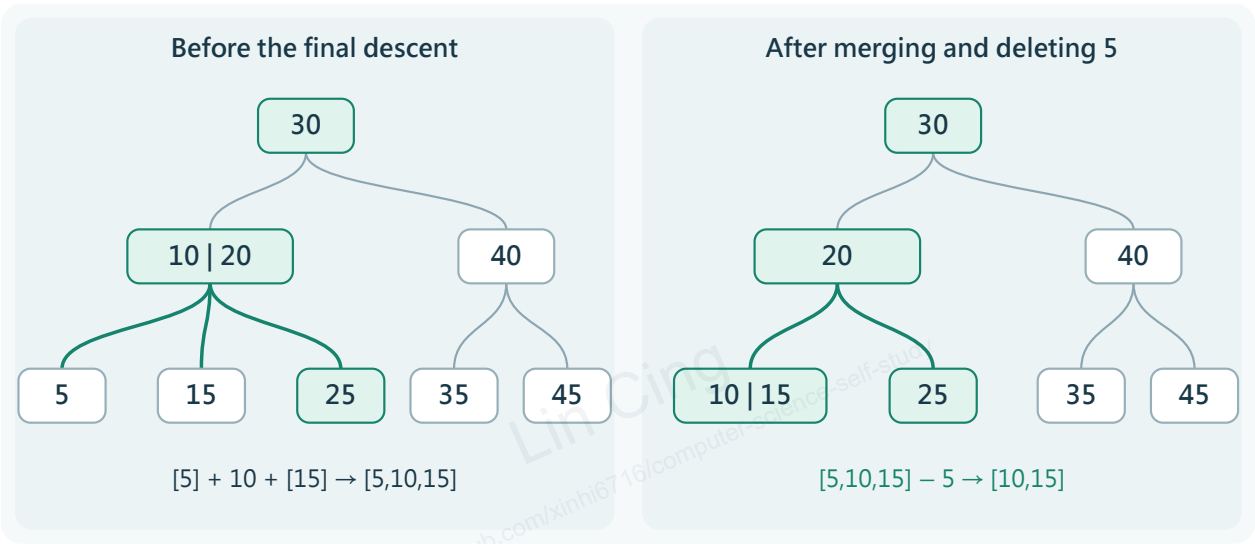
For t=2, delete 5. Left internal node [10] has only one key; right sibling [30,40] has two. First borrow: 20 descends and 30 rises. Subtree [25] lies between 20 and 30 and must become the last child of [10,20]. The figure shows pre-descent repair; 5 has not yet been deleted.

MERGE includes the parent separator

```
MERGE(p,i):                      // adjacent children both t-1 keys
    y=p.children[i]; z=p.children[i+1]
    y.keys.append(p.keys.pop(i))
    y.keys.extend(z.keys)
    y.children.extend(z.children)
    p.children.pop(i+1)
    write y,p; free z
    return y
```

New y has (t−1)+1+(t−1)=2t−1 keys: full but not overflowing. If internal, it has t+t=2t children. The parent loses one key and one child. A nonroot parent entered with at least t keys and retains t−1. An empty root is contracted after the operation.

Continue deleting 5: merge two minimal leaves first



Inside [10,20], target leaf [5] and sibling [15] each have one key. Include parent separator 10 to form [5,10,15], then remove 5, leaving [10,15]. The left internal node is [20] with children [10,15],[25]. Root [30] and right node [40] are unchanged. Inorder loses exactly 5 and all leaves remain at depth 2.

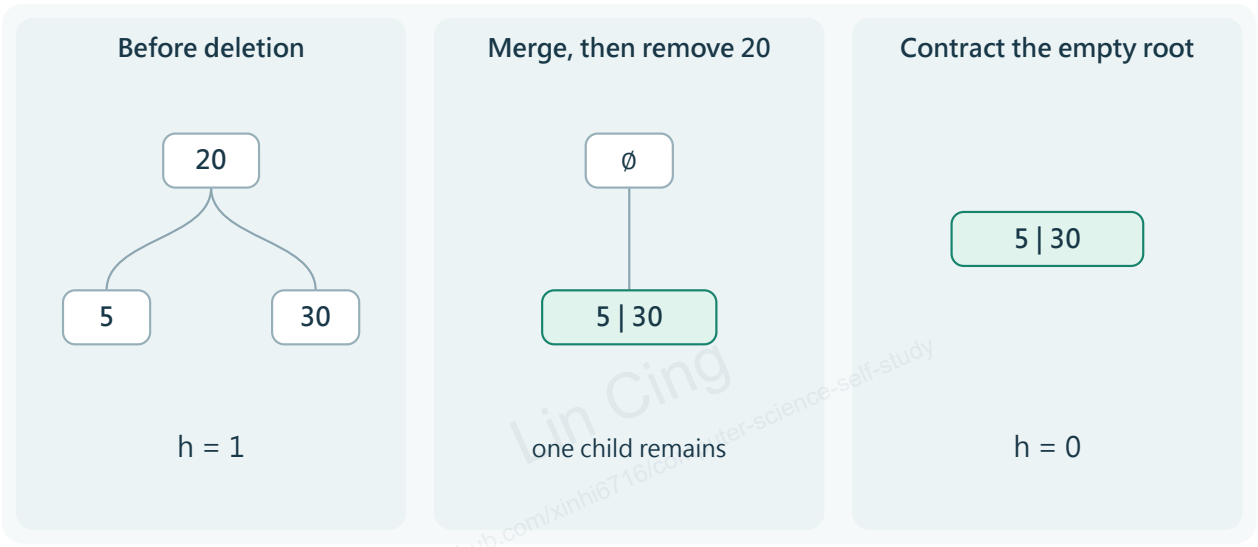
An internal match moves a whole predecessor or successor record

Initial t=2 tree: root and two leaves	Delete	Result
Root [20]; left [5,10], right [30]	20	Predecessor 10 rises: root[10], leaves[5],[30]
Root [20]; left [5], right [30,40]	20	Successor 30 rises: root[30], leaves[5],[40]

Find a predecessor by following the left subtree's rightmost children to a leaf and taking its last key. A successor follows the right subtree's leftmost children and takes the first key. Delete that record from its original subtree; do not retain duplicate parent/leaf copies.

The record pointer must move with its key. Changing only the key while leaving the old payload makes later queries return the wrong record.

Root contraction: only the root may remove a level



Merge root separator [20] with leaves [5],[30] into [5,20,30], then delete 20, leaving [5,30]. The old root has zero keys and one child; replace it by that child, reducing h from 1 to 0. Doing this to an ordinary internal node would shorten only one branch and violate equal leaf depth.

Deletion procedure: the three internal-match cases

```
DELETE-NODE(x,k):
    i=lower_bound(x.keys,k)
    if i<len(x.keys) and x.keys[i]==k:
        if not x.children: erase x.keys[i]; write x; return FOUND
        L=x.children[i]; R=x.children[i+1]
        if len(L.keys)>=t:
            q=MAX-RECORD(L); DELETE-NODE(L,q.key); x.keys[i]=q
        elif len(R.keys)>=t:
            q=MIN-RECORD(R); DELETE-NODE(R,q.key); x.keys[i]=q
        else:
            y=MERGE(x,i); DELETE-NODE(y,k)
        write x; return FOUND
    return DELETE-DESCEND(x,i,k)  // next card
```

Here the keys array contains records; comparisons use their key fields. q must be a saved record value or stable pointer, not an object dereferenced after its original storage is freed.

Deletion descent: borrow first; adjust the index after merging

```
DELETE-DESCEND(x,i,k):
    if not x.children: return NOT_FOUND
    if len(x.children[i].keys)==t-1:
        if i>0 and len(x.children[i-1].keys)>=t:
            BORROW-LEFT(x,i)
        elif i<len(x.keys) and len(x.children[i+1].keys)>=t:
            BORROW-RIGHT(x,i)
        elif i<len(x.keys): MERGE(x,i)
        else: MERGE(x,i-1); i-=1
    return DELETE-NODE(x.children[i],k)
```

Merging left absorbs old child i into child i−1, so decrement i. Public DELETE(k) calls DELETE-NODE(root,k), then **contracts a zero-key internal root on either FOUND or NOT_FOUND**, and returns the original search result.

An absent-key deletion can still change tree shape

Let t=2, root [20], and leaves [5],[30]. Delete 99: choose the right child. It has only t−1=1 key and its sibling cannot lend, so merge into [5,20,30] and change the child index from 1 to 0.

The resulting leaf lacks 99 and returns NOT_FOUND. The wrapper must still contract the empty internal root. The final tree is leaf root [5,20,30]: the set is unchanged, while height falls from 1 to 0.

NOT_FOUND promises no element removal, not an unchanged shape. An API requiring shape preservation on failure can first search for existence and account for that extra path. Empty trees and deletion of the last key retain a zero-key leaf root.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

General deletion proof and two separate cost measures

Induct on remaining height. Leaf removal is correct. Internal replacement deletes the predecessor or successor from its old subtree, removing exactly k overall. Borrowing and merging preserve global order, child counts, and equal leaf depth, while giving the next subproblem at least t keys. A nonroot parent losing a separator retains t−1. Root contraction handles the exception. Every recursive call descends, proving termination.

Only constantly many neighboring pages are touched per level: I/O is $O(h+1)=O(1+\log_t(n+1))$. Array movement costs up to O(t) per level, giving O(t(h+1)) insertion/deletion CPU time. Separate predecessor/successor lookup adds paths totaling O(h+1), not a fresh full-height traversal at every level.

Binary search inside a page gives O((h+1)log t) search comparisons. It does not reduce insertion/deletion array movement to O(log t).

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Fibonacci heaps: defer linking until extract-min

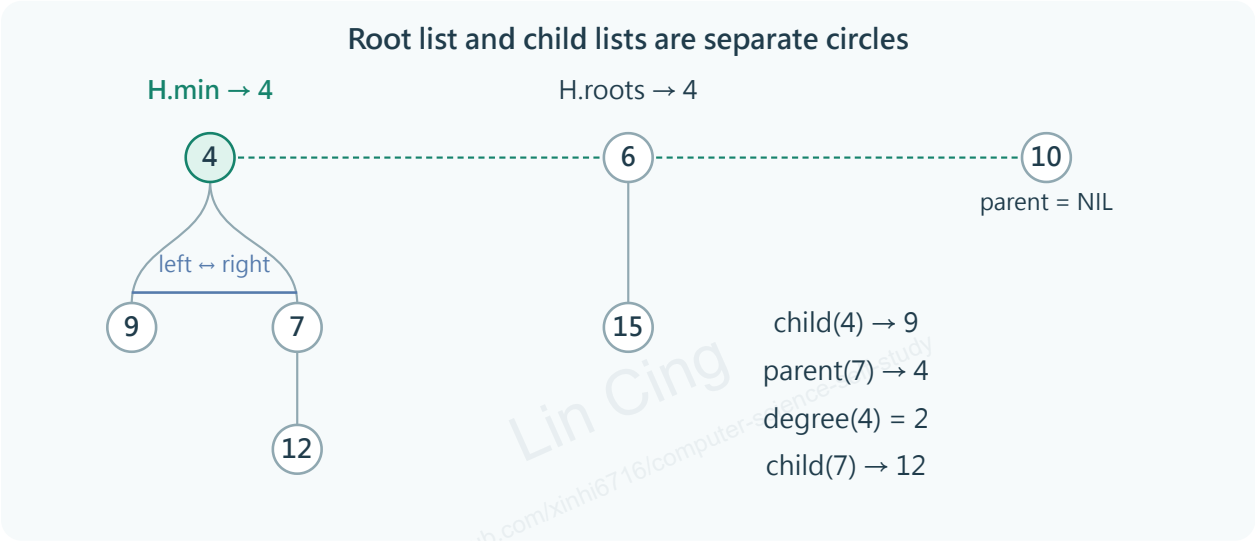
A min-heap requires each parent key to be no greater than its child keys; different subtrees need not be ordered. A Fibonacci heap keeps a list of such trees. Insertion and meld concatenate roots; EXTRACT-MIN later links roots of equal degree.

Degree is the **number of immediate children**, not height. Cuts destroy the exact binomial-tree shape. We will prove degree O(log n); this does not assert logarithmic root-to-leaf paths.

DECREASE-KEY(H,x,k) and DELETE(H,x) receive a valid node handle. Finding x by an arbitrary key may require scanning the whole heap and is excluded from those operation bounds. Equal keys may coexist as distinct nodes.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Separate the root list, child lists, and minimum pointer



Roots are 4,6,10. Node 4 owns 9 and 7; 7 owns 12. Root-list entry and minimum pointer have different roles. Each sibling group has its own circular list.

Node fields and the exact meaning of mark

A node stores key, parent, child, left, right, degree, and mark. The child pointer selects any child; siblings form a circular doubly linked list. A lone child points left and right to itself. H.roots enters the root list, H.min selects a minimum root, and H.n counts all nodes. Both entries are NIL in an empty heap.

A nonroot mark records one child loss since its current attachment. A second loss cuts the node. Our convention clears marks whenever nodes become roots or are linked as children; every root is unmarked.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Circular lists: the mechanism behind constant-time meld

To remove a known node x, connect x.left.right=x.right and x.right.left=x.left. Replace the list entry if it names x; a singleton leaves NIL. Finally isolate x by setting both sibling pointers to itself. Only O(1) pointers change.

For nonempty circles entered at a and b, save al=a.left and bl=b.left. Set al.right=b, b.left=al, bl.right=a, and a.left=bl. Empty-list cases simply choose the other entry.

Below ATTACH-ROOT(H,x) takes an isolated node, splices it into the root list, and sets parent=NIL and mark=false. REMOVE-CHILD(p,x) repairs p.child, sibling links, and degree. These primitives neither search by key nor traverse every sibling.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

INSERT, MINIMUM, and destructive UNION

Operation	Complete action	Actual cost
MAKE	roots=min=NIL; n=0	O(1)
INSERT(x)	New singleton root; update min; increment n	O(1)
MINIMUM	Return min, or NIL if empty	O(1)
UNION(H1,H2)	Splice root circles, choose the smaller non-NIL min, add counts	O(1)

UNION consumes two disjoint heaps. The result owns all their nodes; the old entries must not be used independently. H1=H2 is forbidden. Two empty inputs produce an empty result. This bound excludes rewriting a per-node heap-owner field; an implementation demanding that work cannot claim constant-time meld.

EXTRACT-MIN: promote children, then consolidate

```
EXTRACT-MIN(H):
    z=H.min
    if z==NIL: return NIL
    while z.child!=NIL:
        c=z.child
        REMOVE-CHILD(z,c)
        ATTACH-ROOT(H,c)
    REMOVE-ROOT(H,z)
    H.n-=1
    if H.roots==NIL: H.min=NIL
    else: CONSOLIDATE(H)
    return z
```

Repeatedly remove the current child rather than following sibling pointers invalidated by mutation. Promotion clears parent and mark; isolate z before returning it. One original root does not imply an empty result: its promoted children may remain.

CONSOLIDATE: leave at most one root of each degree

```
CONSOLIDATE(H):
    A=empty degree table; H.min=NIL
    while H.roots!=NIL:
        x=POP-ROOT(H)              // isolate one root
        d=x.degree
        while A[d]!=NIL:           // absent slot means NIL
            y=A[d]; A[d]=NIL
            if y.key<x.key: swap(x,y)
            LINK-ISOLATED(y,x)      // y.parent=x; y.mark=false
            d=x.degree
        A[d]=x
    for each non-NIL x in A:
        ATTACH-ROOT(H,x)
        if H.min==NIL or x.key<H.min.key: H.min=x
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

LINK and the degree-table contract

LINK-ISOLATED joins equal-degree isolated roots: the larger-key y becomes a child of x, increasing x.degree by one. Ties choose either parent. A can be a growing array using O(D(n)+1) slots. Draining the root list avoids revisiting old roots that have already been linked.

```
LINK-ISOLATED(y,x):
    require x!=y; x.degree==y.degree
    require x.key<=y.key; both roots isolated
    y.parent=x; y.mark=false
    splice y into x.child circular list
    x.degree+=1
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Consolidation invariant and termination

At each outer-loop entry, unprocessed roots remain in H.roots; processed nodes lie in disjoint subtrees named by A. Each occupied slot stores one tree whose degree equals the index. Every original node belongs to exactly one of these parts.

The inner loop maintains d=x.degree. Linking an equal-degree y preserves the node set and heap order, replaces two roots by one, and increments d. Every link decreases the finite forest root count, so the loop terminates.

Each outer iteration removes an unprocessed root. At termination A represents all remaining trees, with distinct root degrees. Rebuilding the root list and comparing its roots gives the correct minimum. Distinct degrees are guaranteed only immediately after consolidation; INSERT, UNION, and CUT may create duplicates again.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

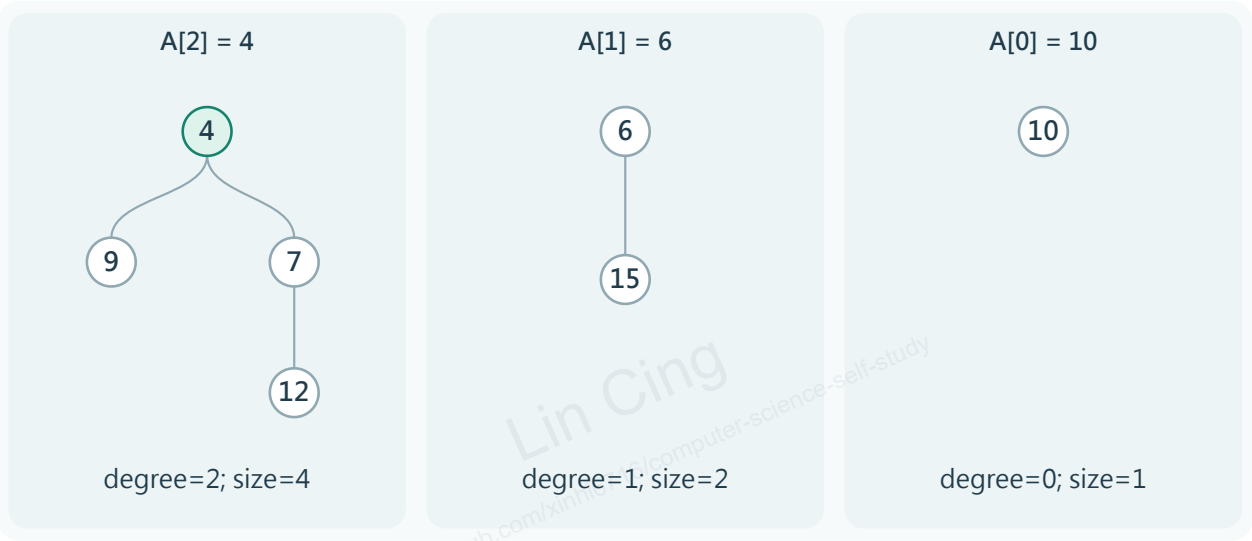
A worked consolidation: follow every degree collision

Start with eight singleton roots 1,4,9,7,12,6,15,10. Extract 1, then process the remaining roots in order 4,9,7,12,6,15,10; each starts with degree 0.

Root taken	Links in this iteration	Occupied A slots (degree:root)
4	None	0:4
9	Attach 9 under 4	1:4
7	None	0:7, 1:4
12	Attach 12 under 7; then 7 under 4	2:4
6	None	0:6, 2:4
15	Attach 15 under 6	1:6, 2:4
10	None	0:10, 1:6, 2:4

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Verify the final forest, degrees, and root count



Root degrees are 2,1,0. Subtree sizes total 4+2+1=7 and min=4. Other roots need not be ordered by key.

The same extraction: fewer roots release stored potential

Root 4 has children 9 and 7; node 7 retains child 12. Root 6 has child 15, and root 10 has no children. Their degrees are 2,1,0, with 4+2+1=7 nodes, and every parent key is no greater than its child key.

Stage	Roots r	Marks m	Potential K(r+2 m)
Before extracting 1: eight singleton roots	8	0	8 K
After removing 1 and completing four links	3	0	3 K

Potential falls by 5 K: removing the minimum eliminates one root and the four links eliminate four more. This connects the example to the amortized formula. **5 K is released potential, not an exact count of five machine instructions.**

These happen to be uncut binomial trees. After CUT, degree d no longer guarantees 2^d nodes; the general lower bound will be F_{d+2}.

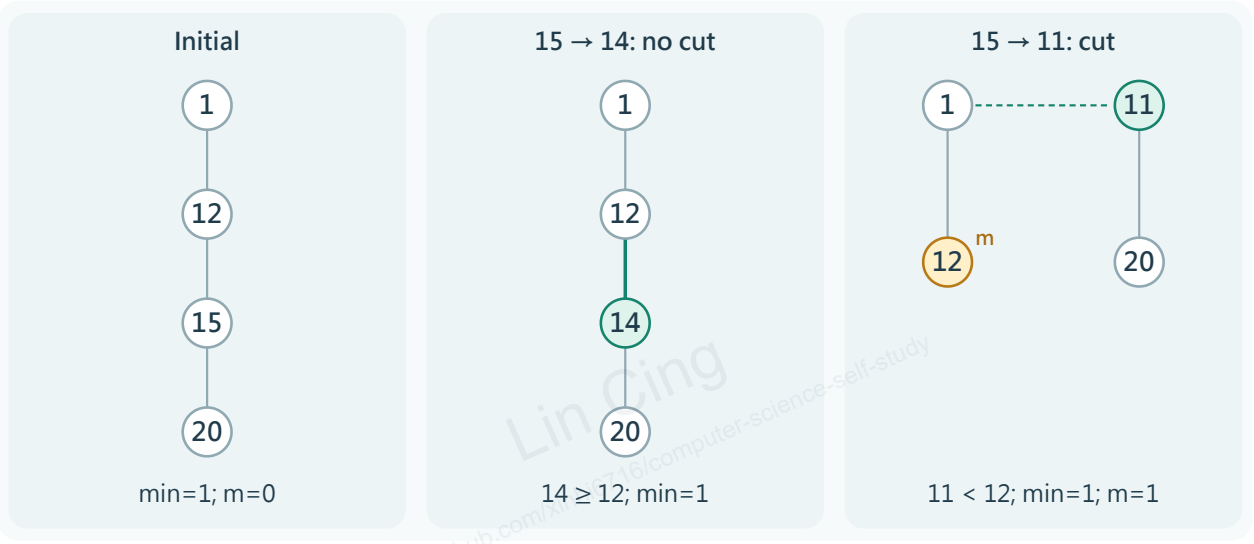
DECREASE-KEY: cut only when parent order is violated

```
DECREASE-KEY(H,x,k):
    if k>x.key: error
    x.key=k; p=x.parent
    if p!=NIL and x.key<p.key:
        CUT(H,x,p)
        CASCADING-CUT(H,p)
    if x.key<H.min.key: H.min=x

CUT(H,x,p):
    REMOVE-CHILD(p,x)       // degree decreases by one
    ATTACH-ROOT(H,x)       // parent=NIL; mark=false
```

Lowering x cannot violate its order relative to its children; only parent→x may fail. A root or a new key still at least its parent key needs no cut. The new minimum is either the old minimum or x, since no other key changed.

Worked decrease: 15→14 stays; 15→11 cuts the subtree



Start both cases from the left tree. Key 14 stays; key 11 is cut with child 20 still attached. Nonroot 12 records its first child loss.

Check both decreases field by field

Root 1 has child 12, which has child 15, which has child 20. Start each case from that same tree. Decreasing 15→14 keeps 14≥12 and changes no links. Decreasing 15→11 violates 11<12 and cuts the **entire subtree**: 20 remains a child of the node now keyed 11.

In the second case, an initially unmarked nonroot 12 becomes marked after its first child loss. Root 1 is unaffected. New root 11 exceeds the old minimum 1, so min stays. Decreasing an existing root, such as 3→0, updates min without a cut.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

CASCADING-CUT: mark the first loss, cut on the second

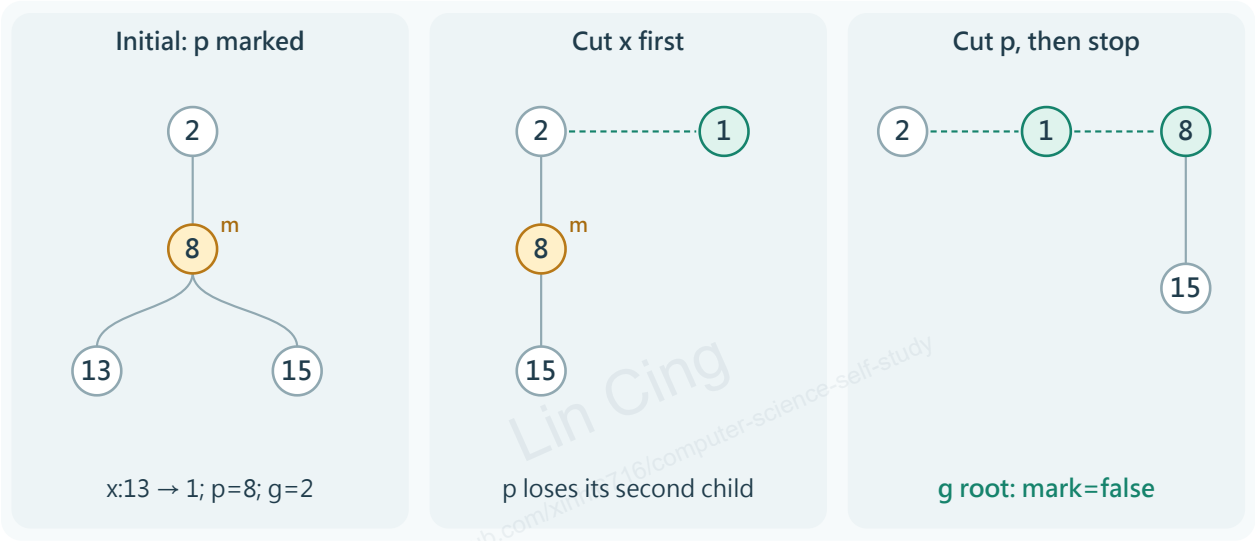
```
CASCADING-CUT(H,y):
    while y.parent!=NIL:
        p=y.parent
        if y.mark==false:
            y.mark=true
            return
        CUT(H,y,p)
        y=p
    return                      // root: stop, do not mark
```

At entry y has just lost a child. If unmarked, record its first loss since this attachment and stop. If marked, this is its second loss: cut y, clear its mark, and continue at p, which has now lost y.

Deleting edges preserves heap order on all remaining edges. Cascading cuts limit how many children a nonroot can lose, preserving the relation between degree and subtree size; they do more than repair key order.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Cascading branch one: stop at a root grandparent



Cut x, then marked p. Stop at root g without marking it. Child 15 remains attached to p.

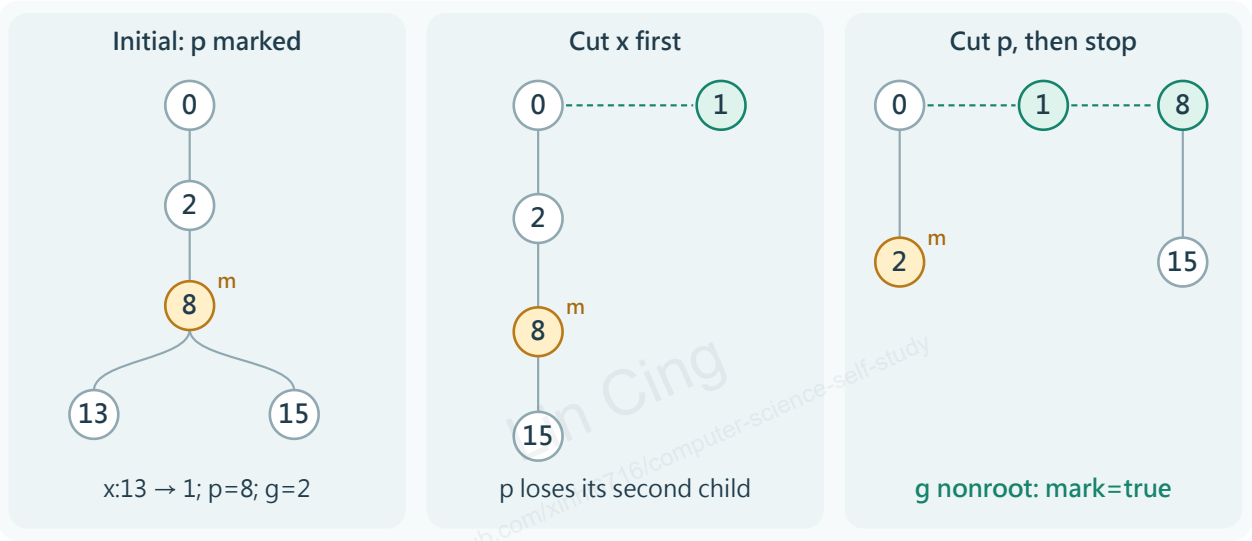
Root branch: root list and potential ledger

Initially the sole root is g=2. Its marked child p=8 has children x=13 and s=15. The mark records a previous child loss. Now decrease x from 13 to 1.

Cut x, then cut already-marked p. At g, parent=NIL, so stop and **do not mark root g**. Final roots are 2,1,8; node 8 still owns child 15. The minimum is 1.

With r roots and m marked nodes, (r,m) changes from (1,1) to (3,0). Unscaled potential r+2m changes from 3 to 3. Clearing p's mark offsets the two additional roots.

Cascading branch two: mark an unmarked nonroot grandparent



Here g has parent q and starts unmarked. After cutting x and p, mark g and stop. Root q=0 remains the minimum.

Nonroot branch: clear p, then mark g

Add root q=0 above the previous local structure, making g=2 an unmarked nonroot; p=8 remains marked. Decrease x=13 to 1. Cut x, then p. Now g.parent=q and g.mark=false, so mark g and stop.

Final roots are q=0, x=1, p=8. Node q retains g=2, and p retains s=15. The minimum stays 0. Roots rise 1→3, while marks stay 1→1: p is cleared and g is newly marked. Potential rises 3→5.

If g had already been marked, cut g as well and continue at q. **Parent and mark determine the endpoint**; marking the grandparent is not an unconditional outcome.

One potential pays for every operation

Choose one fixed constant K>0 and sum over all live, pairwise disjoint heaps:

$$\Phi = K \sum_{H} \big(r(H) + 2m(H)\big), \qquad \hat{c}_i = c_i + \Phi_i - \Phi_{i-1}.$$

Choose K once, sufficiently large to cover the constant-time pointer primitives below. Do not change it between operations. Starting empty gives $\Phi_0=0$ and nonnegative final potential, so $\Sigma c_i \le \Sigma \hat{c}_i$. A preexisting forest adds initial Φ_0 to the sequence bound.

INSERT adds one root and no mark, giving $\Delta\Phi=K$ and O(1) amortized cost. MINIMUM and MAKE leave potential unchanged. UNION replaces two heaps by their disjoint union; root and mark counts add, so global potential is unchanged and both actual and amortized meld costs are O(1).

Why a long cascade has constant amortized cost

With no cut, actual work and potential change are O(1). Otherwise let $c \ge 1$ count every cut, including the first cut of x. Each of the remaining $c-1$ cuts clears a marked nonroot. At most one final nonroot becomes newly marked; any old mark on x is also cleared. Hence

$$\Delta r = c, \qquad \Delta m \le 1 - (c-1) = 2 - c,$$
$$\Delta\Phi \le K(4-c).$$

For a fixed constant a, actual work is at most $a(c+1)$. Choose $K \ge a$:

$$\hat{c} \le a(c+1) + K(4-c) \le a + 4K = O(1).$$

This holds for every legal operation sequence without a random-input assumption. One actual cascade can still follow a long path and take linear time; O(1) is the amortized bound.

Extract-min analysis: many roots release stored potential

Before extraction let n be the node count, r the root count, and D=D(n) a maximum-degree bound. Promoting the removed root's children leaves R≤r−1+D roots. Every link removes a root, so there are at most R links. Child promotion, root processing, and degree-table initialization/rebuilding cost at most a(r+D+1).

After consolidation at most D+1 roots remain. Promotion and linking only clear marks, so

$$\Delta\Phi \le K(D+1-r).$$

$$\hat{c} \le a(r+D+1) + K(D+1-r) \le (a+K)(D+1).$$

Use the same K≥a; empty and singleton heaps cost O(1). This proves O(D+1), not yet logarithmic time: the degree theorem must supply D=O(log(n+1)). After n singleton insertions, the first extraction can take Θ(n) actual time.

The degree lemma: each retained child loses at most one child

Let x currently have d children $y_1, \dots, y_d$, ordered by their **most recent attachment times** to x. For i≥2, all earlier children that still survive were already attached when y_i joined. Thus x then had degree at least i−1.

LINK joins equal-degree roots, so y_i also had degree at least i−1 at attachment. Losing a second child would have cut it from x. Since y_i still remains, it has lost at most one child:

$$\deg(y_1) \ge 0, \qquad \deg(y_i) \ge i-2 \quad (2 \le i \le d).$$

Current sibling degrees need not be sorted. Nor is a nonroot limited to one child loss over its entire lifetime: cutting and later relinking resets the relevant history.

From child degrees to a Fibonacci subtree-size bound

Use strong induction on subtree node count to prove size(x)≥F_{d+2} for every node of degree d. Child subtrees are smaller, so their induction hypotheses apply. Combine the preceding degree lemma with monotonicity of Fibonacci numbers.

$$F_0 = 0, \quad F_1 = 1, \quad F_j = F_{j-1} + F_{j-2}\ (j \geq 2).$$

For d=0, size≥1=F_2. For d=1, x and one child give size≥2=F_3. For d≥2, the first child contributes at least one node and the remaining children give

$$\text{size}(x) \geq 2 + \sum_{i=2}^{d} F_i = F_{d+2}.$$

The identity follows from $\sum_{i=0}^{d} F_i$=F_{d+2}−1: at d=0 it is 0=F_2−1; adding F_{d+1} and using the recurrence proves the next case. Inducting on subtree size avoids assuming an unproved monotonicity of minimum subtree size by degree.

Finish the degree bound: exponential size implies logarithmic degree

Let ϕ=(1+$\sqrt{5}$)/2, so ϕ^2=ϕ+1. Prove F_{d+2}≥φ^d: the bases are 1≥1 and 2≥ϕ. For d≥2,

$$F_{d+2} = F_{d+1} + F_d \geq \varphi^{d-1} + \varphi^{d-2} = \varphi^d.$$

For any node in a nonempty n-node heap, n≥size(x)≥F_{d+2}≥φ^d, giving $d \leq \lfloor \log_\varphi n \rfloor$. Maximum degree, degree-table size, the post-consolidation root count, and amortized EXTRACT-MIN cost are therefore O(log(n+1)).

Degree d	0	1	2	3	4	5	6
Guaranteed minimum size F_{d+2}	1	2	3	5	8	13	21

This is a guaranteed lower bound, not a claim that a specified operation sequence attains equality for every d. Handle the empty heap separately rather than taking log 0.

DELETE: make the designated node uniquely minimum

DELETE(H,x) may first decrease x to a special sentinel below every ordinary comparison key, then call EXTRACT-MIN. For a nonroot, DECREASE-KEY correctly handles the parent's child loss and any cascade. Extraction promotes x's children and consolidates.

Writing “minus infinity” while allowing another node with the same comparison value is insufficient: min could still name the wrong node. Extend comparison to (tag,key), normally with tag=1; only the deletion target temporarily has tag=0. It becomes uniquely minimum. Restore its original key after returning the removed node.

Adding both operations under the same potential cancels the intermediate potential, giving amortized O(1)+O(log(n+1)). Direct internal-node removal is another possible implementation, but it must specify child promotion, parent marks, and cascading repair; promotion alone is incomplete.

Operation bounds and their conditions

Operation	Actual per-operation upper bound	Amortized upper bound
MAKE, MINIMUM, INSERT, UNION	O(1)	O(1)
DECREASE-KEY with a handle	O(n+1)	O(1)
EXTRACT-MIN, DELETE with a handle	O(n+1)	O(log(n+2))
SEARCH by an arbitrary key	O(n+1)	Heap order gives no automatic improvement

The logarithmic bound uses log(n+2) to retain constant cost when n=0. Valid-handle DELETE and DECREASE-KEY require a nonempty heap; empty EXTRACT-MIN is legal and returns NIL. Here n is the relevant heap size; UNION consumes disjoint input heaps. Bounds exclude allocator costs, per-node ownership relabeling, and locating a handle.

Turn operation counts into a graph-algorithm bound

A graph algorithm using O(V) insertions/extractions and O(E) handle-based decreases spends O(E+V log(V+1)) total heap time. Constants, memory usage, and access patterns still affect implementation choices; asymptotic bounds are not measured speed.

For V=1000 and E=8000, suppose each vertex is inserted/extracted at most once and each edge triggers at most one decrease:

Work	Maximum calls	Total Fibonacci-heap amortized bound
INSERT	1000	O(V)
EXTRACT-MIN	1000	O(V log(V+1))
DECREASE-KEY	8000	O(E)

Adding gives O(E+V log(V+1)); the corresponding binary-heap bound is O((E+V)log(V+1)). Counts are call limits, not milliseconds or exact costs: constants differ between representations.

Use integer bits to change the search model

A red-black tree compares keys; a **van Emde Boas tree (vEB)** also uses their bit representation. Fix $S \subseteq \{0, \dots, u-1\}$ with $u = 2^m$, $m \geq 1$, and distinct keys. **n=|S| and universe size u are different parameters.**

Use a word RAM with keys and addresses fitting in one word. Shifts, masks, comparisons, and array indexing cost O(1); arbitrary-precision bit complexity is a different model.

Membership, updates, predecessor, and successor take worst-case $O(1 + \log\log u)$; extrema take O(1). Comparison-sorting lower bounds do not prohibit this: integer operations and initialization/storage costs are additional assumptions.

What direct addressing and naive recursion miss

A bit vector supports membership and updates in O(1), but scanning for the successor of 0 in $S = \{0, u-1\}$ can take Θ(u). A binary summary tree improves this query to O(log u).

Recursively storing roughly $\sqrt{u}$ clusters and their nonempty indices is not sufficient. Finding a cluster and then recursively finding its minimum gives

$$T(u) = 2T(\sqrt{u}) + O(1), \quad R(m) = 2R(m/2) + O(1) = \Theta(m).$$

The result is still Θ(log u). The crucial requirement is **at most one nonconstant recursive call per level**. Cached extrema make that possible.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Odd bit widths need unequal high and low parts

Let $\ell = \lfloor m/2 \rfloor$, $h = \lceil m/2 \rceil$, $L = 2^{\ell}$, and $H = 2^{h}$, so u=HL. For u>2:

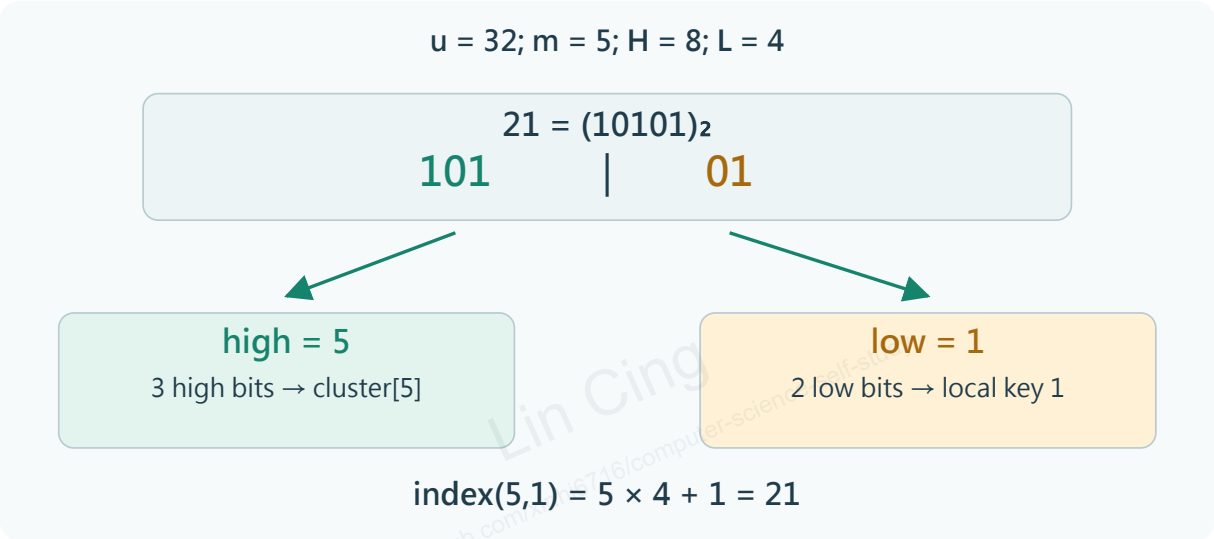
$$\text{high}(x) = \lfloor x/L \rfloor, \quad \text{low}(x) = x \bmod L, \quad \text{index}(i, j) = iL + j.$$

There are H clusters, each with universe L. The summary stores cluster indices and has universe H. **Recompute the split from the local u at every level.**

For u=32: m=5, h=3, ℓ=2, H=8, L=4. Split $21 = (10101)_2$ as $101 \mid 01$: high=5, low=1, and 5×4+1=21. The u=2 base has no recursive structures.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Two coordinates of the same key



high selects a cluster; low is the key passed into it. A 5 in the summary means cluster 5 is nonempty, not that the original set contains key 5.

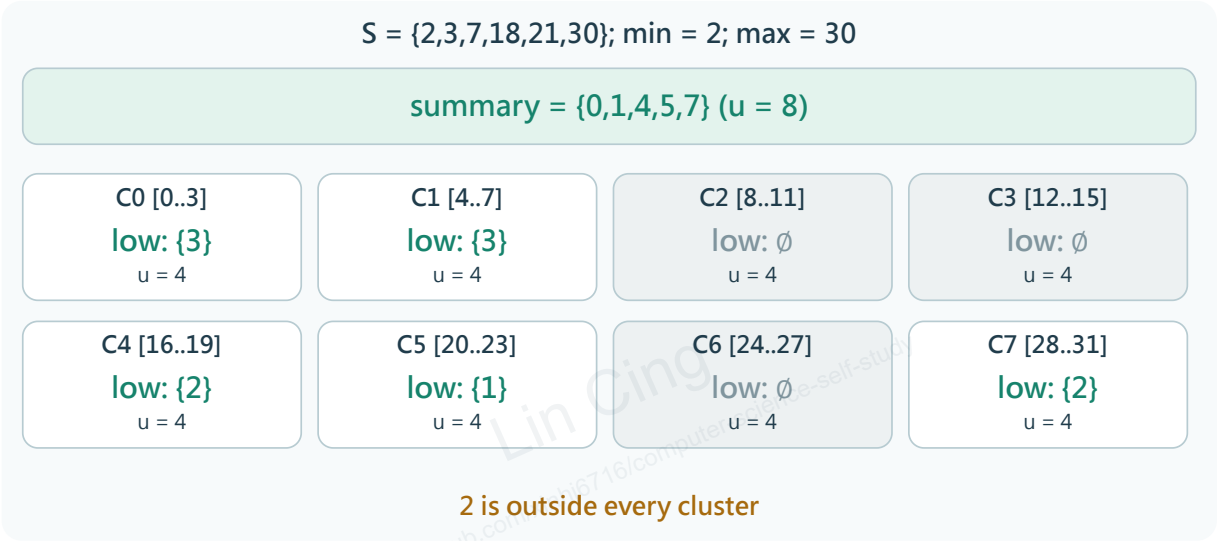
Representation invariant: the minimum stays outside clusters

Each node stores u, min, and max; for u>2 it also owns a summary and cluster[0..H−1]. Use a fully preallocated structure. Empty nodes have min=max=NIL.

For a nonempty set, min is stored **outside all clusters**. Every other key is represented by its low part in the cluster selected by its high part. A singleton has empty clusters and summary. With at least two keys, max also appears in a cluster.

The summary contains exactly the nonempty cluster indices: $i \in \text{summary} \iff C_i.\min \neq \text{NIL}$. Thus min=max detects a singleton in O(1), and inserting into an empty node sets min=max=x without descending.

A u=32 set: distinguish original keys, offsets, and summary



The set is {2,3,7,18,21,30}; top-level min=2 and max=30. C_0 stores only offset 3, not 2. The summary is {0,1,4,5,7}. Only the top level is expanded; every child obeys the same invariant.

Membership and valid arguments

```
MEMBER(V,x):                 # require 0<=x<V.u
    if V.min==NIL: return false
    if x==V.min or x==V.max: return true
    if V.u==2: return false
    return MEMBER(V.cluster[high(x)],low(x))
MINIMUM(V): return V.min
MAXIMUM(V): return V.max
```

high/low use V's local dimensions. Membership follows one cluster; extrema can stop the search early. NIL is a distinct state, never the valid key 0.

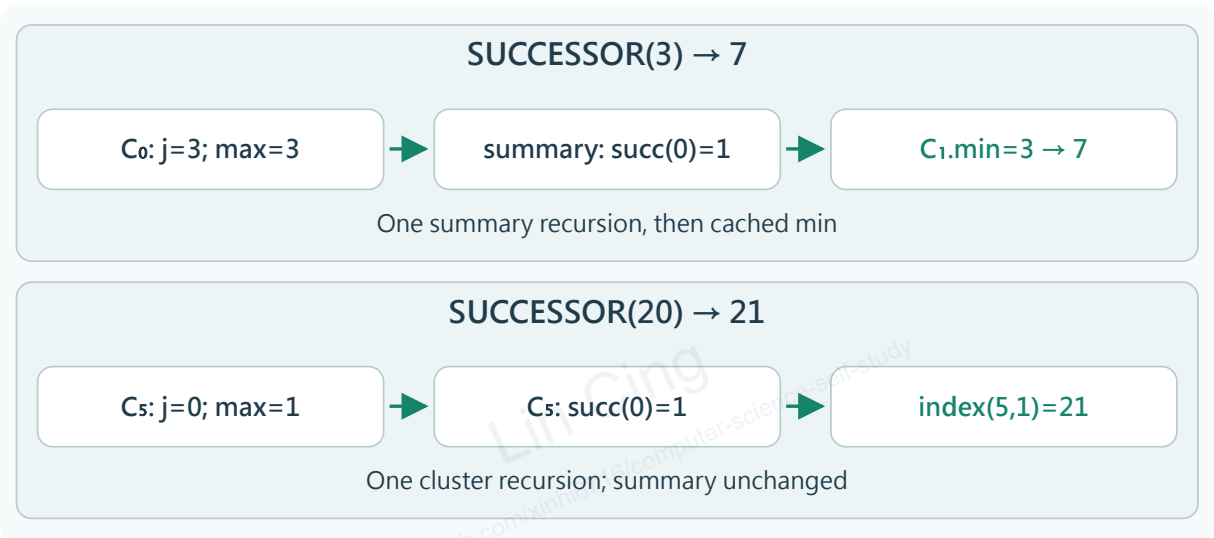
Successor is the smallest stored key **strictly greater than x**; predecessor is the largest strictly smaller key. Queries need not be members but must be inside the universe. Public interfaces reject invalid arguments before array access.

Successor: max chooses the recursive branch

```
SUCCESSOR(V,x):
    if V.min==NIL: return NIL
    if x<V.min: return V.min
    if x>=V.max: return NIL
    if V.u==2: return 1
    i=high(x); j=low(x); C=V.cluster[i]
    if C.max!=NIL and j<C.max:
        return index(i,SUCCESSOR(C,j))
    k=SUCCESSOR(V.summary,i)
    if k==NIL: return NIL
    return index(k,V.cluster[k].min)
```

If the current cluster's max exceeds j, recurse there. Otherwise skip directly to the summary. **Do not perform a failed recursive search first.** Reading the selected cluster's min then costs O(1).

Successor traces from one fixed state



Cached max chooses the cluster or summary branch; never both.

Check every successor coordinate

Use {2,3,7,18,21,30}. SUCCESSOR(3): i=0,j=3 and C_0.max=3, so query the summary. Its successor index is 1; C_1.min=3 gives $1 \times 4+3=7$.

SUCCESSOR(20): i=5,j=0 and C_5.max=1>0, so recurse only into C_5. Offset 1 reconstructs 21. Query 20 need not be stored. SUCCESSOR(30)=NIL because the order is strict.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Predecessor must include the detached minimum

```
PREDECESSOR(V,x):
    if V.min==NIL: return NIL
    if x>V.max: return V.max
    if x<=V.min: return NIL
    if V.u==2: return 0
    i=high(x); j=low(x); C=V.cluster[i]
    if C.min!=NIL and j>C.min:
        return index(i,PREDECESSOR(C,j))
    k=PREDECESSOR(V.summary,i)
    if k==NIL: return V.min
    return index(k,V.cluster[k].max)
```

When k=NIL, return the detached min: earlier guards already established x>V.min, and no lower cluster exists. For example PREDECESSOR(3)=2 even though C_0 has no smaller offset. A symmetric copy of successor would miss 2.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Insert: swap out the old min and update only necessary summaries

```
INSERT-ABSENT(V,x):          # require x not in V
    if V.min==NIL: V.min=V.max=x; return
    if x<V.min: swap(x,V.min)
    if V.u>2:
        i=high(x); j=low(x); C=V.cluster[i]
        if C.min==NIL:
            INSERT-ABSENT(V.summary,i)
            C.min=C.max=j
        else: INSERT-ABSENT(C,j)
    if x>V.max: V.max=x
```

A smaller new key becomes min; **the old min** is then inserted below. An empty cluster requires a recursive summary insertion but only O(1) singleton initialization. A nonempty cluster already has its index in the summary, so recurse only into that cluster. The expensive branches are exclusive.

Insert 1: keep the new min outside and place the old min in C_0

Insert 1 into the original {2,3,7,18,21,30}:

Step	State	Invariant
Swap	min=1; pending x=2	New min stays outside
Split	high(2)=0, low(2)=2	Top-level L=4
C_0	{3} becomes {2,3}	Local min=2, max=3
Summary	Still {0,1,4,5,7}	C_0 was already nonempty
Finish	max remains 30	Largest key unchanged

C_0 has u=4, so its own split uses L=2. Its old local min 3 moves to local cluster 1 as offset 1. Coordinate systems change at each recursive level.

Delete: handle singletons and u=2 first

DELETE-PRESENT requires that x exists. Handle these two cases before the next page's general procedure:

```
DELETE-PRESENT(V,x):
    if V.min==V.max:
        V.min=V.max=NIL; return
    if V.u==2:
        V.min=V.max=1-x; return
    DELETE-GENERAL(V,x)
```

The first branch is a nonempty singleton by contract; all its clusters were already empty. At u=2 with two keys, the set is {0,1}, so deleting x leaves 1−x.

Calling this procedure on an absent key can erase valid data. A public interface first checks MEMBER and returns NOT_FOUND on failure; the precondition must be explicit.

General deletion: promote the next min, then remove its cluster copy

```
DELETE-GENERAL(V,x):         # u>2; at least two keys
    if x==V.min:
        k=V.summary.min
        x=index(k,V.cluster[k].min); V.min=x
    i=high(x); j=low(x); C=V.cluster[i]
    DELETE-PRESENT(C,j)
    if C.min==NIL:
        DELETE-PRESENT(V.summary,i)
        if x==V.max:
            k=V.summary.max
            if k==NIL: V.max=V.min
            else: V.max=index(k,V.cluster[k].max)
    elif x==V.max:
        V.max=index(i,C.max)
```

Why deletion rebinds x to the promoted minimum

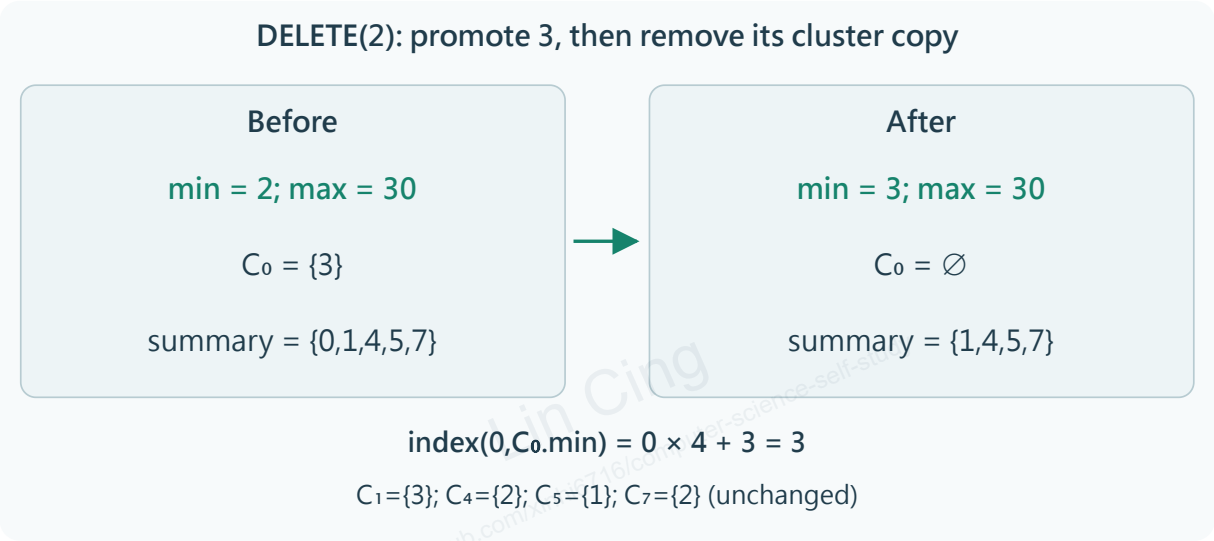
Promotion changes x to the new min. The recursive removal deletes its old cluster representation. Update the cluster and summary before reconstructing max; the previous largest cluster may no longer exist.

For {2,3}, delete 2: promotion sets x=3 and min=3; removing 3 empties C_0 and then the summary. Because the rebound x equals old max 3, set max=min=3, leaving the correct singleton.

With 7,18,21,30 also present, the rebound x=3 differs from max 30, so max stays unchanged. Every later comparison uses the current x, not the original argument 2.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Delete 2: promote 3 without shifting the other keys



Promote 3 to the top-level min, empty C_0, and remove summary index 0.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Check promotion and summary removal

Restart from the original set. summary.min=0 and C_0.min=3, so the new min is 3. Removing offset 3 empties C_0; deleting index 0 from the summary leaves {1,4,5,7}. max stays 30.

The promoted 3 now lives **only at the top level**. Keeping it in C_0 would violate the invariant. Here the cluster deletion removes a singleton and costs O(1); the summary is the only potentially expensive recursive path.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Deleting maxima and the final two elements

From the original set, delete 30,21,18,7,3,2 in order:

Removed	Remaining min / max	Top-level summary
30	2 / 21	{0,1,4,5}
21	2 / 18	{0,1,4}
18	2 / 7	{0,1}
7	2 / 3	{0}
3	2 / 2	Empty
2	NIL / NIL	Empty

After deleting 3, no cluster remains, so max becomes the detached min 2. Never use NIL from an empty summary as an array index. Deleting the final 2 clears extrema; all recursive structures are already empty.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Why two deletion calls still leave one expensive path

General deletion may call DELETE(cluster) and DELETE(summary), but the second occurs only if the cluster becomes empty. That means the cluster **was a singleton**, so its deletion took O(1).

Previous cluster	Cluster deletion	Remove summary index?
Singleton	O(1)	Yes; may recurse
At least two keys	May recurse	No; remains nonempty

Hence the cost is $\max\{T(L), T(H)\} + O(1)$, not $T(L) + T(H) + O(1)$. The representation and branch conditions prove a **worst-case** bound; neither probability nor amortization is needed.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Solve the recurrence in bit width

Let T(u) bound an operation and set $R(m) = T(2^m)$. One nonconstant recursive call reaches a child with at most $\lceil m/2 \rceil$ bits:

$$R(m) \le R(\lceil m/2 \rceil) + c, \qquad R(1) = O(1).$$

For m≥2, $\lceil m/2 \rceil \le 2m/3$, so at most $\lceil \log_{3/2} m \rceil$ levels reach the base. Constant work per level gives $O(1 + \log m) = O(1 + \log \log u)$.

Starting at m=5, a longest width path is 5→3→2→1; a cluster path can be 5→2→1. Odd widths do not have identical square-root child universes. The additive 1 handles the constant-time u=2 base.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Correctness through the represented set

Induct on bit width m. At m=1, enumerate the empty set, {0}, {1}, and {0,1}. Larger sets are uniquely reconstructed from the detached min and all cluster offsets.

Queries: extrema handle boundaries. If a same-cluster answer exists, no higher cluster can beat it; otherwise the summary selects the next nonempty cluster. Predecessor also considers detached min.

Insertion: swapping a new minimum with the old one relocates a key without losing it. Insert the pending key into its cluster and add a summary index exactly when that cluster first becomes nonempty.

Deletion: promote the least cluster key when deleting min, then remove its cluster copy; otherwise remove the requested key. Remove indices of newly empty clusters and reconstruct max from updated state. The induction hypothesis makes child operations correct, establishing the exact set change.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

An empty structure still costs Θ(u) space and initialization

The fully allocated version pays allocation before operations. For u>2, there are H clusters of universe L and a summary of universe H. Local pointer/field space is at most cH:

$$P(u) \le HP(L) + P(H) + cH, \qquad P(2) \le p_0.$$

Prove $P(u) \le A(u-1) - B$ by strong induction. Choose B≥c and A≥B+p_0; substitution gives

$$P(u) \le A[H(L-1) + (H-1)] - B(H+1) + cH \le A(u-1) - B.$$

Thus space is O(u). Cluster-only base nodes already require Ω(u), giving Θ(u). Visiting and initializing all nodes also takes Θ(u). This proof permits unequal H and L.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Public interfaces, duplicate keys, and space tradeoffs

Case	Public contract
Duplicate INSERT(x)	MEMBER first; return ALREADY_PRESENT
Absent DELETE(x)	MEMBER first; return NOT_FOUND
x<0 or x≥u	Reject before indexing
Multiplicity needed	Maintain counts; change key set only on 0↔1
Satellite data	Specify key→record mapping and ownership

A membership check plus an update still costs O(1+log log u). Multiplicity assumes constant-time, nonoverflowing counters; duplicate keys must not enter INSERT-ABSENT directly.

The full tree is appropriate only when u is affordable. Lazy allocation and hash-table clusters require a new space/allocation/expected-time analysis; they do not inherit this deterministic worst-case proof. A balanced O(n)-space tree may be preferable for a sparse, huge universe.

Keep amortized, expected, and worst-case guarantees distinct

Structure / operation	Guarantee	Essential assumptions
Red-black query/update	Worst-case O(log(n+2))	Comparisons; O(n) space
Hash lookup	Expected O(1+α)	Stated hashing model; no ordered neighbors
Fibonacci decrease-key	Amortized O(1)	Valid handle; shared sequence potential
Full vEB predecessor/update	Worst-case O(1+log log u)	Integer word RAM; Θ(u) space/setup

Building a full vEB and performing q operations costs $O(u + q(1 + \log\log u))$. Initialization cannot disappear from the total. Comparison keys, integer universes, and randomized models impose different contracts; a context-free ranking of structures is misleading.

References and attribution

Cormen, Leiserson, Rivest, Stein. Introduction to Algorithms, 3rd ed. (2009), Chapters 10–14, 17–20. MIT 6.046J/18.401J (2004), L09: Balanced Search Trees, Piotr Indyk. MIT 6.046J/18.401J (2005), L08: Hashing II; L09: Randomly Built BSTs; L11: Augmenting Data Structures; L13: Amortized Analysis. Charles E. Leiserson and Erik D. Demaine. Lin Cing, personal handwritten data-structure notes: queues, search trees, red-black trees, multiway search trees, and heaps.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Dynamic programming

Part IV Advanced Design and Analysis Techniques

Scope

Dynamic programming merges repeated recursive calls into states. The challenge is proving that a state retains everything needed for future decisions and reconstructing an actual solution from optimal values. Cutting, matrix parenthesization, common subsequences, and search trees develop this method.

Prerequisites

Recursion, optimization, and overlapping subproblems.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Chapter route

Learning objectives

- Prove optimal substructure for rod cutting and LCS.
- Design state dimensions and evaluation orders.
- Derive interval DP for matrix chains and optimal BSTs.
- Reconstruct choices and reassess DP assumptions in unfamiliar variants.
- 15.1 Rod cutting
- 15.2 Matrix-chain multiplication
- 15.3 Elements of dynamic programming
- 15.4 Longest common subsequence
- 15.5 Optimal binary search trees

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Rod cutting

Given nonnegative integer length n, p_i is the price of an uncut piece of length i. All pieces have positive integer lengths. Leaving the rod uncut is permitted, but the whole rod must be sold; cuts cost nothing. Maximize revenue.

The first decision cuts off length i and optimally handles n−i. With $r_0=0$,

`r_n=max_{1<=i<=n}(p_i+r_{n-i})`.

Necessity. In an optimal cutting with value r_n, let the first piece have length i. If the remaining n−i were not optimal, replacing it with a better r_(n−i) cutting would be legal and improve the total, a contradiction.

Sufficiency. Each p_i+r_(n−i) is a legal cutting and cannot exceed the true optimum. The two inequalities establish the recurrence.

Uncached recursion makes $T(0)=1$ and $T(n)=1+\sum_{j=0}^{n-1}T(j)=2^n$ calls. Memoization or bottom-up evaluation computes each length once, with $\sum_{j=1}^{n}j=\Theta(n^2)$ transitions. T counts calls, not prices or revenue.

Memoization and bottom-up evaluation

Use one-based price array p and evaluate smaller remainders first. Even with negative prices, the whole rod must be sold, so nonempty states start at −∞ rather than silently allowing disposal.

```
r[0] = 0
for length = 1..n:
    r[length] = -∞
    for first = 1..length:
        value = p[first] + r[length-first]
        if value > r[length]:
            r[length] = value
            choice[length] = first
pieces = []
while n > 0:
    append choice[n] to pieces
    n = n-choice[n]
```

At entry to length, all r[0..length−1] are optimal. The inner loop tests every legal first piece, so the recurrence makes r[length] optimal. Reconstruction reduces the remainder by at least1 per step and outputs at most n pieces.

Complete rod table and reconstruction

Let p=[1,5,8,9]. choice[n] stores an optimal first-piece length.

n	Candidate revenues	r[n]	choice[n]
0	Empty	0	0
1	1	1	1
2	5,1+1=2	5	2
3	8,1+5=6,5+1=6	8	3
4	9,1+8=9,5+5=10,8+1=9	10	2

From n=4, choice[4]=2 earns p_2=5 and leaves2. choice[2]=2 earns another5 and leaves0. The optimal cutting is 2+2, revenue10. Reconstruct after computing the table; do not guess cuts while modifying settled optimal values.

Complete rod table and reconstruction (continued)

A fixed cost c per cut can be subtracted from a candidate only if a remainder remains; one length dimension still suffices. A limit of k pieces requires a piece-count dimension. Add state only when the new restriction affects future feasible decisions, not merely because the specification changed.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Matrix-chain multiplication

A_i has dimensions $p_{i-1} \times p_i$. If the final multiplication for A_i..A_j splits after k, its cost is

`m[i,k]+m[k+1,j]+p_{i-1}p_k p_j`.

Minimize over i≤k<j, with m[i,i]=0.

Both optimal subchains must be optimal. Replacing the left by cheaper parentheses leaves the final multiplication cost unchanged and improves the whole chain, contradicting optimality; likewise on the right. Every full parenthesization has a unique final split, so enumerating k=i..j−1 is complete.

Fill by increasing length l=j−i+1, since both dependencies are shorter. Store the best k in split[i,j]. There are Θ(n^2) states and O(n) candidates per state, totaling Θ(n^3).

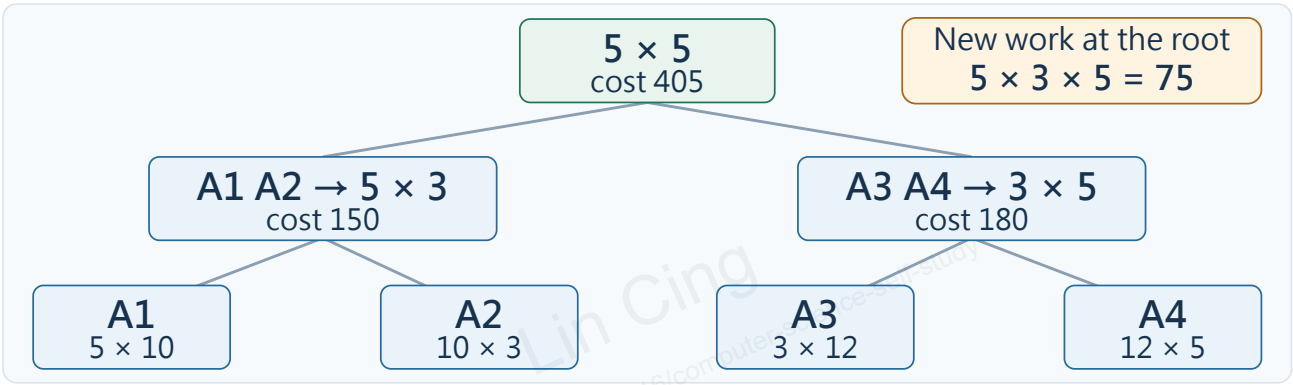
More precisely, the number of candidates is `Σ_(l=2)^n (n−l+1)(l−1)=n(n²−1)/6`. Each uses constantly many reads and arithmetic operations. This proves both bounds; merely multiplying two O bounds would not establish Θ.

Matrix-chain table and parenthesis output

```
for i = 1..n: m[i,i] = 0
for length = 2..n:
    for i = 1..n-length+1:
        j = i+length-1
        m[i,j] = +∞
        for k = i..j-1:
            c = m[i,k]+m[k+1,j]+p[i-1]*p[k]*p[j]
            if c < m[i,j]: m[i,j], split[i,j] = c, k
emit(i,j):
    if i == j: print A_i; return
    k = split[i,j]
    print '('; emit(i,k); emit(k+1,j); print ')'
```

Every dependency is a strictly shorter interval, justifying length order. The reconstruction tree has n leaves and n−1 internal nodes, so emitting parentheses takes Θ(n) node operations, excluding actual matrix-entry multiplication.

Matrix-chain table and parenthesis output (continued)



150 + 180 + 75 = 405; cumulative cost differs from new work

Complete matrix-chain values and reconstruction

For p=[5,10,3,12,5], the upper-triangular table for A1..A4 is:

Length	Values
1	m11=m22=m33=m44=0
2	m12=150,m23=360,m34=180
3	m13=330,m24=330
4	m14=405

`m24=min(0+180+10*3*5,360+0+10*12*5)=330`, with split[2,4]=2.

`m14=min(0+330+5*10*5,150+180+5*3*5,330+0+5*12*5)=405`, with split[1,4]=2.

That split gives(A1A2) and(A3A4). Their splits1 and 3 produce final parenthesization `((A1A2)(A3A4))`.

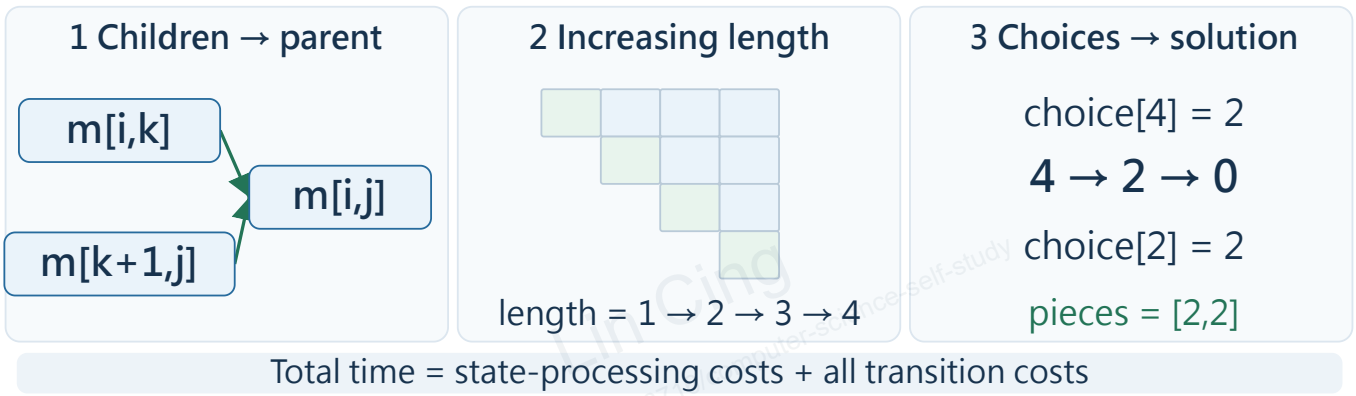
Elements of dynamic programming

A state must describe the remaining work, not merely rename the original problem. Transitions depend on smaller states. Establish four facts:

1. Removing the first decision from an optimum leaves an optimum for the corresponding subproblem.
2. Defined substates compose every candidate decision without omitting legal solutions.
3. Base cases are correct and the dependency graph is acyclic.
4. A topological evaluation order completes all prerequisites before a state is filled.

Together these prove recurrence correctness and valid evaluation. Overlap explains why caching saves repeated work; it alone does not prove optimal substructure. Cyclic dependencies require another justified solver rather than one table pass. Reconstruction can store choices or recheck transitions in a full value table; one final scalar optimum usually does not suffice.

Elements of dynamic programming (continued)



The state graph determines memoization cost

Rod cutting has n+1 states. Length j depends on0..j−1, totaling n(n+1)/2 edges. Constant work per state/edge gives Θ(V+E)=Θ(n^2).

Top-down memo expands all candidates the first time a state is evaluated and immediately returns its stored value afterward. Keep visited status separate from values, for example None for uncomputed, since a negative revenue can be valid. Bottom-up has no recursion stack; memo may have n stack levels. Both use Θ(n) table space.

The longest-simple-path counterexample does not mean every cycle forbids DP. Endpoint-only states fail to record used vertices. Individually longest simple u→v and v→w paths may both use z, so concatenation repeats z and is not simple. Adding a visited set S permits exponential-state DP; a DAG instead allows longest paths directly in topological order.

Longest common subsequence

Let c[i,j] be the LCS length of prefixes X_i,Y_j. Empty-prefix length is 0.

If x_i=y_j, an LCS not ending in that character could append it and become longer. Thus an optimum can end with the common character; removing it gives a candidate for X_(i−1),Y_(j−1). A better prefix subsequence could also append x_i and exceed the original optimum, so the prefix must itself be optimal.

If x_i≠y_j, no common subsequence can depend on both distinct terminal characters as its last character. It omits at least one terminal position, giving

`c[i,j]=max(c[i-1,j],c[i,j-1])`.

The two branches cover every possibility. Each transition costs O(1) and the full table O(mn).

Both LCS inequalities and the full algorithm

When terminal characters match, an LCS not ending with that character uses neither terminal position and can append it, contradicting maximality. Removing the final character bounds its length by c[i−1,j−1]+1. Conversely, any prefix LCS can append the common character legally. Hence c[i,j]=c[i−1,j−1]+1.

```
c[0,0..n] = 0; c[0..m,0] = 0
for i = 1..m:
    for j = 1..n:
        if X[i] == Y[j]: c[i,j] = c[i-1,j-1]+1
        else: c[i,j] = max(c[i-1,j],c[i,j-1])
i=m; j=n; result=[]
while i>0 and j>0:
    if X[i]==Y[j]: append X[i] to result; i--; j--
    elif c[i-1,j]>=c[i,j-1]: i--
    else: j--
reverse result
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Both LCS inequalities and the full algorithm (continued)

The full c table already determines backtracking; a separate direction table is optional. Length alone needs two rows along the shorter string, Θ(min(m,n)) space. This backtracking still requires the full table and cannot run unchanged on only two rows.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Complete LCS table and backtracking

X=ABCBDAB,Y=BDCABA gives c[7,6]=4. Including empty prefixes:

```
     ε B D C A B A
ε    0 0 0 0 0 0 0
A    0 0 0 0 1 1 1
B    0 1 1 1 1 2 2
C    0 1 1 2 2 2 2
B    0 1 1 2 2 3 3
D    0 1 2 2 2 3 3
A    0 1 2 2 3 3 4
B    0 1 2 2 3 4 4
```

Up decreases i; left decreases j.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Complete LCS table and backtracking (continued)

Cell	Action	Next	Reverse characters
(7,6)	Up; both candidates4	(6,6)	Empty
(6,6)	A=A, diagonal	(5,5)	A
(5,5)	D≠B; upper3 larger	(4,5)	A
(4,5)	B=B, diagonal	(3,4)	AB
(3,4)	C≠A; left2 larger	(3,3)	AB
(3,3)	C=C, diagonal	(2,2)	ABC
(2,2)	B≠D; left1 larger	(2,1)	ABC
(2,1)	B=B, diagonal	(1,0)	ABCB

Reverse to get BCBA. Each step decreases at least one index, giving O(m+n) backtracking without scanning all mn cells.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

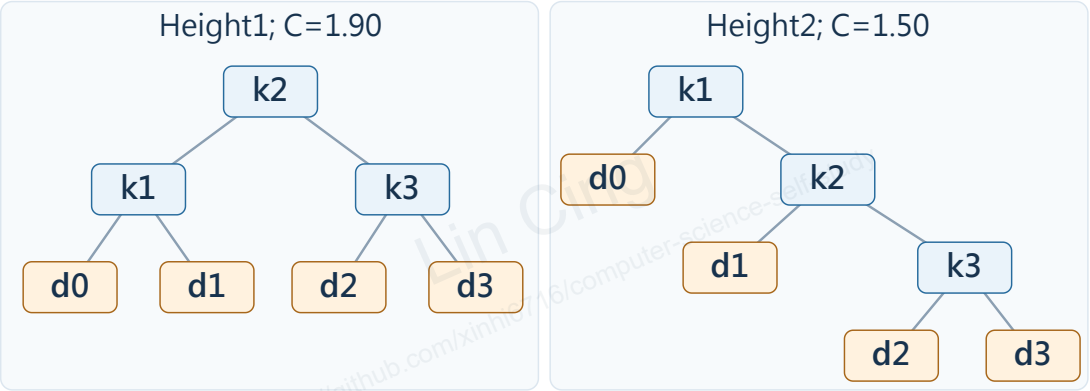
Complete LCS table and backtracking (continued)

i ↓ / j →	0 ε	1 B	2 D	3 C	4 A	5 B	6 A
0 ε	0	0	0	0	0	0	0
1 A	0	0	0	0	1	1	1
2 B	0	1	1	1	1	2	2
3 C	0	1	1	2	2	2	2
4 B	0	1	1	2	2	3	3
5 D	0	1	2	2	2	3	3
6 A	0	1	2	2	3	3	4
7 B	0	1	2	2	3	4	**4**

Green arrows: backtrack; amber: emit; reverse A,B,C,B → BCBA

Example A: lower height can cost more comparisons

Let p=(0.50,0.10,0.05), q=(0.15,0.10,0.05,0.05). Root depth is 0. Height counts real-key edges; C counts only real-key comparisons.



C counts only real-key comparisons; no extra dummy check

Example A: weight every successful and unsuccessful outcome

A successful search compares depth+1 keys. A failed search compares as many keys as its dummy depth, **without an extra dummy check**.

Query	Probability	Balanced: count × prob.	Right chain: count × prob.
k1	0.50	2 × 0.50 = 1.00	1 × 0.50 = 0.50
k2	0.10	1 × 0.10 = 0.10	2 × 0.10 = 0.20
k3	0.05	2 × 0.05 = 0.10	3 × 0.05 = 0.15
d0	0.15	2 × 0.15 = 0.30	1 × 0.15 = 0.15
d1	0.10	2 × 0.10 = 0.20	2 × 0.10 = 0.20
d2	0.05	2 × 0.05 = 0.10	3 × 0.05 = 0.15
d3	0.05	2 × 0.05 = 0.10	3 × 0.05 = 0.15
Total	1.00	**C = 1.90**	**C = 1.50**

Success costs are 1.20/0.85; failure costs are 0.70/0.65. Frequent outcomes k1 and d0 finish earlier in the right chain, outweighing extra comparisons for the other outcomes.

Optimal binary search trees

Given distinct sorted keys k1<…<kn, choose a BST that minimizes expected search cost for a specified query distribution. Key order stays fixed; the root and subtree shapes are the decisions. Include both successful and unsuccessful searches.

A hit on key k_i has probability p_i. Failed searches end in gaps $d_0, \dots, d_n$, with probabilities $q_0, \dots, q_n$. For example, d_0 lies before the smallest key and d_i between adjacent keys k_i, k_{i+1}. A dummy leaf represents an empty stopping position, not a real record to compare. All probabilities sum to 1. Define the extended cost E, including a step at the dummy leaf:

$$E(T) = \sum_{r=1}^{n} p_r(\text{depth}_T(k_r) + 1) + \sum_{r=0}^{n} q_r(\text{depth}_T(d_r) + 1).$$

e[i,j] is optimal cost for k_i..k_j; w[i,j] is all key and dummy probability mass in that interval. An empty interval has one dummy:

$$e[i, i-1] = w[i, i-1] = q_{i-1}.$$

Choosing root k_r increases every left/right-subtree search depth by1, adding its total weight. Thus

Optimal binary search trees (continued)

$$e[i,j] = \min_{i \le r \le j} \{e[i,r-1] + e[r+1,j] + w[i,j]\}.$$

Replacing a purportedly optimal subtree with a cheaper one would improve the whole tree. Every root is enumerated, establishing the recurrence.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Two cost conventions differ by total failure probability

C(T) counts real-key comparisons. E(T) also counts the step reaching a dummy leaf after failure. These are cost conventions, not instructions to compare an imaginary record. The table e below uses E.

$$C(T) = \sum_{r=1}^{n} p_r(\operatorname{depth}_T(k_r) + 1) + \sum_{r=0}^{n} q_r \operatorname{depth}_T(d_r).$$

$$E(T) = C(T) + \sum_{r=0}^{n} q_r.$$

For example A the difference is 0.35: E=2.25 for the balanced tree and 1.85 for the right chain. This shape-independent constant **preserves the optimal tree, but changes its reported cost**.

With no real key, C[i,i−1]=0, while e[i,i−1]=q[i−1]. C does not ignore failed searches: after attachment, a query still compares the parent key, and the parent interval accounts for that step through w. Initialization and later additions must use one consistent convention.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Across courses: search models and cost conventions

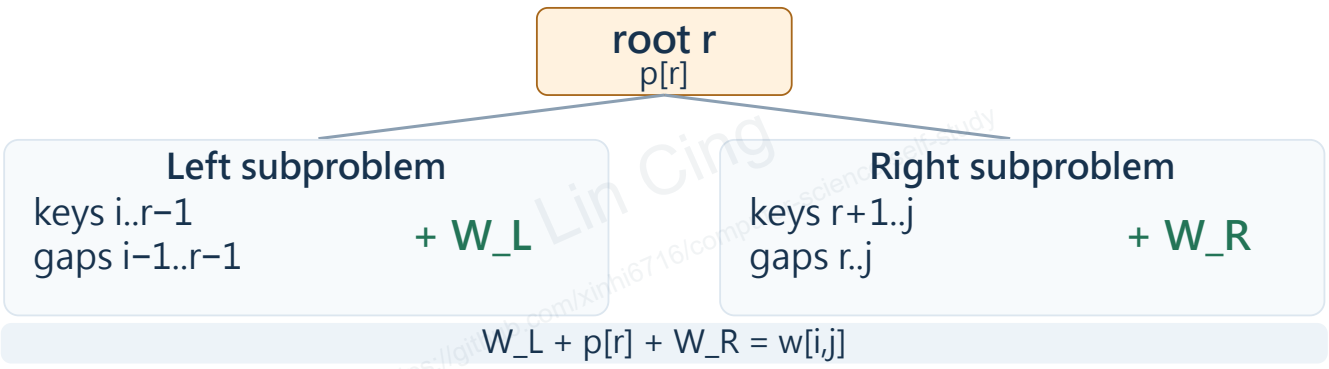
Data Structures Chapter 10 starts with successful searches only. Here, CLRS § 15.5 includes failed queries too. Compare answers only for the same distribution and cost convention.

Convention	Successful searches only	Full query model here
Input weights	p; total 1	p and q; total 1
Empty-interval cost	0	E: q; C: 0
Added interval weight	Successful-query mass	Successful and unsuccessful mass w

Dropping nonzero q and renormalizing p may change the optimal tree. For fixed p and q, E−C=Σq is constant, so optimal trees agree. Also check interval endpoints, empty-tree costs, and the meaning of root entries.

Why add all of w, rather than only the root probability?

$$w[i,j] = \sum_{t=i}^{j} p_t + \sum_{t=i-1}^{j} q_t = W_L + p_r + W_R.$$



W_L, W_R are the two subtree probability masses. One extra level adds $W_L + W_R$; a root hit adds p_r. **Use global probabilities without renormalizing.**

Optimal-BST table and cost convention

```
for i = 1..n+1:
    e[i,i-1] = w[i,i-1] = q[i-1]
for length = 1..n:
    for i = 1..n-length+1:
        j = i+length-1
        w[i,j] = w[i,j-1]+p[j]+q[j]
        e[i,j] = +∞
        for r = i..j:
            c = e[i,r-1]+e[r+1,j]+w[i,j]
            if c < e[i,j]: e[i,j], root[i,j] = c, r
```

This is CLRS's extended-search-tree cost: failed search includes one dummy check. Counting only real-key comparisons gives e[1,n]−Σq_i. This shape-independent constant leaves the optimal tree unchanged. Example B's optimum 2.10 becomes 2.10−0.40=1.70 under real-key comparisons.

Reconstruct build(i,j): if j<i, emit dummy d_(i−1); otherwise r=root[i,j], emit k_r and recursively build(i,r−1),build(r+1,j). Retain empty intervals to deliver the unsuccessful-search leaves correctly.

Example B: complete optimal-BST table and reconstruction

Take p=(0.20,0.30,0.10), q=(0.10,0.05,0.15,0.10).

Interval	w	e	Root
[1,1]	0.35	0.50	k1
[2,2]	0.50	0.70	k2
[3,3]	0.35	0.60	k3
[1,2]	0.80	1.45	k2
[2,3]	0.70	1.35	k2
[1,3]	1.00	2.10	k2

For e[1,2], root k2 gives e[1,1]+e[3,2]+w[1,2]=0.50+0.15+0.80=1.45. root[1,3]=2; roots for[1,1] and[3,3] are 1,3. Thus k2 has children k1,k3 and dummies attach by BST intervals.

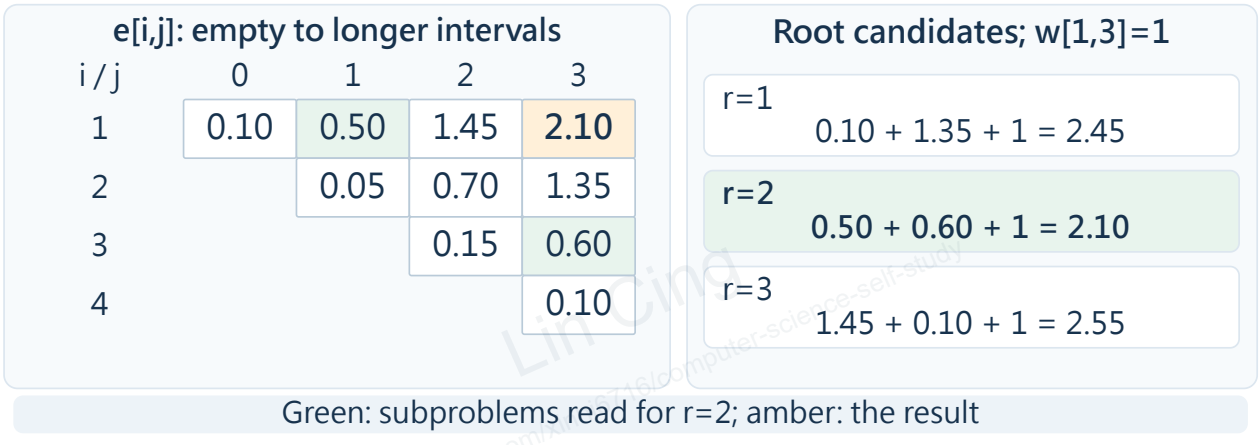
Example B: complete optimal-BST table and reconstruction (continued)

Increasing-length evaluation uses $\Theta(n^2)$ cells each for e,w,root and $\Theta(n^3)$ total candidate roots. This chapter uses that proved version.

Exactly $\Sigma_{l=1}^{n-l+1} l = n(n+1)(n+2)/6$ roots are tested. Each w updates in O(1) from $w[i,j-1]+p_j+q_j$. Rescanning probabilities inside every root candidate would add a factor n and invalidate this bound.

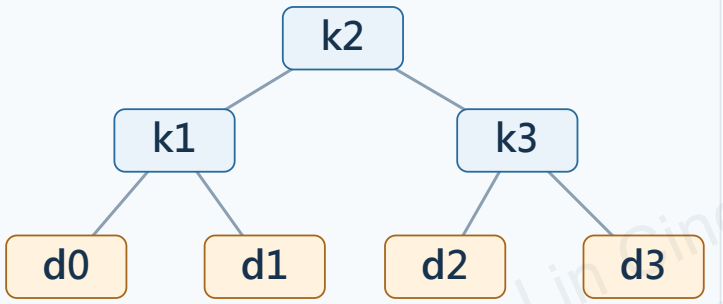
Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Example B: compare every root and read the correct subproblems



Green cells are e[1,1] and e[3,3], read when r=2. Both are shorter intervals. Amber e[1,3] becomes 2.10 after all three candidates are compared. Retain the q cost of empty intervals.

Example B: reconstruct the actual search tree from root



Example B: includes dummy checks

k2: 0.30 × 1 = 0.30

k1,k3: 0.30 × 2 = 0.60

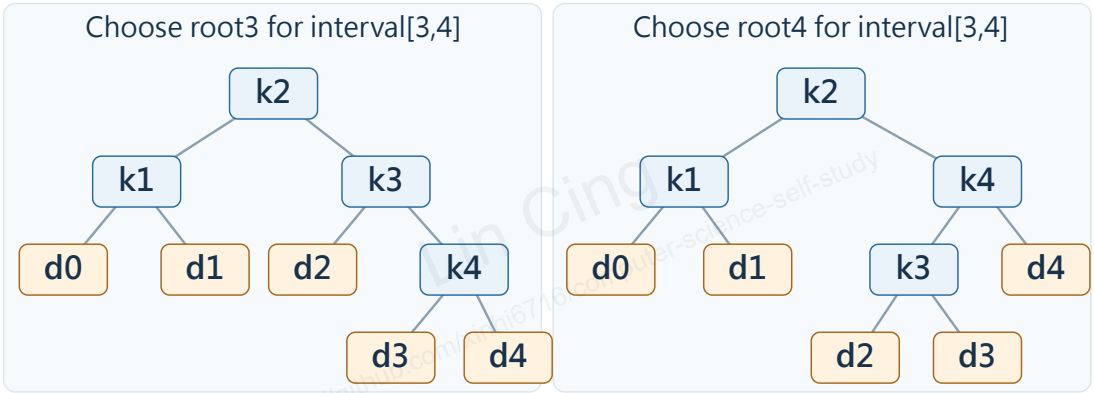
d0..d3: 0.40 × 3 = 1.20

E = 2.10
C = E − 0.40 = 1.70

d0 | k1 | d1 | k2 | d2 | k3 | d3

Example C: integer weights and two optimal trees

Weights P=(3,3,1,1), Q=(2,3,1,1,1) total 16; failure weights total 8. The right interval[3,4] has two optimal roots,3 and 4.



Both trees: S=32; divide by total weight16: C=2 and E=2.50

Sequence alignment: score matches, mismatches, and gaps

Global alignment allows match, mismatch, and gap. Its last column is one of:

- Paired(x_i,y_j), earning match/mismatch score.
- x_i paired with a gap.
- y_j paired with a gap.

For maximum prefix score F[i,j],

`F[i,j]=max(F[i-1,j-1]+score(x_i,y_j),F[i-1,j]+gap,F[i,j-1]+gap).`

This shares LCS's prefix states and backtracking skeleton but needs three score cases. For X=ACGTA,Y=AGCAC, match3,mismatch−2,gap−3, optimum is 1 with alignment `ACGTA- / A-GCAC`. Every gap costs its own fixed score. Comparing only equal-length positions misses displacement.

Unbounded knapsack uses capacity states

With repeatable item weights s_i and values v_i, let f[c] be maximum value at **exact** weight c:

`f[0]=0`, `f[c]=max_{i:s_i<=c}(f[c-s_i]+v_i).`

For capacity at most C, maximize over0≤c≤C; exact fill takes only f[C]. Including f[c−1] in the recurrence instead defines value at capacity at most c. Distinguish the meanings.

For A=(3,5),B=(4,7),C=(7,12), capacity13,

`f[0..13]=[0,-∞,-∞,5,7,-∞,10,12,14,15,17,19,21,22].`

Optimum22 at weight13 uses A+A+A+B. Highest value/weight greedy chooses B×3 and earns21, disproving general ratio-greedy correctness.

Sources

Bibliography

Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 3rd ed., MIT Press, 2009, Chapter15, § § 15.1–15.5.

Lin Cing, personal handwritten data-structures notes on weighted OBST cost and interval recurrences.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

References and attribution

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Greedy algorithms

Part IV Advanced Design and Analysis Techniques

Scope

A greedy algorithm retains only one choice per step, avoiding other candidates. Activity exchanges, Huffman leaf exchanges, and matroid augmentation prove when a choice is safe. Weighted objectives and capacity restrictions show when an old rule fails.

Prerequisites

Sorting, exchange arguments, trees, and axiomatic selection properties.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Chapter route

Learning objectives

- Prove activity selection by exchange.
- Distinguish greedy choice from optimal substructure.
- Build Huffman codes and prove matroid greedy correctness.
- Give counterexamples when safety fails and complete every exchange step.
- 16.1 An activity-selection problem
- 16.2 Elements of the greedy strategy
- 16.3 Huffman codes
- 16.4 Matroids and greedy methods
- 16.5 A task-scheduling problem as a matroid

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Activity selection

Given all start/finish times, maximize the number of pairwise nonoverlapping activities. Assume s_i<f_i, sorted nondecreasing finishes, and half-open intervals[s_i,f_i). Equal finish/start endpoints are compatible.

Theorem. For every nonempty subproblem, some maximum compatible set contains an earliest-finishing activity a_m.

Proof. Let A* be an optimum and a_j its earliest-finishing activity. Since f_m≤f_j, replace a_j with a_m.

- Every later activity in A* starts at or after f_j, hence also after f_m.
- There is no earlier activity in the subproblem's chosen schedule to disrupt.
- Cardinality is unchanged, so the new set remains optimal.

This establishes a safe greedy choice. After a_m, only activities starting at or after f_m remain. Repeating the same exchange inductively proves optimality.

Linear scan and boundaries

```
activity-select(activities sorted by finish):
    result=[]; last_finish=-∞
    for (s,f) in activities:
        if s>=last_finish:
            append (s,f) to result
            last_finish=f
    return result
```

Before processing activity j, result is the compatible set chosen by the rule from the first j−1 activities, and last_finish is its final endpoint. The exchange theorem makes each choice extendible to an optimum. One scan takes Θ(n) when sorted; otherwise add O(n log n) sorting.

Assume positive-duration half-open intervals; equal endpoints may touch. Zero-duration activities need separate same-point rules. Empty input returns an empty set. Earliest start is unsafe: [0,10), [1,2),[2,3) gives it one activity rather than two.

Shortest duration is also unsafe: [0,3),[3,6),[2,4). Choosing the middle length2 activity blocks both length3 activities and gives one rather than the optimum two.

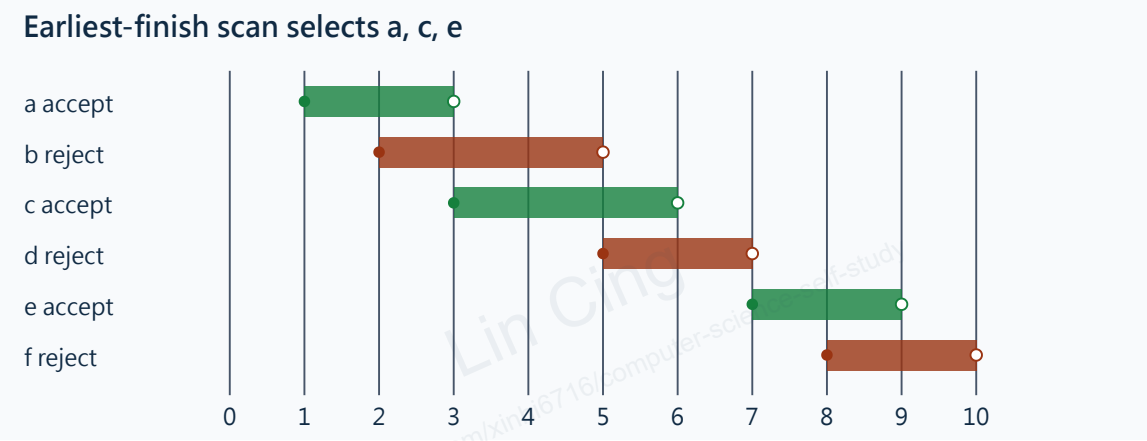
Complete activity scan

Activity	Start	Finish
a	1	3
b	2	5
c	3	6
d	5	7
e	7	9
f	8	10

Select a; reject b since2<3; select c since3≥3; reject d since5<6; select e since7≥6; reject f since8<9. Result{a,c,e}.

The optimum need not be unique: {b,d,e} also has size3. Exchange guarantees **some** optimum includes the earliest finish, not that every optimum must.

Complete activity scan (continued)



Elements of the greedy strategy

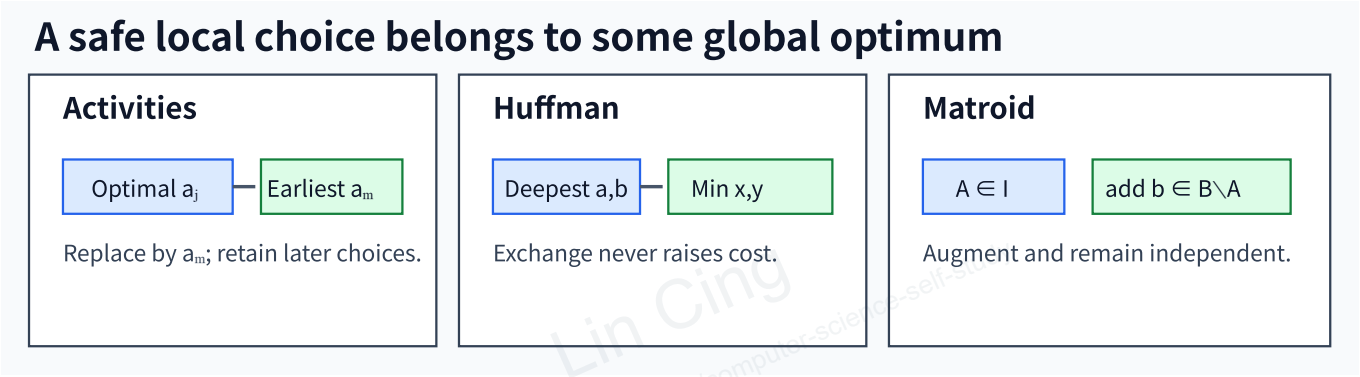
An exchange proof must establish:

1. **Safety.** For greedy g and an optimum A*, identify a differing choice x and form A′ by replacing x with g. Check feasibility and no loss of value.
2. **The residual problem.** After selecting g, the remaining solution belongs to a precisely defined subproblem that retains every relevant candidate.
3. **Termination by induction.** Empty/singleton bases are correct and each decision strictly shrinks the problem.

Only then do safe local choices imply global optimality. Immediate benefit, average performance, or successful examples alone do not suffice.

Largest-coin greedy fails for denominations{1,3,4}, amount6:4+1+1 uses three coins versus3+3 using two. This disproves the general rule; experience with familiar denominations does not prove a property for all denomination families.

Elements of the greedy strategy (continued)



Why weighted activities invalidate the same exchange

Maximizing total value changes the objective from cardinality.

Activity	Interval	Value
short	[0,2)	1
long	[1,3)	100
tail	[3,4)	1

Earliest finish selects short+tail, value2; optimum long+tail has101. Replacing long by short preserves compatibility but lowers value100→1, so it no longer preserves optimality.

The weighted problem still has optimal substructure. Sort by finish with one-based indices. Let p(j) be the largest earlier index finishing no later than s_j, or0 if none. D[j] is optimum value among the first j activities, with D[0]=0:

`D[j]=max(D[j-1],value_j+D[p(j)])`.

Why weighted activities invalidate the same exchange (continued)

Excluding j leaves the first j−1 activities. Including it restricts all remaining choices to the first p(j), which can be optimized independently. These cases cover every solution. Here p=(0,0,2),D=(0,1,100,101). Reconstruct tail at D3 and long at D2. Finish-time greedy guarantees unweighted cardinality only.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Fractional and 0–1 knapsack require different proofs

Assume positive weights/values and C>0. Fractional knapsack allows x_i∈[0,1], maximizing Σv_i x_i. Sort decreasing density ρ_i=v_i/w_i, taking highest densities fully and at most one partial item.

If total weight≤C, taking everything is optimal. Otherwise any optimum fills C: spare space plus a partially untaken positive-value item would allow improvement.

Suppose an optimum uses positive lower-density j weight while higher-density i is not full. Move δ=min(w_i(1−x_i),w_j x_j)>0 weight from j to i. Capacity is unchanged and value increases by δ(ρ_i−ρ_j)≥0. Repeat until the first greedy choice agrees, then induct on remaining capacity. Some optimum follows the whole density order; ties may be arbitrary.

In 0–1 knapsack, x_i∈{0,1}, so that δ exchange is usually illegal. For C=10 and(w,v)=(6,12),(5,9),(5,9), density greedy takes(6,12) and cannot fill the remaining4, earning12. The 0–1 optimum takes both(5,9), earning18. Fractional greedy takes(6,12) and4/5 of one(5,9), earning19.2.

For 0–1 integer-capacity DP, D[i,c] is optimum for the first i items and capacity at most c. Bases D[0,c]=D[i,0]=0. Take max of D[i−1,c] and v_i+D[i−1,c−w_i], the latter only if w_i≤c. Time Θ(nC) is pseudopolynomial because C occupies only Θ(log C) input bits.

Huffman codes

A binary prefix tree's leaf depth is code length. For frequency f_c, total bits are

$B(T)=\Sigma_c f_c d_T(c)$.

Repeatedly extract the two minimum-weight trees x,y and insert their parent of weight f_x+f_y. n symbols need n−1 rounds, each two extract-min and one insert, or3(n−1) heap operations. With linear heap construction, time is O(n log n).

Swapping left/right changes codeword0/1 values but not lengths or total cost. Equal frequencies may yield different optimal trees.

Huffman's safe choice places minimum frequencies at deepest siblings

Assume at least two symbols and positive frequencies. An optimal prefix tree may be full: removing a one-child internal node shortens every code beneath it and lowers cost. A deepest leaf has a leaf sibling at the same depth, since an internal sibling would have deeper descendants.

Let x,y have the two lowest frequencies. First move x to one of these deepest positions. If one holds y, use the other, keeping y available for the second exchange or already in its desired sibling position.

Swapping x with deepest leaf a changes only their contributions:

$B(T')-B(T)=(f_a-f_x)(d_x-d_a)\leq 0$.

Here $f_a \geq f_x$ and $d_x \leq d_a$. With x fixed, all remaining frequencies are at least f_y. Move y to x's deepest sibling without raising cost or moving x. A symbol already in place gives a zero exchange; equal frequencies also permit ties.

Thus some optimal prefix code includes this greedy choice. The argument requires both the minimum-frequency and maximum-depth inequalities, not intuition alone.

Huffman contraction and optimal substructure

Replace x,y by virtual symbol z of frequency $f_z=f_x+f_y$. Expand z in an optimum contracted tree T′ into leaves x,y to obtain T.

Other leaf depths stay unchanged, while x,y have depth $d_{T'}(z)+1$. Hence

$B(T)=B(T')+f_x+f_y$.

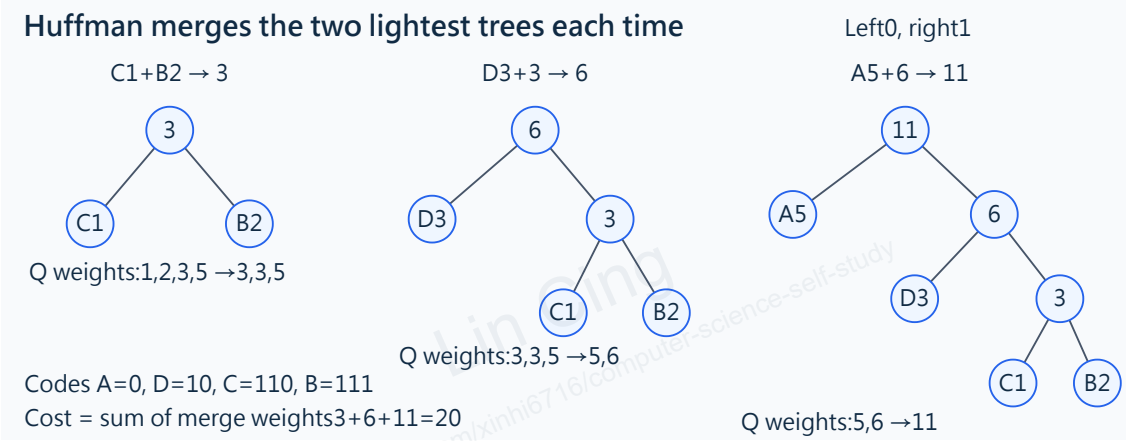
If a better original tree T* existed, the safe-choice lemma permits deepest siblings x,y. Contracting them would give

$B(T^{**})=B(T^*)-f_x-f_y<B(T')$,

contradicting optimality of T′. Expansion is optimal, and induction on alphabet size proves Huffman.

For A5,B2,C1,D3, merge weights are3,6,11. One code is A=0,D=10,C=110,B=111, costing5+6+3+6=20=3+6+11.

Huffman contraction and optimal substructure (continued)



Huffman algorithm and singleton boundary

```
Q = min-heap of all positive-frequency singleton trees
while size(Q)>1:
    x=extract_min(Q); y=extract_min(Q)
    insert(Q, node(x.weight+y.weight, x, y))
return Q's sole tree, or empty if Q is empty
```

Decode from root, following left/right for0/1. At a leaf, emit its symbol and restart at root. Prefix freedom means no complete word is a prefix of another, making boundaries unambiguous. Ending at an internal node means truncated input, not successful decoding.

An empty alphabet has no tree. A singleton may receive an empty code theoretically; practical streams often prescribe0 and transmit a symbol count separately. The equality between weighted length and merge-weight sum uses tree depths and, for the usual nontrivial coding case, at least two leaves. Do not substitute a singleton's forced one-bit implementation convention into merge cost.

Matroids and greedy methods

A matroid $M=(S,I)$ satisfies:

- Empty set belongs to I.
- Heredity: every subset of an independent set is independent.
- Augmentation: if $A,B\in I$ and $|A|<|B|$, some x in B excluding A makes $A\cup\{x\}$ independent.

Assume finite S and $w(x)>0$. Scan decreasing weights and accept if independence remains. Let x be the first independent singleton. If none exists, heredity excludes every nonempty independent set, and empty is optimal.

Take optimum B. If x is absent, start $A=\{x\}$. While $|A|<|B|$, augment using an element of B outside A. Only B elements are added. At equal size, exactly one $y\in B$ has been omitted, so $A=B-\{y\}+\{x\}$. Since $\{y\}$ is independent, first feasible x has $w(x)\geq w(y)$, and $w(A)=w(B)-w(y)+w(x)\geq w(B)$. Thus A is also optimal and contains x.

Do not assert that any arbitrary y can be exchanged. Augmentation first constructs feasible A, and equal cardinality identifies its one omitted y.

Complete proof that contraction remains a matroid

Fix feasible x, let S' be S excluding x, and $I'=\{A\subseteq S':A\cup\{x\}\in I\}$. S' may contain elements that are never independent in the new system; these cause no problem.

- Empty belongs to I' because$\{x\}\in I$.
- If $A\in I'$ and $C\subseteq A$, then $C\cup\{x\}\subseteq A\cup\{x\}$; heredity gives $C\in I'$.
- If $A,B\in I'$ with $|A|<|B|$, augment $A\cup\{x\}$ from $B\cup\{x\}$. The added y lies in B excluding A, since x is in both. Hence $A\cup\{y\}\in I'$.

Removing x maps every feasible solution containing x to I' and subtracts exactly w(x); adding it reverses this mapping. After choosing x, the original problem is exactly maximum-weight independence in the contraction. Induct on $|S|$; later greedy feasibility tests are precisely the I' oracle, proving optimality.

An element rejected now never becomes feasible later: heredity of a later larger feasible set containing it would imply feasibility at rejection, a contradiction. Sorting costs $O(n \log n)$; $O(f(n))$ per oracle gives $O(n \log n+n f(n))$ total. For arbitrary independent sets with negative weights, skip negative elements rather than unconditionally forming a full basis.

Graphic matroids and a nonmatroid counterexample

For fixed undirected G=(V,E), take S=E and I all acyclic edge sets, retaining isolated vertices in V. Heredity holds. Let A's components be V1..Vk. If every B edge stays within a component Vi, B's forest restriction there has at most|Vi|−1 edges. Summing gives|B|≤|V|−k=|A|, contradicting|B|>|A|. Thus B has an edge crossing A-components, and adding it creates no cycle. Augmentation holds.

In a connected graph, bases are n−1-edge spanning trees. To derive minimum-cost MST from the positive maximum-weight theorem, set w′(e)=K−cost(e)>0. Equal basis sizes make maximizing Σw′ equivalent to minimizing original cost. This precisely connects increasing-cost Kruskal to the theorem without confusing arbitrary minimum forests with bases.

Heredity alone does not imply augmentation. For S={a,b,c},weights10,9,9, take I={∅,{a},{b},{c},{b,c}}. A={a},B={b,c} have unequal sizes, yet neither b nor c augments A. Greedy gets10 at a; optimum{b,c} gets18.

Unit-time scheduling as a matroid

Job j takes one unit, has deadline d_j, and late penalty w_j. A set A can all finish on time exactly when sorted deadlines satisfy d_(k)≥k, equivalently for every t,

`N_t(A)=|{j in A:d_j<=t}|<=t.`

Necessity follows from capacity t in the first t slots. For sufficiency, schedule by increasing deadline: job k's deadline≥k makes it on time.

All jobs are ready at0, nonpreemptive, on one machine. Deadlines are positive integers and completion exactly at d_j is on time. n jobs need only n slots, so cap deadlines above n at n without changing feasibility.

Heredity holds. For augmentation with|A|<|B|, let k be the largest integer with N_k(A)=k, including0. Since|A|<|B|≤n, k<n. A has exactly k jobs with deadline≤k, while B has at most k. Thus B has at least|B|−k>|A|−k jobs with deadline>k. Some x∈B outside A has d_x>k.

Adding x leaves N_t unchanged for t≤k. For t>k, maximality implies N_t(A)<t, so integrality gives N_t(A)≤t−1. Adding at most1 preserves feasibility, proving augmentation.

Complete scheduling greedy trace

Job	Deadline	Penalty
a	3	9
b	1	8
c	3	7
d	2	6
e	1	5

Scan decreasing penalty: accept a; accept b with deadlines1,3; accept c with1,3,3. Reject d:1,2,3,3 has fourth deadline3<4. Reject e because N1=2>1.

On-time set{a,b,c} is scheduled b,a,c at completion times1,2,3. Place d,e at4,5; both are indeed late. Penalty is6+5=11. Total penalty35 minus maximum saved24 gives11.

Complete scheduling greedy trace (continued)

A length-n deadline-frequency array yields all prefix N_t and checks independence in O(n). n tests cost $O(n^2)$, plus O(n log n) initial penalty sorting. Sorting separately for every oracle costs O(n log n) each and $O(n^2 \log n)$ overall; sorting is not linear.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Sources

Bibliography

Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 3rd ed., MIT Press, 2009, Chapter16, § § 16.1–16.5.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

References and attribution

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Elementary graph algorithms

Part VI Graph Algorithms

Scope

Search order determines distances, recursion ancestry, and finish times. On one six-vertex directed graph, FIFO layers and the white-path theorem establish the correctness and linear cost of BFS, DFS, topological sorting, and SCC decomposition.

Prerequisites

Graph representations, colors, timestamps, and linear traversals.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Chapter route

Learning objectives

- Choose adjacency matrices or lists.
- Derive the shortest-path property of BFS.
- Use DFS for topological sorting and SCCs.
- 22.1 Representations of graphs
- 22.2 Breadth-first search
- 22.3 Depth-first search
- 22.4 Topological sort
- 22.5 Strongly connected components

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

22.1 Representations of graphs

Graphs here are finite and either directed or undirected, with stable vertex identifiers and explicitly specified edge directions. A path may have length zero, so every vertex reaches itself. Let $n=|V|$ and $m=|E|$, counting each undirected edge once. Unless stated otherwise, shortest distance counts **edges**, rather than arbitrary edge weights.

An adjacency list has n headers. Its lists contain m entries for a directed graph and 2m for an undirected simple graph. Space is $\Theta(n+m)$. Enumerating u's neighbors takes $\Theta(1+\text{outdegree}(u))$; testing (u,v) by scanning an ordinary unsorted list takes that same worst-case time. Hashing or sorting changes lookup guarantees and must not be assumed for an ordinary list.

In an adjacency matrix A, rows are sources and columns are destinations. $A[u,v]=1$ means $u \to v$. Space is $\Theta(n^2)$, edge lookup $\Theta(1)$, and enumerating u's neighbors generally scans n entries. An undirected graph satisfies $A=A^{\mathsf{T}}$. A weighted matrix needs a separate absence marker because zero can be a valid edge weight.

One six-vertex directed graph

BFS, DFS, and SCC below use **these same directed edges**, with outer vertex order and adjacency order as displayed.

Vertex	Adj[u]	outdegree	indegree
a	b,c	2	1
b	c	1	2
c	a	1	2
d	b,e	2	1
e	d	1	2
f	e	1	0

Here n=6,m=8. The list lengths sum to 8, as do the indegrees, rather than 16. To construct G^{T}, initialize n empty lists and scan all m entries, placing u in $\text{Adj}^{\mathsf{T}}[v]$ for every $u \to v$. Time and additional space are $\Theta(n+m)$; the original graph need not change.

One six-vertex directed graph (continued)

One graph as lists and a matrix

Adj[u]: row u → column v. Directed graph, m=8.

Vertex	Adjacency list
a	b,c
b	c
c	a
d	b,e
e	d
f	e

Columns of A

	a	b	c	d	e	f
a	0	1	1	0	0	0
b	0	0	1	0	0	0
c	1	0	0	0	0	0
d	0	1	0	0	1	0
e	0	0	0	1	0	0
f	0	0	0	0	1	0

Green cells are directed edges: 8 cells correspond to 8 list entries.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

22.2 Breadth-first search

BFS maintains distance layers with a FIFO queue. White means undiscovered, gray discovered but not fully scanned, and black fully scanned. The discovery predecessor is π[v]; ∞ means no known distance and NIL no predecessor. Neither can be replaced by zero.

```
BFS(G,s):
  for v in V: color[v]=WHITE; d[v]=∞; π[v]=NIL
  color[s]=GRAY; d[s]=0; Q=empty queue; enqueue(Q,s)
  while Q is not empty:
    u=dequeue(Q)
    for v in Adj[u]:
      if color[v]==WHITE:
        color[v]=GRAY; d[v]=d[u]+1; π[v]=u
        enqueue(Q,v)
    color[u]=BLACK
  return d,π
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

22.2 Breadth-first search (continued)

Change color before enqueueing, so competing predecessors cannot enqueue a vertex twice. At the start of each while iteration, the queue contains exactly the gray vertices. While scanning u, however, u is gray and already dequeued. The invariant does not hold after every individual instruction.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Why FIFO processes nearer vertices first

List the queue from front to back as $q_1,\ldots,q_r$. Distances are nondecreasing and $d[q_r] \leq d[q_1]+1$; regard this condition as true for an empty queue.

Initially only s is queued. If u is removed with $d[u]=k$, the remaining distances, by induction, are k or k+1 in nondecreasing order. Every new vertex receives distance k+1 and is appended, preserving order. If layer k empties, the next front has distance k+1; only a later iteration can generate distance k+2. Dequeued distances are therefore nondecreasing.

Every finite d[v] describes an actual predecessor path. The source has length zero, and discovering v appends $u \to v$ to u's path, giving length $d[u]+1$. Thus true shortest distance $\delta(s,v) \leq d[v]$. Distances decrease by one along predecessor links, so those links cannot cycle.

An actual path gives only an upper bound. Shortest-path correctness still requires ruling out an earlier discovery by a longer route.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Complete proof of BFS shortest distances

Suppose a reachable v ends with $d[v] \neq \delta(s,v)$, and choose such a v with minimum δ. It is not s. Let u precede v on a shortest path. Then $\delta(s,u)=\delta(s,v)-1$ and, by minimality, $d[u]=\delta(s,u)$. Vertex u has finite distance, is enqueued, and is eventually dequeued.

When $u \to v$ is scanned, if v is white, it receives $d[v]=d[u]+1=\delta(s,v)$, a contradiction. Otherwise let w have first discovered v. Vertex w was dequeued no later than u. Queue order gives $d[w] \leq d[u]$, hence $d[v]=d[w]+1 \leq \delta(s,v)$. Together with $\delta \leq d$ from the predecessor path, this again gives equality. Both cases contradict the choice of v. All reachable distances are correct. Discovering an unreachable vertex would give an actual path to it, also a contradiction, so its distance remains ∞.

Every reachable nonsource vertex has one predecessor, and following predecessors reaches s after exactly d[v] edges. These links form a BFS tree. Adjacency order can change the selected predecessors and shortest paths, but not distances.

Initialization takes $\Theta(n)$. Each vertex enters and leaves the queue at most once; each reachable adjacency list is scanned once. Total time is $O(n+m)$, with $O(n)$ additional space for the queue and arrays. A matrix scans n positions for every dequeued vertex, giving worst-case $\Theta(n^2)$.

Complete queue trace from d

The queue shown is its content **after scanning finishes** in that iteration. Parenthesized numbers are distances; newly discovered vertices receive predecessors immediately.

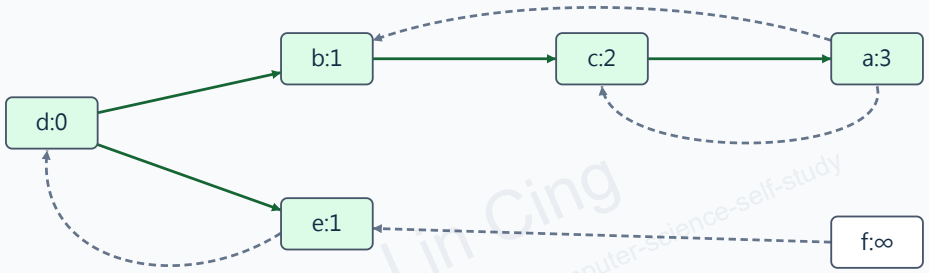
Removed u	Scan result	Queue at iteration end
Before the loop	Initialize d[d]=0	d(0)
d	b white: d[b]=1,π[b]=d; e white: d[e]=1,π[e]=d	b(1),e(1)
b	c white: d[c]=2,π[c]=b	e(1),c(2)
e	d already black; do not enqueue	c(2)
c	a white: d[a]=3,π[a]=c	a(3)
a	b,c already discovered; unchanged	empty

Finally $(d[a],d[b],d[c],d[d],d[e],d[f])=(3,1,2,0,1,\infty)$. A shortest path to a is $d \to b \to c \to a$. Edge $f \to e$ does not permit travel from d backward to f.

Complete queue trace from d (continued)

BFS from d on the same six-vertex graph

Boxes: vertex:distance. Green: predecessor edges. Gray dashed: other edges.



Dequeue: d(0) → b(1) → e(1) → c(2) → a(3). Vertex f stays at ∞.

Directed distances, equal weights, and path counts

For every edge $u \to v$ with reachable u, $\delta(s,v) \leq \delta(s,u)+1$. A directed graph provides no reverse inequality. Here $a \to b$ goes from distance 3 to 1, a difference of 2. Only when both directions of an undirected edge exist do we obtain $|d[u]-d[v]| \leq 1$.

For equal positive weight w, shortest weight is $w \cdot d[v]$. BFS fails for general weights: $s \to t$ of weight 9 competes with $s \to x$ and $x \to t$ of weight 1 each. The one-edge route costs 9, whereas the two-edge route costs 2. With all weights zero, every reachable route costs zero. Negative weights or cycles require a separately specified problem.

To count shortest paths in a unit-weight graph, initialize ways[s]=1 and all other counts to zero. Discover and enqueue white v as usual. For **every** edge $u \to v$ satisfying $d[v]=d[u]+1$, add ways[u] to ways[v], including edges into an already discovered v. Count each simple-graph edge once. All distance-$(k-1)$ vertices finish before distance-k vertices are dequeued, so ways[u] is complete when u is dequeued.

Directed distances, equal weights, and path counts (continued)

For edges 0→1,0→2,1→3,2→3,1→4,2→4,3→5,4→5, distances are (0,1,1,2,2,3), and ways= (1,1,1,2,2,4). This uses O(n+m) additions. Counts may occupy Θ(n) bits, so arbitrary-precision additions are not unconditionally constant-time.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

22.3 Depth-first search

DFS completes the current recursive branch before advancing to another neighbor. The outer loop must cover all vertices, including those unreachable from the first root. Here time counts **discovery and finish events**, not scanned edges.

```
DFS(G,order):
  for v in V: color[v]=WHITE; π[v]=NIL
  time=0; finished=empty list
  for u in order:
    if color[u]==WHITE: Visit(u)
Visit(u):
  time=time+1; tin[u]=time; color[u]=GRAY
  for v in Adj[u]:
    if color[v]==WHITE: π[v]=u; Visit(v)
  color[u]=BLACK; time=time+1; tout[u]=time
  append(finished,u)
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

22.3 Depth-first search (continued)

Each vertex is visited once and each adjacency entry scanned once, taking Θ(n+m) time. Recursion depth is at most n and additional space O(n). An iterative implementation must retain the next neighbor or an explicit finish event in each stack frame. Pushing all neighbors at once with a visited flag does not automatically reproduce the same forest or finish times.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Discovery and finish intervals nest or separate

For distinct u,v, suppose tin[u]<tin[v]. If v is discovered before u finishes, Visit(u) remains on the stack. Vertex v is its descendant and must return first, giving tin[u]<tin[v]<tout[v]<tout[u]. If v is discovered after u finishes, tin[u]<tout[u]<tin[v]<tout[v], so the intervals are disjoint.

Thus tin[u]<tin[v]<tout[u]<tout[v] is impossible. Conversely, strict interval containment means u was still on the stack when v was discovered, so v is a proper descendant. This parenthesis theorem depends on last-in-first-out recursive returns.

DFS on the six-vertex graph in a,b,c,d,e,f order gives the table. Its parenthesis sequence is (a(b(c c)b)a)(d(e e)d)(f f); every timestamp from 1 through 12 appears exactly once.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Discovery and finish intervals nest or separate (continued)

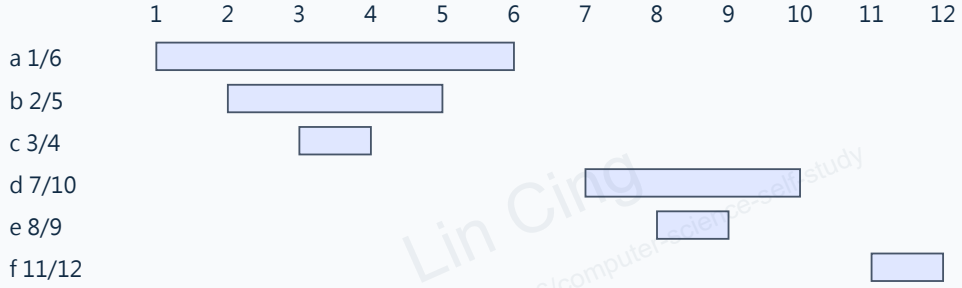
Vertex	π	tin	tout
a	NIL	1	6
b	a	2	5
c	b	3	4
d	NIL	7	10
e	d	8	9
f	NIL	11	12

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Discovery and finish intervals nest or separate (continued)

DFS discovery and finish events

Bars run from tin to tout. Descendants nest; different root intervals are disjoint.



a contains b,c; d contains e; f is the third root. Vertex intervals never interleave.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

The time condition in the white-path theorem

Immediately **before Visit(u) changes u from white to gray**, v is a descendant of u in this DFS, allowing v=u, if and only if there is a u-to-v path whose vertices are all white at that instant. This time convention includes the starting vertex u itself.

For the forward implication, the forest path from u to v uses original graph edges. All proper descendants are discovered after tin[u], so every vertex on this path is white at the specified instant.

Conversely, take the first vertex y on a white path that supposedly fails to become a descendant of u. Its predecessor x is already such a descendant, possibly u. When Visit(x) scans x→y, a still-white y is recursively visited and becomes a descendant. If y has been discovered already, it was white at the initial instant and was therefore discovered during u's unfinished recursive call, also within u's subtree. Both cases contradict the assumption.

The requirement **all white at that instant** is essential. In a→b,a→c,c→b, scan b first. By c's discovery, b is black. Although c reaches b, b will not become a descendant of c. Ordinary reachability cannot replace the white-path condition.

Four edge types in directed DFS

While scanning u→v, u is the deepest gray vertex, and all gray vertices form exactly the stack's ancestor chain. White v gives a tree edge. Gray v gives a back edge, including a self-loop. For black v, tin[u]<tin[v] gives a forward edge; otherwise it is a cross edge.

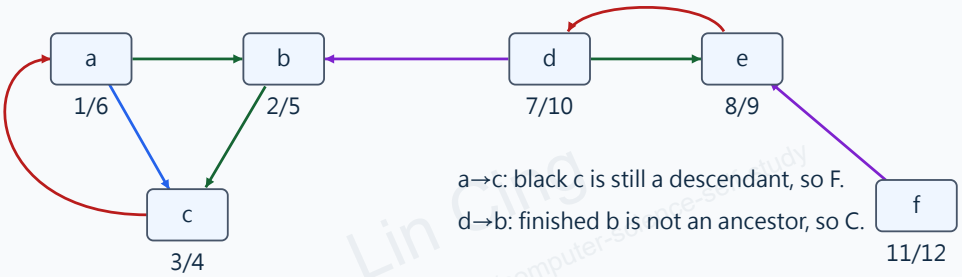
Type	Forest relationship	Timestamp check after DFS
tree	π[v]=u	tin[u]<tin[v]<tout[v]<tout[u]
forward	Nontree edge to a proper descendant	Same inequality; timestamps alone do not distinguish tree/forward
back	v is an ancestor of u, or v=u	For a proper ancestor: tin[v]<tin[u]<tout[u]<tout[v]
cross	Neither is the other's ancestor	tout[v]<tin[u]; preserve the scanned direction

The actual scan order is a→b(T),b→c(T),c→a(B),a→c(F),d→b(C),d→e(T),e→d(B),f→e(C). Edge a→c is scanned after c finishes. A black destination need not mean a cross edge.

Four edge types in directed DFS (continued)

Four directed edge types in one DFS

Green T: tree. Red B: back. Blue F: forward. Purple C: cross.



Labels are tin/tout. All 8 edges remain; their directions determine their types.

Undirected edge types and cycle tests

An undirected edge occurs in two adjacency lists. Classify the edge using the direction of its **first scan**. Suppose tin[u]<tin[v]. Since u lists v, v is discovered before u finishes. If u→v is the first scan, v must still be white, or v→u would already have been scanned; it is a tree edge. If v→u is scanned first, u is gray and it is a back edge. Thus undirected DFS has only tree/back edges, with no genuine forward/cross edges.

In a simple undirected graph, the reverse adjacency entry of the parent edge is not an additional cycle. With parallel edges, skip the same parent edge by **edge identifier**, rather than skipping all edges to the parent. A second parallel edge can form a length-two cycle under the multigraph convention.

A directed graph has a cycle if and only if DFS has a back edge. A back edge u→v, together with the tree path v to u, gives a cycle. Conversely, choose the earliest-discovered vertex v on a cycle. All other cycle vertices are white then. The cycle path to v's predecessor u is a white path, so the white-path theorem makes u a descendant of v. Edge u→v is a back edge. A self-loop satisfies the same conclusion directly.

22.4 Topological sort

A topological order permutes all vertices so every edge u→v satisfies pos[u]<pos[v]. A directed cycle would require strict increases around a loop back to the starting position, which is impossible. Exactly DAGs admit such an order.

Run DFS on a DAG and reverse its finish list; comparison sorting is unnecessary. For every scanned u→v, v cannot be gray, since that would give a cycle. If white, v becomes a descendant and tout[v] <tout[u]. If black, v has finished already and the same inequality holds. Decreasing finish order therefore directs every edge forward. Time is Θ(n+m), with O(n) additional space.

An implementation can report a cycle upon scanning a gray destination, and deliver the order only after validation. A reverse-finish list always exists, but it is not automatically a topological order for an arbitrary directed graph.

For a→c,b→c,c→d and outer order a,b,c,d, DFS gives a=1/6,c=2/5,d=3/4,b=7/8. Reverse finish order is b,a,c,d. Order a,b,c,d is also valid. **Order a,c,b,d is invalid because it reverses b→c.**

Kahn's indegree updates and failure condition

Compute indegrees and enqueue all zero-indegree vertices. Output a removed u, decrement indegree[v] for every u→v, and enqueue v exactly when its value becomes zero. The counts equal indegrees in the subgraph induced by vertices not yet output. Thus u has no unprocessed predecessor when output, and every output prefix respects edge order.

If a nonempty remainder has no zero-indegree vertex, repeatedly follow a remaining incoming edge from any vertex. Finiteness forces a repeated vertex and hence a cycle. Conversely, every nonempty induced subgraph of a DAG has a zero-indegree vertex, so all n vertices are output. Fewer than n outputs means failure. **A remaining vertex can lie downstream from a cycle without lying on one.**

Kahn's indegree updates and failure condition (continued)

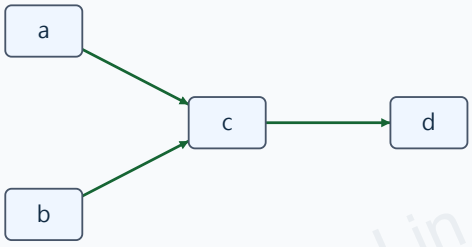
Output u	Remaining indegree of c	Remaining indegree of d	Queue at iteration end
Initial	2	1	a,b
a	1	1	b
b	0	1	c
c	0	0	d
d	0	0	empty

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Kahn's indegree updates and failure condition (continued)

Kahn: indegrees and queue states

DAG: a→c, b→c, c→d. Remove a source, then decrement its destinations.



Operation	in(c)	in(d)	Queue
Initial	2	1	a,b
Remove a	1	1	b
Remove b	0	1	c
Remove c	0	0	d
Remove d	0	0	empty

Output a,b,c,d. Order a,c,b,d reverses b→c and is invalid.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

22.5 Strongly connected components

Define u~v exactly when u reaches v and v reaches u. Zero-length paths give reflexivity; the definition gives symmetry; concatenating paths gives transitivity. This equivalence relation partitions vertices into SCCs, the inclusion-maximal mutually reachable sets, **not merely the class with the greatest cardinality**.

The condensation has one vertex per SCC and an edge for every original edge between distinct classes. It is a DAG: a cycle of distinct classes would make all those classes mutually reachable, contradicting their separation. Reversing every edge reverses paths in both directions, so G and G^T have identical SCCs.

For the six-vertex graph, A={a,b,c},D={d,e},F={f}, with condensation F→D→A. In G^T the direction is A→D→F. Starting arbitrarily at a in G^T can reach all three SCCs. **Transposition alone does not prevent crossing components; root order and visited marks are essential.**

Why finish times order different SCCs

Use timestamps from the **first DFS on G**. Define tin(C)=min tin[v] and F(C)=max tout[v] over v∈C. For distinct SCCs with C→D, F(C)>F(D). There are two discovery cases.

1. If tin(C)<tin(D), the first discovered x in C can follow white paths inside C to the outgoing edge, cross to D, and traverse D internally. The white-path theorem puts every other C vertex and every D vertex below x. Thus F(C)=tout[x]>F(D). Paths between C vertices can be chosen entirely inside C: any outside vertex on such a path would also be mutually reachable with C and therefore belong to it.
2. If tin(D)<tin(C), the first y in D reaches every white vertex of D, so F(D)=tout[y]. No path leads from D to C, or C→D would make them one SCC. Hence y's recursion cannot discover C. C is discovered after y finishes, implying F(C)>F(D).

Why finish times order different SCCs (continued)

Consequently, a cross-component edge C→D in G^T satisfies F(C)<F(D). This compares **maximum finish times per component**, not every pair of individual finish times. For a↔b,a→c, scanning a→b first gives tout[b]=3,tout[c]=5,tout[a]=6. Then F({a,b})=6>F({c})=5 even though b finishes before c. The actual cross-component edge a→c still has tout[a]>tout[c]. An edge between different SCCs cannot be a back edge.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Two complete DFS passes and the boundary proof

```
SCC(G):
  first=DFS(G,a fixed order of all vertices)
  roots=reverse(first.finished)    // save the first finish order
  GT=transpose(G)
  reset every vertex to WHITE, π=NIL for the second pass
  consider vertices in roots order; call Visit(u) on GT only if WHITE
  output each new root's entire DFS tree as one component
```

Inductively, every previously output tree is exactly one SCC. The next white root u lies in a component C with the greatest F(C) among all unprocessed components. If C's maximum-finish vertex were already processed, the induction hypothesis would make all of C processed, contradicting white u.

Paths internal to C reach every C vertex from u. All remaining C vertices are white, so the second pass collects all of them. For a cross edge C→D in G^T, F(D)>F(C), so D must already be black. Visit does not recurse into it, and cannot cross it to another white class. The tree collects neither too few nor too many vertices, completing the induction.

Two complete DFS passes and the boundary proof (continued)

Each DFS takes Θ(n+m), as does transposition. Reversing the appended finish list takes Θ(n); no O(n log n) sort is needed. Total time is Θ(n+m), and the transpose plus arrays use O(n+m) additional space.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Both passes on the same six-vertex graph

The first finish list is c,b,a,e,d,f, giving second-pass candidates f,d,e,a,b,c. The transposed lists, alphabetized internally, are a:[c],b:[a,d],c:[a,b],d:[e],e:[d,f],f:[].

Candidate root	New discoveries in G^T	Cross-component edge encountered	Output
f	f	none	{f}
d	d,e	e→f; f already black	{d,e}
e	skip, already black	none	no new component
a	a,c,b	b→d; d already black	{a,b,c}
b,c	skip, already black	none	no new component

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Both passes on the same six-vertex graph (continued)

The values F(A)=6,F(D)=10,F(F)=12 decrease along the original condensation F→D→A. Reversing discovery order could place c ahead of a, but this particular example need not fail. Accidental success is not a correctness argument. The minimal counterexample is x→y: first-pass discoveries tin[x]=1,tin[y]=2 would start the second pass at y, whose transposed edge y→x incorrectly merges two SCCs.

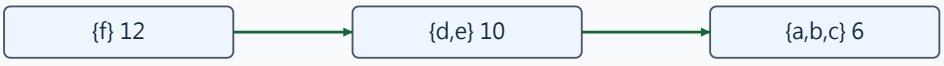
Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Both passes on the same six-vertex graph (continued)

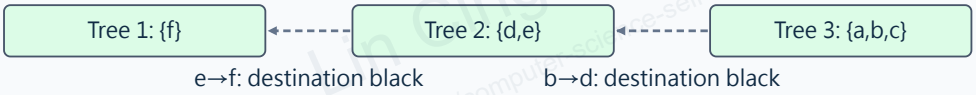
SCC: finish order prevents crossing in pass two

First-pass maxima in G: F({f})=12, F({d,e})=10, F({a,b,c})=6.

Original condensation DAG



Root order and black boundaries in GT



Green boxes collect one SCC each. Gray arrows cannot recurse into black vertices.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

References

Bibliography

Chapter source

Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 3rd ed., MIT Press, 2009, Chapter 22.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

References and attribution

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Disjoint sets: merge groups, never split them

The partition changes as groups merge. The query asks: **are these two elements in the same group now?**

Question	Method
How do we identify a set?	A list's set record, or a forest root
How much must a merge change?	Move the shorter list; attach the lower-rank root
Why can later finds get cheaper?	Path compression and amortized sequence analysis

Prerequisites: pointers, rooted trees, asymptotic notation, and the potential method. Rank, level, and iter are defined here.

Three operations maintain one partition

Every created element belongs to exactly one nonempty set; distinct sets are disjoint.

Operation	Precondition	Result
MAKE-SET(x)	x has not been created	Create singleton {x}
FIND(x)	x exists	Return its set's representative
UNION(x,y)	Both elements exist	Merge their sets; do nothing if already together

A representative identifies a set; it need not be its smallest key. Finds on one set return the same representative; different sets have different representatives.

Count calls separately from successful merges

Create n singletons first. Let m count external MAKE-SET, FIND, and UNION calls, so $m \geq n$. Do not count UNION's internal FIND calls again in m.

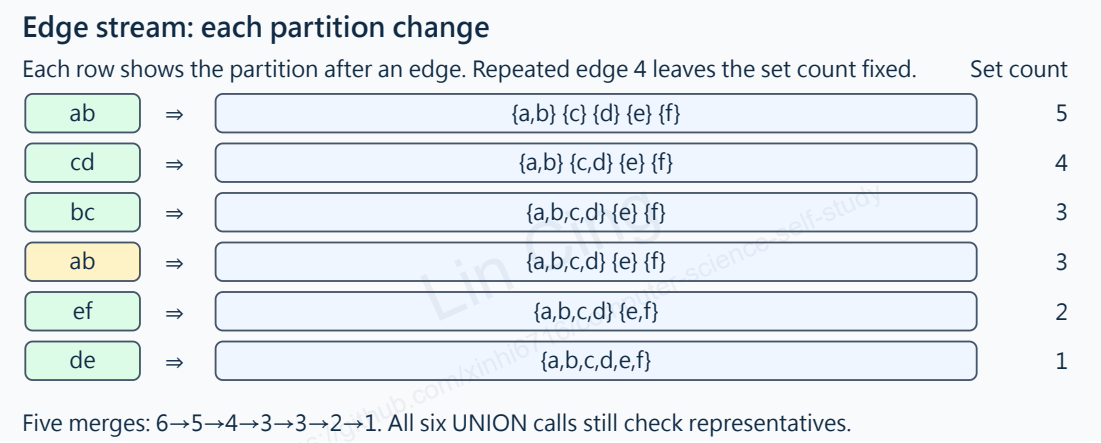
Quantity	Contract
Successful merges	At most n−1, because each removes one set
UNION calls	Repeated calls are allowed; no n−1 bound
LINK(r,s)	Accepts two **distinct roots**, not arbitrary elements

> UNION finds both roots and returns immediately if they agree. **Never LINK a root to itself or increase its rank for a repeated merge.**

FIND may change parents while preserving the partition. Deletion, splitting, and rollback require additional structures and analysis.

Undirected connectivity: what does a new edge merge?

Create a set for each vertex, then UNION each edge. The diagram uses ab, cd, bc, ab, ef, de.



The fourth edge ab is repeated: the call counts, but the partition does not change. Six UNION calls cause five successful merges.

Why are these exactly the connected components?

Invariant: the components of the processed-edge subgraph equal the maintained partition.

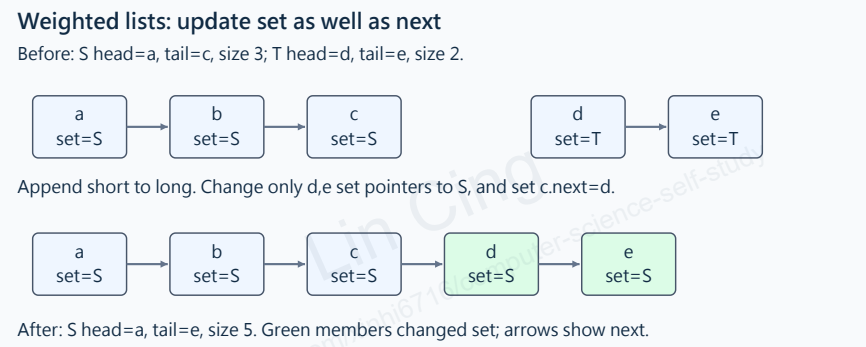
Step	Connectivity	Partition
No edges yet	Each vertex is isolated	One singleton per vertex
New edge within a component	Already connected	UNION changes nothing
New edge between components	Joins exactly those two components	UNION merges exactly their two sets

Induction gives FIND(u)=FIND(v) exactly when an undirected u–v path exists.

DFS/BFS finds static components in $\Theta(n+|E|)$. DSU also supports **connectivity queries as edges arrive**. Ignoring edge directions yields weak components, not SCCs.

Lists: joining tails does not finish the merge

A set record stores head, tail, size. Each member stores next and a set pointer. FIND(x) returns x.set.head in O(1).



> Merging a→b→c with d→e changes tail c→e and size 3→5. **Both d.set and e.set must point to the retained record** before their finds return a.

Finish every step of weighted-list UNION

Let S=x.set and T=y.set. Return for the same record; otherwise retain the longer list as S, with first-set ties.

Order	Update	Purpose
1	S.tail.next=T.head	Join the chains
2	Preserve T's old head; update S.tail and S.size	Keep the traversal start and update metadata
3	Follow T's next links; redirect every member's set to S	Make their finds return S.head
4	Discard T's record only after all redirects	Leave no dangling set pointers

Do not change T's internal next links during traversal. The merge costs $\Theta(1+\min(|S|,|T|))$: **a tail pointer makes joining constant-time, not the whole UNION.**

Why can unweighted lists take quadratic work?

Always append the second set to the first. In UNION(x_2,x_1), UNION(x_3,x_2), …, each new singleton absorbs the existing long list.

Merge	Members whose set pointers change
UNION(x_2,x_1)	1
UNION(x_3,x_2)	2
…	…
UNION(x_n,x_{n-1})	n−1

Total rewrites are n(n−1)/2. There are only n creations and n−1 unions, so m=2n−1, yet the cost is $\Theta(n^2)$.

Moving the shorter list: how often can one member move?

Fix member x. Whenever its set pointer changes, it belonged to the shorter list; its new set is **at least twice as large**.

After k moves its set has at least 2^k members, at most n. Thus $k \leq \lfloor \log_2 n \rfloor$. Sum over members and add the m constant call overheads: **$O(m+n \log n)$**, or $O(m+n \log(n+1))$ including small n.

Quantity	Bound or example
Whole sequence	$O(m+n \log n)$
One merge of two n/2-member sets	Still $\Theta(n)$
Balanced rounds with $n=2^k$	$n/2$ rewrites per round; $(n/2)\log_2 n$ total

> A sequence bound is not a per-call worst-case bound. Weighted union does not make every sequence linear.

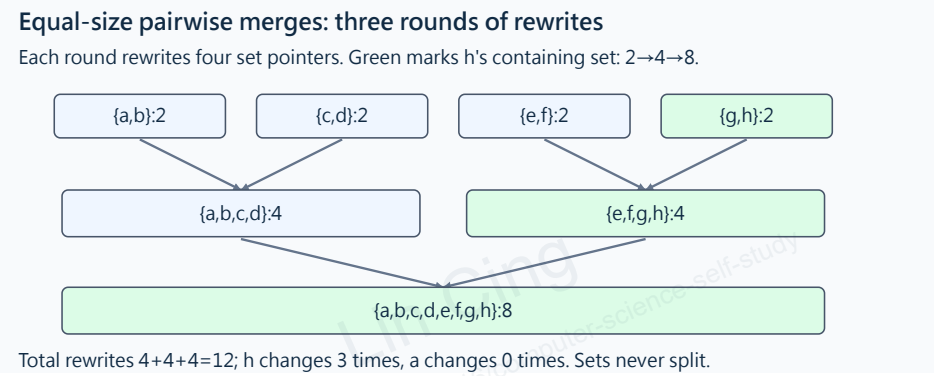
Eight elements: exactly which members move?

Retain the first set on equal sizes. The same operations will later build a forest.

UNION	Old sizes	Members rewritten	New representative / size	Cumulative
a,b	1,1	b	a / 2	1
c,d	1,1	d	c / 2	2
e,f	1,1	f	e / 2	3
g,h	1,1	h	g / 2	4
a,c	2,2	c,d	a / 4	6
e,g	2,2	g,h	e / 4	8
a,e	4,4	e,f,g,h	a / 8	12

Member h follows h→g→e→a as its set grows 1→2→4→8; a never moves. **Twelve total rewrites do not mean every member moves three times.**

One diagram connects round totals to one member's growth



Round totals and one member's move count have different denominators. The table sums to 12 rewrites; h alone moves three times.

Attaching only new singletons to a large set can instead take n−1 rewrites. This meets the upper bound; it does not make every possible sequence linear.

Forests: find the roots before linking

Each element stores parent and rank; a root points to itself. MAKE-SET sets parent[x]=x and rank[x]=0. UNION must first find both roots. Linking arbitrary x directly to y can split an existing set or create a cycle.

```
UNION(x,y):
    r=FIND(x); s=FIND(y)
    if r==s: return r
    return LINK(r,s)
LINK(r,s):                    // r and s are distinct roots
    if rank[r]<rank[s]: swap r,s
    parent[s]=r
    if rank[r]==rank[s]: rank[r]+=1
    return r
```

After linking roots, all old members still reach the new root. One root loses its self-parent, with no reverse path creating a cycle. Unequal ranks leave the retained rank unchanged; equal ranks increase it by one. Rank forests and size-weighted lists are different representations. Rank is not exact set size.

Two-pass FIND: locate the root, then follow the old path

Starting at x, follow parent pointers until reaching a self-parent root. Pass one only finds root r. Pass two redirects each node on the **original search path** directly to r.

```
FIND(x):
    r=x
    while parent[r]!=r: r=parent[r]
    while parent[x]!=x:
        old=parent[x]
        parent[x]=r
        x=old
    return r
```

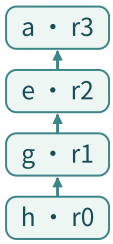
Variable old preserves the next stop before mutation. Using x=parent[x] instead reads the root just written and skips intermediate nodes. Recursive parent[x]=FIND(parent[x]) redirects nodes as calls return.

The two-pass version uses O(1) auxiliary space; the recursive stack scales with original path height. Both take Θ(s) work for s visited nodes, change only this path, and preserve ranks.

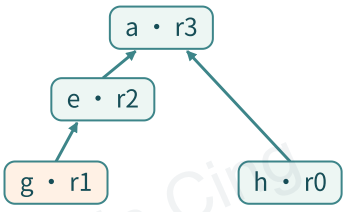
Finding the right root is not full path compression

Arrows point **child→parent**. Only h→g→e→a is drawn; a is self-parent.

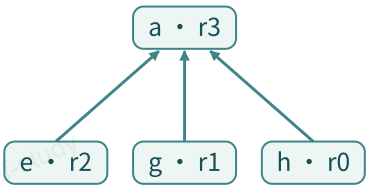
Original path



Jump to root after writing



Save old parent; finish path



Current node in pass two	Save old	Assign parent	Continue at
h	g	a	g
g	e	a	e
e	a	a, unchanged	a, then stop

The same eight elements before and after compression

For the seven earlier unions, equal ranks retain the first root. The successive root ranks are a1,c1,e1,g1,a2,e2,a3. Final parents are b→a, c→a, d→c, e→a, f→e, g→e, h→g; a points to itself.

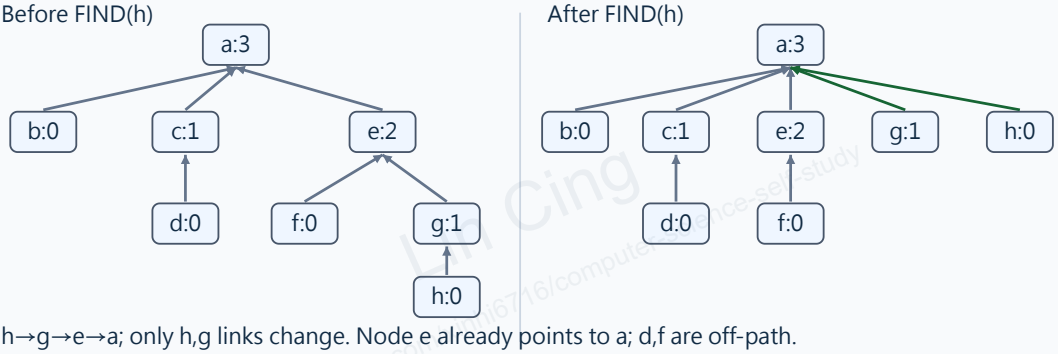
Vertex	a	b	c	d	e	f	g	h
rank	3	0	1	0	2	0	1	0
Parent before FIND(h)	a	a	a	c	a	e	e	g
Parent after FIND(h)	a	a	a	c	a	e	a	a

- Pass one follows h→g→e→a: three parent edges.
- Only **h and g change parent**; e already points to a. Off-path nodes do not change.
- **Every rank stays fixed.** Compression does not replace historical ranks with current heights.

Read the tree: redirect the path, retain the branches

The same eight-node tree: parents before and after compression

Nodes show key:rank; arrows show parent; root self-loops omitted; green links changed.



h→g→e→a; only h,g links change. Node e already points to a; d,f are off-path.

> A later FIND(h) takes one edge; FIND(d) still follows d→c→a. **One FIND compresses only its own search path.**

Why does rank bound tree height?

Rank is an **upper bound**, rather than a measurement of current height. Induct on operations:

Operation	Effect on height	Why height $\leq$ rank survives
Make a singleton	Height is 0	Initial rank is also 0
Merge unequal ranks	Attach the lower-rank root	The rank gap pays for its extra parent edge
Merge equal ranks	Height increases by at most 1	The retained root's rank also increases by 1
Compress a path	Redirect to ancestors, shortening paths	Ranks stay fixed

> **Do not recompute rank after compression.** A rank-2 root may now have height 1.

Rank remembers the members accumulated during merging

A root's rank rises only by merging two different sets of equal rank:

- Rank 0 requires at least one member.
- Two rank-(r−1) sets each contain at least 2^{r-1} members. Their merged rank-r root has at least 2^r.
- Unequal-rank merges add members; FIND preserves set membership.

A nonroot's rank freezes when it loses root status. **Its set then had at least 2^r members.** Thus $2^r \leq n$, giving $r \leq \lfloor \log_2 n \rfloor$. This is not a bound on its current subtree size.

> Rank alone gives $O(\log n)$ per FIND. Compression improves the **amortized sequence bound**; a first deep-leaf query can still traverse $\Theta(\log n)$ edges.

21.4 Fine-grained analysis of rank and compression

Ackermann iteration means composition

Fix the final number of elements n and set $\alpha = \alpha(n)$ throughout the analysis, even while MAKE-SET calls are still creating elements. For $j \geq 1$, define

$$f^{(0)}(j) = j, \quad f^{(i+1)}(j) = f(f^{(i)}(j)), \quad A_0(j) = j + 1, \quad A_k(j) = A_{k-1}^{(j+1)}(j) \ (k \geq 1).$$

The superscript counts **repeated applications to the previous output**. Since $A_0^{(i)}(j) = j + i$, we get $A_1(j) = 2j + 1$. Induction gives $A_1^{(i)}(j) = 2^i(j + 1) - 1$, hence

$$A_2(j) = 2^{j+1}(j + 1) - 1, \qquad A_3(1) = A_2(A_2(1)) = A_2(7) = 2047.$$

Inductively, each A_k is strictly increasing in its input and satisfies $A_k(j) \geq j + 1$: a larger starting input undergoes more iterations, each increasing its input. Since $j + 1 \geq 2$, also $A_k(j) > A_{k-1}(j)$. Both monotonicities are used below.

The chosen inverse-Ackermann convention

Use the single-parameter convention of CLRS third edition, Section 21.4:

$$\alpha(n) = \min\{k \geq 0 : A_k(1) \geq n\}.$$

k	0	1	2	3	4
$A_k(1)$	2	3	7	2047	greater than 10^{80}

Thus $\alpha(2) = 0$, $\alpha(8) = 3$, and $\alpha(2048) = 4$. Other texts may use a two-parameter convention; its numerical values cannot be substituted here.

The complete bound is $O(m(1 + \alpha(n)))$ for m external operations, including n MAKE-SET calls. Only for $n \geq 3$ can it be shortened to $O(m\alpha(n))$. Small instances still require constant work per operation.

Which quantities can change?

Induction on operations shows that a root points to itself and every nonroot has smaller rank than its parent. Linking equal-rank roots raises the retained root's rank by one. Unequal ranks attach the smaller rank under the larger. Compression changes parents, never ranks.

Once a node becomes a nonroot, its rank is fixed. Its parent's rank may increase, or compression may replace the parent by a higher ancestor. Neither event decreases parent rank. This is why a node's level cannot decrease over time.

A rank-r root contains at least 2^r elements. A nonroot retains that historical bound from losing root status. Hence every rank is at most $\lfloor \log_2 n \rfloor < n$, regardless of the current compressed subtree size.

Define level and iter only for positive-rank nonroots. Rank-zero nodes and roots receive separate potential formulas.

Level selects the scale; iter measures progress within it

For a nonroot, write $r = \operatorname{rank}(x) \geq 1$, $p = \operatorname{rank}(\operatorname{parent}(x))$, and define

$$\ell(x) = \max\{k \geq 0 : p \geq A_k(r)\}, \qquad i(x) = \max\{j \geq 1 : p \geq A^{(j)}_{\ell(x)}(r)\}.$$

Here $\ell = \operatorname{level}$ and $i = \operatorname{iter}$. First find the highest scale reached by parent rank; then count iterations at that scale.

Since $p \geq r + 1 = A_0(r)$, we have $\ell \geq 0$. Also $A_\alpha(r) \geq A_\alpha(1) \geq n > p$, so $\ell < \alpha$. The definition gives $i \geq 1$. If $i \geq r + 1$, then

$$p \geq A^{(r+1)}_\ell(r) = A_{\ell+1}(r),$$

contradicting maximality of ℓ. Therefore $0 \leq \ell < \alpha$ **and** $1 \leq i \leq r$.

Nonnegative potential records work still available

Assign a potential to each node and sum over the forest:

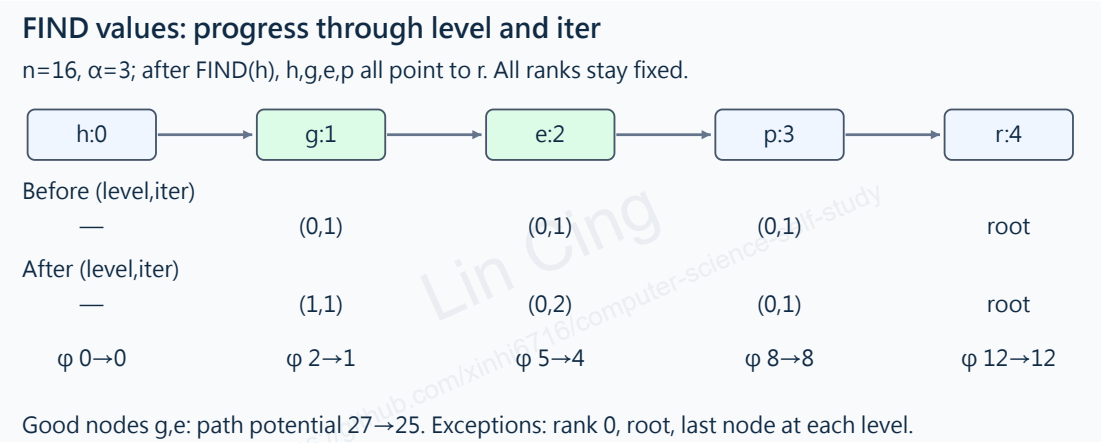
$$\phi(x) = \begin{cases} \alpha r, & x \text{ is a root or } r = 0, \\ (\alpha - \ell(x))r - i(x), & x \text{ is a nonroot and } r \geq 1, \end{cases} \qquad \Phi = \sum_x \phi(x).$$

For nonroots, the lower bound is $(\alpha - (\alpha - 1))r - r = 0$, and the upper bound is $\alpha r - 1$. Roots have potential αr; rank-zero nodes have zero potential. Thus Φ is always nonnegative.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Potential accounting on a five-node path

Take $n = 16$, $\alpha = 3$, and compress a path of ranks 0, 1, 2, 3, 4. The root keeps its self-parent.



Each good node releases one unit: g gains a level; e increases iter within one level. The other three potentials stay fixed, so the path total falls from 27 to 25.

Why can a rising level reset iter?

Fix the rank r of a nonroot. Within one level, increasing parent rank leaves iter unchanged or increases it. Each additional iteration decreases ϕ by one.

If the level rises from k to $k' > k$, iter may reset from i to a smaller i'. Nevertheless,

$$\Delta\phi = -(k' - k)r + (i - i') \leq -r + (r - 1) = -1.$$

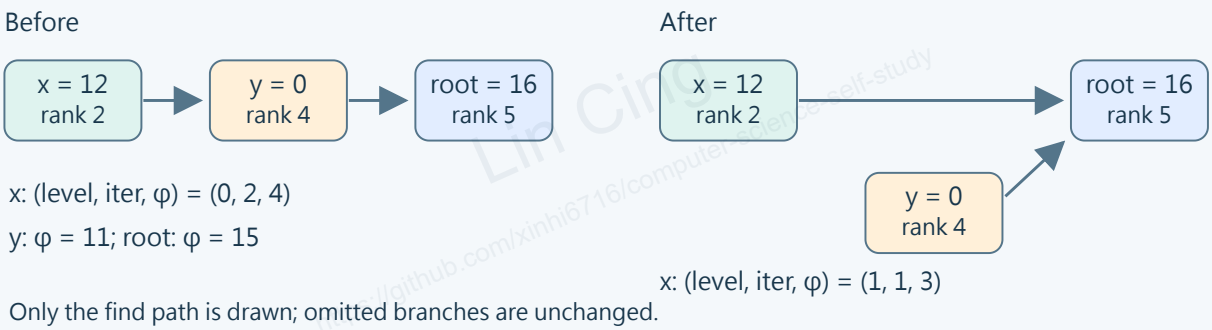
This uses $k' - k \geq 1$, $i \leq r$, and $i' \geq 1$. **The gain from changing level pays for resetting iter.** The pair progresses in lexicographic order, comparing level first, then iter. Its two coordinates need not increase independently.

A legal FIND that really decreases iter

Take $n = 32$ and $\alpha = 3$. The find path is $12 \to 0 \to 16$, with ranks $2, 4, 5$. Unvisited branches are omitted and remain unchanged.

A higher level can reset iter from 2 to 1

n=32, α=3; FIND(12)



Only the find path is drawn; omitted branches are unchanged.

Computing the reset example term by term

Node $x = 12$ keeps rank 2. Only its parent rank changes, from 4 to 5.

Quantity	Before compression	After compression
level	$A_0(2) = 3 \le 4 < A_1(2) = 5$, so 0	$A_1(2) = 5 \le 5 < A_2(2) = 23$, so 1
iter	$A_0^{(2)}(2) = 4 \le 4 < A_0^{(3)}(2) = 5$, so 2	$A_1(2) = 5 \le 5 < A_1^{(2)}(2) = 11$, so 1
$\phi(x)$	$(3-0)2 - 2 = 4$	$(3-1)2 - 1 = 3$

Node 0 keeps potential 11, and the root keeps potential 15. All other branches are unchanged. Thus the forest has $\Delta\Phi = (3 + 11 + 15) - (4 + 11 + 15) = -1$. The displayed totals sum only the path; the other 29 nodes need not have zero potential.

Constructing that path with legal operations

Create singleton keys 0 through 31. Equal-rank merges retain the first root.

Operation	Checkable result
Merge adjacent equal-size blocks of 0..15 in rounds	Root 0 has rank 4; node 12 has rank 2
FIND(12)	Parent of 12 becomes 0; ranks stay fixed
Build 16..31 by the same rounds	Root 16 has rank 4
UNION(16,0)	Root 16 rises to rank 5; parent of 0 becomes 16
FIND(12) again	Path $12 \to 0 \to 16$; parent of 12 becomes 16

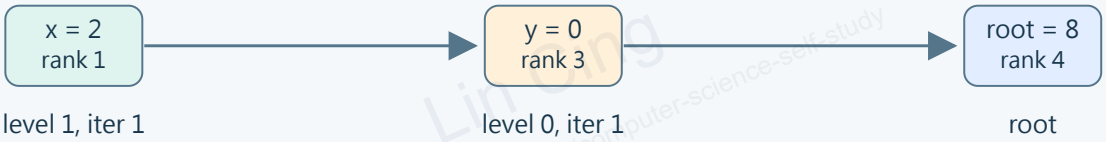
The first half starts with (0,1), (2,3), …, (14,15), then (0,2), (4,6), …, followed by (0,4), (8,12), and finally (0,8). These ranks come from a legal history rather than arbitrary assignments to a drawn chain.

Levels need not increase along a path

Take $n = 16$. Merge 0..7 into root 0 of rank 3, then call FIND(2). Build 8..15 into root 8 of rank 3 and call UNION(8,0). This creates the following legal path.

Increasing ranks do not imply increasing levels

n=16, α=3; FIND(2)



$A_1(1)=3 \le \text{rank}(y) < A_2(1)=7$; $A_0(3)=4 \le \text{rank}(\text{root}) < A_1(3)=7$

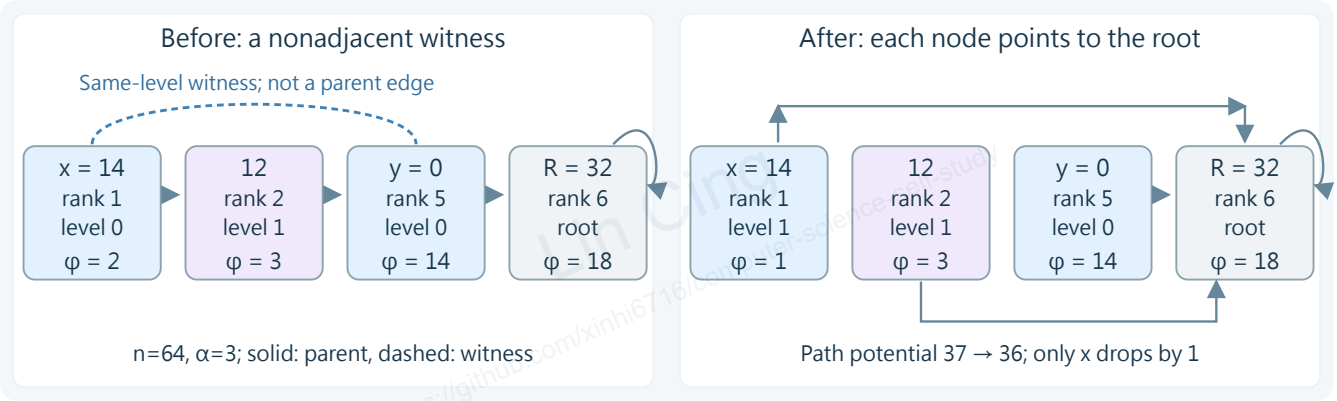
Along this path: ranks 1 < 3 < 4, but levels 1 > 0.

Count the last node of each level; no sorted-level assumption is needed.

Comparing different nodes changes the input rank too. Temporal monotonicity concerns one fixed node.

A same-level witness need not be the next node

A positive-rank nonroot x is good if a later nonroot y on the old path has the same level. **The witness need not be x's parent.**



In 14→12→0→32, levels are 0,1,0. Node 0 witnesses that 14 is good; node 12 may have another level. The root has no level.

Construct the path through legal operations

Start with 64 singletons. Equal-rank UNION retains its first root. Building an interval in rounds means merging adjacent singletons, then adjacent equal-sized blocks.

Operation	Fact established by the history
Build 0..31 in rounds	Root 0 has rank 5; node 12 has rank 2 and node 14 rank 1
FIND(12)	Old path is $12 \to 8 \to 0$; attach 12 to 0, while 14 still points to 12
Build 32..63 in rounds	Root 32 has rank 5
UNION(32,0)	Equal ranks: 32 rises to 6, and 0 points to 32
Before FIND(14)	Path $14 \to 12 \to 0 \to 32$, with ranks 1,2,5,6

FIND(12) follows $12 \to 8 \to 0$ upward to the root, then attaches each visited nonroot directly to that root. It never enters a child subtree, so 14 still points to 12. The later union attaches root 0 to root 32. These ranks follow from legal operations; 32 retains its self-parent.

Compute every level, iteration count, and potential

Fix $n = 64, \alpha = 3$. Level and iter apply only to positive-rank nonroots; the root uses $\phi = \alpha r$.

Node	Rank	Parent rank: old→new	Level: old→new	Iter: old→new	ϕ: old→new
14	1	2→6	0→1	1→1	2→1
12	2	5→6	1→1	1→1	3→3
0	5	6→6	0→0	1→1	14→14
32, root	6	6→6	Not defined	Not defined	18→18

For 14, initially $A_0(1) = 2 \le 2 < A_1(1) = 3$, so level=0,iter=1. Afterwards $A_1(1) = 3 \le 6 < A_2(1) = 7$, and $A_1^{(2)}(1) = 7 > 6$, so level=1,iter=1. Its potential drops from $(3-0)1 - 1 = 2$ to $(3-1)1 - 1 = 1$.

For 12, $5 = A_1(2) \le p < 11 = A_1^{(2)}(2) < A_2(2) = 23$, so level=1,iter=1. For 0, $A_0(5) = 6 \le 6 < A_0^{(2)}(5) = 7 < A_1(5) = 11$, so level=0,iter=1,potential=14. Path total changes $37 \to 36$. All other nodes keep rank,parent rank,root status; **the whole forest also loses exactly one unit**.

The good-node inequality, with a reason for each step

Freeze all quantities before compression. Write $\rho(v) = \mathrm{rank}(v)$, $p_x = \mathrm{parent}(x)$, $p_y = \mathrm{parent}(y)$, and let R be the root. Put $r = \rho(x), k = \ell(x) = \ell(y), i = i(x)$.

Step	Reason
$\rho(R) \geq \rho(p_y)$	p_y is on the root path; rank strictly increases along nonroot parent edges
$\rho(p_y) \geq A_k(\rho(y))$	Definition of $\ell(y) = k$
$A_k(\rho(y)) \geq A_k(\rho(p_x))$	y equals or is above p_x, so $\rho(y) \geq \rho(p_x)$; A_k increases
$A_k(\rho(p_x)) \geq A_k(A_k^{(i)}(r))$	Iter gives $\rho(p_x) \geq A_k^{(i)}(r)$; use monotonicity again
$A_k(A_k^{(i)}(r)) = A_k^{(i+1)}(r)$	One more function composition, not multiplication

The new parent rank is therefore at least $A_k^{(i+1)}(r)$. No step assumes sorted path levels: the proof uses ancestor rank order and monotonicity at one fixed level k.

Crossing a threshold pays for a good node

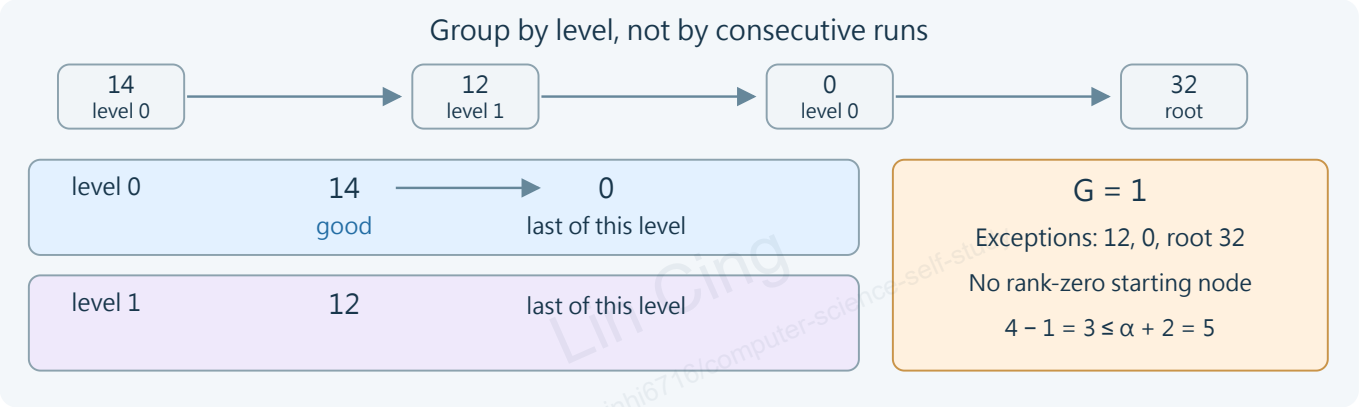
Compression makes R the parent of x, keeping r fixed. The proof crosses the **next iteration threshold of the old level**. It does not guarantee that the new iter exceeds the old one; distinguish two cases.

After compression	Potential change	Why it decreases
Level stays k	$\Delta\phi = -(i' - i) \leq -1$	Iter is at least $i + 1$
Level rises	$\Delta\phi = -(k' - k)r + i - i'$	$k' - k \geq 1, i \leq r, i' \geq 1$, so $\Delta\phi \leq -r + (r - 1) = -1$

In the 64-node example, $x = 14, y = 0, k = 0, i = 1$; the inequalities become $6 \geq 6 \geq 6 \geq 3 \geq 3$. The next old-level threshold is $A_0^{(2)}(1) = 3$. New parent rank 6 also crosses $A_1(1) = 3$, so this is the second case and potential drops by one.

Charge the decrease to good node x, not again to witness y; y's potential need not drop. Other nodes' potentials do not increase, so the good-node decreases may be summed.

Only the last node of each level is an exception



Within one level, every node except the last has a later same-level witness. Here 14 is good; 12 and 0 are the last nodes of levels 1 and 0. Adding root 32 gives three exceptions. Treating consecutive runs 0/1/0 as three groups would miss good node 14.

Convert group counts into total potential decrease

An s-node path has at most α positive-rank nonroot levels, numbered $0, \dots, \alpha - 1$. At most one last node per level, the root, and a possible rank-zero starting node give at most $\alpha + 2$ exceptions. Strictly increasing ranks allow at most one rank-zero node.

$$G \geq g := \max(0, s - (\alpha + 2)), \qquad \Delta\Phi \leq -G \leq -g.$$

G counts actual good nodes; g is only a conservative lower bound. Here $s = 4, \alpha = 3$, hence $g = 0$, yet $G = 1$ and $\Delta\Phi = -1$. A short path need not attain the bound. The next argument uses the general lower bound to pay for long paths.

Fixed constants pay for the entire scan

Suppose two-pass find costs at most cs. Fix $C > c$ and use scaled potential $\Psi = C\Phi$. The amortized cost satisfies

$$\hat{c} = c_{\text{actual}} + \Delta\Psi \le cs - Cg.$$

Path length	Bound after substituting g
$s \le \alpha + 2$	$cs \le c(\alpha + 2)$
$s > \alpha + 2$	$(c - C)s + C(\alpha + 2) \le C(\alpha + 2)$

Thus each FIND has amortized cost $O(1 + \alpha)$. For $s = 7$, $\alpha = 3$, $c = 2$, $C = 3$, the lower bound is $g = 2$, so $\hat{c} \le 14 - 6 = 8$. Keep the same C throughout the sequence.

Only the retained LINK root can gain potential

MAKE-SET adds a rank-zero node with zero potential and still has amortized cost $O(1)$. Suppose LINK attaches old root u under retained root v.

Possibly affected nodes	Reason for the potential change
u	Loses root status without changing rank; potential does not rise
Old children of v	Parent rank cannot fall; the earlier monotonicity argument applies
v	Unequal ranks leave it fixed; equal ranks add one, increasing potential by α

Every other node keeps its rank, parent rank and root status. Therefore $\Delta\Psi \le C\alpha$, and LINK costs $O(1) + C\alpha = O(1 + \alpha)$ amortized. UNION performs two FIND calls and links only distinct roots, expanding into at most three operations.

Recovering actual sequence cost

Let c_j be actual cost and $a_j = c_j + \Psi_j - \Psi_{j-1}$ amortized cost. Summing cancels intermediate potentials:

$$\sum_{j=1}^{m'} c_j = \sum_{j=1}^{m'} a_j + \Psi_0 - \Psi_{m'} \leq \sum_{j=1}^{m'} a_j.$$

The inequality uses the empty initial forest, $\Psi_0 = 0$, and nonnegative final potential. Expanding UNION gives $m' \leq 3m$, so the original m operations cost $O(m(1 + \alpha(n)))$ in total.

This is a worst-case bound on the entire sequence. With rank and compression, a single FIND can still cost $\Theta(\log n)$: build a balanced rank tree and first query a deepest leaf. A linear chain requires removing the rank rule and does not satisfy this theorem's hypotheses.

Potential analysis reference: CLRS, third edition, Section 21.4, printed pp. 576–581.

References and attribution

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Minimum spanning trees

Part VI Graph Algorithms

Scope

Safe edges across cuts reduce the cost of connectivity. Exchange arguments establish Kruskal and Prim. One five-vertex graph tracks components, keys, and parents, leading to independently checkable certificates of optimality and uniqueness.

Prerequisites

Connected undirected graphs, cuts, and weights.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Chapter route

Learning objectives

- Derive MST safety from the cut property.
- Compare Kruskal and Prim, including their structures and costs.
- Identify assumptions about negative weights, disconnected graphs, and uniqueness.
- 23.1 Growing a minimum spanning tree
- 23.2 The algorithms of Kruskal and Prim

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

23.1 Growing a minimum spanning tree

Minimize total tree weight, not each path length

Let G=(V,E) be a finite undirected graph, with n=|V|,m=|E| and comparable finite real edge weights w(e). First assume n≥1 and G connected. A spanning tree T contains all vertices, is connected and acyclic, and has exactly n−1 edges. An MST minimizes $w(T) = \sum_{e \in T} w(e)$. There are finitely many candidate trees, so a minimum exists. Weights may be negative, zero, or repeated.

Negative weights do not invalidate MSTs, but acyclicity remains essential. In a triangle with all weights−1, a tree costs−2, whereas the general connected subgraph using all edges costs−3. An MST need not be a shortest-path tree. For ab:4,ac:5,bc:2, the MST {ab,bc} costs 6, and its a-to-c path has length6; the shortest graph path ac has length5.

For n=1, return no edges and cost 0; for n=0, use the empty-forest convention. A disconnected graph has no spanning tree. Its minimum spanning forest, MSF, consists of an MST in each original component. Self-loops belong to no tree. Parallel edges retain distinct edge IDs, so endpoints alone do not identify a removable edge.

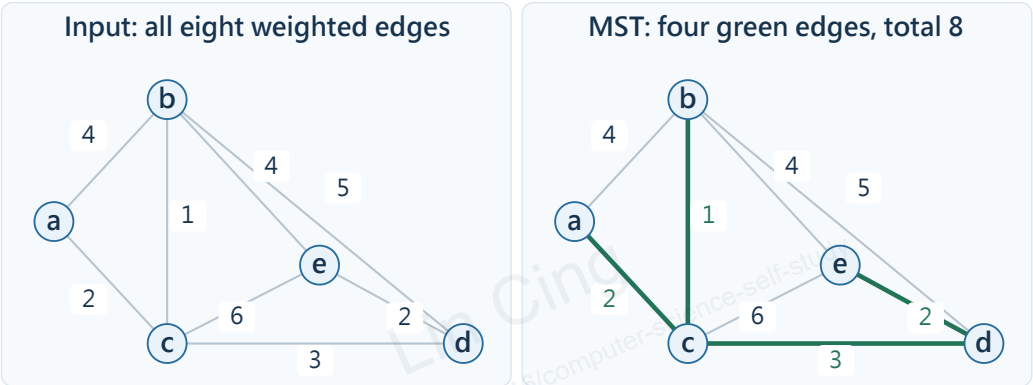
One five-vertex graph, two growth orders

Use the following fixed input throughout. Kruskal breaks weight ties by edge ID.

ID	0	1	2	3	4	5	6	7
Edge	ab	ac	bc	bd	cd	ce	de	be
Weight	4	2	1	5	3	6	2	4

Kruskal selects bc,ac,de,cd; Prim from a selects ac,cb,cd,de. Both end with {bc,ac,de,cd}, costing1+2+2+3=8, but their intermediate forests and grown vertex sets differ. Gray edges also belong to the input. The final green tree alone does not determine the selection process.

One five-vertex graph, two growth orders (continued)



Safe means extendible to an optimal tree

Maintain that $A \subseteq E$ is contained in at least one MST. An edge $e \notin A$ is safe for A if some MST contains $A \cup \{e\}$. This optimal-extension requirement is stronger than adding e without a cycle, which only preserves a forest.

A cut $(S, V-S)$ requires $\varnothing \neq S \neq V$. An edge crosses it when its endpoints lie on opposite sides. The cut respects A if no A edge crosses it. A light edge has minimum weight among the crossing edges, possibly tied.

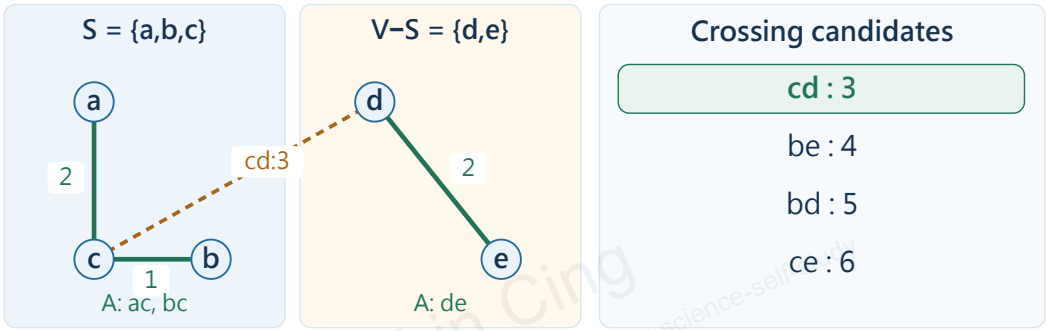
The generic algorithm starts with $A=\varnothing$ and repeatedly adds a safe edge until $|A|=n-1$. The invariant ensures acyclicity. A forest on n vertices with $n-1$ edges is connected, so the result is an MST. The algorithmic task is to establish safety efficiently; choosing a cheap edge is not itself a proof.

Safe-edge theorem and preservation of selected edges

Suppose A extends to an MST, cut (S,V−S) respects A, and e is a light edge of the cut. Choose an MST T containing A. If e∈T, the claim already holds. Otherwise T+e has exactly one cycle. Since e crosses the cut, the cycle must cross it again at some edge f to return to its starting side. Since the cut respects A, f∉A.

Removing f breaks the cycle while preserving connectivity, so $T' = T - \{f\} + \{e\}$ is a spanning tree. Because $w(e) \le w(f)$, its cost is at most w(T). Optimality of T makes T′ an MST containing $A \cup \{e\}$. The exchange proves structural validity, nonincreasing cost, and preservation of A.

Safe-edge theorem and preservation of selected edges (continued)



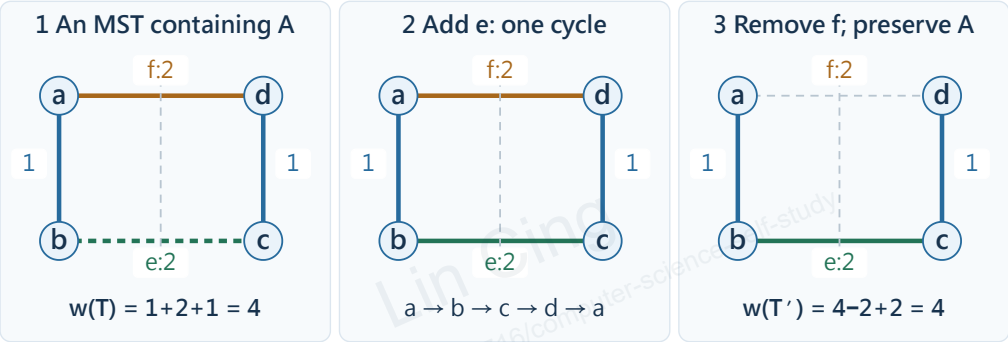
No selected edge crosses the cut; cd is the lightest crossing edge.

Respect A: preserve selected edges, then safely add cd

Here S={a,b,c},A={ac,bc,de}. Crossing edges are cd:3,be:4,bd:5,ce:6, so cd is safe. Edge de lies entirely on the other side.

An exchange in three stages

Four vertices: ab:1, cd:1, ad:2, bc:2, ac:4, bd:5. Fix A={ab,cd} and cut {a,b}|{c,d}.



Blue A={ab,cd} is preserved. f is outside A. Dashed green e is the candidate.

Equal-weight exchange: same optimum, now containing e

T attains the three-edge lower bound 1+1+2=4. Add e=bc, then remove f=ad from cycle a-b-c-d-a. Cost stays 4 and A is preserved. **Some MST contains A and e; the original T omits e.**

Quantifiers for ties, forced edges, and cycles

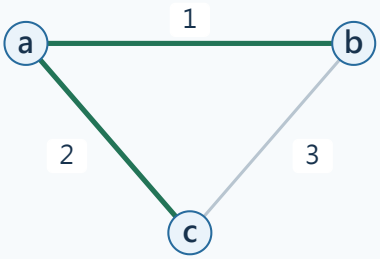
If e is the **unique** light edge of a cut, an MST omitting it could be strictly improved by the exchange. Thus every MST contains e. A tied light edge need only belong to some MST. A triangle with all weights 1 has three MSTs; every edge is light for some cut, but all three cannot be selected together.

Cycle property: a uniquely heaviest edge e on a cycle belongs to no MST. If an MST T contained e, removing it would define a cut. The remaining original cycle path supplies a crossing edge f with w(f)<w(e), giving a strict improvement. If e is only tied for heaviest, a nonincreasing exchange proves that some MST omits it, not that every MST must omit it.

Every component C of forest A defines a respecting cut (C,V−C). Its lightest outgoing edge is therefore safe. Kruskal and Prim share this argument; neither requires selecting the globally cheapest edge at every iteration.

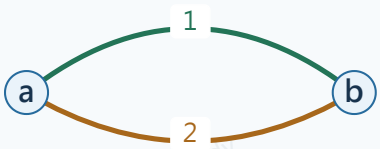
The second-lightest edge depends on the graph model

Simple graph: the first three edges



Second edge can occur; third need not

Parallel edges invalidate the claim



The unique MST uses only weight 1

The second-edge claim requires a connected simple graph

Let e be second by nondecreasing weight, with arbitrary ties. If an MST T omits e, the cycle in T+e has at least two other edges, but at most one edge is strictly lighter than e. Some cycle edge f has $w(f) \geq w(e)$; removing f gives an MST containing e. **Parallel edges invalidate the cycle-length premise. The third edge has no such guarantee.**

23.2 The algorithms of Kruskal and Prim

Kruskal merges components in global weight order

```
KRUSKAL(V,E):
    A=empty set
    for v in V: MAKE-SET(v)
    sort E nondecreasingly by (weight,edge-ID)
    for e=(u,v) in sorted E:
        ru=FIND(u); rv=FIND(v)
        if ru!=rv:
            A.add(e); UNION(ru,rv)
    return A
```

The DSU represents current components of (V,A), not final components of the original graph. Equal roots mean an existing A path, so adding e would cycle; this also rejects self-loops. Distinct roots join two trees with one edge and preserve acyclicity. For connected input, stop after accepting n−1 edges. For an MSF, scanning all input edges is also valid.

Kruskal's correctness and earlier cheap edges

Assume A extends to an MST before the next accepted edge e=(u,v). Let S be u's current DSU component, whose cut respects A. Any crossing edge f preceding e has already been processed. If accepted then, it joined its endpoints permanently, so it cannot cross the current S. If rejected then, its endpoints were already in one component and remain together, also impossible for a crossing edge. Thus no earlier edge crosses this cut.

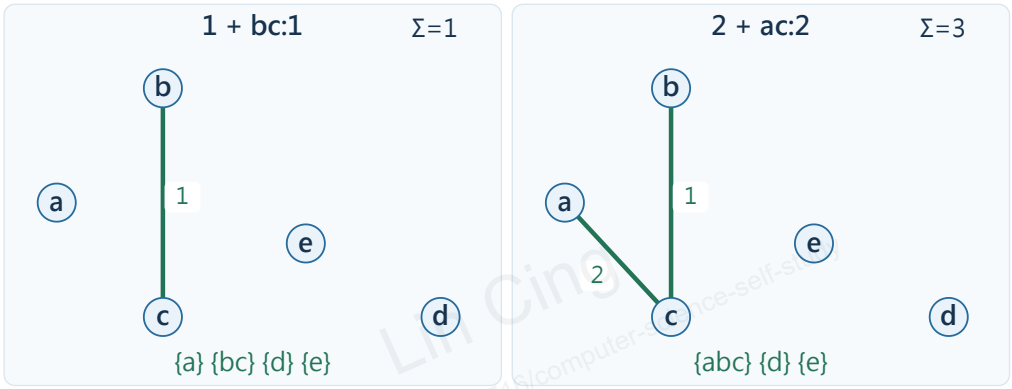
Unscanned edges have weight at least w(e), making e light for the cut. The safe-edge theorem preserves optimal extendibility. At termination, if an original component still consisted of multiple A components, an original connecting path would contain an edge crossing A components. That edge could not have been rejected when scanned, a contradiction. Thus every original component receives an optimal spanning tree, and the output is an MSF.

The proof requires neither distinct weights nor minimum-vertex DSU representatives. Root identities are implementation details.

Every Kruskal state

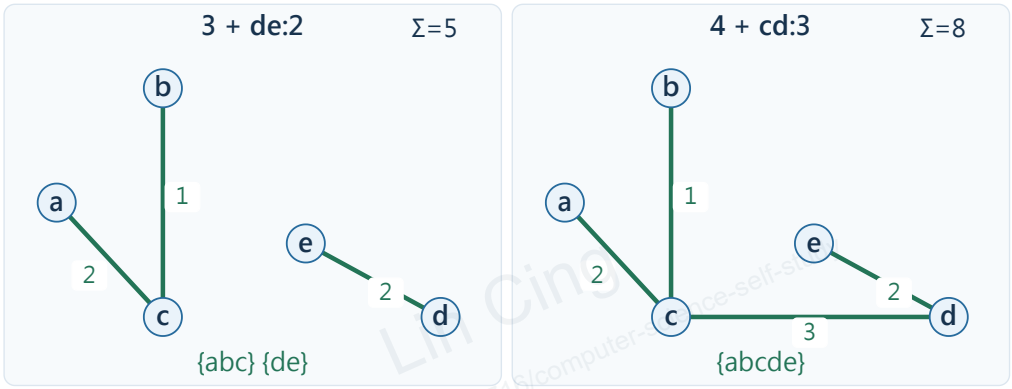
Scanned edge	Endpoint components before scan	Decision	Components afterward	Cumulative cost
bc:1	{b},{c}	accept	{bc},{a},{d},{e}	1
ac:2	{a},{bc}	accept	{abc},{d},{e}	3
de:2	{d},{e}	accept	{abc},{de}	5
cd:3	{abc},{de}	accept	{abcde}	8
ab:4	same component	reject	unchanged	8
be:4	same component	reject	unchanged	8
bd:5	same component	reject	unchanged	8
ce:6	same component	reject	unchanged	8

Every Kruskal state (continued)



Only selected edges shown; component sets appear below

Grow two trees, then join them with one edge



Round 3 grows a separate de tree; Kruskal grows a forest

Step 4 already accepts n−1=4 edges and permits stopping. The final four rows check the full-scan version. At step 3, de need not touch {abc}: Kruskal grows a forest rather than Prim's single tree.

Kruskal's cost and graph assumptions

Comparison sorting costs O(m log(m+1)). Rank-and-compression DSU handles n MAKE-SET calls,2m FIND calls, and at most n−1 UNION calls in O((n+m)(1+α(n))). Under the Chapter 21 convention α(2)=0, so only n≥3 permits the simplification O((n+m)α(n)); retain the constant term for smaller n. Including input gives O(m log(m+1)+(n+m)(1+α(n))). An empty graph returns an empty forest directly. The sorted edge list uses O(m) space, DSU O(n), and output O(n).

For a connected **simple graph**, $n-1 \leq m < n^2$, so log m=O(log n), yielding the familiar O(m log n). Arbitrarily many parallel edges invalidate $m<n^2$; retain log m unless an additional algorithm removes redundant edges. Negative and equal weights do not affect comparison sorting.

For integer weights 0..W, counting sort can replace comparison sorting in O(m+W), but DSU cost remains. A large W can make this worse. This changes the representation assumption behind the comparison lower bound.

Prim's key is one crossing edge, not path distance

For the grown set S, every v∉S maintains $key[v] = \min_{u \in S, (u,v) \in E} w(u, v)$, or∞ when no such edge exists. Its parent records the endpoint in S and edge ID realizing the value. Each unselected vertex has one priority-queue item. Set the first root's key to 0 only to extract it first; this zero is not an output edge weight.

Extract a minimum key each iteration. Except at the initial root, its parent edge is a light edge across (S,V−S) and is safe. After extracting u, compare each new candidate w(u,v) directly with key[v], not key[u]+w(u,v).

For ab:4,ac:5,bc:2, extracting a gives keys b4,c5. Extracting b lowers c's key to 2, although the tree path from a costs 4+2=6. Dijkstra compares d[b]+2=6 and retains d[c]=5. Prim permits negative weights; Dijkstra's correctness has its own nonnegative-edge assumption.

Prim's complete queue contract

```
PRIM-CONNECTED(V,Adj,r):
    for v in V: key[v]=∞; parent[v]=NIL; inQ[v]=true
    key[r]=0; Q=BUILD-INDEXED-MIN-HEAP(V,key)
    A=empty set
    while Q is not empty:
        u=EXTRACT-MIN(Q); inQ[u]=false
        if key[u]==∞: report disconnected input
        if parent[u]!=NIL: A.add(the edge recorded in parent[u])
        for each edge e=(u,v) in Adj[u]:
            if inQ[v] and w(e)<key[v]:
                parent[v]=(u,e.ID)
                DECREASE-KEY(Q,v,w(e)); key[v]=w(e)
    return A
```

DECREASE-KEY synchronizes heap and key storage. Array inQ gives O(1) membership; scanning the heap for membership changes the cost. For an MSF, extracting∞ starts a new original component: make u a root, set key[u]=0, add no parent edge, and scan normally. Do not add∞ to the forest cost.

Prim's invariant, termination, and complete trace

At each iteration start S=V−Q; A is a tree on S, empty before the first iteration, contained in an MST. Each queued vertex's key and parent specify its lightest edge to S. Initialization handles the root separately. Extraction adds a safe edge by the cut theorem, and scanning u supplies exactly all newly available crossing candidates. Thus the invariant is restored. Each iteration removes one vertex, so n iterations produce an n−1-edge MST.

Extracted item	Remaining key(parent) values after scan	Accepted edges	Cost
a:0	b4(a),c2(a),d∞,e∞	none	0
c:2	b1(c),d3(c),e6(c)	ac	2
b:1	d3(c),e4(b)	ac,cb	3
d:3	e2(d)	ac,cb,cd	6
e:2	empty	ac,cb,cd,de	8

Prim's invariant, termination, and complete trace (continued)

Extracted keys0,2,1,3,2 are not nondecreasing. A new vertex can provide a cheaper crossing edge. Optimality follows from cuts, rather than key monotonicity.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

New candidates determine the next safe edge

Prim: adding c changes the next light edge

Parent in parentheses. Right arrows show replaced values.

Before c: S={a}, next c:2		⇒	After c: S={a,c}, next b:1	
b	4 (a)		b	1 (c) ← old 4(a)
c	2 (a)		c	extracted
d	∞		d	3 (c) ← old∞
e	∞		e	6 (c) ← old∞

A key is one lightest edge to S. Adding c lowers b from 4 to 1; extracted keys need not increase.

For S={a}, ac:2 is lightest. Extracting c lowers b from ab:4 to cb:1 and gives d,e their first finite keys. The next extraction must be b:1; having just extracted2 cannot exclude weight 1. Extracting b replaces ce:6 by be:4 for e, and extracting d later replaces that by de:2.

New candidates determine the next safe edge (continued)

Save every parent change together with the corresponding key. The final parent need not be the first encountered neighbor. Vertex extraction order alone cannot reconstruct these competing candidates.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Keep Prim's implementation costs distinct

Queue and representation	Work	Total cost
Indexed binary heap and adjacency lists	n extracts, at most2m scanned entries and m decreases	O((n+m)log n)
Fibonacci heap and adjacency lists	O(n log n) amortized extraction, O(m) amortized decreases	O(m+n log n)
Linear minimum-key search and adjacency lists	n scans of at most n keys,2m edge entries	$O(n^2+m)$
Adjacency matrix and linear keys	n rounds scanning n keys and n neighbor positions	$\Theta(n^2)$

For a simple graph, $m=O(n^2)$, making the third row $O(n^2)$. Handle n=1 directly with an empty tree. Indexed handles locate a decrease in O(1). A lazy heap that pushes replacement entries and discards stale ones can contain O(m) items; the n-item proof does not apply unchanged.

Keep Prim's implementation costs distinct (continued)

For m=Θ(n), both heap variants give O(n log n). For m=Θ(n^2), binary and Fibonacci bounds are respectively O(n^2 log n) and O(n^2), and linear-key scanning also gives O(n^2). The executable example verifies choices using dense Prim; it does not claim a measured Fibonacci-heap speed.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Certify a candidate tree without rerunning the algorithm

Let T be a spanning tree. Quantify only over **non-self-loop nontree edges** e=(u,v),u≠v, and let M_T(u,v) be the maximum weight on the unique T path from u to v. Ignore self-loops; no maximum is assigned to an empty u-to-u path. Then

$$T \text{ is an MST} \quad \Longleftrightarrow \quad \forall e = (u, v) \in E \setminus T,\ u \neq v :\ w(e) \geq M_T(u, v).$$

Necessity follows by adding any e with w(e)<M and removing a maximum path edge, strictly improving T. For sufficiency, remove any f∈T to define a cut. Every other crossing edge e is outside T and its T path uses f, giving w(e)≥M_T≥w(f). Thus f is light across that cut. No other T edge crosses it, so the cut respects any previously selected prefix of T. Add T's edges in any order and repeatedly apply the safe-edge theorem to prove optimality.

For nontree edges ab,bd,ce,be in the fixed graph, pairs (w,M) are (4,2),(5,3),(6,3),(4,3). All inequalities are strict, proving uniqueness here.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Uniqueness and changing one edge weight

Tree T is unique exactly when every non-self-loop nontree edge has w(e)>M_T. Equality allows replacing a maximum path edge with e at equal cost. Conversely, strictness makes f the unique light edge of every T−f cut. Every MST must then contain every edge of T. This does not require **all input weights to differ**: the example has two weight 2 edges and a unique MST.

Lower only one non-self-loop nontree edge e to w′, keeping everything else fixed, and let M be its original T-path maximum. If w′≥M, T still passes every certificate. If w′<M, replace a maximum path edge f by e. The new optimum is w(T)+min(0,w′−M). For a lower bound, a tree omitting e retains original weight at least w(T). For any new tree R containing e, remove e to define a cut. The original T path between its endpoints contains a crossing edge g. Tree R−e+g exists in the original weighted graph, hence w(T)≤w(R)−w′+w(g)≤w(R)−w′+M. This matches the constructed bound.

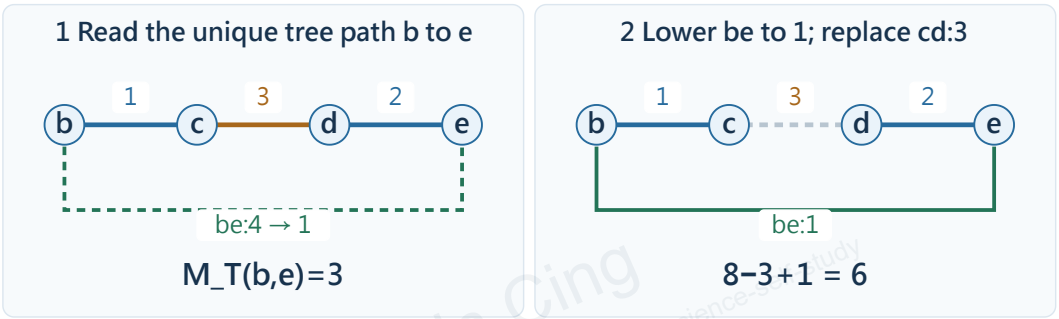
Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Uniqueness and changing one edge weight (continued)

Lowering be from 4 to 1 uses path b-c-d-e with weights 1,3,2, so remove cd:3 and get8−3+1=6. Lowering only to 3 leaves the old tree optimal but creates a tie. A certificate using a separate DFS per path costs O(nm). This update guarantee concerns one changed edge. No self-loop, however negative, can enter a tree; changing its weight leaves the MST unchanged and never triggers a tree-path exchange.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Uniqueness and changing one edge weight (continued)



Edge ac:2 is unchanged; this view enlarges only the affected b-c-d-e path.

Replace a path maximum only when the new weight is smaller: $w' < M_T$

References

Bibliography

Chapter source

Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 3rd ed., MIT Press, 2009, Chapter 23.

References and attribution

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Single-source shortest paths

Part VI Graph Algorithms

Scope

Relaxation maintains upper bounds and predecessors reconstruct paths. We prove the distinct contracts of Bellman–Ford, DAG shortest paths, and Dijkstra, then use negative-cycle certificates, difference constraints, and rooted-tree verification to distinguish finite distance, unreachability, and unbounded cost.

Prerequisites

Edge relaxation, negative weights, and reachability.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Chapter route

Learning objectives

- Maintain shortest-path and triangle-inequality invariants.
- Analyze Bellman–Ford, DAG shortest paths, and Dijkstra separately.
- Convert difference constraints to weighted directed graphs.
- 24.1 The Bellman-Ford algorithm
- 24.2 Single-source shortest paths in directed acyclic graphs
- 24.3 Dijkstra’ s algorithm
- 24.4 Difference constraints and shortest paths
- 24.5 Proofs of shortest-paths properties

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

24.1 The Bellman-Ford algorithm

Three distance meanings; negative edges differ from cycles

Let $G=(V, E)$ be finite and directed, with $n \geq 1$, $s \in V$ and finite real edge weights. Path cost sums edge weights; a walk may repeat vertices and contain cycles. Define $\delta(s, v)$ as the infimum of costs of all s-to-v walks. It is ∞ if unreachable, $-\infty$ if a negative cycle can be visited on the way to v, and otherwise a finite minimum. The source's zero-length walk costs 0, so $\delta(s, s) \leq 0$.

If a walk to v has no usable negative cycle, deleting positive cycles improves its cost and deleting zero cycles preserves it. A simple shortest path of at most $n-1$ edges therefore exists. A negative edge alone does not make distances undefined. A negative cycle affects exactly vertices **reachable onward from that cycle when the cycle is reachable from s**; other branches can retain finite distances.

A single-destination problem becomes single-source after reversing all edges. A single-pair answer is one entry of the single-source result. Representing a negative undirected edge by two opposite arcs creates a negative closed walk; intuition about an isolated directed negative edge no longer applies.

Relaxation only improves tentative upper bounds

```
INITIALIZE(s):
    for v in V: d[v]=∞; parent[v]=NIL
    d[s]=0
RELAX(e=(u, v)):
    if d[u]!=∞ and d[v]>d[u]+w(e):
        d[v]=d[u]+w(e)
        parent[v]=(u, e.ID)
        return true
    return false
```

Induction on relaxations shows every finite $d[v]$ is the cost of an actual s-to-v walk, hence $d[v] \geq \delta(s, v)$. Only strict improvements change parents. Parallel edges require IDs. Initialization is $\Theta(n)$, and each relaxation $O(1)$ under exact unit-cost arithmetic. Fixed-width implementations must avoid overflow and represent ∞ separately. Clamping negative values to an arbitrary floor can destroy negative-cycle detection.

Once d reaches a finite δ, it cannot increase or fall below δ and stays fixed. Finiteness matters: $-\infty$ is not the ordinary finite output of a single relaxation.

Bellman–Ford operations, early stopping, and cost

```
BELLMAN-FORD(G, s):
    INITIALIZE(s)
    repeat at most n−1 rounds:
        changed=false
        for each e in E in a fixed order:
            if RELAX(e): changed=true
        if not changed: return d, parent,"no reachable negative cycle"
    for each (u, v) in E:
        if d[u]!=∞ and d[v]>d[u]+w(u, v):
            return "reachable negative cycle exists"
    return d, parent,"no reachable negative cycle"
```

The last scan tests for a cycle; a still-decreasing finite estimate is not delivered as a shortest distance. After an unchanged round, every reachable edge satisfies $d[v] \leq d[u]+w$. Repeating the same scan cannot change anything, permitting early return. For $n=1$ the final test must still inspect self-loops and detect a negative one.

Bellman–Ford operations, early stopping, and cost (continued)

The full-round version takes $\Theta(n+nm)$, often written $\Theta(nm)$ when $m \geq 1$, $n \geq 2$. Early stopping has best case $\Theta(n+m)$ and the same worst case. Arrays d, parent use $O(n)$ extra space; input uses $O(n+m)$. Complete negative-cycle classification and certificates can use the DP below.

Reverse edge order forces several propagation rounds

Vertices are (s, a, b, c, t, z). Scan the **fixed ordered edges** ct:3, at:8, ac:1, bt:12, bc:5, ba:−2, sa:4, sb:1. Each edge points from its first letter to its second. Vertex z is isolated.

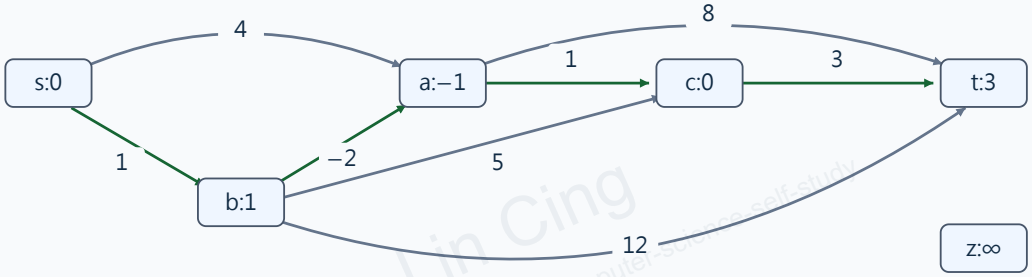
Round end	s	a	b	c	t	z
0	0	∞	∞	∞	∞	∞
1	0	4	1	∞	∞	∞
2	0	−1	1	5	12	∞
3	0	−1	1	0	7	∞
4	0	−1	1	0	3	∞
5	0	−1	1	0	3	∞

In round 2, old a=4 first gives t=12, c=5; only later does ba lower a to−1. In round 3, ct first gives 8, at lowers it to 7, then ac sets c=0. Only round 4 obtains t=3. Round 5 changes nothing. Final predecessors form s→b→a→c→t, cost 1−2+1+3=3.

Reverse edge order forces several propagation rounds (continued)

Bellman–Ford: one input and final predecessors

Nodes: final distances. Green: parent arcs. Gray: other input arcs.



Reverse-order round ends: a 4→−1; c ∞→5→0; t ∞→12→7→3.

Tree path s→b→a→c→t: 1−2+1+3=3. Vertex z is unreachable from s.

Why n−1 rounds suffice and detect negative cycles

Without a reachable negative cycle, every reachable v has a simple shortest path $s=v_0,\ldots, v_k=v$ with $k\le n-1$. Round i scans (v_{i-1}, v_i), after round i−1 has already made $d[v_{i-1}]=\delta$. Convergence makes $d[v_i]=\delta$. Other same-round relaxations cannot spoil a true value. Thus n−1 rounds suffice. This proves the in-place algorithm covers at least i-edge paths; its state need not equal the DP minimum over paths with at most i edges.

If a negative cycle is reachable, all its vertices have finite estimates after n−1 rounds: reachability itself has a simple path of at most n−1 edges, without needing a shortest path. If no cycle edge could improve, summing $d[v]\le d[u]+w$ around it would cancel all distances and give $0\le w(\text{cycle})<0$. Thus some reachable edge must improve. Conversely, without such a cycle all distances are correct and triangle inequalities forbid improvement. These are both directions of the detection proof.

Reconstructing a certificate with an edge-count DP

Let $D_i[v]$ be the minimum cost of a walk with at most i edges, with $D_0[s]=0$ and all other entries∞.

$$D_i[v] = \min\left(D_{i-1}[v], \min_{(u,v)\in E}(D_{i-1}[u] + w(u,v))\right).$$

Each layer reads only the preceding layer. Record skip when retaining an old value, or the edge ID and preceding-layer vertex u when taking an edge. Every walk with at most i edges either has at most i−1, or loses its last edge to become one with at most i−1. Conversely every recurrence candidate is a legal walk, proving equality.

Compute through layer n. If $D_n[v]<D_{n-1}[v]$, backtracking n layers gives an **exactly n-edge** walk: fewer edges would already qualify at layer n−1. Decompose the walk into a simple path and cycles cut at repeated vertices. If every cycle were nonnegative, deleting them would produce an at-most-(n−1)-edge path of no greater cost, contradicting strict improvement. At least one extracted cycle is negative. Its ordered edge IDs and total weight form a checkable certificate.

Reconstructing a certificate with an edge-count DP (continued)

The full-history version takes $O(n+nm)$ time and $O(n^2+m)$ space. Two distance layers suffice for classification but do not retain this backtracking table. Do not claim the in-place algorithm's $O(n)$ auxiliary space for this version.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Exactly which vertices receive $-\infty$

Let seeds $B=\{v:D_n[v]<D_{n-1}[v]\}$. Search from B in the **original edge direction**, marking every reached vertex $-\infty$. Other reachable vertices keep D_{n-1}; unreachable ones remain ∞.

A seed's n-edge walk contains a negative cycle, so the seed and its descendants are affected. Conversely, every reachable negative cycle has finite entries in D_{n-1}. If it had no seed, each of its edges would satisfy $D_{n-1}[v]=D_n[v]\le D_{n-1}[u]+w$; summing gives a contradiction. Every such cycle therefore has a seed, whose forward search reaches all affected descendants. The classification covers exactly the affected set, rather than only cycle vertices.

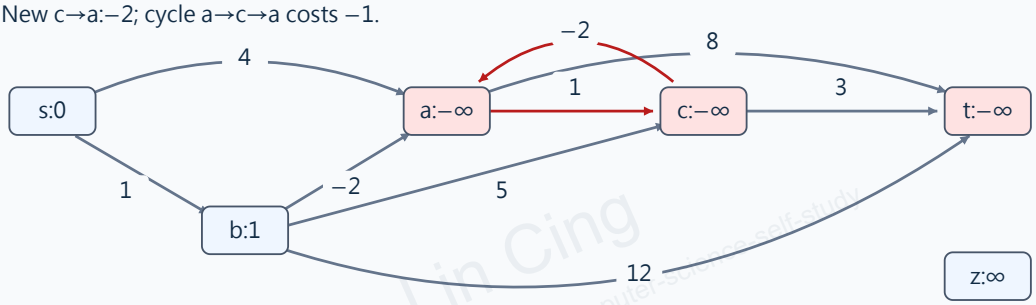
Add $c\to a:-2$ to the earlier graph. Cycle $a\to c\to a$ costs $1-2=-1$. Distances on (s, a, b, c, t, z) become $(0,-\infty,1,-\infty,-\infty,\infty)$. The cycle cannot return to s or b, which stay finite; z stays unreachable. An additional unreachable negative cycle must not cause a false single-source report.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Exactly which vertices receive −∞ (continued)

A reachable negative cycle affects only forward descendants

New c→a:−2; cycle a→c→a costs −1.



Descendants a, c, t receive−∞; b=1, s=0. No cycle exit returns to s or b.

A negative cycle does not erase finite branches or unreachable z.

24.2 Single-source shortest paths in directed acyclic graphs

Topological order delivers every candidate in time

```
DAG-SHORTEST(G, s):
    order=TOPOLOGICAL-SORT(G)
    if a cycle is found: reject this algorithm's input
    INITIALIZE(s)
    for u in order:
        for each edge e in Adj[u]: RELAX(e)
    return d, parent
```

Every s-to-v path is relaxed in its path order, so path relaxation gives δ at its end. Equivalently, induct on topological position: all incoming sources have been finalized and scanned before v is processed, leaving no future better candidate. Unreachable vertices retain∞. Negative edges are allowed because no cycle means no negative cycle. Sorting and scanning together take Θ(n+m), with O(n) auxiliary space.

24.2 Single-source shortest paths in directed acyclic graphs (continued)

The earlier graph is already a DAG. In order s, b, a, c, t, z, scanning s gives a4, b1; b gives a−1, c6, t13; a gives c0, t7; c gives t3. One pass matches four Bellman–Ford rounds. The difference is **topological vertex order**, not the cost definition.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Critical paths: sign reversal works directly in DAGs

Negate every DAG weight and negate the resulting shortest cost to obtain its longest cost. Alternatively initialize L[s]=0, others−∞, and use maximum relaxation. A cyclic graph cannot use the DAG algorithm unchanged; positive cycles make longest walks unbounded.

For vertex jobs of duration[v]≥0, an edge u→v requires u to finish before v starts. In topological order compute F[v]=duration[v]+max({F[u]:u→v}∪{0}). Backtracking a maximizing predecessor gives a critical path.

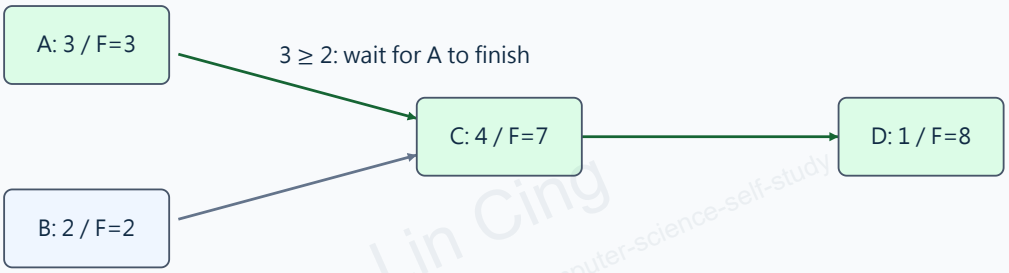
For A:3, B:2, C:4, D:1 with A→C, B→C, C→D, finishes are F[A]=3, F[B]=2, F[C]=4+max(3,2)=7, F[D]=1+7=8. Critical path A, C, D costs 8. With unlimited processors and only these dependencies, starting each eligible job immediately attains this lower bound. Finite processors add resource constraints, so the critical-path bound alone is not a full schedule.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Critical paths: sign reversal works directly in DAGs (continued)

Job DAG: the slower predecessor determines completion

Boxes: job / duration / earliest finish F. Green arrows: critical path.



F[C]=4+max(3,2)=7；F[D]=1+7=8。

Unlimited processors finish at 8. Total work 10 requires at least 10 on one processor.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

24.3 Dijkstra’ s algorithm

Extract the minimum tentative distance and never reopen it

Assume **every edge weight is nonnegative**. An indexed min-heap holds one item per unextracted vertex with key=d. Initialize d[s]=0, others∞, parent=NIL, inQ=true.

```
while Q is not empty:
    u=EXTRACT-MIN(Q); inQ[u]=false
    if d[u]==∞: stop; all remaining vertices are unreachable
    for e=(u, v) in Adj[u]:
        if inQ[v] and d[u]+w(e)<d[v]:
            d[v]=d[u]+w(e); parent[v]=(u, e.ID)
            DECREASE-KEY(Q, v, d[v])
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Extracted vertices S have true distances. Unlike Prim, the candidate is **total path cost d[u]+w**. An extracted vertex is neither updated nor reinserted. A single-pair query may stop when the target is actually extracted, not when first discovered.

The precise use of nonnegative weights

Suppose the next finite extraction u is the first wrong one. On a true shortest path s→…→x→y→…→u, let y be the first unextracted vertex and x its extracted predecessor. The source is extracted first with d[s]=0. Otherwise x already has its true distance and has scanned xy, so convergence gives d[y]=δ(s, y).

The suffix y-to-u has nonnegative weight, giving δ(s, y)≤δ(s, u). Minimum extraction gives d[u]≤d[y]; the upper-bound property gives δ(s, u)≤d[u]. Thus

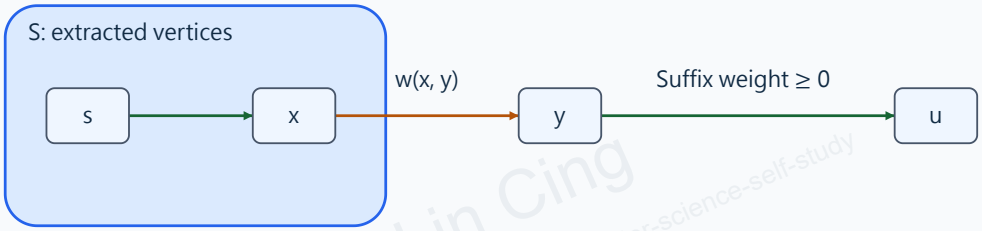
$$\delta(s, u) \le d[u] \le d[y] = \delta(s, y) \le \delta(s, u).$$

All inequalities are equalities, contradicting error. Zero edges and distance ties are allowed; strict increases are unnecessary. If an∞ item is extracted while a reachable vertex remains, the first unextracted vertex on its path would have a finite candidate, contradicting minimum extraction.

The precise use of nonnegative weights (continued)

Dijkstra: bound u using the first unextracted vertex

Follow one true shortest path. Orange: the first arc leaving S.



Vertex x is finalized and xy scanned ⇒ d[y]=δ(s, y).

Minimum extraction and a nonnegative suffix give d[u]≤d[y]=δ(s, y)≤δ(s, u).

Numerical trace and a genuine negative-edge failure

Change only ba in the earlier example to+2. States after scans are

Extraction	Unextracted finite d(parent)	Finalized distances
s0	a4(s), b1(s)	s0
b1	a3(b), c6(b), t13(b)	s0, b1
a3	c4(a), t11(a)	also a3
c4	t7(c)	also c4
t7	none; z∞	also t7

The shortest-path tree is s→b→a→c→t, cost 1+2+1+3=7. Target estimates13→11→7 demonstrate why first discovery is insufficient.

Numerical trace and a genuine negative-edge failure (continued)

For a failure, use s→a:2, s→b:5, b→a:−4, a→t:2. After s, extract a2 and assign t4; extract t4 before b5, which would give a1 and true t3. The standard no-reopening algorithm retains wrong answers. Updating an extracted a without rescanning its outgoing edges also leaves t wrong. This graph is a DAG with no negative cycle. The failed premise is δ(s, b)=5>δ(s, a)=1, contradicting the proof's nonnegative-suffix comparison.

Queue, graph representation, and exact arithmetic costs

An indexed binary heap takes O(n) to build, O(n log n) for n extracts, and O(m log n) for at most one decrease per scanned edge, totaling O((n+m)log n), with n=1 handled directly. Fibonacci decreases take O(1) amortized, giving O(m+n log n). Handles locate heap items, and inQ is an O(1) array lookup.

Linear minimum-distance scanning with adjacency lists costs $O(n^2+m)$, simplifying to $O(n^2)$ for simple graphs. A lazy heap pushes all new candidates and can hold m items; it requires a separate O((n+m)log(m+1)) argument and O(n+m) space. The indexed n-item bound does not transfer automatically. The numerical example uses linear-key scanning and adjacency lists.

These bounds use a real-RAM model. With B-bit integer weights and no negative cycles, simple-path costs need O(B+log n) bits; exact addition and comparison costs must be included separately. Floating-point error or arbitrary epsilons can change equality and cycle decisions. Mathematical correctness alone does not establish numerical stability.

24.4 Difference constraints and shortest paths

Derive the edge direction algebraically

Constraint $x_j - x_i \le b$ corresponds to arc $i \to j:b$ because $x_j \le x_i + b$. Each coefficient row has one+1 and one−1, with unrestricted real variables. Rewrite $x_j - x_i \ge b$ as $x_i - x_j \le -b$. Split an equality into two opposite inequalities.

Add a super-source q with zero-weight arcs to every variable and no incoming arc, making all original negative cycles reachable. Without a negative cycle, set $x_i = \delta(q, i)$. Each edge's triangle inequality gives $x_j - x_i \le b$. Conversely, summing constraints around a negative cycle cancels every variable and gives $0 \le \Sigma b < 0$. Thus feasibility is equivalent to absence of negative cycles.

For n variables and m constraints, the augmented graph has n+1 vertices, m+n edges, giving $O(n^2+nm)$ Bellman–Ford time. Instead initialize all original distances to 0, as though q's arcs have already been processed, and scan the original edges for at most n−1 rounds plus a test. Cost becomes O(n+nm), including initialization when m=0.

Complete feasible and infeasible examples

Constraints $x_2-x_1\le -2$, $x_3-x_2\le -1$, $x_1-x_3\le 4$ give arcs $1\to 2:-2, 2\to 3:-1, 3\to 1:4$. From all zero, the first scan gives $(0,-2,-3)$; $-3+4=1\ge 0$ does not change x_1. The second round is unchanged.

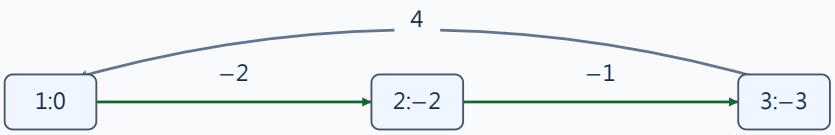
Check each inequality: $-2-0=-2\le -2$; $-3-(-2)=-1\le -1$; $0-(-3)=3\le 4$. Adding 3 gives $(3,1,0)$, also feasible because differences are unchanged. The feasible region is invariant under translation along the all-ones vector and has no fixed origin.

Changing the last constant to 2 makes the sum $0\le -2-1+2=-1$, proving infeasibility. The negative cycle is $1\to 2\to 3\to 1$. Failure of one proposed vector alone would not prove absence of every feasible vector.

Complete feasible and infeasible examples (continued)

Difference constraints: a feasible vector and a cycle contradiction

Arc $i\to j:b$ means $x_j-x_i\le b$. Node values are a feasible vector x.



Differences: $-2-0=-2$; $-3-(-2)=-1$; $0-(-3)=3\le 4$.

Change the last weight 4 to 2: summing gives $0\le -2-1+2=-1$, a contradiction.

Original cycle weight 1: feasible. New weight -1: infeasible. Translation preserves differences.

Extensions that retain the proof

Equality $x_j - x_i = b$ gives $i \to j : b$ and $j \to i : -b$. Together they force equality. To express $x_i \le c$ or $-x_i \le c$, introduce a reference r and respectively $r \to i : c$ or $i \to r : c$. After solving, translate all coordinates so $x_r = 0$ before treating them as absolute bounds. Translation must preserve this reference.

Integer right sides and integer-zero initialization make Bellman–Ford's additions and minima produce an integer feasible solution. If **all** variables must be integers but right sides are real, replace each b by $\text{floor}(b)$: integer differences make the constraints equivalent. Then test for negative cycles. This rowwise rounding is not automatically valid when only some variables must be integers.

Under the normalization $d \le 0$, shortest distances form the coordinatewise greatest feasible vector. Any feasible x satisfies $x_v \le$ the cost of every q-to-v path by repeated substitution, hence $x_v \le \delta(q, v)$. This conclusion includes $x_i \le 0$. With unrestricted translation in the original system, Σx_i can increase arbitrarily.

24.5 Proofs of shortest-paths properties

Optimal substructure, triangle inequality, and convergence

Assume no negative cycle reachable from s. A subpath of a finite shortest path must be shortest: replacing a more expensive subpath gives a cheaper s-to-v walk, a contradiction. Appending $u \to v$ to a shortest s-to-u path gives $\delta(s, v) \le \delta(s, u) + w$. An unreachable u makes the right side ∞ and imposes no additional restriction.

Initialization and the actual-walk interpretation prove the upper-bound property. An unreachable vertex cannot become finite without an actual walk. Immediately after RELAX(u, v), $d[v] \le d[u] + w$, but **only at that instant**: a later decrease in $d[u]$ can require rescanning this edge.

If u already has its true distance and $u \to v$ lies on a shortest path, relaxation gives $d[v] \le \delta(s, u) + w = \delta(s, v)$. The upper bound gives the reverse inequality, so v becomes permanently correct. This is convergence. Starting with s and applying convergence in shortest-path edge order fixes one vertex after another. Interspersed relaxations only lower upper bounds and cannot spoil correct values, establishing path relaxation.

Parent acyclicity follows from update times

Suppose a strict relaxation u→v first creates a parent cycle. Before it, the parent subgraph has a path v→…→u. For every old parent arc x→y, the last assignment of parent[y] made d[y]=d[x]+w. Since y has not been reassigned and x can only decrease, immediately before this update d[y]≥d[x]+w. Summing the old path gives d[u]≥d[v]+w(v→…→u).

The new strict update requires d[v]>d[u]+w(u, v). Combining gives a negative total cycle weight. All its vertices are finite and reachable from s, contradicting the assumption. A negative self-loop follows directly from the same strict improvement. Thus the parent subgraph remains acyclic.

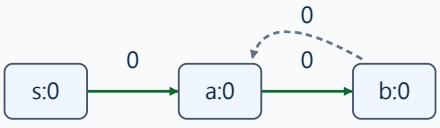
Every finite v≠s received a non-NIL parent by relaxation. The source retains d[s]=δ(s, s)=0 and parent NIL. Parent vertices of finite vertices are also finite. In a finite acyclic graph, backtracking must terminate at the only parentless finite vertex s. Each vertex has one parent, giving a rooted tree on all discovered finite vertices.

Parent acyclicity follows from update times (continued)

Zero edges: equal distances still need rooted parents

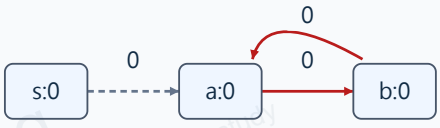
Original arcs s→a:0, a→b:0, b→a:0. All distances are 0 on both sides.

Valid parents: rooted paths from s



parent[a]=s · parent[b]=a

Invalid parents: a, b form a cycle



parent[a]=b · parent[b]=a

Green: valid tree. Red: invalid cycle. Gray dashed: unselected original arcs.

Tightness checks cost. Search parent children from s to verify connectivity and acyclicity.

Correct distances and valid parents give a shortest-path tree

Once finite d equals δ, the last-update argument for every current parent arc u→v gives d[v]≥d[u]+w. The triangle inequality gives the reverse after substituting d=δ, hence equality. Summing on the s-to-v tree path telescopes to path cost d[v]−d[s]=δ(s, v). Unreachable vertices retain∞ and NIL parents, completing all shortest-path-tree requirements.

Do not justify this by strictly decreasing predecessor distances. Zero edges permit equality and negative edges can decrease forward distances. Acyclicity depends on the strict-update-time argument. Choosing arbitrary tight predecessors afterward can form a zero cycle. In s→a:0, a→b:0, b→a:0 all distances are 0; parent[a]=b, parent[b]=a makes both edges tight without any root path.

A linear-time certificate verifies optimality too

Given d and parent edge IDs, compute the actual reachable set R from s. Require finite d in R, d[s]=0, and∞ with NIL parents outside R. For every v∈R−{s}, verify that its parent is an actual input arc u→v with d[v]=d[u]+w. Search the parent-children graph from s, visiting exactly R without repeats, and require parent[s]=NIL. Finally scan all input arcs from reachable u and check d[v]≤d[u]+w.

The tight rooted tree realizes a path of cost d[v], proving δ(s, v)≤d[v]. Summing edge inequalities along any s-to-v walk gives d[v]≤its cost, hence d[v]≤δ(s, v). Both directions establish equality. A reachable negative cycle would contradict the summed inequalities and is rejected. Verification takes Θ(n+m) time, O(n) space without rerunning Bellman–Ford or Dijkstra.

Edge inequalities alone allow unrealistically small values with no realizing paths. Tight parents alone allow zero cycles or overlook cheaper input arcs. Each certificate component proves a separate direction.

References

Bibliography

Chapter source

Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 3rd ed., MIT Press, 2009, Chapter 24.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

References and attribution

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

All-pairs shortest paths

Part VI Graph Algorithms

Scope

All-pairs shortest paths uses two distinct DP states, edge budgets and intermediate vertices. Min-plus products, Floyd–Warshall, and Johnson connect path reconstruction, zero cycles, affected pairs, and the signs and costs of reweighting.

Prerequisites

Min-plus matrices and predecessor representations.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Chapter route

Learning objectives

- Connect min-plus multiplication to shortest paths.
- Derive Floyd–Warshall from intermediate-vertex states.
- Use Johnson reweighting on sparse graphs with negative edges.
- 25.1 Shortest paths and matrix multiplication
- 25.2 The Floyd-Warshall algorithm
- 25.3 Johnson’ s algorithm for sparse graphs

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

25.1 Shortest paths and matrix multiplication

APSP takes a directed weighted graph on n vertices, allowing negative edges, and returns a distance for every pair(i, j). Unreachable pairs have$+\infty$. If i can reach a negative cycle that can reach j, repeating it makes walk cost unbounded below. Define that distance as$-\infty$; no finite shortest path exists for the pair.

First assume no negative cycle. Every repeated-vertex walk contains a nonnegative cycle, which can be removed without increasing cost. Repeating this leaves a simple path. For $i \neq j$, a shortest path therefore uses at most $n-1$ edges. For i=j, the zero-length path costs 0.

Matrix W[i, j] stores the lightest direct edge, or∞ if absent. Set W[i, i]=0 to allow no edge. For negative-self-loop detection, initialize to min(0, w(i, i)) rather than overwriting a negative loop with 0. All diagonals in the following cycle-free example are 0.

State as an upper bound on edge count

Let $L^{(m)}[i, j]$ be the minimum weight of a walk with **at most m edges**, not exactly m. Set $L^{(0)}=I$, with zero diagonal and∞ elsewhere. Splitting at the last edge gives

$L^{(m+1)}[i, j]=\min_k(L^{(m)}[i, k]+W[k, j])$.

Every nonempty candidate splits into an at-most-m-edge prefix and its last edge; every finite recurrence term conversely concatenates to a valid walk. Shorter old paths remain through k=j and W[j, j]=0. Both directions describe the same feasible walks. With negative cycles, a fixed finite edge budget still defines a meaningful finite optimum, but budget $n-1$ no longer gives unrestricted distances.

Define$(A \otimes B)[i, j]=\min_k(A[i, k]+B[k, j])$. Matrices for budgets a, b concatenate to budget a+b. Conversely, split any at-most-(a+b)-edge walk after min(a, its length) edges; the suffix uses at most b. Hence $L^{(a)} \otimes L^{(b)}=L^{(a+b)}$.

Min-plus algebra and two algorithms

Associativity expands both sides to the same finite minimum:$((A\otimes B)\otimes C)[i,j]=\min_{p,\varphi}(A[i,p]+B[p,q]+C[q,j])=(A\otimes(B\otimes C))[i,j]$. Matrix I is the identity;∞ is neutral for min and$\infty+$a finite value$=\infty$. Do not introduce$-\infty$ at this stage, since$\infty+(-\infty)$ is undefined.

```
Extend(A, B):
  C=new n×n matrix filled with∞
  for i, j, k: C[i, j]=min(C[i, j], A[i, k]+B[k, j])
  return C
Slow(W): L=I; repeat n−1 times: L=Extend(L, W); return L
Square(W): L=W; m=1
  while m<n−1: L=Extend(L, L); m=2m
  return L
```

Slow performs $\Theta(n)$ products of $\Theta(n^3)$, totaling $\Theta(n^4)$ for $n\geq 2$. Square performs $\operatorname{ceil}(\log_2(n-1))$ squarings, taking $O(n^3 \log n)$. Two input matrices and a fresh output need $O(n^2)$ working space. Overwriting inputs during a product invalidates its fixed old-budget interpretation. For n=0 return an empty matrix; handle n=1 directly through its diagonal.

Min-plus algebra and two algorithms (continued)

Ordinary multiplication replaces min by addition and addition by multiplication, but Strassen's subtraction formulas do not directly apply to min-plus. The analogy does not transfer fast multiplication bounds for rings.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

One four-vertex, six-edge example

All matrices, Floyd stages, and Johnson states below use1→2:4,1→3:7,2→3:−2,2→4:6,3→4:3,4→2:1. Vertex 1 has no incoming edge. Cycle 2→3→4→2 costs 2, so there is no negative cycle.

Rows of W are[0,4,7,∞],[∞,0,−2,6],[∞,∞,0,3],[∞,1,∞,0].

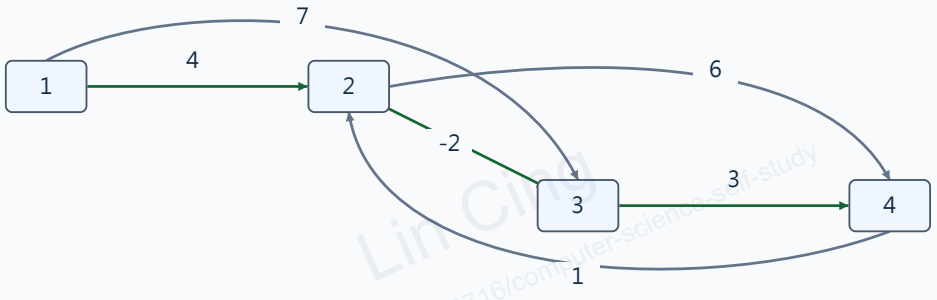
Rows of $L^{(2)}$ are[0,4,2,10],[∞,0,−2,1],[∞,4,0,3],[∞,1,−1,0].

For example $L^{(2)}[1,4]=\min(0+\infty,4+6,7+3,\infty+0)=10$, with two tied two-edge paths. Also $L^{(2)}[2,4]=-2+3=1$. Only the third extension permits 1→2→3→4 and gives $L^{(3)}[1,4]=5$; all other entries stay unchanged. Since n=4, $L^{(3)}$ is the answer, as is the squared $L^{(4)}$.

One four-vertex, six-edge example (continued)

Six directed edges and edge-budgeted paths

Green: best 3-edge route. Gray: other input edges. Labels: original weights.



1→2→4=10 (2 edges); 1→3→4=10 (2 edges); 1→2→3→4=5 (3 edges).

25.2 The Floyd-Warshall algorithm

Here k bounds the **identifiers of allowed intermediate vertices**, not edge count. State $D^{(k)}[i, j]$ is the minimum walk cost when all intermediate vertices belong to$\{1,\ldots, k\}$; endpoints i, j are unrestricted. Without negative cycles, choose a simple path, so k occurs at most once.

An optimum avoiding k belongs to old state $D^{(k-1)}$. An optimum using k splits there into segments with no internal k, giving $D^{(k-1)}[i, k]+D^{(k-1)}[k, j]$. For optimal substructure, replacing a nonoptimal segment with a cheaper legal segment improves the concatenation; deleting any nonnegative cycles cannot worsen it, contradicting optimality. Conversely, concatenating two old-state paths introduces only intermediate k and cannot give an illegally low candidate.

Thus $D^{(0)}=W$ and $D^{(k)}[i, j]=\min(D^{(k-1)}[i, j], D^{(k-1)}[i, k]+D^{(k-1)}[k, j])$. At k=n all intermediates are allowed. New matrices per stage take $\Theta(n^3)$ time. Saving all stages uses $\Theta(n^3)$ space; retaining only old and new uses $\Theta(n^2)$.

In-place updates work, arbitrary loop orders do not

Without negative cycles, D[k, k]=0 at every stage. Updating row k compares D[k, j] with D[k, k]+D[k, j], leaving it unchanged; column k is likewise unchanged. All stage-k inputs D[i, k], D[k, j] therefore remain old-stage values. Updating each D[i, j] once exactly implements the recurrence in one matrix.

```
D=copy(W)
for k=1..n:
  for i=1..n:
    for j=1..n:
      if D[i, k]!=∞ and D[k, j]!=∞:
        D[i, j]=min(D[i, j], D[i, k]+D[k, j])
```

Time is $\Theta(n^3)$, space $\Theta(n^2)$. Fixed-width implementations must prevent an INF sentinel plus a negative number from becoming falsely reachable and must avoid overflow. Check reachability before addition; the mathematical theorem assumes exact arithmetic.

In-place updates work, arbitrary loop orders do not (continued)

For a counterexample, use unit-weight chain1→2→4→3. A single i, j, k pass considers i=1, j=3 before D[1,4] or D[2,3] is finite, so D[1,3] remains∞. Later j=4 computes D[1,4]=2 but never revisits j=3. Correct k, i, j order gives D[1,3]=3.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Complete matrices at every k

Rows and columns use1,2,3,4. $D^{(0)}=W$; vertex 1 has no incoming edge, so $D^{(1)}=W$ too.

Stage	Row1	Row2	Row3	Row4
$D^{(2)}$	0,4,2,10	∞,0,−2,6	∞,∞,0,3	∞,1,−1,0
$D^{(3)}$	0,4,2,5	∞,0,−2,1	∞,∞,0,3	∞,1,−1,0
$D^{(4)}$	0,4,2,5	∞,0,−2,1	∞,4,0,3	∞,1,−1,0

Stage 2 changes(1,3) to 4−2=2,(1,4) to 4+6=10, and(4,3) to 1−2=−1. Stage 3 changes(1,4) to 2+3=5 and(2,4) to−2+3=1. Stage 4 changes(3,2) to 3+1=4. Every other old value is at most its candidate and remains.

Unlike $D^{(2)}$, $L^{(2)}$ already permits 2→3→4 and 3→4→2. Intermediate vertices 3,4 are not allowed in $D^{(2)}$, whose corresponding entries remain6 and∞. A second step in each algorithm means a different state.

Complete matrices at every k (continued)

Six Floyd updates: two old inputs give one new value

Strict updates only. Inputs use the previous stage; unlisted entries stay unchanged.

k	(i, j)	Old	D[i, k]	D[k, j]	New
2	(1,3)	7	4	-2	2
2	(1,4)	∞	4	6	10
2	(4,3)	∞	1	-2	-1
3	(1,4)	10	2	3	5
3	(2,4)	6	-2	3	1
4	(3,2)	∞	3	1	4

Green: new minimum. No updates at k=1; three at k=2, two at k=3, one at k=4.

A predecessor is not an intermediate vertex

Entry Π[i, j] is the vertex **immediately preceding j** on a selected shortest i-to-j path, not arbitrary intermediate k. Initialize Π[i, j]=i for direct edges; use NIL for diagonals and unreachable entries. With two stage buffers, a strict improvement through k assigns Πnew[i, j]=Πold[k, j], because the final edge belongs to the k-to-j suffix. Otherwise preserve the old predecessor. When k=j, no strict improvement occurs, so a zero-length suffix cannot overwrite a valid predecessor with NIL.

Preserve the old route on equal costs; changing $<$ to $\leq$ is unsafe. With only edge 1→2 of weight 1, stage k=2 has equal candidate1+0=1. Assigning Π[1,2]←Π[2,2]=NIL then destroys the existing route. Zero cycles likewise prevent arbitrary tight-edge parent choices without an acyclicity argument.

In the example, Π[1,4] first improves at k=2 to Π[2,4]=2, giving 1→2→4. At k=3 it becomes Π[3,4]=3. With Π[1,3]=2 and Π[1,2]=1, the final reverse chain is 4←3←2←1, cost 4−2+3=5. The next tie rule provides a directly decreasing integer measure for a complete predecessor tree.

Provably acyclic predecessors even with zero cycles

Store pair(cost, hops), minimizing cost first and then hops. Concatenation adds both coordinates. Initialize diagonals(0,0), direct edges(w,1), and absent paths(∞,∞), then apply the same Floyd recurrence. Without negative cycles, positive cycles worsen cost and zero cycles worsen hops. A lexicographic optimum has a simple representative, so the decomposition and in-place proofs still hold. The algorithm uses $\Theta(n^3)$ operations on fixed-size pairs.

For each fixed source s and finite $v \neq s$, choose an original incoming edge $u \to v$ satisfying

$\text{cost}[s, v] = \text{cost}[s, u] + w(u, v)$, $\text{hops}[s, v] = \text{hops}[s, u] + 1$.

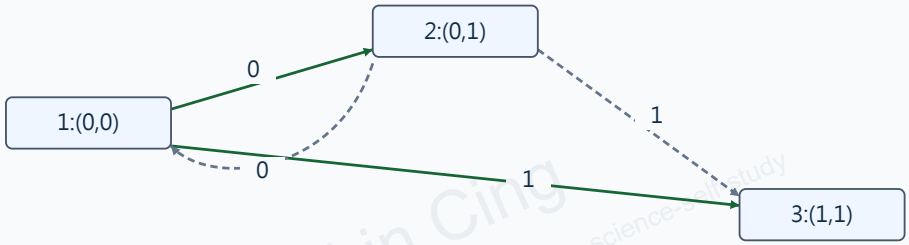
Such a u exists on the final edge of a lexicographically optimal path. A nonoptimal prefix could be replaced to improve the whole path, a contradiction. Set $\Pi[s, v] = u$. Every parent step decreases nonnegative integer hops by one, excluding cycles and infinite descent. All finite nonsource vertices have parents, so chains terminate at s and reconstruct actual shortest paths.

Scanning all m edges for every source costs $O(nm)$, no more than $\Theta(n^3)$ on a simple graph with $m \leq n^2$. Saved cost, hops,Π need $\Theta(n^2)$ space. For zero edges $1 \leftrightarrow 2$ and weight 1 edges $1 \to 3, 2 \to 3$, source 1 has hops=(0,1,1). It cannot acquire parent2 and no$1 \leftrightarrow 2$ parent cycle arises.

Provably acyclic predecessors even with zero cycles (continued)

Safe backtracking through zero cycles: hops decrease

Boxes: vertex:(cost, hops). Green: selected edges; backtrack against their arrows.



$\Pi[1,2] = 1$, $\Pi[1,3] = 1$. Source 1 has no parent.

Route $2 \to 3$ also gives cost 1 but needs 2 hops, losing to direct$1 \to 3$ with 1 hop.

Gray dashed: unselected. Every parent step lowers hops by 1, excluding zero-cycle parents.

Negative-cycle detection and affected pairs

If final Floyd has D[k, k]<0, its finite value concatenates actual edges into a negative closed walk. Decomposing this into simple cycles finds at least one negative cycle. Conversely, take a negative simple cycle and its largest-numbered vertex k. All other vertices are allowed intermediates of its k-to-k walk, so Floyd detects a negative closed walk and some negative diagonal. This proves both detection directions.

With negative cycles, fixed stages produce some finite walk values, which are not shortest distances for affected pairs. The earlier in-place equivalence proof relying on D[k, k]=0 no longer applies. Standard in-place k, i, j still detects cycles: finite values have actual-walk witnesses and are no greater than the corresponding buffered candidates, so negative diagonals cannot be missed. Different update details can produce different temporary negative values.

Negative-cycle detection and affected pairs (continued)

Compute reachability R separately. Mark(i, j) as−∞ exactly when some k satisfies D[k, k]<0, R[i, k], R[k, j]. Sufficiency follows by entering a negative cycle, repeating it arbitrarily, and leaving toward j. For necessity, if the i-to-j relevant region has no negative cycle, deleting all nonnegative cycles leaves one of finitely many simple paths with a finite lower bound. Its distance cannot be−∞. Unreachable pairs remain∞; other Floyd entries remain valid.

Mark affected pairs before returning results, and do not feed−∞ back into ordinary∞ arithmetic. Johnson may reject the whole graph on any negative cycle; partial finite results require these pairwise semantics.

A negative cycle need not affect the whole graph

Use1→2:2,2→3:−3,3→2:1,3→4:4, with isolated5. Cycle 2→3→2 costs−2.

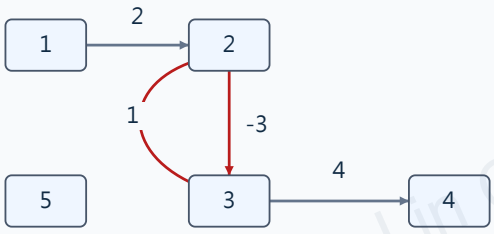
Source / destination	1	2	3	4	5
1	0	−∞	−∞	−∞	∞
2	∞	−∞	−∞	−∞	∞
3	∞	−∞	−∞	−∞	∞
4	∞	∞	∞	0	∞
5	∞	∞	∞	∞	0

Vertex 1 can enter the cycle and leave for 4, giving $\delta(1,4)=-\infty$. There is no return to 1, so $\delta(1,1)=0$. Vertices 4,5 cannot enter the cycle and retain zero self-distances. A negative diagonal does not justify marking every matrix cell as erroneous.

A negative cycle need not affect the whole graph (continued)

A negative cycle affects 9 pairs, not the whole matrix

Red cycle 2→3→2 costs−2. Red cells require both entry to and exit from the cycle.



	1	2	3	4	5
1	0	−∞	−∞	−∞	∞
2	∞	−∞	−∞	−∞	∞
3	∞	−∞	−∞	−∞	∞
4	∞	∞	∞	0	∞
5	∞	∞	∞	∞	0

Vertex 1 can enter but not return, so $\delta(1,1)=0$. Vertices 4,5 cannot enter.

Transitive closure uses the same split with Boolean algebra

Initialize $R^{(0)}[i, j]=(i=j)$ or an edge $i \to j$, including zero-length paths and true diagonals. Replace min/+ by OR/AND:

$R^{(k)}[i, j]=R^{(k-1)}[i, j]$ OR$(R^{(k-1)}[i, k]$ AND $R^{(k-1)}[k, j])$.

Remove repeated vertices from a path regardless of cycle weights and split by whether intermediate k appears. Both directions cover exactly reachability. This proof does not require absence of negative cycles. In-place updates are safe: R[k, k]=true and R[i, k] OR(R[i, k] AND true)=R[i, k], with the same argument for row k.

For the common example, final rows are[1,1,1,1],[0,1,1,1],[0,1,1,1],[0,1,1,1]. Time is $\Theta(n^3)$ Boolean operations, space $\Theta(n^2)$ Boolean values. Bit packing can save storage and exploit word parallelism, but requires an explicit word-size and implementation assumption; arbitrary-length bit strings are not single constant-time words.

25.3 Johnson’ s algorithm for sparse graphs

Johnson makes all edges nonnegative while preserving relative costs of routes with fixed endpoints. For any finite potential $h:V \to R$, define $\hat{w}(u, v)=w(u, v)+h(u)-h(v)$. Summing along $v_0, \ldots, v_r$ cancels internal potentials:

$\hat{w}(path)=w(path)+h(v_0)-h(v_r)$.

Every route with the same endpoints receives the same adjustment, preserving strict comparisons and ties, and therefore the **set** of shortest paths. Different tie-breaking can still select different predecessor trees. Cycles receive adjustment0, preserving all negative cycles. Reweighting cannot repair a negative-cycle problem.

Adding the same constant C per edge instead adds rC to an r-edge route and can change the optimum. For direct$1 \to 4$:6 and two-edge $1 \to 2$:2,$2 \to 4$:2, the original two-edge cost 4 beats6. Adding 3 per edge gives direct9 versus two-edge 10, reversing the result.

Constructing a potential and rejecting negative cycles

Add a super-source s with zero edges to all original vertices and no incoming edges. If the original graph has no negative cycle, neither does the augmented graph. Every vertex is reachable, so Bellman–Ford gives finite $h(v)=\delta(s, v)$. For each edge $h(v) \leq h(u)+w(u, v)$, equivalent to $\hat{w}(u, v) \geq 0$. This is the needed Dijkstra contract.

The n source edges need not be stored. Initialize all $h[v]=0$, as after relaxing them, and synchronously scan original edges for $n-1$ rounds, then test whether $h[v]>h[u]+w(u, v)$ still holds. This gives zero-cost starts at any original vertex and an $n-1$-edge original budget. Without negative cycles a simple shortest path suffices. If any negative cycle exists, its potential inequalities cannot all hold, since summing gives $0 \leq$ a negative number. The final scan must reject it.

In the common example, start $h=(0,0,0,0)$. Round 1 lowers only $h[3]$ to -2; rounds 2,3 do nothing. Six reweighted edges in input order are 4,9,0,6,1,1, all nonnegative. This h is not distance from original vertex 1: $h[2]=0$ whereas $\delta(1,2)=4$.

Constructing a potential and rejecting negative cycles (continued)

Johnson potentials and six reweighted edges

h[1]=0		h[2]=0	h[3]=-2	h[4]=0
Original edge	w	h[u]	h[v]	$\hat{w}=w+h[u]-h[v]$
1→2	4	0	0	4+(0)−(0)=4
1→3	7	0	-2	7+(0)−(-2)=9
2→3	-2	0	-2	-2+(0)−(-2)=0
2→4	6	0	0	6+(0)−(0)=6
3→4	3	-2	0	3+(-2)−(0)=1
4→2	1	0	0	1+(0)−(0)=1

Green weights are nonnegative. The potential uses a new zero-edge source, not original vertex 1.

Run Dijkstra per source, then restore original distances

```
Johnson(G):
  h=Bellman-Ford potential with all-zero initialization
  if any edge can still relax: report a negative cycle and stop
  for each original edge: ŵ(u, v)=w(u, v)+h[u]-h[v]
  for source u in V:
    run Dijkstra on original vertices with nonnegative ŵ, giving distHat
    for v in V:
      D[u, v]=∞ if distHat[v]==∞ else distHat[v]-h[u]+h[v]
  return D
```

From 1, distHat begins(0,∞,∞,∞). Extract 1 to get(0,4,9,∞); extract 2 to lower3 from 9 to 4 and assign4 value 10; extract 3 lowers4 to 5. Extract 4 cannot improve2. Extracted items are 1:0,2:4,3:4,4:5, using smaller IDs on distance ties. Final(0,4,4,5) restores to(0,4,2,5).

Other distHat rows are[∞,0,0,1],[∞,2,0,1],[∞,1,1,0]. Subtract source potential and add destination potential to recover the final Floyd matrix. For 3→2, this gives 2−(−2)+0=4; for 4→3, it gives 1−0+(−2)=−1. Unreachable entries stay∞; do not apply finite arithmetic to a sentinel.

Run Dijkstra per source, then restore original distances (continued)

Dijkstra on nonnegative weights and distance restoration

All reweighted edges are nonnegative. Cells show distHat; smaller IDs break ties.

Action	To 1	To 2	To 3	To 4
Initialize	0	∞	∞	∞
Extract 1	0	4	9	∞
Extract 2	0	4	4	10
Extract 3	0	4	4	5
Extract 4	0	4	4	5

Green: strict improvement. Original distance = distHat − h[1] + h[v].
Final (0,4,4,5) restores to (0,4,2,5), matching Floyd row1.

Sparsity, queues, and the unavoidable output cost

All-zero potential initialization costs $O(n)$, plus $n-1$ scans and a final check of m original edges, giving $O(nm+n)$. Generic Bellman–Ford on the explicitly augmented graph costs $O((n+1)(m+n))$; include its n added edges.

Fibonacci Dijkstra takes $O(m+n \log(n+1))$ amortized. Running it n times and filling n^2 outputs gives $O(nm+n^2 \log(n+1))$. Binary heaps give $O(n(m+n)\log(n+1))$; their log factor on m does not disappear. Graph and per-source work use $O(n+m)$, plus $\Theta(n^2)$ if the full output is retained. Row streaming can reduce resident output space but does not reduce the n^2-entry output itself.

For $n=1000$, $m=5000$, Floyd performs10^9 triple-index updates. Potential computation scans at most about 5×10^6 original edges. Binary repeated-Dijkstra's leading expression is $1000 \cdot 6000 \cdot \log_2 1000 \approx 5.98\times10^7$; Fibonacci's is $5\times10^6+10^6 \cdot \log_2 1000 \approx 1.50\times10^7$. These count different modeled operations and are **not measured execution times or directly divisible speedups**. For dense $m=\Theta(n^2)$, Floyd remains simple and $\Theta(n^3)$, and Johnson's sparse-graph advantage disappears.

References

Bibliography

Chapter source

Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 3rd ed., MIT Press, 2009, Chapter 25.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

References and attribution

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Maximum flow

Part VI Graph Algorithms

Scope

Residual networks, cut certificates, and valid heights prove maximum flow. Complete traces reconstruct augmentation, matching, and relabel-to-front.

Prerequisites

Directed graphs, BFS, and summation notation. This chapter develops conservation, residual networks, and cuts.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Chapter route

Learning objectives

- Define flow networks and net flow value.
- Use residual edges to revise Ford–Fulkerson choices.
- Prove max-flow min-cut and connect bipartite matching.
- 26.1 Flow networks
- 26.2 The Ford-Fulkerson method
- 26.3 Maximum bipartite matching
- 26.4 Push-relabel algorithms
- 26.5 The relabel-to-front algorithm

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

26.1 Flow networks

A capacity network has finite V, distinct source s and sink t, and capacities $c(u, v) \geq 0$. There are no self-loops; nonedges have capacity 0, and same-direction parallel capacities may be combined. Write $n = |V| \geq 2$ and m for the number of positive-capacity directed edges. Isolated vertices and vertices outside all s-to-t paths are allowed.

Use antisymmetric net flow $f(u, v) = -f(v, u)$ with $f(u, v) \leq c(u, v)$. Its full range is $-c(v, u) \leq f(u, v) \leq c(u, v)$. Internal $u \neq s, t$ obeys $\Sigma_v f(u, v) = 0$. Flow value $|f| = \Sigma_v f(s, v)$ is net source outflow, not a sum of absolute edge flows.

Requiring nonnegativity for all ordered net-flow pairs would force every value to 0 by antisymmetry. Capacities may be real. First count exact arithmetic operations; integer and rational bit costs are separate.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Nonnegative arc flow differs from net flow

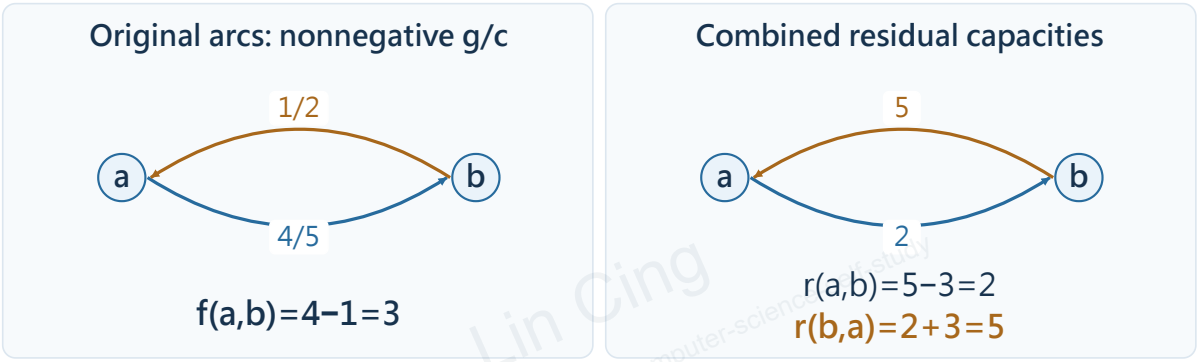
An alternative stores $0 \leq g(u, v) \leq c(u, v)$ on original arcs, without antisymmetry. Define $f(u, v) = g(u, v) - g(v, u)$, preserving conservation and net value. Opposite arc flows can cancel $\min(g(u, v), g(v, u))$ without changing any vertex's balance.

Conversely, for net f define $g(u, v) = \max(f(u, v), 0)$. Capacity bounds give $g \leq c$, and $g(u, v) - g(v, u) = f(u, v)$. Thus the models have the same maximum net value.

With $c(a, b) = 5$, $c(b, a) = 2$ and $g(a, b) = 4$, $g(b, a) = 1$, net values are 3 and -3. Canceling 1 gives arc flows 3 and 0. Forward residual capacity is $5 - 3 = 2$; reverse residual is $2 - (-3) = 5$, combining withdrawal of 3 with 2 units of original reverse capacity.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Original reverse capacity and cancellation are distinct



Reverse capacity 5 = original reverse capacity 2 + cancellation of net flow 3

For this pair of arcs, capacities are 5 and 2, original arc flows are 4 and 1, and net flow is 3. Residual b→a allows 5: undo 3 units of net flow, then use 2 units of original reverse capacity. **A residual arc need not be an original arc; reverse residual capacity need not consist only of cancellation.**

Internal cancellation gives the cut upper bound

A cut(S, T) partitions V with s∈S, t∈T. Its capacity c(S, T)=Σ_(u∈S, v∈T)c(u, v) counts only S-to-T arcs. Sum net outflows over S: source contributes |f| and other vertices contribute0. Internal antisymmetric pairs cancel, giving

$$|f| = \sum_{u \in S, v \in T} f(u,v) \leq \sum_{u \in S, v \in T} c(u,v) = c(S,T).$$

Count net crossing flow, not merely positive contributions. A feasible flow certifies a lower bound on the optimum; any cut certifies an upper bound. Equality is a checkable optimality certificate.

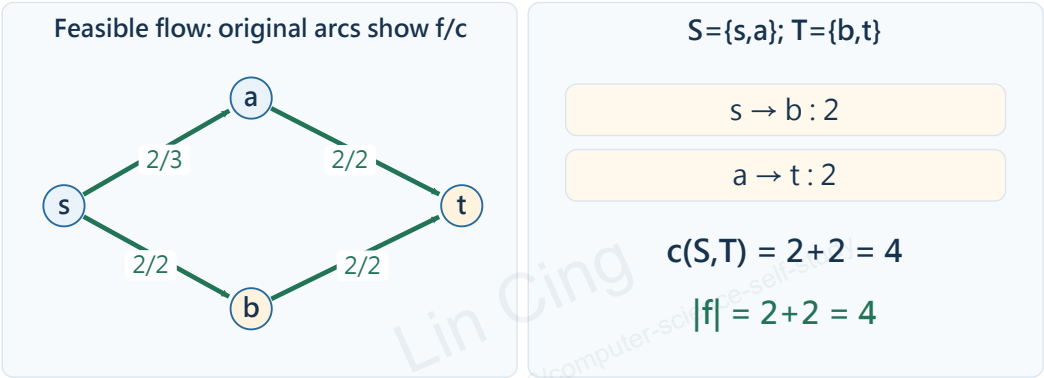
Flow and cut certificates on the same network

Capacities are s→a=3, a→t=2, s→b=2, b→t=2. Send2 along each s-a-t and s-b-t. Conservation and capacity bounds hold, with value 4.

For S={s, a}, T={b, t}, crossing capacities s→b:2 and a→t:2 sum to 4. Thus the maximum is both at least and at most4.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Flow and cut certificates on the same network (continued)



Flow gives a lower bound; cut gives an upper bound; equality proves optimality

Source outcapacity or sink incapacity alone does not establish internal connectivity. The full network, feasible flow, and equal cut establish optimality here.

Multiple sources and sinks with finite added capacities

For disjoint supply set A and demand set B without individual bounds, add $s' \to a$ for each $a \in A$ and $b \to t'$ for each $b \in B$. Use total original capacity C for added capacities instead of an implemented ∞; no useful transport exceeds that total. If individual endpoint limits exist, use those actual limits.

Attach original endpoint net supplies to the new arcs to obtain conservation. Removing those arcs recovers the original supplies and demands at unchanged total value. This establishes both directions of the reduction.

To eliminate antiparallel original arcs in a nonnegative-arc model, replace $u \to v$ of capacity c by $u \to x \to v$ with both capacities c. Conservation at x forces equal segment flow, giving an exact correspondence. Setting both capacities to c/2 would change the problem.

26.2 The Ford-Fulkerson method

Residual capacity is $c_f(u, v) = c(u, v) - f(u, v) \geq 0$. The residual graph contains exactly positive-residual arcs. Neighbor candidates include original reverse directions even when their original capacity is 0.

For a simple residual s-to-t path p, let $\delta = \min_{(uv \in p)} c_f(u, v) > 0$. Update $f(u, v) \mathrel{+}= \delta$ and $f(v, u) \mathrel{-}= \delta$ together. Forward capacity is respected; reverse decrease cannot violate its upper bound, and its lower bound is equivalent to the forward upper bound. Path-internal balance changes cancel. Source net outflow increases by δ, preserving feasibility and strictly increasing value.

```
f=zero net flow
while a residual s-t path p exists:
    delta=min(residual(u, v) for (u, v) in p)
    for (u, v) in p:
        f(u, v)+=delta; f(v, u)-=delta
return f
```

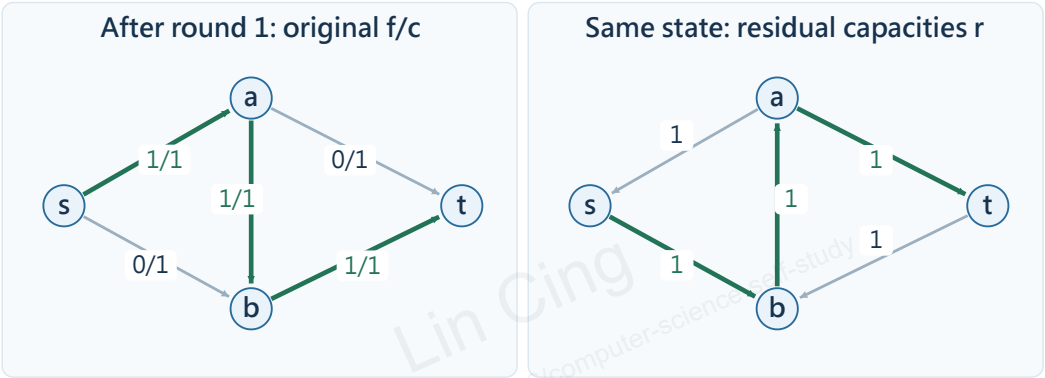
Two augmentations that withdraw an earlier choice

All original capacities s→a, s→b, a→b, a→t, b→t are 1.

Stage	Path and δ	(f_sa, f_sb, f_ab, f_at, f_bt)	Value
Initial	none	0,0,0,0,0	0
First	s→a→b→t; 1	1,0,1,0,1	1
Second	s→b→a→t; 1	1,1,0,1,1	2

Residual b→a cancels old a→b flow from 1 to 0. The final flow sends 1 on s-a-t and 1 on s-b-t. Restricting search to original directions would incorrectly stop after the first augmentation and return 1.

Two augmentations that withdraw an earlier choice (continued)

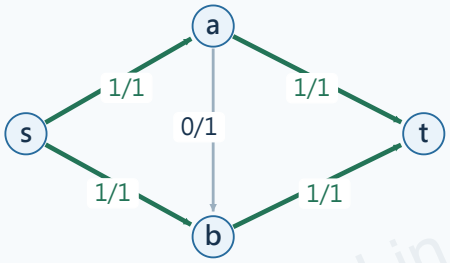


Green residual path s→b→a→t has bottleneck 1; b→a can undo old flow

When storing residual capacities directly, equivalent updates are r(u, v)−=δ, r(v, u)+=δ. Decreasing only the forward entry loses withdrawal capability.

After cancellation, check original arcs and optimality

After round 2: feasible value 2



Check every original arc

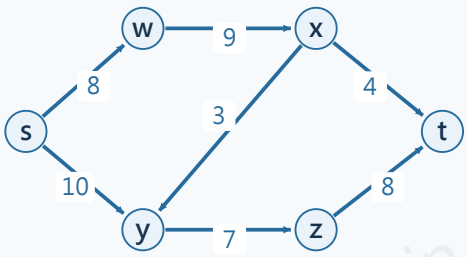
s→a	1 → 1
s→b	0 → 1
a→b	1 → 0
a→t	0 → 1
b→t	1 → 1

a→b falls from 1 to 0; both source arcs are saturated; cut {s} has capacity 2

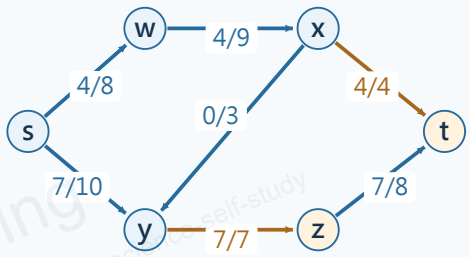
The second augmentation updates s→b, b→a, a→t. Using b→a decreases net f(a,b), without removing its original capacity. Vertices a and b each receive and send 1. Flow value 2 equals cut {s} capacity 2. Residual a→b still has capacity 1, but s cannot reach a, so it permits no further augmentation.

Six vertices: how do we reach the optimum?

Six vertices: original capacities c



After four rounds: original f/c



S={s,w,x,y}: cut arcs x→t:4 and y→z:7 total 11, equal to the flow

The original graph has x→y:3, but no y→x. The right panel shows original f/c after four augmentations, not residual capacities. Use vector order (sw,wx,xy,xt,sy,yz,zt) throughout the trace.

First send x→y, then undo it along y→x

Round	Path; residual capacities before augmentation	δ	Original flows afterward
Start	none	none	(0,0,0,0,0,0,0)
1	s-w-x-y-z-t; (8,9,3,7,8)	3	(3,3,3,0,0,3,3)
2	s-y-x-t; (10,3,4)	3	(3,3,0,3,3,3,3)

Round 1 creates residual y→x:3. Round 2 uses it, reducing original f(x,y) from 3 to 0. It does not add an original reverse arc. Now x receives 3 through wx and sends 3 through xt; y receives 3 through sy and sends 3 through yz. Source net outflow is 3+3=6.

Finish the trace and verify the cut

Vectors remain ordered (sw,wx,xy,xt,sy,yz,zt).

Round	Path; residual capacities before augmentation	δ	Original flows afterward
3	s-y-z-t; (7,4,5)	4	(3,3,0,3,7,7,7)
4	s-w-x-t; (5,6,1)	1	(4,4,0,4,7,7,7)

Value rises 6→10→11. Vertices w,x each receive and send 4; y,z each receive and send 7. Residual reachability gives S={s,w,x,y}. The only original S→T arcs are x→t:4 and y→z:7. **Feasible value 11 and cut capacity 11 prove optimality.** Source outcapacity 8+10=18 alone cannot determine the answer.

Three equivalent max-flow min-cut conditions

For feasible f, these are equivalent:(i) f is maximum;(ii) no residual s-to-t path exists;(iii) some cut satisfies $|f|=c(S, T)$.

(i)⇒(ii): an existing path would augment to a larger feasible value.

(ii)⇒(iii): let S be residual-reachable from s and T its complement. Since $t \notin S$, this is a cut. Every $u \in S$, $v \in T$ has $c_f(u, v)=0$, or v would be reachable. Thus $f(u, v)=c(u, v)$, and the net cut identity gives equality.

(iii)⇒(i): every feasible g has $|g| \leq c(S, T)=|f|$.

Neither integer capacities nor a specific path rule is required. Integrality governs termination and cost, not this equivalence.

Integrality and pseudopolynomial time

With integer capacities and initial zero flow, residuals and δ remain integers. Each augmentation increases value by at least 1. A finite cut bounds the value, so the algorithm terminates with an integral maximum net flow after at most optimum F^* augmentations.

Use adjacency lists, $O(n+m)$ initial allocation, and timestamped visited marks that initialize only touched vertices. Each newly seen nonsource vertex corresponds to a scanned edge, so one path search costs $O(m+1)$. Total time is $O(n+m+(m+1)F^*)$. Since F^* can be exponential in binary input length, this is pseudopolynomial.

Rational capacities can be scaled by the product D of denominators. Its bit length is at most the sum of denominator bit lengths, but its numeric value can be enormous, so augmentation count may still be huge. Irrational capacities do not ensure improvement by at least 1; this argument does not prove termination for arbitrary paths. BFS supplies a capacity-independent bound below.

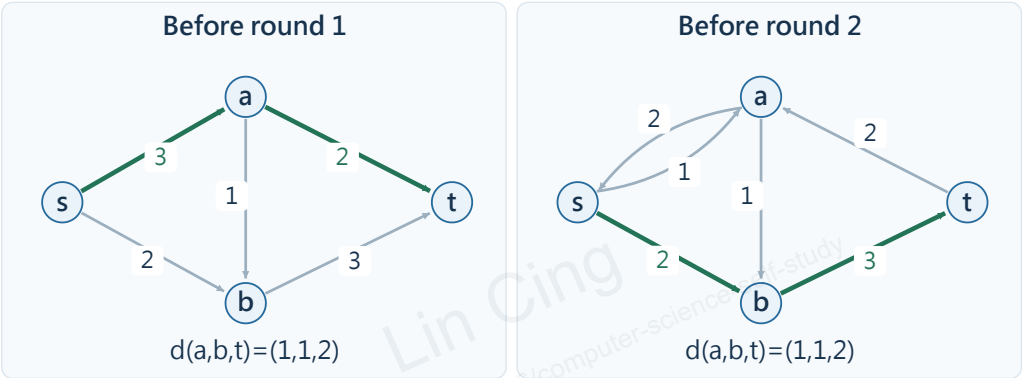
Edmonds–Karp minimizes residual edge count

Use BFS to choose a fewest-edge residual path, then augment through parents. Let d_f(v) be shortest edge count from s, or∞. For s→a=3, s→b=2, a→b=1, a→t=2, b→t=3 and neighbor a before b:

Before augmentation	d(a), d(b), d(t)	Path and δ	Flow tuple afterward
Round 1	1,1,2	s→a→t; 2	2,0,0,2,0
Round 2	1,1,2	s→b→t; 2	2,2,0,2,2
Round 3	1,2,3	s→a→b→t; 1	3,2,1,2,3
Final check	∞,∞,∞	no path	value 5; cut{s} has capacity 5

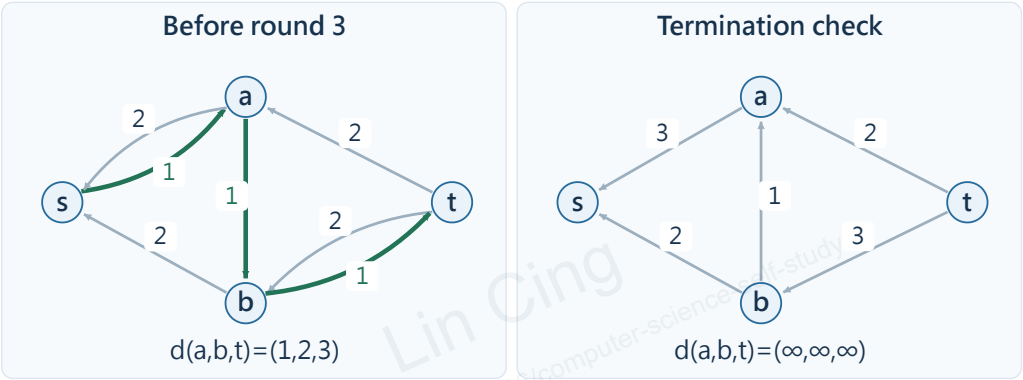
The tuple order is(f_sa, f_sb, f_ab, f_at, f_bt).

Edmonds–Karp minimizes residual edge count (continued)



All positive residual arcs shown; green marks the BFS path; bottlenecks 2, 2

Round 3 and termination: read the cut from residual reachability



Round 3 has δ=1; finally s has no outgoing arc: flow 5 equals cut {s} capacity 5

Path s→a→b→t has bottleneck min(1,1,1)=1, increasing value 4→5. The final residual arcs are a→s:3, b→s:2, b→a:1, t→a:2, t→b:3. Only s is reachable from s, so cut {s} has original capacity 3+2=5, equal to the flow. Distance ∞ means unreachable, not infinite capacity.

Residual BFS distances never decrease

Write d before and d′ after augmentation. Suppose $d'(v)<d(v)$, choosing such a v with smallest $d'(v)$. Its predecessor u on a new shortest path has $d'(u)=d'(v)-1$ and cannot be a smaller decrease counterexample, so $d(u)\le d'(u)$.

If uv was already residual, $d(v)\le d(u)+1\le d'(v)$, a contradiction. Therefore uv is a newly created reverse edge, and vu lies on the last BFS augmentation path. A prefix of that shortest path is shortest, so $d(u)=d(v)+1$.

Combining gives $d(v)=d(u)-1\le d'(u)-1=d'(v)-2$, contradicting $d'(v)<d(v)$. The old-unreachable/new-reachable∞ case also leads to finite old predecessors and hence old reachability, a contradiction. Every distance is nondecreasing.

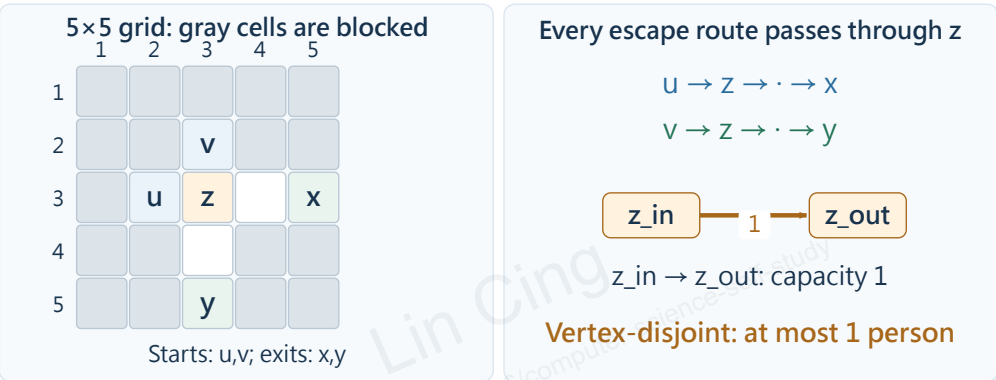
A critical edge's tail rises by at least two before reuse

Every augmentation saturates at least one critical uv at bottleneck δ, with $d(v)=d(u)+1$. For that direction to reappear, a later augmentation must traverse vu. Then $d'(u)=d'(v)+1 \geq d(v)+1=d(u)+2$, using distance monotonicity.

Thus between successive critical uses of one direction, its tail distance increases by at least 2. On an augmenting simple path the tail distance is at most $n-2$, so each direction is critical $O(n)$ times. At most$2m$ candidate directions give $O(nm)$ augmentations.

Each BFS and update takes $O(m+1)$, giving $O(n+m+nm^2)$ for $m \geq 1$, usually written $O(nm^2)$ after initialization. For $m=0$, return zero flow in $O(n)$. Common-denominator scaling of rational capacities keeps all value bit lengths polynomial in input length, and the number of rounds is independent of their magnitude. Thus the algorithm is also polynomial in bit cost; $O(nm^2)$ first counts arithmetic operations.

Escape routes: edge-disjoint does not mean vertex-disjoint



Edge capacities alone give 2; vertex capacities correctly limit escape to 1

Coordinates are (row,column). Open cells are {(3,2),(2,3),(3,3),(3,4),(3,5),(4,3),(5,3)}; all other cells are blocked. Starts are u=(3,2), v=(2,3); boundary exits are x=(3,5), y=(5,3). Unit edge capacities allow two routes sharing z=(3,3). **If no cell may be shared, flow value 2 in that model does not prove both people can escape.**

Split every cell into an entry and an exit

For a $k \times k$ grid, let q be the open-cell count, p the number of distinct occupied starts, and b the open-boundary count. To forbid all shared cells:

Constraint	Network representation
At most one person per cell	$v_{in} \to v_{out}$, capacity 1
Move between adjacent open cells	$u_{out} \to v_{in}$ and $v_{out} \to u_{in}$, each capacity 1
One person per start	source $\to$ start_in, capacity 1
Leave through an open boundary	boundary_out $\to$ sink, capacity 1

Omit blocked cells. A boundary start still traverses its internal edge. The network has $N = 2q + 2$ vertices and $M \le q + 4k(k-1) + p + b$ arcs. For a full open grid, $b = 4k - 4$ requires $k \ge 2$; $b = 1$ when $k = 1$.

Why flow p is equivalent to everyone's escape

Given p vertex-disjoint escape routes, send one unit along each corresponding network path. Conversely, integer capacities admit an integral maximum flow. Decompose it into source–sink paths and cycles; remove cycles without changing value. Value p saturates all p source arcs. Every internal capacity 1 prevents two paths from using the same cell. Mapping back gives one escape route per person. This proves both directions.

Every route in the example passes through z. Edge $z_{in} \to z_{out}$ forms a capacity-1 cut, so at most one person escapes; route $u \to z \to (3,4) \to x$ attains 1. The edge-only model admits two edge-disjoint routes and has value 2. **The answers differ because the constraints differ.**

26.3 Maximum bipartite matching

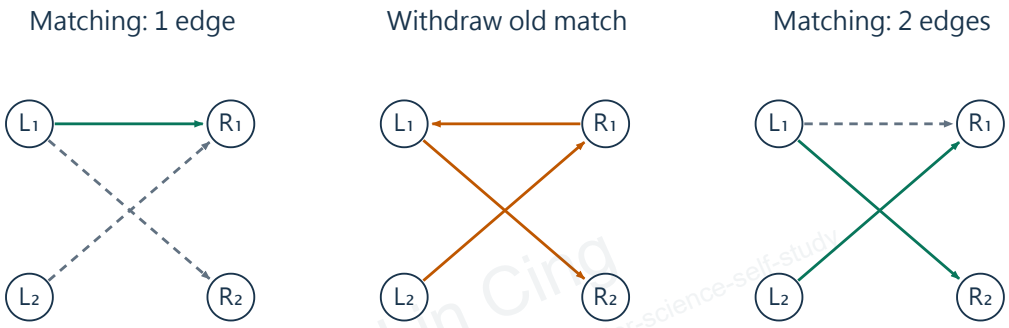
For bipartition $L \cup R$, a matching has no shared endpoints. Build unit arcs $s \to L$, original edges $l \to r$, and $R \to t$.

A matching M sends 1 along each s-l-r-t; disjoint endpoints ensure capacities, and value is $|M|$. Conversely, an integral flow gives each left vertex at most one unit total incoming, so conservation and integrality allow at most one original matching arc with flow 1. The right-side argument is symmetric. Selected unit-flow original arcs form a matching of size equal to the flow value.

Integer augmentation guarantees existence of an integral maximum flow, so maximum matching size equals maximum flow value. It does not make every optimal flow integral: in $K_{2,2}$, put1/2 on all four left-to-right arcs and 1 on source/sink arcs. This optimal value 2 flow does not make all four positive arcs a matching.

Maximum size is at most $\min(|L|,|R|)$, and each augmentation adds 1. With $m'=|E|+|L|+|R|$, direct Ford–Fulkerson costs $O(|V|+m'+m'\min(|L|,|R|))$. Isolated vertices prevent assuming $m'=\Theta(|E|)$ unconditionally.

The actual edges flipped by an alternating path



$L_2 \to R_1 \to L_1 \to R_2$: unmatched, matched backward, unmatched

Green: matching. Gray dashed: unselected original edges. Orange: residual alternating path.

Edges are L_1R_1, L_1R_2, L_2R_1; initially match L_1R_1. Augment along $s \to L_2 \to R_1 \to L_1 \to R_2 \to t$, replacing L_1R_1 by L_2R_1 and L_1R_2. Internal vertices keep one matching edge; both free endpoints become matched, raising size $1 \to 2$.

A constructive proof of Kőnig's equality

For a maximum matching, let Z be residual-reachable from s, and $C=(L \setminus Z) \cup (R \cap Z)$. Consider original lr with $l \in Z$, $r \notin Z$. If unmatched, its forward residual arc reaches r. If matched, the source-to-l arc is saturated; the last arc reaching l must come backward from its matched r, also making r reachable. Both contradict $r \notin Z$. Thus C covers every original edge.

Both endpoints of a matched edge are either reachable or unreachable together: a reachable right endpoint reaches its left mate backward, and a reachable matched left endpoint must have been entered from its right mate. Therefore C contains exactly one endpoint of every matched edge. Unmatched left vertices are source-reachable and outside C. Unmatched right vertices cannot be reachable, or an augmentation would exist, and also lie outside C. Hence $|C|=|M|$.

Any vertex cover must choose a distinct endpoint for every matching edge. Minimum bipartite cover size therefore equals maximum matching size. General graphs need not satisfy this: a triangle has minimum cover2 and maximum matching1.

Sufficiency of Hall's condition

A matching covering L requires $|N(A)| \geq |A|$ for every $A \subseteq L$, because distinct left vertices need distinct neighbors.

If a maximum matching leaves some left vertex unmatched, search alternating paths from unmatched left vertices. Let A, B be reached left and right sets. The preceding forward/backward argument gives $B=N(A)$. Every B vertex is matched, or the search yields an augmentation. Its matched left mate lies in A, which additionally contains an unmatched starting vertex. Hence $|A|>|B|=|N(A)|$, violating Hall's condition.

The condition is sufficient as well as necessary. If L is empty, the empty matching already covers it. Covering all of L is perfect on both sides only when the two side sizes also agree.

26.4 Push-relabel algorithms

A preflow retains antisymmetry and capacity bounds but allows excess $e(u)=\Sigma_v f(v, u)\geq 0$ for $u\neq s$. Active means $u\neq s$, t and $e(u)>0$. The sink accumulates excess but is never discharged.

Valid heights satisfy $h(s)=n$, $h(t)=0$ and $h(u)\leq h(v)+1$ on every residual uv. Initialize other heights and all f, e to 0, then saturate source arcs:$f(s, v)=c(s, v)$, $f(v, s)=-c(s, v)$, $e(v)+=c(s, v)$, $e(s)-=c(s, v)$.

Nonsource excess is nonnegative. No positive residual arc leaves s; other residual arcs start at height0 and satisfy validity. Heights are valid labels, not necessarily current BFS distances.

Lin Cing
https://github.com/xinhi6716/computer-science-self-st

Complete push and relabel preconditions

```
push(u, v): # u active, residual>0, h[u]==h[v]+1
    d=min(e[u], c(u, v)-f(u, v))
    f(u, v)+=d; f(v, u)-=d; e[u]-=d; e[v]+=d
relabel(u): # u active, no admissible outgoing edge
    h[u]=1+min(h[v] for v with residual(u, v)>0)
```

Admissible means positive residual and a height drop of exactly 1. A push is saturating if residual becomes 0; otherwise it is nonsaturating and empties e(u). Count simultaneous saturation and emptying as saturating.

An active vertex must have a positive-residual outgoing arc. Otherwise every $f(u, v)=c(u, v)\geq 0$, giving $e(u)=-\Sigma f(u, v)\leq 0$, a contradiction. Thus relabel minimizes a nonempty set. With no admissible arc, integer validity implies $h(u)\leq$every residual neighbor height, so relabel raises h(u) by at least 1.

Lin Cing
https://github.com/xinhi6716/computer-science-self-st

Operations preserve preflows and valid heights

Push keeps e(u) nonnegative and only increases other nonsource excess. Capacity and antisymmetry follow as for augmentation. Heights remain unchanged. The only possibly new reverse vu has h(v)=h(u)−1, so it remains valid but is not admissible.

Relabel leaves f, e fixed. The new outgoing bound follows from 1+the minimum neighbor height. Incoming wu remains valid because h(w)≤old_h(u)+1≤new_h(u), and cannot be admissible. At least one outgoing arc attaining the minimum becomes admissible.

The admissible network is a DAG: a k-edge cycle would imply h=h−k. Push creates no admissible edge, and relabel can create only its own outgoing admissible edges. These facts support the later list invariant.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Active vertices reach the source, bounding their heights

Let R be residual-reachable from active u, and suppose s∉R. All R vertices have nonnegative excess and u positive, so Σ_(v∈R)e(v)>0. No positive-residual arc leaves R, so f(v, w)=c(v, w)≥0 across its outgoing cut. Internal antisymmetric terms cancel, giving

Σ_(v∈R)e(v)=−Σ_(v∈R, w∉R)f(v, w)≤0,

a contradiction. Thus u reaches s along a simple residual path of at most n−1 edges. Summing valid-label inequalities gives h(u)≤h(s)+n−1=2n−1. Relabeling leaves u active, so its new label has the same bound; at other times its height is unchanged. Each internal vertex is relabeled at most2n−1 times, for $O(n^2)$ total.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Correctness at termination still needs a termination bound

If all internal excesses vanish, the preflow is a feasible flow. A residual s-to-t simple path of $k \leq n-1$ edges would give $n=h(s) \leq h(t)+k=k \leq n-1$, a contradiction. Therefore no augmenting path exists, and max-flow min-cut proves optimality.

Sink excess equals the flow value. Some initial excess can return to the source instead of reaching the sink.

Correctness on termination alone does not prove termination. Bound relabels, saturating pushes, and nonsaturating pushes separately to obtain a finite capacity-independent count.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

There are O(nm) saturating pushes

Fix direction uv. At its first saturation, $h(u)=h(v)+1$ and its residual becomes 0. Another positive uv push requires a reverse vu push restoring residual. At that step $h(v)=h(u)+1$; height monotonicity makes v at least 2 higher than at the prior forward saturation.

At the next uv saturation, u must again be one higher than v, and therefore at least 2 higher than its own previous saturation height. Heights are bounded by $2n-1$, so each direction saturates $O(n)$ times. At most$2m$ directions give $O(nm)$ total.

The bound uses height differences and monotonicity, not push quantity, and remains valid for fractional capacities.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Potential accounting for nonsaturating pushes

Let $\Phi=\Sigma_{\text{(active u)}}h(u)$. Initially it is 0 and always nonnegative. Relabel increases it exactly by its height increase. Across all vertices, total height increase is at most$(n-2)(2n-1)=O(n^2)$.

A saturating push can activate only its destination, adding at most$2n-1$ to Φ. The $O(nm)$ such pushes contribute $O(n^2m)$ total positive increase. A nonsaturating push empties u. A newly activated v adds $h(v)=h(u)-1$; an already-active v or terminal adds nothing. Thus Φ falls by at least 1.

Nonnegativity bounds these decreases by all positive increases, giving $O(n^2+n^2m)$, or $O(n^2m)$ when $m\geq 1$. Combining all basic operations gives $O(n^2m)$. With $m=0$, initialization leaves no active vertex and terminates directly.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

26.5 The relabel-to-front algorithm

The linked list L always contains **all** vertices except s, t, including zero-excess vertices. It is not an active-only list. Fix $N(u)=\{v:c(u, v)>0 \text{ or } c(v, u)>0\}$, including reverse candidates. Preserve current across discharges:

```
discharge(u):
    while e[u]>0:
        v=current[u]
        if v==NIL:
            relabel(u); current[u]=head(N[u])
        elif residual(u, v)>0 and h[u]==h[v]+1:
            push(u, v) # do not advance current here
        else: current[u]=next-neighbor(v)
```

A saturated edge is skipped only on the next loop iteration. If push empties u, return immediately with current still pointing to that neighbor. A different implementation that advances after every push cannot share this trace unchanged.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Why skipped current arcs need no immediate revisit

Fix an interval with unchanged h(u). If uv is skipped for zero residual, only a reverse vu push can restore it. That requires h(v)=h(u)+1, so restored uv points uphill and remains inadmissible. If uv is residual but inadmissible, valid integer heights imply $h(u) \leq h(v)$. Vertex v can only rise, never creating a one-level descent from fixed u.

Equivalently, push creates no admissible edges, and a neighbor's relabel leaves no admissible incoming edge. Only u's own relabel can create new admissible outgoing arcs; that is exactly when current resets.

Reaching NIL therefore means all skipped arcs are still unusable and relabel is legal. Previously inspecting an edge alone would not justify ignoring later state changes.

Read next from the vertex's new position

```
initialize_preflow()
L=all vertices except s, t, in a fixed order
current[u]=head(N[u]) for u in L
u=head(L)
while u!=NIL:
    old=h[u]; discharge(u)
    if h[u]>old: move u to front of L
    u=next(u) # after any move
```

Move only after the whole discharge, not after each internal relabel. After moving an interior u to the head, its next is the old head, so earlier vertices that may have gained excess are revisited. Saving old next would incorrectly skip them.

Two branches of the list invariant

At every outer-loop test:(1) L is a topological order of the admissible network restricted to nonterminals;(2) all vertices before u have zero excess. Initially nonsource heights are 0, so no admissible arc connects list vertices. Any L works, and u begins at its head.

If discharge does not relabel, push creates no admissible edge, so topological order remains. It sends excess only to later list vertices or terminals, preserving the earlier zero-excess prefix. The discharged u is empty, restoring the prefix condition at next(u).

If discharge relabels, the last relabel leaves u without admissible incoming edges, and subsequent pushes create none. Moving u to the head preserves topological order. Only empty u precedes the new next, and other relative positions stay unchanged.

When u=NIL, the entire list is a zero-excess prefix. Conservation and the generic correctness theorem give maximum flow. This proof does not transfer directly to an active-only scheduling list.

Five-vertex initialization and the complete discharge of a

Capacities are s→a=4, s→b=3, s→c=2, a→b=2, a→t=1, b→c=2, b→t=4, c→t=5. Fix N(a)=[b, t, s], N(b)=[c, t, a, s], N(c)=[t, b, s], L=[a, b, c].

Set h(s)=5, other heights0, and saturate source arcs. Record each push/relabel immediately after it finishes, before subsequent neighbor scanning.

Action	e(a, b, c, t)	h(a, b, c)	current(a, b, c)
Initialize	4,3,2,0	0,0,0	b, c, t
a scans b, t, s to NIL; relabel	4,3,2,0	1,0,0	b, c, t
Push a→b by 2	2,5,2,0	1,0,0	b, c, t
Skip b to t; push a→t by 1	1,5,2,1	1,0,0	t, c, t
Scan t, s to NIL; relabel again	1,5,2,1	6,0,0	b, c, t
Skip b, t to s; return 1 along a→s	0,5,2,1	6,0,0	s, c, t

Five-vertex initialization and the complete discharge of a (continued)

Height of a rises from 0 to 6. Move it to the front after discharge; it was already the head, so L remains unchanged. Next process b.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

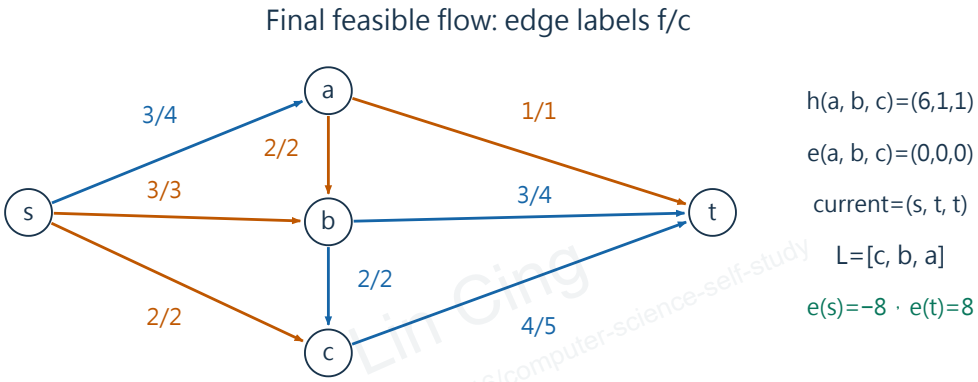
Discharging b, c and the exact move times

Action	e(a, b, c, t)	h(a, b, c)	current(a, b, c)	L
b scans c, t, a, s to NIL; relabel	0,5,2,1	6,1,0	s, c, t	a, b, c
Push b→c by 2	0,3,4,1	6,1,0	s, c, t	a, b, c
Skip c to t; push b→t by 3	0,0,4,4	6,1,0	s, t, t	a, b, c
Move b to front after discharge	0,0,4,4	6,1,0	s, t, t	b, a, c
Revisit a; excess0, no operation	0,0,4,4	6,1,0	s, t, t	b, a, c
c scans t, b, s to NIL; relabel	0,0,4,4	6,1,1	s, t, t	b, a, c
Push c→t by 4; move c after discharge	0,0,0,8	6,1,1	s, t, t	c, b, a

The new next is b, then a, both empty, and finally NIL. Sink excess stays 1 after b→c and becomes 4 only after b→t. Final source net outflow 3+3+2=8 equals sink inflow1+3+4=8.

Lin Cing

Independently verify the final flow by its cut



Vertex a receives 3 and sends 2+1; b receives 3+2 and sends 2+3; c receives 2+2 and sends 4. Residual reachability gives S={s,a}; cut capacity 3+2+2+1=8 equals feasible value 8.

One of the initial 9 units returns from a to s, leaving net value 8. No flow is lost. The cut certificate is independent of heights and scheduling.

Complete $O(n^3)$ accounting

There are $O(n^2)$ relabels. A phase lies between consecutive relabels. Without relabeling, the outer cursor advances through at most n list vertices; a relabeling discharge starts a new phase. Thus there are $O(n^3)$ discharge calls. With valid doubly linked handles, move-front costs $O(1)$.

Between its own relabels, a current pointer scans its neighbors once. Each vertex relabels $O(n)$ times, so advances and relabel minimum scans total $O(n\Sigma deg(u))=O(nm)$. Saturating pushes total $O(nm)$. A nonsaturating push empties u and ends discharge, so at most one occurs per call, for $O(n^3)$.

Total $O(n^3+nm)=O(n^3)$, since the simple capacity network has $m \leq n(n-1)$. Sparse adjacency and reverse records, heights, excesses, current pointers, and list need $O(n+m)$ space. These are arithmetic counts; capacity arithmetic and comparisons also pay their actual bit costs.

Choosing a generic operation is not free

Scanning all vertices for an active one and all neighbors at every step does not realize the $O(n^2m)$ basic-operation bound automatically. Maintain an active set instead: push changes only two excesses, and handles plus membership marks insert or remove in $O(1)$.

Use the same current rule for neighbors. All pointer advances and relabel scans total $O(nm)$, with $O(1)$ per push, realizing $O(n^2m)$ arithmetic time after initialization. Relabel-to-front uses a fixed all-vertex list and discharges to get $O(n^3)$; it is not merely another name for the active set.

Integer Ford–Fulkerson depends on F^*, Edmonds–Karp on nm^2, generic push-relabel on n^2m, and relabel-to-front on n^3. These arise from different worst-case mechanisms and do not directly predict measured machine speed.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

References

Bibliography

Chapter source

Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 3rd ed., MIT Press, 2009, Chapter 26; § 26.5.

Algorithm_wjungle, handwritten notes on flow networks, escape routes, and vertex capacities, physical PDF pages 37–42.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

References and attribution

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Computational geometry

Part VII Selected Topics

Geometric predicates, degenerate inputs, and data-structure order must agree. Orientation, intersection detection, convex hulls, and closest pairs show how local decisions produce correct global results. Prerequisites are vectors, cross products, sorting, and sweep lines.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Chapter map

The sections cover segment properties, detection of any intersecting pair, convex hulls, and closest pairs. The goals are to use cross products for turns, prove a sweep-line intersection detector, and analyze hull and closest-pair algorithms.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Closed segments and orientation

A point is $p = (x, y)$. The closed segment $ab = \{a + t(b - a) : 0 \le t \le 1\}$ includes endpoints. Intersection includes endpoint contact, collinear overlap, and segments reduced to a point, as well as proper crossings.

$$\operatorname{orient}(a, b, c) = (b_x - a_x)(c_y - a_y) - (b_y - a_y)(c_x - a_x).$$

With the x-axis pointing right and y-axis pointing up, a positive value means a counterclockwise left turn from a through b to c. Negative means right turn, and zero means collinear. This is the signed parallelogram area. The triangle area is half its absolute value. Reversing cross-product order reverses the signs and requires reversing the turn convention.

Exact signs and arithmetic limits

If integer coordinates have absolute value at most B, differences are at most $2B$, and the determinant has absolute value at most $8B^2$. The coordinate type may be too narrow for products. Widen **before** subtraction and multiplication or use arbitrary precision. A predicate on b-bit coordinates multiplies $O(b)$-bit integers and is not always constant machine time.

Floating-point cancellation can turn a small nonzero determinant into zero. Fixed epsilon cannot guarantee correct signs at every scale. Scaling coordinates by λ multiplies orientation by λ^2, while the turn stays the same. Treating approximate collinearity as exact can also destroy transitivity of a sorting comparator.

Proofs here use exact real predicates. Reproducible examples use exact integers or rationals. Floating implementations need error-bounded filters followed by increased precision when the filter is inconclusive. An unproved epsilon does not replace a sign test.

Complete segment-intersection test

Let $o_1 = \text{orient}(a,b,c), o_2 = \text{orient}(a,b,d), o_3 = \text{orient}(c,d,a), o_4 = \text{orient}(c,d,b)$. The on test requires both collinearity and closed bounds on both axes.

```
on(a,b,p):
    return orient(a,b,p)==0 and
           min(ax,bx)<=px<=max(ax,bx) and
           min(ay,by)<=py<=max(ay,by)
intersect(a,b,c,d):
    if sign(o1)*sign(o2)<0 and sign(o3)*sign(o4)<0: return true
    return on(a,b,c) or on(a,b,d) or on(c,d,a) or on(c,d,b)
```

Compare signs rather than multiplying large determinant values. Vertical segments need no slope, and on handles point segments. The bounding box alone is sufficient only after collinearity is already known.

Why both sides must straddle

When no orientation is zero, opposite signs for o_1, o_2 place c, d on opposite sides of the line through ab. Their unique line intersection lies inside cd. Opposite signs for o_3, o_4 place that same intersection inside ab, so the segments meet.

Conversely, an interior crossing puts each segment's endpoints on opposite sides of the other's line. Zero orientations cover an endpoint on another segment or collinearity. Two overlapping closed collinear intervals have an endpoint in the other interval, so the four on tests are complete. A point segment also reduces to testing its point on the other segment.

One straddling pair alone is insufficient. A segment may cross the extension of ab beyond its endpoints.

Crossings, contacts, overlap, and separation

Fix $a = (0, 0), b = (4, 0)$.

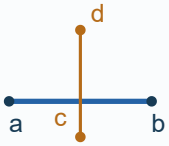
c,d	o1,o2	o3,o4	Result
(2,-1),(2,2)	-4,8	6,-6	Interior crossing (2,0)
(4,0),(6,2)	0,8	8,0	Shared endpoint b
(2,0),(6,0)	0,0	0,0	Overlap [2,4]
(5,0),(6,0)	0,0	0,0	Disjoint bounding boxes

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Crossings, contacts, overlap, and separation (continued)

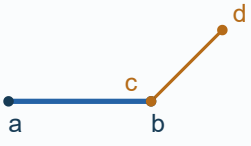
Closed-segment intersection includes endpoints and collinearity

Interior crossing



$o_1 = -4, o_2 = 8$

Shared endpoint



$o_1 = 0, o_2 = 8$

Collinear overlap

Common interval [2,4]

Collinear separation

Disjoint bounding boxes

Blue segment a=(0,0)→b=(4,0). Both axes have the same scale; y points upward.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Detect whether any pair intersects

For n closed segments, return whether some pair intersects. Listing all intersections is a different task. All-pairs testing costs $\Theta(n^2)$, and an input with $\Theta(n^2)$ intersections already needs quadratic output for full enumeration.

First consider nonvertical segments. Order each segment's endpoints as left l, right r, with $l_x < r_x$. At sweep position x, active segments satisfy $l_x \le x \le r_x$. The status orders them by height $y_s(x)$. Events are endpoints. On insertion, test the new segment against its upper and lower neighbors. Before deletion, test the two neighbors that will become adjacent.

The key is current geometric height, not segment ID:

$$y_s(x) = \frac{l_y(r_x - l_x) + (r_y - l_y)(x - l_x)}{r_x - l_x}.$$

Denominators are positive, so compare heights by cross multiplication. With constant-cost exact predicates, each balanced-tree operation costs $O(\log n)$.

Endpoint-only sweep detector

```
process endpoints in (x, left-before-right, y, id) order:
    if left endpoint of s:
        insert s in T at this x
        if intersect(s,below(s)) or intersect(s,above(s)): return true
    else:
        a=below(s); b=above(s)
        if both exist and intersect(a,b): return true
        delete s from T using its stored node handle
return false
```

Skip missing neighbors. Tests use entire segments, so the detector may return before the sweep reaches the intersection. It stops at the first detected pair. Replacing `return` by `print` and continuing loses correctness because status order changes after crossings.

Earliest-intersection proof

Every true return follows an exact segment test, so there are no false positives. Suppose an intersection exists but the detector returns false. Choose an intersection p of smallest x-coordinate. Before it, segments do not cross, so their height order cannot exchange and the status remains valid.

If other segments lie between two segments meeting at p, continuity forces those intermediate segments through p too. Among this finite group, some intersecting pair is adjacent just before p. If contact starts at an endpoint, insertion connects the same-height group. Processing left endpoints before right endpoints retains segments ending at the same point.

The pair first becomes adjacent either when one is inserted or when the last segment between them is deleted. Both events test their entire segments and would return true, a contradiction. Collinear overlap likewise creates an intersecting adjacent pair when overlap starts at an endpoint.

No re-sort at every x is needed. Before detection, no order exchange occurs. If one is about to occur, the earlier neighbor test has already stopped the algorithm.

Deleting a middle segment creates neighbors

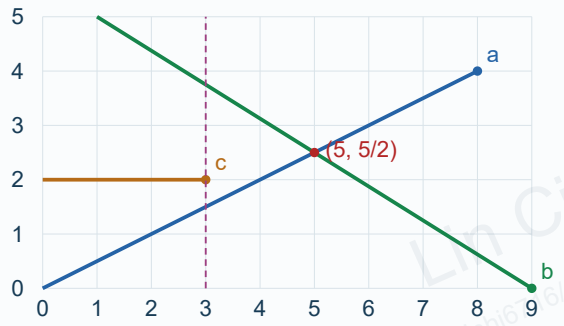
Let $a : (0, 0) \to (8, 4)$, $b : (1, 5) \to (9, 0)$, $c : (0, 2) \to (3, 2)$. Segments a, b meet at $(5, 5/2)$. Segment c meets neither.

Event	Status from low to high	New test
x=0 insert a	a	None
x=0 insert c	a,c	a,c do not meet
x=1 insert b	a,c,b	c,b do not meet
x=3 before deleting c	a,c,b	a,b meet, return true

At x=3, heights are $3/2, 2, 15/4$. Only removing c makes a, b adjacent. Testing insertions alone would miss this event.

Deleting a middle segment creates neighbors (continued)

Deleting c exposes the new neighbor pair a,b



Heights at x=3:

a: 3/2

c: 2, about to be deleted

b: 15/4

Test a,b before deleting c

Accept even if the intersection lies ahead

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Vertical segments, endpoints, and cost

For general closed segments, batch equal-x events. Insert nonvertical segments starting there, process all vertical segments at that x, then delete nonvertical segments ending there. Endpoint contact remains active.

For a vertical segment with y-range $[L, H]$, find the first status segment of height at least L. It intersects exactly when that height is at most H. This is a tree `lower_bound` in $O(\log n)$, not insertion of the vertical segment as one height key. Sort same-x vertical intervals by L and maintain the largest previous H. If the next L is at most that maximum, intervals overlap. Otherwise update the maximum. Point segments belong to this closed-interval group too.

Sorting costs $O(n \log n)$. There are at most $2n$ nonvertical updates and n range queries, each logarithmic. Total sorting across vertical groups is also $O(n \log n)$. The general detector uses $O(n \log n)$ time and $O(n)$ space under constant-cost exact geometric arithmetic. Cross products on b-bit coordinates require the corresponding additional bit cost.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Convex hull and output convention

A convex set contains the segment between every pair of its points. For finite Q, $CH(Q)$ consists of convex combinations $\sum_i \lambda_i p_i$, with $\lambda_i \geq 0$ and $\sum_i \lambda_i = 1$. It is also the smallest convex set containing Q.

Deduplicate coordinates first. Output only extreme vertices in counterclockwise order, without repeating the first at the end. Points in the interior of a hull edge are omitted. Empty input gives no vertices. One unique point gives that point. An all-collinear set with at least two points gives just its two endpoints.

Graham scan sorts by polar angle and maintains a stack. Jarvis march finds a supporting edge at each step and depends on hull size h. Neither needs floating-point `atan2`. Exact orientation and squared distance comparisons suffice for ordering and collinear ties.

Graham scan and equal-angle points

Choose p_0 with smallest y-coordinate, breaking ties by smallest x. Other points lie in the upper half-plane or along positive x. Compare directions u, v using $\text{orient}(p_0, u, v) > 0$. Break collinear ties by squared distance from p_0. Keep only the farthest point on each ray. All others lie between it and p_0, so removing them preserves the hull.

```
S=[]
for p in [p0]+points_in_strictly_increasing_polar_angle:
    while len(S)>=2 and orient(S[-2],S[-1],p)<=0:
        pop S
    push p
return S
```

The size test avoids empty-stack access. Handle all-collinear input by returning endpoints first. For noncollinear input, the result closes to a convex polygon. Using `<=0` removes flat intermediate vertices, matching the extreme-vertex convention. Listing every boundary point needs different same-ray and final-collinear-chain handling.

Why a popped vertex is no longer extreme

For a new point p_i, consider the final two stack points r, t. Their polar angles increase from p_0. If $\text{orient}(r, t, p_i) \leq 0$, then t is not a new hull vertex.

If $r = p_0$, strictly increasing angles spanning less than π give a positive turn, so no pop occurs. Otherwise translate p_0 to zero. Rays r, t, p_i occur in order within a span below π, so t is in the cone of r, p_i. Write $t = \alpha r + \beta p_i$, $\alpha, \beta \geq 0$. With $D = \text{orient}(0, r, p_i) > 0$,

$$\text{orient}(r, t, p_i) = (\alpha + \beta - 1)D.$$

A nonleft turn gives $\alpha + \beta \leq 1$. Thus $t = \alpha r + \beta p_i + (1 - \alpha - \beta)p_0$ is a convex combination in the triangle. Removing it preserves the new hull. Equality puts it on edge rp_i, still not extreme. This containment argument justifies repeated pops.

Stack invariant and correctness

Before processing p_i, the stack is exactly the counterclockwise hull vertices of the processed points. One or two points establish the degenerate base case. The first noncollinear point forms a triangle.

The new point has the largest polar angle, so $p_0 \to p_i$ is a supporting edge. All earlier points lie on its right or on its line. Looking backward from the old hull's end, edges not visible from p_i remain unchanged. A contiguous visible end chain is replaced by new edges. Each pop preserves the hull by the triangle proof. When (r, t, p_i) becomes a left turn, $t \to p_i$ is the other tangent. The remaining convex edges and $p_i \to p_0$ bound a convex polygon.

That polygon contains the old hull and new point, while all its vertices are input points. It both contains and is contained in the new convex hull, proving equality. Induction completes correctness. Each point is pushed once and popped at most once, so the scan takes $O(n)$. Sorting dominates at $O(n \log n)$, with $O(n)$ space.

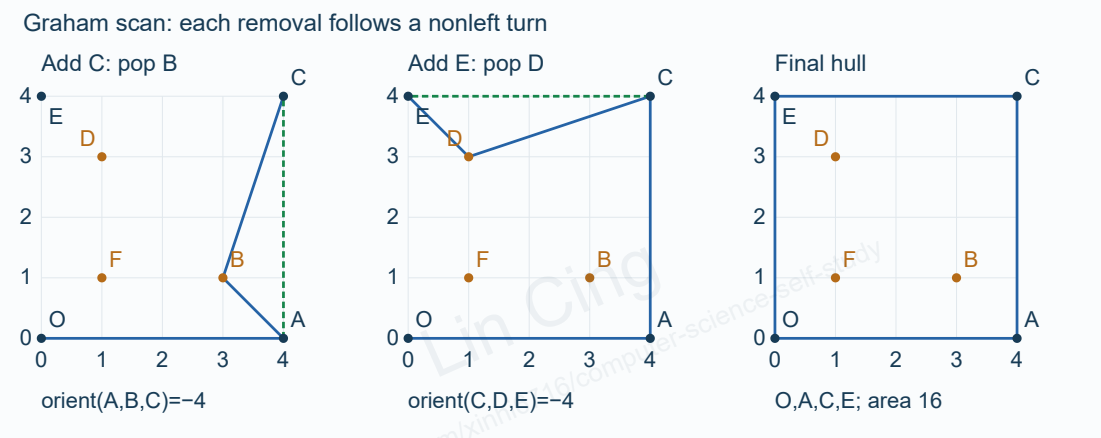
Two numerical pop checks

Use $O(0,0), A(4,0), B(3,1), C(4,4), D(1,3), E(0,4)$ in that angle order. Point $F(1,1)$ shares C's ray and is removed first.

New point	Predicate	Stack after processing
O,A,B	orient(O,A,B)=4	O,A,B
C	orient(A,B,C)=-4, pop B; orient(O,A,C)=16	O,A,C
D	orient(A,C,D)=12	O,A,C,D
E	orient(C,D,E)=-4, pop D; orient(A,C,E)=16	O,A,C,E

Indeed, $B = O/4 + A/2 + C/4$ and $D = O/4 + C/4 + E/2$. The popped points are convex combinations of other points. The hull is the square O,A,C,E, with area 16.

Two numerical pop checks (continued)



Jarvis march finds supporting edges

Start at the lexicographically smallest point p. Choose any $q \neq p$. Scan all $r \neq p$. If $\text{orient}(p, q, r) < 0$, set $q = r$. For collinear points, retain the farther one. At the end, all points lie on or left of $p \to q$, so q is the next counterclockwise extreme vertex. Output p, set $p = q$, and stop upon returning to the start.

Why does one scan suffice? At a hull vertex, all directions to other points lie in a supporting half-plane spanning at most π. Orientation linearly orders the rays there. Each update retains the clockwise-most ray seen so far, and same-ray ties retain the farthest point. The final ray is a supporting edge. Repeated steps go between extreme vertices and do not enter the interior.

Each of h hull edges scans $n - 1$ points, giving $O(nh)$ time and $O(h)$ space including output. Since $h \leq n$, worst-case time is $O(n^2)$. The square example selects A,C,E,O and has $h = 4$. Jarvis can help when h is small, but is not always faster than Graham.

Closest pair and balanced groups

For $n \geq 2$ points with unique IDs, find one pair minimizing Euclidean distance. Repeated coordinates are legal and may give zero. For fewer than two points, report no pair. Compare squared distances instead of computing roots for every candidate.

Sort X by `(x,y,ID)` and Y by `(y,x,ID)`. Split X by rank, assigning its first $\lceil n/2 \rceil$ IDs to L and the rest to R. Choose dividing x-coordinate ℓ between their x-ranges, allowing that same x if tied. Stably partition Y by **ID membership**. Using only $x < \ell$ would fail for equal-x inputs and could destroy balance.

Recurse for δ_L, δ_R, and set $\delta = \min(\delta_L, \delta_R)$. If zero, return immediately. Any improving cross-group pair has both endpoints within δ of the dividing line and vertical separation below δ. Only the width-2δ strip matters.

Closest-pair pseudocode

```
closest(X,Y):
    if len(X)<=3: return brute-force closest pair
    split X by rank into XL,XR; choose dividing x=ell
    split Y by IDs into YL,YR, preserving y order
    best=min(closest(XL,YL), closest(XR,YR))
    delta=distance(best)
    if delta==0: return best
    Z=[p in Y with (px-ell)^2 < delta^2]
    for i=0..len(Z)-1:
        for j=i+1..min(i+7,len(Z)-1):
            if (Z[j].y-Z[i].y)^2 >= distance(best)^2: break
            if distance2(Z[i],Z[j]) < distance2(best): best=(Z[i],Z[j])
    return best
```

The strip uses the fixed δ from the recursive results. The best distance can shrink during comparisons, but at most seven successors are inspected per point. Since only one minimum pair is requested, equal-distance candidates need no update. Strict strip membership and >= termination are valid. Listing all minimum pairs needs explicit equality and output rules.

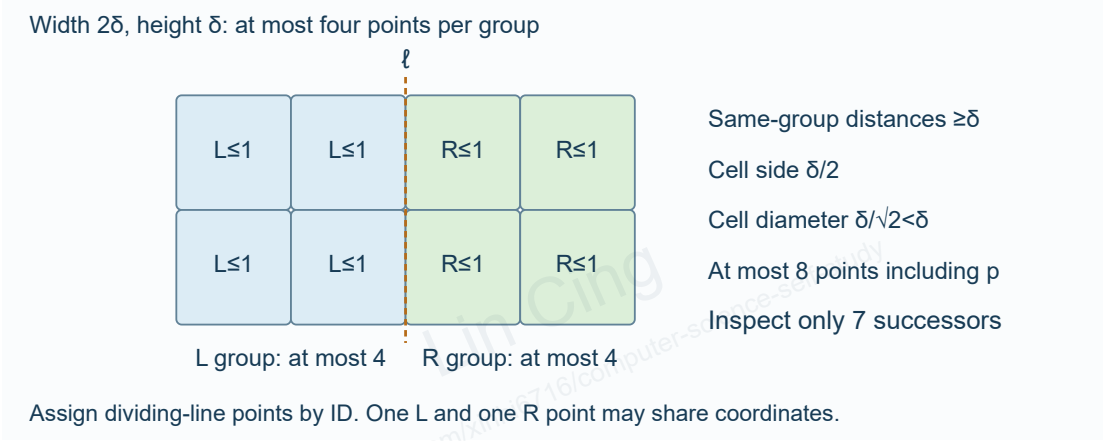
Packing proves seven successors suffice

Fix a strip point p. Any later point q that improves the result has $0 \leq q_y - p_y < \delta$, placing it in a width-$2\delta$, height-$\delta$ rectangle. L points lie in its left δ-square and R points in its right square. Within each group, pairwise distances are at least δ, by recursion.

Divide each half-square into four $\delta/2$-squares. Assign boundary points using fixed half-open conventions. A cell's diameter is $\delta/\sqrt{2} < \delta$, so it contains at most one point from that group. There are at most four points per group and eight total. Excluding p, at most seven remain. Every possible improving q is among the next seven positions in y-order.

Points on the dividing line belong to one group by ID and are counted in that group's half-square. Do not treat points from both groups on that line as if they share one within-group spacing guarantee. Cross-group duplicate coordinates are found by these comparisons. Within-group duplicates already make recursion return zero.

Packing proves seven successors suffice (continued)



A complete merge example

X order is $A(0,0), B(1,4), C(2,1), D(3,7), E(4,1), F(5,5), G(6,0), H(7,8)$. Split A-D from E-H and choose $\ell = 7/2$.

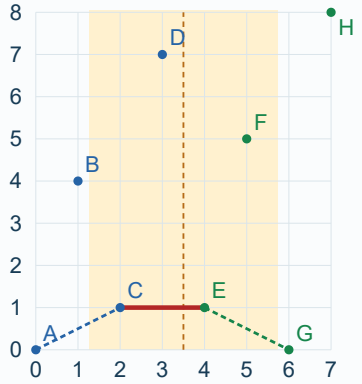
The left best pair A,C has squared distance 5. The right best pair E,G also has squared distance 5. Thus $\delta = \sqrt{5}$. The strip in y-order is C(2,1), E(4,1), F(5,5), D(3,7).

Point	Successor checks	Result
C	E has squared distance 4; next F has y-gap 4	Set best=C,E; 4≥2, stop this row
E	F has y-gap 4	Cannot improve distance 2, stop
F	D has y-gap 2	Equality cannot improve, stop
D	No successor	Done

The cross-group pair C,E has distance 2, improving both subproblems' $\sqrt{5}$. The figure uses the fixed $\sqrt{5}$ strip even after the best distance shrinks.

A complete merge example (continued)

Both sides have distance √5; cross-group C,E improve it to 2



ℓ=7/2, fixed δ=√5

L group A-D; R group E-H

Strip in y-order: C,E,F,D

$|C-E|^2=(4-2)^2+0^2=4$

Both sides have minimum squared distance 5

After cross-group checks: best²=4

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Stable partitioning determines the bound

Rank splitting X, stable ID partitioning of Y, strip construction, and seven comparisons per point take $O(n)$ nonrecursive work. Thus

$$T(n) = T(\lceil n/2 \rceil) + T(\lfloor n/2 \rfloor) + O(n) = O(n \log n).$$

The initial two sorts also take $O(n \log n)$. Sorting Y again in every subproblem instead gives $O(n \log n)$ work per merge level contribution and overall $O(n \log^2 n)$. Mark each ID's group at the current level. Searching the whole L array separately for every Y point would not be a linear partition.

With depth-first recursion and prompt release of temporary arrays, array lengths along one recursive chain sum to $n + n/2 + \cdots = O(n)$. Sibling arrays and return records preserve the $O(n)$ workspace bound, plus $O(\log n)$ stack. Saving every array in the entire recursion tree instead takes $O(n \log n)$ space.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Source

Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 3rd ed., MIT Press, 2009, Chapter 33.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

References and attribution

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

NP-completeness

Part VII Selected Topics

Bit encodings, quantifiers, and verifiable certificates define computational hardness. We construct the core reductions, prove both semantic directions, and bound their size and running time. Prerequisites are decision problems, Turing machines, reductions, and Boolean circuits.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Chapter map

The sections cover polynomial time, polynomial-time verification, reducibility, NP-completeness proofs, and NP-complete problems. The goals are to define P, NP, NP-hardness, and NP-completeness, construct many-one reductions, and prove the SAT, clique, vertex-cover, Hamiltonian-cycle, TSP, and subset-sum connections.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Decision languages and encodings

An abstract problem maps instances to answers. A decision problem has yes/no answers, represented by a language $L = \{x : \text{the answer on } x \text{ is yes}\}$. One object can have different encodings. An integer t needs about $\log_2(t+1)$ bits in binary but t symbols in unary. Polynomial time is measured in actual input bits, so the encoding must be specified.

$$L \in \mathsf{P} \iff \exists \text{ deterministic TM } M, \exists \text{ polynomial } p : \forall x,\ M(x) \text{ halts within } p(|x|), \quad M(x) = 1 \iff x \in L.$$

The quantifier is every x, not some x. Malformed encodings must also be rejected or otherwise handled as specified in polynomial time.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Polynomial composition and practical speed

If A produces a valid encoding within $p_A(n)$ steps and B solves the next problem within p_B steps, their composition takes at most $p_A(n) + p_B(p_A(n))$. Output length cannot exceed production time. Substituting a polynomial into another polynomial remains polynomial.

Thus a problem that reduces in polynomial time to a polynomial-time solvable problem is also in P. Multiple polynomial conversions compose, provided the semantic direction is correct and lengths stay bounded.

Running quickly on one computer is not a membership proof. $O(n^{100})$ is polynomial, while a low-constant exponential algorithm can be more useful for small n. Complexity classes and engineering speed answer different questions.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Pseudopolynomial is not input-bit polynomial

A cost $O(nt)$ may be exponential in the bit length of t. Subset-sum DP has $n(B+1)$ cells and costs $O(nB)$. With binary B, this depends on numerical magnitude rather than input length and is pseudopolynomial.

For a traceable family, let $n = m, B = 2^m, a_i = 2^{m-1} + i, i = 1, \ldots, m, m \geq 2$. Each number has $\Theta(m)$ bits, and B has m+1 bits. Total input length is $N = \Theta(m^2)$, but full-table work is

$$\Theta(m2^m) = 2^{\Theta(m)} = 2^{\Theta(\sqrt{N})}.$$

If bounded by CN^c for fixed c,C, then eventually $m2^m \leq C'm^{2c}$, contradicting $m2^m/m^{2c} \to \infty$. The algorithm can still be useful for small B or unary input. It does not prove binary SUBSET-SUM belongs to P.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

NP's complete quantifiers

Fast verification requires a bounded certificate and a deterministic verifier:

$$L \in \text{NP} \iff \exists V, \exists \text{ polynomials } p, q: \quad \forall x, \quad x \in L \iff \exists y \, (|y| \leq q(|x|) \wedge V(x, y) = 1).$$

The verifier halts on **all** x,y within $p(|x| + |y|)$ steps. The string y is a certificate or witness. Quantifier order matters. A yes instance has some witness. For a no instance, no witness can make the verifier accept. Nondeterministically guessing a witness represents this existential quantifier as an accepting computation path.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Completeness and soundness of a verifier

For relation R(x,y), completeness says every $x \in L$ has a polynomial-length y with R(x,y)=1. Soundness says that existence of an accepted y implies $x \in L$. Both are needed for the equivalence. Completeness alone may accept false instances. Soundness alone may miss true ones.

For Hamiltonian cycle, a certificate is a sequence of n vertices. Verify that its length is n, that it is a permutation of all vertices, that each consecutive pair is an original edge, and that the last vertex connects to the first. Checking only the length is unsound.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

P is contained in NP

If $L \in P$, let M be its deterministic polynomial-time decider. Define V(x,y) to ignore y and run M(x). Use the empty certificate. If x is in L, that certificate accepts. If x is outside L, every y is rejected. Thus soundness and completeness hold and $P \subseteq NP$.

The reverse containment remains open. Failure to find a polynomial algorithm is not a proof that none exists.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Polynomial many-one reductions

The notation $A \leq_p B$ means there is a function f computable in polynomial time on every source instance, producing a valid target instance, such that for every x,

$$x \in A \iff f(x) \in B.$$

The direction converts source A into target B. To prove a new B hard, reduce a known hard A to B. Reducing B to an easy problem cannot establish hardness.

A many-one reduction produces one B instance. It does not adapt the source after a solver response or make multiple oracle calls, unless those computations themselves fit inside the legitimate polynomial-time conversion.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Prove both semantic directions

The forward direction $x \in A \Rightarrow f(x) \in B$ constructs a target yes witness from each source yes witness. The reverse direction $f(x) \in B \Rightarrow x \in A$ shows that any target witness decodes to a source yes instance. Authors sometimes swap the names completeness and soundness depending on viewpoint, so state the actual implication.

For a clique reduction, any target k-clique must carry enough consistent information to reconstruct a satisfying assignment. A gadget, edge rule, or threshold that permits contradictory assignments breaks the reverse implication.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Transitivity with length bounds

Suppose $A \leq_p B$ via f and $B \leq_p C$ via g. Define $h(x) = g(f(x))$. If x belongs to A, f(x) belongs to B and h(x) to C. Conversely, h(x) in C implies f(x) in B and then x in A. Thus $x \in A \iff h(x) \in C$.

If f takes $p_f(n)$ time, then $|f(x)| \leq p_f(n)$. Composition takes at most $p_f(n) + p_g(p_f(n))$, a polynomial. Therefore $A \leq_p C$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

NP-hard and NP-complete

A language L is NP-hard if $A \leq_p L$ for every $A \in NP$. It is NP-complete if it is both NP-hard and in NP.

Given known NP-complete C, prove L is in NP and $C \leq_p L$. For any A in NP, hardness of C gives $A \leq_p C$, and transitivity gives $A \leq_p L$. Without membership, the conclusion is only NP-hard. Without the reduction in this direction, known hardness has not been transferred.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

The first NP-complete problem

CIRCUIT-SAT asks whether a finite acyclic Boolean circuit with fan-in-two AND/OR gates and unary NOT gates has an input making its designated output 1. Original input bits form the certificate. Evaluate gates in topological order and check the output in linear gate-and-wire time, proving NP membership.

For hardness, fix any L in NP, its deterministic verifier V, certificate bound q(n), and time bound. Given x, the free inputs to circuit C_x must represent witness y. They are not an unspecified accepting state of one fixed computation.

Normalize witnesses to exactly $q(n) + b(n)$ bits, where $b(n) = 1 + \lceil \log_2(q(n) + 1) \rceil$. The first b bits encode actual length ℓ; the next q bits store y padded with zeros. The new verifier checks $0 \leq \ell \leq q(n)$, reads only those ℓ witness bits, then simulates V. If q(n)=0, the one-bit length field permits only zero. This preserves existence of a witness without conflating different string lengths.

Expand local machine transitions into gates

Fix a single-tape deterministic TM simulating that verifier, remaining in its terminal state after acceptance or rejection. A multitape-to-single-tape simulation costs at most a polynomial factor. Let T(n) be its polynomial step bound, including the normalized witness. Initial tape length is $N = O(n + q(n) + 1)$, including the nonempty length field.

A head moves at most one cell per step. Represent $O(N + T)$ cells and retain T+1 blank cells beyond both ends. A cell label contains its tape symbol and whether the head is there, with its state if present. Fixed machine states and alphabet use a constant b bits per label.

The label at time t+1 and position j depends only on labels j-1,j,j+1 at time t. Each output bit is a Boolean function of a fixed 3b inputs. Express its truth table as an OR of minterms, each an AND of inputs or negated inputs, then replace large gates by fan-in-two gates. Because the machine and b are fixed, every local update uses $O(1)$ gates.

Circuit size and the simulation equivalence

The initial layer hardwires x, blanks, head location, and initial state. Only normalized witness bits are free. For every time $0, \ldots, T-1$ and cell, copy the fixed local circuit and connect it to the next layer. An OR tree checks for an accepting-state label at time T.

Gate and wire counts are $O(T(N+T))$. Gate indices use $O(\log(T(N+T)))$ bits. Enumerating times, cells, and a fixed truth table produces the circuit in polynomial time, with no hidden exponential hardware.

For each y, induction on t proves every layer is the actual configuration of V(x,y). The initial layer is legal, and local transitions produce exactly the next configuration. Invalid local labels are never reached. Terminal acceptance persists, so $C_x(y) = 1$ iff V accepts.

If x is in L, an accepting witness satisfies the circuit. If the circuit is satisfiable, its free witness bits make V accept, so x is in L. Every NP language reduces this way to CIRCUIT-SAT, proving NP-completeness.

Circuit-to-SAT gate constraints

Give each wire value a variable, sharing the same gate-output variable where needed. Require output z_{out} by a unit clause. Encode every gate equivalently:

Relation	CNF constraints
z=u AND v	$(\neg z \vee u) \wedge (\neg z \vee v) \wedge (z \vee \neg u \vee \neg v)$
z=u OR v	$(z \vee \neg u) \wedge (z \vee \neg v) \wedge (\neg z \vee u \vee v)$
z=NOT u	$(z \vee u) \wedge (\neg z \vee \neg u)$

For AND, the first two clauses force both inputs true if z is true; the last forces z true if both inputs are true. OR and NOT follow by the same two-direction truth cases. Fix constant wires with unit clauses.

Circuit-to-SAT gate constraints (continued)

An accepting circuit input extends to all computed wire values and satisfies the constraints. Conversely, gate equivalences force every wire to equal its actual value in topological order, and the output unit clause forces acceptance. At most three clauses per gate give linear total size in gates and wires. Repeatedly expanding shared subcircuits into formula trees could instead cause exponential growth.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

SAT to three-variable clauses

Parse a formula into unary NOT and binary AND/OR nodes. Replace implication by $\neg a \vee b$. Other fixed-arity connectives have fixed-size truth-table gadgets. Give each internal node a new variable, apply the gate clauses, and require the root true. Remove repeated literals within a clause. Delete clauses containing both a literal and its negation, since they are tautologies.

Clauses now have at most three literals. To require exactly three distinct variables per clause, use fresh padding variables for each clause:

- Replace $(a \vee b)$ by $(a \vee b \vee p) \wedge (a \vee b \vee \neg p)$. If the original is true, any p works. If false, the pair forces both p and its negation.
- Replace unit a by the four clauses $(a \vee p \vee q), (a \vee p \vee \neg q), (a \vee \neg p \vee q), (a \vee \neg p \vee \neg q)$. If a is false, every p,q assignment falsifies one clause. If true, all pass.
- An empty clause is false, so output the padded version of $z \wedge \neg z$. An empty conjunction is true, so output one fixed satisfiable three-variable clause.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

SAT to three-variable clauses (continued)

Original assignments extend through parse-node values. Conversely, the tree constraints recover node semantics. Padding preserves each clause requirement. This is equisatisfiability, not literal equality of functions after adding variables. Constant clauses per node and at most four per padding step give polynomial size and construction.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Splitting a long clause

For $l_1 \vee \cdots \vee l_r$, $r > 3$, a chain also works:

$$(l_1 \vee l_2 \vee y_1) \wedge (\neg y_1 \vee l_3 \vee y_2) \wedge \cdots \wedge (\neg y_{r-3} \vee l_{r-1} \vee l_r).$$

If all original literals are false, the first clause forces y1 true, each middle clause forces the next y true, and the last forces the final y false, a contradiction. If some literal is true, let yj indicate that the first j+1 literals are all false. Every chain clause then holds. This uses r-3 new variables and r-2 clauses. Unrestricted distributive expansion of a formula into CNF may be exponentially large.

For $F = (a \vee b) \wedge \neg a$, set $u = a \vee b, v = \neg a, z = u \wedge v$, add their gate clauses, and require z. Input a=false,b=true extends to u=v=z=true. If a=true, the NOT gadget forces v=false while z and AND require v=true, excluding a false acceptance.

The book's three-CNF definition uses three distinct literals. The main reductions here can further normalize to three distinct variables. Deduplication and padding also handle repeated literals before the occurrence construction.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Three-CNF-SAT to CLIQUE

For a formula F with k clauses, create one vertex per literal occurrence and set the clique threshold to k. For k=0, the empty conjunction is true and maps separately to `(empty graph,0)`. Otherwise k≥1.

Vertices within a clause have no edges. Vertices from different clauses are adjacent exactly when their literals are not complementary. With three occurrences per clause there are 3k vertices.

Let |F| be actual binary encoding length. Intern appearing variable names into n IDs, n≤3k. Checking vertex pairs costs $O(k^2)$ word comparisons. Bit accounting includes $O(\log(n+k))$ bits per ID comparison and edge endpoint. A safe bound is $O(|F|\log|F| + k^2\log(n+k))$. An adjacency matrix has $\Theta(k^2)$ bits. Comparing arbitrarily long variable names is not automatically constant time.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A consistent occurrence clique

$$F = (x_1 \vee \neg x_2 \vee x_3) \wedge (\neg x_1 \vee x_2 \vee \neg x_3) \wedge (x_1 \vee x_2 \vee x_3).$$

Assignment $x_1 = false, x_2 = false, x_3 = true$ satisfies all clauses. Select x3 from the first, ¬x1 from the second, and x3 from the third. They come from different clauses and are pairwise noncomplementary, forming a size-3 clique.

This is not the only witness. Selecting x1 from the first and ¬x1 from the second would fail because they lack an edge. Under the chosen assignment, x1 is false anyway. The graph cannot combine contradictory requirements into a witness.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

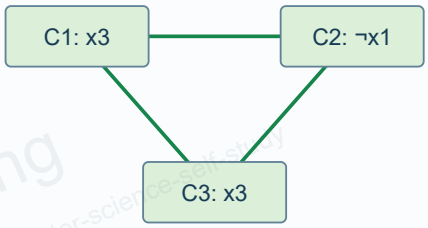
A consistent occurrence clique (continued)

Connect different clauses only for noncomplementary literals

C1 / C2	¬x1	x2	¬x3
x1	×	✓	✓
¬x2	✓	×	✓
x3	✓	✓	×

✓ Edge; × complementary pair, no edge

The three selected occurrences are consistent



x1=0, x2=0, x3=1: one true literal from each group

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Both directions of the clique reduction

If F is satisfiable, fix a satisfying assignment and select one true occurrence from every clause. Two chosen literals cannot be complementary under one assignment. Every pair therefore has an edge, producing a k-clique.

Conversely, a k-clique has at most one vertex per clause since same-clause pairs lack edges. With k clauses, it selects exactly one from each. Every selected pair is noncomplementary, so no variable is required both true and false. Set each selected variable to satisfy its chosen literal and set unselected variables arbitrarily. These consistent choices satisfy every clause.

Thus F is satisfiable iff the constructed graph has a k-clique. A list of k vertices is a polynomial certificate checked by pairwise edges, so CLIQUE is NP-complete.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Clique, independent set, and vertex cover

An independent set has no edge between its members. A vertex cover touches every edge. For a loopless undirected n-vertex graph, its complement has edge uv exactly when u≠v and G lacks that edge. Thus a set is a clique in G iff it is independent in its complement.

If S is independent in H, every H edge has an endpoint outside S, so $V \setminus S$ is a vertex cover. Conversely, a cover $V \setminus S$ forbids any edge with both endpoints in S, so S is independent. For $0 \leq k \leq n$,

$$\mathrm{CLIQUE}(G, k) \iff \mathrm{VC}(\overline{G}, n - k).$$

The reductions $(G, k) \mapsto (\overline{G}, n - k)$ and $(H, \ell) \mapsto (\overline{H}, n - \ell)$ both take $O(n^2)$ pair checks. A clique of at least k contains one of exactly k. A cover of at most n-k can be enlarged to exactly n-k. Handle out-of-range thresholds with fixed yes/no instances rather than negative certificate sizes.

A complement example

For vertices {1,2,3} and edges {12,23}, the set {1,2} is a size-2 clique. The complement has only edge 13. The same set is independent there, and its complement {3} covers the only edge, with size 3-2=1.

The transformation is to VC on the complement, with threshold n-k. Complementation is polynomial and reversible, giving reductions in both directions.

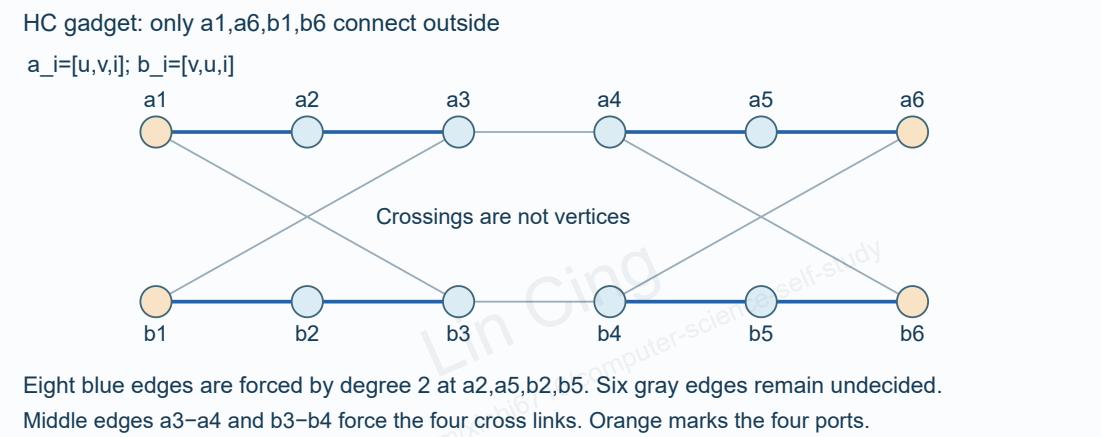
Vertex cover to Hamiltonian cycle

A Hamiltonian-cycle certificate is a permutation of all vertices with valid consecutive edges and closure. This proves NP membership. For hardness, build an undirected gadget for every original edge {u,v}. Crossings in its drawing are not vertices, and no planar crossing gadget is required.

Create twelve vertices a1,...,a6 and b1,...,b6, where ai=[u,v,i] and bi=[v,u,i]. Add fourteen internal edges. Ten form the two chains a1-a2-a3-a4-a5-a6 and b1-b2-b3-b4-b5-b6. Four cross links are a1-b3, a3-b1, a4-b6, a6-b4.

Only a1,a6,b1,b6 may connect outside the gadget. The other eight vertices need two internal edges in any final cycle. In particular a2,a5,b2,b5 already have degree two in the gadget, forcing their eight adjacent chain edges.

Vertex cover to Hamiltonian cycle (continued)



Eight blue edges are forced by degree 2 at a2,a5,b2,b5. Six gray edges remain undecided. Middle edges a3−a4 and b3−b4 force the four cross links. Orange marks the four ports.

Three legal gadget covers

The remaining choices are the middle edges a3-a4 and b3-b4. Once these are selected or omitted, degree two at internal vertices forces every cross link.

Middle edges selected	Cover of all twelve vertices
Both	Two disjoint six-vertex paths, a1 through a6 and b1 through b6
Only b3-b4	One path a1-a2-a3-b1-b2-b3-b4-b5-b6-a4-a5-a6
Only a3-a4	The symmetric full path from b1 to b6
Neither	Two closed six-vertex cycles with no way to join the external cycle, forbidden

These cases exhaust all internal degree configurations. A legal Hamiltonian cycle uses one of the first three. It cannot enter an a port and leave through a b external chain. It must cover every gadget vertex. Locally, at least one endpoint of the original edge must be selected.

Middle edges force external port use

Let x indicate selection of a3-a4 and y selection of b3-b4. Vertex a3 already uses forced edge a2-a3, so a3-b1 has indicator 1-x. Similarly a4-b6 has 1-x. Edges a1-b3 and a6-b4 each have 1-y.

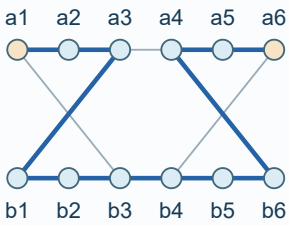
Thus a1,a6 have internal degree 2-y and external degree y. Ports b1,b6 have external degree x. Both ends on one original-vertex side connect externally together or neither does.

For x=y=0, the cycles are a1-a2-a3-b1-b2-b3-a1 and a4-a5-a6-b4-b5-b6-a4. Every vertex already has degree two, so neither cycle connects to a selector. They cannot form the whole graph's single Hamiltonian cycle. The three remaining full path covers are shown below.

Middle edges force external port use (continued)

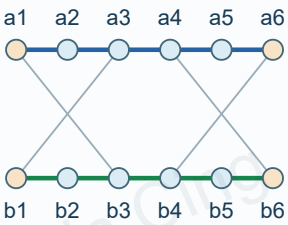
All three legal modes preserve the original-vertex side

u side only: 12 vertices



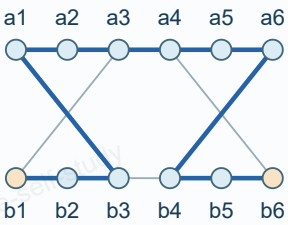
x=0, y=1

6 vertices on each side



x=1, y=1

v side only: 12 vertices



x=1, y=0

Gray edges are unused. Blue/green show paths. Each orange port needs one external edge for degree 2.

Incident chains and selectors

VERTEX-COVER asks for size at most k. Remove isolated vertices without changing the minimum cover. If k<0, or k=0 with an edge present, output a fixed no Hamiltonian-cycle instance. If no edges remain, or k is at least the remaining vertex count, output a fixed yes triangle. The main case has no isolated vertices and $1 \leq k < |V|$. A smaller cover can be enlarged to exactly k nonisolated vertices.

For each original u, order its neighbors u(1),...,u(d) arbitrarily. Connect its gadget sides by edges $[u, u(i), 6] - [u, u(i+1), 1]$, i=1,...,d-1. An external path selecting u must follow all of u's incident gadgets from one chain endpoint to the other.

Add k selectors s1,...,sk. Each selector connects to both ends of every u chain, with no other external edges. There are $12|E| + k$ vertices, $14|E|$ internal edges, at most $2|E|$ chain links, and $2k|V|$ selector links. The size is polynomial. Both source and target graphs are undirected.

Both directions of vertex cover to cycle

Given a cover $C = \{u_1, \ldots, u_k\}$, make one path through each selected vertex's incident chain. If only one endpoint of an original edge is selected, use that side's full twelve-vertex path. If both endpoints are selected, use the two disjoint six-vertex paths. Every edge has a selected endpoint, so every gadget vertex is visited exactly once. Join s1, the u1 chain, s2, the u2 chain, ..., sk, the uk chain, and back to s1. This visits every selector once and gives one Hamiltonian cycle.

Conversely, remove the k selectors from a Hamiltonian cycle. This leaves k disjoint paths. Local gadget modes forbid switching original-vertex identity, so each path follows one original u's entire incident chain. Two paths cannot select the same u, since they would share a chain endpoint or gadget vertex. Select these k original vertices. Every gadget is visited, and every legal mode uses at least one endpoint's chain. Hence every original edge is covered.

Therefore VC(G,k) holds iff the constructed graph has a Hamiltonian cycle. Construction, decoding, and size are polynomial. Fixed yes/no instances handle the preliminary cases without inventing empty chains for selectors.

A full twenty-five-vertex cycle

For original path u-v-w with k=1, choose cover {v}. Build U=W_uv and V=W_vw, each with twelve vertices, and one selector. The b side of U and a side of V represent v. Connect U.b6 to V.a1 and the selector to the two ends of the v chain.

Use these full paths, with every point prefixed by its gadget:

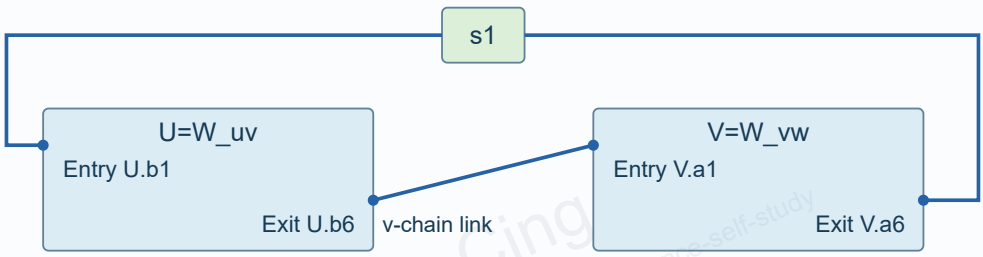
- U: b1,b2,b3,a1,a2,a3,a4,a5,a6,b4,b5,b6.
- V: a1,a2,a3,b1,b2,b3,b4,b5,b6,a4,a5,a6.

The complete cycle starts s1→U.b1, follows all of U, takes U.b6→V.a1, follows all of V, and returns V.a6→s1. Same-named vertices in different gadgets are distinct. Exactly 24 gadget vertices plus one selector are visited once. Every adjacency is an internal edge, chain link, or selector edge.

Both original edges have only v selected, so both gadgets use a full path on v's side. Choosing only u would leave W_vw uncovered and cannot create a cycle.

A full twenty-five-vertex cycle (continued)

Original path u−v−w, cover={v}: one selector closes a 25-vertex cycle



Each box contains the full 12-vertex path listed in the notes. U and V vertices are distinct.

This shows the selected HC. All external edges follow the selector and incident-chain rules.

Hamiltonian cycle to TSP decision

Given an undirected graph with $n \geq 3$, build complete graph K_n. Give original edges weight 1 and missing edges weight 2. Set threshold B=n. Construction takes $O(n^2)$.

A Hamiltonian cycle uses n original edges and becomes a tour of cost n. Conversely, every tour has n edges of weight at least 1. Total cost at most n forces every edge to have weight 1, so every tour edge belongs to the original graph. It is a Hamiltonian cycle there.

Thus HC(G) iff TSP(Kn,w,n). TSP decision is in NP. Its certificate is a vertex permutation, checked for uniqueness, closure, and total cost.

TSP trace and small graphs

For a triangle with edges 12,23,31, all three weights are 1. Tour 1→2→3→1 costs 3 and meets threshold B=3. Removing original edge 31 leaves that edge in the complete target graph with weight 2. Every three-point tour then costs 1+1+2=4>3, matching the absence of an original Hamiltonian cycle.

Simple undirected graphs with n<3 have no Hamiltonian cycle under this chapter's convention. Map them to a fixed no TSP instance. The n-edge argument above assumes n≥3.

Optimization TSP returns a minimum cost and, when requested, a tour. A polynomial exact optimizer could decide the threshold problem by comparison, so hardness of decision also makes exact optimization hard unless P=NP.

Decision, optimization, and search

Optimization counterparts include minimum-cost TSP, minimum SET-COVER, and maximum CLIQUE. An exact polynomial optimizer solves the decision version by comparing its value to the threshold. In the other direction, an exact decision oracle can find an integer optimum by binary search if upper and lower bounds have polynomial bit length and every threshold query is answered correctly.

Finding the optimum value does not itself output a witness. Reconstruction needs additional queries, for example fixing decisions step by step. A known value cannot be presented as an already constructed optimal solution.

Decision, optimization, and search (continued)

Classification	Meaning	Consequence	Does not imply
NP-hard language L	Every NP language reduces to L	If L is in P, P=NP	L is in NP
NP-complete language L	L is in NP and NP-hard	Short certificates and universal NP reductions	An optimization program is automatically a language
CLIQUE decision	Ask whether a k-clique exists	Membership and hardness proved here	Output of a maximum clique
MAX-CLIQUE optimization	Return maximum size, optionally a witness	An exact polynomial solver decides CLIQUE	Direct application of a language definition to an optimization output type

NP-hardness concerns reductions. NP-completeness also requires membership. Decision answers yes/no, optimization returns a value, and search returns a witness. Any claimed equivalence must explain how one output supplies the other's required answer.

Subset-sum membership and dynamic programming

Decision SUBSET-SUM asks whether some subset of given positive integers sums exactly to target t. A certificate lists selected indices. Check bounds, nonrepetition, and exact sum in polynomial time, proving membership in NP.

Let dp[i,s] mean that a subset of the first i integers reaches s. Then

$$dp[i, s] = dp[i - 1, s] \lor (a_i \leq s \land dp[i - 1, s - a_i]).$$

The $O(nt)$ algorithm is pseudopolynomial with binary t. The reduction below uses large values with polynomial bit length, consistently with that DP limitation.

Three-CNF-SAT to a digit table

Normalize to n appearing variables and k clauses, each with three distinct variables and no complementary tautology. Deduplication and padding above achieve this. Map constant true or false formulas separately to fixed yes/no integer instances.

Create n+k decimal columns, first for variables and then clauses. These columns compactly define integers. The final delivered encoding is binary.

- For each xi, create Ti and Fi. Both have 1 in variable column i and 0 in other variable columns. Ti has 1 in each clause containing xi; Fi has 1 in each clause containing ¬xi. Other clause digits are zero.
- For each clause j, add two slack numbers, with digits 1 and 2 respectively in that clause column and zero elsewhere.
- The target has 1 in each variable column and 4 in each clause column.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Three-CNF-SAT to a digit table (continued)

There are 2n+2k positive numbers, each with n+k decimal digits and $O(n+k)$ binary bits. Total output length is $O((n+k)^2)$, and construction is polynomial. If the SUBSET-SUM input format requires a set rather than a multiset, the nondegenerate premises ensure Ti and Fi differ: each variable appears, and positive/negative occurrences cannot coincide in a clause. Different variables have different high columns, and slack columns also distinguish the slack numbers.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

No carry for every candidate subset

An arbitrary subset sums each variable column to at most 2. A clause has three distinct literals, so all truth-value numbers together contribute at most three 1s in its column. Slack adds at most 1+2=3. Thus every clause sum is at most 6<10. **No subset can carry**, not only a correctly constructed witness.

If the formula is satisfiable, choose exactly Ti or Fi according to each variable's value. Variable columns become 1. Each clause has r true literals, r=1,2,3. Fill its deficit 4-r by slack 1+2, 2, or 1, respectively. All columns reach the target.

Conversely, any subset reaching the target has exactly one of Ti,Fi per variable, because no carries can alter its variable column. These choices define a consistent assignment. Each clause target is 4 but its slack contributes at most 3, so some chosen truth-value number contributes 1 there. The corresponding literal is true. Every clause is satisfied.

The two implications and polynomial size prove NP-hardness. Together with NP membership, SUBSET-SUM is NP-complete.

A hand-checkable no-carry sum

Use $F = (x_1 \vee x_2 \vee x_3) \wedge (\neg x_1 \vee x_2 \vee \neg x_3)$. Column order is x1,x2,x3,C1,C2.

Numbers	Decimal column representation
T1,F1	10010, 10001
T2,F2	01011, 01000
T3,F3	00110, 00101
S1,S1′	00010, 00020
S2,S2′	00001, 00002
Target	11144

Assignment (true,true,false) chooses T1+T2+F3=11122. Each clause has two true literals. Add S1′=20 and S2′=2 to reach 11144. No carries occur. Each high digit 1 confirms one truth-value number per variable.

A hand-checkable no-carry sum (continued)

Values can be of order 10^{n+k}, while binary length is only $O(n+k)$. The reduction has polynomial output even though exact $O(nt)$ DP is not polynomial in bit length.

T1+T2+F3=11122; add slack 00022 to reach target 11144

	x1	x2	x3	C1	C2	
T1	1	0	0	1	0	Each variable digit is 1
T2	0	1	0	1	1	Each clause has 2 true literals
F3	0	0	1	0	1	
slack	0	0	0	2	2	Slack 2 fills each to 4
target	1	1	1	4	4	

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Why universal no-carry matters

Proving no carry only for satisfying-assignment selections would not establish the reverse direction. A wrong subset might otherwise forge target digits through carries. Here every subset has variable digits at most 2 and clause sums at most 6, so no carry can occur for any candidate.

Converting an n+k digit decimal number to binary by repeated multiplication by 10 and digit addition has a simple $O((n+k)^2)$ bit implementation. Performing it for all 2n+2k numbers is still polynomial. Verification sums at most $O(n+k)$ numbers of $O(n+k)$ bits, adding only $O(\log(n+k))$ bits to the sum.

For assignment (false,false,false) in the example, the two clauses have 0 and 2 true literals. Even both slack numbers give only 3 in the first clause column, below 4. No carry from another column can conceal the failure.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Encoding invariance, closure, and coNP

If two encodings convert to each other in polynomial time, P membership is unchanged. Convert then decide; conversion bounds output length, and polynomial composition applies. Binary and unary integers lack this two-way guarantee. Writing 2^{b-1} in unary from b binary bits requires exponential output.

P is closed under complement, union, and intersection by negating or combining polynomial deciders. For concatenation L1L2, test all n+1 split positions with the two deciders. For L*, use reachable[0]=true and test substrings x[i+1..j] for i<j as possible next pieces. $O(n^2)$ polynomial tests plus DP decide reachable[n]; accept the empty string directly. Both operations preserve P.

Define $coNP = \{L : \overline{L} \in NP\}$. It describes short certificates for no instances, not languages outside NP. Neither P=NP nor NP=coNP is known. If an NP-complete C belongs to coNP, take any A in NP with $A \leq_p C$. The same map gives $\overline{A} \leq_p \overline{C}$. Compose the reduction with the NP verifier for $\overline{C}$ to obtain $\overline{A} \in NP$, so NP⊆coNP. Taking complements yields equality. This is a conditional implication, not a proved separation.

Reduction chains and hypotheses

The main chain is CIRCUIT-SAT ≤p SAT ≤p 3-CNF-SAT ≤p CLIQUE. Complementation connects CLIQUE, INDEPENDENT-SET, and VERTEX-COVER. Another chain is VERTEX-COVER ≤p HAMILTONIAN-CYCLE ≤p TSP decision. The no-carry construction gives 3-CNF-SAT ≤p SUBSET-SUM.

Each arrow retains its own premises. HC here is undirected. The TSP complete graph has polynomial size. Subset-sum uses binary target encoding. Optimization TSP and its decision language have different output specifications.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Proving a new NP-complete language

1. Specify the decision yes-set, instance parameters, and output.
2. Give a certificate, polynomial length bound, verifier, and acceptance conditions to prove NP membership.
3. Construct a reduction from a known NP-complete source to the new target.
4. Prove source yes implies target yes.
5. Prove target yes implies source yes.
6. Bound construction time, output size, and verification time by polynomials.
7. Handle empty graphs, k=0, repeated literals, disconnected graphs, and unreachable values as applicable.

A suggestive gadget without both implications is not a reduction proof. Converting the new problem back to a known hard problem also has the wrong direction for establishing hardness.

Source

Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 3rd ed., MIT Press, 2009, Chapter 34.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

References and attribution

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Approximation algorithms

Part VII Selected Topics

Approximation trades the cost of exact optimization for a proved bound. Covering, metric TSP, and subset-sum illustrate lower bounds, approximation errors, and bit costs. Constant ratios, logarithmic ratios, and FPTAS guarantees are distinct. Prerequisites are optimization, approximation ratios, and NP-hard problems.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Chapter map

The sections cover vertex cover, traveling salesperson, set cover, randomization and linear programming, and subset sum. The goals are to prove approximation ratios and distinguish randomized guarantees, LP rounding, PTAS, and FPTAS.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Objectives, feasibility, and denominators

Objectives are nonnegative. A minimization ρ-approximation returns a feasible solution on every input with $ALG \leq \rho OPT$. Maximization requires $ALG \geq OPT/\rho$, $\rho \geq 1$. When OPT=0, use the inequalities rather than 0/0. Minimization must return zero, while any nonnegative feasible maximization value satisfies its inequality.

A good result on one example is not an approximation guarantee. The bound must hold on every legal input, and time is measured in encoded input length. A minimization proof often finds $LB \leq OPT$ and proves $ALG \leq \rho LB$. Maximization can use $UB \geq OPT$ and prove $ALG \geq UB/\rho$.

A PTAS runs in polynomial input length N for every fixed $\epsilon > 0$. For example, $N^{1/\epsilon}$ can qualify. An FPTAS is polynomial jointly in N and the numerical value $1/\epsilon$, not merely its logarithm. Very small epsilon can still be costly.

Unweighted vertex cover

For a simple undirected graph, a cover touches every edge. Minimizing its size is NP-hard. This algorithm's guarantee is for unweighted size. With no edges, return the empty cover and OPT=0.

```
C=empty; matched[v]=false for every v
for each edge (u,v), in a fixed order:
    if not matched[u] and not matched[v]:
        add u,v to C
        matched[u]=matched[v]=true
return C
```

Equivalently, choose an uncovered edge, add both endpoints, and delete covered edges. Marking endpoints avoids repeatedly rebuilding the graph and takes $O(|V| + |E|)$ time and $O(|V|)$ workspace, excluding input storage.

A maximal matching is a lower bound

Let M be the selected edges. Marking both endpoints immediately ensures no selected edges share a vertex, so M is a matching. An unselected edge already had a marked endpoint when scanned, and is covered in the final C. Thus C is feasible.

The matching is maximal, meaning no edge can be added, but need not be maximum in cardinality. Every cover, including an optimum C*, must choose at least one endpoint for each M edge. Disjoint endpoints require distinct charged vertices, so

$$|C| = 2|M| \leq 2|C^*|.$$

The proof needs neither OPT nor a maximum matching. Arbitrary adjacent edges cannot be charged independently to obtain an |E| lower bound, since they may share one selected vertex. A matching prevents that double counting.

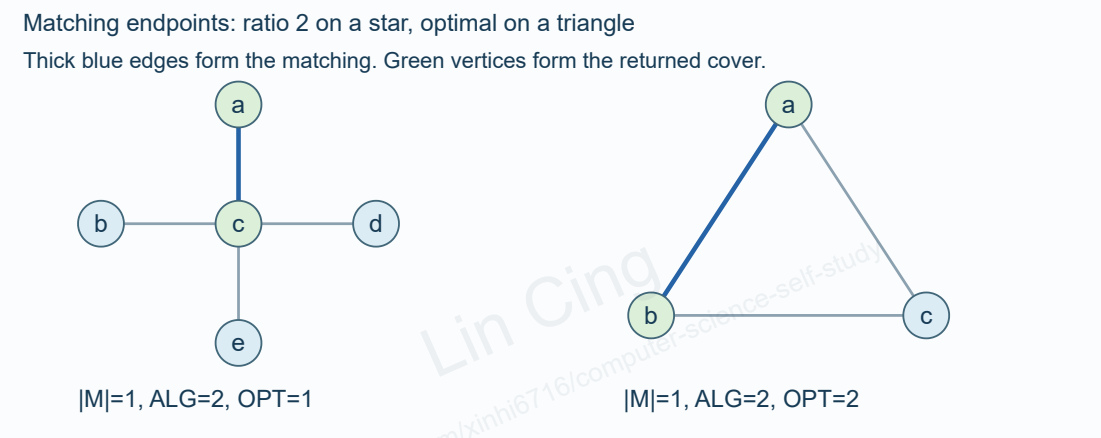
Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Ratio two and exact optimality

For a four-leaf star centered at c, choose edge (c,a). The output is {c,a}, while {c} is optimal, attaining ratio 2. For a triangle, choosing ab gives {a,b}. Every one-vertex choice misses its opposite edge, so OPT=2 and this output is optimal.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Ratio two and exact optimality (continued)



Weights invalidate this proof for weighted cost. A single edge with endpoint weights 1 and 100 gives algorithm cost 101 and optimum 1. Weighted vertex cover needs a weight-sensitive bound, such as LP rounding below.

Metric TSP and double tree

The main input is a complete undirected graph with n≥3, symmetric nonnegative costs, and triangle inequality $c(u, w) \leq c(u, v) + c(v, w)$. This is metric TSP. Distinct vertices may have zero distance, a pseudometric; the proofs still hold.

A tour visits every vertex once and returns to its start. Double tree computes an MST T, makes a full DFS walk from any root, takes vertices in first-visit order, and closes the resulting tour. Completeness ensures each shortcut edge exists. This construction cannot be applied directly to an arbitrary sparse graph.

Simple Prim on a full cost matrix takes $O(n^2)$, with linear DFS and output, for $O(n^2)$ word arithmetic operations and comparisons. Large rational weights require additional bit costs.

Each inequality in the factor-two proof

Deleting any edge from an optimal tour leaves a spanning tree. Nonnegativity means deletion does not increase cost, so MST minimality gives $c(T) \leq OPT$.

The full DFS walk W traverses each tree edge twice, giving $c(W) = 2c(T)$. It repeats vertices and is not yet a TSP tour.

Replace a walk section $u \to v_1 \to \cdots \to v_k \to w$, whose interior vertices were already visited, by $u \to w$. Repeated triangle inequality makes the shortcut no more expensive. Keeping first visits and final closure yields a feasible tour H with

$$c(H) \leq c(W) = 2c(T) \leq 2OPT.$$

The lower bound uses nonnegativity, shortcutting uses the metric inequality, and feasibility uses completeness. These assumptions have different roles.

Every walk edge and its actual cost

Use Manhattan distances for A(0,0),B(2,0),C(2,1),D(0,1). The coordinatewise absolute-value triangle inequalities imply the metric inequality.

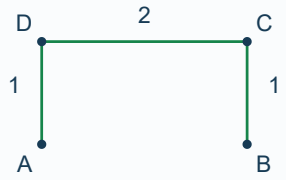
Take MST edges AD=1, DC=2, CB=1, totaling 4. The six complete-graph costs are 1,1,2,2,3,3. Every spanning tree uses three edges and therefore costs at least 1+1+2=4, proving this chain minimal.

Root at A. The full walk A,D,C,B,C,D,A costs 1+2+1+1+2+1=8. First-visit tour A,D,C,B,A costs 1+2+1+2=6. Any closed tour reaching both x-extremes has total x variation at least 4 and total y variation at least 2. Cost is at least 6, attained around the rectangle, so OPT=6. Walk cost 8 and tour cost 6 are different quantities.

Every walk edge and its actual cost (continued)

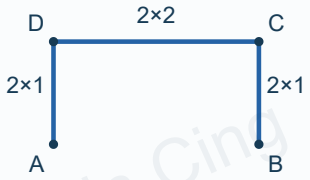
Same rectangle and Manhattan costs: MST 4 → walk 8 → tour 6

Minimum spanning tree



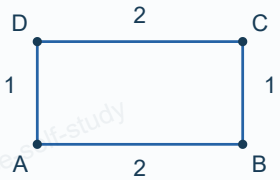
AD+DC+CB=1+2+1=4

Traverse each tree edge twice



A,D,C,B,C,D,A; cost 8

First visits, then return to A



A,D,C,B,A; cost 6

A(0,0), B(2,0), C(2,1), D(0,1). Axes have equal scale. Labels are Manhattan costs.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Christofides' three-halves guarantee

Under the same metric assumptions, let O be the odd-degree vertices of an MST. The degree sum is twice the edge count, so |O| is even. Find a minimum-weight perfect matching M on O and add it to T as a multigraph. Every odd degree becomes even, all other degrees remain even, and connectivity remains. An Euler closed walk exists; shortcut it to a tour.

Take an optimal tour and shortcut it to the vertices of O in visit order. Its closed cycle costs at most OPT. Alternating cycle edges form two perfect matchings. If |O|=2, view the cycle as two return edges. Their combined cost is at most OPT, so one costs at most OPT/2. Minimum matching therefore has $c(M) \leq OPT/2$. The final tour costs at most $c(T) + c(M) \leq 3OPT/2$.

This uses the subroutine theorem that general-graph minimum-weight perfect matching is polynomial-time solvable. An arbitrary greedy matching cannot replace the minimum matching in this proof. Euler traversal and shortcutting are polynomial too.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A constant ratio for nonmetric TSP would decide HC

Assume a deterministic polynomial-time algorithm A guarantees fixed ratio $\rho \geq 1$ on every nonnegative complete graph. Fix integer $R \geq \rho$. For an n-vertex HC input, give original edges weight 1 and nonedges weight $M = Rn + 1$.

With an HC, OPT=n, so A returns cost at most $\rho n \leq Rn$. Without an HC, every tour uses a nonedge, costing at least $M + n - 1 = Rn + n > Rn$. Comparing A's output to Rn decides HC in polynomial time, implying P=NP.

M has $O(\log n)$ binary bits, and the target graph has $O(n^2 \log n)$ bits. Large values conceal no superpolynomial output. These costs generally violate triangle inequality and do not contradict metric algorithms. The conclusion is conditional on P≠NP and applies to the stated deterministic guarantee. An expected-value randomized guarantee is not automatically a deterministic decider.

Weighted set cover

Let universe X have N elements and family $F = \{S_1, \ldots, S_M\}$, each a subset of X, with union X. Unweighted cover minimizes selected set count. Weighted cover has positive rational costs and minimizes total cost. Empty X returns the empty cover. If the union is insufficient, report infeasibility.

```
U=X; C=empty
while U is nonempty:
    choose S with |S intersect U|>0 minimizing w(S)/|S intersect U|
    C=C union {S}; U=U minus S
return C
```

For unit weights, this chooses maximum new coverage. Every round covers a new element, and previously chosen sets have zero new coverage. There are at most min(N,M) rounds. A direct incidence-matrix implementation computes counts in O(NM) per round and O(NM min(N,M)) operations overall, polynomial. Compare rational ratios exactly by cross multiplication.

Weighted set cover (continued)

Exact set cover is NP-hard. Map vertex cover to X=E, with one unit-cost set Sv of incident edges per vertex. At most k sets cover X iff at most k vertices cover the original edges. Incidence count is 2|E|, so size and construction are polynomial. Weighted set cover includes the unit-cost case. This does not claim every restricted family is hard.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Element charges yield a harmonic bound

When the greedy algorithm buys S of cost w covering k new elements, charge each new element w/k. Each element is charged exactly once, so total charge equals ALG.

Fix any candidate set A of size dA. Order its elements by first coverage as e1,...,edA, arbitrarily within one round. At the start of the round covering ej, A has at least dA-j+1 uncovered elements and is a valid alternative. Greedy's charge therefore satisfies

$$charge(e_j) \le \frac{w(A)}{d_A - j + 1}.$$

If other A elements were covered earlier in the same round, the number uncovered at the round's start is only larger, preserving the bound. Summing gives $\sum_{e \in A} charge(e) \le w(A) H_{d_A}$, where $H_d = \sum_{j=1}^{d} 1/j$ and H0=0.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Summing over an optimum cover

Let C* be optimal and d=max|Sj|. Every element belongs to at least one optimum set and has nonnegative charge. Thus

$$ALG = \sum_{e \in X} charge(e) \leq \sum_{A \in C^*} \sum_{e \in A} charge(e) \leq \sum_{A \in C^*} w(A) H_{|A|} \leq H_d OPT.$$

Repeated elements on the right increase the upper bound, so the inequality direction is correct. For j≥2, $1/j \leq \int_{j-1}^{j} dx/x$, proving $H_d \leq 1 + \ln d$.

Weighted and unweighted cases share the proof but use different selection rules. Choosing only maximum new count with arbitrary weights does not minimize charge per element and has no guarantee from this argument.

Three greedy sets versus two optimal sets

Let X={1,2,3,4,5,6}, S1={1,2,3}, S2={4,5,6}, S3={1,2,4,5}, all cost 1. Break the later tie in favor of S1.

Round	U before selection	New counts S1,S2,S3	Choice	New charges
1	1,2,3,4,5,6	3,3,4	S3	1,2,4,5 each pay 1/4
2	3,6	1,1,0	S1	3 pays 1
3	6	0,1,0	S2	6 pays 1

ALG=3. Sets S1,S2 cover X, and no single set does, so OPT=2 and ratio=3/2. Charges sum to 3. Each optimum set receives total 3/2, at most $H_3 = 11/6$, as the general proof predicts.

Three greedy sets versus two optimal sets (continued)

S3 covers four first, but greedy needs three sets; S1+S2 need two

	1	2	3	4	5	6	
S1	1	1	1	0	0	0	Initially: 1,2,3,4,5,6
S2	0	0	0	1	1	1	After S3: 3,6
S3	1	1	0	1	1	0	After S1: 6
							After S2: ∅

charge(1,2,4,5)=1/4; charge(3)=charge(6)=1

Total charge 3 equals greedy cost. Optimal S1∪S2={1,…,6}; no single set covers X.

Randomized maximum three-CNF satisfaction

MAX-3-CNF-SAT maximizes satisfied clauses among m clauses. Require three distinct variables per clause and no complementary pair. Assign variables independently and fairly. Each clause has probability 1/8 of being entirely false and 7/8 of being true.

Exact polynomial optimization would decide three-CNF satisfiability by checking OPT=m, so exact optimization is NP-hard. It still admits the following approximation. Let Ij indicate satisfaction and Y=sum Ij. Clauses may share variables, so their indicators need not be independent. Linearity of expectation gives

$$E[Y] = \sum_j E[I_j] = 7m/8 \geq 7OPT/8.$$

An expected-objective 8/7 approximation does not promise that every run reaches 7OPT/8 or that $E[OPT/Y] \leq 8/7$. A single clause gives Y=0 with probability 1/8, preventing a finite expected-ratio statement. Repeated literals such as (a∨a∨a) have only 1/2 satisfaction probability. A clause containing a literal and its negation is always true and can be counted separately.

From expectation to a success probability

For m=0, any assignment gives Y=OPT=0 and succeeds with probability 1. For m>0, let $p = \Pr[Y \geq 3m/4]$. Since 0≤Y≤m and E[Y]=7m/8,

$$7m/8 \leq (1-p)3m/4 + pm,$$

so p≥1/2. Independently rerun r times and retain the best assignment. Failure to reach 3m/4 has probability at most 2^{-r}. This is a high-probability 4/3 threshold guarantee. It is not a high-probability 7/8 threshold guarantee.

For a deterministic 7/8 objective bound, fix variables by conditional expectation instead.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Conditional expectation never decreases

Given a partial assignment, a clause with a true literal has conditional satisfaction probability 1. A fully assigned false clause has probability 0. Otherwise, with r unassigned distinct variables, it has probability $1 - 2^{-r}$. Sum these probabilities to compute the conditional expectation exactly.

For the next variable x, the current expectation is the average of its values after fixing x=0 and x=1. Choose the larger branch. After all variables are fixed, expectation equals actual satisfaction count, at least $7m/8 \geq 7OPT/8$.

For clauses (a∨b∨c), (¬a∨b∨c), (a∨¬b∨c), (a∨b∨¬c), initial expectation is 7/2. Branch a=0 gives 13/4 and a=1 gives 15/4, so choose 1. Then b=0 gives 7/2 and b=1 gives 4, so choose 1. Either c works; choose 0 and all four clauses hold.

Recomputing m clauses at each of n variables takes O(nm) fixed-precision fraction operations. Denominators here are at most 8, with no hidden exponential precision.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Weighted vertex-cover LP and threshold rounding

With positive rational weights, minimize $\sum_v w_v x_v$, subject to $x_u + x_v \geq 1$ on every edge and integral x in {0,1}. Relax to 0≤xv≤1. Every integral cover remains feasible, so $LP^* \leq OPT$.

Find an optimal feasible LP solution x* and choose C={v:xv*≥1/2}. If both endpoints of an edge were omitted, both values would be below 1/2, violating the edge constraint. Thus C is a cover. For every vertex, its selection indicator is at most 2xv*. Multiply by nonnegative weights and sum:

$$w(C) \leq 2 \sum_v w_v x_v^* = 2LP^* \leq 2OPT.$$

Rational LP has polynomial-bit-complexity algorithms. Frequent practical success of simplex does not prove its polynomial worst case. The proof uses an actually feasible LP solution. Numerical tolerances need their own feasibility and cost analysis. The number 0.4999 is not exactly 1/2.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

LP optimum differs from integer optimum

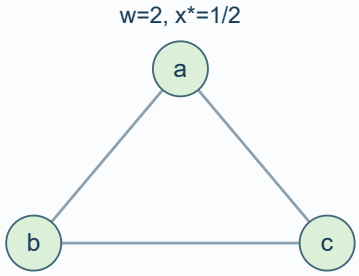
For a triangle with weights 2,3,4, the half-value solution costs 9/2. Multiply edge constraints ab,ac,bc by 1/2,3/2,5/2. Their left side becomes $2x_a + 3x_b + 4x_c$, with right side 9/2. Every LP solution therefore costs at least 9/2, proving the half solution optimal.

An integer cover needs at least two vertices. Their costs are 5,6,7, so OPT=5. Threshold rounding takes all three, giving ALG=9=2LP*≤2OPT=10. Its actual ratio is 9/5. The LP value 4.5 is not the original optimum.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

LP optimum differs from integer optimum (continued)

Weighted triangle: LP*=9/2, integer OPT=5, rounded cost=9



Edge multipliers: ab 1/2, ac 3/2, bc 5/2

Sum: 2x_a+3x_b+4x_c ≥9/2

Half values attain 9/2, proving LP optimality

Integer cover a,b costs 2+3=5

Threshold ≥1/2 selects all three: cost 9

Green vertices all pass the threshold. LP and integer optima are different.

Randomized set-cover rounding

For N≥2, use the LP constraints $\sum_{S \ni e} x_S \geq 1, x_S \geq 0$, with nonnegative costs. Set integer $k = 3\lceil \log_2 N \rceil \geq 3 \ln N$, and select each set independently with $p_S = \min(1, kx_S)$. Rational x gives rational probabilities, implementable by uniform integer sampling over their denominators. No irrational logarithmic probability is required.

If some set containing e has p=1, e is certainly covered. Otherwise, its omission probability is

$$\prod_{S \ni e} (1 - p_S) \leq e^{-\sum p_S} \leq e^{-k} \leq N^{-3}.$$

A union bound gives probability at most $N^{-2} \leq 1/4$ of any uncovered element. Expected cost is at most kLP*. By Markov, cost exceeds 2kLP* with probability at most 1/2. Adding the two failure probabilities gives at most 3/4, so each trial simultaneously covers X and meets the cost bound with probability at least 1/4. The ratio 2k is O(log N).

Randomized set-cover rounding (continued)

Check actual coverage and cost, and independently retry failures. Expected trials are at most four. This is an expected polynomial algorithm with a verifiable success condition, not a guaranteed single successful trial. For N=0 return empty. For N=1 choose the cheapest set containing its element. If LP*=0, nonnegative cost and zero expected cost imply cost zero with certainty. The cost check then automatically passes; only coverage needs retries, without dividing by zero in Markov's inequality.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Optimization subset sum

Given positive integers a1,...,an and integer t≥0, maximize a subset sum at most t. Items are distinguished by indices, even if values repeat. Remove ai>t first. For n=0 or t=0 return the empty subset with value zero.

The exact list algorithm starts L0={0}. At step i, merge and deduplicate Li-1 and Li-1+ai, then remove values exceeding t. Both lists are sorted, so two-pointer merging is linear without re-sorting.

Every subset of the first i items either excludes i or includes it, corresponding to those two lists. Positivity means an oversized sum can never decrease later, so removal is safe. Induction retains every feasible exact sum. There can be 2^i distinct sums, for example powers of two with a sufficiently large target. Linear merging per round does not make the overall algorithm polynomial.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

TRIM retains smaller, actual sums

For a sorted list and δ>0:

```
R=[]
for y in L, in increasing order:
    if R is empty or y>(1+delta)*R[-1]: append y to R
return R
```

If y is omitted, the retained last value z satisfies $z \leq y \leq (1+\delta)z$. If retained, choose z=y. Every candidate therefore has a representative with $y/(1+\delta) \leq z \leq y$. Zero is kept, and the first positive value cannot be removed by zero.

Retained values are actual integer subset sums. TRIM does not divide or round their numerical values. Smaller representatives preserve capacity feasibility when later items are added. A larger representative can exceed the target even when the original y remains feasible.

Merge, trim, and remove oversized values

For $0 < \epsilon < 1, n \geq 1$, set $\delta = \epsilon/(2n)$.

```
L=[0]
for i=1..n:
    B=merge_unique(L, [z+a[i] for z in L])
    L=trim(B,delta)
    remove all values >t from L
return max(L)
```

Every retained value is an actual sum from the first i items, so the returned sum is feasible and at most t. To return its subset, retain immutable parent pointers and whether the current item was used. When equal values merge, choose any valid provenance. Backtracking takes at most n steps, with strictly decreasing item indices and no item reuse.

Value-only computation can roll its lists. Retaining reconstruction nodes for all rounds instead needs O(nK) nodes, where K bounds one list. It cannot still be claimed to use only O(K) space.

Induction includes the selected-item branch

For every exact feasible sum y≤t from the first i items, prove there is $z \in L_i$ with $y/(1+\delta)^i \le z \le y$. At i=0, only y=z=0 exists.

If item i is excluded, induction gives a prior representative u with $y/(1+\delta)^{i-1} \le u \le y$. Candidate B contains u. TRIM supplies z≥u/(1+δ), yielding the bound. Also z≤y≤t, so final removal does not discard it.

If item i is included, write y=y′+ai. Positivity ensures y′≤t. Induction gives u between $y'/(1+\delta)^{i-1}$ and y′. Candidate B contains u+ai, and

$$u + a_i \ge y'/(1+\delta)^{i-1} + a_i \ge (y' + a_i)/(1+\delta)^{i-1}.$$

TRIM loses at most one additional factor 1+δ. Its representative is at most u+ai≤y≤t, so survives capacity removal. The second inequality explicitly uses ai≥0. With negative items, both early removal and this proof need revision.

Accumulated error stays within epsilon

Apply the invariant to OPT. The maximum retained value is at least its representative, so $OPT/z_{max} \le (1+\delta)^n$ when OPT>0. For OPT=0, use the approximation inequality directly.

Since $\ln(1+x) \le x$,

$$n \ln(1 + \epsilon/(2n)) \le \epsilon/2.$$

For 0<ε<1, $\ln(1+\epsilon) = \int_0^\epsilon (1+x)^{-1} dx \ge \epsilon/(1+\epsilon) \ge \epsilon/2$. Therefore $(1+\epsilon/(2n))^n \le 1+\epsilon$, giving

$$z_{max} \ge OPT/(1+\epsilon) \ge (1-\epsilon)OPT.$$

The proof controls multiplication of per-round errors. Setting every round's δ equal to global ε would allow factor $(1+\epsilon)^n$ and would not establish the same global guarantee.

Four rounds with integer lists

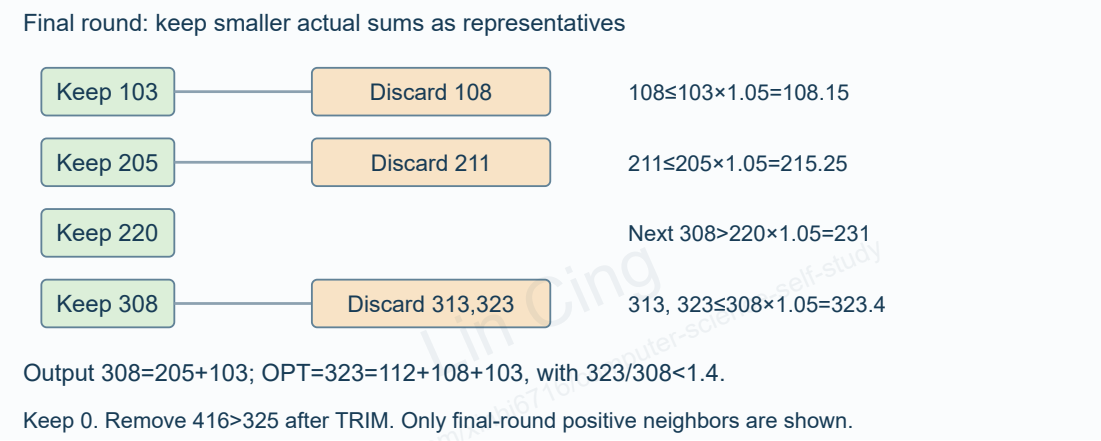
Use a=[112,108,205,103], t=325, ε=2/5, n=4, δ=1/20.

i	Merged unique B	After TRIM	After removing values above 325
1	0,112	0,112	0,112
2	0,108,112,220	0,108,220	0,108,220
3	0,108,205,220,313,425	Keep all	0,108,205,220,313
4	0,103,108,205,211,220,308,313,323,416	0,103,205,220,308,416	0,103,205,220,308

At round 2, 112≤108・1.05=113.4, so discard that candidate sum. At round 4, 108≤103・1.05=108.15, 211≤205・1.05=215.25, and both 313,323≤308・1.05=323.4. Each is represented by a smaller actual sum.

The output is 308=205+103. Exact OPT=323=112+108+103. The actual ratio is about 1.049≤1.4, and 308≤325. Discarding candidate sum 112 at one round does not delete that input item.

Four rounds with integer lists (continued)



List length, bit cost, and FPTAS

Adjacent positive trimmed values have ratio greater than 1+δ. The smallest is at least 1 and the largest at most t. There are at most $1 + \ln t / \ln(1+\delta)$ positive entries, plus zero. Since $\ln(1+\delta) \geq \delta/(1+\delta) \geq \delta/2$,

$$K = O(1 + n\log(t+1)/\epsilon).$$

Linear merging, trimming, and filtering over n rounds use $O(nK) = O(n + n^2\log(t+1)/\epsilon)$ arithmetic operations and comparisons. Values alone need O(K) workspace. The logarithm is input bit length, unlike the numerical t in exact DP's O(nt).

For exact rational $\epsilon = \alpha/\beta$, test trimming by $2n\beta y > (2n\beta + \alpha)last$, avoiding floating relative error. Reading input and removing ai>t costs O(N) bit work, with original bit length N. Later candidates are at most 2t. Intermediate bit lengths are polynomially controlled by log(t+1), log n, and the encoding of α,β. Including each large-integer multiplication still gives time polynomial in input length and 1/ε, completing the FPTAS proof.

Guarantees that must remain distinct

Method	Feasibility	Objective guarantee	Conditions
Matching endpoints for unweighted VC	Every run	ALG≤2OPT	Does not cover arbitrary weights
Double-tree TSP	Every run	ALG≤2OPT	Complete, nonnegative, symmetric metric
Weighted set-cover greedy	Every run	ALG≤Hd OPT	Choose cost per newly covered element
Random MAX-3-CNF	Every run returns an assignment	E[ALG]≥7OPT/8	Three distinct variables per clause
LP threshold weighted VC	Every run	ALG≤2OPT	Optimal feasible LP and nonnegative weights
Subset-sum TRIM	Every run	ALG≥OPT/(1+ε)	Nonnegative items and δ=ε/(2n)

Guarantees that must remain distinct (continued)

Expectation, deterministic inequalities, and probability-of-failure statements answer different questions. An approximation scheme must account for precision, list length, encoded input size, and reconstruction cost. Removing nearby values alone is not a running-time proof.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Source

Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 3rd ed., MIT Press, 2009, Chapter 35.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

References and attribution

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

String matching

Part VII Selected Topics

Matching reuses known equal prefixes and suffixes as state. We derive finite automata and KMP from character comparison and modular hashing, proving the failure chain, correctness, and total linear cost. Prerequisites are pattern prefixes, hashing, and finite automata.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Chapter map

The sections cover naive matching, Rabin-Karp, finite-automaton matching, and Knuth-Morris-Pratt. The goals are to define valid shifts and borders, compare these four methods, and prove the prefix function and KMP time bound.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Indices, shifts, and boundary cases

Text T has length n, pattern P has length m, and characters belong to finite alphabet Σ. Equality costs $O(1)$. Character indices start at 1, but shifts start at 0. A valid shift satisfies $0 \le s \le n - m$ and $P[j] = T[s + j]$ for **every** $j = 1, \ldots, m$. Output all shifts, including overlaps.

Write $P_k = P[1..k]$, with $P_0 = \epsilon$. A proper prefix or suffix cannot equal the whole string. A border is both a prefix and a suffix.

For $m = 0$, the specification outputs all $n + 1$ boundaries $0, \ldots, n$. For $m > n$, output nothing. Handle these before the main algorithms, which assume $1 \le m \le n$, so they never read a nonexistent $P[1]$.

Naive matching checks every shift

```
for s=0..n-m:
    j=1
    while j<=m and P[j]==T[s+j]: j++
    if j==m+1: output s
```

Before each inner comparison, $P[1..j-1] = T[s+1..s+j-1]$. Equality extends the invariant. A mismatch invalidates this shift. Reaching $j = m + 1$ means every character agrees. The outer loop enumerates exactly the possible shifts, proving both soundness and completeness.

There are $n - m + 1$ windows and at most m comparisons each. The worst case is $\Theta((n - m + 1)m)$, attained by $T = a^n, P = a^m$. Many true matches alone can force this cost. Extra workspace is $O(1)$, excluding output.

All windows of one text

For $T = \texttt{ababaca}$, $P = \texttt{aba}$, stop at the first mismatch.

Shift	Window	Comparisons in order	Count	Result
0	aba	a=a, b=b, a=a	3	Output 0
1	bab	a≠b	1	Reject
2	aba	a=a, b=b, a=a	3	Output 2
3	bac	a≠b	1	Reject
4	aca	a=a, b≠c	2	Reject

There are ten comparisons. Matches at shifts 0 and 2 share text position 3 and are both valid. Naive matching rechecks alignments. KMP uses known equal sections to rule out impossible shifts.

Rabin-Karp rolling identity

Encode characters injectively as $0, \ldots, d-1$. For a length-m string, $H(X) = \sum_{j=1}^{m} X[j] d^{m-j}$. Arithmetic on an arbitrary m-digit integer is not constant time. Store $h_q(X) = H(X) \bmod q$, $q \geq 2$, and verify characters whenever hashes agree.

Let $p = h_q(P), t_s = h_q(T[s+1..s+m]), h = d^{m-1} \bmod q$. Removing the highest digit, shifting, and adding a digit gives

$$t_{s+1} = \big(d(t_s - T[s+1]h) + T[s+m+1]\big) \bmod q.$$

The identity holds for every $q \geq 2$. Primality helps some randomized analyses but does not by itself give collision probability $1/q$ for fixed strings. Normalize negative remainders to $[0, q-1]$ and prevent overflow in intermediate products.

Hashes reject; characters accept

```
h=1; p=0; t=0
for j=1..m-1: h=d*h mod q
for j=1..m:
    p=(d*p+P[j]) mod q
    t=(d*t+T[j]) mod q
for s=0..n-m:
    if p==t and P[1..m]==T[s+1..s+m]: output s
    if s<n-m:
        t=(d*(t-T[s+1]*h)+T[s+m+1]) mod q
```

Horner evaluation and the rolling identity inductively maintain the current window hash. Equal strings have equal hashes, so no match is lost. Different hashes reject. Equal hashes trigger exact comparison, preventing false reports. Collisions affect running time rather than correctness in this verified version. Omitting verification creates a different algorithm that can report false matches.

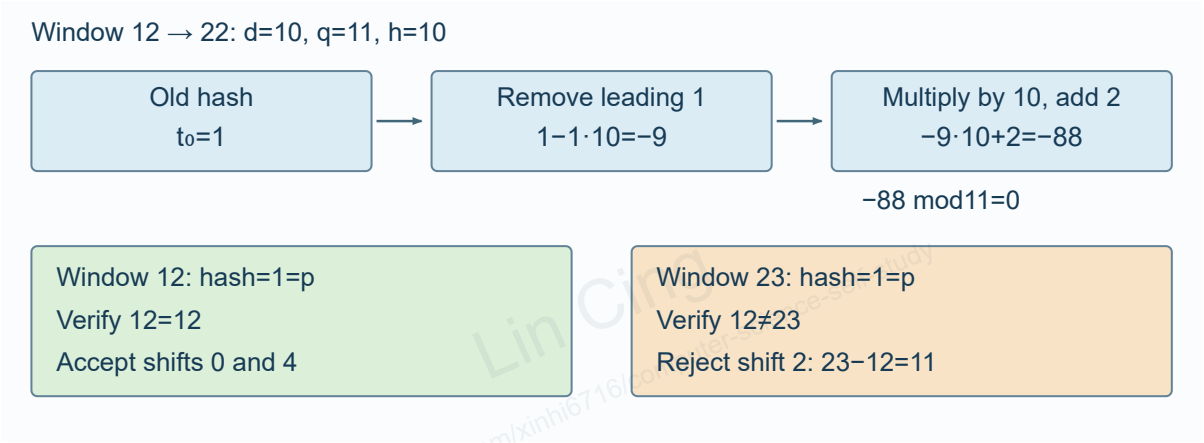
Match, collision, match

Take $T = 122312, P = 12, d = 10, q = 11$. Then $h = 10, p = 1$.

Shift	Window	Hash	Next hash calculation	Verification
0	12	1	$(10(1 - 1 \cdot 10) + 2) \bmod 11 = 0$	Match, output 0
1	22	0	$(10(0 - 2 \cdot 10) + 3) \bmod 11 = 1$	Different hash, skip
2	23	1	$(10(1 - 2 \cdot 10) + 1) \bmod 11 = 9$	$12 \neq 23$, reject collision
3	31	9	$(10(9 - 3 \cdot 10) + 2) \bmod 11 = 1$	Different hash, skip
4	12	1	Final window, read no new character	Match, output 4

A spurious hit has equal hashes but different strings. Here $23 - 12 = 11$ vanishes modulo the prime modulus. Prime moduli still permit collisions.

Match, collision, match (continued)



True and spurious hits both cost time

In the word-RAM model, characters, q, and modular products fit in constant-cost word operations, or constant-cost modular multiplication is available. The intermediate $d(t - \text{out} \cdot h)$ must also fit. Normalizing the difference before multiplication keeps that product below dq. Arbitrary-precision implementations must include bit costs.

Preprocessing takes $\Theta(m)$, and rolling takes $\Theta(n - m + 1)$. With z true matches and f spurious hits, verification takes at most m comparisons per hit, giving $O(n + m + m(z + f))$.

Either z or f can be $\Theta(n - m + 1)$, so worst-case time remains $\Theta((n - m + 1)m)$. All-a inputs have only true hits and still repeat full verification. Output alone costs $\Omega(z)$, while this version spends another m comparisons per true hit.

A proved random-base hash bound

Fix prime $q > d - 1$, so distinct character codes remain distinct in $\mathbb{F}_q$. **After fixing the input**, choose a uniform base $b \in \mathbb{F}_q$, independently of the input. Hash with $\sum_j X[j] b^{m-j} \bmod q$. Replace d with b in the rolling identity. The identity also handles $b = 0$, with $h = b^0 = 1$ when $m = 1$.

For fixed distinct strings X, Y, their difference polynomial is nonzero and has degree at most $m - 1$. A nonzero field polynomial has at most its degree many roots. At a root, factor out its linear factor and induct on degree. Thus

$$\Pr[h_b(X) = h_b(Y)] \le \min(1, (m-1)/q).$$

Sum the expectations of collision indicators over nonmatching windows. Independence between windows is unnecessary, and $E[f] \le (n - m + 1 - z)(m - 1)/q$. A loose expected-time bound is $O(n + m + mz + mnm/q)$. A large enough modulus reduces spurious-hit cost but not the mz cost of true matches. Its bit length and modular arithmetic costs must still be counted.

Finite-automaton state

A finite automaton consists of $(Q, q_0, A, \Sigma, \delta)$, a finite state set, initial state, accepting set, alphabet, and transition function for every state-character pair. For matching use $Q = \{0, \dots, m\}$, $q_0 = 0$, $A = \{m\}$.

Define $\sigma(x) = \max\{k \le m : P_k \text{ is a suffix of } x\}$. The empty string is always a suffix, so the maximum exists. The state $q = \sigma(x)$ is the longest pattern prefix currently available for extension, rather than the number of completed matches.

Set $\delta(q, a) = \sigma(P_q a)$. Read one character and take one transition. State m reports the shift ending at that position. Accepting states still have outgoing transitions, allowing overlapping matches.

Why the longest prefix is sufficient

We prove $\delta(\sigma(x), a) = \sigma(xa)$. Let $q = \sigma(x), r = \sigma(xa)$. If $r > 0$, remove the last character from its match. Then P_{r-1} is a suffix of x, so $r - 1 \leq q$. Since both are suffixes of x, the shorter P_{r-1} is a suffix of P_q. Appending a makes P_r a suffix of $P_q a$, hence $r \leq \sigma(P_q a)$. This inequality also holds for $r = 0$.

Conversely, $P_q a$ is a suffix of xa. Every suffix of $P_q a$ that is a pattern prefix is therefore a suffix of xa. Thus $\sigma(P_q a) \leq \sigma(xa) = r$, proving equality.

Starting with $\sigma(\epsilon) = 0$, induction gives $q = \sigma(T[1..i])$ after each transition. State m holds exactly when P is a suffix of that text prefix, equivalent to a match at shift $i - m$. This proves both directions of acceptance.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Complete transition table and overlap

For $P = \texttt{abab}, \Sigma = \{a, b, c\}$:

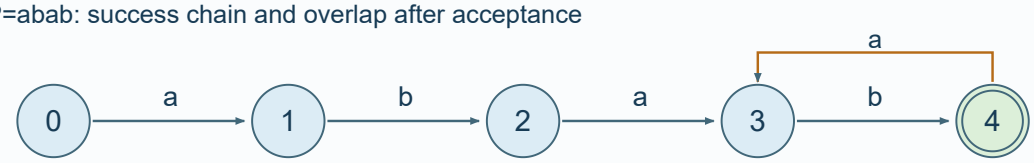
q	a	b	c
0	1	0	0
1	1	2	0
2	3	0	0
3	1	4	0
4	3	0	0

Scanning ababab gives states 1,2,3,4,3,4, reporting shifts 0 and 2 at positions 4 and 6. The transition $\delta(4, a) = 3$ uses suffix aba of ababa. Resetting to zero after a match would lose the second occurrence.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Complete transition table and overlap (continued)

P=abab: success chain and overlap after acceptance



The full transition table is in the notes. Only the success chain and δ(4,a)=3 are shown.

T[i]	a	b	a	b	a	b
State	1	2	3	4	3	4

Accept at positions 4 and 6: shifts 0 and 2. Do not reset after a match.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Table construction versus scanning

A direct construction tries candidate suffix lengths from $\min(m, q+1)$ downward for each q, a, comparing characters. It can take $O(m^2)$ per cell and $O(m^3|\Sigma|)$ overall. It is correct but slow. Scanning is still $\Theta(n)$.

Using the prefix function π below, construct in increasing q:

```
for a in Sigma: delta(0,a) = (1 if a==P[1] else 0)
for q=1..m:
    for a in Sigma:
        if q<m and a==P[q+1]: delta(q,a)=q+1
        else: delta(q,a)=delta(pi[q],a)
```

After mismatch, the next candidate is a proper border of P_q. The longest has length $\pi[q]$, and all shorter candidates are suffixes of it. Reuse its transition on the same character. Since $\pi[q] < q$, dependencies are already computed. Computing π takes $\Theta(m)$. The table takes $\Theta(m|\Sigma|)$ time and space. Constant-time character lookup assumes constant-time alphabet indexing.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

The prefix function

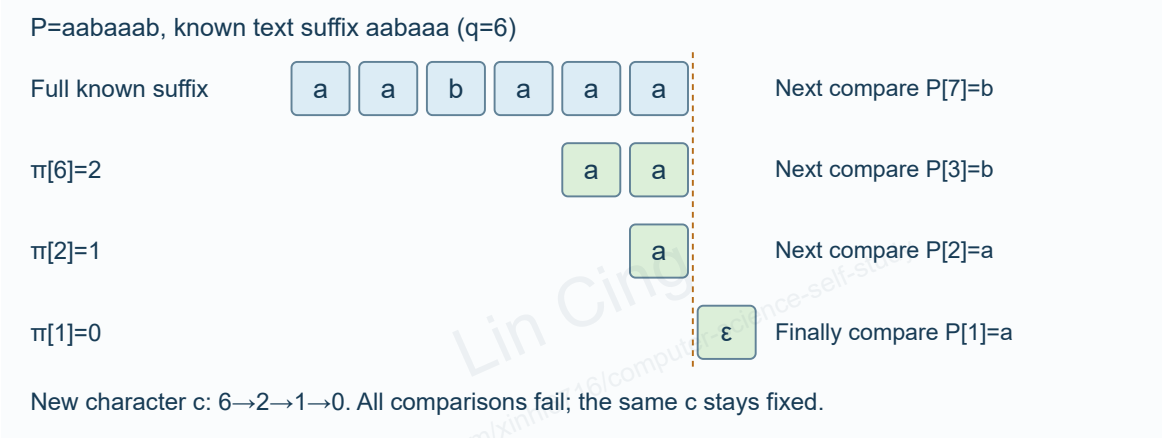
Define $\pi[q] = \max\{k < q : P_k \text{ is a suffix of } P_q\}$, for $q = 1, \ldots, m$. Thus $\pi[1] = 0$ and $\pi[q] < q$. It depends only on the pattern.

For aabaaab, $\pi = [0, 1, 0, 1, 2, 2, 3]$. The proper borders of $P_6 =$ **aabaaa** are aa, a, and the empty string. The chain is $6 \to 2 \to 1 \to 0$. Length 6 is the starting state, not a proper border.

KMP stores no full transition table. On a mismatch, follow π to a shorter candidate while keeping the new text character fixed. Previously read text need not be read again.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

The prefix function (continued)



Lin Cing
https://github.com/xinhi6716/computer-science-self-study

The failure chain contains every candidate

Let $B(q)$ be all proper border lengths of P_q, and let $b = \pi[q] > 0$. For every $k < b$,

$$k \in B(q) \quad \iff \quad k \in B(b).$$

Since P_b is a suffix of P_q, their length-k suffixes coincide. Their length-k prefixes also coincide with P_k. Thus the prefix-suffix equality is identical in both directions. No border has $b < k < q$, by maximality. If $b = 0$, no positive proper border exists.

Hence $B(q) = \{b\} \cup B(b)$. Induction on strictly decreasing lengths proves that $\pi[q], \pi[\pi[q]], \ldots, 0$ lists every proper border. This completeness justifies fallback.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Construct the prefix function

```
pi[1]=0; k=0
for i=2..m:
    while k>0 and P[k+1]!=P[i]: k=pi[k]
    if P[k+1]==P[i]: k++
    pi[i]=k
```

Before processing i, $k = \pi[i-1]$. If P_i has a positive proper border of length r, deleting its last character leaves a proper border of P_{i-1} of length $r-1$. Conversely, a border of that length extends when $P[r] = P[i]$.

Start at the largest candidate $\pi[i-1]$ and follow the complete decreasing failure chain. The first extendible candidate gives the largest legal r. At zero, unequal characters give answer zero. Only smaller, already-correct prefix values are used, proving induction. Since $\pi[i-1] \le i-2$, the entire P_i cannot be mistaken for its own proper border.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Every prefix-table step

For `aabaaab`:

i	Character	Initial k	Comparison and fallback	pi[i]
1	a	None	A single character has no nonempty proper border	0
2	a	0	P[1]=a, extend	1
3	b	1	P[2]=a≠b; 1→0; P[1] also differs	0
4	a	0	P[1]=a	1
5	a	1	P[2]=a	2
6	a	2	P[3]=b≠a; 2→1; P[2]=a	2
7	b	2	P[3]=b	3

The value $\pi[6] = 2$, not 3, because suffix `aaa` differs from prefix `aab`. Fallback keeps known possible borders rather than always subtracting one.

KMP scan and the state after a match

```
pi=prefix_function(P); q=0
for i=1..n:
    while q>0 and P[q+1]!=T[i]: q=pi[q]
    if P[q+1]==T[i]: q++
    if q==m:
        output i-m
        q=pi[m]
```

At the acceptance check, $q = \sigma(T[1..i])$, as in the automaton. If $q < m$, retain it. At $q = m$, output first, then retain $\pi[m]$, the longest proper pattern prefix that is a suffix of the read text. A new match only needs a proper border, since a full length-m prefix cannot extend to length $m + 1$.

Every shorter candidate after mismatch lies on the failure chain. The first that extends with $T[i]$ gives exactly the automaton transition. If none extends, use zero. This proves completeness and ensures $q < m$ before accessing $P[q + 1]$.

Overlap after a full match

For $P = \texttt{aabaaab}$, $\pi[7] = 3$. Scan $T = \texttt{aabaaabaaab}$.

i	Character	q before comparison	q after extension	q retained after output
1	a	0	1	1
2	a	1	2	2
3	b	2	3	3
4	a	3	4	4
5	a	4	5	5
6	a	5	6	6
7	b	6	7, output 0	3
8	a	3	4	4
9	a	4	5	5

Overlap after a full match (continued)

i	Character	q before comparison	q after extension	q retained after output
10	a	5	6	6
11	b	6	7, output 4	3

The matches share `aab` at positions 5 through 7. Resetting to zero discards that known equality.

Many comparisons for one character, linear total

From state 6 on new character `c`, compare against $P[7] = b, P[3] = b, P[2] = a, P[1] = a$. All differ. Follow $6 \to 2 \to 1 \to 0$ and retain zero. The same `c` is compared repeatedly, but the text index never retreats.

During the scan, q starts at zero and increases by at most one per character, for total increase at most n. Every while-body iteration decreases it by at least one. Post-match fallback also only decreases it, and q stays nonnegative. Thus there are at most n while iterations, plus constant work per character and output. Scanning takes $\Theta(n)$. The same accounting gives at most $m - 1$ increases and while iterations in prefix construction, hence $\Theta(m)$.

Total time is $\Theta(n + m)$, with $\Theta(m)$ workspace excluding output. This is a sequence bound, not a worst-case constant bound for each character. A full automaton table provides that per-character bound at a higher space cost.

Assumptions of the four methods

Method	Preprocessing	Worst-case scan	Pattern workspace
Naive	None	$\Theta((n - m + 1)m)$	$O(1)$
Verified Rabin-Karp	$\Theta(m)$	$\Theta((n - m + 1)m)$	$O(1)$
Full automaton using pi	$\Theta(m\|\Sigma\|)$	$\Theta(n)$	$\Theta(m\|\Sigma\|)$
KMP	$\Theta(m)$	$\Theta(n)$	$\Theta(m)$

The table assumes $1 \leq m \leq n$, constant-cost characters, and word modular arithmetic. Preprocessing is separate. Fast expected Rabin-Karp requires explicit randomization and collision conditions. KMP requires neither random input nor a random pattern. Unicode implementations must choose bytes, code points, or grapheme clusters as the matching units. Different encoded character sequences are not automatically identical strings.

Source

Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 3rd ed., MIT Press, 2009, Chapter 32.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

References and attribution

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Matrix operations

Part VII Selected Topics

Scope

Elimination must preserve permutations and earlier multipliers together. Complete LUP leads to inversion, Schur complements, and multiplication reductions. SPD matrices and orthogonal residuals connect Cholesky to least squares while separating exact arithmetic, rank, and numerical sensitivity.

Prerequisites

Linear algebra, Gaussian elimination, and least squares.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Chapter route

Learning objectives

- Solve systems by LUP and explain permutations.
- Analyze inversion through blocks and LU.
- Handle SPD matrices and least-squares normal equations.
- 28.1 Solving systems of linear equations
- 28.2 Inverting matrices
- 28.3 Symmetric positive-definite matrices and least-squares approximation

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

28.1 Solving systems of linear equations

First use real exact arithmetic. A nonsingular $n \times n$ matrix A has a unique solution to $Ax=b$ for every b. A singular A can give no solution or infinitely many, so an elimination zero does not imply every b is inconsistent. Nonsingularity does not guarantee elimination without row swaps: A=[0 1;1 0] is invertible but its first diagonal pivot is 0.

The LUP contract is **PA=LU**. Matrix P permutes rows, L is unit lower triangular, and U upper triangular with nonzero diagonal. Array p means current row i came from original row p[i], so(Pb)[i]=b[p[i]]. This permutes equations, not columns or unknown x.

From PAx=Pb and PA=LU, solve Ly=Pb then Ux=y. Substitution gives $Ax=P^{-1}Ly=P^{-1}Pb=b$. Nonzero triangular diagonals and invertible P establish uniqueness. Only P=I permits A=LU; assuming that equality while replacing b by Pb without explanation is inconsistent.

Forward and backward substitution dependencies

For unit lower L, process increasing i with $y[i]=b[p[i]]-\Sigma_{j<i}L[i, j]y[j]$. All required earlier y[j] are known, and each step satisfies the first i equations of Ly=Pb.

Process decreasing i with $x[i]=(y[i]-\Sigma_{j>i}U[i, j]x[j])/U[i, i]$. Later unknowns have already been solved, and the last n−i+1 equations of Ux=y now hold. Nonzero diagonals justify division. These invariants complete every row rather than guessing all unknowns simultaneously.

```
Solve(L, U, p, b):
  for i=1..n:
    y[i]=b[p[i]]-sum(j=1..i-1, L[i, j]*y[j])
  for i=n..1:
    x[i]=(y[i]-sum(j=i+1..n, U[i, j]*x[j]))/U[i, i]
  return x
```

Each substitution has n(n−1)/2 summation products; with additions and n divisions, total time is $\Theta(n^2)$, extra vector space O(n). Reuse one factorization for r right sides in $\Theta(n^3+rn^2)$. Refactoring the same A for every b discards reusable work.

Elimination, Schur complements, and existence

For nonzero first pivot a, write the active matrix$[a\ w^T; v\ B]$. Elimination uses

$[a\ w^T; v\ B]=[1\ 0; v/a\ I]\cdot[a\ w^T;0\ B-vw^T/a]$.

The lower-right $S=B-vw^T/a$ is the Schur complement. The left factor is invertible, so a nonsingular original gives a nonsingular right factor. If nonzero z satisfied $Sz=0$, vector$(-w^Tz/a, z)$ would lie in that factor's nullspace. Thus S is nonsingular and recursion can continue.

An all-zero pivot column makes the active S singular. Together with previously nonzero pivots, this implies original A singular. Conversely, a nonsingular active S has a nonzero entry in its first column. Swap that entry to the pivot and continue to completion. Hence **every nonsingular A has LUP**, without proving existence of unpermuted LU for every A.

In-place LUP swaps earlier multipliers too

Store U on and above the diagonal of work, and L's multipliers below it; unit L diagonal is implicit. Swap whole rows, including columns 1..k−1 holding previous multipliers.

```
work=copy(A); p=[1,.., n]
for k=1..n:
  r=argmax(i=k..n, abs(work[i, k]))
  if work[r, k]==0: report singular
  swap(work[k,1..n], work[r,1..n]); swap(p[k], p[r])
  for i=k+1..n:
    work[i, k]=work[i, k]/work[k, k]
    for j=k+1..n:
      work[i, j]=work[i, j]-work[i, k]*work[k, j]
```

Before step k, the block invariant is $PA=[L_{11}\ 0; L_{21}\ I]\cdot[U_{11}\ U_{12};0\ S]$. An active-row permutation R changes S to RS and must also change L_{21} to RL_{21}. Otherwise the lower product rows refer to wrong original equations. New elimination multipliers then extend L, preserving PA=LU.

There are $\Sigma_k(n-k)^2=\Theta(n^3)$ trailing-block updates and $\Theta(n^2)$ pivot-search work. These count arithmetic operations, not bit costs for arbitrarily long rationals.

Complete example with two row swaps

Take A=[0 2 1;1 1 0;2 3 4], b=(0,0,7)^T. The largest first-column magnitude is 2. Swap rows 1,3, so p=(3,2,1). Multipliers $m_{21}=1/2$, $m_{31}=0$ leave trailing block[−1/2 −2;2 1].

The second column again selects magnitude 2. Swap current rows 2,3, so p=(3,1,2). **Earlier L multiplier 1/2 moves from row 2 to row 3.** New $m_{32}=(-1/2)/2=-1/4$ gives final pivot$-2-(-1/4)\cdot 1=-7/4$.

Thus L=[1 0 0;0 1 0;1/2 −1/4 1], U=[2 3 4;0 2 1;0 0 −7/4].

Multiplying gives LU=[2 3 4;0 2 1;1 1 0], exactly PA in original-row order 3,1,2. Swapping only the trailing block leaves an erroneous row 2 multiplier 1/2 and breaks LU=PA.

Complete example with two row swaps (continued)

Two pivots: swap trailing blocks and saved L together

Green: saved L multipliers. Blue: U or the active trailing block. Unit L diagonal is implicit.

Original A

0	2	1
1	1	0
2	3	4

p=(1,2,3)

→ Swap rows 1/3, then eliminate

2	3	4
1/2	-1/2	-2
0	2	1

p=(3,2,1)

→ Swap rows 2/3, then eliminate

2	3	4
0	2	1
1/2	-1/4	-7/4

p=(3,1,2)

Swap two moves first-column 1/2 from current row 2 to row 3.

Final m31=1/2, m32=−1/4, U33=−7/4, satisfying PA=LU.

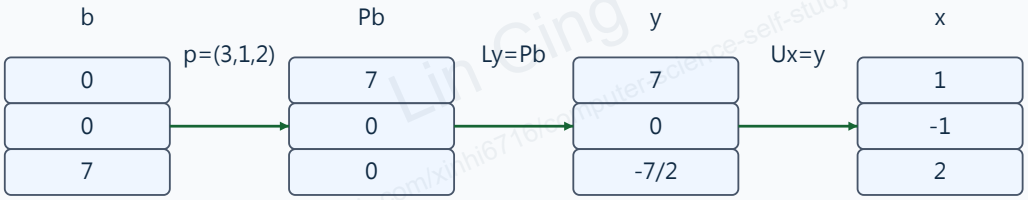
Substitution on the example and numerical boundaries

Here $Pb=(7,0,0)^T$. Forward substitution gives $y_1=7$, $y_2=0$, $y_3=0-(1/2)\cdot 7-(-1/4)\cdot 0=-7/2$.

Backward substitution gives $x_3=(-7/2)/(-7/4)=2$, $x_2=(0-1\cdot 2)/2=-1$, $x_1=(7-3\cdot(-1)-4\cdot 2)/2=1$. Original row checks are $0-2+2=0$,$1-1=0$,$2-3+8=7$.

From b to Pb, y, x: two triangular systems

Same A, b. Row permutation reorders right-side equations, not unknown x.



Forward: $y3=0-(1/2)\cdot 7=-7/2$.

Backward: $x3=2$, $x2=-1$, $x1=1$. Original A gives $(0,0,7)$.

Substitution on the example and numerical boundaries (continued)

Partial pivoting selects the largest absolute entry only in the current column. Exact arithmetic guarantees $|m_{ik}|\leq 1$ for this step, limiting column multipliers. It **does not guarantee small floating-point error for every matrix**. Element growth can occur, and ill-conditioning can amplify small input perturbations. Mathematical nonzero pivots differ from implementation tolerance thresholds; near-singularity decisions need scale and error assumptions.

28.2 Inverting matrices

Column j of A^{-1} solves $Ax=e_j$. One LUP plus n substitutions costs $\Theta(n^3)+n\Theta(n^2)=\Theta(n^3)$. Check $AA^{-1}=I$; for square matrices in exact arithmetic, a right inverse is the unique inverse. Solving one $Ax=b$ normally uses substitution directly rather than computing an inverse first.

For A=[1 1;2 5], L=[1 0;2 1], U=[1 1;0 3]. For e_1, forward y=(1,−2), then $x_2=-2/3$, $x_1=5/3$. For e_2, y=(0,1), then $x_2=1/3$, $x_1=-1/3$. Thus A^{-1}=[5/3 −1/3;−2/3 1/3], not entrywise reciprocals.

Multiplication checks 5/3−2/3=1,−1/3+1/3=0,10/3−10/3=0,−2/3+5/3=1. A singular A can still solve selected right sides, but not all unit vectors uniquely, and has no inverse under this definition.

Block-inverse order and assumptions

Write A=[B C; D E], require invertible B, and define $S=E-DB^{-1}C$. Factor A=[I 0; DB^{-1} I] · [B C;0 S]. If A is invertible, so is S. Invert the factors in **reverse order**:

```
A⁻¹ = [ B⁻¹+B⁻¹CS⁻¹DB⁻¹    −B⁻¹CS⁻¹ ]
      [ −S⁻¹DB⁻¹              S⁻¹     ]
```

Matrix factors do not commute. Do not exchange C, D, or inverse positions. In the scalar example B=1, C=1, D=2, E=5, S=3; the top-left term 1+1 · 1 · (1/3) · 2 · 1=5/3 matches columnwise inversion.

This block recursion does not directly cover every invertible A. Matrix[0 1;1 0] is invertible but leading B=0 cannot be inverted. SPD properties below make leading blocks and Schur complements invertible. General matrices require permutations or the following theoretical reduction.

Block-inverse order and assumptions (continued)

Schur complement: the exact block after elimination

Left-multiply by $[I\ 0;-DB^{-1}\ I]$. Matrix B must be invertible; factor order matters.

A

B	C
D	E

After elimination

B	C
0	$E-DB^{-1}C$

$S=E-DB^{-1}C$; $A=[I\ 0;\ DB^{-1}\ I]\cdot[B\ C;0\ S]$.

Example $B=1, C=1, D=2, E=5$: $S=3$, $A^{-1}=[5/3\ -1/3;-2/3\ 1/3]$.

For $B=0$, B^{-1} is undefined. Even invertible A can have singular B.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Reductions between multiplication and inversion

For $n\times n$ B, C, construct $3n\times 3n$ $T=[I\ B\ 0;0\ I\ C;0\ 0\ I]$. Its inverse is$[I\ -B\ BC;0\ I\ -C;0\ 0\ I]$; multiplication gives upper-right $BC-BC=0$. If inversion cost $I(n)=\Omega(n^2)$ and $I(3n)=O(I(n)),\Theta(n^2)$ construction plus one inverse extracts BC in $O(I(n))$. Dimension and regularity assumptions are essential.

Conversely, first invert SPD blocks. Leading B and Schur S are SPD, as proved next, so invert two half-size blocks and perform a constant number of products:$T(n)\leq 2T(n/2)+O(M(n))$. Assume $M(n)=\Omega(n^2)$, padding below 2n changes only constants, and $M(n/2)\leq cM(n)$ for $c<1/2$. Recurrence levelℓ costs at most$(2c)^\ell O(M(n))$; the geometric sum gives $T(n)=O(M(n))$. Powers $M(n)=\Theta(n^\omega),\omega\geq 2$ satisfy this.

For general real nonsingular A, Gram $G=A^TA$ is SPD because $z^TGz=||Az||^2>0$. Formula $A^{-1}=G^{-1}A^T$ follows by multiplying it by A to get I, and retains $O(M(n))$ cost. Pad nonpowers of two as diag(A, I), whose original inverse block is unchanged. This is an **exact-arithmetic complexity reduction**. Forming A^TA solely for this proof is unsuitable as a default floating-point strategy because it squares the condition number. Complex matrices require conjugate transpose A^*, not ordinary transpose.

28.3 Symmetric positive-definite matrices and least-squares approximation

Real SPD means $A=A^T$ and $z^TAz>0$ for **every** nonzero z. Singularity would give a nonzero $Az=0$ and zero quadratic form, so SPD implies nonsingularity. For any leading block B, pad its nonzero vector z to(z,0); positivity of the full quadratic form proves B SPD.

For SPD $A=[B\ C;\ C^T\ E]$, $S=E-C^TB^{-1}C$ is symmetric. For nonzero y, set $z=-B^{-1}Cy$. Then

$(z, y)^TA(z, y)=z^TBz+2z^TCy+y^TEy=y^T(E-C^TB^{-1}C)y=y^TSy>0$.

Thus S is SPD. The first pivot $e_1^TAe_1=A_{11}$ is positive, and elimination recursively handles SPD Schur complements, making every pivot positive. This proves no-pivot elimination avoids zero. Positive original diagonals alone are insufficient:[1 2;2 1] has second pivot−3.

Computing LDL^T and Cholesky

Symmetric elimination gives positive pivots d_j and unit-lower multipliers. Induction gives $A=LDL^T$ with positive diagonal D. Set $C=LD^{1/2}$ to obtain Cholesky $A=CC^T$, with lower C and positive diagonal. A direct column algorithm is

$C[j, j]=\mathrm{sqrt}(A[j, j]-\Sigma_{k<j}C[j, k]^2)$,

$C[i, j]=(A[i, j]-\Sigma_{k<j}C[i, k]C[j, k])/C[j, j]$, for $i>j$.

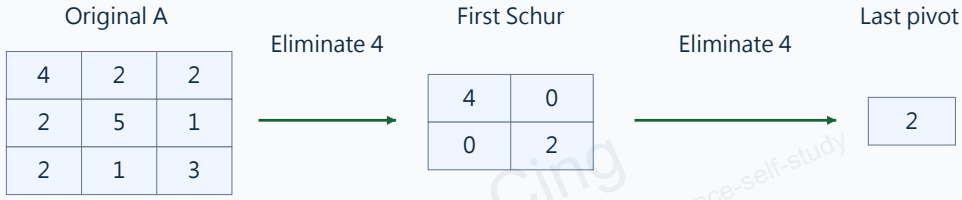
SPD Schur complements keep every square-root argument positive. Triple summations take $\Theta(n^3)$, storing only one triangle. A common flop convention gives leading $n^3/3$, versus about $2n^3/3$ for general LU. This does not promise exactly twice the speed on every machine.

For $A=[4\ 2\ 2;2\ 5\ 1;2\ 1\ 3]$, $C_{11}=2$, $C_{21}=C_{31}=1$, $C_{22}=\mathrm{sqrt}(5-1)=2$, $C_{32}=(1-1\cdot 1)/2=0$, $C_{33}=\mathrm{sqrt}(3-1-0)=\sqrt{2}$. Hence $C=[2\ 0\ 0;1\ 2\ 0;1\ 0\ \sqrt{2}]$. Its product has diagonals 4,5,3 and off-diagonals 2,2,1, matching A.

Computing LDL^T and Cholesky (continued)

SPD Schur complements make Cholesky legal

Every SPD elimination pivot is positive; here 4,4,2.



C rows=[2,0,0]、[1,2,0]、[1,0,√2].

Product CC^T has diagonals 4,5,3 and off-diagonals 2,2,1, matching every original entry.

Least squares: necessity and global sufficiency

For $A \in R^{m \times n}$, $b \in R^m$, minimize $f(x)=||Ax-b||_2^2$. Linearity is in coefficients. Basis functions $A[i, j]=f_j(t_i)$ may be nonlinear in t; for example 1, t, t^2 still give a linear coefficient model.

For any direction z, expand $f(x+tz)=f(x)+2t\, z^T A^T(Ax-b)+t^2||Az||^2$. At a minimum, each directional linear term must vanish; otherwise a small t of the opposite sign decreases f. Therefore $A^TAx=A^Tb$. Conversely, if these normal equations hold, setting t=1 gives $f(x+z)=f(x)+||Az||^2 \geq f(x)$. The stationary point is a global minimum.

Gram A^TA is symmetric positive semidefinite, with $z^TA^TAz=||Az||^2$. It is SPD **if and only if the columns are linearly independent**. Full column rank gives unique coefficients. Rank deficiency permits multiple coefficients but one prediction Ax:two minimizers differ by z with Az=0. At least one minimizer always exists. Choose independent columns as a basis, solve their full-rank SPD normal equations, and observe that residual orthogonality to the basis gives orthogonality to all columns. For all-zero A, take x=0, p=0, residual b.

Full-column-rank pseudoinverse and projection

For full column rank, define $n \times m$ $A^{+}=(A^{T}A)^{-1}A^{T}$, so $\hat{x}=A^{+}b$. Although $A^{+}A=I_n$, generally $AA^{+} \neq I_m$. Instead $Q=A(A^{T}A)^{-1}A^{T}$ satisfies $Q^{T}=Q$, $Q^{2}=Q$, and $QAz=Az$ for every column-space vector Az. It projects b onto Col(A), with $A^{T}(I-Q)b=0$.

For the next 3×2 design, A^{+} rows are$(5/6,1/3,-1/6),(-1/2,0,1/2)$, mapping $b=(0,2,1)$ to$(1/2,1/2)$. Q rows are$(5/6,1/3,-1/6),(1/3,1/3,1/3),(-1/6,1/3,5/6)$. This is not I_3; it maps b to$(1/2,1,3/2)$, not b itself.

For rank-deficient A, the inverse$(A^{T}A)^{-1}$ is undefined and this formula cannot be applied. The general Moore–Penrose inverse can be constructed by SVD. The formula and coefficient uniqueness here require full column rank.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Complete fit to three noncollinear data points

Use$(t, y)=(0,0),(1,2),(2,1)$, coefficients $x=(c, m)^{T}$, and model $\hat{y}=c+mt$.

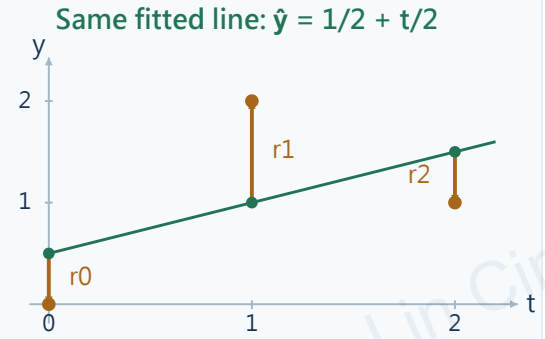
$A=[1\ 0;1\ 1;1\ 2]$, $b=(0,2,1)^{T}$, $A^{T}A=[3\ 3;3\ 5]$, $A^{T}b=(3,4)^{T}$. Subtract $3c+3m=3$ from $3c+5m=4$ to get $m=1/2$, $c=1/2$.

t	Observed y	Prediction $\hat{y}$	$r=y-\hat{y}$
0	0	1/2	−1/2
1	2	1	1
2	1	3/2	−1/2

The components of $A^{T}r$ are$-1/2+1-1/2=0$ and $0 \cdot (-1/2)+1 \cdot 1+2 \cdot (-1/2)=0$. Squared residual norm is 3/2. Independent columns$(1,1,1),(0,1,2)$ establish unique coefficients. **Collinearity of data points does not determine this design's column rank.** Keeping t and replacing y by$(1,2,3)$ makes the data collinear without changing A; only the residual becomes 0.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Complete fit to three noncollinear data points (continued)

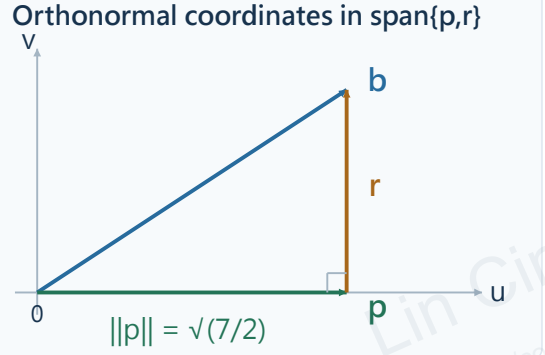


t	y	ŷ	r
0	0	1/2	−1/2
1	2	1	1
2	1	3/2	−1/2

$r^T r = 1/4 + 1 + 1/4 = 3/2$

Amber: vertical error at fixed t. Green: predicted values and fitted line.

Treat the three observations as one vector in R^3



Coordinates in R^3

b = (0, 2, 1)
p = (1/2, 1, 3/2)
r = (−1/2, 1, −1/2)

u = p / ||p||
v = r / ||r||

$A^T r = (0, 0)$
$||r||^2 = 3/2$

Section E=span{p,r}; its horizontal line is E∩Col(A)=span{p}

For these nonzero orthogonal p,r, use orthonormal axes u=p/||p|| and v=r/||r||. The listed b,p,r retain their original R^3 coordinates. Since $A^T r=0$, r is perpendicular to all of Col(A). Section E=span{p,r} meets Col(A) only along span{p}; the horizontal line denotes this intersection. Col(A) remains a two-dimensional plane in R^3.

Orthogonal residuals, conditioning, and rank deficiency

Let $p=A\hat{x}$, $r=b-p$. Equation $A^{T}r=0$ makes r orthogonal to all Col(A), with $b=p+r$. For coefficients $\hat{x}+z$, the residual is $r-Az$, with orthogonal terms. Squared error is $||r||^{2}+||Az||^{2}$, connecting normal equations to minimum-distance geometry.

In the(t, y) scatterplot, r_{i} is the **vertical y-error at fixed t_{i}**, not the shortest point-to-line distance. The other picture treats all b as one vector in R^{3}; there r is the shortest displacement from b to subspace Col(A). These pictures use different spaces.

With repeated columns A=[1 1;1 1;1 1], b=(0,2,1), only $c_{1}+c_{2}=1$ is fixed. Every(s,1−s) is optimal, while prediction(1,1,1) and residual(−1,1,0) remain unique.

For full column rank,$\kappa_{2}(A^{T}A)=\kappa_{2}(A)^{2}$ because singular values are squared. Matrix diag(1,10^{-3}) has condition 10^{3}, its Gram 10^{6}. This measures sensitivity, not operation count. QR or SVD often solves floating-point least squares, especially near rank deficiency. Exact normal-equation correctness does not guarantee stability for every floating dataset.

References

Bibliography

Chapter source

Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*,3rd ed., MIT Press,2009, Chapter 28.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

References and attribution

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Polynomials and FFT

Part VII Selected Topics

Polynomial multiplication becomes pointwise multiplication after evaluation at distinct points. We derive the DFT and its inverse from root orthogonality, then check the even/odd split, bit reversal, butterfly indices, and zero padding. Exact arithmetic and floating-point arithmetic require different guarantees.

Prerequisites are complex roots, value representations, and recursive divide and conquer.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Chapter map

- Compare multiplication in coefficient and value representations.
- Derive the DFT and inverse DFT.
- Implement radix-2 FFT and state the conditions on the roots of unity.

The sections cover polynomial representations, the DFT and FFT, and efficient FFT implementations.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Coefficient representation

A polynomial $A(x) = \sum_{j=0}^{n-1} a_j x^j$ has **degree bound** n, meaning $\deg A < n$. Its leading coefficients may be zero, so its actual degree may be smaller. Code may represent the zero polynomial by `[0]`. The degree bound is not the location of the first nonzero entry. Coefficients here run from low degree to high degree.

Addition of equal-length coefficient vectors takes $\Theta(n)$. Horner evaluation at z starts with $v = a_{n-1}$ and updates $v = a_j + zv$ for $j = n-2, \ldots, 0$. After the update at j, the invariant is $v = \sum_{r=j}^{n-1} a_r z^{r-j}$. The result is $A(z)$, using $n-1$ multiplications and additions. Independent Horner evaluations at n arbitrary points cost $\Theta(n^2)$.

For input lengths r, s, convolution gives $c_k = \sum_{i+j=k} a_i b_j$ and output length $r + s - 1$. Direct convolution uses rs coefficient products, or $\Theta(rs)$ operations. FFT improves the entire conversion-and-multiplication process.

Interpolation exists and is unique

Given distinct $x_0, \ldots, x_{n-1}$ and values $y_0, \ldots, y_{n-1}$, define

$$\ell_k(x) = \prod_{j \neq k} \frac{x - x_j}{x_k - x_j}, \qquad A(x) = \sum_k y_k \ell_k(x).$$

Every denominator is nonzero. Each basis polynomial has degree less than n and satisfies $\ell_k(x_i) = \delta_{ik}$, proving existence.

If two polynomials of degree less than n match the values, their difference has n distinct roots. A nonzero polynomial of degree d has at most d roots. To prove this, factor out $x - \alpha$ at a root α. Every other root is a root of the quotient, so induction on degree applies. The difference must be zero. With fewer than n points, adding any multiple of $\prod_j (x - x_j)$ over the known points usually gives distinct solutions within the degree bound.

Vandermonde matrices and interpolation cost

Write $V[i,j] = x_i^j$ and $Va = y$. The Vandermonde determinant is $\det V = \prod_{j<k}(x_k - x_j)$. Equal points give equal rows, so every difference is a factor of the determinant polynomial. Its total degree is exactly $n(n-1)/2$. The coefficient of $x_1 x_2^2 \cdots x_{n-1}^{n-1}$ is 1, fixing the constant factor. Distinct points therefore make V invertible. General LUP interpolation costs $\Theta(n^3)$.

Lagrange interpolation can be converted to coefficients in $O(n^2)$ arithmetic operations. Build $Q(x) = \prod_j (x - x_j)$ one factor at a time in total quadratic time. For each k, use linear-time synthetic division to obtain $Q/(x - x_k)$, evaluate its denominator in $O(n)$, and accumulate coefficients. Rebuilding every basis polynomial from scratch does not justify that quadratic bound.

A product needs enough values

At the **same N distinct points**, multiplication needs only N pointwise products. Unique reconstruction of lengths r, s requires $N \geq r + s - 1$. Pad both coefficient arrays to N. Radix-2 FFT chooses a power of two at least this large.

For $A = 1 + 2x$ and $B = 1 - x$, the product is $1 + x - 2x^2$, which needs three coefficients. Choose $N = 4$, even though each input has length 2.

At the Nth roots of unity, $x^N = 1$. With too few points, the inverse returns convolution modulo $x^N - 1$. Position k receives $c_k + c_{k+N} + c_{k+2N} + \cdots$. For $(1+x)^2$, $N = 2$ gives `[2,2]` because the x^2 coefficient wraps into the constant coefficient. Ordinary multiplication gives `[1,2,1]`. Zero padding prevents this aliasing.

Roots of unity and the forward DFT

Use the **positive-exponent forward convention** throughout:

$$\omega_n = e^{2\pi i/n}, \qquad y_k = \sum_{j=0}^{n-1} a_j \omega_n^{jk} = A(\omega_n^k).$$

Some fields use a negative forward exponent. Either paired convention works, but mixing them does not.

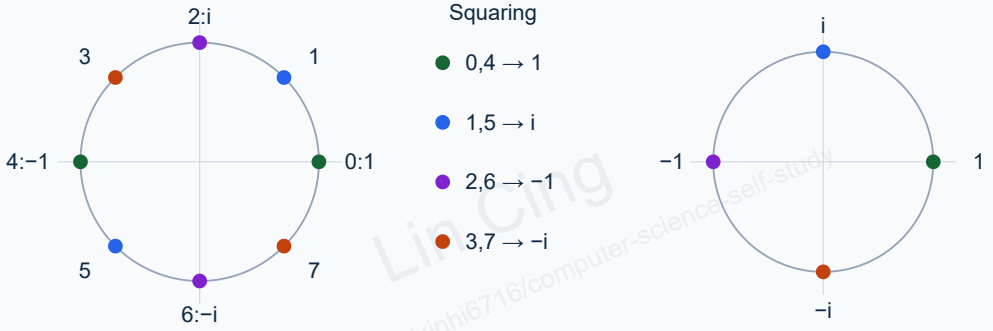
The root is primitive, so $\omega_n^k = 1$ exactly when $n \mid k$. Its n powers are distinct interpolation points. Cancellation gives $\omega_{dn}^{dk} = \omega_n^k$ for positive integers d. For even n, $\omega_n^{n/2} = -1$ and $(\omega_n^k)^2 = \omega_{n/2}^k$. Opposite roots square to the same value, leaving $n/2$ distinct roots, each twice.

The geometric sum $\sum_{j=0}^{n-1} \omega_n^{jt}$ is n if $n \mid t$. Otherwise it is $(1 - (\omega_n^t)^n)/(1 - \omega_n^t) = 0$. Primitivity makes the denominator nonzero in the latter case. The value 1 cannot replace a primitive root for arbitrary n.

Roots of unity and the forward DFT (continued)

Eight roots square to four distinct points

Left label k denotes ω_8^k. Same-color opposite points square to one point on the right.



Orthogonality gives the inverse

Let $F[k, j] = \omega_n^{jk}$ and $G[j, k] = \omega_n^{-jk}/n$. Then

$$(GF)[j, \ell] = \frac{1}{n} \sum_k \omega_n^{k(\ell-j)} = \delta_{j\ell}.$$

For distinct $j, \ell \in [0, n-1]$, $n \nmid \ell - j$, so orthogonality applies. Thus $GF = I$, $G = F^{-1}$, and

$$a_j = \frac{1}{n} \sum_k y_k \omega_n^{-jk}.$$

Use negative twiddles and divide by n once at the end. Normalization may instead be distributed across levels, but dividing by n at every level is wrong.

Applying the forward transform twice and dividing by n gives $(F^2 a/n)_j = a_{(-j) \bmod n}$. This is cyclic index reversal, not conjugation for arbitrary complex input. For $a = [2, 5+i, 7, -1]$, the result is `[2,-1,7,5+i]`.

Even and odd subproblems

For a positive power of two n, set $A_e(z) = a_0 + a_2 z + \cdots$ and $A_o(z) = a_1 + a_3 z + \cdots$. Then $A(x) = A_e(x^2) + x A_o(x^2)$. Root halving reduces evaluation at all n roots to two $n/2$-point DFTs, E and O.

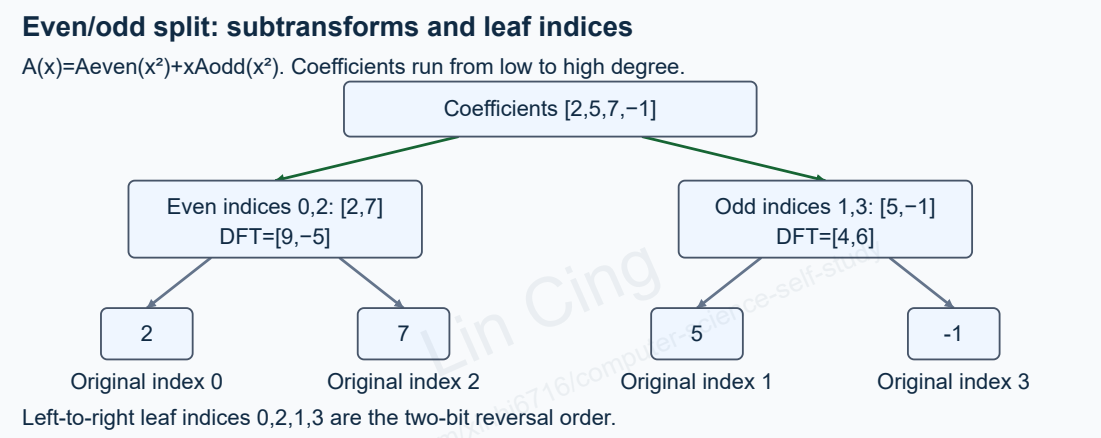
For $0 \le k < n/2$,

$$y_k = E_k + \omega_n^k O_k, \qquad y_{k+n/2} = E_k - \omega_n^k O_k.$$

Compute $t = \omega_n^k O_k$ once, then add and subtract. This is a butterfly. The base case $n = 1$ returns `[a[0]]`. Assuming the two recursive transforms are correct, the displayed identities produce every required value, proving correctness by induction.

For `[2,5,7,-1]`, the even coefficients `[2,7]` give `[9,-5]`, and the odd coefficients `[5,-1]` give `[4,6]`. Twiddle 1 gives $y_0 = 13, y_2 = 5$. Twiddle i gives $y_1 = -5 + 6i, y_3 = -5 - 6i$. Distinct intermediate values expose incorrect wiring.

Even and odd subproblems (continued)



Recursive FFT and its cost

```
FFT(a, sign):
    n = len(a)                  # require a positive power of two
    if n == 1: return [a[0]]
    E = FFT([a[0], a[2], ...], sign)
    O = FFT([a[1], a[3], ...], sign)
    w = 1; wm = exp(sign * 2*pi*i/n)
    for k = 0 .. n/2-1:
        t = w * O[k]
        y[k] = E[k] + t
        y[k+n/2] = E[k] - t
        w = w * wm
    return y
```

Forward uses `sign=+1`. Inverse uses `sign=-1` and divides the outer result by n. Splitting and combining cost $\Theta(n)$ per level. Thus $T(n) = 2T(n/2) + \Theta(n) = \Theta(n \log n)$ complex arithmetic operations. There are $n/2$ butterflies per level and $(n/2)\log_2 n$ in total.

Recursive FFT and its cost (continued)

A sequential implementation that copies subarrays and promptly releases child results needs $O(n)$ peak extra space and $O(\log n)$ stack space. Retaining all levels takes $O(n \log n)$. Recursion depth alone does not bound the arrays to logarithmic space.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

All three transforms in a convolution

For $A = [1, 2, 0, 0]$, $B = [1, -1, 0, 0]$, $N = 4$, and $\omega_4 = i$:

k	DFT(A)	DFT(B)	Pointwise product
0	3	0	0
1	$1 + 2i$	$1 - i$	$3 + i$
2	-1	2	-2
3	$1 - 2i$	$1 + i$	$3 - i$

The negative-exponent DFT of `[0,3+i,-2,3-i]` is `[4,4,-8,0]` before normalization. Dividing by 4 gives `[1,1,-2,0]`. Keep the first three coefficients to recover $1 + x - 2x^2$.

Two forward transforms, one inverse, and $\Theta(N)$ pointwise products cost $\Theta(N \log N)$. Because the output degree is below N, cyclic convolution equals ordinary convolution.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

All three transforms in a convolution (continued)

Convolution: frequency values and inverse

A=1+2x, B=1−x, padded to N=4. Positive forward exponent, $\omega_4=i$.

k	DFT(A)	DFT(B)	Pointwise product
0	3	0	0
1	1+2i	1−i	3+i
2	−1	2	−2
3	1−2i	1+i	3−i

Negative DFT: [4,4,−8,0]. Divide by 4: [1,1,−2,0].

Green cells are product values. Keep three coefficients: $1+x-2x^2$.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Bit reversal and stage 1

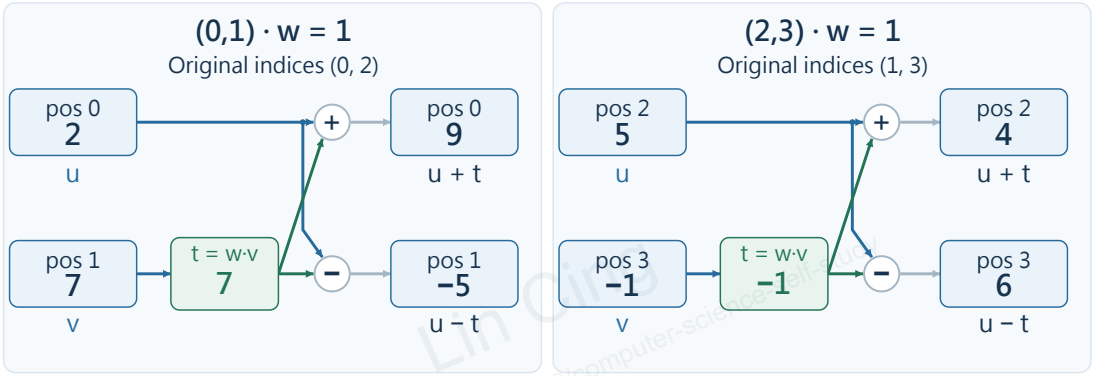
Recursive splitting reads the lowest index bit first, then the next bit. For $n = 2^b$, the leaf order reverses the b bits of each original index. Bit reversal is its own inverse, so both `A[rev(j)]=a[j]` and `A[j]=a[rev(j)]` describe the permutation.

For $n = 4$, `00,01,10,11` become `00,10,01,11`. The input `[2,5,7,-1]` becomes `[2,7,5,-1]`. Stage 1 pairs **physical positions** `(0,1)` and `(2,3)`, which hold original indices `(0,2)` and `(1,3)`. These are different kinds of index.

Both stage-1 twiddles equal 1. The array becomes `[9,-5,4,6]`. Stage 2 uses $m = 4$, pairs `(0,2)` and `(1,3)`, and uses twiddles 1 and i. Pairing adjacent positions again would perform a different computation.

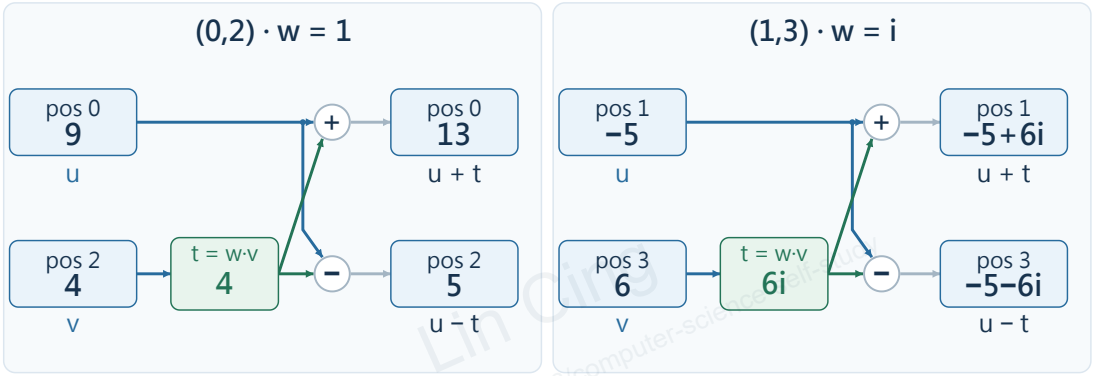
Lin Cing https://github.com/xinhi6716/computer-science-self-study

Bit reversal and stage 1 (continued)



Stage 1: physical array [2,7,5,−1] → [9,−5,4,6]

Stage 2 combines across half-blocks



Stage 2: read positions 0,1,2,3 as [13,−5+6i,5,−5−6i]

j	$u = A[j]$	$t = \omega_4^j A[j+2]$	**New** $A[j]$	**New** $A[j+2]$
0	9	$1 \cdot 4 = 4$	13	5
1	-5	$i \cdot 6 = 6i$	$-5 + 6i$	$-5 - 6i$

Stage 2 combines across half-blocks (continued)

In physical output order, the result is `[13,-5+6i,5,-5-6i]`.

The same rules take `[1,2,0,-1]` through bit reversal `[1,0,2,-1]`, stage 1 `[1,1,1,3]`, and stage 2 `[2,1+3i,0,1-3i]`. Repeated intermediate 1s can conceal wiring mistakes. The earlier example makes the sources of each operation distinct.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Iterative FFT index contract

```
place a into A in bit-reversed order
for s = 1 .. log2(n):
    m = 2^s; wm = exp(sign * 2*pi*i/m)
    for base = 0, m, 2*m, ..., n-m:
        w = 1
        for j = 0 .. m/2-1:
            u = A[base+j]
            t = w * A[base+j+m/2]
            A[base+j] = u+t
            A[base+j+m/2] = u-t
            w = w*wm
for inverse only, divide every final entry by n
```

Lin Cing

At the start of stage s, each length-m block has the DFT of its even subproblem in its left half and the DFT of its odd subproblem in its right half. Butterflies perform the recursive combination, leaving a complete m-point DFT in each block. Singletons establish the stage-1 base case. Induction reaches $m = n$. Save `u` before overwriting the upper cell, or the lower subtraction will use a new value.

Iterative FFT index contract (continued)

Each stage has $n/2$ butterflies, so arithmetic work is $\Theta(n \log n)$. Reversing b bits separately for each index takes $O(n \log n)$ bit steps. Reorder with an extra length-n array or in-place swaps of the permutation's two-cycles. Twiddles depend on m, j, not `base`. All blocks in a stage can share a table. Tables for all stages contain $\Theta(n)$ values and do not change the main arithmetic bound.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Circuit work and depth

With fixed twiddles and bit reversal treated as wires, a bounded-fan-in circuit runs all $n/2$ disjoint butterflies in a stage at once. Only the next stage reads their outputs. For $n = 8$, there are 3 stages and 12 butterflies. Work is $\Theta(n \log n)$, while depth is $\Theta(\log n)$, since every input-to-output path crosses $\log_2 n$ combinations.

This model assumes constant-depth complex addition and multiplication. It does not include serial twiddle generation, processor communication, or physical wire lengths. Software threads sharing an array need a barrier after each stage. Within a stage each butterfly writes a different pair of cells. With only P processors, the lower bound W/P still applies. A logarithmic-depth drawing does not give logarithmic running time on arbitrary hardware.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Exact arithmetic and floating point

The $\Theta(n \log n)$ bound counts complex arithmetic operations. Floating-point roots and arithmetic introduce errors. Rounding an integer convolution is valid only with an absolute error below $1/2$ in every output cell. A few successful round trips cannot guarantee all lengths and coefficient magnitudes.

An exact alternative is an NTT over a prime field $\mathbb{F}_p$. Require $N \mid p - 1$ and a root of order exactly N. Then N is invertible and the same geometric-sum and inverse proofs apply. Over a composite modulus, a primitive root alone does not ensure that the required denominators are units. The statement here is restricted to prime fields.

For the same inputs, use $p = 17, N = 4, \omega = 13$. We have $\omega^2 = 16 \neq 1$, $\omega^4 = 1$, and $4^{-1} = 13$. The transforms are `[3,10,16,9]` and `[0,5,2,14]`. Pointwise multiplication modulo 17 gives `[0,16,15,7]`, and the inverse gives `[1,1,15,0]`. Decode 15 as -2.

Signed reconstruction needs a magnitude bound

If all integer input coefficients have absolute value at most B, with lengths r, s, every convolution coefficient has absolute value at most $\min(r, s)B^2$. Choose $p > 2\min(r, s)B^2$ for unique signed reconstruction from one modulus. Here $r = s = 2, B = 2$, and $17 > 16$, so reconstruction is unique. Correct modular arithmetic alone cannot rule out wraparound of larger integers.

The NTT operation bound counts arithmetic modulo p. If p grows with coefficient magnitude, bit complexity must include multiplication and addition of $\log p$-bit integers. Arbitrarily large integer multiplication is not one constant-time operation.

Source

Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 3rd ed., MIT Press, 2009, Chapter 30.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

References and attribution

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Number-theoretic algorithms

Part VII Selected Topics

Divisibility, groups, and CRT support modular equations, powers, RSA, and randomized primality tests. A returned factor is a deterministic certificate. Pollard rho's time estimate needs an additional assumption. Prerequisites are divisibility, modular arithmetic, groups, and primes.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Chapter map

The sections cover elementary number theory, greatest common divisors, modular arithmetic, modular linear equations, the Chinese remainder theorem, powers of an element, RSA, primality testing, and integer factorization.

The goals are to use Euclid and extended Euclid, solve modular equations and CRT systems, and distinguish RSA correctness, randomized testing guarantees, and factorization assumptions.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Divisibility and input length

For integers d, a, $d \mid a$ means $a = kd$ for some integer k. A prime $p \geq 2$ has exactly the positive divisors 1 and p. The integer 1 is not prime. The gcd is nonnegative and positive unless both inputs are zero. Define $\gcd(0, 0) = 0$. Code first takes absolute values of negative inputs.

For $n \geq 1$, $a \equiv b \pmod{n}$ means $n \mid a - b$. The representative `a mod n` lies in $[0, n - 1]$, even for negative inputs. For example, $-7 \bmod 5 = 3$. Congruences preserve sums, differences, and products because their differences remain multiples of n. Division needs additional conditions.

A β-bit integer can have value near 2^β. A loop up to $\sqrt{n}$ is exponential in input length. We distinguish arithmetic operation counts from bit costs. Schoolbook multiplication and long-division remainder on $O(\beta)$-bit integers each take $O(\beta^2)$ bit operations.

Bezout, cancellation, and factorization

For a, b not both zero, let g be the smallest positive integer of the form $ax + by$. Divide a by g. The remainder $a - qg$ is another integer linear combination. A positive remainder below g contradicts minimality, so $g \mid a$. Similarly $g \mid b$. Every common divisor divides g, proving $g = \gcd(a, b)$ and Bezout's identity $ax + by = g$.

If $\gcd(a, n) = 1$, write $au + nv = 1$. Multiplication by b gives $b = abu + nbv$, so $n \mid ab$ implies $n \mid b$. Coprimality permits cancellation. Modulo 6, $2 \cdot 1 \equiv 2 \cdot 4$, but $1 \not\equiv 4$.

If prime $p \nmid a$, then $\gcd(a, p) = 1$, so $p \mid ab$ implies $p \mid b$. Every $n \geq 2$ factors into primes by recursively factoring smaller divisors. Compare two factorizations. A prime in the first divides some prime in the second and therefore equals it. Cancel and induct to prove unique prime factorization.

Euclid's algorithm

For $a = qb + r$, $0 \le r < b$, a common divisor of a, b divides $r = a - qb$. A common divisor of b, r divides $a = qb + r$. Thus $\gcd(a, b) = \gcd(b, r)$.

```
euclid(a,b):
    a=abs(a); b=abs(b)
    while b != 0: (a,b)=(b,a mod b)
    return a
```

The second argument strictly decreases to zero. The current gcd equals the original gcd, and the final $\gcd(a, 0) = a$ proves correctness. If $0 < b \le a$, set $r = a \bmod b$. If $r \le b/2$, the second argument already halves. Otherwise the next remainder is $b \bmod r = b - r < b/2$. Every two divisions halve it, giving $O(\log b)$ divisions, or $O(\beta)$ for β-bit inputs.

Fibonacci chains and bit bounds

If $a > b \ge 1$ takes k divisions to reach zero, tracing backward gives each earlier remainder at least the sum of the next two. With $F_1 = F_2 = 1$, this yields $a \ge F_{k+2}, b \ge F_{k+1}$. Fibonacci numbers at least double every two positions, so $k = O(\log b)$. Consecutive Fibonacci inputs have quotient 1 except at the final division and attain the same order of chain length.

The coarse per-division bound $O(\beta^2)$ times $O(\beta)$ rounds gives a safe $O(\beta^3)$ bit bound. This is not optimal. Tracking shrinking operands gives the sharper classical $O(\beta^2)$ gcd bound, but we do not use that refinement to claim measured speed.

Keeping two remainders uses $O(\beta)$ bits. Saving every trace row has additional storage cost. Euclid needs no known prime factorization.

Extended Euclid

If recursion on (b, r) returns $g = bx' + ry'$, substituting $r = a - qb$ gives $g = ay' + b(x' - qy')$. Return $(x, y) = (y', x' - qy')$. The base case $b = 0$ returns $(g, x, y) = (a, 1, 0)$.

```
extended(a,b):                  # a,b >= 0
    if b == 0: return (a,1,0)
    (g,x1,y1)=extended(b,a mod b)
    return (g,y1,x1-floor(a/b)*y1)
```

For 252 and 198, the divisions are $252 = 198 + 54$, $198 = 3 \cdot 54 + 36$, $54 = 36 + 18$, $36 = 2 \cdot 18$. Back substitution gives $18 = 54 - 36 = 4 \cdot 54 - 198 = 4 \cdot 252 - 5 \cdot 198$. The gcd is 18 and coefficients are $(4, -5)$. Adjust corresponding coefficient signs for negative original inputs.

Extended Euclid (continued)

Euclid: every remainder is an integer linear combination

Remainder r	252 coefficient x	198 coefficient y	Check 252x+198y
252	1	0	252+(0)=252
198	0	1	0+(198)=198
54	1	-1	252+(-198)=54
36	-3	4	-756+(792)=36
18	4	-5	1008+(-990)=18

Quotients 1,3,1,2. Last nonzero remainder: 18 = 4×252 − 5×198.

Modular groups and units

A group operation is closed and associative, has an identity, and gives every element an inverse. An abelian group also has a commutative operation. Inverses are unique. If b, c are inverses of a, then $b = b(ac) = (ba)c = c$.

The additive group $(\mathbb{Z}_n, +)$ is finite and abelian, with identity 0 and inverse $-a$. Multiplication on all of $\mathbb{Z}_n$ is generally not a group, since 0 has no inverse. For $n \geq 2$, the units $U(n) = \{a \in [0, n-1] : \gcd(a, n) = 1\}$ form the multiplicative group.

Bezout gives $au + nv = 1$ when $\gcd(a, n) = 1$, so u is an inverse. Conversely, if $au \equiv 1$, every common divisor divides $au - nv = 1$, so the gcd is 1. Products of units remain units because $(ab)^{-1} = b^{-1}a^{-1}$. Division by a has a unique modular meaning precisely when a is a unit.

Lagrange's theorem through cosets

A subgroup $H \subseteq G$ contains the identity and is closed under products and inverses. A left coset gH has $|H|$ elements because multiplication by g is a bijection. If $gh_1 = kh_2$, then $k^{-1}g = h_2h_1^{-1} \in H$, so $gH = kH$. Otherwise the cosets are disjoint. They partition finite G, proving $|H| \mid |G|$. A proper subgroup has size at most $|G|/2$.

A nonempty product-closed subset of a finite group is already a subgroup. For each a, finiteness gives $a^i = a^j$, $i < j$, so $a^{j-i} = e$ belongs to the subset. If $j - i > 1$, a^{j-i-1} is an inverse in the subset. If $j - i = 1$, then $a = e$, whose inverse is itself.

The order of a is the least positive t with $a^t = e$. The elements $a^0, \ldots, a^{t-1}$ are distinct and powers repeat modulo t, so its generated subgroup has size t. Lagrange gives $t \mid |G|$, hence $a^{|G|} = e$. Finiteness matters. The additive group of integers has elements of infinite order.

Totients and distinct operations

Define $\varphi(n) = |U(n)|$. For prime p, the nonunits modulo p^α are exactly the $p^{\alpha-1}$ multiples of p. Hence $\varphi(p^\alpha) = p^{\alpha-1}(p-1)$. CRT below identifies units of a coprime product with the Cartesian product of component unit groups, giving $\varphi(n) = \prod_i p_i^{\alpha_i-1}(p_i-1)$.

For 15, $U(15) = \{1, 2, 4, 7, 8, 11, 13, 14\}$, so $\varphi(15) = 8$. Since $7 \cdot 13 \equiv 1$, the modular fraction $4/7$ means $4 \cdot 13 \equiv 7$, not integer quotient 0.

In additive $\mathbb{Z}_8$, 2 generates $\{0, 2, 4, 6\}$, with order 4. In multiplicative $U(7)$, 2 has powers 1,2,4 and order 3. The powers of 3 are 1,3,2,6,4,5 and its order is 6. The notation $\langle a \rangle$ depends on the operation.

All modular linear solutions

For $ax \equiv b \pmod{n}$, $n \geq 1$, set $g = \gcd(a, n)$. Necessarily $g \mid b$, since $ax + ny = b$. Extended Euclid gives $au + nv = g$. If $b = gh$, then $x_0 = uh$ is a solution, proving sufficiency.

Divide by g to get $(a/g)x \equiv b/g \pmod{n/g}$. Its coefficient is coprime to the new modulus, so it has one solution modulo n/g. Subtracting two solutions and cancelling gives $n/g \mid x - x_0$. All g distinct solutions modulo n are therefore

$$x = x_0 + k(n/g) \pmod{n}, \qquad k = 0, \ldots, g-1.$$

This proves existence, number, completeness, and an algorithm. If $a = 0$, then $g = n$. Every representative solves it when $n \mid b$, and none do otherwise. For $n = 1$, the only representative is 0. We do not assert general unit division for that modulus.

Equation example and output cost

For $18x \equiv 30 \pmod{42}$, $g = 6 = -2 \cdot 18 + 42$, so $x_0 = -10 \equiv 4 \pmod{7}$. The six solutions modulo 42 are 4,11,18,25,32,39. For example, $18 \cdot 4 \equiv 30$, and increasing x by 7 increases the product by $126 = 3 \cdot 42$. Replacing 30 with 31 gives no solution, since $6 \nmid 31$. The inverse of 18 modulo 42 does not exist.

The compressed triple $(x_0, n/g, g)$ avoids enumeration. Listing all solutions requires $\Omega(g)$ outputs. After normalizing $a \bmod n$, Euclid plus enumeration takes $O(\log n + g)$ arithmetic operations. Each output still has $O(\log n)$ bits, which must be counted.

Chinese remainder construction

For pairwise coprime $m_1, \ldots, m_k \geq 2$, let $M = \prod_i m_i$, $M_i = M/m_i$, $z_i = M_i^{-1} \pmod{m_i}$, $e_i = M_i z_i$, and $x = \sum_i a_i e_i \pmod{M}$.

For fixed j, every e_i with $i \neq j$ is zero modulo m_j, while $e_j \equiv 1$. Thus $x \equiv a_j \pmod{m_j}$, proving existence. If x, x' both solve the system, each m_i divides their difference. Pairwise coprimality and cancellation imply the product divides the difference, proving uniqueness modulo M.

The map $x \mapsto (x \bmod m_1, \ldots, x \bmod m_k)$ is a bijection preserving addition and multiplication. A residue is a unit exactly when every component is a unit. Otherwise it shares a prime factor with some component modulus. This proves multiplicativity of the totient for coprime products.

CRT selectors reconstruct 29

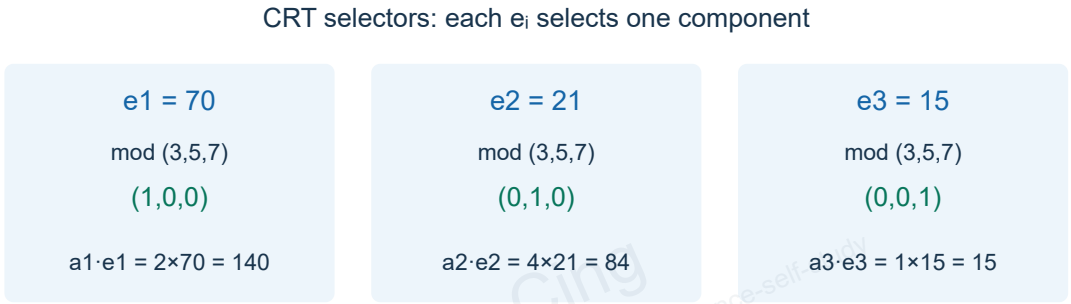
Solve $x \equiv 2 \pmod 3, x \equiv 4 \pmod 5, x \equiv 1 \pmod 7$. Here $M = 105$, $M_i = (35, 21, 15)$, inverses are $(2, 1, 1)$, and selectors are $(70, 21, 15)$.

$$x = 2 \cdot 70 + 4 \cdot 21 + 15 = 239 \equiv 29 \pmod{105}.$$

Indeed, 29 has residues 2,4,1. The selector coordinates are $(1, 0, 0), (0, 1, 0), (0, 0, 1)$. Modulo M, $e_i^2 = e_i$, $e_i e_j = 0$ for $i \neq j$, and $\sum_i e_i = 1$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

CRT selectors reconstruct 29 (continued)



Intermediate integers must not overflow. The bit length of M is of order $\sum_i \log m_i$. Use large integers and reduction as needed. The numerical value of M is not its memory cost in bits.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Noncoprime compatibility

For $x \equiv a \pmod{m}, x \equiv b \pmod{n}$, let $g = \gcd(m, n)$. Substituting $x = a + mk$ gives $mk \equiv b - a \pmod{n}$. A solution exists exactly when $g \mid b - a$, and its period is $\operatorname{lcm}(m, n) = mn/g$, not mn.

For $x \equiv 2 \pmod{6}, x \equiv 5 \pmod{9}$, the difference 3 is divisible by $g = 3$. The equation $6k \equiv 3 \pmod{9}$ becomes $2k \equiv 1 \pmod{3}$, so $k \equiv 2$, giving $x \equiv 14 \pmod{18}$. Replacing 5 by 4 gives no solution.

Merge general systems one congruence at a time, checking compatibility and updating the lcm. The pairwise-coprime selector formula cannot be applied directly to 6 and 9, because the inverses do not exist.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Powers, orders, and logarithms

For $a \in U(n)$, Lagrange gives Euler's theorem $a^{\varphi(n)} \equiv 1 \pmod{n}$. For prime p, this becomes Fermat's theorem $a^{p-1} \equiv 1$. Every residue satisfies $a^p \equiv a \pmod{p}$, with the zero case holding directly.

An element g of order $\varphi(n)$ is a primitive root and generates all units. Each unit equals g^k for a unique $k \pmod{\varphi(n)}$, its discrete logarithm $\operatorname{ind}_{n,g}(a)$. By the order definition, $g^x = g^y$ exactly when $x \equiv y \pmod{\varphi(n)}$.

Modulo 7, base 3 maps exponents 0,1,2,3,4,5 to 1,3,2,6,4,5, so $\operatorname{ind}(4) = 4$. Base 2 reaches only 1,2,4 and has no logarithm for 3. A nongenerator need not represent every unit.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Finite fields have multiplicative generators

In a finite abelian group, elements of coprime orders r, s have a product of order rs. If $(ab)^k = 1$, then $a^k = b^{-k}$ lies in both generated subgroups. Their intersection has order dividing both r, s, so it contains only the identity. Thus $r \mid k, s \mid k$. Conversely the rsth power is 1.

Let L be the lcm of element orders in $\mathbb{F}_p^\times$. For each prime ℓ, some element order supplies the highest power ℓ^v in L. A suitable power of that element has order exactly ℓ^v. Multiplying the elements for distinct primes gives an element of order L.

All $p-1$ nonzero elements are roots of $X^L - 1$. A nonzero degree-L polynomial over a field has at most L roots, by repeated factorization, so $p - 1 \leq L$. Lagrange makes each order divide $p-1$, giving $L \leq p-1$. Hence $L = p-1$, and a generator exists. Arbitrary finite groups need not be cyclic.

Generators for odd prime powers

For odd prime p, take a generator g modulo p. Modulo p^2,

$$(g+p)^{p-1} \equiv g^{p-1} + (p-1)g^{p-2}p.$$

The second term divided by p is nonzero modulo p, so at least one $h \in \{g, g+p\}$ satisfies $h^{p-1} = 1 + cp$, $p \nmid c$.

If $v_p(z-1) = j \geq 1$, the binomial expansion gives $v_p(z^p - 1) = j + 1$. The first term is $p(z-1)$. All later terms have at least one more factor of p, including the last because $p \geq 3$, so they cannot cancel the first. Thus $v_p(h^{(p-1)p^k} - 1) = k + 1$.

Reduction modulo p makes the order modulo p^α divisible by $p-1$. Lagrange makes it divide $p^{\alpha-1}(p-1)$. The valuation excludes every smaller power of p, so the order equals the totient. Hence $U(p^\alpha)$ is cyclic. Reusing a generator modulo p unchanged requires the p^2 check.

Exactly which unit groups are cyclic

For $n > 1$, the possibilities are exactly $2, 4, p^\alpha, 2p^\alpha$, with p odd prime. The previous proof covers odd prime powers. CRT gives $U(2p^\alpha) \cong U(2) \times U(p^\alpha)$, whose first component is trivial. The cases 2 and 4 are direct.

If $8 \mid n$, reduction maps $U(n)$ onto $U(8)$. Every element of $U(8)$ squares to 1, but the group has four elements and is not cyclic. A cyclic group's homomorphic image must be cyclic, a contradiction. Surjectivity follows by choosing an odd residue in the power-of-two component and unit residues in the odd prime-power components via CRT.

If n has two distinct odd prime factors, independent CRT signs give at least four units squaring to 1. If $4 \mid n$ and an odd prime divides n, its two components also give four roots. A cyclic group has at most two solutions to $g^{2k} = 1$ modulo its order. These cases exclude all remaining forms.

Nontrivial square roots expose factors

For an odd prime power p^α, $x^2 \equiv 1$ implies $p^\alpha \mid (x-1)(x+1)$. The gcd of the two factors divides 2, so the odd prime p cannot divide both. One factor contains all of p^α, giving $x \equiv \pm 1$.

If $x^2 \equiv 1 \pmod{n}$ but $x \not\equiv \pm 1$, then n is composite and both $\gcd(x-1, n)$ and $\gcd(x+1, n)$ are proper nontrivial factors. Neither equals n, or x would equal a sign. Neither equals 1, since cancellation would force the other factor to be divisible by n.

For $n = 77$, $x = 34$, $34^2 = 1156 \equiv 1$, yet $34 \neq 1, 76$. The gcds are 11 and 7. CRT interprets 34 as +1 modulo 11 and -1 modulo 7, with unsynchronized signs.

Square and multiply

For $n \geq 1, b \geq 0$, normalize $a \bmod n$ and initialize $r = 1 \bmod n$. Read every exponent bit from highest to lowest.

```
r = 1 mod n
for bit in binary(b), highest first:     # empty string for b=0
    r = r*r mod n
    if bit == 1: r = r*a mod n
return r
```

After reading prefix v, the invariant is $r \equiv a^v \pmod{n}$. The next bit δ changes it to $2v + \delta$, exactly the effect of squaring and optionally multiplying by a. The empty prefix has value 0. The final prefix is b, proving correctness. Exponent 0 returns $1 \bmod n$, including 0 when $n = 1$. Coprimality is unnecessary. Negative exponents need an inverse and a separate input contract.

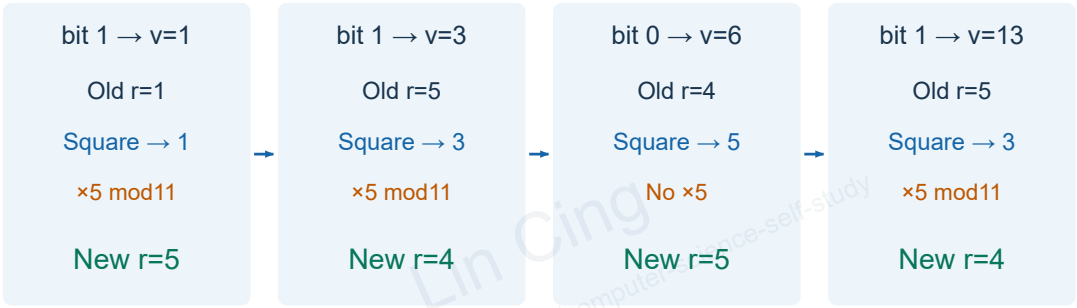
Every bit of 5 to the thirteenth

The binary representation of 13 is 1101. Starting with $r = 1$, scan all four bits modulo 11.

Bit	New prefix	Old r	Square modulo 11	Multiply by 5 or retain
1	1	1	1	5
1	3	5	3	4
0	6	4	5	5
1	13	5	3	4

Every bit of 5 to the thirteenth (continued)

5^{13} mod11: scan all bits 1101, starting at r=1



Invariant $r=5^{v}$ mod11. Prefixes 1 → 3 → 6 → 13; return 4.

Initializing $r = a$ means the leading 1 was already read, so only 101 remains. Starting at a but scanning all of 1101 instead follows exponents 3,7,14,29 and values 4,3,9,9, incorrectly returning 9. A k-bit exponent uses at most $2k$ modular multiplications. With a β-bit modulus, ordinary bit cost is $O(k\beta^2)$, or $O(\beta^3)$ when $k = O(\beta)$. Initial reduction of an exceptionally long base has additional cost.

RSA on every residue

Choose distinct odd primes p, q, set $n = pq, \varphi = (p-1)(q-1)$, and choose $1 < e < \varphi$ with $\gcd(e, \varphi) = 1$. Extended Euclid gives $d = e^{-1} \pmod{\varphi}$, $1 \le d < \varphi$. Publish (n, e), retain d, p, q. For $0 \le m < n$, encrypt $c = m^e \bmod n$ and decrypt $m' = c^d \bmod n$.

Write $ed = 1 + k(p-1)(q-1)$. For any m, modulo p, if $p \mid m$, both m^{ed} and m are zero. Otherwise Fermat gives $m^{p-1} = 1$, so $m^{ed} = m$. The same applies modulo q. CRT uniqueness proves $m^{ed} \equiv m \pmod{n}$, including nonunits. Euler's theorem alone would omit those legal messages.

The private and public powers also undo each other in the opposite order, so they are inverse mathematical permutations. This correctness is separate from protocol security.

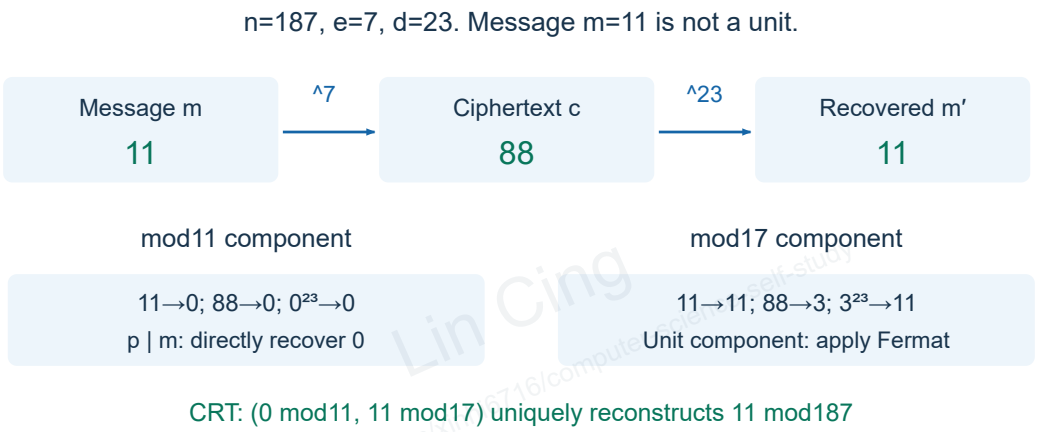
A nonunit RSA message

Take $p = 11, q = 17, n = 187, \varphi = 160, e = 7$. Since $1 = 23 \cdot 7 - 160$, $d = 23$. The message $m = 11$ is a legal nonunit.

Encryption uses $11^2 \equiv 121, 11^4 \equiv 55$, and $11^7 \equiv 55 \cdot 121 \cdot 11 \equiv 88 \pmod{187}$. Decryption uses $88^2 \equiv 77, 88^4 \equiv 132, 88^8 \equiv 33, 88^{16} \equiv 154$. Since $23 = 16 + 4 + 2 + 1$, multiply the corresponding values. The reductions are $154 \cdot 132 \equiv 132$, $132 \cdot 77 \equiv 66$, and $66 \cdot 88 \equiv 11$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A nonunit RSA message (continued)



A fixed-bit-length public exponent costs $O(\beta^2)$ with ordinary arithmetic. A private exponent of $O(\beta)$ bits costs $O(\beta^3)$. Factoring n reveals p, q, hence φ, d. This does not prove the converse claim that RSA inversion is equivalent to factoring.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Algebraic limits of textbook RSA

Without encoding, RSA is deterministic. Equal messages give equal ciphertexts, so a small message space can be enumerated. Also $E(m_1)E(m_2) \equiv E(m_1 m_2)$. Given $c = E(m)$ and $r \in U(n)$, the ciphertext cr^e represents mr. This is malleability, not a security proof.

Using m^d directly as a signature also lacks a complete security construction. Choose any σ, then set $m = \sigma^e$. This creates a pair that passes verification without an authorized signature on a meaningful message. Practical schemes separately specify randomized encodings, hashes, and verification formats. This chapter derives the permutation rather than deployment parameters.

These algebraic weaknesses can coexist with hard factorization of large integers. Decryption correctness cannot replace an attack model and security definition.

Primality tests

Trial division needs divisors only through $\lfloor \sqrt{n} \rfloor$. If $n = ab$ and both factors exceed $\sqrt{n}$, their product exceeds n. Absence of a small factor therefore proves primality. For β-bit n, $\sqrt{n}$ is about $2^{\beta/2}$, not polynomial in bit length.

The prime number theorem gives $\pi(X) \sim X / \ln X$. Uniform sampling in a comparable-magnitude interval has approximate prime density $1/\ln X$, or $2/\ln X$ among odd candidates. This asymptotic estimate does not guarantee a prime within $\ln X$ trials or prove the test's error bound.

A Fermat test finding $a^{n-1} \not\equiv 1$ proves compositeness. Equality gives only a probable prime. For $341 = 11 \cdot 31$, $2^{10} = 1024 \equiv 1$, so $2^{340} = 1$ despite compositeness. Carmichael numbers let every unit pass. For $561 = 3 \cdot 11 \cdot 17$, all of 2,10,16 divide 560, so Fermat and CRT give $a^{560} = 1$ for every unit.

Miller-Rabin boundaries and square chains

Handle $n < 2$ as nonprime, $n = 2, 3$ as prime, and all other even n as composite. For odd $n \geq 5$, write $n - 1 = 2^s d$, d odd, $s \geq 1$. Choose a uniformly among integers in $[2, n - 2]$.

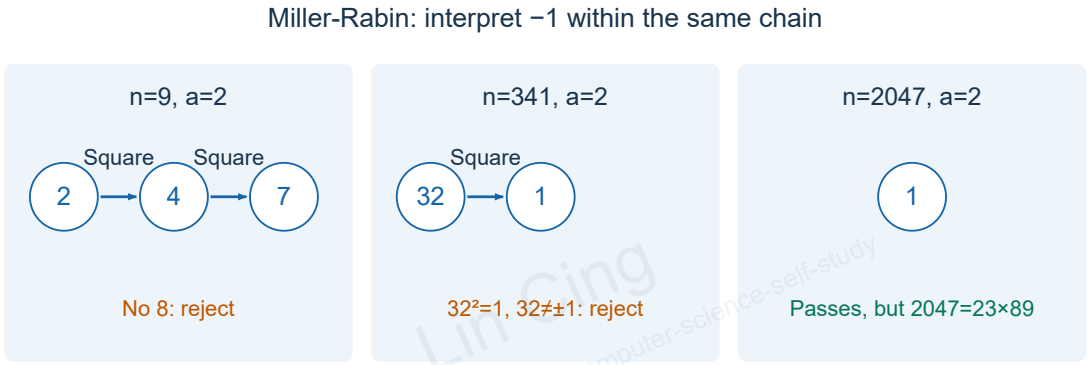
```
x = modular_power(a,d,n)
if x == 1 or x == n-1: pass this round
else:
    repeat s-1 times:
        x = x*x mod n
        if x == n-1: pass this round
    if no -1 occurred: return COMPOSITE
```

Repeat with independent bases and return probably prime only if every round passes. For prime n, Fermat gives $a^{n-1} = 1$. If the chain does not start at 1, the value before its first 1 must be -1, since a field has only square roots ± 1 of 1. Thus primes always pass, and every rejection proves compositeness. Without the special case for 3, sampling would use the empty interval $[2, 1]$.

Strong testing separates Fermat examples

n, a	Decomposition	Initial power	Later squares	Outcome
5,2	$4 = 2^2 \cdot 1$	2	$4 = -1$	Pass
9,2	$8 = 2^3 \cdot 1$	2	4,7	Reject, no -1
341,2	$340 = 2^2 \cdot 85$	32	1	Reject, nontrivial root 32
2047,2	$2046 = 2 \cdot 1023$	1	None needed	Pass, but $2047 = 23 \cdot 89$

Strong testing separates Fermat examples (continued)



Passing means probable prime. Repeating a fixed base does not amplify confidence.

A fixed single base cannot prove primality for general integers. The error is acceptance of a composite. Primes are never rejected. The proof below bounds random-base acceptance for each fixed composite.

Count roots in prime-power components

Fix odd composite $n = \prod_{i=1}^{r} p_i^{\alpha_i}$, with $n - 1 = 2^s d$. Write $p_i - 1 = 2^{t_i} u_i$, with odd u_i. Let $D_i = \gcd(d, u_i)$, $\tau = \min(s, t_1, \ldots, t_r)$. A passing base must be a unit. If $p_i \mid a$, every positive power in that component is zero, never either sign.

The group $\mathbb{F}_{p_i}^{\times}$ has a generator. Write $a = g^k$. The equation $a^d = 1$ has D_i roots. The equation $a^{2^j d} = -1$ becomes a linear congruence in k. The coefficient gcd is $2^{\min(j,t_i)} D_i$, which divides $(p_i - 1)/2$ exactly when $j < t_i$. In that case there are $2^j D_i$ roots.

Each root lifts uniquely to $p_i^{\alpha_i}$. If z solves $X^h - \eta = 0$ modulo p_i^a, try $z + cp_i^a$. At the next level solve

$$(z^h - \eta)/p_i^a + hz^{h-1}c \equiv 0 \pmod{p_i}.$$

For $h = 2^j d$, $p_i \nmid h$ because $p_i \nmid n - 1$. Also z is a unit. The derivative is nonzero, so c is unique. The root counts therefore hold at every prime-power component.

CRT counts synchronized strong liars

Passing cases are disjoint. Either $a^d = 1$, or a unique $j \in [0, s-1]$ satisfies $a^{2^j d} = -1$. Two levels cannot both be -1, because squaring the first gives 1 thereafter. CRT makes components independent, but every component must reach -1 at the **same level**.

Thus the exact strong-liar count in $[1, n-1]$ is

$$L = \left(1 + \sum_{j=0}^{\tau-1} 2^{jr}\right) \prod_i D_i.$$

The first term counts initial 1 in every component. The term for j counts synchronized -1 at that level. Combining components that reach -1 at different levels overcounts bases that the test rejects. Factorization is used for analysis. The algorithm does not need the factors.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

One-quarter bound with one or at least three factors

If $r = 1$, then $n = p^\alpha, \alpha \geq 2$. Since $p - 1 \mid p^\alpha - 1$, $\tau = t, D = u$, so $L = p - 1$. Therefore

$$\frac{L}{n-1} = \frac{p-1}{p^\alpha - 1} \leq \frac{1}{p+1} \leq \frac{1}{4}.$$

If $r \geq 3$, use $\varphi(n) = \prod_i p_i^{\alpha_i - 1} 2^{t_i} u_i$, $D_i \leq u_i$, and $t_i \geq \tau$ to get

$$\frac{L}{\varphi(n)} \leq \frac{1 + \sum_{j=0}^{\tau-1} 2^{jr}}{2^{r\tau}} = \frac{1}{2^r - 1} + \frac{2^r - 2}{(2^r - 1)2^{r\tau}}.$$

This decreases for $\tau \geq 1$. Its maximum at 1 is $2^{1-r} \leq 1/4$. Since $\varphi(n) \leq n - 1$, $L \leq (n-1)/4$. For $n = 9$, $L = 2 = (n-1)/4$, so this constant cannot be lowered on the full representative domain.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

The necessary two-factor case

For $r = 2$, if $t_1 \neq t_2$ or $s < \min(t_1, t_2)$, then $t_1 + t_2 \geq 2\tau + 1$. Thus

$$\frac{L}{\varphi(n)} \leq \frac{1 + \sum_{j=0}^{\tau-1} 4^j}{2^{2\tau+1}} = \frac{4^\tau + 2}{6 \cdot 4^\tau} \leq \frac{1}{4}.$$

Otherwise $t_1 = t_2 = \tau \leq s$. If some $\alpha_i \geq 2$, the extra prime-power factor in the totient is at least 3. If some $D_i < u_i$, the odd integer u_i/D_i is at least 3. At least one occurs. Otherwise $n = pq$ is squarefree and $D_i = u_i$, so both $p - 1, q - 1$ divide $n - 1$. Reducing $pq - 1$ modulo $p - 1$ gives $p - 1 \mid q - 1$. The reverse gives $q - 1 \mid p - 1$, forcing $p = q$, a contradiction.

The extra divisor of at least 3 gives $L/\varphi(n) \leq (4^\tau + 2)/(9 \cdot 4^\tau) \leq 1/6 < 1/4$. Every odd composite case is covered without an unspecified subgroup or Carmichael exception.

Sampling and posterior probability

Bases 1 and $n - 1$ always pass. Removing them from $L \leq (n - 1)/4$ gives $(L - 2)/(n - 3) \leq 1/4$ on $[2, n - 2]$. For each **fixed composite** n, R independent uniform rounds therefore accept with probability at most 4^{-R}. This is not directly the probability that an accepted candidate is composite.

If the candidate distribution has prior prime probability $\pi > 0$, prime pass probability is 1 and composite acceptance is at most $\epsilon = 4^{-R}$. Bayes gives

$$\Pr(\text{composite} \mid \text{pass}) \leq \frac{\epsilon(1 - \pi)}{\pi + \epsilon(1 - \pi)}.$$

The candidate distribution must supply π. An adversarial fixed input cannot be assigned a uniform prime density without evidence.

Each round uses $O(\log n)$ modular multiplications and ordinary bit cost $O(\beta^3)$. R rounds cost $O(R\beta^3)$, plus sampling. A fixed base list is deterministic only on a proved input range. Reusing one randomly selected base does not give independence. Independent sampling with replacement may happen to repeat a value and still satisfies the contract.

Factorization and Pollard rho

Proving compositeness need not reveal factors. Trial division finds them deterministically but can require $\sqrt{n}$ trials. Pollard rho seeks a collision modulo an unknown factor.

For $f(x) = x^2 + c \bmod n$ and $p \mid n$, reducing $f(x)$ modulo p equals applying the same polynomial to $x \bmod p$. If $x_i \equiv x_j \pmod{p}$, then $p \mid x_i - x_j$, so $d = \gcd(|x_i - x_j|, n) > 1$. If $x_i \not\equiv x_j \pmod{n}$, then $d < n$, yielding a proper factor.

This implication is deterministic. Verify a returned value by $1 < d < n, d \mid n$. The uncertain parts are the collision time and whether all components collide together. The latter gives $d = n$, which is not successful factorization.

Floyd and checkpoint schedules

```
x=y=x0
repeat until work budget exhausted:
    x=f(x); y=f(f(y))
    d=gcd(abs(x-y),n)
    if 1<d<n: return d
    if d==n: restart with another x0 or c
return no factor found within budget
```

Floyd advances the slow pointer by one and the fast pointer by two. A finite functional graph has tail length μ and cycle length λ. Once both are on the cycle, their relative position changes by one per round, so they meet within λ more rounds. Cost is $O(\mu + \lambda)$ function evaluations and a constant number of stored integers.

Another schedule fixes a checkpoint $y = x_{2^k}$ while x advances to the next power of two. Once the checkpoint is on the cycle and the interval is at least λ, the interval contains a matching component. This is a different schedule, so its trace must not be mixed with Floyd's. Both need a budget and a rule for $d = n$.

Three rounds factor 8051

Use $n = 8051, x_0 = 2, f(x) = x^2 + 1 \bmod 8051$, with Floyd's schedule.

Round	x	y	Absolute difference	gcd with 8051
1	5	26	21	1
2	26	7474	7448	1
3	677	871	194	97

In round 3, 677 and 871 both have residue 95 modulo 97, while their residues modulo 83 are 13 and 41. The collision yields 97 rather than 8051. Division gives 83. Trial division through their square roots confirms both are prime, completing $8051 = 83 \cdot 97$.

Three rounds factor 8051 (continued)

Floyd round 3: x=677, y=871, difference 194

mod97: collision

677 mod97 95

871 mod97 95

mod83: no collision yet

677 mod83 13

871 mod83 41

97 | 194, but 83 ∤ 194, so gcd(194,8051)=97

Check 8051 = 83×97. Success requires a proper divisor.

Birthday estimates and their assumptions

If iteration modulo unknown prime factor p behaves approximately as a uniform random mapping, it explores new points until the first collision. The probability of no collision in k steps is approximately

$$\prod_{j=0}^{k-1}(1 - j/p) \leq \exp(-k(k-1)/(2p)).$$

Typical collision scale is $\sqrt{p}$, and tail-cycle methods detect it in the same order of steps. A fixed polynomial $x^2 + c$ is not independent randomness. This assumption gives neither a worst-case time for that polynomial nor a success probability for each restart.

For balanced semiprimes, p is about $\sqrt{n}$, giving heuristic scale $n^{1/4} = 2^{\beta/4}$, still not polynomial in β. Every round also pays for a β-bit modular square and gcd. Using this chapter's safe coarse gcd bound gives $O(\beta^3)$, rather than constant cost.

Birthday estimates and their assumptions (continued)

After finding a factor, recursively factor both sides. Remove even factors first and specify the confidence of prime or probable-prime leaves. Exhausting a budget means no factor was found within it, not that the input is prime. The factor certificate is certain. Completion-time estimates remain conditional on the heuristic.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Sources

Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 3rd ed., MIT Press, 2009, Chapter 31.

M. O. Rabin, "Probabilistic Algorithm for Testing Primality," *Journal of Number Theory* 12(1), 1980, doi:10.1016/0022-314X(80)90084-0.

L. Monier, "Evaluation and Comparison of Two Efficient Probabilistic Primality Testing Algorithms," *Theoretical Computer Science* 12(1), 1980, doi:10.1016/0304-3975(80)90007-9.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

References and attribution

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Linear programming

Part VII Selected Topics

Scope

Equivalent models, exact pivots, and cycle prevention establish two-phase simplex. Dual solutions and improving rays certify its three possible outcomes.

Prerequisites

Convex polyhedra, standard form, and duality.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Chapter route

Learning objectives

- Convert models to standard and slack forms.
- Run simplex from a basic feasible solution.
- Use weak/strong duality and feasibility initialization.
- 29.1 Standard and slack forms
- 29.2 Formulating problems as linear programs
- 29.3 The simplex algorithm
- 29.4 Duality
- 29.5 The initial basic feasible solution

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

29.1 Standard and slack forms

Our standard form is $\max\ c^{\mathsf{T}}x,\ Ax \le b,\ x \ge 0$, with A∈R^(m×n). Constraints are non-strict and variables continuous real numbers. Finite binary-input bit complexity assumes rational coefficients. Other texts call equality form standard; check conventions first.

A feasible point satisfies all constraints; an optimum maximizes the objective over feasible points. Infeasible means no feasible point. Unbounded means feasible with arbitrarily large objective, not merely an unbounded feasible region. For example, max(−x) over x≥0 has an unbounded region and optimum 0.

Intersections of finitely many linear half-spaces are convex polyhedra:Ax≤b, Ay≤b imply A(λx+(1−λ)y)≤b for 0≤λ≤1. Convexity neither guarantees integer solutions nor makes nonlinear products linear constraints.

Four conversions must recover original variables

Convert minimizing f to maximizing−f and negate the returned objective. Multiply $a^{\mathsf{T}}x \ge b$ by−1. Split equality into opposite inequalities. Replace unrestricted u by $u^{+}-u^{-}$ with both nonnegative. Every u has representation(max(u,0), max(−u,0)); every such pair recovers an original value by subtraction. Although not one-to-one, the optimum values correspond.

For $\min\ 2u-v;\ u+v=3;\ u-2v\ge-3;\ v\ge0;\ u\ \text{free}$, maximize$-2u^{+}+2u^{-}+v$ subject to

$u^{+}-u^{-}+v\le3;\ -u^{+}+u^{-}-v\le-3;\ -u^{+}+u^{-}+2v\le3$.

Substituting u=3−v gives v≤2 and original objective 6−3v, so v=2, u=1 attains minimum 0, corresponding to maximum 0. Starting with n variables, m constraints, sign splitting uses at most 2n variables and equality splitting at most 2m constraints, adding only constant coefficient bit length.

Slack form, bases, and basic feasibility

Introduce $s=b-Ax\geq 0$. Let $M=[A\ I_m]$, $q=(x,s)\geq 0$, so $Mq=b$. Slacks are uniquely determined, not freely allocated independent decisions.

Choose m independent columns as basis B and let N be the others. Invertibility gives

$q_B=M_B^{-1}b-M_B^{-1}M_Nq_N;\ z=v+\rho_N^Tq_N$.

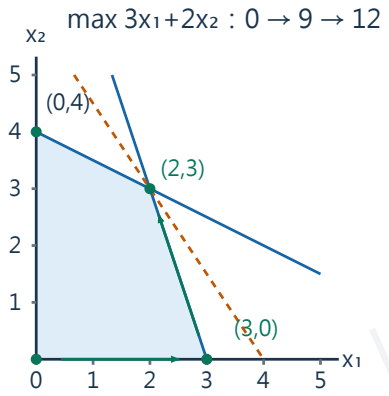
Set all nonbasic variables to 0 for a basic solution. It is feasible exactly when $M_B^{-1}b\geq 0$ componentwise. Initial slack basis I is feasible exactly when $b\geq 0$. Basic values need not be positive; zero basic variables cause degeneracy. A fixed basis determines a unique dictionary because the inverse is unique.

Geometry, level lines, and basis on one example

Use $\max\ 3x_1+2x_2;\ x_1+2x_2\leq 8;\ 3x_1+x_2\leq 9;\ x\geq 0$. Feasible vertices are(0,0),(3,0),(2,3),(0,4). The vertex path improves objective 0→9→12, ending at(2,3).

Initially $x_3=8-x_1-2x_2;\ x_4=9-3x_1-x_2;\ z=3x_1+2x_2$, with basis$\{x_3, x_4\}$ and slacks(8,9).

Geometry, level lines, and basis on one example (continued)



Blue: two resource constraints

$x_1+2x_2=8$

$3x_1+x_2=9$

Green: two simplex moves

Orange dashed: $3x_1+2x_2=12$

Intersection(2,3), objective 12

An entire edge can be optimal. For objective x_1+2x_2, the segment(0,4) to(2,3) has value 8. A vertex method returns one optimal basic solution without asserting uniqueness.

29.2 Formulating problems as linear programs

Specify each variable's meaning, unit, and domain. Map every model-feasible solution to an original plan and every original feasible plan back. Matching objectives while omitting capacity or conservation does not give equivalent models.

For shortest paths, use potentials d_v, fix d_s=0, and constrain d_v−d_u≤w_uv per edge. Maximize d_t. Potentials are unrestricted; negative paths can require d_t<0.

First assume all vertices are source-reachable, no negative cycle exists, and t is reachable. Summing inequalities along any s-to-t path gives d_t≤its length. Every feasible potential is therefore a lower bound on the shortest distance. Actual distances δ(s, v) satisfy all triangle inequalities and attain δ(s, t). Maximization, not minimization, finds the shortest distance.

Potential boundaries are not program Infinity values

Restricting to the s-reachable subgraph gives finite true distances when no negative cycle exists. Report unreachable t separately;∞ is not a real LP variable. Keeping an unreachable subgraph with a negative cycle instead makes the whole potential LP infeasible even if the source-to-target distance is ordinary. That assumption cannot be omitted.

If the entire graph has no negative cycle, a zero-edge super-source gives globally feasible potentials. For t unreachable from s, increase every unreachable vertex's potential by arbitrary $K>0$. No reachable-to-unreachable edge exists, so all differences stay feasible. Then d_t is unbounded, corresponding to unreachability, not a negative-length path.

For $s\to a:2$, $a\to t:-1$, $s\to t:4$, constraints are $d_a\le 2$, $d_t\le d_a-1$, $d_t\le 4$. Values(0,2,1) are feasible and the two-edge path bounds $d_t\le 1$, so optimum 1. Erroneously requiring all $d\ge 0$ would exclude correct solutions on other negative-total-path examples.

Flow models use nonnegative variables per original arc

Use arc flow g_{uv} rather than the preceding chapter's antisymmetric net flow. Constrain $0\le g_{uv}\le c_{uv}$, internal outflow minus inflow 0, and maximize net source outflow. Antiparallel original arcs each retain their own variable and capacity. Do not also impose $g_{uv}=-g_{vu}$.

Every original flow satisfies the model, and capacity/conservation make every model solution a flow. There are m variables and $O(n+m)$ constraints, without n^2 nonedge variables.

Minimum-cost flow additionally fixes source net outflow D and sink net inflow D, and minimizes $\Sigma a_{uv}g_{uv}$. Demand is exact; maximizing flow then reporting cost does not solve the same objective. Finite original capacities bound negative-cost circulation, which need not make the LP unbounded. Prohibiting circulation requires an additional model restriction.

Complete minimum-cost-flow example

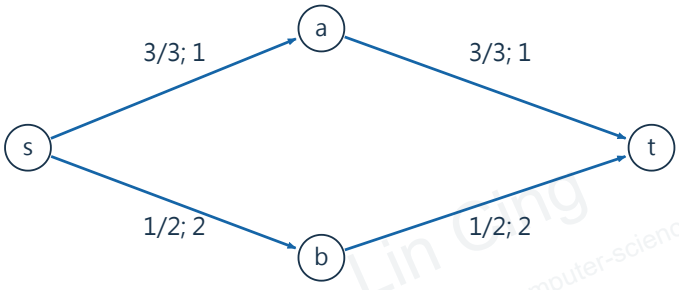
Arcs s→a, a→t each have capacity 3, unit cost 1. Arcs s→b, b→t each have capacity 2, unit cost 2. Demand D=4. Conservation makes route flows u, v, with u+v=4,0≤u≤3,0≤v≤2, and objective min 2u+4v.

Substitute v=4−u to get cost 16−2u and feasible 2≤u≤3. Thus u=3, v=1 costs 10. This specifies every arc flow, demand, feasible interval, and optimum.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Complete minimum-cost-flow example (continued)

Demand D=4. Labels: flow / capacity; unit cost.



u+v=4

2≤u≤3

Cost=16−2u

Optimum u=3, v=1

Upper cost 3×(1+1)=6; lower cost 1×(2+2)=4; total 10

Using u+v≤4 would incorrectly make zero flow at cost 0 optimal. Omitting u≤3 would incorrectly select u=4 at cost 8.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Multicommodity flow shares capacities

Commodity k has(s_k, t_k, D_k) and per-arc g^k_uv≥0. Each commodity separately conserves flow and meets endpoint net supplies±D_k. Shared capacity is Σ_kg^k_uv≤c_uv. A feasibility objective may be0; costs can also be included.

Two commodities each uniquely requiring a capacity 1 bottleneck u→v with demand 3/4 are individually feasible but jointly need 3/2>1. Independent max-flow solutions miss this shared resource.

With k commodities, m arcs, n vertices, there are km variables and O(kn+m) constraints, a polynomial-size LP for rational data. This models splittable flow. One-path-per-commodity or integral-flow requirements create different discrete problems, without the same general LP guarantees.

29.3 The simplex algorithm

Fix dictionary signs as `x_i=b_i-Σ_(j∈N)a_ijx_j; z=v+Σ_(j∈N)c_jx_j`. Entry a is the coefficient being subtracted, opposite the displayed signed coefficient. All ratios and pivots use this contract.

From feasible b≥0, choose c_e>0 to enter, set x_e=θ≥0, and keep other nonbasics 0. Basic x_i=b_i−a_ieθ. Rows a_ie≤0 impose no upper bound; positive a_ie requires θ≤b_i/a_ie. Choose the minimum nonnegative ratio with a positive denominator,**including zero**.

If no denominator is positive, direction q_e=1, q_i=−a_ie≥0, others 0 gives feasible ray x(θ)=x(0)+θq with objective v+c_eθ→∞. Do not write x_e=∞; infinity is not a feasible real value.

Pivoting retains the leaving variable's coefficient

For leaving l and a_le≠0, solve its row exactly:

$x_e=b_l/a_{le}-x_l/a_{le}-\Sigma_{(j\in N\setminus\{e\})}(a_{lj}/a_{le})x_j$.

Substitute into every other row and objective, then exchange B, N. Variable x_l becomes nonbasic, not absent. Omitting−x_l/a_le changes the feasible set.

New entries are a_el=1/a_le, b_e=b_l/a_le, a_ej=a_lj/a_le. For i≠l, b′_i=b_i−a_ieb_e, a′_il=−a_ie/a_le, a′_ij=a_ij−a_iea_lj/a_le. The objective becomes v′=v+c_eb_e, c′_l=−c_e/a_le, c′_j=c_j−c_ea_lj/a_le.

This invertible substitution preserves every point satisfying the equations and its objective, not merely the current basic point. With nonnegativity, it defines the same LP.

General feasibility proof for the ratio test

The new entering value b′_e=θ=b_l/a_le≥0. Other b′_i=b_i−a_ieθ stay nonnegative:for positive a_ie, the minimum ratio bounds θ; for nonpositive a_ie, the new value is at least b_i≥0. With new nonbasics 0, the new basic solution is feasible.

Objective improvement is c_eθ≥0, strict for θ>0. At θ=0, the point and objective may stay unchanged while the basis changes. This proves both why negative-denominator rows do not restrict the step and why zero ratios cannot be excluded.

```
while some reduced cost c[e]>0:
    choose entering e
    candidates={i in B : a[i, e]>0}
    if candidates empty: return feasible point + improving ray
    l=minimum-ratio row using a specified anti-cycling rule
    pivot(l, e)
return current basic solution
```

Two pivots on the same two-dimensional example

Initially B={3,4}. Enter x_1:ratios 8/1=8,9/3=3 make x_4 leave with θ=3:

$x_1=3-x_2/3-x_4/3;\ x_3=5-5x_2/3+x_4/3;\ z=9+x_2-x_4.$

Enter x_2:ratios 3/(1/3)=9,5/(5/3)=3 make x_3 leave:

$x_2=3-3x_3/5+x_4/5;\ x_1=2+x_3/5-2x_4/5;\ z=12-3x_3/5-4x_4/5.$

Nonbasic $x_3=x_4=0$ gives(2,3), tight in both constraints. Every reduced cost is nonpositive, so every feasible point has objective≤ 12. No further numerical search of the polygon is needed.

Zero steps are legal pivots

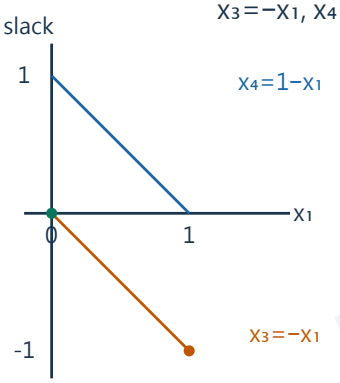
Use $\max\ z=x_1;\ x_3=-x_1;\ x_4=1-x_1$, all variables nonnegative. Initially B={3,4}, N={1}, basic values(0,1).

Basic row	Positive denominator	Ratio	At θ=0	Incorrect θ=1
x_3	1	0	0	−1, infeasible
x_4	1	1	1	0

Let x_3 leave. New $x_1=-x_3$, $x_4=1+x_3$, $z=-x_3$ has the same point(0,0,1), objective 0, but now nonpositive reduced costs prove optimality. Ignoring zero ratios immediately violates feasibility.

Zero steps are legal pivots (continued)

$x_3=-x_1$, $x_4=1-x_1$: do not skip a zero ratio



$x_1=0$: $x_3=0$, $x_4=1$, feasible

Ratios: 0/1=0 and 1/1=1

Legal step $\theta=0$; x_3 leaves

Wrong $\theta=1$: $x_3=-1$, infeasible

Only $x_1=0$ is feasible

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Finitely many bases alone do not prevent cycling

Matrix M=[A I] has m rows, and a basis selects m independent columns. There are at most C(n+m, m) possibilities. A fixed basis determines one dictionary, so an infinite run repeats a basis. Degenerate steps may preserve objective, so mere nondecrease does not forbid repetition.

Use exact lexicographic perturbation for a self-contained anti-cycling proof. From an initial feasible dictionary $q_B=\beta-Dq_N$, replace its RHS by $\beta_i+\varepsilon^i$ for i=1,…, m. Here ε is a positive infinitesimal,**not an arbitrary floating decimal**. Nonnegative constants and a positive first ε coefficient make the initial dictionary strictly feasible in the ordered field.

Represent comparisons by coefficient vectors:constant first, then $\varepsilon,\varepsilon^2,\ldots$; the first nonzero coefficient determines sign. Continue minimum-ratio tests with positive denominators, using later coefficients to resolve equal real ratios.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Why the lexicographic rule terminates

Let the initial equation matrix be $M_0=[I\ D]$, with RHS $\beta+\Sigma e_i\varepsilon^i$. For any invertible basis K, its basic values are

$M_0,\ K^{-1}\beta+\Sigma(M_0,\ K^{-1}e_i)\varepsilon^i$.

No inverse row is entirely zero, so no basic value is the identically-zero polynomial. Feasibility therefore makes every basic value strictly positive. Two positive-denominator ratios cannot coincide identically, or two inverse rows would be proportional, contradicting invertibility. Every legal θ is strictly positive, and $c_e>0$ makes the lexicographic objective strictly increase.

A fixed basis has one dictionary/objective, so it cannot repeat. Finite bases imply termination. Taking constant coefficients returns an original-LP feasible basic point. Reduced costs do not depend on RHS; nonpositive costs establish its optimality. An improving ray likewise has ε-independent direction and certifies original unboundedness from the constant feasible point. No guessed sufficiently-small floating ε is required.

A reproducible lexicographic tie

Use $x_3=2-x_1;\ x_4=2-x_1-x_2;\ \max\ x_1+x_2$, with RHS $2+\varepsilon,2+\varepsilon^2$. Entering x_1 compares $2+\varepsilon$ against $2+\varepsilon^2$; the latter is smaller, so x_4 leaves.

New rows are $x_1=2+\varepsilon^2-x_2-x_4;\ x_3=\varepsilon-\varepsilon^2+x_2+x_4;\ z=2+\varepsilon^2-x_4$. No reduced cost is positive. The constant solution is $x_1=2$, other variables 0, objective 2. Although x_3 has zero constant,$\varepsilon-\varepsilon^2$ is strictly positive and separates degeneracy symbolically.

Another common anti-cycling choice is Bland's rule:smallest-index positive reduced-cost entering variable, and smallest-index leaving variable among minimum-ratio ties. Both parts matter. The complete termination proof here uses lexicographic perturbation and does not claim arbitrary choices terminate.

Per-pivot cost is not total simplex cost

A dense ordinary dictionary updates O(mn) coefficients per pivot. This does not prove polynomial total time; basis count C(n+m, m) can be exponential. Lexicographic RHS adds m+1 coefficients per row and a polynomial update/comparison factor, but retains only a finite/exponential general worst-case pivot guarantee.

Rational exact pivots use fractions. Basis solutions are ratios of subdeterminants. After clearing original denominators, a determinant has at most m! terms, giving bit length bounded polynomially by dimension and input bit length. A single basis thus needs no unrestricted representation growth, but pivot count need not be polynomial.

Floating implementations need separate feasibility/reduced-cost tolerances, near-zero ratio handling, and residual recomputation. Clamping negative b to 0 can change the problem. Ellipsoid and interior-point methods are different algorithms; their polynomial theory does not follow from these pivot counts.

29.4 Duality

Primal $\max\ c^{\mathsf{T}}x,\ Ax \le b,\ x \ge 0$ has dual $\min\ b^{\mathsf{T}}y,\ A^{\mathsf{T}}y \ge c,\ y \ge 0$. Its m resource rows give m prices and n primal variables give n price inequalities. Transpose A; merely exchanging max/min is insufficient.

For feasible x, y, nonnegativity and both constraint systems give

$c^{\mathsf{T}}x \le x^{\mathsf{T}}A^{\mathsf{T}}y = y^{\mathsf{T}}Ax \le y^{\mathsf{T}}b$.

This is weak duality. A dual feasible point bounds every primal objective above. Equality certifies optimality on both sides. Equal objective numbers without checking feasibility are not a certificate.

Constructing a dual from the terminal basis

Let M=[A I], d=(c,0), and terminal basis matrix M_B be invertible. Define row $y^T=d_B^T M_B^{-1}$. Substituting basic variables gives

$z=y^T b+\Sigma_(j\in N)(d_j-y^T M_j)q_j$.

For every column, reduced cost ρ_j=d_j−yᵀM_j, and basic costs are 0. Terminal $\rho\le 0$ gives $-y_i\le 0$ from slack columns e_i, hence $y\ge 0$. Decision columns give $c_j-y^T A_j\le 0$, hence $A^T y\ge c$.

The current feasible basic objective is $y^T b$, attaining weak-duality equality. A finite optimal primal therefore has a same-value dual. Read y_i as the negative reduced cost of original slack i, or0 if that slack is basic. This follows from the full dictionary identity, not coefficient matching at only one point.

The same example's dual and complementary slackness

The two-dimensional example has dual $\min\ 8y_1+9y_2;\ y_1+3y_2\ge 3;\ 2y_1+y_2\ge 2;\ y\ge 0$. Terminal $z=12-3x_3/5-4x_4/5$ gives y=(3/5,4/5). Both dual rows are tight, and 24/5+36/5=12 matches x=(2,3).

The general gap decomposes exactly as

$b^T y-c^T x=y^T(b-Ax)+x^T(A^T y-c)$.

For feasible sides, every summand is nonnegative. Gap 0 is therefore equivalent to all products y_i(b_i−A_ix)=0 and x_j((Aᵀy)_j−c_j)=0. This proves necessity and sufficiency of complementary slackness. Positive primal slack forces its price 0; positive primal variable forces its dual row tight.

For $\max\ 2x_1+x_2;\ x_1+x_2\le 4;\ x_1\le 2;\ x\ge 0$, dual y=(1,1) gives upper bound 6. Point(2,2) has value 6 and all complementary products 0. Point(1,3) has value 5, with the sole positive product $y_2(2-x_1)=1$.

The same example's dual and complementary slackness (continued)

$$\text{gap} = y^T(b - Ax) + x^T(A^Ty - c) \geq 0$$

$x=(2,2)$

$y=(1,1)$, dual value 6

primal 6

$y_2(2-x_1) = 0$

Other three products are 0

$x'=(1,3)$

$y=(1,1)$, dual value 6

primal 5

$y_2(2-x_1) = 1$

Other three products are 0

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Unboundedness and infeasibility are not interchangeable

An unbounded primal has no feasible dual, or one fixed dual value would bound all primal values. A downward-unbounded dual similarly makes primal infeasible. The converse need not hold:an infeasible primal need not have an unbounded dual, because both can be infeasible.

Primal `max-x; 0x≤-1; x≥0` is infeasible; dual `min-y; 0y≥-1; y≥0` is feasible and downward-unbounded. Changing primal objective to max x gives dual constraint $0 \geq 1$, making both sides infeasible.

For other sign conventions, convert to standard first. A primal equality has a free dual multiplier; an unrestricted primal variable gives a dual equality. These rules follow by combining split nonnegative variables or opposite inequalities.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

29.5 The initial basic feasible solution

Negative entries in initial b prevent starting a ratio loop that assumes feasibility. For $Ax \leq b$, $x \geq 0$, add $a \geq 0$ and solve auxiliary $\max -a;\ Ax - a1 \leq b$. It is always feasible:take $x=0$, $a=\max(0,-\min_i b_i)$. Its objective is at most 0 and cannot be upward-unbounded.

Original feasibility allows the same x with $a=0$, attaining auxiliary optimum 0. Conversely, auxiliary optimum 0 gives $a=0$ and an original feasible x. A strictly negative optimum proves infeasibility. A currently positive artificial variable alone is insufficient; first establish auxiliary optimality.

When $m=0$, use the origin directly. A positive c_j gives an unbounded coordinate ray; otherwise the origin is optimal. No minimum over an empty row set is needed.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A special pivot initializes auxiliary feasibility

Write $s_i=b_i-A_ix+a$ and choose r with minimum $b_r<0$. Let a enter, s_r leave. The pivot coefficient is -1; this is **not** the improving ratio step from an already-feasible basis.

Solving gives $a=-b_r+A_rx+s_r$; other rows become $s_i=b_i-b_r+(A_r-A_i)x+s_r$. Set new nonbasic $x=s_r=0$. Then $a=-b_r>0$ and $s_i=b_i-b_r \geq 0$, establishing a feasible basis. Only now use simplex and lexicographic anti-cycling.

Start symbolic RHS perturbation at this feasible dictionary and take constant parts afterward. Reduced costs ignore RHS, so the result is the original auxiliary optimum and feasible point, rather than a false infeasibility judgment from an ε objective.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Zero artificial value still requires basis cleanup

At auxiliary optimum 0, delete a's column if it is nonbasic. If a is basic with value 0, its row is $a=-\Sigma a_{ajx_j}$. A nonzero coefficient permits that x_j to enter and a to leave. This zero-step algebraic pivot leaves the point unchanged; its coefficient may have either sign and it is not an improving ratio step.

If every coefficient is 0, the equation a=0 is redundant. Delete its row and a. General equality or multi-artificial implementations need this branch. Full independent slack rows can sometimes exclude it by rank, but never justify division by 0.

Finally substitute every currently basic variable appearing in the **original objective** to recover a nonbasic-only objective. Changing a title or retaining auxiliary−a does not restore the original problem. This completes Phase I; Phase II begins with that feasible basis and restored objective.

Complete Phase I and original-objective example

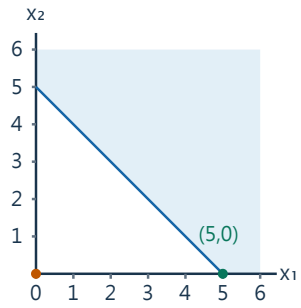
Original $\min\ z=2x_1+x_2;\ x_1+x_2\geq 5;\ x\geq 0$ uses surplus $x_3\geq 0$ with $x_1+x_2-x_3=5$. Add artificial $a=5-x_1-x_2+x_3$, minimizing w=a in Phase I, equivalent to maximizing−a.

State	Basic row	At nonbasics 0	Objective
Initial	$a=5-x_1-x_2+x_3$	a=5	w=5
x_1 enters, a leaves	$x_1=5-x_2+x_3-a$	$x_1=5$	w=a, min 0
Delete a; substitute original objective	$x_1=5-x_2+x_3$	$x_1=5$	$z=10-x_2+2x_3$
x_2 enters, x_1 leaves	$x_2=5-x_1+x_3$	$x_2=5$	$z=5+x_1+x_3$

Nonnegative final minimization coefficients prove optimum 5. Original origin(0,0) is infeasible; adding a=5 makes it feasible only for the auxiliary problem. This one-equation example uses one artificial column; the shared-a construction above covers arbitrary $Ax\leq b$ systems.

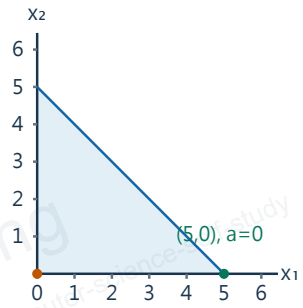
Complete Phase I and original-objective example (continued)

Original: $x_1+x_2\geq 5$



Origin infeasible

Auxiliary slice $x_3=0$: $a=5-x_1-x_2$



Origin with $a=5$ is feasible

Artificial variables change only the auxiliary problem. Setting $a=0$ restores original constraints.

Infeasibility needs a completed certificate

Original $x\leq 1$, $x\geq 2$, $x\geq 0$ becomes $x-a\leq 1$, $-x-a\leq -2$. Summing gives $-2a\leq -1$, so $a\geq 1/2$. Values $x=3/2$, $a=1/2$ satisfy both tightly, proving auxiliary max $-a=-1/2<0$ and original infeasibility.

The complete dictionary starts $s_1=1-x+a$, $s_2=-2+x+a$. Special a-entry, s_2-exit gives $a=2-x+s_2$, $s_1=3-2x+s_2$, $w=-2+x-s_2$. Enter x:ratios 2,3/2 make s_1 leave, producing

$x=3/2-s_1/2+s_2/2;\ a=1/2+s_1/2+s_2/2;\ w=-1/2-s_1/2-s_2/2.$

Both reduced costs are nonpositive, so this is truly optimal. A positive artificial amount alone would not suffice. The original inequalities also sum to $0\leq -1$, an independent contradiction certificate.

Three outcomes and the proof dependencies

Phase I starts from a feasible auxiliary basis. Lexicographic simplex terminates finitely, and auxiliary objective is bounded above. A negative optimum means infeasible; a zero optimum permits artificial cleanup and objective restoration.

Phase II also starts feasible and terminates under the lexicographic rule. No positive ratio denominator produces a feasible point plus improving ray, certifying unboundedness. Otherwise nonpositive terminal reduced costs produce a feasible basic point and same-value dual, certifying a finite attained optimum.

Thus every LP in this standard form is exactly one of infeasible, feasible with unbounded objective, or feasible with a finite attained optimum. Finite-dimensional structure and non-strict constraints matter. Problem $\max\ x;\ 0 \le x < 1$ has unattained supremum 1 and lies outside this trichotomy's assumptions.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

References

Bibliography

Chapter source

Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*,3rd ed., MIT Press,2009, Chapter 29.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

References and attribution

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Multithreaded algorithms

Part VII Selected Topics

Scope

Dynamic multithreading separates total work, critical paths, and scheduling through a dependency DAG. Spawn/sync semantics and race repairs lead to complete matrix-product and parallel-merge/sort algorithms, proofs, and space assumptions. Average parallelism does not define a processor-benefit cutoff.

Prerequisites

DAG scheduling, parallel cost models, and recurrence analysis.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Chapter route

Learning objectives

- Express dynamic multithreading with spawn and sync.
- Distinguish work, span, and parallelism.
- Analyze parallel limits of matrix multiplication and merge sort.
- 27.1 The basics of dynamic multithreading
- 27.2 Multithreaded matrix multiplication
- 27.3 Multithreaded merge sort

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

27.1 The basics of dynamic multithreading

Programs specify computations allowed to run concurrently; the scheduler decides when and on which processor they run. A logical task is not a permanently bound OS thread, and spawning does not promise an immediate second core.

Assume P identical ideal processors and sequentially consistent shared memory. Instructions interleave in an order respecting dependencies; each load/store is one step, but a combined read-add-write is not automatically atomic. Arithmetic, indexing, and spawn/sync have constant unit work. Scheduler overhead, bandwidth, caches, and communication are outside this ideal bound.

A nonempty finite computation is a DAG. Nodes can be strands, sequential instruction sequences without parallel control, with positive execution time. Remove empty strands; split positive integral strands into unit-instruction chains for unit-node proofs. Edges express control and data dependencies. Nodes merely drawn at the same level need not be independent.

Exact semantics of spawn, call, and sync

`x = spawn F(...)` allows F to run with the caller's continuation. An ordinary `y = G(...)` requires the caller to await G's return. A `sync` waits for all children previously spawned by this procedure; it is not a system-wide barrier.

Spawn arguments are evaluated in the parent first. Each child owns its call-local variables, but shared-array references can still lead several tasks to the same cell. Different variable names do not prove independence. Reading a spawned result requires a dependency, usually an explicit sync.

Procedure return has an implicit sync here. However, if `return x+y` evaluates its expression before returning, that implicit wait does not permit an earlier read of x. Explicitly sync before computing the return expression. Removing spawn/sync and replacing parallel for by ordinary for gives a serialization. Correct serialization alone does not prove every parallel interleaving correct.

Fibonacci code and correctness

```
P-FIB(n):                // nonnegative integer n
    if n <= 1: return n
    x = spawn P-FIB(n-1)
    y = P-FIB(n-2)
    sync
    return x+y
```

Children read their own n and return separate results, with no shared writes. Decreasing n ensures termination. Bases F0=0, F1=1 and induction, followed by synchronized addition, prove Fn. Spawning both children still requires waiting before addition and changes constants rather than work/span orders.

This version deliberately omits memoization and recomputes parameters. High parallelism does not imply work efficiency: sequential iteration uses O(n) arithmetic operations, whereas occupying more cores does not excuse exponential work.

Work and span in one explicit strand model

Split each nonbase call into three unit strands:pre through spawning, cont before the second ordinary call, and join for addition/return after sync. A base call is one unit strand. Pre precedes the left child and cont; cont precedes the right child; both child finishes precede join.

With total work W and longest dependency-path cost D,

$$W_n = W_{n-1} + W_{n-2} + 3, \qquad W_0 = W_1 = 1;$$
$$D_n = 2 + \max(D_{n-1}, 1 + D_{n-2}), \qquad D_0 = D_1 = 1.$$

n	Fn	Wn	Dn
2	1	5	4
3	2	9	6
4	3	17	8

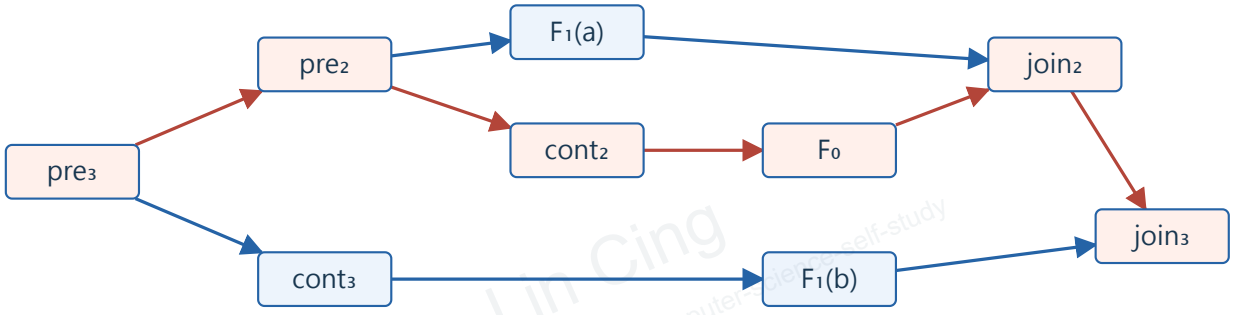
Work and span in one explicit strand model (continued)

Induction verifies Wn=4F(n+1)−3 and Dn=2n for n≥2. Let $\varphi = (1+\sqrt{5})/2$. Since ϕ²=ϕ+1, induction gives ϕ^(n−2)≤Fn≤ϕ^(n−1) for n≥2:check2,3, and add adjacent bounds to obtain the next power. Thus W=Θ(ϕ^n), D=Θ(n). A complete recursive call is not one O(1) node in this model.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Trace all dependencies among nine strands

All nine strands of P-FIB(3): arrows express dependencies



Each box is one unit strand: W=9. The red six-box critical path gives D=6.

Pre3 forks to pre2 and cont3; cont3 calls the right F1. Pre2 forks to its F1 and cont2; cont2 calls F0. Both bases precede join2, and join2 plus the other F1 precede join3.

Trace all dependencies among nine strands (continued)

The red path pre3→pre2→cont2→F0→join2→join3 has 6 nodes, so W=9, D=6. Path pre3→cont3→F1→join3 has only 4. Arrows express dependencies, not a single sequential chain through every node of a call.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Work law, span law, and achievable speedup

Write $W = T_1$,$D = T_\infty$, and actual P-processor time T_P. At most P units of work finish per time unit, and a critical path cannot parallelize, so

$$T_P \geq \max\{W/P, D\}.$$

The discrete unit-node model also gives $T_P \geq \lceil W/P \rceil$. Speedup $S_P = W/T_P \leq \min(P, W/D)$, and efficiency S_P/P≤1. Average parallelism A=W/D limits speedup. P>A precludes perfect P-fold acceleration, but does not imply processor P+1 has no benefit.

Sequential composition adds both W and D. Independent fork/join branches add work and take the maximum branch span, plus explicit fork/join costs. An empty computation has W=D=0 and time 0;0/0 is not defined parallelism.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Complete greedy-scheduling upper-bound proof

At each step execute P ready nodes if possible, otherwise execute all ready nodes. Let C be complete steps and I incomplete steps, T=C+I. Every incomplete step before completion executes at least one node, so W≥PC+I.

An incomplete step removes all sources of the remaining DAG. Any longest path remaining afterward can be extended backward in the previous DAG to at least one removed source. If nothing remains, the earlier span was at least 1. Thus each incomplete step decreases remaining span by at least 1, and I≤D. Consequently

$$T \leq (W - I)/P + I \leq (W - D)/P + D \leq W/P + D.$$

The substitution uses coefficient1−1/P≥0. Positive integral strand durations can be split into chains for this proof. It does not give the same guarantee in a different model with arbitrary indivisible tasks of unknown lengths.

Two-approximation and quantified slackness

An optimal scheduler also obeys $T_P^* \geq \max(W/P, D)$, so greedy T≤W/P+D≤2T*. The sharper bound is(2−1/P)T*. Constant-factor performance requires no future-DAG knowledge.

Define slackness $\sigma = W/(PD) = A/P$. For D>0, the bounds give

$$1 \leq T_P/(W/P) \leq 1 + (1 - 1/P)/\sigma \leq 1 + 1/\sigma.$$

Large σ brings ideal greedy time near W/P. This is a mathematical guarantee, not measured bandwidth or scheduler overhead. An optimal schedule can ignore excess processors, so its time cannot increase with P. Measured implementations also depend on overhead and scheduling choices.

Parallelism30 still benefits from processor 31

Complete697 independent unit tasks before starting a23-unit dependency chain. Then W=697+23=720, D=1+23=24, A=30.

P	Independent-stage allocation	Stage time	Chain time	Exact optimum
30	23 rounds of 30, then7	24	23	47
31	22 rounds of 31, then15	23	23	46

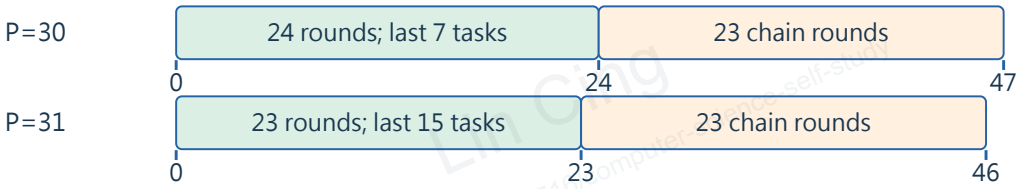
The chain cannot start until every predecessor finishes. Thus ceil(697/P)+23 is both an attainable schedule and a lower bound. Speedups720/47≈15.32 and 720/46≈15.65 both stay below30, yet the added processor helps. Only P≥697 makes the first stage one round and attains span24.

Parallelism30 still benefits from processor 31 (continued)

Same DAG: processor 31 still improves on 30

W=697+23=720
D=1+23=24

Average parallelism W/D=30. Both rows use the same time scale.



$T_{30}=47$, $T_{31}=46$. Speedup≤30 does not preclude benefit from processor 31.

Less work but greater span can reverse the winner

Two correct versions have(W, D)=(1024,4) and(768,16), both with ideal greedy scheduling.

At P=8, the first takes at least 1024/8=128; the second takes at most768/8+16=112, so the second is definitely faster. At P=128, the first takes at most1024/128+4=12; the second takes at least 16, so the first is faster.

Disjoint upper/lower bounds justify these conclusions. W/P+D is an upper bound, not exact time. Overlapping feasible-time intervals may leave the ordering unresolved from W, D alone. A single small-core measurement also cannot identify separate work and span changes.

Implementing parallel for by balanced fork/join

Use zero-based half-open intervals[l, r), with l=r empty, here and in merge algorithms.

```
PAR-FOR(l, r, body):
    if l == r: return
    if r-l == 1: body(l); return
    m = floor((l+r)/2)
    spawn PAR-FOR(l, m, body)
    PAR-FOR(m, r, body)
    sync
```

The n nonempty iterations give n leaves and n−1 internal nodes. Constant control work per internal node adds O(n), not n log n. The longest call/join chain has depth Θ(log n), so span is Θ(log n) plus the largest iteration span, provided iterations are independent.

An ordinary loop spawning n tasks serially has a Θ(n) issuing chain in its parent and cannot claim the same logarithmic span. The final sync also makes all results ready before the next phase reads them.

Matrix-vector multiplication safely parallelizes rows

```
MAT-VEC(A, x):
    n=A.rows
    allocate y[n]
    parallel for i=0 to n-1:
        y[i]=0
        for j=0 to n-1: y[i]+=A[i, j]*x[j]
    return y
```

Iteration i exclusively owns y[i], with A, x read-only. After inner step j, the entry is the sum of the first j+1 products, proving correctness. Work is $\Theta(n^2)$, span $\Theta(\log n)+\Theta(n)=\Theta(n)$, parallelism $\Theta(n)$. For n=0 return an empty vector.

Making the inner loop parallel as well causes unsynchronized read-modify-writes to one y[i], a determinacy race. Two logically parallel accesses conflict when they touch one cell and at least one writes. Even equal written values or one successful run does not prove independence.

Sequential consistency still permits lost updates

Lost updates: loads, local registers, and final stores

Initially x=0. Each task preserves load → add → store.

Step	Task 1	Task 2	Shared x
1	load $r_1 \leftarrow x = 0$		0
2		load $r_2 \leftarrow x = 0$	0
3	add $r_1 \leftarrow r_1+1 = 1$		0
4	store $x \leftarrow r_1$		1
5		add $r_2 \leftarrow r_2+1 = 1$	1
6		store $x \leftarrow r_2$	1

After sync both read 1. Waiting cannot make three instructions one atomic operation.

The interleaving respects each task's load→add→store but ends at x=1. Running one task completely before the other gives 2. Sequential consistency orders individual instructions; it does not make the three-instruction compound operation atomic.

Sequential consistency still permits lost updates (continued)

Use disjoint ownership or reduction to remove the race. Atomic additions or locks can impose dependencies, but their serialization, work, and span need separate accounting. They do not retain an analysis assuming all additions complete simultaneously.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A reduction parallelizes the dot product correctly

```
DOT(Arow, x, l, r):
    if l == r: return 0
    if r-l == 1: return Arow[l]*x[l]
    m=floor((l+r)/2)
    left=spawn DOT(Arow, x, l, m)
    right=DOT(Arow, x, m, r)
    sync
    return left+right
```

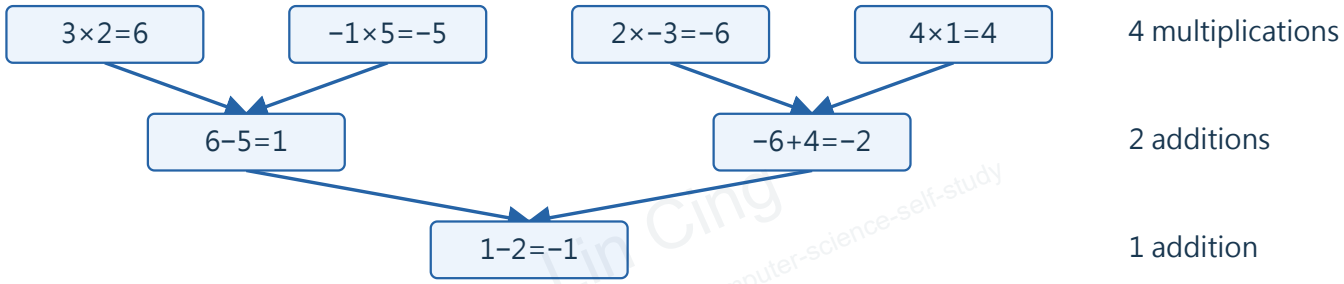
Children return separate values rather than sharing an accumulator. Associativity and induction give the interval dot product. Work is $\Theta(n)$, span $\Theta(\log(n+1))$. Replacing each MAT-VEC row by DOT gives $\Theta(n^2)$ work and $\Theta(\log n)$ span for $n \geq 2$.

For row=[3,−1,2,4], x=[2,5,−3,1], products are[6,−5,−6,4], first-level sums[1,−2], and result−1.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A reduction parallelizes the dot product correctly (continued)

Dot product: four private products combine in a tree



Arithmetic DAG: work=7, span=3. Fork/join adds Θ(log n) control depth.

Floating-point tree addition may round differently from sequential accumulation. The proof assumes exact associative arithmetic and does not promise bitwise-identical floating results.

Granularity and the allocation model

Group n constant-cost iterations into batches of at most g,$1 \le g \le n$, and issue batches with balanced PAR-FOR. Work is Θ(n), span $\Theta(\log\lceil n/g \rceil + g)$. Larger g reduces task overhead constants but increases leaf serial time; the tradeoff depends on P and slackness.

Here allocate obtains a descriptor for uninitialized, nonoverlapping storage at constant abstract span. If actual allocation initializes each cell, count that work and parallelize initialization to retain logarithmic span. Serial zeroing of n^2 cells is not hidden inside O(1).

Matrix blocks use views(row origin, column origin, stride, order), without hidden full-block copies. Task descriptors, allocators, caches, and memory bandwidth have implementation costs beyond the stated ideal model.

27.2 Multithreaded matrix multiplication

```
P-MULTIPLY(A, B):
    n=A.rows                // both n by n
    allocate C[n, n]
    parallel for i=0 to n-1:
        parallel for j=0 to n-1:
            C[i, j]=0
            for k=0 to n-1: C[i, j]+=A[i, k]*B[k, j]
    return C
```

Each(i, j) owns one output cell and accumulates its inner loop sequentially. Work is $\Theta(n^3)$. The two issuing trees contribute $\Theta(\log n)+\Theta(\log n)$, then the inner loop contributes $\Theta(n)$, so span $\Theta(n)$, parallelism $\Theta(n^2)$. Two parallel keywords do not remove each cell's n-term dependency.

Using a DOT-style reduction for each Cij preserves $\Theta(n^3)$ work but reduces span to $\Theta(\log n)$, giving $\Theta(n^3/\log n)$ parallelism. Concurrent writes to Cij are not this reduction algorithm.

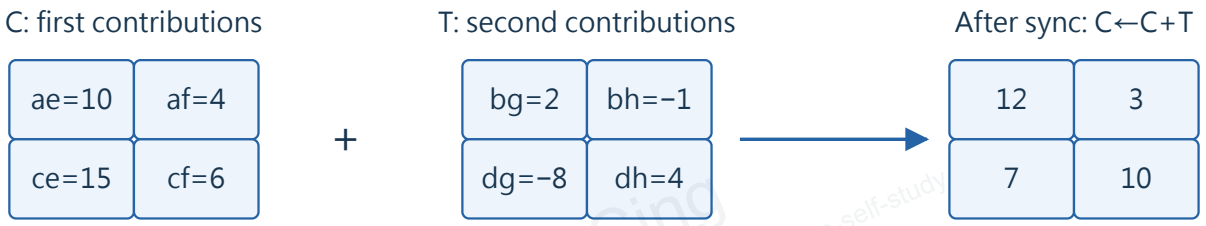
Eight subproducts need eight distinct destinations

Write A=[[a, b],[c, d]], B=[[e, f],[g, h]], where letters are equal-order blocks. Compute C=[[ae, af],[ce, cf]] and temporary T=[[bg, bh],[dg, dh]]. Every one of eight products has its own destination. Sync before C=C+T.

For A=[[2,−1],[3,4]], B=[[5,2],[−2,1]], temporary C=[[10,4],[15,6]], T=[[2,−1],[−8,4]], giving[[12,3],[7,10]].

Eight subproducts need eight distinct destinations (continued)

Eight independent products: separate destinations before addition



Each product writes one C or T cell. Await all products, then add cellwise.
A=[[2,−1],[3,4]], B=[[5,2],[−2,1]]. No simultaneous += contributions to one cell.

Two tasks executing C11+=ae and C11+=bg conflict. The temporary storage removes a shared-write dependency; spawning alone cannot do that.

Complete recursive matrix-product procedure

```
P-REC(C, A, B):
    n=A.rows
    if n == 1: C[0,0]=A[0,0]*B[0,0]; return
    allocate T[n, n]
    view A as [[a, b],[c, d]], B as [[e, f],[g, h]]
    view C, T as four n/2 by n/2 blocks
    spawn P-REC(C11, a, e); spawn P-REC(C12, a, f)
    spawn P-REC(C21, c, e); spawn P-REC(C22, c, f)
    spawn P-REC(T11, b, g); spawn P-REC(T12, b, h)
    spawn P-REC(T21, d, g); P-REC(T22, d, h)
    sync
    parallel for i=0 to n-1:
        parallel for j=0 to n-1: C[i, j]+=T[i, j]
```

Inputs are equal-order square matrices of power-of-two order, A, B read-only, and C disjoint from them. Pad any other positive order with zeros to the next power of two and extract the original result. Return an empty matrix for n=0. T and child workspaces are independent. Eight spawn/call operations are a fixed constant, not an n-long issuing chain.

Inductive correctness and eight-way costs

The base writes the correct scalar product. Inductively, eight half-size products are correct and race-free through distinct destinations. Sync makes them available before cellwise addition. Block multiplication identities then yield AB. Shared reads of A, B are harmless.

$$W_M(n) = 8W_M(n/2) + \Theta(n^2) = \Theta(n^3),$$
$$D_M(n) = D_M(n/2) + \Theta(\log n) = \Theta(\log^2 n).$$

Span follows only the longest child, not the sum of eight. Each level's nested parallel combination has Θ(log n) issuing/join depth, not O(1). For n=2^L, the sum1+…+L=Θ(L^2) gives both bounds. Order4 has 8^2=64 scalar-product leaves; parallelization has not removed multiplications.

Three dependency stages for parallel Strassen

Use Chapter 4's M numbering. First prepare block sums/differences(a+d),(e+h),(c+d),(f−h),(g−e),(a+b),(c−a),(e+f),(b−d),(g+h) with cellwise parallel operations. Once inputs are ready, run seven products:

$$\begin{aligned} M_1 &= (a+d)(e+h), & M_2 &= (c+d)e, \\ M_3 &= a(f-h), & M_4 &= d(g-e), \\ M_5 &= (a+b)h, & M_6 &= (c-a)(e+f), \\ M_7 &= (b-d)(g+h). \end{aligned}$$

Sync after all seven, then combine in parallel:C11=M1+M4−M5+M7, C12=M3+M5, C21=M2+M4, C22=M1−M2+M3+M6. Products cannot consume unfinished sums, and combination cannot consume unfinished products.

Strassen's work, span, and parallel workspace

Seven half-size children and a constant number of matrix sums give

$$W_S(n) = 7W_S(n/2) + \Theta(n^2) = \Theta(n^{\log_2 7}),$$
$$D_S(n) = D_S(n/2) + \Theta(\log n) = \Theta(\log^2 n).$$

Correctness follows from Chapter 4's expanded block identities and recursive induction, with the three additional parallel dependency stages. Parallelism $\Theta(n^{(\log_2 7)}/\log^2 n)$ is smaller than for eight-way multiplication but work is lower. Maximizing a parallelism ratio alone is not the objective.

Sequential depth-first $O(n^2)$ temporaries do not transfer to simultaneous full child expansion. If each call keeps its own $\Theta(n^2)$ workspace through join, eight-way space satisfies $Q(n)=8Q(n/2)+\Theta(n^2)=\Theta(n^3)$; seven-way space is $\Theta(n^{(\log_2 7)})$. Finite P and reuse can lower peaks, but require a consistent schedule/span analysis rather than an unimplemented space claim.

Executable specification of parallel Strassen

PADD(X, Y, s) allocates independent R and uses nested parallel loops to write $R[i, j]=X[i, j]+s \cdot Y[i, j]$, waiting before return. For $s=\pm 1$, work is $\Theta(n^2)$, span $\Theta(\log n)$. Pad equal-order A, B to powers of two; the outer wrapper handles n=0.

```
P-STRASSEN(A, B):
    n=A.rows
    if n == 1: return [[A[0,0]*B[0,0]]]
    view A as [[a, b],[c, d]], B as [[e, f],[g, h]]
    S1=PADD(a, d,+1); S2=PADD(e, h,+1)
    S3=PADD(c, d,+1); S4=PADD(f, h,-1)
    S5=PADD(g, e,-1); S6=PADD(a, b,+1)
    S7=PADD(c, a,-1); S8=PADD(e, f,+1)
    S9=PADD(b, d,-1); S10=PADD(g, h,+1)
    M1=spawn P-STRASSEN(S1, S2)
    M2=spawn P-STRASSEN(S3, e)
    M3=spawn P-STRASSEN(a, S4)
    M4=spawn P-STRASSEN(d, S5)
```

Executable specification of parallel Strassen (continued)

```
    M5=spawn P-STRASSEN(S6, h)
    M6=spawn P-STRASSEN(S7, S8)
    M7=P-STRASSEN(S9, S10)
    sync
    allocate C[n, n], viewed as four blocks
    parallel for i=0 to n/2-1:
        parallel for j=0 to n/2-1:
            C11[i, j]=M1[i, j]+M4[i, j]-M5[i, j]+M7[i, j]
            C12[i, j]=M3[i, j]+M5[i, j]
            C21[i, j]=M2[i, j]+M4[i, j]
            C22[i, j]=M1[i, j]-M2[i, j]+M3[i, j]+M6[i, j]
    return C
```

Ten PADD calls are sequential here, but their fixed count retains Θ(log n) preparation span. Finish all before spawning products, consistent with the three-stage contract. No arbitrarily long serial issuing chain is ignored.

27.3 Multithreaded merge sort

Parallel recursive sorts with a serial merge are insufficient:

```
PAR-SORT-SERIAL-MERGE(A, l, r):
    if r-l <= 1: return
    m=floor((l+r)/2)
    spawn PAR-SORT-SERIAL-MERGE(A, l, m)
    PAR-SORT-SERIAL-MERGE(A, m, r)
    sync
    serial MERGE the sorted halves using an auxiliary buffer
```

Work satisfies2W(n/2)+Θ(n)=Θ(n log n), but span D(n/2)+Θ(n)=Θ(n), giving only Θ(log n) parallelism. Sync before merge prevents reading unfinished runs. Reducing span further requires parallel merge itself.

Define half-open cuts and tie order first

Read-only T contains runs[l1, r1),[l2, r2). Output O[o, o+N), where N is total length, must not overwrite T. Compare records by(key, original position), a total order with unique original positions, so equal keys retain original order.

```
LOWER-BOUND(T, l, r, x):
    while l < r:
        m=floor((l+r)/2)
        if T[m] < x: l=m+1
        else: r=m
    return l
```

The original run's boundary between values<x and≥x remains in current[l, r]. Positions before current l are<x; positions at and beyond current r within the original run are≥x. Each comparison preserves a possible cut and strictly reduces interval length. An empty run returns l without reading nonexistent T[l]. Work and span are O(1+log(n2+1)), including constant empty handling.

Complete parallel merge, including empty runs

```
P-MERGE(T, l1, r1, l2, r2, O, o):
    n1=r1-l1; n2=r2-l2
    if n1 < n2: swap (l1, r1),(l2, r2); swap n1, n2
    if n1 == 0: return
    m=floor((l1+r1)/2)
    j=LOWER-BOUND(T, l2, r2, T[m])
    q=o+(m-l1)+(j-l2)
    O[q]=T[m]
    spawn P-MERGE(T, l1, m, l2, j, O, o)
    P-MERGE(T, m+1, r1, j, r2, O, q+1)
    sync
```

The longer run occupies the first descriptor. Swapping views neither moves nor relabels records. Empty first means both empty. Empty second still permits recursive median placement from the first without reading an empty interval. Each call removes one pivot, leaving total child size N−1, so recursion terminates.

General proof of position, correctness, and stability

There are m−l1 first-run and j−l2 second-run records smaller than the pivot, total q−o. Every remaining record is larger in the total order. The pivot therefore belongs at q. Child output ranges[o, q),[q+1, o+N) are disjoint from one another and from the pivot.

Induct on N. Empty merge is correct. Both children are smaller and correctly merge their designated records by induction. Concatenating left result, pivot, right result is sorted and contains every input record exactly once. T is read-only and output ranges have exclusive writers, so no determinacy race occurs. Sync makes the complete answer available.

Original positions order equal keys, proving stability. A key-only comparison with the same lower_bound rule after swapping longer-run roles can place an equal right-run record first. Correct key sorting alone does not prove stability.

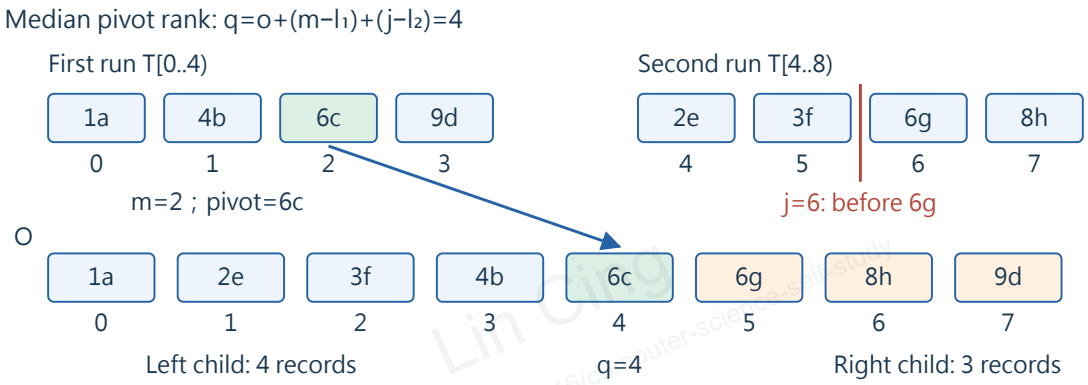
Root cut and binary-search steps for eight records

Use T=[1a,4b,6c,9d | 2e,3f,6g,8h], where letters a..h identify original positions0..7. Compare keys first, then positions. Set l1=0, r1=4, l2=4, r2=8, o=0. The first length 4 run has m=2, pivot6c.

Search[l, r)	mid	Comparison with 6c	Update
[4,8)	6	6g>6c: equal key, later position	r=6
[4,6)	5	3f<6c	l=6
[6,6)	none	empty interval	return j=6

Position q=0+(2−0)+(6−4)=4. Left child contains 4 records, right child3.

Root cut and binary-search steps for eight records (continued)



Compare(key, original position): 6c<6g. Left, pivot, right outputs are disjoint.

Every output write on the same example

One legal depth-first serialization writes pivots in the following order. Parallel interleavings may differ, but destinations stay fixed and each is written once.

Write	Pivot	q, zero-based	Meaning
1	6c	4	Root division
2	4b	3	Last record of root's left region
3	3f	2	Median after swapping longer run
4	2e	1	Small-run cut
5	1a	0	Other run empty
6	8h	6	Median after right-region swap
7	6g	5	Right region's left child
8	9d	7	Right region's right child

Every output write on the same example (continued)

Final O=[1a,2e,3f,4b,6c,6g,8h,9d], with c still before g. Running the right child first gives the same answer because T is read-only and every destination unique. Sortedness does not rely on a global sequential write order.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Upper and lower bounds on worst-case merge span

For n1≥n2, either child has size at most⌊n1/2⌋+n2≤(N+n2)/2≤3N/4, even if the other cut lands at an endpoint. Binary search costs O(log(N+1)), so

$$D_M(N) \le D_M(3N/4) + O(\log(N+1)) = O(\log^2(N+1)).$$

Constant-factor shrinkage gives O(log N) levels with logarithmic sizes O(log N−cr); their arithmetic sum gives the square.

For a lower bound, merge interleaved runs[0,2,…,2m−2] and[1,3,…,2m−1], with power-of-two m. The left child retains the same equal-length interleaving structure, halving length each level. Its other-run median search takes Θ(log m) comparisons. Summing log m,(log m−1),… gives Ω(log²m), hence worst-case Θ(log²N). If the other run stays empty, search costs constant and span is only Θ(log N). Worst-case order is not exact cost for every input.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Writing each record once does not pay for searches

Every nonempty call removes a distinct pivot, so there are N nonempty calls. Each creates two children, for 2N+1 total calls including empty returns. The N output writes give Ω(N), but an O(N) upper bound must also pay each binary search.

A size-m node has nonrecursive cost at most c log(m+1). Divide it among its m input records, charging each c log(m+1)/m. A fixed record x is charged along one ancestor chain until chosen as pivot.

Count r levels upward from the last call containing x. The 3/4 child bound makes that ancestor's size at least(4/3)^r. Sum these charges below, including empty children, without taking log0 or falsely assuming both small children have size at least N/4.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Each record's total search charge is bounded

Function $g(u) = \log_2(u+1)/u$ decreases for u>0. The sign of its derivative numerator is u/(u+1)−ln(u+1)<0, using $\int_0^u (1+t)^{-1}dt > u/(1+u)$. Thus the charge r levels upward is at most

$$c\frac{\log_2((4/3)^r+1)}{(4/3)^r} \le c[1 + r\log_2(4/3)](3/4)^r.$$

Both Σq^r=1/(1−q) and Σr q^r=q/(1−q)² converge at q=3/4; the latter follows by differentiating finite sums and taking limits. Each record pays O(1), for O(N) total. The N+1 empty calls add constant work each. With the output lower bound, W_M(N)=Θ(N).

This proof covers arbitrary legal cuts, empty runs, and unbalanced small children. A finite example is not the upper-bound proof.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Complete two-buffer parallel merge sort

Attach original positions to records and initialize separate arrays X, Y of length n to the same input, using one parallel copy. Call SORT(X, Y,0, n), then read Y.

```
SORT(src, dst, l, r):
    if l == r: return
    if r-l == 1: dst[l]=src[l]; return
    m=floor((l+r)/2)
    spawn SORT(dst, src, l, m)
    SORT(dst, src, m, r)
    sync
    P-MERGE(src, l, m, m, r, dst, l)
```

Children swap buffer roles, leaving their sorted runs in the parent's src. Sync before merging into dst. P-MERGE input and output must be different physical buffers. Two global buffers avoid allocating a new length-n temporary per call and avoid overwriting unread input.

Buffer-swap correctness and race checks

At each SORT entry, both buffers' current intervals still contain that interval's original input. The root copy establishes this. Before launching children, the parent has not altered its interval, and each child touches only its disjoint half, so the entry property holds recursively.

A singleton copies to dst. By induction, children leave two sorted unchanged-multiset runs in parent src. After sync, P-MERGE stably merges them to dst. The parent need not promise src equals dst on exit; the equality condition is needed only on entry.

Even with role swaps, children write disjoint index ranges of the same two arrays. Parent merging waits for both. Data workspace is O(n), peak task descriptors O(n), and each call chain O(log n). Initial copying takes Θ(n) work,Θ(log n) span, without changing sorting orders.

A tight sort bound needs one compatible worst-case input

Every sort level has Θ(n) merge work and there are Θ(log n) levels, so W_S(n)=Θ(n log n). Span obeys

$$D_S(n) \le D_S(\lceil n/2 \rceil) + O(\log^2 n) = O(\log^3 n).$$

For the lower bound, the same input must force expensive merges at all relevant levels. For n=2^L, place keys in L-bit-reversed index order; for n=8, use[0,4,2,6,1,5,3,7]. Root halves sort into interleaved even and odd runs. Each child's relative ranks have the same construction. Every level therefore has an Ω(log²(size)) merge, and parent merging starts only after both children finish.

Along a child chain,Σ_(r=1..L)r²=Θ(L³), giving worst-case Θ(log³n) for powers of two. Average parallelism is Θ(n/log²n). Merge's squared-log span is not the sort's span.

Compatible expensive levels for nonpowers of two

For any n, recursively assign relative ranks. If n is even, put even ranks in the first half and odd in the second. If odd, put odd ranks in the shorter first half and even in the longer second. Recursively arrange each half's ordered key set the same way. Every sorted pair of runs interleaves with length difference at most1. For powers of two, this is bit reversal.

At each such merge, the longer-run median has m same-run records on its left, and m or m+1 other-run records. The left child remains interleaved with length difference at most1 and total size between N/2−1 and N/2+1. For N≥8, both runs have Θ(N) size. The selected path has Θ(log N) levels and logarithmic search cost at each shrinking scale, giving Θ(log²N) span.

The same recursive rank arrangement gives this property at every sort node. Follow the larger child, whose size roughly halves. Its completion precedes parent merge, so critical paths concatenate and give Ω(log³n). This supplies the tight lower bound for arbitrary n, without choosing incompatible worst inputs separately at different levels.

Compare work and worst span in one model

Algorithm	Work	Worst span	Average parallelism
Recursive P-FIB	$\Theta(\varphi^n)$	$\Theta(n)$	$\Theta(\varphi^n/n)$
Row-parallel matrix-vector	$\Theta(n^2)$	$\Theta(n)$	$\Theta(n)$
Reduction matrix-vector	$\Theta(n^2)$	$\Theta(\log n)$	$\Theta(n^2/\log n)$
Two outer parallel loops for matrix product	$\Theta(n^3)$	$\Theta(n)$	$\Theta(n^2)$
Eight-way recursive matrix product	$\Theta(n^3)$	$\Theta(\log^2 n)$	$\Theta(n^3/\log^2 n)$
Parallel Strassen	$\Theta(n^{\log_2 7})$	$\Theta(\log^2 n)$	$\Theta(n^{\log_2 7}/\log^2 n)$
DOT-reduction matrix product	$\Theta(n^3)$	$\Theta(\log n)$	$\Theta(n^3/\log n)$
Parallel merge	$\Theta(n)$	$\Theta(\log^2 n)$	$\Theta(n/\log^2 n)$
Parallel merge sort	$\Theta(n \log n)$	$\Theta(\log^3 n)$	$\Theta(n/\log^2 n)$

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Compare work and worst span in one model (continued)

Asymptotic entries assume sufficiently large n; empty and singleton cases use explicit bases. Work efficiency, small span, and low space are distinct objectives. For fixed P, compare W/P, D, and memory together rather than substituting one parallelism ratio for the full analysis.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

References

Bibliography

Chapter source

Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*,3rd ed., MIT Press,2009, Chapter 27; §§27.1–27.3.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

References and attribution

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Summations

Part VIII Appendix

Scope

Derive arithmetic, power, geometric, telescoping, and product identities from finite sums. Analyze loops with fixed-constant induction, tail ratios, dyadic blocks, and integral bounds. State convergence assumptions before using infinite series or changing summation order.

Prerequisites

Arithmetic and geometric sequences, logarithms, and integral estimates.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Appendix route

Learning objectives

- Use summation formulas and linearity.
- Bound sums by integration, splitting, and geometric series.
- Distinguish finite sums from infinite series.
- A.1 Summation formulas and properties
- A.2 Bounding summations

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Summation formulas and properties

The finite sum $\sum_{k=1}^{n} a_k$ equals 0 when $n = 0$. The finite product $\prod_{k=1}^{n} a_k$ equals 1 when $n = 0$. These empty expressions use the additive and multiplicative identities, so formulas also cover loops with zero iterations. Finite terms may be reordered arbitrarily, and

$$\sum_{k=1}^{n}(ca_k + b_k) = c\sum_{k=1}^{n} a_k + \sum_{k=1}^{n} b_k.$$

An infinite series $\sum_{k=1}^{\infty} a_k$ is the limit of its partial sums S_n. It is not obtained by substituting ∞ into a finite formula. If the partial sums have no finite limit, the series diverges. Convergence requires $a_k \to 0$, because $a_k = S_k - S_{k-1}$. That condition is insufficient. The terms $1/k$ tend to zero, but the harmonic series diverges.

Absolute convergence means that $\sum_{k=1}^{\infty} |a_k|$ converges. It permits arbitrary reordering without changing the sum. A conditionally convergent series, such as $\sum_{k=1}^{\infty}(-1)^{k+1}/k$, need not preserve its sum under reordering, which changes partial-sum cancellation. Termwise calculus below requires a convergence interval. Finite linearity does not justify arbitrary infinite operations.

Arithmetic sums by pairing and induction

Write $S_n = 1 + 2 + \cdots + n$ and reverse the order to obtain $n + (n-1) + \cdots + 1$. Adding the two expressions gives $2S_n = n(n+1)$, so

$$S_n = \frac{n(n+1)}{2}.$$

The formula includes $n = 0$. Induction gives a second verification. The base case is $S_0 = 0$, and the induction step is

$$S_{n+1} = \frac{n(n+1)}{2} + (n+1) = \frac{(n+1)(n+2)}{2}.$$

The induction coefficient stays fixed as n changes. More generally,

$$\sum_{j=0}^{n-1}(a + jd) = na + \frac{dn(n-1)}{2}.$$

A loop with $j = 3, \ldots, 7$ that performs $2j - 1$ units of work has total cost

Arithmetic sums by pairing and induction (continued)

$$\begin{aligned}\sum_{j=3}^{7}(2j-1) &= \left(2\sum_{j=1}^{7}j-7\right)-\left(2\sum_{j=1}^{2}j-2\right)\\ &= 49-4=45\\ &= 5+7+9+11+13.\end{aligned}$$

Changing the starting index requires subtracting the correct prefix. The upper index 7 is not the number of terms, which is 5.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Square and cube sums from telescoping

Sum $(k+1)^3-k^3=3k^2+3k+1$ for $k=0,\dots,n$. The left side telescopes, giving

$$(n+1)^3=3\sum_{k=0}^{n}k^2+\frac{3n(n+1)}{2}+(n+1).$$

Solving for the sum gives

$$\sum_{k=0}^{n}k^2=\frac{n(n+1)(2n+1)}{6}.$$

Similarly, sum $(k+1)^4-k^4=4k^3+6k^2+4k+1$ and substitute the preceding square and arithmetic sums:

$$\begin{aligned}(n+1)^4 &= 4\sum_{k=0}^{n}k^3+6\sum_{k=0}^{n}k^2+4\sum_{k=0}^{n}k+(n+1),\\ \sum_{k=0}^{n}k^3 &= \frac{n^2(n+1)^2}{4}=\left(\frac{n(n+1)}{2}\right)^2.\end{aligned}$$

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Square and cube sums from telescoping (continued)

Both formulas hold at $n = 0$. At $n = 4$, the checks are

$$0 + 1 + 4 + 9 + 16 = 30 = \frac{4 \cdot 5 \cdot 9}{6},$$
$$0 + 1 + 8 + 27 + 64 = 100 = \left(\frac{4 \cdot 5}{2}\right)^2.$$

Telescoping proves the formulas for general n. The small cases check arithmetic and transcription.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Finite, infinite, and boundary geometric sums

Let $G_n(x) = \sum_{k=0}^{n} x^k$. Subtract G_n from xG_n. Interior terms cancel, so

$$(x - 1)G_n(x) = x^{n+1} - 1.$$

For $x \neq 1$, this gives $G_n(x) = (x^{n+1} - 1)/(x - 1)$. At $x = 1$, evaluate separately to obtain $G_n(1) = n + 1$; division by zero is invalid.

If $|x| < 1$, then $x^{n+1} \to 0$, and

$$\sum_{k=0}^{\infty} x^k = \frac{1}{1 - x}.$$

At $x = 1$, partial sums are unbounded. At $x = -1$, they alternate between 1 and 0. If $|x| > 1$, the terms do not tend to zero.

For $x = -1/2$ and $n = 3$, the finite sum is $1 - 1/2 + 1/4 - 1/8 = 5/8$, whereas the infinite sum is $2/3$. When $|x| < 1$, the exact tail and its absolute bound are

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Finite, infinite, and boundary geometric sums (continued)

$$\sum_{k=n+1}^{\infty} x^k = \frac{x^{n+1}}{1-x}, \qquad \left|\frac{x^{n+1}}{1-x}\right| \le \frac{|x|^{n+1}}{1-|x|}.$$

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Weighted geometric sums and valid calculus

Finite polynomials may be differentiated term by term. A direct derivation starts with $W_n = \sum_{k=0}^{n} kx^k$. Subtracting xW_n gives

$$(1-x)W_n = x + x^2 + \cdots + x^n - nx^{n+1}.$$

Substitute the finite geometric formula and rearrange:

$$W_n = \frac{x - (n+1)x^{n+1} + nx^{n+2}}{(1-x)^2}, \qquad x \neq 1.$$

At $x = 1$, $W_n = n(n+1)/2$. For $|x| < 1$, $n|x|^n \to 0$, because an exponential with a fixed base dominates a polynomial. Taking the limit gives

$$\sum_{k=0}^{\infty} kx^k = \frac{x}{(1-x)^2}.$$

This derivation does not interchange differentiation with an infinite sum without proof. Power series permit termwise differentiation and integration inside their radius of convergence, which is 1 here. Endpoints require separate checks.

Weighted geometric sums and valid calculus (continued)

Apply $x\,d/dx$ once more inside that radius:

$$\sum_{k=0}^{\infty} k^2 x^k = \frac{x(1+x)}{(1-x)^3}.$$

Thus $x = 1/2$ gives $\sum_{k=0}^{\infty} k/2^k = 2$ and $\sum_{k=0}^{\infty} k^2/2^k = 6$. Terms tending to zero alone would not establish these convergent sums.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Telescoping leaves boundary terms

Each interior term occurs once with each sign, so

$$\sum_{k=1}^{n} (a_k - a_{k-1}) = a_n - a_0.$$

Align indices and endpoints before cancellation. For example,

$$\frac{1}{k(k+2)} = \frac{1}{2}\left(\frac{1}{k} - \frac{1}{k+2}\right),$$

which gives

$$\sum_{k=1}^{n} \frac{1}{k(k+2)} = \frac{1}{2}\left(1 + \frac{1}{2} - \frac{1}{n+1} - \frac{1}{n+2}\right).$$

At $n = 3$, $1/3 + 1/8 + 1/15 = 21/40$, matching $\frac{1}{2}(3/2 - 1/4 - 1/5)$. Letting $n \to \infty$ gives $3/4$, since the terminal denominators tend to infinity.

Products telescope as well. For $n \geq 2$,

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Telescoping leaves boundary terms (continued)

$$\prod_{k=2}^{n}\left(1-\frac{1}{k^2}\right)=\left(\prod_{k=2}^{n}\frac{k-1}{k}\right)\left(\prod_{k=2}^{n}\frac{k+1}{k}\right)$$
$$=\frac{1}{n}\frac{n+1}{2}=\frac{n+1}{2n}.$$

At $n=1$, the empty product is 1, as the formula also predicts.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Products as sums and indispensable premises

If every $a_k>0$, then

$$\ln\left(\prod_{k=1}^{n}a_k\right)=\sum_{k=1}^{n}\ln a_k.$$

A zero or negative factor has no real logarithm. For example,

$$\prod_{k=1}^{n}2^{3k+1}=2^{\sum_{k=1}^{n}(3k+1)}=2^{3n(n+1)/2+n}.$$

For $0\le u_k<1$, logarithm bounds give

$$-\frac{u_k}{1-u_k}\le\ln(1-u_k)\le-u_k.$$

These bounds control products through sums. As u_k approaches 1, the lower-bound denominator cannot be treated as a fixed constant.

If fixed constants satisfy $0<c_1f(k)\le h(k)\le c_2f(k)$ for every $k\ge1$, summing gives

Products as sums and indispensable premises (continued)

$$c_1 \sum_{k=1}^{n} f(k) \le \sum_{k=1}^{n} h(k) \le c_2 \sum_{k=1}^{n} f(k).$$

If the inequalities hold only on a tail, retain the finite prefix separately. When $\sum_{k=1}^{n} f(k) \to \infty$ and the prefix is finite, the prefix does not change the Θ order. Constants must be uniform over k. An unbounded choice of a different constant at every index proves no common Θ bound.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Bounding summations

Decide whether an exact value, an upper bound, or a two-sided order is needed. Induction, termwise bounds, fixed ratios, splitting, and integration serve different purposes. A rough upper bound alone does not establish tightness.

For $S_n = \sum_{k=0}^{n} 4^k$, seek $S_n \le c4^n$. The base case $n = 0$ requires $c \ge 1$. The induction step gives

$$S_{n+1} \le c4^n + 4^{n+1} = \left(\frac{c}{4} + 1\right) 4^{n+1}.$$

It closes when $c/4 + 1 \le c$, or $c \ge 4/3$. The fixed choice $c = 4/3$ works for all n. The last term gives $S_n \ge 4^n$, so $S_n = \Theta(4^n)$.

The false induction claiming that $\sum_{k=1}^{n} k = O(n)$ remains linear after adding $n + 1$ hides a growing constant. An assumption $\sum_{k=1}^{n} k \le cn$ would require

$$cn + n + 1 \le c(n + 1), \qquad \text{or} \qquad n + 1 \le c.$$

No fixed c satisfies this for every n.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Maximum-term upper bounds and tail lower bounds

For nonnegative a_k and any subset $J \subseteq \{1, \dots, n\}$,

$$\sum_{k=1}^{n} a_k \le n \max_{1 \le k \le n} a_k, \qquad \sum_{k=1}^{n} a_k \ge \sum_{k \in J} a_k.$$

Upper bounds may use all terms. Lower bounds often retain enough large terms. For example, $\sum_{k=1}^{n} k \le n^2$. The indices $\lceil n/2 \rceil, \dots, n$ contain at least $n/2$ terms, each at least $n/2$. Thus $\sum_{k=1}^{n} k \ge n^2/4$ for both odd and even n, proving $\Theta(n^2)$ without an exact formula.

For a fixed real $r \ge 0$, the same argument gives

$$2^{-r-1} n^{r+1} \le \sum_{k=1}^{n} k^r \le n^{r+1}.$$

Hence the sum is $\Theta(n^{r+1})$. Fixing r is essential. If $r = n$, the lower coefficient 2^{-r-1} depends on n, and this proof gives no uniform order.

A geometric tail requires a fixed ratio

Suppose $a_k \ge 0$ and a fixed $0 < \rho < 1$ satisfies $a_{k+1} \le \rho a_k$ for every $k \ge K$. Induction gives $a_{K+j} \le a_K \rho^j$, so

$$\sum_{k=K}^{\infty} a_k \le \frac{a_K}{1 - \rho}.$$

The multiplicative inequality also covers zero terms without division by zero.

For $a_k = k^2/3^k$ and $k \ge 2$,

$$\frac{a_{k+1}}{a_k} = \frac{1}{3}\left(1 + \frac{1}{k}\right)^2 \le \frac{3}{4}.$$

The prefix at $k = 0, 1$ contributes $1/3$. The tail starts at $a_2 = 4/9$, giving

$$\sum_{k=2}^{\infty} a_k \le \frac{4/9}{1/4} = \frac{16}{9}, \qquad \sum_{k=0}^{\infty} a_k \le \frac{19}{9}.$$

A geometric tail requires a fixed ratio (continued)

The exact weighted geometric formula gives $3/2$, a sharper finite bound. Harmonic ratios $k/(k+1) < 1$ tend to 1, so no fixed $\rho < 1$ bounds the whole tail. A prefix that depends on n, such as two terms each equal to n^2, is not $O(1)$ merely because it contains only two terms.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Dyadic blocks reveal harmonic layers

Let $H_n = \sum_{k=1}^{n} 1/k$. The block $k = 2^j, \dots, 2^{j+1} - 1$ has 2^j terms, each between $1/2^{j+1}$ and $1/2^j$. Each complete block contributes between $1/2$ and 1.

Complete the last partial block for an upper bound. There are $\lfloor \log_2 n \rfloor + 1$ blocks, each contributing at most 1. The first $\lfloor \log_2 n \rfloor$ complete blocks give a lower bound. For $n \geq 2$,

$$\frac{\lfloor \log_2 n \rfloor}{2} \leq H_n \leq \lfloor \log_2 n \rfloor + 1.$$

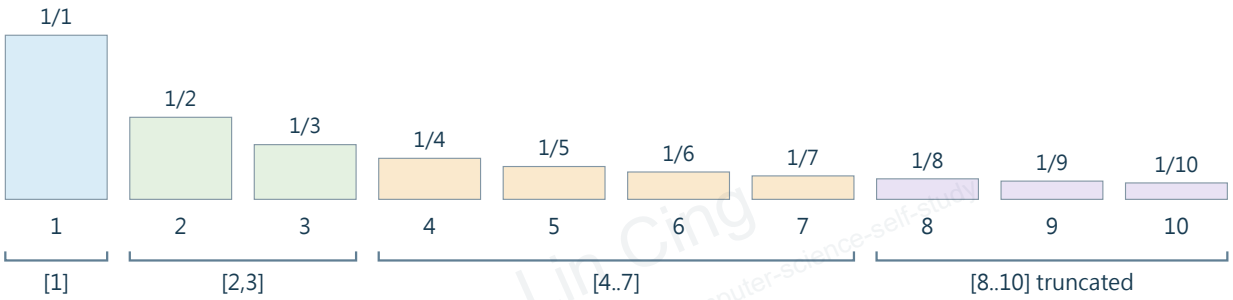
The lower bound is $\Omega(\log n)$, so $H_n = \Theta(\log n)$, also proving that the infinite series diverges.

At $n = 10$, the blocks $[1]$, $[2, 3]$, $[4, 5, 6, 7]$, and $[8, 9, 10]$ contribute 1, $5/6$, $319/420$, and $121/360$. Their total is $H_{10} = 7381/2520 \approx 2.928968$. The last block has three terms, not eight.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Dyadic blocks reveal harmonic layers (continued)

n=10: each full dyadic block sums between1/2 and1



Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Integral bounds follow from unit intervals

If f is integrable and nondecreasing on $[m-1, n+1]$, each integer k satisfies

$$\int_{k-1}^{k} f(x)\,dx \le f(k) \le \int_{k}^{k+1} f(x)\,dx.$$

Summing gives

$$\int_{m-1}^{n} f(x)\,dx \le \sum_{k=m}^{n} f(k) \le \int_{m}^{n+1} f(x)\,dx.$$

For a nonincreasing f, the inequalities reverse:

$$\int_{m}^{n+1} f(x)\,dx \le \sum_{k=m}^{n} f(k) \le \int_{m-1}^{n} f(x)\,dx.$$

The endpoint shifts control the error. A sum is not simply replaced by an integral. Since $1/x$ decreases for $x > 0$,

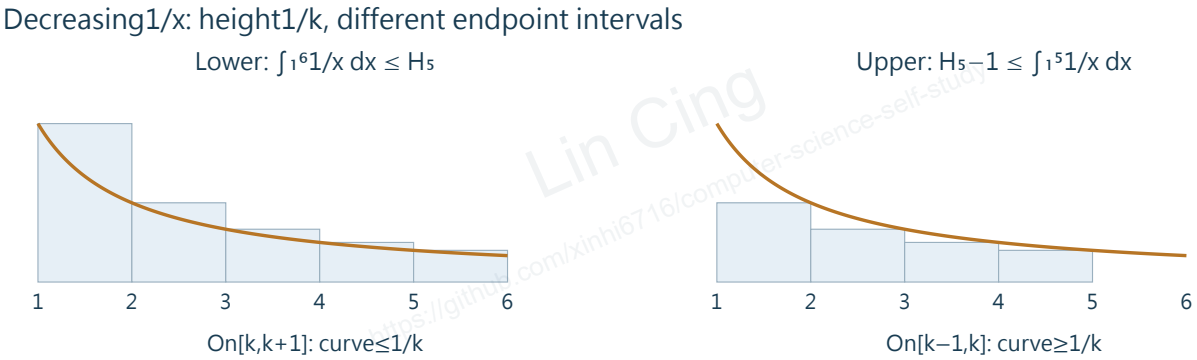
$$H_n \ge \int_{1}^{n+1} \frac{dx}{x} = \ln(n+1).$$

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Integral bounds follow from unit intervals (continued)

Integrating from zero for an upper bound encounters a singularity. Remove the first term, then bound the terms $k = 2, \ldots, n$:

$$H_n \leq 1 + \int_1^n \frac{dx}{x} = 1 + \ln n.$$



Recomputable two-sided integral bounds

At $n = 5$, $H_5 = 137/60 \approx 2.283333$ lies between $\ln 6 \approx 1.791759$ and $1 + \ln 5 \approx 2.609438$. This bracket has an explicit width; it does not assert equality with $\ln n$. In general,

$$\ln(n+1) \leq H_n \leq \ln n + 1, \qquad H_n = \ln n + O(1).$$

For a fixed $p > 1$,

$$\sum_{k=1}^{n} \frac{1}{k^p} \leq 1 + \int_1^n x^{-p}\, dx \leq 1 + \frac{1}{p-1}.$$

The nonnegative partial sums increase and are bounded, so they converge. The case $p = 1$ diverges. For $0 \leq p < 1$, the integral lower bound grows as n^{1-p}, and the sum is $\Theta(n^{1-p})$. Vanishing terms alone do not imply convergence.

For a fixed $r \geq 0$, integration also gives

$$\frac{n^{r+1}}{r+1} \leq \sum_{k=1}^{n} k^r \leq \frac{(n+1)^{r+1} - 1}{r+1}.$$

Recomputable two-sided integral bounds (continued)

This again proves $\Theta(n^{r+1})$, with constants allowed to depend on the fixed parameter r.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Nested loops and changing summation order

Loops with $i = 1, \dots, n$ and $j = i, \dots, n$ perform constant work

$$\sum_{i=1}^{n} (n - i + 1) = \frac{n(n+1)}{2}$$

times. Counting by j instead gives $i = 1, \dots, j$, hence $\sum_{j=1}^{n} j$. Both count the index set $\{(i, j) : 1 \leq i \leq j \leq n\}$.

If an iteration costs j, then

$$\sum_{i=1}^{n} \sum_{j=i}^{n} j = \sum_{j=1}^{n} \sum_{i=1}^{j} j = \sum_{j=1}^{n} j^2 = \frac{n(n+1)(2n+1)}{6}.$$

At $n = 3$, the row sums are $6, 5, 3$, totaling 14. The column count $1 \cdot 1 + 2 \cdot 2 + 3 \cdot 3$ also totals 14.

Finite reordering is safe. An infinite double sum with signed terms requires absolute convergence before arbitrary interchange of order. Finite identities alone do not justify that operation.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Sources

Bibliography

Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 3rd ed., MIT Press, 2009, Appendix A.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

References and attribution

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Sets, relations, functions, graphs, and trees

Part VIII Appendix

Scope

Use elementwise logic and general proofs to connect sets, equivalence partitions, functions, graphs, and trees. State graph, path, direction, empty-set, and binary-slot conventions explicitly. Prove the six equivalent tree conditions and the bipartite criterion.

Prerequisites

Basic discrete-mathematics notation.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Appendix route

Learning objectives

- Apply definitions of sets, relations, and functions precisely.
- Recognize graph and tree structure.
- Connect Cartesian products, equivalence relations, and recursive structures.
- B.1 Sets
- B.2 Relations
- B.3 Functions
- B.4 Graphs
- B.5 Trees

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Sets

A set is unordered and has no repetition, so $\{u, v, u\} = \{v, u\}$. A multiset retains multiplicity, while a sequence retains positions. These are different objects. Membership is written $x \in A$. The condition $A \subseteq B$ means that every $x \in A$ also lies in B; $A \subsetneq B$ additionally requires $A \neq B$.

The empty set is a subset of every set A, but need not be an element of A. The set $\{\varnothing\}$ has one element and is not empty. Equality requires both inclusions, $A \subseteq B$ and $B \subseteq A$.

Here $\mathbb{N} = \{0, 1, \ldots\}$, $\mathbb{Z}$ denotes the integers, and $\mathbb{R}$ denotes the reals. The expression $\{x \in \mathbb{Z} : 3 \mid x\}$ specifies a subset by a property, rather than an ordered list.

Set operations express elementwise logic. Membership in $A \cup B$ uses "or," membership in $A \cap B$ uses "and," and $x \in A \setminus B$ means $x \in A$ and $x \notin B$. A complement requires a specified universe U, with $A^c = U \setminus A$. Changing U changes the complement.

Set identities quantify over every element

Property	Union form	Intersection form
Identity and zero	$A \cup \varnothing = A$	$A \cap \varnothing = \varnothing$
Idempotence	$A \cup A = A$	$A \cap A = A$
Commutativity	$A \cup B = B \cup A$	$A \cap B = B \cap A$
Associativity	$A \cup (B \cup C) = (A \cup B) \cup C$	$A \cap (B \cap C) = (A \cap B) \cap C$
Distributivity	$A \cup (B \cap C) = (A \cup B) \cap (A \cup C)$	$A \cap (B \cup C) = (A \cap B) \cup (A \cap C)$
Absorption	$A \cup (A \cap B) = A$	$A \cap (A \cup B) = A$
De Morgan	$(B \cup C)^c = B^c \cap C^c$	$(B \cap C)^c = B^c \cup C^c$

For an arbitrary element x,

Set identities quantify over every element (continued)

$$\begin{aligned} x \in A \setminus (B \cap C) &\iff x \in A \ \wedge\ \neg(x \in B \wedge x \in C) \\ &\iff (x \in A \wedge x \notin B) \ \vee\ (x \in A \wedge x \notin C) \\ &\iff x \in (A \setminus B) \cup (A \setminus C). \end{aligned}$$

This two-way equivalence proves the identity for every x. A finite Venn example cannot replace that universal quantifier.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Partitions, cardinality, and overlapping counts

A partition of S is a collection of nonempty, pairwise disjoint blocks whose union is S. Every $x \in S$ belongs to exactly one block. For $S = \varnothing$, the empty collection of blocks is allowed. Congruence modulo 3 partitions $\{0, \dots, 7\}$ into $\{0, 3, 6\}$, $\{1, 4, 7\}$, and $\{2, 5\}$.

For finite A, B, split their union into the disjoint parts $A \setminus B$, $A \cap B$, and $B \setminus A$. This gives

$$|A \cup B| = |A| + |B| - |A \cap B|.$$

For $A = \{0, 2, 4, 6\}$ and $B = \{2, 3, 6\}$, the union has size $4 + 3 - 2 = 5$. Simply adding cardinalities counts the intersection twice.

Three-set inclusion-exclusion adds the three singleton sizes, subtracts the three pair intersections, and adds the triple intersection. In general, an element that appears in $r \geq 1$ sets receives coefficient

$$\sum_{j=1}^{r} (-1)^{j+1} \binom{r}{j} = 1,$$

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Partitions, cardinality, and overlapping counts (continued)

because $(1-1)^r = 0$. An element in no set receives coefficient 0. This proves inclusion-exclusion element by element.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Power sets versus Cartesian products

The power set is $\mathcal{P}(S) = \{T : T \subseteq S\}$. For n distinguishable elements, each subset chooses whether each element is included. This is a bijection with the n-bit strings, so $|\mathcal{P}(S)| = 2^n$. In particular, $\mathcal{P}(\varnothing) = \{\varnothing\}$ has one element.

An ordered pair (a, b) retains positions and generally differs from (b, a). Define

$$A \times B = \{(a, b) : a \in A,\ b \in B\}.$$

For finite sets, each fixed a gives $|B|$ pairs. The $|A|$ disjoint rows give $|A \times B| = |A||B|$. An n-fold product counts ordered tuples and multiplies the finite factor sizes. The set A^0 contains the unique empty tuple, so it has size 1.

For $A = \{p, q\}$ and $B = \{0, 1, 2\}$, $A \times B$ has 6 elements. By contrast, $\mathcal{P}(A)$ has the four elements $\varnothing$, $\{p\}$, $\{q\}$, and $\{p, q\}$. A sequence is a function from positions to values. Equal values at distinct positions remain distinct entries.

Infinite cardinality requires bijections

Two sets have equal cardinality when a bijection exists. Being infinite alone does not establish equal cardinality. Define a map from $\mathbb{N}$ to $\mathbb{Z}$ by

$$f(0) = 0, \qquad f(2k-1) = k, \qquad f(2k) = -k \quad (k \geq 1).$$

Every integer has exactly one preimage, so $\mathbb{Z}$ is countable. The map $n \mapsto 2n$ is a bijection onto the even natural numbers. An infinite set can have the same cardinality as a proper subset.

To prove that $\mathbb{R}$ is uncountable, suppose $(0, 1)$ has a list $x_1, x_2, \ldots$. Choose decimal representations that do not end in infinitely many 9s. Define the ith digit of y to be 1 unless the ith digit of x_i is 1, in which case choose 2. Then $y \in (0, 1)$, its representation is unambiguous, and it differs from every x_i at digit i. This contradicts the claimed complete list.

For finite sets of equal size, injection implies surjection. For infinite sets, $f(n) = n + 1$ on $\mathbb{N}$ is injective but misses 0. The finite premise is essential.

Relations

A binary relation is a subset $R \subseteq A \times B$. A relation on one set has $R \subseteq A \times A$. Write aRb when $(a, b) \in R$. A relation need not assign a partner to every input, and a partner need not be unique.

Property	Requirement for all $a, b, c \in A$
Reflexive	aRa
Symmetric	$aRb \implies bRa$
Antisymmetric	$(aRb \land bRa) \implies a = b$
Transitive	$(aRb \land bRc) \implies aRc$

Symmetry and antisymmetry are not negations. Equality satisfies both. For distinct a, b, c, a relation containing $a \leftrightarrow b$ and $b \to c$, but lacking $c \to b$, can satisfy neither. On a nonempty A, the empty relation is vacuously symmetric, antisymmetric, and transitive, but is not reflexive. Symmetry and transitivity do not guarantee the existence of any related pair.

Equivalence relations induce partitions

An equivalence relation is reflexive, symmetric, and transitive. Define $[a] = \{b \in A : aRb\}$. Reflexivity makes each class nonempty and ensures that every a is covered.

Suppose $c \in [a] \cap [b]$. Then aRc and bRc imply aRb by symmetry and transitivity. For any $x \in [a]$, symmetry gives bRa, and transitivity with aRx gives bRx. Thus $[a] \subseteq [b]$. Exchanging a and b gives the reverse inclusion. Classes coincide or are disjoint, and removing duplicates yields a partition.

Conversely, given a partition, define aRb exactly when a and b share a block. Every element lies in one block, proving reflexivity. Sharing a block is symmetric. If a, b share one block and b, c share another, uniqueness of the block containing b makes all three share a block. This proves transitivity. The block containing a is exactly $[a]$, so the constructions correspond.

Congruence modulo 3 illustrates the axioms. The difference $a - b$ is a multiple of 3. Zero proves reflexivity, negation proves symmetry, and addition proves transitivity. The integer classes have residues $0, 1, 2$; restricting them to the earlier finite set gives its three blocks.

Partial orders, total orders, and extrema

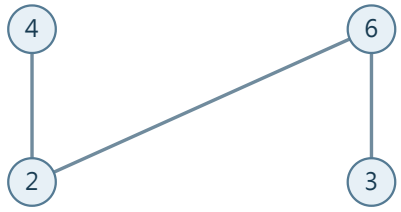
A partial order is reflexive, antisymmetric, and transitive. A total order also compares every pair. A total preorder requires reflexivity, transitivity, and comparability, but permits distinct elements tied in both directions.

A maximum M satisfies $x \leq M$ for every x. A maximal element m merely has no strictly larger element. Minimum and minimal are the dual notions. If M, N are both maxima, then $M \leq N$ and $N \leq M$, so antisymmetry gives $M = N$. A maximum is maximal, but a maximal element need not be a maximum.

On $S = \{2, 3, 4, 6\}$ ordered by divisibility, the minimal elements are $2, 3$ and the maximal elements are $4, 6$. Neither a minimum nor a maximum exists. A Hasse edge is a cover $x < y$ with no z satisfying $x < z < y$. Draw larger elements above smaller ones and omit reflexive and transitive edges.

Partial orders, total orders, and extrema (continued)

S={2,3,4,6}: divisibility Hasse diagram



Upward: larger divisible element

covers：2–4、2–6、3–6

minimal：2、3

maximal：4、6

No minimum or maximum

Lines are covers, not numerical distances. Reflexive and transitive edges are omitted.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Functions

A function $f : A \to B$ is a relation assigning exactly one $b \in B$ to every $a \in A$. The set A is the domain, B is the codomain, and the range $f(A)$ may be smaller than B. A partial function undefined at some a is not a total function under this definition.

An injection satisfies $f(a) = f(a') \implies a = a'$. A surjection gives every $b \in B$ at least one preimage. A bijection satisfies both properties. The map $f(n) = 2n$ from $\mathbb{N}$ to $\mathbb{N}$ is injective but not onto. Changing its codomain to the nonnegative even integers makes it bijective. The map $n \mapsto \lfloor n/2 \rfloor$ from $\mathbb{N}$ to $\mathbb{N}$ is onto but not injective, since both 0 and 1 map to 0.

For finite sets, an injection uses distinct codomain elements, so $|A| \leq |B|$. A surjection must supply every codomain element, so $|A| \geq |B|$. With equal finite sizes, either property implies the other. The infinite counterexample above prevents extending that inference.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Inverses, preimages, and composition

For a bijection, $f^{-1}(b)$ is the unique a with $f(a) = b$, and

$$f^{-1} \circ f = \mathrm{id}_A, \qquad f \circ f^{-1} = \mathrm{id}_B .$$

An injection alone has an inverse only on its range. Without injection, the inverse relation generally is not a function.

For any function and $T \subseteq B$, the set preimage is

$$f^{-1}(T) = \{a \in A : f(a) \in T\}.$$

It is defined without invertibility. It shares notation with an inverse function but is a different object.

For $f : A \to B$ and $g : B \to C$, composition satisfies $(g \circ f)(a) = g(f(a))$. If both maps are injective, equality of composite values lets us cancel g, then f, proving equality of inputs. If both are onto, given $c \in C$, choose $b \in B$ with $g(b) = c$, then $a \in A$ with $f(a) = b$. This proves surjectivity of the composite. For bijections,

$$(g \circ f)^{-1} = f^{-1} \circ g^{-1},$$

Inverses, preimages, and composition (continued)

with the order reversed.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Graph conventions and degree sums

A finite digraph $G = (V, E)$ has $E \subseteq V \times V$, permitting self-loops but no parallel edges. A simple undirected graph uses unordered pairs of distinct vertices and has no loops. A multigraph must preserve edge identities and specify how loops contribute to degree.

A directed edge (u, v) leaves u and enters v. A self-loop contributes one to each of indegree and outdegree. Undirected degree counts incident edges. Each undirected edge has two endpoints, while each directed edge has one tail and one head. Thus

$$\sum_{v \in V} \deg(v) = 2|E|,$$

$$\sum_{v \in V} \operatorname{outdeg}(v) = \sum_{v \in V} \operatorname{indeg}(v) = |E|.$$

The first identity implies that the number of odd-degree vertices is even. Moving vertices or bending edges does not change E. Crossing lines are not junctions unless marked as vertices. Legends or edge lists must determine directions, identities, and values.

Walks, simple paths, and cycles

A walk $v_0, \ldots, v_k$ follows an edge between each adjacent pair. Its length is k edges, rather than $k + 1$ vertices. CLRS permits repetition in a path; that object is called a walk here. A simple path has distinct vertices. Every vertex has a length-zero path to itself.

For undirected acyclicity, a cycle has length at least 3 and repeats only the start and end vertex. Traversing one edge from u to v and back is not a cycle. Directed simple cycles may have length 1 for a loop or 2 for opposite arcs, and must follow every direction. A general closed walk may repeat other vertices and differs from a simple cycle.

If a walk from u to v repeats $v_i = v_j$ with $i < j$, remove the closed segment between those occurrences. The remaining sequence is a shorter walk with the same endpoints. Repeated removal strictly decreases a finite length and eventually leaves a simple path. This proves equivalence of reachability by walks and by simple paths without listing infinitely many walks.

Components, SCCs, and direction

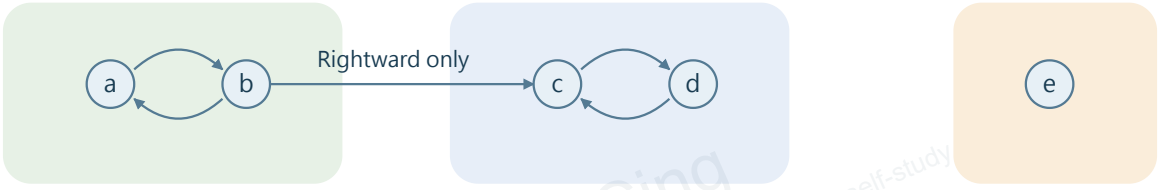
Undirected reachability is reflexive by zero-length paths, symmetric by reversal, and transitive by concatenation. Components are its equivalence classes. Directed reachability is reflexive and transitive, but need not be symmetric. Mutual directed reachability has all three properties; its classes are strongly connected components, or SCCs.

Take the arcs $a \to b$, $b \to a$, $b \to c$, $c \to d$, $d \to c$, and an isolated vertex e. The SCCs are $\{a, b\}$, $\{c, d\}$, and $\{e\}$. Vertex a reaches d, but d cannot reach a, so the first two SCCs remain distinct. Ignoring directions gives only the components $\{a, b, c, d\}$ and $\{e\}$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Components, SCCs, and direction (continued)

Reachability is directional; mutual reachability differs



a reaches d; d cannot reach a. Ignore direction: a/b/c/d together, e still isolated.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Subgraphs, isomorphism, and contraction

A subgraph selects vertices and some original edges, keeping both endpoints of every selected edge. An induced subgraph $G[W]$ keeps all original edges with both endpoints in W. For the preceding example, $W = \{a, b, c\}$ induces exactly $a \to b$, $b \to a$, and $b \to c$.

An isomorphism is a vertex bijection $f : V \to V'$ satisfying

$$(u, v) \in E \iff (f(u), f(v)) \in E'.$$

Preservation in one direction alone is not the general definition. Equal vertex counts, edge counts, and degree multisets are necessary but insufficient. The cycle C_6 and two disjoint triangles both have 6 vertices, 6 edges, and degree 2 everywhere, but only the cycle is connected.

Contracting an undirected edge $\{u, v\}$ merges its endpoints into x, redirects other incident edges, and removes the contracted edge. A simple representation merges parallel copies to a common neighbor and discards loops. Algorithms that need parallel edges, including some randomized contractions, must explicitly use a multigraph instead of silently deduplicating.

Graph classes and the bipartite criterion

The complete graph K_n joins every distinct pair and has $n(n-1)/2$ edges. A bipartite graph has a partition $V = L \sqcup R$ with every edge crossing the two sides, equivalently a proper two-coloring. A DAG is directed and acyclic; trees and forests here are undirected. Absence of directed cycles alone does not make a tree.

A bipartite graph has no odd cycle. Every edge switches sides, and an odd number of switches cannot return to the original side. Conversely, suppose there is no odd simple cycle. Root each component and color vertices by the parity of their shortest undirected distance from the root. An edge joining equal parities would create an odd closed walk by following a root-to-u path, the edge, and a reversed root-to-v path.

If that odd closed walk repeats an internal vertex, split there into two shorter closed walks. Their lengths sum to an odd number, so one is odd. Repeatedly keep the odd part until no internal repetition remains. This is an odd simple cycle, a contradiction. Hence every edge crosses colors, proving bipartiteness.

Graph classes and the bipartite criterion (continued)

A hyperedge may contain many vertices. Its incidence bipartite graph puts original vertices on one side and hyperedges on the other, with an edge for each membership. These are different node types; a hyperedge is not an ordinary vertex pair.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Trees and forests

A free tree here is a nonempty finite simple undirected graph that is connected and has no simple cycle. A forest is acyclic and may contain multiple components or isolated vertices. Empty forests are allowed. Empty graphs are outside the six-equivalence theorem below.

Every tree with $n \geq 2$ has a leaf. Take an endpoint v of a longest simple path. A neighbor outside the path would extend it. An additional neighbor elsewhere on the path would create a cycle. Thus v has only its adjacent path neighbor and has degree 1.

Removing a leaf and its edge preserves connectivity because simple paths between other vertices cannot use a leaf internally. It also preserves acyclicity. Starting with one vertex and zero edges, induction gives $n - 1$ edges in a tree. For a forest with c components, sum the component tree counts:

$$|E| = \sum_{i=1}^{c} (n_i - 1) = n - c.$$

The identity includes the empty forest, where $n = c = 0$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Six equivalent tree conditions

For a nonempty finite simple undirected graph with n vertices and m edges, these conditions are equivalent:

1. The graph is a tree.
2. Each pair of vertices has exactly one simple path between them.
3. The graph is connected, and deleting any edge disconnects it.
4. The graph is connected, with $m = n - 1$.
5. The graph is acyclic, with $m = n - 1$.
6. The graph is acyclic, and adding any missing edge creates a cycle.

$(1) \implies (2)$**.** Connectivity gives a path. If two distinct simple paths existed, their first divergence and subsequent first meeting would delimit internally disjoint segments forming a cycle. This contradicts acyclicity.

$(2) \implies (3)$**.** An edge $\{u, v\}$ is itself the unique simple u-to-v path. Deleting it destroys that reachability.

Six equivalent tree conditions (continued)

$(3) \implies (1)$**.** If a cycle existed, deleting one of its edges would leave the other arc as a replacement in any path using that edge. The graph would remain connected, a contradiction. The leaf-induction count gives $(1) \implies (4)$ and $(1) \implies (5)$. Existence of a path and uniqueness of that path are separate assertions.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Remaining directions and a spanning-tree construction

Every connected graph has a spanning tree. Start with one visited vertex, forming a set S. While $S \neq V$, connectivity supplies an edge from S to its complement. Add that edge and its one new endpoint. Each step attaches one new vertex and creates no cycle. After $n-1$ steps, the selected edges form a spanning tree with $n-1$ edges. In particular, every connected graph has $m \geq n-1$.

$(4) \implies (1)$**.** The constructed spanning tree uses all $n-1$ graph edges. There are no extra edges, so the graph equals that tree.

$(5) \implies (1)$**.** The forest identity $m = n - c$, together with $m = n-1$, gives $c = 1$, proving connectivity.

$(1) \implies (6)$**.** Every new edge $\{u, v\}$ closes the tree's unique u-to-v path into a cycle.

$(6) \implies (1)$**.** If two components existed, adding an edge between them could not create a cycle. This contradicts condition (6), so the graph is connected.

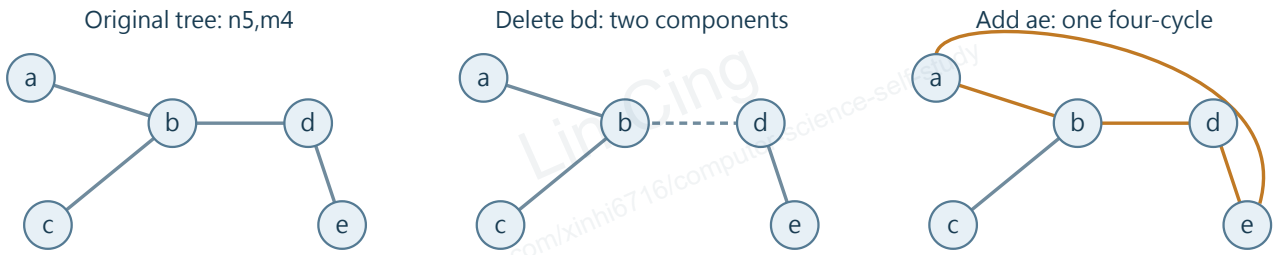
Remaining directions and a spanning-tree construction (continued)

These implications establish all six equivalences. The equality $m = n-1$ alone is insufficient. A triangle plus an isolated vertex has $n = 4, m = 3$ but is not a tree. Acyclicity alone also does not ensure connectivity.

The same tree under deletion and cycle creation

Let $V = \{a, b, c, d, e\}$ and $E = \{ab, bc, bd, de\}$, so $n = 5, m = 4$. The unique simple path from a to e is a, b, d, e, of length 3. Deleting bd gives components $\{a, b, c\}$ and $\{d, e\}$. Adding ae creates the four-cycle a, b, d, e, a.

Same vertices: remove an edge to disconnect, add one to cycle



Middle dashed edge is deleted. Right orange cycle follows a–b–d–e–a via new edge ea.

A graph with a cycle can still have a spanning tree, although the graph itself is not a tree. Minimum-spanning-tree optimization adds a weight objective that is absent from the definition of a tree.

Rooted and ordered trees

Choose a root r in a free tree. Every nonroot vertex x has as its parent the preceding vertex on its unique simple path from r. Its other, downward neighbors are its children. Here ancestors and descendants include the node itself; proper ancestors and descendants exclude it. The subtree rooted at x contains all descendants of x.

Depth counts edges from the root to a node. A node's height counts edges to its deepest leaf. Leaves have height 0, the root has depth 0, and tree height is the maximum depth. Rooted degree counts children. Graph degree also counts the parent, so it is larger by 1 for each nonroot node.

Root the preceding example at b. Its children are a, c, d, and d has child e. Vertices a, c, d have depth 1, and e has depth 2. The heights are 2 for b, 1 for d, and 0 for the others. Vertex b has both graph degree and child count 3. Vertex d has graph degree 2 and one child.

An ordered tree also orders each child list. Replacing the order (a, c, d) at b by (c, a, d) keeps the rooted relationships but changes the ordered tree.

Binary and positional trees retain empty slots

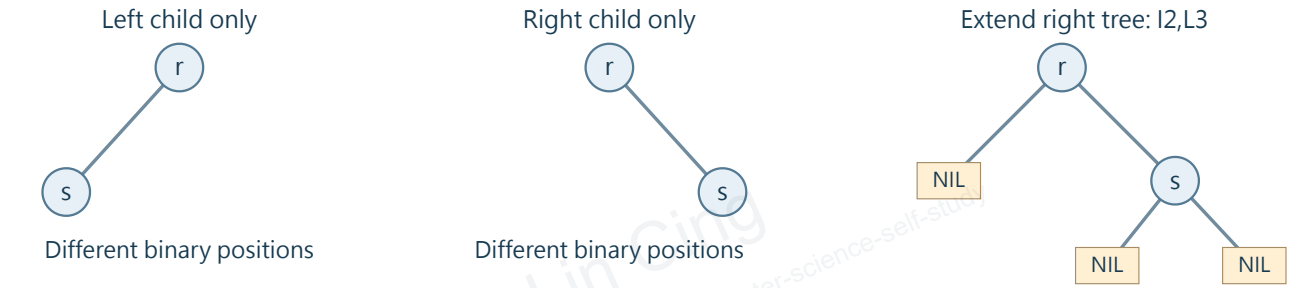
A binary tree is recursively either the empty tree `NIL` or a root with disjoint left and right binary subtrees. A single child in the left slot differs from a single child in the right slot. An ordered tree with one child has no such slot distinction, so an ordered tree of degree at most 2 does not fully specify a binary tree.

A positional k-ary tree has fixed slots $1, \ldots, k$, including meaningful empty positions. Replacing every missing binary child by an external `NIL` leaf gives a full binary tree. Each internal node has two children, and the left and right positions are preserved.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Binary and positional trees retain empty slots (continued)

Empty slots retain position: left and right differ



Lin Cing
https://github.com/xinhi6716/computer-science-self-study

For a nonempty binary tree, n real nodes have $2n$ child slots. The $n - 1$ real parent-child edges occupy that many slots, leaving

$$2n - (n - 1) = n + 1$$

missing positions. The extended tree has n internal nodes and $n + 1$ external leaves. Sharing one `NIL` sentinel object does not change the number of logical positions or require $n + 1$ actual allocations.

Full, complete, and height counts

A full k-ary tree gives each internal node exactly k children. Leaves may have different depths. With I internal nodes and L leaves, counting edges gives

$$kI = I + L - 1, \qquad L = (k-1)I + 1.$$

For binary trees, $L = I + 1$.

CLRS's appendix uses "complete k-ary" to require that all leaves also have depth h; this is often called "perfect" elsewhere. For $k \geq 2$, level d has k^d nodes, and

$$L = k^h, \qquad I = \sum_{d=0}^{h-1} k^d = \frac{k^h - 1}{k - 1}.$$

At $k = 3, h = 2$, the level sizes are $1, 3, 9$, so $I = 4, L = 9$, the total is 13, and there are $12 = 3I$ edges. For $k = 1$, a chain has $I = h, L = 1$; do not divide by $k - 1 = 0$.

Full, complete, and height counts (continued)

Heap chapters often use "complete binary" for a possibly partial final level filled from the left. Check which convention applies. A general nonempty binary tree of height h has at most $2^{h+1} - 1$ nodes, giving

$$h \geq \lceil \log_2(n+1) \rceil - 1.$$

A chain has height $n - 1$. Without balance, binary-tree height need not be $\Theta(\log n)$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Sources

Bibliography

Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 3rd ed., MIT Press, 2009, Appendix B.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

References and attribution

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Counting and probability

Part VIII Appendix

Scope

Define sample objects and common denominators before using counting, conditional probability, Bayes' rule, and indicators. Derive geometric and binomial moments and tail bounds. Distinguish levels of independence, zero-probability and support boundaries, a centered deviation r, and a total threshold t.

Prerequisites

Permutations and combinations, probability axioms, and random variables.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Appendix route

Learning objectives

- Count by products, disjoint sums, and complements.
- Derive expectation and variance for geometric and binomial distributions.
- Bound binomial tails using exponential moments.
- C.1 Counting
- C.2 Probability
- C.3 Discrete random variables
- C.4 The geometric and binomial distributions
- C.5 The tails of the binomial distribution

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Counting starts with the sample object

Define what one outcome records. The sum rule requires disjoint cases. The product rule requires a known branch count after each prefix. The available choices may depend on that prefix, provided each level's count is fixed. Variable branching requires summing branches rather than multiplying unspecified averages.

From six students, two distinct office holders give $6 \cdot 5 = 30$ ordered assignments. An unordered two-person committee is counted twice by those assignments, so there are 15 committees. Allowing one person to hold both offices gives $6^2 = 36$ assignments. These count three different objects.

There are q^m length-m strings over q symbols, retaining positions and allowing repetition. Substrings are contiguous; subsequences may skip positions. Specify whether identical contents at distinct positions count separately. For an n-character string and $k \geq 1$, the number of positional length-k substrings is $\max(0, n - k + 1)$. This need not equal the number of distinct contents.

Permutations, combinations, and uniform overcounting

There are $n!$ full permutations of n distinct records. Ordered selection of k records without replacement has

$$P(n, k) = \frac{n!}{(n - k)!}, \qquad 0 \leq k \leq n.$$

Every unordered k-subset has exactly $k!$ orderings, giving

$$\binom{n}{k} = \frac{n!}{k!(n - k)!}.$$

The case $k = 0$ gives one empty choice. Extend the coefficient to 0 for $k < 0$ or $k > n$.

Distinct record identities still give $n!$ permutations when keys repeat. If only key strings matter and the multiplicities are $m_1, \ldots, m_s$, each string has exactly $\prod_{i=1}^{s} m_i!$ identity permutations. Thus the number of distinct strings is $n! / \prod_{i=1}^{s} m_i!$. Prove equal overcounting before dividing.

Complementation gives $\binom{n}{k} = \binom{n}{n-k}$. Partitioning subsets by whether they include a specified element gives Pascal's identity:

Permutations, combinations, and uniform overcounting (continued)

$$\binom{n}{k} = \binom{n-1}{k} + \binom{n-1}{k-1}.$$

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Binomial expansion and entropy bounds

In $(x+y)^n$, there are $\binom{n}{k}$ ways to choose the k factor positions contributing x. Hence

$$(x+y)^n = \sum_{k=0}^{n} \binom{n}{k} x^k y^{n-k}.$$

Setting $x = y = 1$ gives $\sum_{k=0}^{n} \binom{n}{k} = 2^n$, also counting subsets by size. For $1 \le k \le n$,

$$\binom{n}{k} = \prod_{j=0}^{k-1} \frac{n-j}{k-j} \ge \left(\frac{n}{k}\right)^k,$$

because each factor is at least n/k. Also,

$$\binom{n}{k} \le \frac{n^k}{k!} \le \left(\frac{en}{k}\right)^k,$$

using $k! \ge (k/e)^k$.

Let $\alpha = k/n \in (0,1)$. The nonnegative kth term in the expansion of $(\alpha + 1 - \alpha)^n = 1$ is at most 1. Therefore

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Binomial expansion and entropy bounds (continued)

$$\binom{n}{k} \le \alpha^{-k}(1-\alpha)^{-(n-k)} = 2^{nH(\alpha)},$$

where $H(\alpha) = -\alpha \log_2 \alpha - (1-\alpha)\log_2(1-\alpha)$. Extend to the endpoints with $0 \log_2 0 = 0$, giving coefficient 1. Since $H'(\alpha) = \log_2((1-\alpha)/\alpha)$, the maximum is $H(1/2) = 1$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Probability axioms and uniform outcomes

An elementary outcome records one complete distinguishable result. An event is a measurable subset of the sample space S. On the finite or countable discrete spaces here, every subset is an event. An uncountable space requires a specified event family closed under complements and countable unions; arbitrary subsets are not automatically measurable.

Probability is nonnegative, satisfies $P(S) = 1$, and is finitely or countably additive on pairwise disjoint events. It follows that

$$P(\varnothing) = 0, \qquad P(A^c) = 1 - P(A), \qquad A \subseteq B \implies P(A) \le P(B).$$

In a discrete space, $P(A) = \sum_{s \in A} P(\{s\})$.

The formula $P(A) = |A|/|S|$ requires finitely many equally likely elementary outcomes. Two independent fair bits have four ordered outcomes, each with probability $1/4$. Compressing them to the number of ones gives values $0, 1, 2$ with probabilities $1/4, 1/2, 1/4$, respectively. Those three values are not uniform.

Union bounds need no independence

Disjoint decomposition into the A-only part, the intersection, and the B-only part gives

$$P(A \cup B) = P(A) + P(B) - P(A \cap B) \leq P(A) + P(B).$$

For many events, define

$$B_1 = A_1, \qquad B_i = A_i \setminus \bigcup_{j<i} A_j.$$

The B_i are disjoint, have the same union as the A_i, and satisfy $B_i \subseteq A_i$. Additivity and monotonicity give

$$P\left(\bigcup_i A_i\right) = \sum_i P(B_i) \leq \sum_i P(A_i).$$

This also holds for countably many events and needs no independence. If the right side exceeds 1, cap it at 1. By contrast, factoring a general collection of intersection probabilities requires mutual independence.

Union bounds need no independence (continued)

If three failure events each have probability at most 0.01, their union has probability at most 0.03, regardless of dependence. If the three events are identical and each has probability 0.01, their union has probability 0.01. The expression $1 - 0.99^3$ requires independence.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Conditional probability and the chain rule

For $P(B) > 0$, conditional probability renormalizes the outcomes retained in B:

$$P(A \mid B) = \frac{P(A \cap B)}{P(B)}, \qquad P(A \cap B) = P(B)P(A \mid B).$$

Repeated application yields

$$P\left(\bigcap_{i=1}^{n} A_i\right) = P(A_1) \prod_{i=2}^{n} P\left(A_i \,\middle|\, \bigcap_{j=1}^{i-1} A_j\right).$$

Every conditioning prefix must have positive probability. A zero-probability prefix already makes the total intersection zero; subsequent undefined ratios must not be multiplied.

A bag contains three blue and two red balls. Drawing twice without replacement gives $P(\text{both blue}) = (3/5)(2/4) = 3/10$. The second draw's blue marginal remains $3/5$, but conditional on a first blue it is $1/2$. Thus the draws are dependent. Replacement with fresh uniform selection gives $(3/5)^2$ instead.

Conditional probability and the chain rule (continued)

Knowing that at least one occurrence happened usually differs from knowing that a specified occurrence happened. Observation and selection rules belong to the model. Two remaining verbal cases need not each have probability $1/2$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Independence, disjointness, and pairwise independence

Events A, B are independent when $P(A \cap B) = P(A)P(B)$. This includes probability-zero events. The equivalent expression $P(A \mid B) = P(A)$ additionally needs $P(B) > 0$. Disjoint events with positive probabilities are dependent. The empty event is independent of every event.

Mutual independence requires the intersection probability of every finite subcollection to factor into the product of its marginals. Pairwise independence checks only pairs and cannot justify factoring three or more events.

Let U, V be independent fair bits, and define $W = U \operatorname{XOR} V$. The outcomes $000, 011, 101, 110$ each have probability $1/4$. Each of the pairs (U, V), (U, W), and (V, W) has all four combinations equally likely, so every pair is independent. Yet

$$P(U = V = W = 1) = 0 \neq \frac{1}{8} = P(U = 1)P(V = 1)P(W = 1).$$

The three bits are not mutually independent.

Independence, disjointness, and pairwise independence (continued)

Pairwise independent, but not mutually independent

U	V	W=U XOR V	P(outcome)
0	0	0	1/4
0	1	1	1/4
1	0	1	1/4
1	1	0	1/4

Each pair:00/01/10/11, each1/4

All three P(=1) marginals are1/2

But111 is absent from sample space

0 ≠ (1/2)³=1/8

Pair checks do not factor triple intersections. Variances add here; triple products do not.

Total probability and Bayes denominators

Let the B_i form a countable partition of S, with $P(B_i) > 0$. The event A is the disjoint union of $A \cap B_i$, so

$$P(A) = \sum_i P(B_i) P(A \mid B_i).$$

When $P(A) > 0$, Bayes' rule gives

$$P(B_j \mid A) = \frac{P(B_j) P(A \mid B_j)}{P(A)}.$$

A monitor has fault prior 0.02, sensitivity 0.9, and false-positive rate 0.1. Its alarm probability and posterior fault probability are

$$P(\text{alarm}) = 0.02 \cdot 0.9 + 0.98 \cdot 0.1 = 0.116,$$

$$P(\text{fault} \mid \text{alarm}) = \frac{0.018}{0.116} = \frac{9}{58} \approx 0.155172.$$

Sensitivity 0.9 is not posterior probability 0.9. Normal cases contribute 0.098 false alarms to the denominator. If two tests are independent conditional on fault status, then

Total probability and Bayes denominators (continued)

$$P(\text{two alarms}) = 0.02 \cdot 0.9^2 + 0.98 \cdot 0.1^2 = 0.026,$$

$$P(\text{fault} \mid \text{two alarms}) = \frac{0.0162}{0.026} = \frac{81}{130} \approx 0.623077.$$

Squaring the likelihoods requires that conditional independence assumption.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Continuous uniform probabilities

Let X be continuous uniform on $[a, b]$, where $a < b$. For $a \le c \le d \le b$,

$$P(c \le X \le d) = \frac{d - c}{b - a}.$$

Each singleton has probability 0, while the whole interval has probability 1. Countable additivity does not call for summing uncountably many singleton masses.

On $[2, 8]$, $P(3 \le X \le 5) = 2/6 = 1/3$, and removing endpoints changes nothing. The variable $Z = \lfloor X \rfloor$ has values $2, \dots, 7$, each with probability $1/6$. The value $Z = 8$ occurs only when $X = 8$, with probability 0. Do not assign equal weights to seven integer labels.

The following sections concern discrete random variables and probability mass. A continuous density may exceed 1 even though every singleton has probability zero. Integrate a density over intervals rather than treating it as point probability.

Discrete random variables and joint masses

A random variable $X : S \to \mathbb{R}$ assigns a numerical value to an outcome. The event $\{X = x\}$ is a preimage. Its probability mass function is $p_X(x) = P(X = x)$, with nonnegative values satisfying $\sum_x p_X(x) = 1$. The word “mass” distinguishes discrete probabilities from continuous density.

The joint mass $p(x, y) = P(X = x, Y = y)$ gives marginals

$$p_X(x) = \sum_y p(x, y), \qquad p_Y(y) = \sum_x p(x, y).$$

For $p_Y(y) > 0$, the conditional mass is $p(x, y)/p_Y(y)$. Independence means $p(x, y) = p_X(x) p_Y(y)$ for every pair (x, y).

Let A, B be independent fair four-sided dice, and let $M = \max(A, B)$. Then

$$P(M \le m) = \frac{m^2}{16}, \qquad P(M = m) = \frac{m^2 - (m - 1)^2}{16} = \frac{2m - 1}{16}.$$

For $m = 1, 2, 3, 4$, the masses are $1, 3, 5, 7$ divided by 16. They are not uniform.

Expectation need not be a possible outcome

Expectation is $E[X] = \sum_x x p_X(x)$. Finite support always permits evaluation. On countable signed support, $E[|X|] < \infty$ ensures a finite expectation that may be reordered. A nonnegative X may have infinite expectation. The expression $\infty - \infty$ is not valid arithmetic.

For the dice maximum,

$$E[M] = \frac{1 + 6 + 15 + 28}{16} = \frac{50}{16} = \frac{25}{8},$$
$$E[M^2] = \frac{1 + 12 + 45 + 112}{16} = \frac{170}{16} = \frac{85}{8}.$$

The mean 3.125 is outside the integer support and is still correct.

For any g satisfying $E[|g(X)|] < \infty$, grouping outcomes by their X-preimages gives

$$E[g(X)] = \sum_x g(x) p_X(x).$$

There is no need to list the distinct values of $g(X)$ first. In general, $E[g(X)] \neq g(E[X])$; for the square function, the difference is variance.

Linearity and indicators

For integrable X, Y, expand using their joint mass:

$$\begin{aligned} E[X + Y] &= \sum_x \sum_y (x + y) p(x, y) \\ &= \sum_x x \sum_y p(x, y) + \sum_y y \sum_x p(x, y) \\ &= E[X] + E[Y]. \end{aligned}$$

Absolute integrability justifies splitting and reordering. Independence was not used. Also $E[aX + b] = aE[X] + b$, and repeated application gives linearity for finite sums.

The indicator I_A equals 1 on A and 0 otherwise, so $E[I_A] = P(A)$. If X counts events, then $X = \sum_i I_i$ gives $E[X] = \sum_i P(A_i)$, even with dependence. In a uniform permutation of n distinct keys, each inversion-pair indicator has expectation $1/2$, giving total expected inversions $n(n-1)/4$.

For independent integrable X, Y, joint factorization gives

Linearity and indicators (continued)

$$E[XY] = \left(\sum_x x p_X(x)\right)\left(\sum_y y p_Y(y)\right) = E[X]E[Y].$$

This step uses independence. If $X = Y$ is one fair bit, $E[XY] = 1/2 \neq 1/4$. Products of many variables generally require mutual independence, rather than pairwise independence alone.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Tail sums and Jensen

A nonnegative integer variable satisfies, outcome by outcome,

$$X = \sum_{i \geq 1} I_{\{X \geq i\}}.$$

Take expectations of finite truncations, then use monotone limits of nonnegative terms. This gives

$$E[X] = \sum_{i \geq 1} P(X \geq i),$$

possibly with value ∞. A signed variable cannot use this formula directly.

A convex function satisfies

$$f(\lambda x + (1-\lambda)y) \leq \lambda f(x) + (1-\lambda) f(y), \qquad 0 \leq \lambda \leq 1.$$

For a finite support $x_1, \ldots, x_m$ with weights w_i, let $s = \sum_{i<m} w_i$. When $0 < s < 1$, combine the first $m-1$ points into $z = \sum_{i<m} w_i x_i / s$. Two-point convexity and induction give

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Tail sums and Jensen (continued)

$$\begin{aligned} f\left(\sum_{i=1}^{m} w_i x_i\right) &= f(sz + w_m x_m) \\ &\le s f(z) + w_m f(x_m) \\ &\le \sum_{i=1}^{m} w_i f(x_i). \end{aligned}$$

Zero weights may be omitted, and the single-point case is equality. Thus $f(E[X]) \le E[f(X)]$.

For the general integrable case, assume f is convex on an interval containing the support and $\mu = E[X]$. A supporting line at an interior μ has

$$f(x) \ge f(\mu) + a(x - \mu).$$

Taking expectations gives $E[f(X)] \ge f(\mu) + a(E[X] - \mu) = f(\mu)$. At a degenerate endpoint, X is concentrated there. The relevant expectations must be defined. Jensen is not permission to move an arbitrary function through expectation.

Variance, covariance, and dependence

For $E[X^2] < \infty$ and $\mu = E[X]$,

$$\operatorname{Var}(X) = E[(X - \mu)^2] = E[X^2] - \mu^2 \ge 0.$$

The standard deviation is $\sigma = \sqrt{\operatorname{Var}(X)}$. Scaling gives $\operatorname{Var}(aX + b) = a^2 \operatorname{Var}(X)$. For the dice maximum,

$$\operatorname{Var}(M) = \frac{85}{8} - \left(\frac{25}{8}\right)^2 = \frac{55}{64}, \qquad \sigma = \frac{\sqrt{55}}{8}.$$

Define $\operatorname{Cov}(X, Y) = E[XY] - E[X]E[Y]$. Expanding squares gives

$$\begin{aligned} \operatorname{Var}(X + Y) &= \operatorname{Var}(X) + \operatorname{Var}(Y) + 2\operatorname{Cov}(X, Y), \\ \operatorname{Var}\left(\sum_i X_i\right) &= \sum_i \operatorname{Var}(X_i) + 2\sum_{i<j} \operatorname{Cov}(X_i, X_j). \end{aligned}$$

Pairwise independence and finite second moments suffice to make every covariance zero. Mutual independence is unnecessary for variance addition.

Variance, covariance, and dependence (continued)

Identical marginals alone do not suffice. If all n variables are the same fair bit, their sum equals 0 or n with equal probability and has variance $n^2/4$. Adding individual variances would give only $n/4$. Omitting covariance underestimates uncertainty.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Markov and Chebyshev assumptions

Let $X \geq 0$ have finite mean, and let $t > 0$. The pointwise inequality

$$X \geq tI_{\{X \geq t\}}$$

gives $E[X] \geq tP(X \geq t)$ after taking expectations. Thus Markov's inequality is

$$P(X \geq t) \leq \frac{E[X]}{t}.$$

It requires no independence. It cannot be applied directly to signed variables, where positive and negative cancellation may hide a large positive tail.

For Y with finite second moment, set $X = (Y - \mu)^2$ and use the threshold a^2, where $a > 0$. Markov gives Chebyshev's inequality:

$$P(|Y - \mu| \geq a) = P((Y - \mu)^2 \geq a^2) \leq \frac{\mathrm{Var}(Y)}{a^2}.$$

This is a two-sided bound. It also bounds either one-sided event, although it may be loose.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Markov and Chebyshev assumptions (continued)

For the maximum M, use $\mu = 25/8$, variance $55/64$, and $a = 2$. The bound is $55/256 \approx 0.214844$. The actual event contains only $M = 1$, with probability $1/16$. A bound is not an equality; values above 1 may be capped at 1.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Geometric and binomial distributions

A Bernoulli variable with parameter p equals 1 with probability p and 0 with probability $q = 1 - p$. Since $X^2 = X$, it has $E[X] = p$ and $\operatorname{Var}(X) = p - p^2 = pq$. The trial sequences below additionally assume mutual independence and a common p. Equal marginals alone are insufficient.

Here the geometric variable G counts trials including the first success. Its support is $1, 2, \ldots$, with $0 < p \leq 1$. The event $G = k$ requires $k - 1$ failures followed by success, so

$$P(G = k) = q^{k-1}p.$$

At $p = 0$, success never arrives and $G = \infty$, outside this finite-valued distribution.

Counting earlier failures gives $F = G - 1$, with support $0, 1, \ldots$, mass $P(F = f) = q^f p$, and mean q/p. Do not use the trial mean $1/p$ for both conventions. At $p = 1$, $G \equiv 1$, $F \equiv 0$, and both variances are zero.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Geometric normalization and moments

The geometric masses normalize because

$$\sum_{k\ge 1} q^{k-1}p = \frac{p}{1-q} = 1.$$

For an integer $t \ge 0$, $P(G > t) = q^t$, hence $P(G \ge i) = q^{i-1}$. Tail summation gives

$$E[G] = \sum_{i\ge 1} q^{i-1} = \frac{1}{1-q} = \frac{1}{p}.$$

The weighted geometric identity yields

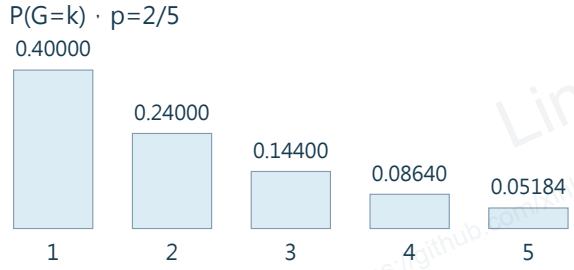
$$\sum_{k\ge 1} k^2 q^{k-1} = \frac{1+q}{(1-q)^3}, \qquad E[G^2] = \frac{1+q}{p^2}.$$

Subtracting the squared mean gives $\operatorname{Var}(G) = q/p^2$. Handle $q = 0$ directly as $G \equiv 1$, without dividing by q.

Geometric normalization and moments (continued)

At $p = 2/5$, the first five masses are $2/5$, $6/25$, $18/125$, $54/625$, and $162/3125$. Their sum is $2882/3125$, and the remaining probability is $P(G > 5) = 243/3125$. The mean is $5/2$ and the variance is $15/4$. A mean of 2.5 averages repeated complete experiments; it does not place a success on a fractional trial.

Fix the counting convention before masses and tails



One outcome: F → F → F → S

G=4: includes the success trial

F=G−1=3: failures only

No success in five: q⁵=243/3125

This tail remains beyond the first five bars

Axis counts trials including first success. E[G]=2.5,E[F]=1.5; the counts differ.

Memorylessness and retry-model boundaries

For $0 < p < 1$ and integers $s, t \geq 0$,

$$P(G > s + t \mid G > s) = \frac{q^{s+t}}{q^s} = q^t = P(G > t).$$

Past failures do not change the next success rate because the trials are independent with a common p. A long failure streak does not make success due.

Changing retry probabilities, persistent shared faults, and sampling without replacement violate that model. Independent trials with unequal p_i instead have tail $\prod_{i=1}^{t}(1 - p_i)$. Dependence may require conditional probabilities rather than q^t.

A fixed trial cost c gives expected total cost c/p. Different costs for failure and success give

$$\text{total cost} = c_{\text{fail}}(G - 1) + c_{\text{success}},$$
$$E[\text{total cost}] = c_{\text{fail}}\frac{q}{p} + c_{\text{success}}.$$

Specify how costs relate to trials before applying linearity.

Binomial mass, mean, and variance

For n mutually independent trials with a common success probability p, let $X = \sum_{i=1}^{n} I_i \in \{0, \ldots, n\}$. A specified set of k success positions has probability $p^k q^{n-k}$. There are $\binom{n}{k}$ disjoint such position sets, so

$$b(k; n, p) = \binom{n}{k} p^k q^{n-k}.$$

Normalization follows from $(p + q)^n = 1$.

Linearity gives $\mu = np$ without independence. Variance addition requires zero pairwise covariances, giving $\operatorname{Var}(X) = npq$. The complete binomial mass formula used mutual independence. Pairwise independence suffices for the mean and variance but does not imply the distribution.

At $n = 0$ or $p = 0$, X is concentrated at 0; at $p = 1$, it is concentrated at n. The endpoint convention $0^0 = 1$ represents an empty product.

Binomial mass, mean, and variance (continued)

Mutually independent trials with unequal p_i instead have a Poisson-binomial distribution, with mean $\sum_i p_i$ and variance $\sum_i p_i(1-p_i)$. Substituting the average p into a binomial mass formula does not make the distributions equal.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A fully checkable binomial table

For $n=8, p=1/4, q=3/4$, the common denominator is $4^8=65536$. The numerators are $\binom{8}{k}3^{8-k}$.

k	0	1	2	3	4	5	6	7	8
Numerator	6561	17496	20412	13608	5670	1512	252	24	1

The numerators sum to 65536. Their k-weighted sum is 131072, giving $E[X]=2$. Their k^2-weighted sum is 360448, giving $E[X^2]=11/2$ and variance $11/2-4=3/2$.

The exact tail is

$$P(X \ge 5)=\frac{1512+252+24+1}{65536}=\frac{1789}{65536}\approx 0.027298.$$

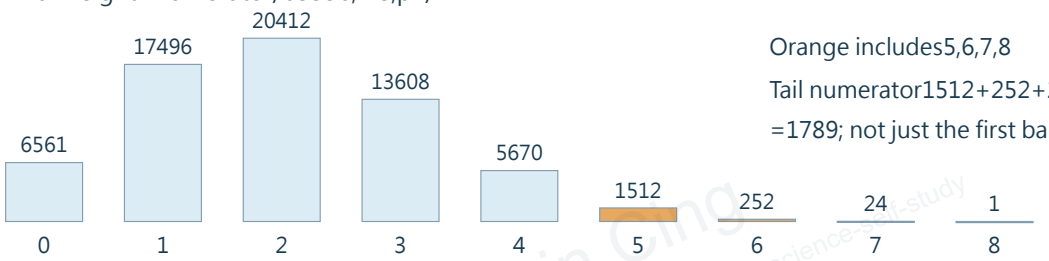
It includes every term through $k=8$, rather than only $b(5)$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A fully checkable binomial table (continued)

A single mass differs from a tail; threshold5 includes5

Bar height=numerator/65536; n8,p1/4



Orange includes5,6,7,8

Tail numerator1512+252+24+1

=1789; not just the first bar

Numerators sum65536; mean2,variance1.5; P(X≥5)=1789/65536≈.027298.

Modes follow adjacent ratios

For $0 < p < 1$ and $1 \le k \le n$,

$$\frac{b(k)}{b(k-1)} = \frac{(n-k+1)p}{kq}.$$

The ratio exceeds 1 exactly when $k < (n+1)p$, and equals 1 when $k = (n+1)p$.

If $(n+1)p$ is not an integer, the unique mode is $\lfloor (n+1)p \rfloor$. If it is an integer m, both $m-1$ and m are modes. For $n = 8, p = 1/4$, the mode is 2 and coincidentally equals the mean 2. For $n = 4, p = 1/2$, $(n+1)p = 2.5$ gives mode 2. For $n = 3, p = 1/2$, the modes are $1, 2$, while the mean 1.5 has no integer probability bar.

The coefficient and entropy bound gives

$$b(k) \le \left(\frac{np}{k}\right)^k \left(\frac{nq}{n-k}\right)^{n-k}.$$

Interpret endpoints by continuous extension or the 0^0 convention. This bounds one mass, rather than a complete tail.

Binomial tails and union bounds

For an integer-valued X, $P(X \geq a)$ sums from $\lceil a \rceil$, whereas $P(X > a)$ starts at $\lfloor a \rfloor + 1$. For an integer k, $P(X < k)$ ends at $k - 1$, unlike $P(X \leq k)$.

For $X \sim \mathrm{Bin}(n, p)$, the event $X \geq k$ implies that some specified set of k trials all succeed. Each such set has probability p^k. Applying the union bound over the $\binom{n}{k}$ sets gives

$$P(X \geq k) \leq \binom{n}{k} p^k, \qquad 0 \leq k \leq n.$$

The events need not be disjoint. At $k = 0$, the bound is 1; at $k > n$, the actual event is empty. Apply the same argument to failures $Y = n - X \sim \mathrm{Bin}(n, q)$ to get

$$P(X \leq k) \leq \binom{n}{k} q^{n-k}, \qquad 0 \leq k \leq n.$$

At $n = 8, p = 1/4, k = 5$, the upper bound is $56/1024 = 7/128 \approx 0.054688$, compared with the exact tail $1789/65536 \approx 0.027298$. Outcomes with six or more successes are counted repeatedly, which explains why the bound is usually strict.

Deriving a geometric left-tail ratio bound

Assume $0 < p < 1$ and an integer $0 < k < np$. For $1 \leq i \leq k$,

$$\frac{b(i-1)}{b(i)} = \frac{iq}{(n-i+1)p} < \frac{kq}{(n-k)p} = \rho < 1.$$

Working backward gives $b(k-j) < \rho^j b(k)$ for $j = 1, \ldots, k$. Consequently,

$$\begin{aligned} P(X < k) &= \sum_{i=0}^{k-1} b(i) < b(k) \sum_{j \geq 1} \rho^j \\ &= \frac{\rho}{1-\rho} b(k) = \frac{kq}{np - k} b(k). \end{aligned}$$

The denominator is positive by assumption. Substituting $k = np$ is invalid.

If $0 < k \leq np/2$, the coefficient is at most $q \leq 1$, so $L_k = P(X < k) < b(k)$. Since $L_{k+1} = L_k + b(k)$,

$$\frac{L_k}{L_{k+1}} = \frac{L_k}{L_k + b(k)} < \frac{1}{2}.$$

Moving one integer threshold farther into the left tail at least halves its probability.

Right tails change both variables and thresholds

Let $Y = n - X \sim \mathrm{Bin}(n, q)$. For an integer $np < k < n$, we have $n - k < nq$. Apply the left-tail bound at threshold $n - k$:

$$\begin{aligned} P(X > k) &= P(Y < n - k) \\ &< \frac{(n-k)p}{nq - (n-k)} b(n-k; n, q) \\ &= \frac{(n-k)p}{k - np} b(k; n, p). \end{aligned}$$

If $k > (np + n)/2$, the left-tail halving result gives

$$P(X > k) < \frac{1}{2} P(X > k - 1).$$

This exchanges the roles of success and failure, including their thresholds.

At $n = 8, p = 1/4, k = 4$, the coefficient is

$$\frac{(8-4)(1/4)}{4-2} = \frac{1}{2}.$$

Right tails change both variables and thresholds (continued)

Thus $P(X > 4) < 2835/65536 \approx 0.043259$, compared with the exact tail $1789/65536$. This improves the union bound in this example, but need not do so for every parameter choice.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Exponential moments turn tails into nonnegative quantities

Let I_i be mutually independent Bernoulli indicators with parameters p_i. Write $X = \sum_i I_i$ and $\mu = \sum_i p_i$. For $\lambda > 0$, monotonicity and Markov give

$$P(X \geq t) \leq e^{-\lambda t} E[e^{\lambda X}].$$

Independence permits factorization. Using $1 + z \leq e^z$,

$$\begin{aligned} E[e^{\lambda X}] &= \prod_i E[e^{\lambda I_i}] \\ &= \prod_i [1 + p_i(e^\lambda - 1)] \\ &\leq \exp[\mu(e^\lambda - 1)]. \end{aligned}$$

Therefore $P(X \geq t) \leq \exp[\mu(e^\lambda - 1) - \lambda t]$. For $\mu > 0, t > \mu$, differentiating the exponent gives $\mu e^\lambda - t = 0$. The minimum occurs at $\lambda = \ln(t/\mu) > 0$, because the second derivative is $\mu e^\lambda > 0$. Substitution gives

$$P(X \geq t) \leq e^{-\mu} \left(\frac{e\mu}{t}\right)^t.$$

Exponential moments turn tails into nonnegative quantities (continued)

Unequal p_i are allowed; mutual independence is required. If $\mu = 0$, every indicator is zero almost surely. All positive-threshold tails are zero directly, without evaluating $\ln(t/0)$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Centered r and total t are different thresholds

The looser CLRS form is

$$P(X-\mu \geq r) \leq \left(\frac{e\mu}{r}\right)^r, \qquad r > \mu > 0.$$

To derive it directly, for $\lambda > 0$ bound each centered exponential moment:

$$\begin{aligned} E[e^{\lambda(I_i - p_i)}] &= p_i e^{\lambda(1-p_i)} + (1-p_i)e^{-\lambda p_i} \\ &\leq p_i e^{\lambda} + 1 \leq \exp(p_i e^{\lambda}). \end{aligned}$$

Factor the moments using mutual independence and apply Markov:

$$\begin{aligned} P(X-\mu \geq r) &\leq e^{-\lambda r} \prod_i E[e^{\lambda(I_i - p_i)}] \\ &\leq \exp(\mu e^{\lambda} - \lambda r). \end{aligned}$$

The exponent is minimized at $\lambda = \ln(r/\mu) > 0$, giving the stated bound.

Here r is the deviation above the mean, so the total threshold is $t = \mu + r$. In the preceding formula, t counts total successes. With $\mu = 10, r = 40$, the event is $X \geq 50$. The two bounds are

Centered r and total t are different thresholds (continued)

$$\begin{aligned} \left(\frac{e}{4}\right)^{40} &\approx 1.947 \times 10^{-7}, \\ e^{-10}\left(\frac{e}{5}\right)^{50} &\approx 2.650 \times 10^{-18}. \end{aligned}$$

Exchanging 40 and 50 changes the derivation. When r only slightly exceeds μ, the centered bound may exceed 1; cap it at 1. For an integer binomial variable, the exact tail starts at $\lceil \mu + r \rceil$, never a rounded-down threshold.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Compare guarantees on one binomial tail

Let $X \sim \mathrm{Bin}(1000, 0.01)$, so $\mu = 10$, and consider $X \geq 50$.

Bound	Calculation	Upper bound
Markov	$10/50$	0.2
Chebyshev	$9.9/40^2$	0.0061875
Centered, $r = 40$	$(e/4)^{40}$	$\approx 1.947 \times 10^{-7}$
Total threshold, $t = 50$	$e^{-10}(e/5)^{50}$	$\approx 2.650 \times 10^{-18}$

The exact value remains

$$\sum_{k=50}^{1000} \binom{1000}{k} 0.01^k 0.99^{1000-k}.$$

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Compare guarantees on one binomial tail (continued)

An adjacent-ratio recurrence or log masses avoid floating-point overflow from large factorials. Markov requires nonnegativity and the mean. Chebyshev requires the true variance, which is 9.9 here. Both exponential bounds require mutually independent indicators.

If all indicators instead share the same Bernoulli variable with parameter 0.01, then X is 0 or 1000, and $P(X \geq 50) = 0.01$. Markov still applies, but the binomial variance and exponential-product factorization do not. Equal marginals do not justify the binomial or Chernoff model.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Sources

Bibliography

Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 3rd ed., MIT Press, 2009, Appendix C.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

References and attribution

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Matrices

Part VIII Appendix

Scope

Entries, dimensions, and field assumptions define matrix arithmetic. Complete elimination, rank factorizations, and inverse traces support proofs of equal row and column rank, determinant laws, and Gram positive definiteness. Real, complex, and finite fields require different inner-product conventions.

Prerequisites

Vector spaces, linear maps, and matrix multiplication.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Appendix route

Learning objectives

- Apply matrix arithmetic and transposition.
- Relate rank, determinant, and inverse.
- State the conditions for symmetric positive definiteness and spectral reasoning.
- D.1 Matrices and matrix operations
- D.2 Basic matrix properties

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Matrices and matrix operations

Unless stated otherwise, dimensions are positive integers, entries are real, and vectors are columns. A rank-zero factorization allows zero intermediate columns and rows, with empty sums defined as 0. A matrix $A \in \mathbb{R}^{m \times n}$ has m rows and n columns. The entry a_{ij} uses the row index first and the column index second. Equality requires equal dimensions and equality of every corresponding entry. Addition and scalar multiplication are defined entrywise.

For example,

$$A = \begin{bmatrix} 1 & 2 & 0 \\ 0 & 1 & 1 \end{bmatrix}, \qquad A^T = \begin{bmatrix} 1 & 0 \\ 2 & 1 \\ 0 & 1 \end{bmatrix}$$

have dimensions 2×3 and 3×2, respectively. Transposition means $(A^T)_{ji} = a_{ij}$. Row-major and column-major storage are memory conventions; the mathematical transpose is defined by this index correspondence.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Matrices and matrix operations (continued)

The vector $x = (1, -1, 2)^T$ is 3×1, while x^T is 1×3. The standard basis vector e_i has a 1 in position i and zeros elsewhere, with dimension specified by context. Zero matrices also require dimensions. The scalar 0 and a 2×3 zero matrix are different types of object.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Exact zero positions in common square structures

Type	Entry condition for an $n \times n$ matrix
Diagonal	$i \neq j \Rightarrow a_{ij} = 0$
Identity I_n	Diagonal entries are 1; all others are 0.
Tridiagonal	$\lvert i - j \rvert > 1 \Rightarrow a_{ij} = 0$
Upper triangular	$i > j \Rightarrow a_{ij} = 0$
Lower triangular	$i < j \Rightarrow a_{ij} = 0$
Unit triangular	Triangular, with every diagonal entry equal to 1.
Permutation	Exactly one 1 in each row and each column; all other entries are 0.
Real symmetric	$A = A^T$

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Exact zero positions in common square structures (continued)

These classes can overlap. The identity is diagonal, upper and lower triangular, symmetric, and a permutation matrix. A tridiagonal matrix need not be symmetric because opposite diagonals may differ. Unit triangular does not mean that every entry is 1. A unit vector likewise need not be an all-ones vector.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Check dimensions and the contracted index

For $A \in \mathbb{R}^{m \times n}$ and $B \in \mathbb{R}^{n \times p}$, the product $C = AB$ belongs to $\mathbb{R}^{m \times p}$, with

$$c_{ij} = \sum_{k=1}^{n} a_{ik} b_{kj}, \qquad 1 \le i \le m, \quad 1 \le j \le p.$$

The shared index k is summed out; the outer indices i, j remain. This is not entrywise multiplication.

Using the earlier A and $B = \begin{bmatrix} 2 & 1 \\ -1 & 3 \\ 4 & 0 \end{bmatrix}$ gives

Entry	Calculation
$(AB)_{11}$	$1 \cdot 2 + 2(-1) + 0 \cdot 4 = 0$
$(AB)_{12}$	$1 \cdot 1 + 2 \cdot 3 + 0 \cdot 0 = 7$
$(AB)_{21}$	$0 \cdot 2 + 1(-1) + 1 \cdot 4 = 3$
$(AB)_{22}$	$0 \cdot 1 + 1 \cdot 3 + 1 \cdot 0 = 3$

Check dimensions and the contracted index (continued)

Thus $AB = \begin{bmatrix} 0 & 7 \\ 3 & 3 \end{bmatrix}$. The reverse product is also defined, but has size 3×3:

$$BA = \begin{bmatrix} 2 & 5 & 1 \\ -1 & 1 & 3 \\ 4 & 8 & 0 \end{bmatrix}.$$

The products differ even in size. Square products also need not commute; scalar commutativity does not transfer to matrix multiplication.

Check dimensions and the contracted index (continued)

Check sizes, then sum three aligned k terms

A : 2 rows × 3 columns

1	2	0
0	1	1

×

B : 3 rows × 2 columns

2	1
-1	3
4	0

=

C=AB : 2×2

0	7
3	3

C[1,2] = row1 of A · column2 of B = 1×1+2×3+0×0 = 7

Yellow row/column lengths3 share k. Output retains row1,column2; not entrywise multiplication.

Implementable multiplication and exact cost

```
MULTIPLY(A[m,n], B[n,p])
    allocate independent C[m,p]
    for i=1..m
        for j=1..p
            s = A[i,1] * B[1,j]
            for k=2..n
                s = s + A[i,k] * B[k,j]
            C[i,j] = s
    return C
```

For $n \geq 1$, this uses mnp scalar multiplications, $mp(n-1)$ additions, and mp output writes, plus loop control. Constant-cost scalar arithmetic in the RAM model gives $\Theta(mnp)$ time, or $\Theta(n^3)$ for square matrices. Initializing $s = 0$ and doing n additions instead adds mp additions. That is a different exact count with the same asymptotic order.

Output entries must not alias input entries that will be read later, since writes could corrupt those reads. Arbitrary-precision integers and field elements require separate bit costs. Performing n^3 scalar multiplications does not guarantee n^3 time for arbitrary bit lengths.

Associativity, distributivity, and compatible dimensions

Let A be $m \times n$, B be $n \times p$, and C be $p \times q$. For every valid output index (i, j),

$$\begin{aligned}((AB)C)_{ij} &= \sum_{k=1}^{p} \sum_{\ell=1}^{n} a_{i\ell} b_{\ell k} c_{kj} \\ &= \sum_{\ell=1}^{n} a_{i\ell} \left(\sum_{k=1}^{p} b_{\ell k} c_{kj} \right) = (A(BC))_{ij}.\end{aligned}$$

Reordering is valid because the sums are finite. Distributivity follows from scalar distributivity at each entry. Identity dimensions must match: $I_m A = A I_n = A$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Transposition reverses factor order

For $A \in \mathbb{R}^{m \times n}$ and $B \in \mathbb{R}^{n \times p}$, compare position (j, i) on both sides of $(AB)^T = B^T A^T$:

$$\begin{aligned}((AB)^T)_{ji} &= \sum_{k=1}^{n} a_{ik} b_{kj}, \\ (B^T A^T)_{ji} &= \sum_{k=1}^{n} (B^T)_{jk} (A^T)_{ki} = \sum_{k=1}^{n} b_{kj} a_{ik}.\end{aligned}$$

The entries agree by **scalar** commutativity. The matrix product itself need not commute; transposition reverses the factor order.

For real symmetric matrices A, B of the same size, their sum and difference remain symmetric. Their product is symmetric if and only if $AB = BA$, since $(AB)^T = BA$. Two symmetric inputs alone do not imply a symmetric product.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Matrix-vector, inner, and outer products

Write the columns of $A \in \mathbb{R}^{m \times n}$ as $a_1, \dots, a_n$. For $x \in \mathbb{R}^n$, the product $Ax = \sum_{j=1}^{n} x_j a_j$ is an m-vector. The earlier example gives

$$Ax = \begin{bmatrix} 1 \\ 0 \end{bmatrix} - \begin{bmatrix} 2 \\ 1 \end{bmatrix} + 2 \begin{bmatrix} 0 \\ 1 \end{bmatrix} = \begin{bmatrix} -1 \\ 1 \end{bmatrix},$$

which matches the row dot products.

For equal-length real vectors u, v, the inner product $u^T v$ is a scalar and the outer product uv^T is a matrix. With $u = (1, 2)^T$ and $v = (3, -1)^T$,

$$u^T v = 1, \qquad uv^T = \begin{bmatrix} 3 & -1 \\ 6 & -2 \end{bmatrix}.$$

Column j is $v_j u$, so the rank is at most one. If both vectors are nonzero, at least one column is a nonzero multiple of u and the rank is one. If either vector is zero, the rank is zero.

The real norm is $\|x\|_2 = \sqrt{x^T x}$. Complex vectors require $x^* x = \sum_i |x_i|^2$, where $*$ denotes conjugate transpose. For $x = [i]$, ordinary transpose gives $x^T x = -1$, which is not squared length.

Permutations act differently on rows and columns

For

$$P = \begin{bmatrix} 0 & 0 & 1 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \end{bmatrix},$$

one has $Px = (x_3, x_1, x_2)^T$. If A has three rows, PA selects its old rows in the order $3, 1, 2$. In $(PA)_{ij} = \sum_{k=1}^{3} p_{ik} a_{kj}$, each row of P selects one term.

For a matrix M with three columns, MP selects its old columns in the order $2, 3, 1$, because the columns of P are e_2, e_3, e_1. Derive the direction from entries or basis columns, rather than memorizing one order for both sides.

Distinct basis columns give $P^T P = I_3 = PP^T$, so $P^{-1} = P^T$. Products of permutation matrices remain permutation matrices because composing two basis reorderings is a bijection.

Rank and null vectors

Vectors $v_1, \ldots, v_n$ are linearly independent if $\sum_{j=1}^{n} c_j v_j = 0$ forces every coefficient $c_j = 0$; otherwise they are dependent. Column rank is the largest number of independent columns, and row rank is defined analogously. Zero matrices have rank zero.

Since $Ax = \sum_j x_j a_j$, column independence is equivalent to $Ax = 0$ having only the solution $x = 0$. A nonzero solution is a nonzero null vector. The null space always contains 0; absence of a nonzero null vector does not mean the null space contains no vectors.

The matrix

$$M = \begin{bmatrix} 1 & 2 & 3 \\ 2 & 4 & 6 \\ 0 & 1 & 1 \end{bmatrix}$$

has eight nonzero entries but rank two. Its third column is the sum of the first two, which are independent. A null vector is $(-1, -1, 1)^T$. Rank is neither the nonzero-entry count nor the number of nonzero entries on the original diagonal.

Elimination operations and intermediate states

Row swaps, nonzero row scaling, and adding a multiple of another row are invertible operations. Each is left multiplication by an invertible elementary matrix. They preserve column-dependence coefficients and row span.

For M, the complete elimination trace is

$$M \xrightarrow{R_2 \leftarrow R_2 - 2R_1} \begin{bmatrix} 1 & 2 & 3 \\ 0 & 0 & 0 \\ 0 & 1 & 1 \end{bmatrix} \xrightarrow{R_2 \leftrightarrow R_3} \begin{bmatrix} 1 & 2 & 3 \\ 0 & 1 & 1 \\ 0 & 0 & 0 \end{bmatrix}.$$

$$\begin{bmatrix} 1 & 2 & 3 \\ 0 & 1 & 1 \\ 0 & 0 & 0 \end{bmatrix} \xrightarrow{R_1 \leftarrow R_1 - 2R_2} R = \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 1 \\ 0 & 0 & 0 \end{bmatrix}.$$

Pivot columns $1, 2$ leave x_3 free, with $x_1 = x_2 = -x_3$. Hence the null space is $\{t(-1, -1, 1)^T : t \in \mathbb{R}\}$. The two nonzero rows have distinct leading positions and are independent. The nonpivot column is the sum of the pivot columns.

Elimination operations and intermediate states (continued)

M=BC: dimension2 from original pivot columns

Original M:3×3,rank2

1	2	3
2	4	6
0	1	1

=

B: original pivot columns

1	2
2	4
0	1

×

C: column coordinates

1	0	1
0	1	1

M.column3=column1+column2, so C.column3=$(1,1)^{T}$.

B retains original vectors. Reduced unit columns give coordinates and cannot replace B.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Gauss–Jordan without assuming diagonal pivots

```
R = copy(A[m,n]); r = 0; pivots = empty list
for j=1..n
    find t in r+1..m with R[t,j] != 0
    if none: continue
    r = r+1; swap rows r,t
    divide row r by R[r,j]
    for i=1..m with i != r
        a = R[i,j]
        row i = row i - a * row r
    append j to pivots
return R, pivots
```

Before each step, the first r pivot columns are distinct unit columns, and unused rows are zero in those columns. A new pivot comes from a remaining row. Normalizing it and eliminating elsewhere preserves old pivot columns because its old pivot coordinates are zero. Skip a column only when every remaining row is zero there. The finite loops produce r pivots and zero rows in $O(mn\min(m,n))$ field operations.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Gauss–Jordan without assuming diagonal pivots (continued)

Exact comparison with zero follows the field arithmetic. Floating-point arithmetic needs pivoting, scale, and tolerance analysis. A fixed rule such as $|\text{pivot}| < 10^{-12}$ is not a universally correct mathematical definition of rank.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Why row rank equals column rank

Let $R = EA$ be the reduced row echelon form, with E invertible. Applying the same reversible map to every column preserves exactly their dependence coefficients. The r pivot columns of R are independent, and every other column is their combination. Thus the column rank of A is r.

Invertible row operations preserve row span. The r nonzero rows of R are independent: their successive pivot coordinates force each coefficient in a dependence relation to vanish. Thus the row rank of A is also r. Write $\text{rank}(A)$ for this common value, between 0 and $\min(m, n)$.

The nullity is $n - r$. Free coordinates uniquely determine the pivot coordinates. Set one free variable at a time to 1 and the others to 0 to construct a null-space basis. These vectors are independent by their free coordinates and span every solution. This proves the dimension statement generally, beyond the example M.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Rank factorization and product bounds

Choose the r original pivot columns of A to form $B \in \mathbb{R}^{m \times r}$. Put the coordinates of every original column relative to B in $C \in \mathbb{R}^{r \times n}$. Then $A = BC$, the columns of B are independent, and C contains identity columns. Conversely, if $A = UV$ with s intermediate columns, every column of A lies in the span of U, so $\operatorname{rank}(A) \leq s$. Therefore r is the minimum possible intermediate dimension.

For M,

$$B = \begin{bmatrix} 1 & 2 \\ 2 & 4 \\ 0 & 1 \end{bmatrix}, \qquad C = \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 1 \end{bmatrix}, \qquad M = BC.$$

The pivot vectors of R are not the original vectors of M; row operations changed their entries.

For compatible A, D, factor $A = B_r C_r$ to get $AD = B_r(C_r D)$, so $\operatorname{rank}(AD) \leq \operatorname{rank}(A)$. Factoring $D = U_s V_s$ similarly gives $AD = (AU_s)V_s$ and the bound by $\operatorname{rank}(D)$. Thus

$$\operatorname{rank}(AD) \leq \min\{\operatorname{rank}(A), \operatorname{rank}(D)\}.$$

Rank factorization and product bounds (continued)

For an invertible left factor P, the identity $A = P^{-1}(PA)$ proves the reverse inequality, giving $\operatorname{rank}(PA) = \operatorname{rank}(A)$. Invertible right multiplication is analogous.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Inverse existence and uniqueness

An inverse B of a square $n \times n$ matrix A satisfies $AB = I_n = BA$. If B, C are both inverses, $B = B(AC) = (BA)C = C$. For invertible A, B, the product inverse is $(AB)^{-1} = B^{-1}A^{-1}$, as multiplication on both sides verifies. Transposing the two inverse equations gives $(A^{-1})^T = (A^T)^{-1}$.

Square full rank is equivalent to no nonzero null vector, hence to injectivity of $x \mapsto Ax$. The n independent columns form a basis of n-space. Each e_j therefore has a unique solution $Ab_j = e_j$. Assemble these columns into B to obtain $AB = I_n$. Then $A(BA - I_n) = 0$ and injectivity imply $BA - I_n = 0$ columnwise. Thus full rank is equivalent to invertibility. Conversely, multiplying $Ax = 0$ by the inverse gives $x = 0$.

A rectangular one-sided inverse need not be two-sided. For $A = \begin{bmatrix} 1 & 0 \end{bmatrix}$ and $B = (1, 0)^T$, one has $AB = I_1$ but $BA = \operatorname{diag}(1, 0) \neq I_2$. The square premise cannot be dropped.

A fully checkable inverse trace

Let $H = \begin{bmatrix} 2 & 1 \\ 3 & 2 \end{bmatrix}$. Apply each row operation to both halves of $[H \mid I_2]$:

$$\left[\begin{array}{cc|cc} 2 & 1 & 1 & 0 \\ 3 & 2 & 0 & 1 \end{array}\right] \xrightarrow{R_2 \leftarrow R_2 - (3/2)R_1} \left[\begin{array}{cc|cc} 2 & 1 & 1 & 0 \\ 0 & 1/2 & -3/2 & 1 \end{array}\right].$$

$$\left[\begin{array}{cc|cc} 2 & 1 & 1 & 0 \\ 0 & 1/2 & -3/2 & 1 \end{array}\right] \xrightarrow{R_2 \leftarrow 2R_2} \left[\begin{array}{cc|cc} 2 & 1 & 1 & 0 \\ 0 & 1 & -3 & 2 \end{array}\right].$$

Finish and verify the inverse

Subtract row two from row one, then normalize row one:

$$\left[\begin{array}{cc|cc} 2 & 1 & 1 & 0 \\ 0 & 1 & -3 & 2 \end{array}\right] \xrightarrow{R_1 \leftarrow R_1 - R_2} \left[\begin{array}{cc|cc} 2 & 0 & 4 & -2 \\ 0 & 1 & -3 & 2 \end{array}\right].$$

$$\left[\begin{array}{cc|cc} 2 & 0 & 4 & -2 \\ 0 & 1 & -3 & 2 \end{array}\right] \xrightarrow{R_1 \leftarrow R_1/2} \left[\begin{array}{cc|cc} 1 & 0 & 2 & -1 \\ 0 & 1 & -3 & 2 \end{array}\right].$$

The right half is $H^{-1} = \begin{bmatrix} 2 & -1 \\ -3 & 2 \end{bmatrix}$. Both checks give the identity:

$$HH^{-1} = \begin{bmatrix} 4-3 & -2+2 \\ 6-6 & -3+4 \end{bmatrix} = I_2, \qquad H^{-1}H = \begin{bmatrix} 4-3 & 2-2 \\ -6+6 & -3+4 \end{bmatrix} = I_2.$$

Elimination applies E to the left half while turning the right half into E. When $EH = I_2$, E is the inverse. Without all n left-side pivots for an $n \times n$ matrix, the right half cannot simply be declared an inverse.

Determinants, minors, and cofactors

For a square matrix A, the minor matrix $A[ij]$ removes row i and column j. The cofactor is $C_{ij} = (-1)^{i+j} \det A[ij]$. For $n = 1$, define $\det[a] = a$. The optional 0×0 convention $\det(\varnothing) = 1$ unifies cofactor formulas. Expansion along row i gives $\det A = \sum_{j=1}^{n} a_{ij} C_{ij}$.

For $N = \begin{bmatrix} 2 & 1 & 0 \\ 1 & 3 & 2 \\ 0 & -1 & 1 \end{bmatrix}$, first-row signs $+, -, +$ give

$$\det N = 2\big(3 \cdot 1 - 2(-1)\big) - 1(1 \cdot 1 - 2 \cdot 0) + 0 = 10 - 1 = 9.$$

A minor is a matrix, its determinant is a scalar, and a cofactor adds a sign. These are distinct objects.

In general, with S_n the permutations of $\{1, \ldots, n\}$,

$$\det A = \sum_{\sigma \in S_n} \operatorname{sgn}(\sigma) \prod_{i=1}^{n} a_{i,\sigma(i)}.$$

Each term selects one entry per row and per column, with a permutation-parity sign. Grouping terms by the selected column in a fixed row gives cofactor expansion.

Reasons for determinant laws

The permutation formula is linear separately in every column. With two equal columns, pair permutations that exchange those columns; the terms cancel, giving determinant zero. Swapping columns changes the sign. The same arguments apply to rows, and the bijection $\sigma \mapsto \sigma^{-1}$ gives $\det(A^T) = \det A$. Scaling a row by α scales the determinant by α. Adding a multiple of another row leaves it unchanged because the extra term from multilinearity has a repeated row.

For any alternating multilinear scalar function F of the columns, expand each column in the standard basis. Repeated basis indices vanish. The remaining permutation terms satisfy

$$F(e_{\sigma(1)}, \dots, e_{\sigma(n)}) = \operatorname{sgn}(\sigma) F(I_n).$$

Thus $F(B) = F(I_n) \det B$. Fix A and take $F(B) = \det(AB)$. Each product column is A times a column of B, so F is alternating multilinear and $F(I_n) = \det A$. This proves $\det(AB) = \det A \det B$ without assuming A invertible. If $\det A = 0$, the function is identically zero.

Determinants and singularity beyond the 2×2 formula

For an upper-triangular matrix, a nonzero permutation term requires $\sigma(i) \geq i$ for every i. Since $\sum_i \sigma(i) = \sum_i i$, every inequality is an equality. Only the diagonal term remains, so the determinant is the diagonal product. The lower-triangular case follows symmetrically or by transpose. A square echelon matrix with a zero row has determinant zero; with all n pivots nonzero, it has a nonzero diagonal product.

Swaps and nonzero scalings multiply the determinant by nonzero factors; row addition preserves it. Therefore

$$\begin{aligned} \det A = 0 &\iff \text{a pivot is missing} \\ &\iff \operatorname{rank}(A) < n \\ &\iff A \text{ is singular.} \end{aligned}$$

A nonzero determinant proves invertibility, without guaranteeing good numerical conditioning. For $A = \operatorname{diag}(1, \varepsilon)$ with $\varepsilon \neq 0$, exact full rank holds but the inverse contains $1/\varepsilon$. Small ε can amplify errors. Stable factorizations usually solve $Ax = b$ without explicitly forming the inverse. Mathematical existence and numerical reliability are different issues.

Real symmetric positive definiteness and Gram matrices

Here positive definite means a **real symmetric** matrix S with $x^T S x > 0$ for every nonzero real x. Positive semidefinite replaces $>$ with $\geq$. Without symmetry, a quadratic form sees only $(S + S^T)/2$; positivity alone does not imply $S = S^T$.

For real $A \in \mathbb{R}^{m \times n}$, the Gram matrix $G = A^T A$ is symmetric and

$$x^T G x = (Ax)^T (Ax) = \|Ax\|_2^2 \geq 0.$$

Equality holds exactly when $Ax = 0$. Thus G is positive definite exactly when A has no nonzero null vector, or equivalently has full column rank. This is impossible when $n > m$. A merely nonzero A is insufficient.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Check Gram positivity on all vectors

For

$$X = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{bmatrix}, \qquad X^T X = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix},$$

the vector $x = (s, t)^T$ gives $x^T X^T X x = s^2 + t^2 + (s + t)^2 > 0$ unless $s = t = 0$.

For

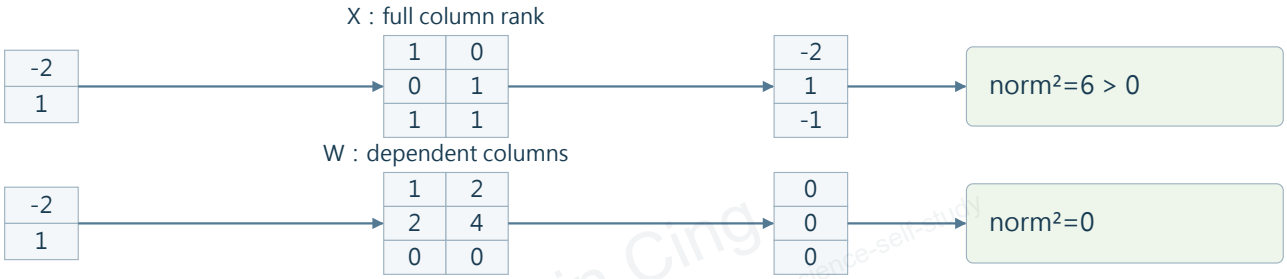
$$W = \begin{bmatrix} 1 & 2 \\ 2 & 4 \\ 0 & 0 \end{bmatrix}, \qquad W^T W = \begin{bmatrix} 5 & 10 \\ 10 & 20 \end{bmatrix},$$

the quadratic form is $5(s + 2t)^2$. The nonzero vector $(-2, 1)^T$ has $Wx = 0$, so this Gram matrix is only positive semidefinite.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Check Gram positivity on all vectors (continued)

Compare x^TA^TAx=norm(Ax)² at two ranks



Same nonzero $x=(-2,1)^T$. Positive definiteness needs all nonzero x; one null vector refutes W.

Complex and finite fields change some premises

Field arguments for multiplication, inverses, rank, and determinants extend to $\mathbb{C}$ and prime fields $\mathbb{F}_p$. Real order and Euclidean inner products do not transfer unchanged. Arithmetic modulo a composite number forms a ring in which nonzero elements may be noninvertible. For example, 2 modulo 6 has no inverse, so pivot normalization and field-rank arguments require reconsideration.

The complex conjugate transpose is A^*. A Hermitian matrix satisfies $S^* = S$. The Gram matrix A^*A is positive semidefinite and is positive definite for full column rank because

$$x^*A^*Ax = \sum_i |(Ax)_i|^2.$$

Ordinary A^TA need not be positive. For $A = [i]$, one has $A^TA = [-1]$ but $A^*A = [1]$.

For real or complex square A, the algebraic transpose identity is $\det(A^TA) = (\det A)^2$. The conjugate version is $\det(A^*A) = |\det A|^2 \geq 0$. The first expression implies nonnegativity only for real entries. Distinguish algebraic transposition from conjugation in inner products.

Sources

Bibliography

Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 3rd ed., MIT Press, 2009, Appendix D.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

References and attribution

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study